Functional Encryption and its Applications to Decentralized Systems

Submitted in partial fulfillment of the requirements for

the degree of

Doctor of Philosophy

in

Electrical and Computer Engineering

Nikhil Vanjani

B.Tech., Computer Science and Engineering, Indian Institue of Technology Kanpur
M.S., Information Security, Carnegie Mellon University

Carnegie Mellon University
Pittsburgh, PA

December 2025

© Nikhil Vanjani, 2025
All rights reserved.

To my parents Lalit and Manisha, my brother Yash and my partner Ayushi.

Abstract

Classical public-key encryption secures communication but provides only *all-or-nothing* access to encrypted data. Modern decentralized systems such as cloud platforms and blockchain networks require *fine-grained* access to sensitive data while preserving privacy. For instance, in healthcare, strict privacy laws prevent sharing patient data across hospitals or research institutions, even when such sharing could improve treatment or advance research. Functional encryption (FE) offers a path forward: it enables learning specific functions on encrypted data without revealing the whole data, thus ensuring privacy. This thesis develops new constructions and applications of functional encryption to make this vision practical, focusing on three application domains: decentralized access control, fair exchange, and anonymous communication.

The first part addresses *decentralized access control*, where the goal is to ensure only authorized users can access certain data. Attribute-Based Encryption (ABE), a special form of FE, addresses this challenge by binding ciphertexts to access policies and decryption keys to user attributes. However, ABE involves a single trusted authority, creating a central point of failure that limits real-world deployment. Multi-Authority ABE (MA-ABE) overcomes this limitation by distributing trust among multiple authorities managing disjoint sets of attributes, but prior schemes lacked either strong security or efficiency. We present the first fully secure and efficient MA-ABE scheme, developing a clever entropy control mechanism within the dual system encryption framework that simplify and improve the classic Lewko–Waters [Eurocrypt 2011] scheme. Building on this, we further extend MA-ABE to support arithmetic span programs, enabling expressive and native support for access policies such as comparisons.

The second part explores *fair exchange*, where mutually distrustful parties want to exchange digital items (for example, data for payment) such that either both parties receive their expected items, or neither does. While classical two-party fair exchange is impossible without a trusted third party, recently blockchains have been used as a trusted third party to overcome this barrier. This thesis studies richer scenarios, such as a buyer paying to train an AI model on a proprietary dataset (for example, patient records), where both fairness and privacy must hold: the buyer should only learn a specific function evaluation on the data (for example, fine-tuned model parameters). We introduce a new primitive called *Functional Adaptor Signatures (FAS)*, and design protocols using it that ensure both fairness and privacy. Our constructions include a FE-based FAS for selling linear functions and a fully homomorphic encryption (FHE)–based FAS for selling arbitrary functions. Together, they establish the foundations of fair exchange for computing insights on sensitive data.

The third part studies *anonymous communication*, where users wish to send messages without revealing their identities. Classical systems such as DC-Nets and Tor are interactive and depend on threshold assumptions about honest routers; if too many are corrupted, anonymity is lost. To remove these assumptions, Shi and Wu [Eurocrypt 2021] introduced the non-interactive anonymous routing (NIAR)

model, treating all routers as potentially malicious. Their FE-based construction, however, required quadratic router computation and assumed static corruptions. We present the first NIAR scheme with quasi-linear router computation and prove security in the presence of adaptive corruptions. Our scheme relies on indistinguishability obfuscation (iO), a powerful primitive related to FE. It employs a network of small obfuscated programs and introduces new authentication techniques for reasoning about their security.

Acknowledgements

I would like to express my deepest gratitude to everyone who has supported me throughout my PhD journey.

First and foremost, I am profoundly thankful to my advisor, Elaine, for sharing her deep passion for research, for her invaluable guidance, and for her unwavering belief in my potential. Her mentorship has been instrumental in shaping my academic and professional growth. I will always cherish the countless discussions, brainstorming sessions, and the collaborative spirit that she fostered in our research group. I am also grateful for the many times she organized group outings and activities that helped build a strong sense of community among us.

I would also like to extend my sincere thanks to my thesis committee members, Pratik, Pratish, Aayush, and Bryan, for their insightful feedback and encouragement throughout my research. I am grateful to have Pratik as a committee member, collaborator, and mentor. He was like a second advisor to me, always ready to help me with any questions I had, and providing valuable feedback on my work. I am grateful to Pratish for the internship opportunity at NTT Research, which has since blossomed into a fulfilling collaboration and for sharing his expertise in functional encryption. Aayush has been a great mentor, always encouraging me to think critically and broaden my research horizons. While I did not have the chance to work closely with Bryan, I appreciate his willingness to be part of my committee and provide valuable insights.

I am thankful to my collaborators Junichi, Aravind, Brent, Rex, Garrett and Yannis for their contributions to our joint research endeavors. Working with all of you has been a rewarding experience, and I have learned a great deal from each of you. I learnt a lot about ABE and pairing based cryptography from Junichi during our multiple collaborations on MA-ABE. My first applied cryptography project was with Aravind, and I am grateful for his insights and enthusiasm in exploring real-world applications of cryptography. Brent has been an inspiring collaborator, and I have learned a great deal from his deep understanding of cryptographic principles and his feedback on my work. I learnt many LaTeX tricks from Rex that made my writing process smoother, and I am also grateful to him for providing me with career advice and connecting me with others who could offer diverse perspectives. The first junior researcher I collaborated with was Garrett, and I am thankful for his dedication to our projects. Mentoring him has also helped me reflect on my research journey and appreciate how far I have come. Yannis has been a great source of knowledge on implementation challenges in cryptography and has helped me gain a deeper perspective.

I would like to acknowledge the support of my lab members Ke, Hao, Ashrujit, Abhiram, Lisa, Sagnik, Justin, Quang, Alper, Afonso, Andrew, Mingxun, and Tianyao. I learnt so much from all of you, and your friendship made my time at CMU truly enjoyable. Special shout out to Ke for helping me during the initial

days of my PhD, to Hao for being a great officemate and for all the fun times we had, and to Ashrujit and Abhiram for all the career advice. I will miss our crypto group's Thursday lunches.

Thank you to the people I met during my two summers at NTT Research: Alexis, Surya, Kabir, Riddhi, Eli, Sacha, Aditi, Swagata and Aditya. Your support and friendship made my internship experiences memorable. I would also like to thank my colleagues from my Spring internship at 0xPARC: Brian and Brian, for sharing the real-world challenges of deploying cryptography and for sharing your passion and ambition for building programmable cryptography.

On a personal note, I am grateful to my brothers from IIT Kanpur who have been a constant source of support and encouragement throughout my academic journey: Ankit, Shaurya, and Rohan. Thank you for always being there for me, and for the countless memories we have shared together. Ankit, thank you for being a great listener and for giving me a place to stay on multiple occasions when I needed to do focused work around deadlines (also thanks to Abhinav, who has since become a friend, for his support). Shaurya, thank you for being my go-to person for all things academic. Rohan, thank you for supporting me through personal challenges and for being a great friend. I would also like to thank my IITK wingies Nishit, Mohit, Raghav, Kanishik, Anuj, Animesh, Prabhat, and Apoorve for making an effort to stay connected over the years and meet whenever possible. I would also like to thank my friends from CMU: Swapnil, Sreyash, Mehul, Pratapaditya, and Rohit. You all have made my time at CMU more enjoyable and memorable.

Finally, I would like to express my deepest gratitude to my family for their unwavering love and support throughout my academic journey. To my mummy and papaa, thank you for your sacrifices, encouragement, and belief in me. To my brother Yash, thank you for our shared time at CMU. We were at the same place after so many years apart, and your presence made my journey easier. To my partner Ayushi, thank you for your endless patience, understanding, and love. Your support has been my anchor, and I am grateful to have you by my side.

Committee: Elaine Shi (Chair), Pratish Datta (NTT Research), Aayush Jain, Bryan Parno, Pratik Soni (University of Utah).

Funding Acknowledgement: This research was supported by a DARPA SIEVE grant, a Packard Fellowship, a Simons Investigator award, a grant from Algorand Foundation, NSF awards under the grant numbers CNS-2128519, CNS-2044679, CNS-2212746, CNS-1908611, CNS-2318701, a grant from Office of Naval Research under the award number N000142212064, the State of Utah's Governor's Office of Economic Opportunity (IAA #230632319) through a seed grant from the Stena Center for Financial Technology, and by gifts from the Stellar Development Foundation Academic Research Grants program. The views and con-

clusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of any sponsoring institution, the U.S. government or any other entity.

Contents

Contents x

List of Figures xv

1 Introduction 1

1.1 Decentralized Access Control . . . 2
1.2 Fair Exchange . . . 3
1.3 Anonymous Communication . . . 5
1.4 Summary . . . 6

2 Multi-Authority Attribute-Based Encryption 7

2.1 Introduction . . . 8
2.2 Technical Overview . . . 14
2.2.1 Background on MA-ABE and Fully Adaptive Security . . . 15
2.2.2 The Lewko–Waters Construction and its Limitations in supporting Adaptive Authority Corruption . . . 17
2.2.3 Our Approach for Supporting Adaptive Authority Corruption . . . 19
2.2.4 Our composite-order Fully Adaptive MA-ABE scheme for BSP . . . 20
2.2.5 Our Composite-order Fully Adaptive MA-ABE scheme for ASP . . . 26
2.2.6 A Compiler to support corruption of authorities related to the challenge ciphertext . . . 27
2.3 Preliminaries . . . 27
2.3.1 Notations . . . 27
2.3.2 Access Structures . . . 28
2.3.3 Boolean Span Program . . . 28
2.3.4 Arithmetic Span Programs . . . 29
2.3.5 Secret Sharing Schemes . . . 29

2.3.6 MA-ABE . . . 29
2.3.7 Assumptions . . . 33
2.4 MA-ABE for monotone BSP from prime-order groups . . . 34
2.4.1 Construction . . . 34
2.4.2 Security Analysis . . . 36
2.4.3 Missing Proofs from Section 2.4.2 . . . 43
2.5 MA-ABE for NC^1 with Multi-Use Security . . . 52
2.5.1 Core 1-ABE . . . 52
2.5.2 Our Construction . . . 53
2.5.3 Proof of Theorem 2.5.3 . . . 53
2.6 MA-ABE for ASP from prime-order groups . . . 62
2.6.1 Definition . . . 63
2.6.2 Construction . . . 65
2.6.3 Proof of Theorem 2.6.4 . . . 66
2.6.4 Compiler for MA-ABE for ASP: boosting security . . . 78
2.6.5 Proof Sketch of Theorem 2.6.17 . . . 79
2.7 Efficiency Analysis: Detailed . . . 81

3 Fair Exchange **83**
3.1 Introduction . . . 84
3.1.1 Our Contributions in [VST24a] . . . 87
3.1.2 Our Contributions in [VGTS26] . . . 90
3.2 Technical Overview 1: [VST24a] . . . 92
3.2.1 Functional Adaptor Signatures for Fair Functional Payments . . . 92
Security of FAS . . . 92
Strawman Construction of FAS . . . 93
3.2.2 Our Techniques: Functional Encryption + Adaptor Signatures . . . 94
3.2.3 Our Construction . . . 98
3.2.4 Instantiations . . . 100
Prime-Order Groups based Instantiation. . . . 100
Lattice based Instantiation. . . . 101
3.2.5 Towards Non-Interactive Pre-Signing . . . 102
3.3 Technical Overview 2: [VGTS26] . . . 102

3.3.1 Our FAS Template: HE + Adaptors . . . 103
3.3.2 Efficiency Improvements . . . 104
3.3.3 Defining Functional Fair Exchange . . . 107
3.4 Preliminaries . . . 107
3.4.1 Hard Relation . . . 108
3.4.2 Adaptor Signatures . . . 109
3.4.3 Inner Product Functional Encryption . . . 112
3.4.4 Lattice Preliminaries . . . 115
3.4.5 Public Key Encryption . . . 116
3.4.6 Non-Interactive Zero Knowledge (NIZK) Arguments . . . 118
3.4.7 Homomorphic Encryption . . . 120
3.5 Definition of FAS . . . 121
3.6 Strongly-Secure FAS Construction from FE . . . 131
3.7 FAS from Groups of Prime-Order . . . 135
3.7.1 Schnorr Adaptor Signature . . . 135
3.7.2 IPFE from Groups of Prime-Order . . . 136
3.7.3 FAS Construction . . . 137
3.8 FAS from lattices . . . 138
3.8.1 Lattice-based Adaptor Signature . . . 138
3.8.2 IPFE from Lattices . . . 140
3.8.3 FAS Construction . . . 142
3.8.4 Modifying FAS Construction in Figure 3.2. . . . 143
3.9 Performance Evaluation of FE-based FAS . . . 144
3.10 Proofs for Generic Construction of FAS . . . 146
3.10.1 Correctness . . . 146
3.10.2 Advertisement soundness . . . 147
3.10.3 Pre-Signature Validity . . . 148
3.10.4 Unforgeability . . . 148
3.10.5 Witness Extractability . . . 153
3.10.6 Pre-Signature Adaptability . . . 156
3.10.7 Zero-Knowledge . . . 156
3.11 Weakly-Secure FAS Construction from FE . . . 162
3.11.1 Witness Indistinguishability . . . 164

3.12 FAS for General Functions from FHE . . . 168
3.12.1 Building Blocks . . . 168
3.12.2 $\mathsf{NIZK}_{\mathsf{eq}}$ Instantiation . . . 170
3.12.3 FAS Construction . . . 170
3.13 Implementation and Evaluation of HE-based FAS . . . 172
3.13.1 FHE FAS . . . 173
Core Optimization: efficient REval . . . 173
Applications . . . 175
3.13.2 LHE FAS . . . 177
3.14 Functional Fair Exchange (FFE) . . . 178
3.14.1 FFE protocol from FAS . . . 181
3.14.2 FAS security definitions: Informal . . . 181
3.14.3 Proof Sketch of Theorem 3.14.3 . . . 182
3.15 Proofs: FAS from FHE . . . 183
3.16 Proofs: Cut-and-choose based $\mathsf{NIZK}_{\mathsf{eq}}$ . . . 218
3.17 FAS for Linear Functions (Revisited) . . . 220

4 Non-Interactive Anonymous Routing 222

4.1 Introduction . . . 222
4.1.1 Main Result . . . 224
4.1.2 Additional Results . . . 225
4.1.3 Related Work and Open Questions . . . 226
4.2 Technical Roadmap . . . 228
4.2.1 Single Selective Challenge and Static Corruptions . . . 228
4.2.2 Removing the Selective Challenge Restriction . . . 232
4.2.3 Achieving Security for Adaptive Corruptions . . . 233
4.3 Definitions for NIAR . . . 234
4.3.1 Syntax . . . 235
4.3.2 NIAR Security under Static Corruption . . . 235
4.3.3 NIAR Full Security . . . 236
4.4 Preliminaries . . . 238
4.4.1 Notations . . . 238
4.4.2 Routing Networks . . . 239

4.4.3 Puncturable PRF . . . 241
4.4.4 Indistinguishability Obfuscation . . . 242
4.4.5 Selective Opening Security for PRFs . . . 242
4.5 Somewhere Statistically Unforgeable (SSU) Signatures . . . 243
4.5.1 Definition . . . 244
4.6 NIAR for a Static and All-Receiver-Corrupting Adversary . . . 245
4.6.1 Construction . . . 246
4.6.2 Efficiency Analysis . . . 251
4.6.3 Static Security Theorem . . . 252
4.7 Upgrading Security Under Static Corruption . . . 252
4.7.1 Definition: Selective Single-Challenge Security w.r.t. a Single Inversion . . . 253
4.7.2 Upgrade Theorem for Static Corruption . . . 254
4.7.3 Proof of the Upgrade Theorem . . . 255
Removing the Selective Restriction . . . 255
Removing the Single Challenge Restriction . . . 257
Removing the Single Inversion Restriction . . . 258
Completing the Proof . . . 259
4.8 Proof of Selective Single-Challenge Security w.r.t. a Single Inversion . . . 259
4.8.1 The Hybrids for Theorem 4.8.1 . . . 261
Informal Hybrids . . . 261
Formal Hybrids . . . 265
4.8.2 Proofs of Indistinguishability . . . 273
4.9 From Static Corruption to Adaptive Corruption . . . 284
4.9.1 A Compiler for Upgrading from Static to Adaptive Corruption . . . 284
4.9.2 Adaptive Corruption Under Single-Inversion and All-Receiver-Corruption . . . 285
4.9.3 Removing the Single Inversion Restriction . . . 286
4.9.4 Removing the All Receiver Corruption Restriction . . . 287
4.9.5 Completing the Proof of Theorem 4.9.1 . . . 290

5 Conclusion and Future Directions **291**

Bibliography **294**

List of Figures

2.1 Security game for MA-ABE. . . . 33
2.2 Construction: MA-ABE for monotone BSP from prime-order groups . . . 35
2.3 CPA-secure symmetric encryption scheme in [KW19] . . . 52
2.4 Construction: MA-ABE scheme for ASP from prime-order groups . . . 65
2.5 Compiler Construction: MA-ABE for ASP . . . 79

3.1 Functional payments via simplified FAS interface. . . . 87
3.2 Construction: Functional Adaptor Signatures. We note that the syntax here is slightly different from the one in Definition 3.5.1 for ease of presentation. In particular, the function input is now represented via the vector $\mathbf{y}$. $(\mathsf{aux}_{\mathbf{y}}, \pi_{\mathbf{y}})$ denote $\widetilde{\sigma}_1$, and $\widetilde{\sigma}$ simply denotes $\widetilde{\sigma}_2$ in the figure for brevity. $\mathsf{FPreSign}_1$ does not need X, x as inputs, $\mathsf{FPreVerify}_1$ does not need X as input, $\mathsf{FPreSign}_2$ does not need $\pi_{\mathbf{y}}$ as input, Adapt does not need $\pi_{\mathbf{y}}$ as input, and FExt does not need $\mathsf{vk}, \pi_{\mathbf{y}}$ as inputs. So, those inputs are omitted for brevity. . . . 99
3.3 Runtime comparison for VST24 and our LHE-FAS (weighted sums), our BGN-FAS (weighted standard deviation), and SNARK-based strawmans over databases of varying dimensions with weights ≤ 1000. . . . 103
3.4 Witness Extractability experiment of adaptor signatures . . . 111
3.5 Selective, IND-security experiment of Inner Product Functional Encryption . . . 113
3.6 Semantic Security Game G_b . . . 117
3.7 Exponential ElGamal Encryption Algorithm Definitions . . . 117
3.8 Unforgeability experiment for Functional Adaptor Signature Schemes . . . 125
3.9 Witness Extractability experiment for Functional Adaptor Signature Schemes . . . 126
3.10 Witness Indistinguishability experiment of FAS . . . 128
3.11 Witness Hiding experiment of FAS . . . 128
3.12 Zero Knowledge (non-abort) experiments for FAS . . . 129
3.13 Zero Knowledge (abort) experiments for FAS . . . 130

3.14 Unforgeability Proof: Games G_0, G_1, G_2 . . . 132
3.15 Abdalla et al. [ABDP15] IPFE scheme augmented with PubKGen algorithm . . . 136
3.16 Digital signature scheme Lyu′ from SIS and LWE . . . 139
3.17 AS w.r.t. Lyu′ signature scheme and hard relations R_{ISIS} and R'_{ISIS} . . . 139
3.18 Agrawal et al. [ALS16] IPFE scheme augmented with PubKGen algorithm. Note that KGen here is stateless. . . . 141
3.19 Unforgeability Proof: Games G_0, G_1, G_2 . . . 149
3.20 Reduction $\mathcal{B}$ for proof of Claim 3.10.4 . . . 150
3.21 Reduction $\mathcal{B}$ for proof of Claim 3.10.5 . . . 151
3.22 Reduction $\mathcal{B}$ for proof of Claim 3.10.6 . . . 152
3.23 Witness Extractability proof: Games G_0 and G_1 . . . 154
3.24 Reduction B for proof of Claim 3.10.9 . . . 155
3.25 Zero-Knowledge Simulator Sim . . . 157
3.26 Zero-knowledge security of FAS: $\mathsf{faZKReal}_{\mathcal{A},\mathsf{FAS}}$ is the real world experiment, $\mathsf{Hyb}^{\mathsf{NIZK.Sim}}_{\mathcal{A},\mathsf{FAS}}$ is an intermediate hybrid experiment, $\mathsf{faZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$ is the ideal world experiment. . . . 158
3.27 Oracle descriptions for experiments $\mathsf{faZKReal}_{\mathcal{A},\mathsf{FAS}}$, $\mathsf{Hyb}^{\mathsf{NIZK.Sim}}_{\mathcal{A},\mathsf{FAS}}$, $\mathsf{faZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$. . . . 159
3.28 Reduction $\mathcal{B}$ for proof of Claim 3.10.13 . . . 161
3.29 Construction: Weakly-secure FAS . . . 163
3.30 Witness Indistinguishability proof: Games G_0^b and G_1^b for $b \in \{0,1\}$ . . . 165
3.31 Distinguisher $\mathcal{D}^b$ for proof of Claim 3.11.5 . . . 166
3.32 Reduction $\mathcal{B}$ for proof of Claim 3.11.6 . . . 167
3.33 Cut-and-choose based $\mathsf{NIZK}_{\mathsf{eq}}$ for proving plaintext equality across HE and ElGamal ciphertexts. 171
3.34 Description of algorithms in our HE-based FAS construction. . . . 172
3.35 TFHE encryption scheme from [tfh, CGGI20]. . . . 174
3.36 TFHE.HEval and TFHE.REval for non-linear f in [tfh, CGGI20]. . . . 174
3.37 Runtime of FHE-FAS algorithms for polynomial evaluations. We plot times for HEval, NIZK Prover and Verifier. "Rest" includes $\mathsf{FPreSign}_2$, $\mathsf{FPreVerify}_2$, Adapt, and Vf. . . . 176
3.38 Runtime of FHE-FAS algorithms for ML model fine-tuning model task. See Figure 3.37's caption for descriptions of reported algorithms. NIZK times are estimates based on Figure 3.37. . . . 176
3.39 Runtime of FExt for VST24 and LHE-FAS for varying witness bounds. For example, "VST24: 100" means VST24 with a witness bound of 100. . . . 177

3.40 Runtime of LHE-FAS algorithms for different database sizes. We report times for $\mathsf{FPreVerify}_1$, $\mathsf{FPreSign}_1$, FExt, with "Rest" including total combined time for $\mathsf{FPreSign}_2$, $\mathsf{FPreVerify}_2$, Adapt, and Vf. . . . 177
3.41 SFReal, SFIdeal, SFReal^i and $\mathsf{SFIdeal}^i$ games for Seller Fairness. . . . 180
3.42 Unforgeability Proof: Games G_0, G_1, G_2 . . . 184
3.43 Reduction for prover $\mathcal{P}$ playing the adaptive soundness game with challenger $\mathcal{C}$ . . . 186
3.44 Reduction for $\mathcal{B}$ playing the witness extractability game for AS with challenger $\mathcal{C}$ . . . 189
3.45 Reduction for $\mathcal{B}'$ which breaks the $\mathcal{F}$-hardness of the relation R. . . . 191
3.46 Witness Extractability Proof: Games G_0, G_1, G_2 . . . 195
3.47 Reduction for prover $\mathcal{P}$ playing the adaptive soundness game with challenger $\mathcal{C}$ . . . 196
3.48 Reduction for $\mathcal{B}$ playing the witness extractability game for AS with challenger $\mathcal{C}$ . . . 198
3.49 Simulator Sim for zero knowledge (non-abort) of FAS proof . . . 202
3.50 Zero Knowledge (non-abort) Proof: Games $\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}$, $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}$, $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}$, and $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$ 203
3.51 Simulator Sim for zero knowledge (abort) of FAS proof . . . 212
3.52 Zero Knowledge (abort) Proof: Games $\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}$, $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}$, $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}$, and $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$ . . . 213
3.53 Description of REval for our LHE scheme. . . . 221

4.1 The circuit $\mathsf{Gate}_{(\ell,g)}$. . . . 247
4.2 The Setup algorithm . . . 249
4.3 The Setup^* algorithm for selective single-challenge security w.r.t. inversion under static corruption as defined in Definition 4.7.1. . . . 260
4.4 The circuit $\mathsf{Gate}_{(\ell,g)}$ in hybrid experiment $\mathsf{Hyb}^{(b)}_2$. . . . 265
4.5 The circuit $\mathsf{Gate}_{(\ell,g)}$ in hybrid experiment $\mathsf{Hyb}^{(b)}_3$. . . . 266
4.6 The circuit $\mathsf{Gate}_{(\ell,g)}$ in hybrid experiment $\mathsf{Hyb}^{(b)}_5$. . . . 267
4.7 The circuit $\mathsf{Gate}_{(\ell,g)}$ in hybrid experiment $\mathsf{Hyb}^{(b)}_{6,\ell,1}$. . . . 268
4.8 The circuit $\mathsf{Gate}_{(\ell,g)}$ in hybrid experiment $\mathsf{Hyb}^{(b)}_{6,\ell,2}$. . . . 269
4.9 The circuit $\mathsf{Gate}_{(\ell,g)}$ in hybrid experiment $\mathsf{Hyb}^{(b)}_{6,\ell,3}$. . . . 270
4.10 The circuit $\mathsf{Gate}_{(\ell,g)}$ in hybrid experiment $\mathsf{Hyb}^{(b)}_{6,\ell,4}$. . . . 270
4.11 The circuit $\mathsf{Gate}_{(\ell+1,g)}$ in hybrid experiment $\mathsf{Hyb}^{(b)}_{6,\ell,4}$. . . . 270
4.12 The circuit $\mathsf{Gate}_{(\ell,g)}$ in hybrid experiment $\mathsf{Hyb}^{(b)}_{6,\ell,5}$. . . . 271
4.13 The circuit $\mathsf{Gate}_{(\ell,g)}$ in hybrid experiment $\mathsf{Hyb}^{(b)}_9$: notation and hardcoded values. . . . 272
4.14 The circuit $\mathsf{Gate}_{(\ell,g)}$ in hybrid experiment $\mathsf{Hyb}^{(b)}_9$: procedure. . . . 273
4.15 A compiler from static corruption to adaptive corruption . . . 284

Chapter 1

Introduction

Classical public-key encryption has been widely deployed for securing communication, but it only provides *all-or-nothing* access to encrypted data: suppose a ciphertext encrypts a message "Meet me at 5pm". A user with the decryption key can learn the entire message, while a user without the key learns nothing. However, Modern distributed systems such as cloud storage, cloud computing, and content delivery networks demand *fine-grained* access to data that is often sensitive in nature. For instance, in healthcare, strict privacy laws prevent sharing patient data across hospitals or researchers—even when such sharing could improve treatment or train better AI models. Notably, data access here is typically enforced by a central authority that users must trust. But, all privacy is lost if this central authority is compromised. This motivates the need to make such systems *decentralized*, where no single party is fully trusted. Advanced cryptographic tools such as functional encryption offer a path to overcome these challenges: they allow fine-grained access to *encrypted data*, thus ensuring privacy regulations are met. Further, decentralized variants of FE can help obtain decentralization at the system level. This thesis, at the intersection of applied and theoretical cryptography, develops new techniques for functional encryption and explores its applications to decentralized systems.

There are three main themes of this thesis:

- Building new functional encryption schemes for decentralized access control.
- Applications of functional encryption towards realizing fair exchange.
- Applications of functional encryption towards realizing anonymous communication.

I summarize these contributions below, which form the core chapters of this thesis.

1.1 Decentralized Access Control

Access control is a fundamental problem in computer security, where the goal is to ensure that only authorized users can access certain data. Attribute-Based Encryption (ABE) provides fine-grained access control over encrypted data by binding decryption privileges directly with user credentials (attributes). While ABE has been studied extensively over the past two decades, traditional ABE schemes rely on a single central authority possessing a master secret key to issue attribute-based decryption keys. This centralized trust assumption is a critical barrier to large-scale, real-world deployment. Multi-Authority ABE (MA-ABE) addresses this limitation by enabling multiple independent authorities to concurrently manage disjoint sets of attributes. Lewko and Waters [LW11a] introduced the first truly decentralized MA-ABE scheme; however, they could only prove security in a model allowing static corruption of authorities. This does not model the real-world where an adversary can adaptively corrupt authorities. A recent work [DKW23] designed a new construction, achieving resistance against adaptive corruption of authorities for the first time, but at the cost of poor efficiency. The main reason for this inefficiency is the new and complex proof techniques of 'dual system with dual-subsystems' to handle adaptive corruptions.

While this is great progress, MA-ABE still does not enjoy many of the features of traditional ABE schemes that are critical for real-world deployment. Let us try to understand these via a deployment pipeline: (i) a real-world access policy needs to first be converted into a MA-ABE friendly intermediate representation, and (ii) we can invoke the underlying MA-ABE scheme to enforce it. First, existing MA-ABE schemes only support boolean formulas as access policies, which have limited expressivity. This is the first source of inefficiency. In contrast, traditional ABE schemes support computation models such as arithmetic span programs, circuits, finite automata, Turing machines, and random access machines. Second, the system parameters of MA-ABE are also a source of inefficiency, and this can show-up across multiple dimensions such as: (i) support for attribute-reuse within a policy (one-use vs multi-use), (ii) attribute universe size (small-universe (polynomial) vs large-universe (super-polynomial)), (iii) number of attributes appearing per policy or user key (bounded vs unbounded). The state-of-the-art MA-ABE schemes [DKW23, CCG+23] are multi-use, but small-universe and bounded. In contrast, we know of traditional ABE schemes that support multi-use, large-universe, and unbounded attributes simulataneously. On a technical level, addressing the first source of inefficiency is much more challenging in the multi-authority setting due to the decentralized nature of attribute management. Our premise is that the proof techniques of [DKW23, CCG+23] are quite complex, and it is difficult to extend them to achieve the above features. Thus, a key question is:

> *Can we design a fully secure MA-ABE scheme, while avoiding complex proof techniques such as 'dual system with dual-subsystems'?*

Simplifying adaptive security proofs. We answer this question affirmatively in our work [DTV25]. In particular, we prove that the classic Lewko–Waters [LW11a] scheme is in fact secure against adaptive corruptions. Towards achieving this surprising result, we introduce a new security proof for their construction involving novel dual system encryption techniques. These new proof techniques are much simpler than prior works [DKW23,CCG+23] and avoid the need for complex 'dual-subsystems' altogether. There are two immediate impacts of our proof techniques: (i) we obtain a tigher security analysis of the Lewko-Waters scheme, and (ii) we obtain efficiency gain in both computational and communication complexity compared to [DKW23].

Extending to multi-use attributes. This MA-ABE construction supports policies that can be expressed by monotone Boolean Span Programs (BSP), is small-universe and is restricted to one-use setting (all features matching Lewko-Waters). Similar to how [CCG+23] extends [DKW23] to multi-use setting, we also extend our scheme to support multi-use of attributes in access policies. This is enabled by the techniques of [KW19].

Having simplified the proof techniques for MA-ABE supporting monotone BSP policies, we then turn to addressing the first source of inefficiency:

Can we design a MA-ABE scheme supporting a richer policy class?

Supporting arithmetic span programs. Building upon our simplified construction and novel proof technique, we design the first MA-ABE scheme supporting arithmetic span programs (ASP), thereby enabling richer policies. In particular, ASPs subsume boolean span programs, arithmetic branching programs, and boolean branching programs. Our construction supports adaptive corruption of user secret keys, and adaptive corruption of authorities,but the latter comes with some restrictions that we believe are related to fundamental barriers in the multi-authority setting for ASP policies.

We present all these results in Chapter 2. Our contributions significantly advance the state-of-the-art of MA-ABE, both in terms of efficiency and expressivity of access policies.

1.2 Fair Exchange

Consider a scenario where a buyer wants to pay a seller in exchange for data, but neither party trusts the other to fulfill their side of the bargain. Fairness requires that either both parties receive what they expect, or neither does. To address these challenges, the fundamental problem of *fair exchange* has been extensively studied in cryptography, and unfortunately, it suffers from impossibility results in two-party settings

without a trusted third party. Recently, blockchains have been used as a trusted third party to enable fair exchange in a decentralized manner. In more advanced scenarios, such as a buyer paying to train an AI model f on a proprietary dataset x (such as, patient records), the exchanged items are not simple digital assets with an "all-or-nothing" flavor, but rather insights derived from sensitive data. Crucially, in addition to fairness, the seller also has a privacy requirement: it wants to ensure that the buyer only learns a specific function evaluation $f(x)$ on the data (such as, fine-tuned model parameters) without revealing the entire dataset x.

Solutions in trustless digital environments like blockchain-based Web3 systems typically fall into two categories: (1) Smart contract-powered solutions and (2) cryptographic solutions leveraging tools such as adaptor signatures. The former approach offers atomic transactions where the buyer learns the function evaluation $f(x)$ (and not x entirely) upon payment. However, this approach is often inefficient, costly, lacks privacy for the seller's data, and is incompatible with systems that do not support smart contracts with required functionalities. In contrast, the adaptor signature-based approach addresses all of the above issues but comes with an "all-or-nothing" guarantee, where the buyer fully extracts x and does not support functional extraction of the sensitive data. Thus, a key question is:

> *Can we design efficient cryptographic protocols that achieve fair exchange with privacy, and compatibility with systems that do not support complex smart contracts?*

Fair Exchange for selling linear functions. In our work [VST24a], we introduce a new primitive called *Functional Adaptor Signatures (FAS)* and present protocols using it that meet all of the above requirements. Notably, the protocol involves posting only a single signature on-chain for the payment, thus reducing on-chain costs to the bare minimum. On a technical level, our protocol enables selling linear functions by using *functional encryption (FE)* techniques alongwith simple adaptor signatures. Here, functional encryption enables seller privacy and adaptor signatures enable fairness.

Supporting arbitrary functions. While the above work lays the foundations of *functional fair exchange*, it is limited to linear functions due to the incompatability of existing FE schemes with adaptor signatures. To overcome the barrier, in a follow-up work [VGTS26], we introduce new techniques to build FAS from *fully homomorphic encryption (FHE)*, thus significantly broadening the functionality to arbitrary functions. On a technical level, we built an efficient NIZK for proving equality of ElGamal and FHE encryptions. We also introduced implementation optimizations to mitigate some of the heavy overheads from using FHE. Our results thus lay the foundation of fair exchanges for selling functions such as fine-tuning ML models.

Formalizing Functional Fair Exchange. Additionally, we formalize the notion of *Functional Fair Exchange (FFE)*, which captures the requirements of fair exchange with privacy, and show how to construct FFE using FAS. This formalization generalizes the FAS-based fair-exchange protocols, providing a standalone target for future work to explore alternative constructions or different trade-offs, such as improved efficiency at the cost of more interaction.

We present these results in Chapter 3. These contributions significantly advance the state-of-the-art of fair exchange with privacy.

1.3 Anonymous Communication

Anonymous communication enables users to send messages without revealing their identities or communication patterns. Traditionally, this has been achieved by using a decentralized network of routers that relay messages while hiding the sender-receiver relationships. Classical protocols such as DC-Nets and Tor are interactive and rely on threshold assumptions about honest routers; if the assumption is broken, no anonymity is left.

To address these limitations, Shi and Wu [SW21] proposed the non-interactive anonymous routing (NIAR) model, which treats all routers as potentially malicious, thus making no threshold trust assumptions. They showed how to build NIAR using a decentralized variant of functional encryption (FE) called multi-client FE [SW21]. However, their construction required $O(n^2)$ router computation, where n is the number of senders/receivers. This quadratic overhead is a significant barrier to real-world deployment and much worse than classical protocols. In terms of security, in addition to corrupting all routers, they also allowed the adversary to statically corrupt a subset of senders and receivers. While corrupting senders and receivers is natural, static corruption does not model real-world adversaries that can adaptively corrupt them. Thus, a key question is:

Can we design NIAR schemes with improved efficiency and stronger security guarantees?

Improving efficiency and security. In our work [FSS+23], we introduce the first NIAR scheme with quasi-linear router computation and proved security in presence of adaptive corruption of senders/receivers. We achieved the efficiency gain by using indistinguishability obfuscation (iO), a primitive related to FE. Our approach employs a network of small obfuscated programs, and we developed new authentication techniques to reason about its security. Further, we design a compiler that converts any NIAR scheme secure against static corruptions into one secure against adaptive corruptions. While our iO-based scheme is asymptotically more efficient, its reliance on iO means it is less practical than Shi and Wu's scheme. But

an upside of our compiler is that it can be applied to their scheme as well, yielding a NIAR scheme with quadratic router computation and adaptive security.

We present these results in Chapter 4. These contributions significantly advance the state-of-the-art of NIAR.

1.4 Summary

In this thesis, we advance the state-of-the-art of functional encryption by presenting new techniques and applications. First, we build new techniques for functional encryption in the context of decentralized access control. Then, we explore applications of functional encryption to fair exchange, and anonymous communication. For fair exchange, we also introduce protocols that broaden functionality and improve efficiency by using homomorphic encryption.

Chapter 2

Multi-Authority Attribute-Based Encryption

In this chapter, we explore functional encryption in the application context of *decentralized access control*, where the goal is to ensure that only authorized users can access certain data. Attribute-Based Encryption (ABE), a special type of functional encryption, is a powerful cryptographic primitive that helps achieve this goal. In ABE, user decryption keys are associated with a set of attributes (example, roles, permissions), and ciphertexts are associated with an access policy (example, a boolean formula over attributes). However, traditional ABE schemes rely on a single trusted authority to manage attributes and issue keys, which can be a single point of failure and a bottleneck in decentralized systems. To address this limitation, we focus on *Decentralized Multi-Authority Attribute-Based Encryption (MA-ABE)*, where multiple independent authorities manage disjoint sets of attributes and issue keys to users.

Lewko and Waters [LW11a] introduced the first truly decentralized MA-ABE scheme. However, they could only prove security in a model that allows static corruption of authorities. Due to the dynamic nature of decentralized MA-ABE, it is more realistic to consider an *adaptive corruption* model, where the adversary can choose which authorities to corrupt at any time. Addressing this challenge was a long-standing open problem, and was recently resolved by Datta et al. [DKW23], who proposed the first MA-ABE scheme secure against adaptive corruptions. However, their security proof relies on a complex "dual system with dual-subsystems" technique, which leads to making the new scheme less efficient than Lewko-Waters.

In this chapter, we present a new decentralized MA-ABE scheme that is secure against adaptive corruptions, while being significantly more efficient than the scheme of Datta et al. [DKW23]. Our scheme is in fact a small modification of the Lewko-Waters [LW11a] construction. We prove the security of our scheme using novel dual system techniques, which eventually help us replace the last transition of Lewko-Waters security proof from a computational step to an information-theoretic step. Additionally, we show how to extend our scheme to support *multi-use* of attributes in access policies. Furthermore, we

demonstrate how our techniques can be applied to construct an efficient MA-ABE scheme for *arithmetic span programs*, which are more expressive than the boolean formulas supported by prior MA-ABE schemes.

As the lead author of this work, I was responsible for designing the core components of our scheme, as well as proving its security. I also played a key role in developing the techniques that enabled us to extend our scheme to support multi-use attributes and arithmetic span programs.

2.1 Introduction

Attribute-Based Encryption (ABE) [SW05, GPSW06] provides fine-grained control over encrypted data access by associating decryption privileges directly with user credentials (attributes). Such schemes enable authorized users possessing appropriate attributes to decrypt specific messages without revealing unintended information. Over the years, designing robust and practical ABE schemes has been a vibrant research area, leading to numerous advances that balance critical trade-offs among expressiveness, efficiency, security, and cryptographic assumptions. [BSW07, OSW07, Wat09, LOS+10, LW10, OT10, AFV11, LW11b, Wat11, LW12, OT12, Wat12, Boy13, GGH+13b, GVW13, Att14, BGG+14, Wee14, CGW15, Att16, BV16, ABGW17, GKW17, CGKW18, Att19, AMY19, GWW19, KW19, Tsa19, KW20, AY20, BV20, GW20, LL20b, LL20a, GW20, AT20, Wee21a, Wee21b, Wee22, RW22, AYY22, LLL22, CW23, ARYY23, JLL23, HLL23, CW24, Wee24, DHM+24, AKY24, HLL24, WW24, Wee25, CW25]. Alongside these exciting advances, ABE has seen growing adoption in practical applications [GPSW06, ETS18, VAH23, LVV+23] and standardization efforts by NIST [PB23] and ETSI [ETS18].

Multi-Authority Attribute-Based Encryption (MA-ABE). Traditional ABE schemes mentioned above rely on a single central authority possessing a master secret key to issue attribute-based decryption keys. To address the inherent limitations of centralized trust – a critical barrier to large-scale, real-world deployment – Chase [Cha07] introduced the concept of Multi-Authority Attribute-Based Encryption (MA-ABE), which enables multiple independent authorities to concurrently manage disjoint sets of attributes. Each authority issues secret keys to users for attributes under its individual control, without requiring coordination or interaction with other authorities. Consequently, a user holding attributes from multiple authorities can decrypt ciphertexts associated with an access policy by seamlessly combining keys obtained from the relevant attribute authorities.

Fully Adaptive Security for MA-ABE. Similar to standard ABE, Multi-Authority ABE naturally demands collusion resistance against unauthorized users; however, MA-ABE introduces an additional challenge: the possibility that some attribute authorities themselves may become corrupted and collude with ma-

licious users. Addressing this critical security aspect, prior works have formalized various corruption models, typically allowing adaptive corruption of users' secret keys while limiting authorities to static corruption—chosen by an adversary at the outset of the security game. Given the inherently decentralized and dynamic nature of MA-ABE, such assumptions seem overly restrictive and unrealistic, as they imply that adversaries must commit upfront to corrupting specific authorities even before observing any issued keys or interactions. Also, in practical scenarios, it is natural to assume authorities may join or leave dynamically over time. Acknowledging these considerations, recent research has advanced toward a more robust and realistic notion known as the fully adaptive security model, where both attribute authorities and user keys can be corrupted adaptively at any point during the security game. This strengthened security definition—which captures realistic threats and dynamic environments—is the central focus of this chapter.

The State-of-the-Art in MA-ABE. Despite substantial progress in designing expressive and efficient single-authority ABE schemes with strong security guarantees, extending these desirable features to the multi-authority setting remains a challenging and delicate task. As a result, only a handful of MA-ABE constructions exist, and all suffer from limitations – whether in terms of security, efficiency, or overall complexity. After several early proposals with notable shortcomings [Cha07, LCLS08, MKE09a, CC09, MKE09b], Lewko and Waters [LW11a] introduced the first truly decentralized MA-ABE scheme. In their construction, any party can act as an independent authority by simply publishing a public key and issuing secret keys to users for attributes under its control—without requiring any global coordination beyond an initial trusted setup. Authorities operate independently, need not be aware of each other's existence, and can join the system at any time, with no bound on the total number of authorities over the system's lifetime. Their scheme supports all access policies expressible as monotone boolean span programs (BSP) [Bei96a], offering expressive policy enforcement in a fully decentralized environment. The security of the scheme is established in the random oracle model using the powerful dual system encryption framework [Wat09, LW10, LOS+10], under subgroup-decision style assumptions in composite-order bilinear groups.

Following the foundational work of Lewko and Waters [LW11a], a number of extensions and refinements to decentralized MA-ABE was proposed. Okamoto and Takashima [OT20a] presented a construction over prime-order bilinear groups under the Decision Linear (DLIN) assumption [BBS04]. Subsequent works [RW15, AG21, Ven23, dlPVA23] improved the efficiency of pairing-based MA-ABE schemes, but at the expense of weaker security guarantees and/or reliance on less standard q-type assumptions or the Generic Group Model (GGM). Similarly, in the lattice setting, existing decentralized MA-ABE constructions [DKW21, WWW22, CLW25] continue to face limitations in expressiveness, offer weaker security models, and often

rely on nonstandard knowledge-based assumptions.

Despite significant progress, all MA-ABE schemes discussed thus far fall short of supporting adaptive corruption of attribute authorities. In fact, achieving fully adaptive security in a decentralized MA-ABE setting remained an open challenge for over a decade. Strikingly, this problem represents a rare instance where standard complexity leveraging or guessing-style arguments fail—even when one tolerates sub-exponential security loss [SW05, BB11, JW16, JKK+17, KW20]. The issue is that when applied in this context, such arguments incur an exponential security loss proportional to the maximum number of authorities involved in a ciphertext. Thus, such techniques require fixing this maximum number in advance and adjusting the security parameter accordingly, severely limiting the practicality and flexibility of the resulting schemes.

Breaking this long-standing barrier, Datta et al. [DKW23] were the first to construct a fully adaptively secure decentralized MA-ABE scheme for access policies expressed as monotone BSPs. Their construction supports an arbitrary polynomial number of adaptive user key and authority corruption queries—marking a major milestone in the field. They presented two variants of their scheme: one based on composite-order bilinear groups, and a more efficient instantiation over prime-order bilinear groups [Fre10, Gui13, dlPVA22], under the Subgroup Decision and Matrix Diffie-Hellman (MDDH) assumptions, respectively. Despite this progress, their scheme inherits a key limitation from earlier MA-ABE constructions with static authority corruption under static computational assumptions —namely, the "one-use" restriction, which permits each attribute to appear at most once in any access policy. Chen et al. [CCG+23] addressed this by extending the prime-order scheme of [DKW23] to support multi-use of attributes in the context of NC^1 access policies. Their enhancement integrates the celebrated multi-use technique of Kowalczyk and Wee [KW19], originally developed for centralized ABE systems, and retains security under the MDDH assumption. More recently, Garg et al. [GGL24] constructed a fully adaptively secure decentralized MA-ABE scheme for access policies representable by polynomial-depth monotone circuits, albeit with support for only bounded collusion.

Motivation. This work revisits the construction and proof techniques of Datta et al. [DKW23] with the goal of developing a fully adaptive decentralized MA-ABE scheme that supports arbitrary collusions of users and attribute authorities, while offering improved efficiency and a significantly simpler security analysis. Although the schemes of Datta et al. [DKW23] and its follow-up by Chen et al. [CCG+23] remain the state-of-the-art in achieving fully adaptive security against arbitrary collusions under standard static assumptions, their security proofs are notably intricate. In particular, they rely on a highly sophisticated extension of the dual system encryption technique, termed the "Dual System with Dual Sub-systems". This approach constructs two parallel systems—a "main system" and a "shadow system"—and carefully

injects entropy into the shadow system, which is then gradually transferred to the main system through a delicate sequence of hybrid transitions, involving both computational and information-theoretic arguments. To motivate this complexity, Datta et al. [DKW23] argued in their technical overview (Section 2.3) that earlier approaches based on the framework of Lewko and Waters [LW11a], and its many extensions, are fundamentally inadequate for handling adaptive authority corruptions. The key challenge lies in the structure of the authority master secret keys: in [LW11a], these keys are composed of exponents, which cannot be directly embedded into dual system proofs, as such proofs operate over group elements and rely on subgroup decision-style assumptions for computational indistinguishability. To overcome this limitation, Datta et al. [DKW23] introduced a pivotal modification: they replaced a portion of the authority master secret keys with a global public group element that is independent of any specific authority. This enabled the inclusion of corrupted authorities within the scope of the dual system proof framework. However, this innovation also introduced new technical hurdles, necessitating the development of their elaborate dual system with dual sub-system methodology to complete the proof.

While the techniques developed in [DKW23] were a major breakthrough—resolving a long-standing open problem—there has been surprisingly little follow-up work exploring whether simpler or more efficient approaches could achieve fully adaptive security. In particular, it remains unclear whether the use of two parallel subsystems—a central component of their proof technique—is truly necessary for supporting fully adaptive security under arbitrary collusions. Could a single-system design suffice instead? If so, this would yield not only a conceptually simplified and tighter security analysis, but also an immediate efficiency gain in both computation and communication complexity, while preserving the same strong security guarantees.

A second motivation of our work is to initiate the study of MA-ABE schemes where access policies are expressed in the arithmetic model of computation. To date, all existing MA-ABE constructions encode access policies using the Boolean model, and no scheme has been designed specifically for arithmetic representations. As a natural first step, we focus on access policies realizable by Arithmetic Span Programs (ASPs)—a powerful abstraction that captures a wide range of computations. ASPs can naturally represent various arithmetic operations such as sparse polynomial evaluation, mean, and variance, as well as combinatorial tasks like string matching, finite automata, and decision trees. Importantly, Boolean/arithmetic formulas, Boolean span programs, and Boolean/arithmetic branching programs can all be efficiently converted into ASPs with only polynomial blow-up [IW14a], making ASPs a strictly more expressive model for encoding policies. While in principle one could simulate arithmetic relations using general MA-ABE schemes for Boolean circuits [GGL24, CLW25] by translating each field operation into an equivalent Boolean sub-circuit, this approach is not scalable. While providing reasonable asymptotic

efficiency in theory (example, via fast integer multiplication techniques [Für07]), the concrete overhead of this approach is enormous. Moreover, there are practical settings where attribute values must be treated as atomic field elements, rather than decomposed into bits, rendering such Boolean encodings inapplicable. Note that in view of similar efficiency and applicability issues with boolean computations, arithmetic variants of various important cryptographic primitives have already been considered in the last few years both within attribute-based cryptography [AIK11, PHGR13, KOS16] and beyond [DOT19, LL20a, LL20b].

Our work builds on this direction by asking: Can we design a decentralized MA-ABE scheme that natively supports access policies defined by ASPs, avoiding the inefficiencies of Boolean encodings?

Our Results

We present the following contributions in this work:

1. Fully Adaptive Decentralized MA-ABE with Simpler Proof and Improved Efficiency. We construct a fully adaptively secure decentralized MA-ABE scheme for monotone BSP access policies that resists arbitrary collusions of users and attribute authorities. Our scheme achieves significant efficiency gains over the current state-of-the-art constructions [DKW23, CCG+23], while also offering a substantially simpler security proof. In particular, our analysis avoids the intricate dual system with dual-subsystem framework introduced in [DKW23], making the security argument more streamlined and transparent. As in [DKW23], we present two instantiations of our construction: one in composite-order bilinear groups (outlined in the Technical Overview, Section 2.2.4) and another in asymmetric prime-order bilinear groups (detailed formally in Section 2.4). The former relies on the Subgroup Decision assumption, while the latter is based on the MDDH assumption. Our prime-order scheme is derived via the composite to prime order translation technique of Chen et al. [CGKW18], which builds on prime-order conversion frameworks developed in [CGW15, GDCC16]. Along the way, we further optimize the required subspace dimensions to achieve improved performance.

At a technical level, our approach fundamentally differs from [DKW23]. Rather than embedding the master secret keys of corrupted authorities into the dual system framework—as pursued in [DKW23]—we keep these keys entirely outside the dual system machinery. Our main contribution lies in carefully revisiting and restructuring the original Lewko–Waters [LW11a] security proof with only minor adjustments to their scheme architecture. Our key insight is that the primary limitation of [LW11a]—their reliance on a final *computational* transition based on an assumption analogous to Decisional Bilinear Diffie-Hellman (DBDH) [BF01], which inherently demands static knowledge of corrupted authorities—can be overcome by replacing this step with a purely *information-theoretic* transition. However, applying this idea naively fails

due to excessive leakage of honest authorities' master key information in their existing hybrid argument. We resolve this by carefully restructuring their security analysis to strictly limit information leakage about honest authority master keys. This critical refinement yields a simpler and tighter fully adaptive security proof without significantly modifying the original [LW11a] construction. Please refer to Sections 2.2.2 to 2.2.4 below for details.

This yields a surprising and insightful discovery: despite their deep expertise in dual system techniques, neither the authors of [LW11a] nor [DKW23] identified that the construction in [LW11a] could, with a restructured proof, be elevated to support adaptive authority corruptions. In particular, Datta et al. explicitly argued in their technical overview (Section 2.3) that incorporating the authority master secret keys into the dual system framework was essential for achieving fully adaptive security – a barrier that motivated the development of their complex dual system with dual sub-systems methodology. Our work shows that this perceived necessity can, in fact, be circumvented entirely.

Furthermore, since our security proof does not rely on target-group-based computational assumptions, we are able to relocate all target group components from the original Lewko–Waters construction [LW11a]—both in the ciphertext and the authority public keys—into the first source group, which is significantly more compact and efficient. As we demonstrate in Table 2.1, our MA-ABE scheme offers substantial improvements in both size and performance over the existing fully adaptive constructions that support arbitrary collusions of users and authorities [DKW23, CCG+23]. Specifically, compared to these prior schemes, our construction achieves 33% reduction in the size of authority master secret keys and ciphertexts, and a 66% reduction in user secret key size. On the computational side, encryption requires no pairings, and decryption involves 50% fewer pairing operations.

Since our construction closely follows [LW11a], it inherits their one-use restriction, allowing each attribute to appear at most once per access policy. To overcome this and support multi-use of attributes, we adopt the technique of Kowalczyk and Wee [KW19] – as applied by Chen et al. [CCG+23] – and integrate it with our prime-order MA-ABE scheme. This results in a fully adaptively secure decentralized MA-ABE scheme for NC^1 access policies that supports attribute reuse while achieving the same efficiency gain over [CCG+23] as our single-use scheme does over [DKW23].

2. Decentralized MA-ABE for Arithmetic Span Programs (ASPs). Our second contribution is the first decentralized MA-ABE scheme for Arithmetic Span Programs (ASPs). We present constructions in both the composite-order bilinear group setting (outlined in the Technical Overview, Section 2.2.5) and the prime-order setting (Section 2.6), with security proven under the Subgroup Decision and MDDH assumptions, respectively. The security analysis builds directly upon our proof framework developed for the BSP case.

Similar to our BSP scheme, our ASP-based construction supports an unbounded number of attribute authorities, each capable of independently joining the system at any time and also tolerates adaptive corruption of arbitrarily many user secret keys. However, our security proof holds under the assumption that no authority appearing in a ciphertext is corrupted.

While the inability to ensure security for ciphertexts involving corrupted authorities may seem unsatisfying, we argue in Section 2.2.5 that this limitation reflects fundamental barriers inherent to the problem. At a high level, security for access structures represented by ASPs is guaranteed only when a user possesses secret keys for at most one value per attribute. However, in the decentralized MA-ABE setting, if an adversary corrupts an attribute authority, it gains the ability to issue keys for multiple values of the same attribute to the same user. This capability undermines the security of ASPs and enables unauthorized decryption—breaking the intended security guarantees. This limitation is closely related to why MA-ABE schemes cannot directly support access policies realizable by non-monotone linear secret sharing scheme (LSSS) [DKW21]. The only viable approach in that case is to first reduce the non-monotone LSSS to an equivalent monotone form and then apply a secure MA-ABE construction for monotone LSSS access policies [OT20a, DKW21].

Additionally, we provide a generic workaround that allows the authorities related to the challenge ciphertext to be corrupted, but the cost of weakening the functionality of the scheme: decryptors must have keys from all authorities featured in a ciphertext policy in order to decrypt the ciphertext. Security holds as long as either the adversary corrupts no authority appearing in the challenge ciphertext policy or for each GID queried, there exists an honest authority appearing in the challenge ciphertext policy who did not issue any secret key. Thus, the modified construction achieves the security model of Cini et al. [CLW25], but unlike [CLW25], our approach does not require a very-selective security model or bounds on number of authorities, and it supports fully adaptive queries, including corruption. The details of this modification can be found in Sections 2.2.6 and 2.6.4.

2.2 Technical Overview

In this section, we present the core technical ideas behind our work. We begin by explaining how we transform the construction and the security analysis of the classic Lewko–Waters MA-ABE scheme [LW11a] to achieve fully adaptive security—supporting both adaptive user key queries and adaptive corruption of attribute authorities. As discussed in the Introduction, this refinement leads to a significantly efficient construction for fully adaptively secure MA-ABE schemes for monotone BSP access policies, improving

[1]Our MA-ABE for ASP satisfies full adaptive security with Type 1 restriction as defined in Definition 2.6.2

Table 2.1: Efficiency Comparison: Comparison of fully adaptively secure decentralized MA-ABE in prime order groups. All schemes are instantiated from k-MDDH assumption in prime-order asymmetric pairing groups, with $k = 1$ (that is, Symmetric External Diffie-Hellman (SXDH) assumption). n and ℓ denote the number of rows and columns in the policy matrix $\mathbf{M}$ (and also matrix $\mathbf{N}$ in case of ASP) respectively.

(a) Communication Comparision: We omit the size of c_0 and access policy in $|\mathsf{ct}|$.

Scheme	$\lvert\mathsf{msk}_i\rvert$	$\lvert\mathsf{pk}_i\rvert$	$\lvert sk_{i,\mathsf{GID}}\rvert$	$\lvert\mathsf{ct}\rvert$	Many-use?	Policy class
DKW23 [DKW23]	$18\lvert\mathbb{Z}_q\rvert$	$6\lvert\mathbb{G}_1\rvert$	$6\lvert\mathbb{G}_2\rvert$	$12n\lvert\mathbb{G}_1\rvert$	No	monotone BSP
Section 2.4	$12\lvert\mathbb{Z}_q\rvert$	$6\lvert\mathbb{G}_1\rvert$	$2\lvert\mathbb{G}_2\rvert$	$8n\lvert\mathbb{G}_1\rvert$	No	monotone BSP
Section 2.6[1]	$24\lvert\mathbb{Z}_q\rvert$	$12\lvert\mathbb{G}_1\rvert$	$2\lvert\mathbb{G}_2\rvert$	$14n\lvert\mathbb{G}_1\rvert$	No	ASP
CCG+23 [CCG+23]	$18\lvert\mathbb{Z}_q\rvert$	$6\lvert\mathbb{G}_1\rvert$	$6\lvert\mathbb{G}_2\rvert$	$12n\lvert\mathbb{G}_1\rvert$	Yes	NC^1
Section 2.5	$12\lvert\mathbb{Z}_q\rvert$	$6\lvert\mathbb{G}_1\rvert$	$2\lvert\mathbb{G}_2\rvert$	$8n\lvert\mathbb{G}_1\rvert$	Yes	NC^1

(b) Computation Comparison: S denotes the set of attributes with respect to which decryption is performed. For each algorithm, we specify two values: #exponentiations, #pairings. For #exponentiations, $(\mathsf{count})_i$ denotes the count for group $\mathbb{G}_i$ where $i \in \{1, 2, T\}$.

Scheme	AuthSetup	KGen	Enc	Dec
DKW23 [DKW23]	$(18)_1, 0$	$(18)_2, 0$	$(6n\ell{+}12n{+}6\ell{-}3)_1 + (3)_T, 3$	$(\lvert S\rvert)_T, 12\lvert S\rvert$
Section 2.4	$(12)_1, 0$	$(7)_2, 0$	$(14n)_1 + (1)_T, 0$	$(3\lvert S\rvert)_1 + (\lvert S\rvert)_T, 6\lvert S\rvert$
Section 2.6	$(24)_1, 0$	$(7)_2, 0$	$(26n)_1 + (1)_T, 0$	$(9\lvert S\rvert)_1 + (\lvert S\rvert)_T, 6\lvert S\rvert$
CCG+23 [CCG+23]	$(18)_1, 0$	$(18)_2, 0$	$(18n)_1 + (6)_T, 3$	$(\lvert S\rvert)_T, 12\lvert S\rvert$
Section 2.5	$(12)_1, 0$	$(7)_2, 0$	$(14n)_1 + (1)_T, 0$	$(3\lvert S\rvert)_1 + (\lvert S\rvert)_T, 6\lvert S\rvert$

upon the constructions in [DKW23, LW11a, CCG+23]. We then briefly sketch how we use the technique of Kowalczyk and Wee [KW20] to extend our scheme to support multi-use of attributes within access policies.

Finally, we outline our decentralized MA-ABE construction for ASPs and elaborate on the inherent barriers to supporting authority corruptions in this more expressive arithmetic setting.

2.2.1 Background on MA-ABE and Fully Adaptive Security

Our MA-ABE schemes—like all known MA-ABE constructions—assume that each user in the system is associated with a unique global identifier GID, drawn from a universe of global identifiers $\mathcal{GID} \subset \{0,1\}^*$. For simplicity of exposition, we assume in this chapter (without loss of generality) that each attribute is managed by a distinct authority, allowing us to use the terms "authority" and "attribute" interchangeably. (We note that this restriction can be relaxed to support an a priori bounded number of attributes per authority [LW11a].) We denote the universe of attribute authorities by $\mathcal{AU}$.

Let us briefly recall the syntax of a decentralized MA-ABE scheme. Such a scheme consists of five core algorithms: GlobalSetup, AuthSetup, KGen, Enc, Dec.

- The GlobalSetup procedure takes as input the security parameter (in unary) and outputs global public parameters gp. All subsequent procedures rely on gp, though we may omit them in the notation when clear from context.

- The AuthSetup procedure can be invoked by any authority $u \in \mathcal{AU}$, generating a corresponding public-secret key pair $(\mathsf{pk}_u, \mathsf{sk}_u)$.
- Using sk_u , the authority can issue a user-specific secret key $\mathsf{sk}_{\mathsf{GID},u}$ for a user identified by global identifier $\mathsf{GID} \in \mathcal{GID}$
- At any point, using the public keys $\{\mathsf{pk}_u\}$ of selected authorities, anyone can encrypt a message msg under an access policy $(\mathbb{A}, \rho)$ to obtain a ciphertext ct.
- A user possessing a collection of secret keys $\{\mathsf{sk}_{\mathsf{GID},u}\}$ corresponding to a consistent GID can decrypt a ciphertext ct if and only if the set of attributes associated with their keys satisfies the access policy embedded in the ciphertext.

In constructions built within the random oracle model[2]—including ours —a public hash function H_1 is assumed. This function maps global identifiers GID to an appropriate domain and is specified by GlobalSetup. In the security analysis, H_1 is modeled as a random oracle.

Fully Adaptive Security. Just like standard ABE, the security of an MA-ABE scheme demands collusion resistance, that is, no group of colluding users, none of whom is individually authorized to decrypt a ciphertext, should be able to decrypt the same when they pull their secret key components together. However, in case of MA-ABE, it is further required that collusion resistance should hold even if some of the authorities collude with the adversarial users and thereby those users can freely obtain secret keys corresponding to the attributes controlled by those corrupt authorities. Decentralized MA-ABE further allows the public and secret keys of the corrupt authorities to be generated in a malicious way and still needs collusion resistance. This is crucial since, in a decentralized MA-ABE scheme, anyone is allowed to act as an attribute authority by generating its public and secret keys locally and independently of everyone else in the system. The fully adaptive security is roughly defined by the following game:

Global Setup: The challenger generates global public parameters.

Query Phase 1: The attacker is allowed to adaptively make a polynomial number of queries of the following form: (i) *Authority Setup Query* : the challenger runs AuthSetup to create a public/master key pair for an authority specified by the adversary, (ii) *Secret Key Query* : the challenger runs KGen to create a secret key for a given attribute. (iii) *Authority Master Key Query* : the challenger provides the attacker the master secret key of an authority of the adversary's choice.

[2]In fact, all known MA-ABE schemes except [WWW22] are built in the random oracle model. While [WWW22] is constructed without random oracles, it relies on the evasive LWE assumption, which is a strong, knowledge-type assumption whose variants have recently come under cryptanalytic scrutiny [BÜW24, HJL25, AMYY25].

Challenge Phase: The adversary submits two messages $\mathsf{msg}_0, \mathsf{msg}_1$, and an access structure $\mathbb{A}$ along with a set of public keys of authorities involved in the access structure. It gets back from the challenger an encryption of one of the messages (chosen at random) with respect to the access structure. It is crucial that the adversary does not hold enough secret keys/authority master keys to decrypt a message that is encrypted with respect to the access structure.

Query Phase 2: Same as in Query Phase 1 (while making sure that the constraint from the challenge phase is not violated).

Guess: The attacker guesses which message was encrypted.

The Lewko-Waters MA-ABE schemes [LW11a] consider a much weaker definition where the adversary must commit during the Global Setup phase on the set of authorities in the system as well as on the subset of corrupted authorities. Already at that point, the private/public key pairs of all non-corrupt authorities are created by the challenger and the public keys are given to the attacker. (That is, during Query Phase I and II, only queries of form: (ii) *Secret Key Query* are allowed.) The fully adaptive security definition [DKW23, CCG+23] is much more realistic given the distributed nature of MA-ABE.

2.2.2 The Lewko–Waters Construction and its Limitations in supporting Adaptive Authority Corruption

As with any centralized ABE scheme, a central challenge in MA-ABE is achieving collusion resistance. In standard ABE, this is typically accomplished by having a central authority—who knows the master secret key—generate user keys in such a way that the individual key components (corresponding to different attributes) are "tied together" using fresh randomness unique to each user. This user-specific randomization ensures that the components are compatible only within a given user's key, and cannot be combined across users to violate access policies. In the multi-authority setting, however, we aim to achieve two goals simultaneously: (i) *autonomous key generation*, where each authority independently issues keys, and (ii) *collusion resistance*, ensuring that unauthorized users cannot pool their keys to decrypt unauthorized ciphertexts.

This autonomy requirement fundamentally limits the applicability of traditional randomization techniques—since there is no central party coordinating the key components. In decentralized MA-ABE, each key component may originate from a distinct authority, with no shared state or even awareness of one another. To address this, all existing decentralized MA-ABE schemes (with the sole exception of [WWW22]) use the output of a public hash function applied to the user's global identifier GID as a source of deterministic "shared randomness" across authorities. This ensures that all key components

issued to a given user are cryptographically tied together, while maintaining full decentralization. We now briefly describe the Lewko–Waters MA-ABE scheme for monontone BSP access policies and the core challenges it faces in supporting adaptive corruption of authorities.

The Lewko-Waters MA-ABE scheme is described as follows:

- $\mathsf{gp} = (N, \mathbb{G}, \mathbb{G}_T, e, g_1, \mathsf{H}_1 : \{0,1\}^* \to \mathbb{G})$ where $(N = p_1 p_2 p_3, \mathbb{G}, \mathbb{G}_T, e)$ a composite-order bilinear group with $\mathbb{G}_{p_1}, \mathbb{G}_{p_2}, \mathbb{G}_{p_3}$ being the subgroups of $\mathbb{G}$ of order p_1, p_2, p_3 respectively and $g_1 \in \mathbb{G}_{p_1}$
- $\mathsf{pk}_u = (e(g_1, g_1)^{\alpha_u}, g_1^{y_u}), \mathsf{sk}_u = (\alpha_u, y_u)$ with $\alpha_u, y_u \leftarrow \mathbb{Z}_N$ for all $u \in \mathcal{AU}$
- $\mathsf{ct} = (C_0, \{C_{1,x}, C_{2,x}, C_{3,x}\}_{x \in [\ell]})$, where $C_0 = \mathsf{msg} \cdot e(g_1, g_1)^s$, for all $x \in [\ell]$: $C_{1,x} = e(g_1, g_1)^{\lambda_x + \alpha_{\rho(x)} r_x}$, $C_{2,x} = g_1^{r_x}$, $C_{3,x} = g_1^{\omega_x + y_{\rho(x)} r_x}$ where $(\mathbf{M}, \rho)$ is the BSP access policy with $\mathbf{M} \in \mathbb{Z}_N^{\ell \times d}$, $\rho : [\ell] \to \mathcal{AU}$ is injective, $\lambda_x = \mathbf{M}_x \cdot \mathbf{v}, \omega_x = \mathbf{M}_x \cdot \mathbf{w}$, $\mathbf{v} = (s, v_2, \ldots, v_d)$, $\mathbf{w} = (0, w_2, \ldots, w_d)$, where $v_2, \ldots v_d, w_2, \ldots w_d$ are uniform random in $\mathbb{Z}_q$.
- $\mathsf{sk}_{\mathsf{GID},u} = g_1^{\alpha_u} \mathsf{H}_1(\mathsf{GID})^{y_u}$

The security proof of the Lewko–Waters construction [LW11a] relies on the dual system encryption technique originally introduced by Waters [Wat09]. In this framework, ciphertexts and keys can exist in either normal form or various semi-functional forms. Semi-functional components are not part of the actual system—they are introduced solely for the purpose of the security proof. The core idea is as follows: a normal key can correctly decrypt both normal and semi-functional ciphertexts, and similarly, a normal ciphertext can be decrypted by either normal or semi-functional keys. However, when a semi-functional key is used to decrypt a semi-functional ciphertext, decryption fails. This asymmetry is leveraged to gradually transition from the real world to a simulated one in which the adversary learns no useful information. Security is established via a sequence of indistinguishable hybrid games. The first game corresponds to the real security game, where both the ciphertext and all user keys are in normal form. In the second game, the ciphertext is switched to semi-functional form while keeping all keys normal. Then, for an adversary making q secret key queries, the proof proceeds through q hybrids: in the kth game, the first k keys are semi-functional, and the remaining are normal. In the final game—where all keys and the challenge ciphertext are semi-functional—none of the keys are useful for decrypting the challenge ciphertext, thereby ensuring security.

The security proof of Lewko and Waters [LW11a] follows the dual system encryption framework. In the penultimate step of the proof, all user secret keys and the challenge ciphertext are in semi-functional form. The final hybrid transition switches the semi-functional challenge ciphertext from an encryption of the original message to an encryption of a random message, thereby completing the proof of security. To

argue the indistinguishability of this transition, the authors rely on a target-group-based computational assumption—specifically, Assumption 4 in [LW11a], which is a variant of the DBDH assumption. More precisely, they simulate the masking secret s as abc, where $a, b, c \leftarrow \mathbb{Z}_N$ are random exponents and unknown to the simulator. To accomplish this simulation without knowing the exponents, the reduction performs two steps depending on whether the authority associated with the challenge access policy is corrupt or honest. First, the reduction disables the effect of the rows in the challenge BSP access policy matrix that correspond to corrupted authorities by selecting a vector that is orthogonal to those rows. Such a vector is guaranteed to exist, since the set of corrupted authorities must, by definition, be unauthorized. This orthogonality allows the simulator to effectively "ignore" the need for master secret keys corresponding to the corrupted authorities. For the honest authorities, the situation is even more delicate. The reduction must embed elements of the underlying hard problem instance directly into their public keys. More precisely, the reduction has to embed the term ab within the master key components α of honest authorities, so that it can simulate the ciphertext components associated with honest authorities containing the shares of s by canceling out ab in the exponent. Crucially, this embedding must be performed before any authority is created. Otherwise, if the reduction embeds the unknown exponents from the challenge problem instance into the master secret key of an authority that is later corrupted, it will be unable to provide that master secret key to the adversary – breaking the simulation. These two technical dependencies – (1) selecting an orthogonal vector for unauthorized rows and (2) embedding problem instances into the public keys of honest authorities – require the reduction to know in advance which authorities will be corrupted. As a result, the security proof can only support static corruption of authorities.

2.2.3 Our Approach for Supporting Adaptive Authority Corruption

We begin with the same high-level objective as Datta et al. [DKW23]: avoiding the need to simulate authority master keys based on instances of computational hardness assumptions. In their work, Datta et al. achieved this by adhering to a critical design principle throughout their security proof – ensuring that all transitions involving game conditions or information about corrupted authorities are handled via information-theoretic arguments. In particular, transitions between such adjacent hybrids are shown to be statistically close by implicitly redefining all relevant components, including the authority master secret keys, without invoking computational assumptions. Our first step is to examine why this information-theoretic paradigm could not be directly applied to the original Lewko–Waters construction [LW11a], and why the invocation of the highly sophisticated "dual system with dual subsystems" framework was deemed necessary by [DKW23]. Upon careful analysis, we observe that the Lewko–Waters proof already adheres to this theme for the

most part, except in the final step, where they rely on a computational transition based on a target-group hardness assumption as explained above. A natural question that arises at this point is whether the computational transition in [LW11a] can be replaced with a purely information-theoretic step, rather than undertaking the more elaborate task of bringing the authority master keys into the fold of dual system encryption, as pursued by [DKW23]. Notably, this alternative approach leads to a significantly simpler and tighter security analysis.

However, due to the specific structure of the Lewko–Waters scheme [LW11a] and its associated hybrid transitions, we observe that the final hybrid cannot be directly transformed into an information-theoretic transition. Roughly speaking, just prior to the final hybrid step, the challenge ciphertext and all user secret keys are in semi-functional form. In this form, both the ciphertext and secret key components contain segments in the $\mathbb{G}_{p_3}$ subgroup. Notably, the $\mathbb{G}_{p_3}$ portions of the ciphertext components $\{C_{3,x}\}$ do not encode secret shares of zero, but instead encode shares of a random, independent value. Our objective in the final information-theoretic step is to transport the entropy present in the $\{C_{3,x}\}$ components into the $\mathbb{G}_{p_3}$ segments of the $\{C_{1,x}\}$ components, thereby randomizing the shares of the masking secret s. For corrupted authorities, this can be achieved by leveraging an orthogonal vector – an information-theoretic tool also used in [DKW23]. Importantly, using this vector does not violate adaptive corruption constraints since the transition remains entirely information-theoretic. For honest authorities, however, this entropy transfer must be achieved using the p_3 components of the master secret keys, conditioned on the adversary's knowledge of user secret keys issued by those authorities. Unfortunately, in the security proof of [LW11a], the semi-functional form of the user secret keys fully reveals the p_3 components of an authority's master secret key when just two such keys are issued for distinct GIDs. To resolve this, we must redesign the $\mathbb{G}_{p_3}$ segments of user secret keys so that even a polynomial number of such keys leaks only limited information about the corresponding authority master secret keys. This, in turn, necessitates a re-orchestration of the entire sequence of hybrid transitions in [LW11a].

2.2.4 Our composite-order Fully Adaptive MA-ABE scheme for BSP

We start by introducing several modifications to the original construction [LW11a]. First, inspired by the approach of [DKW23], we include an element $h \leftarrow \mathbb{G}$ in the global parameters and mask the ciphertext payload msg as $\mathsf{msg} \oplus \mathsf{H}_2(e(g_1, h)^s)$, where $\mathsf{H}_2 : \mathbb{G}_T \to \{0,1\}^*$ is a universal hash function [CW77, CS02] (used as a randomness extractor [V+12]) and $s \leftarrow \mathbb{Z}_N$. In the security proof, we shift g_1 from being an element of the p_1 subgroup to an element $g_1 g_3$ in the $p_1 p_3$ subgroup. As a result, the payload becomes masked by $\mathsf{H}_2(e(g_1, h)^s \boxed{e(g_3, h)^s})$, allowing us to utilize the entropy in $s \mod p_3$ to fully randomize the

payload in the final game. Additionally, since the only step in [LW11a] that relies on a target-group-based computational assumption is the final transition – which our proof replaces with information-theoretic transition – we no longer need to embed the secret shares of s in the target group. Instead, we provide ElGamal-style encryptions of the secret shares in the source group $\mathbb{G}$, under the corresponding authority public keys. Specifically, for each row x in the BSP access structure $(\mathbf{M}, \rho)$, we include: $C_{1,x} = g_1^{\lambda_x + \alpha_{\rho(x)} r_x}, C_{2,x} = g_1^{r_x}, C_{3,x} = g_1^{\omega_x + y_{\rho(x)} r_x}$ for all rows x of the associated BSP access structure $(\mathbf{M}, \rho)$.

This transformation results in smaller ciphertexts and faster encryption, since source group elements are significantly more compact than target group elements and exponentiations are much faster there. For user secret keys, instead of generating them as $\mathsf{sk}_{\mathsf{GID},u} = g_1^{\alpha_u} \mathsf{H}_1(\mathsf{GID})^{y_u}$ — as done in the original [LW11a] construction—we define them as $\mathsf{sk}_{\mathsf{GID},u} = h^{\alpha_u} \mathsf{H}_1(\mathsf{GID})^{y_u}$. Observe that, unlike [DKW23], we do not use the element h to eliminate the α components of the authority master keys. Instead, we retain α as an integral part of the authority's secret.

Security Analysis. We now outline our security proof strategy. For simplicity of exposition in this overview, we exclude the case of malicious authorities—that is, authorities created by the adversary itself. This simplification does not impact the validity or structure of the security argument. The formal security proofs of our constructions, presented in the main body, fully account for maliciously generated authorities and handle them accordingly.

Hyb_0**:** This is the real fully adaptive security game described in Section 2.2.1.

Hyb_1**:** Modify the random oracle H_1 to return random elements from $\mathbb{G}_{p_1}$. This change is indistinguishable under the subgroup decision assumption between $\mathbb{G}_{p_1}$ and $\mathbb{G}$ (Assumption 4.1 in [DKW23]). Consequently, for any uncorrupted authority u, the y_u value modulo p_2 and p_3 are *information theoretically* hidden no matter how many keys the attacker requests from the authority u.

Hyb_2**:** Add a $\mathbb{G}_{p_3}$ component to each part of the challenge ciphertext. That is, the components of the challenge ciphertext takes the form $C_0 = \mathsf{msg}_b \oplus \mathsf{H}_2(e(g_1, h)^s \boxed{e(g_3, h)^{s''}})$, for all $x \in [\ell]$: $C_{1,x} = g_1^{\lambda_x + \alpha_{\rho(x)} r_x} \boxed{g_3^{\lambda''_x + \alpha_{\rho(x)} r''_x}}, C_{2,x} = g_1^{r_x} \boxed{g_3^{r''_x}}, C_{3,x} = g_1^{\omega_x + y_{\rho(x)} r_x} \boxed{g_3^{\omega''_x + y_{\rho(x)} r''_x}}$, where $b \leftarrow \{0,1\}, s'', \{r''_x\}_{x \in [\ell]} \leftarrow \mathbb{Z}_N$, $\{\lambda''_x\}_{x \in [\ell]}, \{\omega''_x\}_{x \in [\ell]}$, are secret shares of s'' and zero with respect to the challenge BSP access policy $(\mathbf{M}, \rho)$. This transition follows from the subgroup decision assumption between $\mathbb{G}_{p_1}$ and $\mathbb{G}_{p_1 p_3}$ (Assumption 4.2 in [DKW23]).

Hyb_3: We change the $\mathbb{G}_{p_3}$ components of $\{C_{3,x}\}_{x\in[\ell]}$ to include shares of a random value instead of zero that is, $\{\omega''_x\}_{x\in[\ell]}$ are now shares of some $w'' \leftarrow \mathbb{Z}_N$. This is an information theoretic step relying on two facts. (1) That the attacker has no information on $y_u \pmod{p_3}$ of any uncorrupted authority u per our step in Hyb_1. The fact that $y_u \mod p_3$ is hidden (and each authority appears at most once in a ciphertext) means that $C_{3,x}$ cannot be distinguished from random in the $\mathbb{G}_{p_3}$ subgroup. Thus, the share is hidden when row x corresponds to an uncorrupted authority u. (2) That the rows of the challenge matrix $(\mathbf{M}, \rho)$ associated with the corrupted authorities are unauthorized for decryption. Hence, they are insufficient for learning the value of the p_3 component of the shared secret.

Critically, this step relies on an information-theoretic argument, which eliminates the challenges associated with embedding a reduction to a computational assumption in the presence of adaptive corruptions. As highlighted earlier, this principle is a central theme of our entire reduction strategy. Throughout the proof, we separate computational and information-theoretic components. All aspects of the analysis that depend on the adversary's corruption behavior are confined to the information-theoretic segments, where adaptivity poses no difficulty.

Hyb_4: Add a $\mathbb{G}_{p_2}$ component to each part of challenge ciphertext. That is, the challenge ciphertext takes the form $C_0 = \mathsf{msg}_b \oplus \mathsf{H}_2(e(g_1,h)^s \boxed{e(g_2,h)^{s'}} e(g_3,h)^{s''})$, for all $x \in [\ell]$: $C_{1,x} = g_1^{\lambda_x+\alpha_{\rho(x)}r_x} \boxed{g_2^{\lambda'_x+\alpha_{\rho(x)}r'_x}} g_3^{\lambda''_x+\alpha_{\rho(x)}r''_x}$, $C_{2,x} = g_1^{r_x} \boxed{g_2^{r'_x}} g_3^{r''_x}$, $C_{3,x} = g_1^{\omega_x+y_{\rho(x)}r_x} \boxed{g_2^{\omega'_x+y_{\rho(x)}r'_x}} g_3^{\omega''_x+y_{\rho(x)}r''_x}$, where $b \leftarrow \{0,1\}, s', \{r'_x\}_{x\in[\ell]} \leftarrow \mathbb{Z}_N, \{\lambda'_x\}_{x\in[\ell]}$, $\{\omega'_x\}_{x\in[\ell]}$, are secret shares of s' and zero with respect to the challenge access policy $(\mathbf{M}, \rho)$. This transition follows from subgroup decision assumption between $\mathbb{G}_{p_1}$ and $\mathbb{G}_{p_1p_2}$ (Assumption 4.3 in [DKW23]).

Hyb_5: Modify the random oracle H_1 to return random elements from $\mathbb{G}_{p_1p_3}$ with the restriction that the $\mathbb{G}_{p_3}$ *components of all the random oracle* H_1 *outputs being the same.* The proof that this change is indistinguishable actually goes through a sequence of sub-hybrids where we change the oracle queries one by one. Intuitively, changing the random oracle H_1 output for a certain GID is akin to making the secret key components for GID to be semi-functional. Thus, the proof will need to leverage the fact that the key components acquired by GID do not satisfy the challenge ciphertext access structure even when combined with corrupt authorities. For each GID the proof will first establish this in the $\mathbb{G}_{p_2}$ subgroup to be "temporarily semi-functional", then use this to move it to the "permanent semi-functional" space in $\mathbb{G}_{p_3}$. While doing this latter movement, the proof will carefully ensure that the $\mathbb{G}_{p_3}$ component is the same across all the random oracle H_1 queries. Finally, undo the work in the $\mathbb{G}_{p_2}$ space to make it available for moving the next GID over.

We consider the following sequence of sub-hybrids for each query GID_j for H_1.

- First modify the random oracle output $\mathsf{H}_1(\mathsf{GID}_j)$ to be a random element in $\mathbb{G}_{p_1p_2}$ instead of $\mathbb{G}_{p_1}$. This change is indistinguishable under the subgroup decision (Assumption 4.3 in [DKW23]) between $\mathbb{G}_{p_1}$ and $\mathbb{G}_{p_1p_2}$.

- Modify the $\mathbb{G}_{p_2}$ components of $\{C_{3,x}\}_{x\in[\ell]}$ to involve shares of a random value as opposed to zero, that is, $\{\omega'_x\}_{x\in[\ell]}$ are now shares of some random $w' \leftarrow \mathbb{Z}_N$. This is an information theoretic step which uses the fact that the rows of the challenge matrix $(\mathbf{M}, \rho)$ associated with the corrupted authorities in conjunction with all those rows for which the adversary requests a secret key for GID_j are unauthorized for decryption. The adaptive corruption of the authority as well as the adaptive key requests for GID_j do not cause any problems. We emphasize that since this information theoretic argument is done over the $\mathbb{G}_{p_2}$ subgroup, it does not matter whether the adversary has information about the $\mathbb{G}_{p_3}$ from keys for other global identities. This is the benefit for modifying keys one by one in an isolated subspace.

- Next, add the same $\mathbb{G}_{p_3}$ component to $\mathsf{H}_1(\mathsf{GID}_j)$ that was added to all the prior H_1 queries. We prove the indistinguishable of this transition under a subgroup decision assumption between $\mathbb{G}_{p_1p_2}$ and $\mathbb{G}$ which is a slight variant of Assumption 4.4 of [DKW23]. More precisely, the assumption we use states that an element of the form $T_1T_2 \in \mathbb{G}_{p_1p_2}$ is indistinguishable from $T_1T_2X_3 \in \mathbb{G}$ given $((N = p_1p_2p_3, \mathbb{G}, \mathbb{G}_T, e), g_1, g_2, X_1X_3, Z_2Z_3)$, where $g_1, X_1, T_1 \leftarrow \mathbb{G}_{p_1}, g_2, Z_2, T_2 \leftarrow \mathbb{G}_{p_2}, X_3, Z_3 \leftarrow \mathbb{G}_{p_3}$. This new assumption can be proven to hold under the original Assumption 4.3 of [DKW23] in three hybrid steps[3].

- Modify the $\mathbb{G}_{p_2}$ components of $\{C_{3,x}\}_{x\in[\ell]}$ to again involve shares of zero, that is, $\{\omega'_x\}_{x\in[\ell]}$ are, once again, shares of zero. This is again an information theoretic step similar to the Hyb_4.

- Remove the $\mathbb{G}_{p_2}$ component of the random oracle output $\mathsf{H}_1(\mathsf{GID}_j)$, that is, make it a random element from $\mathbb{G}_{p_1p_3}$ with the restriction that its $\mathbb{G}_{p_3}$ component is now the same as all prior H_1 query. This transition is indistinguishable under the subgroup decision assumption between $\mathbb{G}_{p_1}$ and $\mathbb{G}_{p_1p_2}$ (Assumption 4.3 of [DKW23]).

Note that in the above sequence of sub-hybrids, the $\mathbb{G}_{p_2}$ subgroup is repeatedly used to "escort" values to the $\mathbb{G}_{p_3}$ subgroup. So far, the structure of the proof broadly mirrors that of [LW11a, DKW23], although

[3]The original Assumption 4.3 of [DKW23] ensures indistinguishability between random elements of $\mathbb{G}_{p_1p_2}$ and those of $\mathbb{G}$ given the same auxiliary information as our structured variant. We can reduce our assumption to Assumption 4.3 of [DKW23] via the following hybrid steps. In the first step applying Assumption 4.3 of [DKW23], we move from a random $T_1T_2 \leftarrow \mathbb{G}_{p_1p_2}$ to a random $T_1T_2Y_3 \leftarrow \mathbb{G}$, where $Y_3 \leftarrow \mathbb{G}_{p_3}$ independent of $X_3 \in \mathbb{G}_{p_3}$. The second transition is an information-theoretic step where we visualize $T_1T_2Y_3$ as $T_1T_2(U_3X_3)$ where $U_3 \leftarrow \mathbb{G}_{p_3}$. This holds since p_3 is prime. In the third step, we move from $T_1T_2U_3X_3$ to $T_1T_2X_3$ via another application of Assumption 4.3 from [DKW23]. Here, given an instance of Assumption 4.3 of [DKW23] where the challenge element is T, we prepare our challenge element as TX_1X_3. It is easy to observe that if $T \leftarrow \mathbb{G}$, then we simulate $T_1T_2U_3X_3$ while if $T \leftarrow \mathbb{G}_{p_1p_2}$, we simulate $T_1T_2X_3$.

there are important differences in the low-level details. In particular, unlike [LW11a], which employs a single semi-functional form of the ciphertext, our approach introduces multiple semi-functional forms to handle the more complex setting of adaptive authority corruption, in addition to adaptive secret key queries. Another critical distinction from both [LW11a] and [DKW23] lies in the treatment of $\mathsf{H}_1(\mathsf{GID}_j)$: at the end of this sub-hybrid sequence, all such outputs share the same $\mathbb{G}_{p_3}$ component. This uniformity is crucial in the information-theoretic step that follows, as it tightly bounds the amount of information revealed to the adversary about $\alpha_u \mod p_3$ and $y_u \mod p_3$ for honest authorities u via secret key queries. It is at this next step that we sharply depart from both [LW11a] and [DKW23]. The former could not support adaptive authority corruption due to reliance on a computational transition at this point, while the latter resorted to an intricate proof involving a long sequence of interleaved computational and information-theoretic steps across two parallel subsystems. By contrast, we handle this transition in a single, information-theoretic step, offering both conceptual simplicity and proof efficiency.

Hyb_6**:** Change the $\mathbb{G}_{p_3}$ components of $\{C_{1,x}\}_{x\in[\ell]}$ to include shares of an independent random value as opposed to the secret s'' used in the masking term of C_0. In order to show that this change is statistically indistinguishable, we argue that the game transcript after Hyb_5 is identically distributed to that in Hyb_6 once we make some implicit adjustments. First, note that the shares $\{\lambda''_x\}_{x\in[\ell]}$ and in the set $\{\omega''_x\}_{x\in[\ell]}$ for all the rows x of the challenge access matrix $\mathbf{M}$ associated with corrupt authorities are information theoretically revealed to the adversary. However, by the game restriction the subspace spanned by those rows does not include the vector $(1,0,\ldots,0)$. We may assume that this holds modulo p_3. This means that there must exist a vector $\mathbf{z} \in \mathbb{Z}_N^d$ such that $\mathbf{z}$ is orthogonal to all these rows of $\mathbf{M}$ but is not orthogonal to $(1,0,\ldots,0)$, (i.e., the first entry of $\mathbf{z}$ is nonzero). Let vector $\mathbf{v}''$ and $\mathbf{w}''$ be the vectors generating the shares $\{\lambda''_x\}_{x\in[\ell]}$ and $\{\omega''_x\}_{x\in[\ell]}$ respectively, that is, s'' be the first entry of vector $\mathbf{v}''$. Consider the vectors $\hat{\mathbf{v}}'' = \mathbf{v}'' + t\mathbf{z}$ and $\hat{\mathbf{w}}'' = \mathbf{w}'' - \frac{t}{c}\mathbf{z}$ where $t \leftarrow \mathbb{Z}_N$ and $X_3 = h_3^c$ is the $\mathbb{G}_{p_3}$ component of $\mathsf{H}_1(\mathsf{GID})$. Since the first entry of $\mathbf{z}$ is nonzero and t is uniformly and independently distributed over $\mathbb{Z}_N$, it follows that the first entry of $\hat{\mathbf{v}}''$ is uniformly and independently distributed from s''. Also, since $\mathbf{w}''$ is already uniformly distributed so becomes $\hat{\mathbf{w}}''$. Hence, it follows that if we generate the shares $\{\lambda''_x\}_{x\in[\ell]}$ and $\{\omega''_x\}_{x\in[\ell]}$ using $\hat{\mathbf{v}}''$ and $\hat{\mathbf{w}}''$ respectively, then we arrive at Hyb_6. It is sufficient to show that the entire game transcript when the shares $\{\lambda''_x\}_{x\in[\ell]}$ and $\{\omega''_x\}_{x\in[\ell]}$ are generated using $\hat{\mathbf{v}}''$ and $\hat{\mathbf{w}}''$ respectively instead is statistically close to that at the end of Hyb_5. We divide our argument into the following three cases:

1. For rows x of $\mathbf{M}$ corresponding to corrupt authorities, the shares λ''_x and ω''_x remain unaffected by this transformation, as the vector $\mathbf{z}$ is chosen specifically to be orthogonal to these rows.

2. For rows x of $\mathbf{M}$ associated with honest authorities for which no user keys have been issued, the $\mathbb{G}_{p_3}$ segments of $C_{1,x}$ and $C_{3,x}$ that are revealed to the adversary remain unchanged if we implicitly redefine the p_3 segments of the master secret keys for these authorities as $\hat{\alpha}_{\rho(x)} = \alpha_{\rho(x)} - (r''_x)^{-1}(\mathbf{M}_x \cdot t\mathbf{z})$ and $\hat{y}_{\rho(x)} = y_{\rho(x)} + (c \cdot r''_x)^{-1}(\mathbf{M}_x \cdot t\mathbf{z})$. Since no user keys are queried for these authorities, the values $\alpha_{\rho(x)} \mod p_3$ and $y_{\rho(x)} \mod p_3$ remain information-theoretically hidden from the adversary, ensuring that this implicit change remains completely unnoticed.

3. For rows x of $\mathbf{M}$ associated with honest authorities for which one or more user keys have been issued, the $\mathbb{G}_{p_3}$ segments of $C_{1,x}$ and $C_{3,x}$ that are information-theoretically revealed to the adversary remain unaltered under our implicit redefinition of the p_3 segments of the authority master keys, as previously described. However, in contrast to the previous case, the issuance of one or more user keys (now in semi-functional form, each containing a $\mathbb{G}_{p_3}$ segment) means that the values $\alpha_{\rho(x)} \mod p_3$ and $y_{\rho(x)} \mod p_3$ are no longer information-theoretically hidden from the adversary. Nevertheless, due to our careful structuring at the end of Hyb_5 – specifically, ensuring that all H_1 queries yield identical $\mathbb{G}_{p_3}$ segments – the only information about the p_3 segments of the master secret keys of such authorities revealed to the adversary is the sum $\alpha_{\rho(x)} + c \cdot y_{\rho(x)} \mod p_3$, even when multiple keys are issued to distinct users. Crucially, our implicit redefinition satisfies $\hat{\alpha}_{\rho(x)} + c \cdot \hat{y}_{\rho(x)} \mod p_3 = \alpha_{\rho(x)} + c \cdot y_{\rho(x)} \mod p_3$. Consequently, this change remains completely indistinguishable from the adversary's perspective, even given the information available from the issued user keys.

Importantly, all the changes administered in this hybrid transition are implicit. Thus, as mentioned above, the adaptive corruption of authorities is not an issue.

Hyb_7**:** Replace C_0 with a random value unrelated to the message msg_b. Due to the work done so far, $s'' \mod p_3$ is information theoretically hidden and so s'' has at least $\log(p_3)$ bits of entropy. At this point, the security of the universal hash function H_2 hides the message.

Supporting Multi-use of Attributes. For enabling our scheme to support multi-use of attributes, we can use the same technique as [CCG+23] which built on the core 1-ABE framework by [KW19]. In the multi-use setting, some information theoretical steps in the above hybrid transition need to be changed into computational ones. The crux of this technique is that it allows us to guess the rows of LSSS, for which neither the corresponding authority is corrupted nor the corresponding secret key is queried. Hence, we can embed a computational problem into authority public keys without caring about adaptive corruptions and secret key queries. Specifically, some information theoretic steps in $\mathsf{Hyb}_3, \mathsf{Hyb}_5, \mathsf{Hyb}_6$ are changed to computational ones.

2.2.5 Our Composite-order Fully Adaptive MA-ABE scheme for ASP

Our construction of MA-ABE for BSP can be applied to arithmetic span programs. Recall that ASP is represented by two matrices and a map $(\mathbf{M}, \mathbf{N}, \rho)$ where the heights of $\mathbf{M}, \mathbf{N}$ are ℓ and $\rho : [\ell] \to \mathcal{AU}$. The ASP is satisfied by $\mathbf{z} \in \mathbb{Z}_N^S$ for $S \subset \mathcal{AU}$ if and only if $z_{\rho(x)}\mathbf{M}_x + \mathbf{N}_x$ for $x \in S$ spans $(1, 0, ..., 0)$. Informally, our construction of MA-ABE for ASP in composite-order groups is as follows:

- $\mathsf{gp} = (N, \mathbb{G}, \mathbb{G}_T, e, g_1, h, \mathsf{H}_1 : \{0,1\}^* \to \mathbb{G})$ is the same as in our scheme for Boolean LSSS.
- $\mathsf{pk}_u = g_1^{\alpha_u}, g_1^{\alpha'_u}, g_1^{y_u}, g_1^{y'_u}, \mathsf{sk}_u = (\alpha_u, \alpha'_u, y_u, y'_u)$ with $\alpha_u, y_u \leftarrow \mathbb{Z}_N$ for all $u \in \mathcal{AU}$
- $\mathsf{ct} = (C_0, \{C_{1,x}, ..., C_{5,x}\})$ where

$$C_0 = \mathsf{msg} \oplus \mathsf{H}_2(e(g_1, h)^s),\ C_{1,x} = g_1^{r_x},\ C_{2,x} = g_1^{\lambda_x + \alpha_{\rho(x)} r_x},\ C_{3,x} = g_1^{\lambda'_x + \alpha'_{\rho(x)} r_x}$$
$$C_{4,x} = g_1^{\omega_x + y_{\rho(x)} r_x},\ C_{5,x} = g_1^{\omega'_x + y'_{\rho(x)} r_x}$$

  with $\lambda_x = \mathbf{M}_x \cdot \mathbf{v},\ \lambda'_x = \mathbf{N}_x \cdot \mathbf{v},\ \omega_x = \mathbf{M}_x \cdot \mathbf{w},\ \omega'_x = \mathbf{N}_x \cdot \mathbf{w}$ for all $x \in [\ell]$, where $(\mathbf{M}, \mathbf{N}, \rho)$ is the arithmetic span program, the first element of $\mathbf{v}$ is s, the first element of $\mathbf{w}$ is 0, and ρ is injective
- $\mathsf{sk}_{\mathsf{GID},u} = h^{z_u \alpha_u + \alpha'_u} \mathsf{H}_1(\mathsf{GID})^{z_u y_u + y'_u}$ where $z_u \in \mathbb{Z}_N$ is the attribute value.

The security proof is also quite similar to our Boolean LSSS. Via a series of hybrids, ω_x and ω'_x in $\mathbb{G}_{p_3}$ are changed to the shares of the same random element, and all the secret keys are gradually changed into the semi-functional form. In the final step, we argue that s mod p_3 is information theoretically hidden to the adversary. A caveat is that we require that the authorities related to the challenge ciphertext are not corrupted in the security proof. This is because if the adversary obtains an authority secret key sk_u, it can generate a *malformed* secret key $\widetilde{\mathsf{sk}}_{\mathsf{GID},u} = h^{z_u \alpha_u} \mathsf{H}_1(\mathsf{GID})^{z_u y_u}$ for $z_u \in \mathbb{Z}_N$. The problem is that this secret key is a valid secret key that allows the adversary to incorporate the row $z_{\rho(x)}\mathbf{M}_x$ into the ASP computation in decryption, while only the rows of the form $z_{\rho(x)}\mathbf{M}_x + \mathbf{N}_x$ are supposed to be valid. This may allow the adversary to decrypt the challenge ciphertext without violating admissibility.

Preventing this kind of attack seems quite challenging. Intuitively, the authority should have the power to generate two secret keys for the same GID with different values, i.e., $h^{z_u \alpha_u + \alpha'_u} \mathsf{H}_1(\mathsf{GID})^{z_u y_u + y'_u}$ and $h^{z'_u \alpha_u + \alpha'_u} \mathsf{H}_1(\mathsf{GID})^{z'_u y_u + y'_u}$ for any $z_u, z'_u \in \mathbb{Z}_N$ s.t. $z_u \neq z'_u$. However, one divided by the other results in the malformed key. Hence, we need to remove this homomorphic structure from secret keys to avoid this attack, retaining the information theoretic security property for the security proof. It is unclear how we can overcome this challenge.

2.2.6 A Compiler to support corruption of authorities related to the challenge ciphertext

While we do not have a solution for the above problem, we present a generic workaround that allows the authorities related to the challenge ciphertext to be corrupted, but at the cost of weakening the functionality of the scheme: decryptors must possess keys from all authorities featured in a ciphertext policy in order to decrypt the ciphertext. Security holds as long as either the adversary corrupts no authority appearing in the challenge ciphertext policy or for each GID queried, there exists an honest authority appearing in the challenge ciphertext policy who did not issue any secret key. Thus, the modified construction achieves the security model of Cini et al. [CLW25], but unlike [CLW25], our approach does not require a very-selective security model or bounds on number of authorities, and it supports fully adaptive queries, including corruption.

Informally, the modified construction is obtained via a compiler consisting of two layers: an inner layer of MA-ABE for ASP with full adaptive security subject to the above caveat, and an outer layer of MA-ABE for conjunctions with full adaptive security (implied by MA-ABE for monotone BSP). In the new scheme, to encrypt a message under an ASP policy, we encrypt the message with inner scheme, then encrypt the result under policy "conjunction of all authorities in set U" using outer scheme, where U is set of authorities appearing in the ASP policy. Decryption uses all keys from set U to first decrypt outer ciphertext, recovering the inner one, which is then decrypted using the ASP scheme. The security proof proceeds by guessing at the outset whether the adversary's challenge will avoid corrupted authorities. If yes, we reduce the security to that of inner ASP scheme; if no, to the conjunction scheme. For full details, we refer to Section 2.6.4.

2.3 Preliminaries

2.3.1 Notations

Throughout, we use λ to denote the security parameter. Boldface letters such as $\mathbf{x}$ denote column vectors, and normal-font letters such as x to denote scalars. Let $\mathbf{A}$ denote a matrix over $\mathbb{Z}_q$. We use $\mathsf{span}(\mathbf{A})$ to denote the column span of $\mathbf{A}$, and we use $\mathsf{span}^m(\mathbf{A})$ to denote matrices with width m where each column lies in $\mathsf{span}(\mathbf{A})$; this means $\mathbf{M} \xleftarrow{\$} \mathsf{span}^m(\mathbf{A})$ is a random matrix m where each column is chosen uniformly from $\mathsf{span}(\mathbf{A})$. We use $\mathsf{basis}(\mathbf{A})$ to denote a basis of $\mathsf{span}(\mathbf{A})$.

Notation for pairing groups. Let PGGen be a probabilistic polynomial time algorithm that takes input a security parameter 1^λ and outputs a pairing group description $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, q, g_1, g_2, g_T)$ where $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ are groups of prime order q and $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ defines a pairing operation. Further, g_1, g_2 are random

generator in $\mathbb{G}_1, \mathbb{G}_2$ respectively and $g_T := e(g_1, g_2)$. As a shorthand, the notation $[\![x]\!]_i$ means g_i^x for $i \in \{1, 2, T\}$. For a vector $\mathbf{x} = (x_1, \ldots, x_n)^T$, for $i \in \{1, 2, T\}$, $[\![\mathbf{x}]\!]_i$ denotes a vector of n group elements $(g_i^{x_1}, \ldots, g_i^{x_n})^T$. Similarly, for a matrix $\mathbf{A} = (\mathbf{a}_1, \ldots, \mathbf{a}_\ell)$ consisting of ℓ vectors, $[\![\mathbf{A}]\!]_i$ denotes a matrix of group elements of dimensions $n \times \ell$: $([\![\mathbf{a}_1]\!]_i, \ldots, [\![\mathbf{a}_\ell]\!]_i)$. For two matrices $\mathbf{A}$ and $\mathbf{B}$ of appropriate dimensions, define $e([\![\mathbf{A}]\!]_1, [\![\mathbf{B}]\!]_2) := [\![\mathbf{AB}]\!]_T$. Further, we denote group operations with $\cdot$, i.e., $[\![\mathbf{A}]\!]_i \cdot [\![\mathbf{B}]\!]_i = [\![\mathbf{A} + \mathbf{B}]\!]_i$. Also, we define $\mathbf{B} \odot [\![\mathbf{A}]\!]_i = [\![\mathbf{BA}]\!]_i$ and $[\![\mathbf{A}]\!]_i \odot \mathbf{B} = [\![\mathbf{AB}]\!]_i$.

For distributions D_0 and D_1, we write $D_0 \approx_c D_1$ if they are computationally indistinguishable and $D_0 \approx_s D_1$ if they are statistically indistinguishable.

2.3.2 Access Structures

In this subsection, we present the definition of access structures.

Definition 2.3.1 (Access Structures [BL88, Bei96b]). *Let $\mathbb{U}$ be the attribute universe. An access structure on $\mathbb{U}$ is a collection $\mathbb{A} \subseteq 2^{\mathbb{U}} \setminus \emptyset$ of non-empty sets of attributes. The sets in $\mathbb{A}$ are called the authorized sets and the sets not in $\mathbb{A}$ are called the unauthorized sets. An access structure is called monotone if $\forall B, C \in 2^{\mathbb{U}}$ if $B \in \mathbb{A}$ and $B \subseteq C$, then $C \in \mathbb{A}$.*

2.3.3 Boolean Span Program

Definition 2.3.2 (Boolean Span Program [KW93, AHY15]). *Let $\mathcal{U} = \{u_1, \ldots, u_m\}$ be a set of variables. For each u_i, denote $\neg u_i$ as a new variable. Intuitively, u_i and $\neg u_i$ correspond to positive and negative attributes, respectively. Also let $\mathcal{U}' = \{\neg u_1, \ldots, \neg u_m\}$. A boolean span program (BSP) over $\mathbb{Z}_q$ is specified by a pair $(\mathbf{M}, \rho)$ of a matrix $\mathbf{M} \in \mathbb{Z}_q^{n \times \ell}$ and a labeling function $\rho : [n] \to \mathcal{U} \cup \mathcal{U}'$ for some integers n and ℓ. Intuitively, the map ρ labels row j with attribute $\rho(j)$.*

A boolean span program accepts or rejects an input by the following criteria. For an input $\mathbf{x} = (x_1, \ldots, x_m) \in \{0,1\}^{1 \times m}$, we define the sub-matrix $\mathbf{M}_\mathbf{x}$ of $\mathbf{M}$ to consist of the rows whose labels are set to 1 by the input $\mathbf{x}$. That is, it consists of either rows labelled by some u_i such that $x_i = 1$ or rows labelled by some $\neg u_i$ such that $x_i = 0$. We say that

$$\mathbf{x} \in \{0,1\}^{1 \times m} \text{ satisfies } (\mathbf{M}, \rho) \text{ iff } \mathbf{e}_1 \in \mathsf{rowSpan}(\mathbf{M}_\mathbf{x}),$$

where $\mathbf{e}_1 := (1, 0, \ldots, 0) \in \mathbb{Z}_q^{1 \times \ell}$.

Monotone BSP. A BSP is called *monotone* if the labels of the columns consist of only the positive literals, that is, the set $\mathcal{U}$.

2.3.4 Arithmetic Span Programs

In this section, we recall the definition of arithmetic span programs (ASP) as defined by [IW14b].

Definition 2.3.3 (Arithmetic Span Program [IW14b]). *An arithmetic span program over $\mathbb{Z}_q$ is specified by a tuple $(\mathbf{M}, \mathbf{N}, \rho)$ of matricies $\mathbf{M}, \mathbf{N} \in \mathbb{Z}_q^{n \times \ell}$, and a labeling function $\rho : [n] \to [m]$ for some integers n, ℓ and m. Intuitively, the map ρ labels row j of both matrices $\mathbf{M}$ and $\mathbf{N}$ with $\rho(j)^{th}$ attribute.*

An arithmetic span program accepts or rejects an input by the following criteria. For $j \in [n]$, let $\mathbf{M}_j$ and $\mathbf{N}_j$ denote the jth rows of $\mathbf{M}$ and $\mathbf{N}$ respectively. For an input $\mathbf{x} = (x_1, \ldots, x_m) \in \mathbb{Z}_q^{1 \times m}$, we say that

$$\mathbf{x} \in \mathbb{Z}_q^{1 \times m} \text{ satisfies } (\mathbf{M}, \mathbf{N}, \rho) \text{ iff } \mathbf{e}_1 \in span(\{x_{\rho(j)}\mathbf{M}_j + \mathbf{N}_j\}_{j \in [n]}),$$

where $\mathbf{e}_1 := (1, 0, \ldots, 0) \in \mathbb{Z}_q^{\ell}$ and span refers to linear span of a collection of row vectors.

2.3.5 Secret Sharing Schemes

In this subsection, we present the definition of secret sharing schemes.

Definition 2.3.4 (Secret Sharing Scheme [Sha79]). *A secret sharing scheme is a tuple of algorithms* (Share, Reconstruct) *where* Share *is the share algorithm and* Reconstruct *is the reconstruct algorithm. The share algorithm* Share *takes as input a secret $z \in \mathbb{Z}_q$ and a function $f : \{0,1\}^n \to \{0,1\}$, and outputs a set of shares $\{z_i\}_{i \in [m]}$ together with $\rho : [m] \to \{0, 1, \ldots, n\}$.*

- *Correctness requires that for every $x \in \{0,1\}^n$, if $f(x) = 1$, then*

$$\mathsf{Reconstruct}(f, x, \{z_i\}_{\rho(i)=0 \cup x_{\rho(i)}=1}) = z.$$

- *Security requires that for every $x \in \{0,1\}^n$, if $f(x) = 0$, then the shares $\{z_i\}_{\rho(i)=0 \cup x_{\rho(i)}=1}$ perfectly hide z.*

Definition 2.3.5 (Linear Secret Sharing Scheme (LSSS) [Sha79]). *A secret sharing scheme that is said to be linear if* Share *is a linear function of the secret z and randomness over $\mathbb{Z}_q$, and* Reconstruct *computes a linear function of the shares over $\mathbb{Z}_q$, that is, $z = \sum_{\rho(i)=0 \cup x_{\rho(i)}=1} w_i z_i$ for some constants $w_i \in \mathbb{Z}_q$.*

2.3.6 MA-ABE

An MA-ABE scheme MA-ABE = (GlobalSetup, AuthSetup, KGen, Enc, Dec) consists of five algorithms whose syntax is given below. Let $\mathcal{AU}$ denote the authority universe. Let $\mathcal{GID}$ denote the universe of global identifiers of the users. Let $\mathbb{M}$ denote the message space. We define MA-ABE for access policies specified by an access structure $\mathbb{A}$ defined on a set $U \subseteq \mathcal{AU}$. We assume that each authority control one attribute, and hence we would use the terms "authority" and "attribute" interchangably. This definition naturally

generalizes to the situation in which each authority can potentially control an arbitrary (bounded or unbounded) number of attributes (see [LW11a, RW15]). Our definition below follows [DKW23].

- $\mathsf{gp} \leftarrow \mathsf{GlobalSetup}(1^\lambda)$: The global setup algorithm takes in the security parameter λ in unary representation and outputs the global public parameters gp for the system. We assume that gp includes the descriptions of the universe of attribute authorities $\mathcal{AU}$ and universe of the global identifiers of the users $\mathcal{GID}$. Note that both $\mathcal{AU}$ and $\mathcal{GID}$ are given by $\{0,1\}^\lambda$ in case there is no bound on the number of authorities and users in the system.

- $(\mathsf{pk}_i, \mathsf{msk}_i) \leftarrow \mathsf{AuthSetup}(\mathsf{gp}, i)$: The authority $i \in \mathcal{AU}$ calls the authority setup algorithm during its initialization with the global parameters gp as input and receives back its public and master secret key pair pk_i, msk_i.

- $\mathsf{sk}_{i,\mathsf{GID}} \leftarrow \mathsf{KGen}(\mathsf{gp}, \mathsf{msk}_i, \mathsf{GID})$: The key generation algorithm takes as input the global parameters gp, a master secret key msk_i of an authority $i \in \mathcal{AU}$, and a user's global identifier $\mathsf{GID} \in \mathcal{GID}$. It outputs a secret key $\mathsf{sk}_{i,\mathsf{GID}}$ for the user.

- $\mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{gp}, \mathsf{msg}, \mathbb{A}, \{\mathsf{pk}_i\}_{i\in U})$: The encryption algorithm takes in the global parameters gp, a message $\mathsf{msg} \in \mathbb{M}$, an access policy $\mathbb{A}$ defined on a set $U \subseteq \mathcal{AU}$, and the set $\{\mathsf{pk}_i\}_{i\in U}$ of public keys for all the authorities in the set U. It outputs a ciphertext ct. We assume that the ciphertext implicitly contains $\mathbb{A}, U$.

- $\mathsf{msg}' \leftarrow \mathsf{Dec}(\mathsf{gp}, \mathsf{ct}, \{\mathsf{sk}_{i,\mathsf{GID}}\}_{i\in S})$: The decryption algorithm takes in the global parameters gp, a ciphertext ct generated with respect to some access policy $\mathbb{A}$, and a collection of keys $\{\mathsf{sk}_{i,\mathsf{GID}}\}_{i\in S}$ corresponding to attribute-user ID pairs $\{(i, \mathsf{GID})\}_{i\in S}$ possessed by a user with global identifier GID. It outputs a message msg' when the collection of attributes associated with the secret keys satisfy the access policy $\mathbb{A}$. Otherwise, decryption fails.

Correctness. An MA-ABE scheme is said to be correct if for every $\lambda \in \mathbb{N}$, $\mathsf{msg} \in \mathbb{M}$, $\mathsf{GID} \in \mathcal{GID}$, every access policy $\mathbb{A}$ defined on a set $U \subseteq \mathcal{AU}$, and every subset of attributes $S \subseteq U$ satisfying the access structure (i.e., $S \in \mathbb{A}$), it holds that

$$\Pr\left[\mathsf{msg}' = \mathsf{msg} : \begin{array}{r} \mathsf{gp} \leftarrow \mathsf{GlobalSetup}(1^\lambda), \\ \forall i \in U : (\mathsf{pk}_i, \mathsf{msk}_i) \leftarrow \mathsf{AuthSetup}(\mathsf{gp}, i) \\ \forall i \in S : \mathsf{sk}_{i,\mathsf{GID}} \leftarrow \mathsf{KGen}(\mathsf{gp}, \mathsf{msk}_i, \mathsf{GID}) \\ \mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{gp}, \mathsf{msg}, \mathbb{A}, \{\mathsf{pk}_i\}_{i\in U}) \\ \mathsf{msg}' \leftarrow \mathsf{Dec}(\mathsf{gp}, \mathsf{ct}, \{\mathsf{sk}_{i,\mathsf{GID}}\}_{i\in S}) \end{array}\right] = 1.$$

Fully Adaptive Security. We define the fully adaptive (chosen-plaintext) security for a decentralized MA-ABE scheme, namely, we consider a security game where there could be adaptive secret key queries, adaptive authority corruption queries, and adaptive challenge ciphertext query. This is formalized in the following game between a challenger and an adversary. Note that we will consider two types of authority public keys, those which are honestly generated by the challenger and those which are supplied by the adversary itself where the former type of authority keys can be corrupted by the adversary at any point of time during the game and the latter type of authority keys can potentially be malformed.

The game $\mathsf{MA\text{-}ABE}_{\mathcal{A}}^{\mathsf{fully\text{-}adaptive}}(\lambda)$ consists of the following phases:

Global Setup: The challenger runs $\mathsf{GlobalSetup}$ to generate global public parameters gp and gives it to the adversary.

Query Phase 1: The adversary is allowed to adaptively make a polynomial number of queries of the following types:

- Authority Setup Queries: The adversary request to set up an authority $i \in \mathcal{AU}$ of its choice. If an authority setup query for the same authority i has already been queried before, the challenger aborts. Otherwise, the challenger runs $\mathsf{AuthSetup}$ to create a public/master key pair $(\mathsf{pk}_i, \mathsf{msk}_i)$ for the authority i. The challenger provides pk_i to the adversary and stores $(\mathsf{pk}_i, \mathsf{msk}_i)$. Note that the challenger does not return the generated public/master key pair to the adversary.

- Secret Key Queries: The adversary makes a secret key query by submitting a pair (i, GID) to the challenger, where $\mathsf{GID} \in \mathcal{GID}$ is a global identifier and $i \in \mathcal{AU}$ is an attribute authority. If an authority setup query for the authority i has not been made already, the challenger aborts. Otherwise, the challenger runs KGen using the public/master key pair it already created in response to authority setup query for i and generates a secret key $\mathsf{sk}_{i,\mathsf{GID}}$. The challenger provides $\mathsf{sk}_{i,\mathsf{GID}}$ to the adversary.

- Authority Master Key Queries: The adversary requests the master secret key of an authority $i \in \mathcal{AU}$ to the challenger. If an authority setup query for the authority i has not been made previously, the challenger aborts. Otherwise, the challenger provides the adversary the master secret key msk_i for authority i it created in response to the authority setup query for i.

Challenge Phase: The adversary submits two messages, $\mathsf{msg}_0, \mathsf{msg}_1 \in \mathbb{M}$ and an access policy $\mathbb{A}$ defined on a set $U \subseteq \mathcal{AU}$. The adversary also submits the public keys $\{\mathsf{pk}_i\}$ for a subset of attribute authorities appearing in the access structure $\mathbb{A}$. The authority public keys $\{\mathsf{pk}_i\}$ supplied by the adversary can

potentially be malformed, i.e., can fall outside the range of AuthSetup. The access structure $\mathbb{A}$ and the authority public keys $\{\mathsf{pk}_i\}$ must satisfy the following constraints.

1. Let $U_{\mathcal{A}} \subseteq U$ denote the set of attribute authorities for which the adversary supplied the authority public keys $\{\mathsf{pk}_i\}$. Also let $U_{\mathcal{B}}$ denote the set of attribute authorities for which the challenger created the master public key pairs in response to the authority setup query of the adversary so far. Then, it is required that $U_{\mathcal{A}} \cap U_{\mathcal{B}} = \emptyset$.

2. Let S denote the subset of U containing the authorities in $U_{\mathcal{A}}$ plus the authorities for which the adversary made a master key query so far. For each global identifier $\mathsf{GID} \in \mathcal{GID}$, let S_{GID} denote the subset of U containing authorities i such that the adversary queried a secret key for the pair (i, GID). For each $\mathsf{GID} \in \mathcal{GID}$, it is required that $S \cup S_{\mathsf{GID}} \notin \mathbb{A}$.

The challenger flips a random coin $b \xleftarrow{\$} \{0,1\}$ and generates a ciphertext ct by running the Enc algorithm that encrypts msg_b under the access structure $(\mathbf{M}, \rho)$. The challenger sends ct to the adversary.

Query Phase 2: The adversary is allowed to make all types of queries as in Query Phase 1 as long as they do not violate the constraints Properties 1 and 2 above.

Guess: The adversary must submit a guess $b' \in \{0,1\}$ for b. The adversary wins if $b = b'$.

The game is formally defined in Figure 2.1. The advantage of an adversary $\mathcal{A}$ in this game is defined as

$$\mathsf{Adv}_{\mathcal{A}}^{\text{MA-ABE,fully-adaptive}}(\lambda) = |\Pr[\text{MA-ABE}_{\mathcal{A}}^{\text{fully-adaptive}}(\lambda) = 1] - 1/2|.$$

Definition 2.3.6 (Fully adaptive security of MA-ABE)**.** *An MA-ABE scheme for access structure $\mathbb{A}$ is fully adaptively secure if for any p.p.t. adversary $\mathcal{A}$ there exists a negligible function $\mathsf{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$, we have $\mathsf{Adv}_{\mathcal{A}}^{\text{MA-ABE,fully-adaptive}}(\lambda) \leq \mathsf{negl}(\lambda)$.*

Remark 2.3.7 (Fully adaptive security of MA-ABE in the Random Oracle Model)**.** *: Similar to [LW11a, RW15, OT20b], we additionally consider the aforementioned notion of fully adaptive security in the random oracle model. In this context, we assume a global hash function H published as part of the global public parameters and accessible by all the parties in the system, including the adversary. In the security proof, we model H as a random function and allow it to be programmed by the challenger. Therefore, in the fully adaptive security game described above, we further let the adversary adaptively submit H-oracle queries to the challenger, along with the key queries it makes both before and after the challenge ciphertext query.*

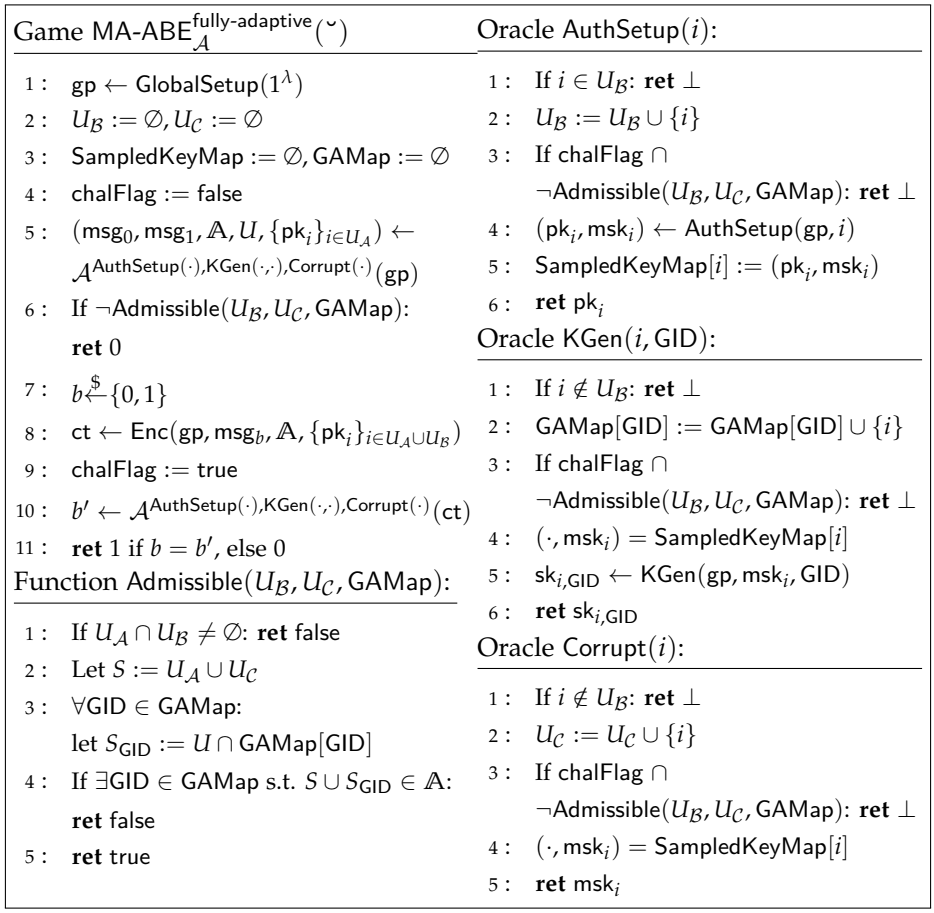


Figure 2.1: Security game for MA-ABE.

2.3.7 Assumptions

Assumption 2.3.8 (Matrix Decisional Diffie Hellman Assumption: $\mathsf{MDDH}^{\mathbb{G}_i}_{k,\ell}$ [EHK+13])**.** *Let $\ell > k > 1$. We say that the $\mathsf{MDDH}^{\mathbb{G}_i}_{k_\ell}$ assumption holds with respect to PGGen if for all p.p.t. adversary $\mathcal{A}$ and for all $i \in \{1, 2, T\}$, the following advantage is negligible in λ.*

$$\mathsf{Adv}^{\mathsf{MDDH}^{\mathbb{G}_i}_{k,\ell}}_{\mathcal{A}}(\lambda) := |\Pr[\mathcal{A}(\mathcal{D}, [\![\mathbf{t}_0]\!]_i) = 1] - \Pr[\mathcal{A}(\mathcal{D}, [\![\mathbf{t}_1]\!]_i) = 1]|$$

where $\mathcal{PG} := (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, q, g_1, g_2, g_T) \leftarrow \mathsf{PGGen}(1^\lambda)$, $\mathcal{D} = (\mathcal{PG}, [\![\mathbf{X}]\!]_i), \mathbf{t}_0 = \mathbf{Xu}, \mathbf{t}_1 \xleftarrow{\$} \mathbb{Z}_q^\ell$ such that $\mathbf{X} \xleftarrow{\$} \mathbb{Z}_q^{\ell \times k}$, $\mathbf{u} \xleftarrow{\$} \mathbb{Z}_q^k$.

When $\ell = k + 1$, we simply call it k-MDDH assumption over group $\mathbb{G}_i$.

Next, we describe some lemmas that are prime-order analogues of Subgroup Decision assumptions over composite-order bilinear groups. We describe these with respect to following set of matrices: Fix parameters $\ell_1, \ell_2, \ell_3, \ell_W$. Pick random

$$\mathbf{B}_1 \xleftarrow{\$} \mathbb{Z}_q^{\ell \times \ell_1}, \mathbf{B}_2 \xleftarrow{\$} \mathbb{Z}_q^{\ell \times \ell_2}, \mathbf{B}_3 \xleftarrow{\$} \mathbb{Z}_q^{\ell \times \ell_3},$$

where $\ell := \ell_1 + \ell_2 + \ell_3$. Let $(\mathbf{B}_1^*, \mathbf{B}_2^*, \mathbf{B}_3^*)^T = (\mathbf{B}_1, \mathbf{B}_2, \mathbf{B}_3)^{-1}$ so that $\mathbf{B}_i^T \mathbf{B}_i^* = \mathbf{I}$ (known as *non-degeneracy*) and $\mathbf{B}_i^T \mathbf{B}_j^* = \mathbf{0}$ if $i \neq j$ (known as *orthogonality*).

Assumption 2.3.9 (Subgroup Decision Assumption $\mathsf{SD}^{\mathbb{G}_2}_{\mathbf{B}_i \to \mathbf{B}_i, \mathbf{B}_j}$ for $i, j \in \{1,2,3\}$ [CGKW18, GHKW16, GDCC16]). *For all $i, j \in \{1,2,3\}$ such that $i \neq j$, the $\mathsf{SD}^{\mathbb{G}_2}_{\mathbf{B}_i \to \mathbf{B}_i, \mathbf{B}_j}$ assumptions states that for any p.p.t. adversary $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for any security parameter $\lambda \in \mathbb{N}$ and for all $k \in \{1,2,3\} \setminus \{i, j\}$,*

$$\mathsf{Adv}^{\mathsf{SD}^{\mathbb{G}_2}_{\mathbf{B}_i \to \mathbf{B}_i, \mathbf{B}_j}}_{\mathcal{A}}(\lambda) := |\Pr[\mathcal{A}(\mathcal{D}, [\![\mathbf{t}_0]\!]_2) = 1] - \Pr[\mathcal{A}(\mathcal{D}, [\![\mathbf{t}_1]\!]_2) = 1]| \leq \mathsf{negl}(\lambda)$$

where

$$\mathcal{PG} := (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, q, g_1, g_2, g_T) \leftarrow \mathsf{PGGen}(1^\lambda),$$

$$\mathcal{D} = (\mathcal{PG}, [\![\mathbf{B}_1]\!]_2, [\![\mathbf{B}_2]\!]_2, [\![\mathbf{B}_3]\!]_2, \mathsf{basis}(\mathbf{B}_i^*), \mathsf{basis}(\mathbf{B}_k^*), \mathsf{basis}(\mathbf{B}_i^*, \mathbf{B}_j^*)),$$

$$\mathbf{t}_0 \leftarrow \mathsf{span}(\mathbf{B}_i), \mathbf{t}_1 \leftarrow \mathsf{span}(\mathbf{B}_i, \mathbf{B}_j).$$

2.4 MA-ABE for monotone BSP from prime-order groups

Let $\mathcal{AU}$ denote the authority universe and let each authority control one attribute. Let $\mathcal{GID}$ denote the universe of global identifiers of the users. We construct MA-ABE for access policies specified by a monotone boolean span program (BSP) denoted by a pair $(\mathbf{M}, \rho)$ of a matrix $\mathbf{M} \in \mathbb{Z}_q^{n \times \ell}$ and a labeling function $\rho : [n] \to U$, where $U \subseteq \mathcal{AU}$ denotes a subset of authorities.

2.4.1 Construction

Our construction is as in Figure 2.2.

Correctness. We first provide an informal sketch of correctness followed by the formal details. Let's look at the three components of the partial decryptions d_x for all $\rho(x) \in S$ where S is the set of attributes possessed by the decrypter. $e(c_{2,x}, [\![\mathbf{k}]\!]_2)$ transforms an encryption of λ_x under the public key $[\![\mathbf{A}\mathbf{V}_{\rho(x)}]\!]_1$ to an encryption of $\lambda_x \mathbf{k}$ under the key $[\![\mathbf{A}\mathbf{V}_{\rho(x)}\mathbf{k}]\!]_T$. Similarly, $e(c_{3,x}, \mathsf{H}_1(\mathsf{GID}))$ transforms an encryption of ω_x under the public key $[\![\mathbf{A}\mathbf{U}_{\rho(x)}]\!]_1$ to an encryption of $\omega_x \mathbf{h}_{\mathsf{GID}}$ under the key $[\![\mathbf{A}\mathbf{U}_{\rho(x)}\mathbf{h}_{\mathsf{GID}}]\!]_T$, where $[\![\mathbf{h}_{\mathsf{GID}}]\!]_2 := \mathsf{H}_1(\mathsf{GID})$. The product of these two terms is thus an encryption of $\lambda_x \mathbf{k} + \omega_x \mathbf{h}_{\mathsf{GID}}$ under the key $[\![\mathbf{A}(\mathbf{V}_{\rho(x)}\mathbf{k} + \mathbf{U}_{\rho(x)}\mathbf{h}_{\mathsf{GID}})]\!]_T$. Finally, $e(c_{1,x}, \mathsf{sk}_{\rho(x),\mathsf{GID}})$ transforms secret key $\mathsf{sk}_{\rho(x),\mathsf{GID}} = [\![\mathbf{V}_{\rho(x)}\mathbf{k} + \mathbf{U}_{\rho(x)}\mathbf{h}_{\mathsf{GID}}]\!]_2$ to a secret key $[\![\mathbf{A}(\mathbf{V}_{\rho(x)}\mathbf{k} + \mathbf{U}_{\rho(x)}\mathbf{h}_{\mathsf{GID}})]\!]_T$. Thus, d_x is essentially a partial decryption resulting in $[\![\lambda_x \mathbf{k} + \omega_x \mathbf{h}_{\mathsf{GID}}]\!]_T$. Since λ_x are secret shares of $\mathbf{t}^T$ and ω_x are secret shares of $\mathbf{0}^T$, thus if the policy is satisfied by S, then the final value d is $[\![\mathbf{t}^T\mathbf{k} + \mathbf{0}^T\mathbf{h}_{\mathsf{GID}}]\!]_T = [\![\mathbf{t}^T\mathbf{k}]\!]_T$. Thus, the decryption outputs $\mathsf{msg} = c_0/d$.

$\mathsf{GlobalSetup}(1^\lambda)$:

1 : $\mathcal{PG} := (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, q, g_1, g_2, g_T) \leftarrow \mathsf{PGGen}(1^\lambda)$

2 : $\mathbf{A}_1 \xleftarrow{\$} \mathbb{Z}_q^{k\times(k+1)}$, $\mathbf{k} \xleftarrow{\$} \mathbb{Z}_q^{2k+1}$

3 : Sample hash function: $\mathsf{H}_1 : \mathcal{GID} \to \mathbb{G}_2^{2k+1}$

4 : **ret** $\mathsf{gp} := (\mathcal{PG}, [\![\mathbf{A}_1]\!]_1, \mathbf{k}, \mathsf{H}_1)$

$\mathsf{AuthSetup}(\mathsf{gp}, i)$:

1 : $\mathbf{V}_i, \mathbf{U}_i \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times(2k+1)}$

2 : $\mathsf{pk}_{i,0} := [\![\mathbf{A}_1\mathbf{V}_i]\!]_1, \mathsf{pk}_{i,1} := [\![\mathbf{A}_1\mathbf{U}_i]\!]_1$

3 : **ret** $\mathsf{msk}_i := (\mathbf{V}_i, \mathbf{U}_i)$, $\mathsf{pk}_i := (\mathsf{pk}_{i,0}, \mathsf{pk}_{i,1})$

$\mathsf{Enc}(\mathsf{gp}, \mathsf{msg} \in \mathbb{G}_T, (\mathbf{M}, \rho), \{\mathsf{pk}_{\rho(i)}\}_{i\in[n]})$:

1 : $\mathbf{t} \xleftarrow{\$} \mathbb{Z}_q^{2k+1}$, $\mathbf{T}_{\mathsf{bot}} \xleftarrow{\$} \mathbb{Z}_q^{(\ell-1)\times(2k+1)}$, $\mathbf{T} := \begin{pmatrix} \mathbf{t}^T \\ \mathbf{T}_{\mathsf{bot}} \end{pmatrix}$

2 : $\mathbf{W}_{\mathsf{bot}} \xleftarrow{\$} \mathbb{Z}_q^{(\ell-1)\times(2k+1)}$, $\mathbf{W} := \begin{pmatrix} \mathbf{0}^T \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix}$

3 : $\forall x \in [n]$: let $\lambda_x := \mathbf{M}_x\mathbf{T} \in \mathbb{Z}_q^{1\times(2k+1)}$, $\omega_x := \mathbf{M}_x\mathbf{W} \in \mathbb{Z}_q^{1\times(2k+1)}$

4 : $c_0 := \mathsf{msg} \cdot [\![\mathbf{t}^T\mathbf{k}]\!]_T \in \mathbb{G}_T$

5 : $\forall x \in [n]$: $\mathbf{s}_x \xleftarrow{\$} \mathbb{Z}_q^k$, $c_{1,x} := [\![\mathbf{s}_x^T\mathbf{A}_1]\!]_1 \in G_1^{1\times(k+1)}$

6 : $\forall x \in [n]$: $c_{2,x} := [\![\lambda_x]\!]_1 \cdot (\mathbf{s}_x^T \odot \mathsf{pk}_{\rho(x),0}) \in \mathbb{G}_1^{1\times(2k+1)}$

7 : $\forall x \in [n]$: $c_{3,x} := [\![\omega_x]\!]_1 \cdot (\mathbf{s}_x^T \odot \mathsf{pk}_{\rho(x),1}) \in \mathbb{G}_1^{1\times(2k+1)}$

8 : **ret** $\mathsf{ct} := (c_0, \{c_{1,x}, c_{2,x}, c_{3,x}\}_{x\in[n]})$

$\mathsf{KGen}(\mathsf{gp}, \mathsf{msk}_i, \mathsf{GID})$:

1 : Compute $[\![\mathbf{h}_{\mathsf{GID}}]\!]_2 := \mathsf{H}_1(\mathsf{GID})$

2 : **ret** $\mathsf{sk}_{i,\mathsf{GID}} := [\![\mathbf{V}_i\mathbf{k} + \mathbf{U}_i\mathbf{h}_{\mathsf{GID}}]\!]_2$

$\mathsf{Dec}(\mathsf{gp}, (\mathbf{M}, \rho), \mathsf{ct}, \mathsf{GID}, \{\mathsf{sk}_{\rho(x),\mathsf{GID}}\}_{\rho(x)\in S})$:

1 : If $(1, 0, \ldots, 0) \notin \mathit{RowSpan}(\mathbf{M}_x)$: **ret** $\perp$

2 : Let $\{w_x\}_{\rho(x)\in S}$ be constants s.t. $\sum_{\rho(x)\in S} w_x\mathbf{M}_x = (1, 0, \ldots, 0)$

3 : $\forall \rho(x) \in S$: $d_x := e(c_{2,x}, [\![\mathbf{k}]\!]_2) \cdot \frac{e(c_{3,x}, \mathsf{H}_1(\mathsf{GID}))}{e(c_{1,x}, \mathsf{sk}_{\rho(x),\mathsf{GID}})}$

4 : Compute $d := \prod_{\rho(x)\in S} d_x^{w_x}$

5 : **ret** c_0/d

Figure 2.2: Construction: MA-ABE for monotone BSP from prime-order groups

Formally, this looks as follows. d_x can be simplified as follows:

$$\begin{aligned} d_x &= e(c_{2,x}, [\![\mathbf{k}]\!]_2) \cdot \frac{e(c_{3,x}, \mathsf{H}_1(\mathsf{GID}))}{e(c_{1,x}, \mathsf{sk}_{\rho(x),\mathsf{GID}})} \\ &= [\![(\lambda_x + \mathbf{s}_x^T\mathbf{A}\mathbf{V}_{\rho(x)})\mathbf{k} + (\omega_x + \mathbf{s}_x^T\mathbf{A}\mathbf{U}_{\rho(x)})\mathbf{h}_{\mathsf{GID}} - (\mathbf{s}_x^T\mathbf{A})(\mathbf{V}_{\rho(x)}\mathbf{k} + \mathbf{U}_{\rho(x)}\mathbf{h}_{\mathsf{GID}})]\!]_T \\ &= [\![\lambda_x\mathbf{k} + \omega_x\mathbf{h}_{\mathsf{GID}}]\!]_T \\ &= [\![\mathbf{M}_x(\mathbf{T}\mathbf{k} + \mathbf{W}\mathbf{h}_{\mathsf{GID}})]\!]_T. \end{aligned}$$

Then, d can be simplified as follows:

$$\begin{aligned} d &= \prod_{\rho(x)\in S} d_x^{w_x} \\ &= [\![\sum_{\rho(x)\in S} w_x \mathbf{M}_x(\mathbf{Tk} + \mathbf{Wh}_{\mathsf{GID}})]\!]_T \\ &= [\![(1,0,\ldots,0)(\mathbf{Tk} + \mathbf{Wh}_{\mathsf{GID}})]\!]_T \\ &= [\![\mathbf{t}^T\mathbf{k} + \mathbf{0}^T\mathbf{h}_{\mathsf{GID}}]\!]_T \qquad \text{(because first rows of } \mathbf{T} \text{ and } \mathbf{W} \text{ are } \mathbf{t}^T \text{ and } \mathbf{0}^T\text{)} \\ &= [\![\mathbf{t}^T\mathbf{k}]\!]_T. \end{aligned}$$

Thus, it follows that $c_0/d = \mathsf{msg}$. Thus, correctness holds.

2.4.2 Security Analysis

Theorem 2.4.1. *The MA-ABE construction for monotone BSP in Figure 2.2 is fully adaptively secure (Definition 2.3.6). if all of the following hold true.*

- *game condition holds and ρ is injective.*
- *k-MDDH assumption holds in groups $\mathbb{G}_1$ and $\mathbb{G}_2$ (Assumption 2.3.8).*
- $\mathsf{SD}^{\mathbb{G}_2}_{\mathbf{B}_1\to\mathbf{B}_1,\mathbf{B}_2}$ *assumption holds (Assumption 2.3.9).*
- $\mathsf{SD}^{\mathbb{G}_2}_{\mathbf{B}_2\to\mathbf{B}_2,\mathbf{B}_3}$ *assumption holds (Assumption 2.3.9).*

Next, we prove Theorem 2.4.1. We prove via a sequence of hybrid games. Suppose the adversary makes Q queries to the random oracle H_1. The hybrid games are as follows: $\mathsf{Hyb}_{\mathsf{Real}}, \mathsf{Hyb}'_{\mathsf{Real}}, \mathsf{Hyb}_1, \mathsf{Hyb}_2, \{\mathsf{Hyb}_{3,j,1}, \mathsf{Hyb}'_{3,j,1}, \mathsf{Hyb}_{3,j,2}, \mathsf{Hyb}'_{3,j,2}, \mathsf{Hyb}_{3,j,3}\}_{j\in[Q]}, \mathsf{Hyb}_4, \mathsf{Hyb}_5$.

Hybrid $\mathsf{Hyb}_{\mathsf{Real}}$. This is the real-world game $\mathsf{MA\text{-}ABE}^{\mathsf{fully\text{-}adaptive}}_{\mathcal{A}}$.

Hybrid $\mathsf{Hyb}'_{\mathsf{Real}}$. This is same as $\mathsf{Hyb}_{\mathsf{Real}}$ except that the challenger computes $\boxed{(\mathsf{gp},\mathsf{st}) \leftarrow \mathsf{GlobalSetup}^*(1^\lambda)}$ and provides gp to the adversary. Here, $\mathsf{GlobalSetup}^*$ runs the same computation as GlobalSetup to compute gp and additionally also computes the following:

$$\mathbf{A}_2 \xleftarrow{\$} \mathbb{Z}_q^{1\times(k+1)}, \mathbf{B}_1, \mathbf{B}_2 \xleftarrow{\$} \mathbb{Z}_q^{(2k+1)\times k}, \mathbf{B}_3 \xleftarrow{\$} \mathbb{Z}_q^{(2k+1)\times 1}$$

$$\left(\mathbf{A}_1^*, \mathbf{A}_2^*\right) = \begin{pmatrix}\mathbf{A}_1\\ \mathbf{A}_2\end{pmatrix}^{-1}, \left(\mathbf{B}_1^*, \mathbf{B}_2^*, \mathbf{B}_3^*\right) = \left(\left(\mathbf{B}_1, \mathbf{B}_2, \mathbf{B}_3\right)^{-1}\right)^T.$$

Then, $\mathsf{st} = (\mathbf{A}_1, \mathbf{A}_2, \mathbf{A}_1^*, \mathbf{A}_2^*, \mathbf{B}_1, \mathbf{B}_2, \mathbf{B}_3, \mathbf{B}_1^*, \mathbf{B}_2^*, \mathbf{B}_3^*)$. Observe that $\forall i,j \in \{1,2\}$: $\mathbf{A}_i\mathbf{A}_j^* = \mathbf{I}$ if $i = j$, and $\mathbf{0}$ if $i \neq j$, and $\forall i,j \in \{1,2,3\}$: $\mathbf{B}_i^T\mathbf{B}_j^* = \mathbf{I}$ if $i = j$, and $\mathbf{0}$ if $i \neq j$.

Hybrid Hyb_0**.** This is same as $\mathsf{Hyb}_{\mathsf{Real}}$ except that the hash function H_1 is programmed to output all hash values in $\mathsf{span}(\mathbf{B}_1)$ as follows: on input GID, sample $\mathbf{h}_{\mathsf{GID}} \xleftarrow{\$} \boxed{\mathbb{Z}_q^k}$ and output $\mathsf{H}_1(\mathsf{GID}) = [\![\boxed{\mathbf{B}_1\mathbf{h}_{\mathsf{GID}}}]\!]_2$.

Hybrid Hyb_1**.** This is same as Hyb_0 except that the ciphertext is changed to semi-functional form. Specifically, let $U_{\mathcal{A}}$ denote the rows of $\mathbf{M}$ that correspond to the public keys provided by the adversary $\mathcal{A}$, that is $U_{\mathcal{A}} = \{i \in [n] : \rho(i) \in U_{\mathcal{A}}\}$. Let $\overline{U_{\mathcal{A}}} = [n] \setminus U_{\mathcal{A}}$. Then, we can write the normal ciphertexts as follows: $\mathsf{ct} := (c_0, \{c_{1,x}, c_{2,x}, c_{3,x}\}_{x\in[n]})$, where $c_0 := \mathsf{msg}_b \cdot [\![\mathbf{t}^T\mathbf{k}]\!]_T$, $\forall x \in [n]$: $c_{1,x} := [\![\mathbf{c}_x^T]\!]_1 := [\![\mathbf{s}_x^T\mathbf{A}_1]\!]_1$, $c_{2,x} := [\![\lambda_x]\!]_1 \cdot (\mathbf{s}_x^T \odot \mathsf{pk}_{\rho(x),0})$ if $x \in U_{\mathcal{A}}$, else $c_{2,x} := [\![\lambda_x]\!]_1 \cdot (\boxed{\mathbf{c}_x^T \odot [\![\mathbf{V}_{\rho(x)}]\!]_1})$, $c_{3,x} := [\![\omega_x]\!]_1 \cdot (\mathbf{s}_x^T \odot \mathsf{pk}_{\rho(x),1})$ if $x \in U_{\mathcal{A}}$, else $c_{3,x} := [\![\omega_x]\!]_1 \cdot (\boxed{\mathbf{c}_x^T \odot [\![\mathbf{U}_{\rho(x)}]\!]_1})$. Given this notation, the ciphertext in Hyb_1 is same as the normal ciphertext except that for all $x \in \overline{U_{\mathcal{A}}}$: $\boxed{c_{1,x} := [\![\mathbf{c}_x^T]\!]_1}$, where $\boxed{\mathbf{c}_x \xleftarrow{\$} \mathbb{Z}_q^{k+1}}$. We call this semi-functional ciphertext. Observe that in total this means that semi-functional ciphertext changes $c_{1,x}, c_{2,x}, c_{3,x}$ for $x \in \overline{U_{\mathcal{A}}}$.

Hybrid Hyb_2**.** This is same as Hyb_1 except that the ciphertext structure is changed as follows: the first row of matrix $\mathbf{W}$ is changed from $\mathbf{0}^T$ to $\gamma\mathbf{B}_3^{*T}$, where $\gamma \xleftarrow{\$} \mathbb{Z}_q$, that is, $\mathbf{W} = \begin{pmatrix} \boxed{\gamma\mathbf{B}_3^{*T}} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix}$.

Hybrid $\mathsf{Hyb}_{3,j-1}$ **for** $j \in [q+1]$**.** This hybrid is same as Hyb_2 except that for the ith global identifier GID_i for $i \leq j-1$, the challenger programs the output $\mathsf{H}_1(\mathsf{GID}_i)$ of the random oracle H_1 as $\mathsf{H}_1(\mathsf{GID}_i) = [\![\boxed{\mathbf{B}_1\mathbf{h}_{\mathsf{GID}_i} + \mathbf{B}_3}]\!]_2$, where $\mathbf{h}_{\mathsf{GID}_i} \xleftarrow{\$} \mathbb{Z}_q^k$, while for $i > j-1$, it programs the output $\mathsf{H}_1(\mathsf{GID}_i)$ of the random oracle H_1 as $\mathsf{H}_1(\mathsf{GID}_i) = [\![\mathbf{B}_1\mathbf{h}_{\mathsf{GID}_i}]\!]_2$ as earlier.

Observe that $\mathsf{Hyb}_{3,0}$ is same as Hyb_2. We introduce a sequnce of intermediate hybrids $\mathsf{Hyb}_{3,j,1}$, $\mathsf{Hyb}'_{3,j,1}$, $\mathsf{Hyb}_{3,j,2}$, $\mathsf{Hyb}'_{3,j,2}$, $\mathsf{Hyb}_{3,j,3}$ between $\mathsf{Hyb}_{3,j-1}$ and $\mathsf{Hyb}_{3,j}$ for all $j \in [q]$ as defined below.

Hybrid $\mathsf{Hyb}_{3,j,1}$ **for** $j \in [Q]$**.** This hybrid is same as $\mathsf{Hyb}_{3,j-1}$ except that for the jth global identifier GID_j, the challenger programs the output $\mathsf{H}_1(\mathsf{GID}_j)$ of the random oracle H_1 as $\mathsf{H}_1(\mathsf{GID}_j) = [\![\boxed{\mathbf{B}_1\mathbf{h}_{\mathsf{GID}_j} + \mathbf{B}_2\mathbf{h}'_{\mathsf{GID}_j}}]\!]_2$, where $\mathbf{h}_{\mathsf{GID}_j}, \mathbf{h}'_{\mathsf{GID}_j} \xleftarrow{\$} \mathbb{Z}_q^k$.

Hybrid $\mathsf{Hyb}'_{3,j,1}$**.** This is same as $\mathsf{Hyb}_{3,j,1}$ except that the ciphertext structure is changed as follows: the first row of matrix $\mathbf{W}$ is changed from $\gamma\mathbf{B}_3^{*T}$ to $(\mathbf{B}_2^*\delta + \gamma\mathbf{B}_3^*)^T$, where $\gamma \xleftarrow{\$} \mathbb{Z}_q$ and $\delta \xleftarrow{\$} \mathbb{Z}_q^k$ that is, $\mathbf{W} = \begin{pmatrix} \boxed{(\mathbf{B}_2^*\delta + \gamma\mathbf{B}_3^*)^T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix}$.

Hybrid $\mathsf{Hyb}_{3,j,2}$**.** This is same as $\mathsf{Hyb}'_{3,j,1}$ except that for the jth global identifier GID_j, the challenger programs the output $\mathsf{H}_1(\mathsf{GID}_j)$ of the random oracle H_1 as $\mathsf{H}_1(\mathsf{GID}_j) = \llbracket \boxed{\mathbf{B}_1\mathbf{h}_{\mathsf{GID}_j} + \mathbf{B}_2\mathbf{h}'_{\mathsf{GID}_j} + \mathbf{B}_3} \rrbracket_2$, where $\mathbf{h}_{\mathsf{GID}_j}, \mathbf{h}'_{\mathsf{GID}_j} \xleftarrow{\$} \mathbb{Z}_q^k$.

Hybrid $\mathsf{Hyb}'_{3,j,2}$**.** This is same as $\mathsf{Hyb}_{3,j,2}$ except that the ciphertext structure is changed as follows: the first row of matrix $\mathbf{W}$ is changed from $(\mathbf{B}_2^*\delta + \gamma\mathbf{B}_3^*)^T$ to $\gamma\mathbf{B}_3^{*T}$, where and $\gamma \xleftarrow{\$} \mathbb{Z}_q$, that is, $\mathbf{W} = \begin{pmatrix} \boxed{\gamma\mathbf{B}_3^{*T}} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix}$.

Hybrid Hyb_4**.** This is same as hybrid $\mathsf{Hyb}_{3,Q}$ except that the ciphertext structure is changed as follows: the first row of matrix $\mathbf{T}$ is changed from $\mathbf{t}^T$ to $(\mathbf{t} + \tau\mathbf{B}_3^*)^T$, where $\tau \xleftarrow{\$} \mathbb{Z}_q$, that is $\mathbf{T} = \begin{pmatrix} \boxed{(\mathbf{t} + \tau\mathbf{B}_3^*)^T} \\ \mathbf{T}_{\mathsf{bot}} \end{pmatrix}$. Crucially, we note that c_0 remains unchanged, that is, the masking term is still $[\![\mathbf{t}^T\mathbf{k}]\!]_T$.

Hybrid Hyb_5**.** This is same as Hyb_4 except that the ciphertext is changed to an encryption of a random value, that is, c_0 is changed from $c_0 := \mathsf{msg}_b \cdot [\![\mathbf{t}^T\mathbf{k}]\!]_T$ to $c_0 := \zeta \cdot [\![\mathbf{t}^T\mathbf{k}]\!]_T$, where $\zeta \xleftarrow{\$} \mathbb{G}_T$.

Claim 2.4.2. *Hybrids* $\mathsf{Hyb}_{\mathsf{Real}}$ *and* $\mathsf{Hyb}'_{\mathsf{Real}}$ *are identically distributed.*

Claim 2.4.3. *If* $\mathsf{MDDH}^{\mathbb{G}_2}_{k,2k+1}$ *holds, then* $\mathsf{Hyb}'_{\mathsf{Real}} \approx_c \mathsf{Hyb}_0$.

Claim 2.4.4. *If* $\mathsf{MDDH}^{\mathbb{G}_1}_{k,k+1}$ *holds, then* $\mathsf{Hyb}_0 \approx_c \mathsf{Hyb}_1$.

Claim 2.4.5. *If game condition holds and* ρ *is injective, then,* $\mathsf{Hyb}_1 \approx_s \mathsf{Hyb}_2$.

Claim 2.4.6. *If* $\mathsf{SD}^{\mathbb{G}_2}_{\mathbf{B}_1 \to \mathbf{B}_1,\mathbf{B}_2}$ *holds, then* $\mathsf{Hyb}_{3,j-1} \approx_c \mathsf{Hyb}_{3,j,1}$ *for all* $j \in [Q]$.

Claim 2.4.7. *If game condition holds and* ρ *is injective, then,* $\mathsf{Hyb}_{3,j,1} \approx_s \mathsf{Hyb}'_{3,j,1}$.

Claim 2.4.8. *If* $\widetilde{\mathsf{SD}}^{\mathbb{G}_2}_{\mathbf{B}_2 \to \mathbf{B}_2,\mathbf{B}_3}$ *holds , then* $\mathsf{Hyb}'_{3,j,1} \approx_c \mathsf{Hyb}_{3,j,2}$ *for all* $j \in [Q]$.

Here, we note that the $\widetilde{\mathsf{SD}}^{\mathbb{G}_2}_{\mathbf{B}_2 \to \mathbf{B}_2,\mathbf{B}_3}$ assumption is a variant of the Subgroup Decision assumption. We formally state this assumption below and show that it reduces to the original Subgroup Decision assumption in Claim 2.4.16.

Claim 2.4.9. *If game condition holds and* ρ *is injective, then,* $\mathsf{Hyb}_{3,j,2} \approx_s \mathsf{Hyb}'_{3,j,2}$.

Claim 2.4.10. *If* $\mathsf{SD}^{\mathbb{G}_2}_{\mathbf{B}_1 \to \mathbf{B}_1,\mathbf{B}_2}$ *holds , then* $\mathsf{Hyb}'_{3,j,2} \approx_c \mathsf{Hyb}_{3,j}$ *for all* $j \in [Q]$.

Claim 2.4.11. *If game condition holds and* ρ *is injective, then,* $\mathsf{Hyb}_{3,Q} \approx_s \mathsf{Hyb}_4$.

Claim 2.4.12. $\mathsf{Hyb}_4 \approx_s \mathsf{Hyb}_5$.

Claim 2.4.13. *In* Hyb_5*, adversary* $\mathcal{A}$*'s winning advantage is* 0*.*

Thus, Claims 2.4.2 to 2.4.13 and hybrid argument imply that Theorem 2.4.1 holds.

In the rest of this section, we present the $\widetilde{\mathsf{SD}}^{\mathbb{G}_2}_{\mathbf{B}_2\to\mathbf{B}_2,\mathbf{B}_3}$ assumption and the proofs of Claims 2.4.11 and 2.4.12 since they are the most non-trivial ones when compared to previous work. Proofs of other claims are deferred to Section 2.4.3.

We start by presenting the $\widetilde{\mathsf{SD}}^{\mathbb{G}_2}_{\mathbf{B}_2\to\mathbf{B}_2,\mathbf{B}_3}$ assumption. It is defined with respect to following set of matrices: Fix parameters ℓ_1, ℓ_2, ℓ_3. Pick random

$$\mathbf{B}_1 \xleftarrow{\$} \mathbb{Z}_q^{\ell\times\ell_1}, \mathbf{B}_2 \xleftarrow{\$} \mathbb{Z}_q^{\ell\times\ell_2}, \mathbf{B}_3 \xleftarrow{\$} \mathbb{Z}_q^{\ell\times\ell_3},$$

where $\ell := \ell_1 + \ell_2 + \ell_3$. Let $(\mathbf{B}_1^*, \mathbf{B}_2^*, \mathbf{B}_3^*)^T = (\mathbf{B}_1, \mathbf{B}_2, \mathbf{B}_3)^{-1}$ so that $\mathbf{B}_i^T \mathbf{B}_i^* = \mathbf{I}$ (known as *non-degeneracy*) and $\mathbf{B}_i^T \mathbf{B}_j^* = \mathbf{0}$ if $i \neq j$ (known as *orthogonality*).

Assumption 2.4.14 (Subgroup Decision Assumption Variant $\widetilde{\mathsf{SD}}^{\mathbb{G}_2}_{\mathbf{B}_2\to\mathbf{B}_2,\mathbf{B}_3}$)**.** *Fix parameter* $\ell_3 = 1$*, that is* $\mathbf{B}_3 \in \mathbb{Z}_q^{\ell}$*, where* $\ell = \ell_1 + \ell_2 + 1$*. The* $\widetilde{\mathsf{SD}}^{\mathbb{G}_2}_{\mathbf{B}_2\to\mathbf{B}_2,\mathbf{B}_3}$ *assumptions states that for any p.p.t. adversary* $\mathcal{A}$*, there exists a negligible function* $\mathsf{negl}(\cdot)$ *such that for any security parameter* $\lambda \in \mathbb{N}$*,*

$$\mathsf{Adv}^{\mathsf{SD}^{\mathbb{G}_2}_{\mathbf{B}_i\to\mathbf{B}_i,\mathbf{B}_j}}_{\mathcal{A}}(\lambda) := |\Pr[\mathcal{A}(\mathcal{D}, [\![\mathbf{t}_0]\!]_2) = 1] - \Pr[\mathcal{A}(\mathcal{D}, [\![\mathbf{t}_1]\!]_2) = 1]| \leq \mathsf{negl}(\lambda)$$

where

$$\mathcal{PG} := (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, q, g_1, g_2, g_T) \leftarrow \mathsf{PGGen}(1^\lambda),$$
$$\mathcal{D} = (\mathcal{PG}, [\![\mathbf{B}_1]\!]_2, [\![\mathbf{B}_2]\!]_2, [\![\mathbf{B}_3]\!]_2, \mathsf{basis}(\mathbf{B}_1^*), \mathsf{basis}(\mathbf{B}_2^*), \mathsf{basis}(\mathbf{B}_2^*, \mathbf{B}_3^*)),$$
$$\mathbf{t}_0 := \mathbf{B}_2\mathbf{h}, \mathbf{t}_1 := \mathbf{B}_2\mathbf{h} + \mathbf{B}_3, \mathbf{h} \xleftarrow{\$} \mathbb{Z}_q^{\ell_2}.$$

Remark 2.4.15. *Unlike the standard Subgroup Decision assumption where* $\mathbf{t}_1 := \mathbf{B}_2\mathbf{h} + r\mathbf{B}_3$ *where* $r \xleftarrow{\$} \mathbb{Z}_q$*, here the coefficient of* $\mathbf{B}_3$ *is fixed to be* 1*. Note that this assumption is crucially used for changing the hash function output to be of the form* $\mathbf{B}_1\mathbf{h} + \mathbf{B}_2\mathbf{h}' + \mathbf{B}_3$*. Eventually, the final semi-functional form of the secret key will have hash output of the form* $\mathbf{B}_1\mathbf{h} + \mathbf{B}_3$ *and this fixed (non-random)* $\mathbf{B}_3$ *term is crucial for proving Claim 2.4.11 which is the main technical component of this work.*

Claim 2.4.16. *If* $\mathsf{SD}^{\mathbb{G}_2}_{\mathbf{B}_2\to\mathbf{B}_2,\mathbf{B}_3}$ *(Assumption 2.3.9) holds, then* $\widetilde{\mathsf{SD}}^{\mathbb{G}_2}_{\mathbf{B}_2\to\mathbf{B}_2,\mathbf{B}_3}$ *(Assumption 2.4.14) holds.*

Proof of Claim 2.4.16. We will instantiate the $\mathsf{SD}^{\mathbb{G}_2}_{\mathbf{B}_2\to\mathbf{B}_2,\mathbf{B}_3}$ assumption with $\ell_3 = 1$. Then in this assumption, $\mathbf{t}_0 \leftarrow \mathsf{span}(\mathbf{B}_2)$ can be equivalently written as $\mathbf{t}_0 = \mathbf{B}_2\mathbf{h}$ for some $\mathbf{h} \xleftarrow{\$} \mathbb{Z}_q^{\ell_2}$. Also, $\mathbf{t}_1 \leftarrow \mathsf{span}(\mathbf{B}_2, \mathbf{B}_3)$ can be equivalently written as $\mathbf{t}_1 = \mathbf{B}_2\mathbf{h} + r\mathbf{B}_3$ for some $\mathbf{h} \xleftarrow{\$} \mathbb{Z}_q^{\ell_2}, r \xleftarrow{\$} \mathbb{Z}_q$. To prove the claim, we will consider a sequence of four hybrid distributions $\mathsf{Hyb}_0, \mathsf{Hyb}_1, \mathsf{Hyb}_2, \mathsf{Hyb}_4$ containing $(\mathcal{D}, [\![\mathbf{t}]\!]_2)$, where $\mathbf{t}$ is equal to the

following across the hybrids: $\mathbf{B}_2\mathbf{h}$ in Hyb_0, $\mathbf{B}_2\mathbf{h} + r\mathbf{B}_3$ in Hyb_1, $\mathbf{B}_2\mathbf{h} + (r+1)\mathbf{B}_3$ in Hyb_2, and $\mathbf{B}_2\mathbf{h} + \mathbf{B}_3$ in Hyb_3. From $\mathsf{SD}^{\mathbb{G}_2}_{\mathbf{B}_2 \to \mathbf{B}_2,\mathbf{B}_3}$ assumption, we know that Hyb_0 and Hyb_1 are computationally indistinguishable. Next, observe that r and $r+1$ are identically distributed for a uniform random r. Then, by post-processing lemma it follows that Hyb_1 and Hyb_2 are identically distributed.

Lastly, we argue that Hyb_2 and Hyb_3 are computationally indistinguishable assuming the $\mathsf{SD}^{\mathbb{G}_2}_{\mathbf{B}_2 \to \mathbf{B}_2,\mathbf{B}_3}$ assumption. For this, we can create a simple reduction that obtains $(\mathcal{D}, [\![\mathbf{t}]\!]_2)$ from the $\mathsf{SD}^{\mathbb{G}_2}_{\mathbf{B}_2 \to \mathbf{B}_2,\mathbf{B}_3}$ assumption challenger and sends $(\mathcal{D}, [\![\mathbf{t}]\!]_2 \cdot [\![\mathbf{B}_3]\!]_2)$ to the adversary. If $\mathbf{t} = \mathbf{B}_2\mathbf{h}$, then the adversary sees Hyb_3 distribution and if $\mathbf{t} = \mathbf{B}_2\mathbf{h} + r\mathbf{B}_3$, then the adversary sees Hyb_2 distribution. □

Proof of Claim 2.4.11. Observe that the only difference between $\mathsf{Hyb}_{3,Q}$ and Hyb_4 is in ciphertext components $c_{2,x}$ for all $x \in [n]$. Observe that $c_{2,x}$ contains $[\![\lambda_x]\!]_1$, where λ_x is a secret share of $\mathbf{t}^T \in \mathbb{Z}_q^{1\times(2k+1)}$ in $\mathsf{Hyb}_{3,Q}$, but it is a secret share of $(\mathbf{t} + \tau\mathbf{B}_3^*)^T$ in Hyb_4. Therefore, to prove that the hybrids are statistically indistinguishable, we will argue that $\tau\mathbf{B}_3^{*T}$ is information theoretically hidden to the adversary $\mathcal{A}$ in Hyb_4.

Suppose the challenge access policy $(\mathbf{M}, \rho)$ is defined over a set of authorities $U \subseteq \mathcal{AU}$, that is, $\rho : [n] \to U$. Recall from Section 2.3.6 that the game condition requires that $U_{\mathcal{A}} \cap U_{\mathcal{B}} = \emptyset$ and for each $\mathsf{GID} \in \mathcal{GID}$, it is required that $S \cup S_{\mathsf{GID}} \notin (\mathbf{M}, \rho)$.

To show that $\tau\mathbf{B}_3^{*T}$ is information theoretically hidden from the adversary $\mathcal{A}$ in Hyb_4, we only need to rely on $S \notin (\mathbf{M}, \rho)$ which is implied by the second game condition. Here, $S \notin (\mathbf{M}, \rho)$ is a shorthand for $(1, 0, \ldots, 0) \notin \mathsf{rowSpan}(\{\mathbf{M}_x\}_{\rho(x)\in S})$.

Note that the vectors $\mathbf{M}_x\mathbf{T}$ for all rows x of the challenge access matrix $\mathbf{M}$ labeled by corrupt authorities (that is, $\rho(x) \in S$) are information theoretically revealed to $\mathcal{A}$. However, by the game condition the subspace spanned by those rows does not include the vector $(1, 0, \ldots, 0)$. This means that there must exist some vector $\mathbf{u}^T \in \mathbb{Z}_q^{1\times\ell}$ such that $\mathbf{u}^T$ is orthogonal to all these rows of $\mathbf{M}$ (that is, $\mathbf{M}_x\mathbf{u} = 0$) but is not orthogonal to $(1, 0, \ldots, 0)$, that is, the first entry of $\mathbf{u}^T$ must be non-zero. We consider a basis $\mathbb{U}$ of $\mathbb{Z}_q^\ell$ involving the vector $\mathbf{u}$ and write $\mathbf{T} = \begin{pmatrix} (\mathbf{t} + \tau\mathbf{B}_3^*)^T \\ \mathbf{T}_{\mathsf{bot}} \end{pmatrix} = \widetilde{\mathbf{U}} + \mathbf{u}\mathbf{b}^T$ for some $\mathbf{b} \in \mathbb{Z}_q^{2k+1}$ and some $\widetilde{\mathbf{U}} \in \mathbb{Z}_q^{\ell\times(2k+1)}$ such that each column of $\widetilde{\mathbf{U}}$ lies in the column span of $\mathbb{U} \setminus \mathbf{u}$. Hence, $\widetilde{\mathbf{U}}$ reveals no information about $\mathbf{b}$. Now since the first entry of $\mathbf{u}$ is non-zero, it follows that the first row of $\mathbf{T}$, that is, $(\mathbf{t} + \tau\mathbf{B}_3^*)^T$, depends on $\mathbf{b}$. But $\mathbf{M}_x\mathbf{T}$ for all the corrupted rows of $\mathbf{M}$ contains no information about $\mathbf{b}$ since $\mathbf{u}$ is orthogonal to all these rows. Thus, it follows that these rows do not leak information of $(\mathbf{t} + \tau\mathbf{B}_3^*)^T$.

Therefore, the only possible way for $\mathcal{A}$ to get information about $\tau\mathbf{B}_3^{*T}$ is through the ciphertext components $c_{2,x}$ corresponding to the uncorrupted rows of $\mathbf{M}$. However, for each such row x, $\mathcal{A}$ can only recover $\mathbf{c}_x^T$, $\mathbf{M}_x\mathbf{T} + \mathbf{c}_x^T\mathbf{V}_{\rho(x)}$ information theoretically. Without loss of generality, we can compute $\mathbf{V}_{\rho(x)} := \mathbf{V}_{\rho(x),1}\mathbf{B}_1^{*T} + \mathbf{V}_{\rho(x),2}\mathbf{B}_2^{*T} + \mathbf{V}_{\rho(x),3}\mathbf{B}_3^{*T}$, where $\mathbf{V}_{\rho(x),1}, \mathbf{V}_{\rho(x),2} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times k}$, and $\mathbf{V}_{\rho(x),3} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times 1}$. Let

the first entry of $\mathbf{M}_x$ be m_x, that is, $\mathbf{M}_x = (m_x, \ldots)$. Then, observe that we can write

$$\begin{aligned}
&\mathbf{M}_x \begin{pmatrix} (\mathbf{t} + \tau \mathbf{B}_3^*)^T \\ \mathbf{T}_{\mathsf{bot}} \end{pmatrix} + \mathbf{c}_x^T \mathbf{V}_{\rho(x)} \\
&= \mathbf{M}_x \begin{pmatrix} \mathbf{t}^T \\ \mathbf{T}_{\mathsf{bot}} \end{pmatrix} + m_x \tau \mathbf{B}_3^{*T} + \mathbf{c}_x^T \mathbf{V}_{\rho(x)} \\
&= \mathbf{M}_x \begin{pmatrix} \mathbf{t}^T \\ \mathbf{T}_{\mathsf{bot}} \end{pmatrix} + (m_x \tau + \mathbf{c}_x^T \mathbf{V}_{\rho(x),3}) \mathbf{B}_3^{*T} + \mathbf{c}_x^T \mathbf{V}_{\rho(x),1} \mathbf{B}_1^{*T} + \mathbf{c}_x^T \mathbf{V}_{\rho(x),2} \mathbf{B}_2^{*T} \\
&= \mathbf{M}_x \begin{pmatrix} \mathbf{t}^T \\ \mathbf{T}_{\mathsf{bot}} \end{pmatrix} + (\mathbf{c}_x^T \mathbf{V}'_{\rho(x),3}) \mathbf{B}_3^{*T} + \mathbf{c}_x^T \mathbf{V}_{\rho(x),1} \mathbf{B}_1^{*T} + \mathbf{c}_x^T \mathbf{V}_{\rho(x),2} \mathbf{B}_2^{*T}
\end{aligned}$$

where we can write $\mathbf{V}'_{\rho(x),3} = \mathbf{V}_{\rho(x),3} + \Delta$ such that $m_x \tau = \mathbf{c}_x^T \Delta$. Therefore, to complete the proof, it suffices to argue that $\mathbf{V}_{\rho(x),3}$ and $\mathbf{V}'_{\rho(x),3}$ are identically distributed. We show this next.

Observe that since ρ is injective, hence it follows that $\mathbf{V}_{\rho(x)}$ is a fresh random matrix and the only other place it appears is in secret keys $\mathsf{sk}_{\rho(x),\mathsf{GID}}$. Specifically, $\mathsf{sk}_{\rho(x),\mathsf{GID}}$ information theoretically reveals $\mathbf{V}_{\rho(x)} \mathbf{k} + \mathbf{U}_{\rho(x)} \mathbf{B}_1 \mathbf{h}_{\mathsf{GID}} + \mathbf{U}_{\rho(x)} \mathbf{B}_3$. Since $\mathbf{k}$ is uniform random, we can equivalently write it as $\mathbf{k} := \mathbf{B}_1 \mathbf{k}_1 + \mathbf{B}_2 \mathbf{k}_2 + k_3 \mathbf{B}_3$ for uniform random $\mathbf{k}_1 \xleftarrow{\$} \mathbb{Z}_q^k$, $\mathbf{k}_2 \xleftarrow{\$} \mathbb{Z}_q^k$, $k_3 \xleftarrow{\$} \mathbb{Z}_q$. Further, we can write $\mathbf{U}_{\rho(x)} := \mathbf{U}_{\rho(x),1} \mathbf{B}_1^{*T} + \mathbf{U}_{\rho(x),2} \mathbf{B}_2^{*T} + \mathbf{U}_{\rho(x),3} \mathbf{B}_3^{*T}$, where $\mathbf{U}_{\rho(x),1}, \mathbf{U}_{\rho(x),2} \xleftarrow{\$} \mathbb{Z}_q^{(k+1) \times k}$, and $\mathbf{U}_{\rho(x),3} \xleftarrow{\$} \mathbb{Z}_q^{(k+1) \times 1}$. Then observe that we can write

$$\begin{aligned}
&\mathbf{V}_{\rho(x)} \mathbf{k} + \mathbf{U}_{\rho(x)} \mathbf{B}_1 \mathbf{h}_{\mathsf{GID}} + \mathbf{U}_{\rho(x)} \mathbf{B}_3 \\
&\quad = \mathbf{V}_{\rho(x),1} \mathbf{k}_1 + \mathbf{V}_{\rho(x),2} \mathbf{k}_2 + k_3 \mathbf{V}_{\rho(x),3} + \mathbf{U}_{\rho(x),1} \mathbf{h}_{\mathsf{GID}} + \mathbf{U}_{\rho(x),3} \\
&\quad = \mathbf{V}_{\rho(x),1} \mathbf{k}_1 + \mathbf{V}_{\rho(x),2} \mathbf{k}_2 + k_3 \boxed{\mathbf{V}'_{\rho(x),3}} + \mathbf{U}_{\rho(x),1} \mathbf{h}_{\mathsf{GID}} + \boxed{\mathbf{U}'_{\rho(x),3}}
\end{aligned}$$

where we can write $\mathbf{V}'_{\rho(x),3} = \mathbf{V}_{\rho(x)} + \Delta$ and $\mathbf{U}'_{\rho(x)} = \mathbf{U}_{\rho(x)} - k_3 \Delta$, where Δ is as defined above, that is, choose Δ such that $m_x \tau = \mathbf{c}_x^T \Delta$. Therefore, $\mathbf{V}_{\rho(x),3}$ and $\mathbf{V}'_{\rho(x),3}$ are identically distributed as long as $\mathbf{U}_{\rho(x),3}$ and $\mathbf{U}'_{\rho(x),3}$ are identically distributed. We show this next. Observe that since ρ is injective, hence it follows that $\mathbf{U}_{\rho(x)}$ is a fresh random matrix and other than $\mathsf{sk}_{\rho(x),\mathsf{GID}}$, the only other place it appears is in ciphertext components $c_{3,x}$. For ciphertext components $c_{3,x}$ corresponding to the uncorrupted rows of $\mathbf{M}$, $\mathcal{A}$ can

recover $\mathbf{M}_x\mathbf{W} + \mathbf{c}_x^T\mathbf{U}_{\rho(x)}$ information theoretically. Observe that we can write

$$\begin{aligned}
&\mathbf{M}_x \begin{pmatrix} \gamma\mathbf{B}_3^{*T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + \mathbf{c}_x^T\mathbf{U}_{\rho(x)} \\
&= \mathbf{M}_x \begin{pmatrix} \gamma\mathbf{B}_3^{*T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + \mathbf{c}_x^T\mathbf{U}_{\rho(x),3}\mathbf{B}_3^{*T} + \mathbf{c}_x^T\mathbf{U}_{\rho(x),2}\mathbf{B}_2^{*T} + \mathbf{c}_x^T\mathbf{U}_{\rho(x),1}\mathbf{B}_1^{*T} \\
&= \mathbf{M}_x \begin{pmatrix} \gamma\mathbf{B}_3^{*T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + \mathbf{c}_x^T(k_3\Delta)\mathbf{B}_3^{*T} + \mathbf{c}_x^T\mathbf{U}'_{\rho(x),3}\mathbf{B}_3^{*T} + \dots \\
&= \mathbf{M}_x \begin{pmatrix} \gamma\mathbf{B}_3^{*T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + m_x k_3\tau\mathbf{B}_3^{*T} + \mathbf{c}_x^T\mathbf{U}'_{\rho(x),3}\mathbf{B}_3^{*T} + \dots \\
&= \mathbf{M}_x \begin{pmatrix} (\gamma + k_3\tau)\mathbf{B}_3^{*T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + \mathbf{c}_x^T\mathbf{U}'_{\rho(x)} \\
&= \mathbf{M}_x \begin{pmatrix} \gamma'\mathbf{B}_3^{*T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + \mathbf{c}_x^T\mathbf{U}'_{\rho(x)}
\end{aligned}$$

where we can write $\gamma' = \gamma + k_3\tau$. Therefore to complete this part of the proof, it suffices to argue that γ and γ' are identically distributed. This holds true because γ, k_3, τ are uniform random, that is, $\gamma, k_3, \tau \xleftarrow{\$} \mathbb{Z}_q$.

To complete the proof, we argue that substituting $(\mathbf{U}_{\rho(x),3}, \mathbf{V}_{\rho(x),3}, \gamma)$ with $(\mathbf{U}'_{\rho(x),3}, \mathbf{V}'_{\rho(x),3}, \gamma')$ (as described above) for all rows x of matrix $\mathbf{M}$ for which the challenger sampled the authority keys (that is, uncorrupted rows plus the rows for which the adversary queried the master secret key) allows us to move from Hyb_4 to $\mathsf{Hyb}_{3,Q}$. We have already argued that this substitution does not change the distribution of the secret keys and ciphertext obtained by the adversary $\mathcal{A}$ for the uncorrupted rows of $\mathbf{M}$. For the case of rows x of $\mathbf{M}$ for which the adversary queried the master secret key, the adversary additionally learns $\mathbf{U}'_{\rho(x),3}, \mathbf{V}'_{\rho(x),3}$ and $\mathbf{M}_x \begin{pmatrix} \gamma'\mathbf{B}_3^{*T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix}$ in $\mathsf{Hyb}_{3,Q}$ and $\mathbf{U}_{\rho(x),3}, \mathbf{V}_{\rho(x),3}$ and $\mathbf{M}_x \begin{pmatrix} \gamma\mathbf{B}_3^{*T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix}$ in Hyb_4 and we argue that this does not help the adversary $\mathcal{A}$ to distinguish between $\mathsf{Hyb}_{3,Q}$ and Hyb_4. This is because $\mathbf{U}_{\rho(x),3}$ and $\mathbf{U}'_{\rho(x),3}$ are identically distributed, $\mathbf{V}_{\rho(x),3}$ and $\mathbf{V}'_{\rho(x),3}$ are identically distributed, and $\mathbf{M}_x \begin{pmatrix} \gamma\mathbf{B}_3^{*T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} = \mathbf{M}_x \begin{pmatrix} \gamma'\mathbf{B}_3^{*T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} = \mathbf{M}_x \begin{pmatrix} \mathbf{0}^T \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix}$ due to the game condition.

Therefore, it follows that $\mathsf{Hyb}_{3,Q}$ and Hyb_4 are statistically indistinguishable. This completes the proof of Claim 2.4.11. □

Proof of Claim 2.4.12. Observe that in Hyb_4 and Hyb_5, $\mathbf{t}$ is sampled randomly as $\mathbf{t} \xleftarrow{\$} \mathbb{Z}_q^{2k+1}$. Alternately,

we can also sample it as $\mathbf{t} := \mathbf{B}_1^* \mathbf{t}_1 + \mathbf{B}_2^* \mathbf{t}_2 + t_3 \mathbf{B}_3^*$, where $\mathbf{t}_1 \xleftarrow{\$} \mathbb{Z}_q^k, \mathbf{t}_2 \xleftarrow{\$} \mathbb{Z}_q^k, t_3 \xleftarrow{\$} \mathbb{Z}_q$. Note that the distribution of $\mathbf{t}$ is identical in both of the above ways. So, we will use the latter form to show that Hyb_4 and Hyb_5 are statistically indistinguishable. Observe that in both Hyb_4 and Hyb_5 the vector $\mathbf{t}$ shows up in ciphertext components c_0 and $c_{2,x}$. In $c_{2,x}$, it shows up in matrix $\mathbf{T}$ used for computing λ_x. And recall that $\mathbf{T} = \begin{pmatrix} (\mathbf{t} + \tau \mathbf{B}_3^*)^T \\ \mathbf{T}_{\mathsf{bot}} \end{pmatrix}$. Hence, the first row of $\mathbf{T}$ can be written as $(\mathbf{B}_1^* \mathbf{t}_1 + \mathbf{B}_2^* \mathbf{t}_2 + (t_3 + \tau)\mathbf{B}_3^*)^T$. Since $\tau \xleftarrow{\$} \mathbb{Z}_q$, it follows that $c_{2,x}$ reveals no information about $t_3 \in \mathbb{Z}_q$. So, we will use the entropy of t_3 to argue that c_0 can be switch from encoding msg_b in Hyb_4 to encoding a random ζ in Hyb_5.

Without loss of generality, suppose $\mathbf{k}$ is of the form $\mathbf{k} = \mathbf{B}_1 \mathbf{k}_1 + \mathbf{B}_2 \mathbf{k}_2 + k_3 \mathbf{B}_3$ for some $\mathbf{k}_1, \mathbf{k}_2, k_3$. Then observe that $[\![\mathbf{t}^T \mathbf{k}]\!]_T = [\![\mathbf{t}_1^T \mathbf{k}_1]\!]_T \cdot [\![\mathbf{t}_2^T \mathbf{k}_2]\!]_T \cdot [\![t_3 k_3]\!]_T$. It follows then that $[\![t_3 k_3]\!]_T$ is uniform random in $\mathbb{G}_T$ as long as $k_3 \neq 0$. Consequently, $\mathsf{msg}_b \cdot [\![t_3 k_3]\!]_T$ and $\zeta \cdot [\![t_3 k_3]\!]_T$ are identically distributed. Thus, c_0 in Hyb_4 and Hyb_5 are identically distributed.

Therefore, it follows that Hyb_4 and Hyb_5 are statistically indistinguishable, where the statistical security loss comes from the scenario when $k_3 = 0$. This completes the proof of Claim 2.4.12. □

2.4.3 Missing Proofs from Section 2.4.2

Before we present the missing proofs, we first define the Q-fold MDDH assumption and comment on the random self-reducibility of the MDDH assumption since these will help simplify the proofs. The Q-fold MDDH assumption is considers Q many independent instances of the MDDH assumption. While using a standard hybrid argument, it can be shown that this is equivalent to the MDDH assumption with a loss of Q in the reduction, but random self-reducibility allows us to give a tighter bound when $Q > \ell - k$ that we state below.

Assumption 2.4.17 (Q-fold MDDH Assumption: $\mathsf{MDDH}^{\mathbb{G}_i}_{k,\ell,Q}$ [EHK+13])**.** *We say that the* $\mathsf{MDDH}^{\mathbb{G}_i}_{k_\ell}$ *assumption holds with respect to* PGGen *if for all p.p.t. adversary* $\mathcal{A}$ *and for all* $i \in \{1, 2, T\}$*, the following advantage is negligible in* λ*.*

$$\mathsf{Adv}^{\mathsf{MDDH}^{\mathbb{G}_i}_{k,\ell,Q}}_{\mathcal{A}}(\lambda) := |\Pr[\mathcal{A}(\mathcal{D}, [\![\mathbf{T}_0]\!]_i) = 1] - \Pr[\mathcal{A}(\mathcal{D}, [\![\mathbf{T}_1]\!]_i) = 1]|$$

where

$$\mathcal{PG} := (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, q, g_1, g_2, g_T) \leftarrow \mathsf{PGGen}(1^\lambda),$$
$$\mathcal{D} = (\mathcal{PG}, [\![\mathbf{X}]\!]_i),$$
$$\mathbf{T}_0 = \mathbf{XU}, \mathbf{T}_1 \xleftarrow{\$} \mathbb{Z}_q^{\ell \times Q}$$

such that $\mathbf{X} \xleftarrow{\$} \mathbb{Z}_q^{\ell \times k}$, $\mathbf{U} \xleftarrow{\$} \mathbb{Z}_p^{k \times Q}$.

Lemma 2.4.18 (Random Self-reducibility of MDDH [EHK+13]). *For all $i \in \{1, 2, T\}$, $\mathsf{MDDH}^{\mathbb{G}_i}_{k,\ell}$ assumption is random self-reducible. Concretely, for any Q,*

$$\mathsf{Adv}^{\mathsf{MDDH}^{\mathbb{G}_i}_{k,\ell,Q}}_{\mathcal{A}}(\lambda) \leq \begin{cases} Q \cdot \mathsf{Adv}^{\mathsf{MDDH}^{\mathbb{G}_i}_{k,\ell}}_{\mathcal{A}}(\lambda) & , \text{if } 1 \leq Q \leq \ell - k \\ (\ell - k) \cdot \mathsf{Adv}^{\mathsf{MDDH}^{\mathbb{G}_i}_{k,\ell}}_{\mathcal{A}}(\lambda) + \frac{1}{q-1} & , \text{if } Q > \ell - k \end{cases},$$

and the probability is taken over $\mathcal{PG} := (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, q, g_1, g_2, g_T) \leftarrow \mathsf{PGGen}(1^\lambda)$, $\mathbf{X} \xleftarrow{\$} \mathbb{Z}_q^{\ell \times k}$, $\mathbf{U} \xleftarrow{\$} \mathbb{Z}_p^{k \times Q}$, $\mathbf{T}_1 \xleftarrow{\$} \mathbb{Z}_q^{\ell \times Q}$ and the random coins of $\mathcal{A}$.

Proof of Claim 2.4.2. $\mathsf{Hyb}_{\mathsf{Real}}$ and $\mathsf{Hyb}'_{\mathsf{Real}}$ are identically distributed because the distribution of gp in $\mathsf{GlobalSetup}$ and $\mathsf{GlobalSetup}^*$ is the same. □

Proof of Claim 2.4.3. We show that if there exists a PPT adversary $\mathcal{A}$ that can distinguish between $\mathsf{Hyb}'_{\mathsf{Real}}$ and Hyb_0, then we can use $\mathcal{A}$ to construct a PPT adversary $\mathcal{B}$ that can break the $\mathsf{MDDH}^{\mathbb{G}_2}_{k,2k+1,Q}$ assumption in $\mathbb{G}_2$, where Q is the number of queries made by $\mathcal{A}$ to the H_1 oracle.

Let $\mathcal{B}$ be an adversary that is given a $\mathsf{MDDH}^{\mathbb{G}_2}_{k,2k+1,Q}$ challenge by the challenger $\mathcal{C}$: $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, q, g_1, g_2, g_T, [\![\mathbf{B}_1]\!]_2, [\![\mathbf{H}]\!]_2)$ where $\mathbf{B}_1 \xleftarrow{\$} \mathbb{Z}_q^{(2k+1)\times k}$ and $\mathbf{H} \in \mathbb{Z}_q^{(2k+1)\times Q}$. Here $\mathcal{C}$ samples $b \xleftarrow{\$} \{0,1\}$ and sets $\mathbf{H} = \mathbf{B}_1\mathbf{Z}$ if $b = 0$ and $\mathbf{H} \xleftarrow{\$} \mathbb{Z}_q^{(2k+1)\times Q}$ if $b = 1$. The goal of $\mathcal{B}$ is to output a bit b' and $\mathcal{B}$ wins if $b' = b$.

$\mathcal{B}$ will internally run $\mathcal{A}$ as follows. For simulating the view to $\mathcal{A}$, $\mathcal{B}$ will run exactly the same as $\mathsf{Hyb}'_{\mathsf{Real}}$, except the following modifications:

- $\mathcal{B}$ will use $[\![\mathbf{B}_1]\!]_2$ obtained from the MDDH challenger $\mathcal{C}$ instead of computing it as in $\mathsf{GlobalSetup}^*$ in $\mathsf{Hyb}'_{\mathsf{Real}}$. While $\mathcal{B}$ can sample $\mathbf{B}_2$ and $\mathbf{B}_3$ on its own as in $\mathsf{Hyb}'_{\mathsf{Real}}$, it will not be able to compute $\mathbf{B}_1^*, \mathbf{B}_2^*, \mathbf{B}_3^*$ as in $\mathsf{GlobalSetup}^*$ since it does not know the value of $\mathbf{B}_1$ in the clear. But this is okay since these values are not needed to simulate the view of $\mathcal{A}$.
- $\mathcal{B}$ will embed the challenge $[\![\mathbf{H}]\!]_2$ in the H_1 oracle queries from $\mathcal{A}$ as follows. $\mathcal{B}$ parses $([\![\mathbf{h}_1]\!]_2, \ldots, [\![\mathbf{h}_Q]\!]_2) = [\![\mathbf{H}]\!]_2$, where $[\![\mathbf{h}_j]\!] \in \mathbb{G}_2^{2k+1}$ for all $j \in [Q]$. For the jth query to the H_1 oracle, $\mathcal{B}$ programs $\mathsf{H}_1(\mathsf{GID}_j) := [\![\mathbf{h}_j]\!]_2$.

Observe that when $\mathcal{C}$ chooses $b = 0$, that is, $\mathbf{H} = \mathbf{B}_1\mathbf{Z}$, then $\mathcal{B}$ is able to simulate the view of $\mathcal{A}$ in Hyb_0. Similarly, when $\mathcal{C}$ chooses $b = 1$, that is, $\mathbf{H} \xleftarrow{\$} \mathbb{Z}_q^{(2k+1)\times Q}$, then $\mathcal{B}$ is able to simulate the view of $\mathcal{A}$ in $\mathsf{Hyb}'_{\mathsf{Real}}$. Thus, if $\mathcal{A}$ can distinguish between $\mathsf{Hyb}'_{\mathsf{Real}}$ and Hyb_0, then $\mathcal{B}$ can distinguish between the two cases of the MDDH challenge.

□

Proof of Claim 2.4.4. Recall that the only difference between Hyb_0 and Hyb_1 is in how the challenge ciphertext is generated. Specifically, for all rows $x \in \overline{U_{\mathcal{A}}}$, the two hybrids differ in how the vector $\mathbf{c}_x \in \mathbb{Z}_q^{k+1}$

is computed. In Hyb_0, $\mathbf{c}_x^T := \mathbf{s}_x^T \mathbf{A}_1$, or equivalently, $\mathbf{c}_x = \mathbf{A}_1^T \mathbf{s}_x$ where $\mathbf{s}_x \xleftarrow{\$} \mathbb{Z}_q^k$ is a random vector. In Hyb_1, $\mathbf{c}_x \xleftarrow{\$} \mathbb{Z}_q^{k+1}$ is a random vector.

We show that if there exists a PPT adversary $\mathcal{A}$ that can distinguish between Hyb_0 and Hyb_1, then we can use $\mathcal{A}$ to construct a PPT adversary $\mathcal{B}$ that can break the $\mathsf{MDDH}^{\mathbb{G}_1}_{k,k+1,|\overline{U_\mathcal{A}}|}$ assumption in $\mathbb{G}_1$, where $\overline{U_\mathcal{A}}$ denotes the subset of rows of challenge access policy matrix $\mathbf{M}$ that are honestly generated by the challenger.

Let $\mathcal{B}$ be an adversary that is given a $\mathsf{MDDH}^{\mathbb{G}_1}_{k,k+1,|\overline{U_\mathcal{A}}|}$ challenge by the challenger $\mathcal{C}$: $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, q, g_1, g_2, g_T, [\![\mathbf{A}_1^T]\!]_1, [\![\mathbf{C}]\!]_1)$ where $\mathbf{A}_1^T \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times k}$ and $\mathbf{C} \in \mathbb{Z}_q^{(k+1)\times|\overline{U_\mathcal{A}}|}$. Here $\mathcal{C}$ samples $b \xleftarrow{\$} \{0,1\}$ and sets $\mathbf{C} = \mathbf{A}_1^T\mathbf{Z}$ if $b = 0$ and $\mathbf{C} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times|\overline{U_\mathcal{A}}|}$ if $b = 1$. The goal of $\mathcal{B}$ is to output a bit b' and $\mathcal{B}$ wins if $b' = b$. $\mathcal{B}$ will internally run $\mathcal{A}$ as follows. For simulating the view to $\mathcal{A}$, $\mathcal{B}$ will run exactly the same as Hyb_0, except the following modifications:

- $\mathcal{B}$ will use $[\![\mathbf{A}_1]\!]_1$ obtained from the MDDH challenger $\mathcal{C}$ instead of computing it as in $\mathsf{GlobalSetup}^*$ in Hyb_0. While $\mathcal{B}$ can sample $\mathbf{A}_2$ on its own as in Hyb_0, it will not be able to compute $\mathbf{A}_1^*, \mathbf{A}_2^*$ as in $\mathsf{GlobalSetup}^*$ since it does not know the value of $\mathbf{A}_1$ in the clear. But this is okay since these values are not needed to simulate the view of $\mathcal{A}$.

- $\mathcal{B}$ will embed the challenge $[\![\mathbf{C}]\!]_1$ in the challege ciphertext query $(\mathsf{msg}_0, \mathsf{msg}_1, (\mathbf{M}, \rho), \{\mathsf{pk}_x\}_{x\in U_\mathcal{A}})$ as follows: $\mathcal{B}$ parses $([\![\mathbf{c}_1]\!]_1, \ldots, [\![\mathbf{c}_{|\overline{U_\mathcal{A}}|}]\!]_1) = [\![\mathbf{C}]\!]_1$, where $[\![\mathbf{c}_x]\!]_1 \in \mathbb{G}_1^{k+1}$ for all $x \in [|\overline{U_\mathcal{A}}|]$. Without loss of generality suppose that $\overline{U_\mathcal{A}} = \{1, 2, \ldots, |\overline{U_\mathcal{A}}|\}$. Then, for all $x \in \overline{U_\mathcal{A}}$, $\mathcal{B}$ uses the challenge $[\![\mathbf{c}_x]\!]_1$ to compute the challenge ciphertext components $c_{1,x}, c_{2,x}, c_{3,x}$ as follows:

$$
\begin{aligned}
c_{1,x} &:= [\![\mathbf{c}_x^T]\!]_1, \\
c_{2,x} &:= [\![\lambda_x]\!]_1 \cdot ([\![\mathbf{c}_x^T]\!]_1 \odot \mathbf{V}_{\rho(x)}), \\
c_{3,x} &:= [\![\omega_x]\!]_1 \cdot ([\![\mathbf{c}_x^T]\!]_1 \odot \mathbf{U}_{\rho(x)}).
\end{aligned}
$$

Observe that when $\mathcal{C}$ chooses $b = 0$, that is, $\mathbf{C} = \mathbf{A}_1^T\mathbf{Z}$, then $\mathcal{B}$ is able to simulate the view of $\mathcal{A}$ in Hyb_0. Similarly, when $\mathcal{C}$ chooses $b = 1$, that is, $\mathbf{C} \xleftarrow{\$} \mathbb{Z}_q^{(2k+1)\times Q}$, then $\mathcal{B}$ is able to simulate the view of $\mathcal{A}$ in Hyb_1. Thus, if $\mathcal{A}$ can distinguish between Hyb_0 and Hyb_1, then $\mathcal{B}$ can distinguish between the two cases of the MDDH challenge.

□

Proof of Claim 2.4.5. Observe that the only difference between Hyb_1 and Hyb_2 is that in ciphertext component $c_{3,x}$ for all $x \in [n]$: $c_{3,x}$ contains $[\![\omega_x]\!]_1$, where ω_x is a secret share of $\mathbf{0}^T \in \mathbb{Z}_q^{1\times(2k+1)}$ in Hyb_1, but it is a secret share of $\gamma\mathbf{B}_3^{*T}$ in Hyb_2. Therefore, to prove that the hybrids are statistically indistinguishable, we will argue that $\gamma\mathbf{B}_3^{*T}$ is information theoretically hidden from the adversary $\mathcal{A}$ in Hyb_2.

Suppose the challenge access policy $(\mathbf{M}, \rho)$ is defined over a set of authorities $U \subseteq \mathcal{AU}$, that is, $\rho : [n] \to U$. Recall from Section 2.3.6 that the game condition requires that $U_{\mathcal{A}} \cap U_{\mathcal{B}} = \emptyset$ and for each $\mathsf{GID} \in \mathcal{GID}$, it is required that $S \cup S_{\mathsf{GID}} \notin (\mathbf{M}, \rho)$, where

- $U_{\mathcal{A}} \subseteq U$ denotes the set of attribute authorities for which the adversary supplied the authority public keys $\{\mathsf{pk}_i\}$,
- $U_{\mathcal{B}}$ denote the set of attribute authorities for which the challenger created the master public key pairs in response to the authority setup query of the adversary so far,
- S denotes the subset of U containing the authorities in $U_{\mathcal{A}}$ plus the authorities for which the adversary made a master key query so far (in other words, S denotes the set of corrupt authorities),
- for each global identifier $\mathsf{GID} \in \mathcal{GID}$, S_{GID} denotes the subset of U containing authorities i such that the adversary queried a secret key for the pair (i, GID).

To show that $\gamma \mathbf{B}_3^{*T}$ is information theoretically hidden from the adversary $\mathcal{A}$ in Hyb_2, we only need to rely on $S \notin (\mathbf{M}, \rho)$ which is implied by the second game condition. Here, $S \notin (\mathbf{M}, \rho)$ is a shorthand for $(1, 0, \ldots, 0) \notin \mathsf{rowSpan}(\{\mathbf{M}_x\}_{\rho(x) \in S})$.

Note that the vectors $\mathbf{M}_x \mathbf{W}$ for all rows x of the challenge access matrix $\mathbf{M}$ labeled by corrupt authorities (that is, $\rho(x) \in S$) are information theoretically revealed to $\mathcal{A}$. However, by the game condition the subspace spanned by those rows does not include the vector $(1, 0, \ldots, 0)$. This means that there must exist some vector $\mathbf{u} \in \mathbb{Z}_q^{\ell}$ such that $\mathbf{u}$ is orthogonal to all these rows of $\mathbf{M}$ (that is, $\mathbf{M}_x \mathbf{u} = 0$) but is not orthogonal to $(1, 0, \ldots, 0)$, that is, the first entry of $\mathbf{u}$ must be non-zero. We consider a basis $\mathbb{U}$ of $\mathbb{Z}_q^{\ell}$ involving the vector $\mathbf{u}$ and write $\mathbf{W} = \begin{pmatrix} \gamma \mathbf{B}_3^{*T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} = \widetilde{\mathbf{U}} + \mathbf{u}\mathbf{b}^T$ for some $\mathbf{b} \in \mathbb{Z}_q^{2k+1}$ and some $\widetilde{\mathbf{U}} \in \mathbb{Z}_q^{\ell \times (2k+1)}$ such that each column of $\widetilde{\mathbf{U}}$ lies in the column span of $\mathbb{U} \setminus \mathbf{u}$. Hence, $\widetilde{\mathbf{U}}$ reveals no information about $\mathbf{b}$. Now since the first entry of $\mathbf{u}$ is non-zero, it follows that the first row of $\mathbf{W}$, that is, $\gamma \mathbf{B}_3^{*T}$, depends on $\mathbf{b}$. But $\mathbf{M}_x \mathbf{W}$ for all the corrupted rows of $\mathbf{M}$ contains no information about $\mathbf{b}$ since $\mathbf{u}$ is orthogonal to all these rows. Thus, it follows that these rows do not leak information of $\gamma \mathbf{B}_3^{*T}$.

Therefore, the only possible way for $\mathcal{A}$ to get information about $\gamma \mathbf{B}_3^{*T}$ is through the ciphertext components $c_{3,x}$ corresponding to the uncorrupted rows of $\mathbf{M}$. However, for each such row x, $\mathcal{A}$ can only recover $\mathbf{c}_x^T$, $\mathbf{M}_x \mathbf{W} + \mathbf{c}_x^T \mathbf{U}_{\rho(x)}$ information theoretically. Without loss of generality, we can compute $\mathbf{U}_{\rho(x)} := \mathbf{U}_{\rho(x),1} \mathbf{B}_1^{*T} + \mathbf{U}_{\rho(x),2} \mathbf{B}_2^{*T} + \mathbf{U}_{\rho(x),3} \mathbf{B}_3^{*T}$, where $\mathbf{U}_{\rho(x),1} \xleftarrow{\$} \mathbb{Z}_q^{(k+1) \times k}$, $\mathbf{U}_{\rho(x),2} \xleftarrow{\$} \mathbb{Z}_q^{(k+1) \times k}$, and $\mathbf{U}_{\rho(x),3} \xleftarrow{\$} \mathbb{Z}_q^{(k+1) \times 1}$.

Let the first entry of $\mathbf{M}_x$ be m_x, that is, $\mathbf{M}_x = (m_x, \ldots)$. Then, observe that we can write

$$\begin{aligned}
\mathbf{M}_x \begin{pmatrix} \gamma \mathbf{B}_3^{*T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + \mathbf{c}_x^T \mathbf{U}_{\rho(x)} &= \mathbf{M}_x \begin{pmatrix} \mathbf{0} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + \mathbf{M}_x \begin{pmatrix} \gamma \mathbf{B}_3^{*T} \\ \mathbf{0} \end{pmatrix} + \mathbf{c}_x^T \mathbf{U}_{\rho(x)} \\
&= \mathbf{M}_x \begin{pmatrix} \mathbf{0} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + (m_x, \ldots) \begin{pmatrix} \gamma \mathbf{B}_3^{*T} \\ \mathbf{0} \end{pmatrix} + \mathbf{c}_x^T \mathbf{U}_{\rho(x)} \\
&= \mathbf{M}_x \begin{pmatrix} \mathbf{0} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + m_x \gamma \mathbf{B}_3^{*T} + \mathbf{c}_x^T \mathbf{U}_{\rho(x)} \\
&= \mathbf{M}_x \begin{pmatrix} \mathbf{0} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + (m_x \gamma + \mathbf{c}_x^T \mathbf{U}_{\rho(x),3}) \mathbf{B}_3^{*T} + \mathbf{c}_x^T \mathbf{U}_{\rho(x),1} \mathbf{B}_1^{*T} + \mathbf{c}_x^T \mathbf{U}_{\rho(x),2} \mathbf{B}_2^{*T} \\
&\equiv \mathbf{M}_x \begin{pmatrix} \mathbf{0} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + (\mathbf{c}_x^T \mathbf{U}'_{\rho(x),3}) \mathbf{B}_3^{*T} + \mathbf{c}_x^T \mathbf{U}_{\rho(x),1} \mathbf{B}_1^{*T} + \mathbf{c}_x^T \mathbf{U}_{\rho(x),2} \mathbf{B}_2^{*T}
\end{aligned}$$

where we can write $\mathbf{U}'_{\rho(x),3} = \mathbf{U}_{\rho(x),3} + \Delta$ such that $m_x \gamma = \mathbf{c}_x^T \Delta$. Therefore, to complete the proof, it suffices to argue that $\mathbf{U}_{\rho(x),3}$ and $\mathbf{U}'_{\rho(x),3}$ are identically distributed. We show this next. Observe that since ρ is injective, hence it follows that $\mathbf{U}_{\rho(x)}$ is a fresh random matrix and the only other place it appears is in secret keys $\mathsf{sk}_{\rho(x),\mathsf{GID}}$. We argue that $\mathsf{sk}_{\rho(x),\mathsf{GID}}$ information theoretically leaks no information about $\mathbf{U}_{\rho(x),3}$ and hence $\mathbf{U}_{\rho(x),3}$ and $\mathbf{U}'_{\rho(x),3}$ are identically distributed. To see this, observe that $\mathsf{sk}_{\rho(x),\mathsf{GID}}$ information theoretically reveals $\mathbf{V}_{\rho(x)}\mathbf{k} + \mathbf{U}_{\rho(x)}\mathbf{B}_1\mathbf{h}_{\mathsf{GID}}$, where $\mathbf{U}_{\rho(x)}\mathbf{B}_1\mathbf{h}_{\mathsf{GID}} = \mathbf{U}_{\rho(x),1}\mathbf{h}_{\mathsf{GID}}$ since $\mathbf{B}_1^{*T}\mathbf{B}_1 = \mathbf{I}$, $\mathbf{B}_2^{*T}\mathbf{B}_1 = \mathbf{0}$ and $\mathbf{B}_3^{*T}\mathbf{B}_1 = \mathbf{0}$, thus no information about $\mathbf{U}_{\rho(x),3}$ is revealed.

To complete the proof, we argue that substituting $\mathbf{U}_{\rho(x),3}$ with $\mathbf{U}'_{\rho(x),3}$ (as described above) for all rows x of matrix $\mathbf{M}$ for which the challenger sampled the authority keys (that is, uncorrupted rows plus the rows for which the adversary queried the master secret key) allows us to move from Hyb_2 to Hyb_1. We have already argued that this substitution does not change the distribution of the secret keys and ciphertext obtained by the adversary $\mathcal{A}$ for the uncorrupted rows of $\mathbf{M}$. For the case of rows x of $\mathbf{M}$ for which the adversary queried the master secret key, the adversary additionally learns $\mathbf{U}'_{\rho(x),3}$ and $\mathbf{M}_x \begin{pmatrix} \mathbf{0}^T \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix}$ in Hyb_1 and $\mathbf{U}_{\rho(x),3}$ and $\mathbf{M}_x \begin{pmatrix} \gamma \mathbf{B}_3^{*T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix}$ in Hyb_2 and we argue that this does not help the adversary $\mathcal{A}$ to distinguish between Hyb_1 and Hyb_2. This is because $\mathbf{U}_{\rho(x),3}$ and $\mathbf{U}'_{\rho(x),3}$ are identically distributed and $\mathbf{M}_x \begin{pmatrix} \gamma \mathbf{B}_3^{*T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} = \mathbf{M}_x \begin{pmatrix} \mathbf{0}^T \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix}$ due to the game condition.

Therefore, it follows that Hyb_1 and Hyb_2 are statistically indistinguishable. This completes the proof of Claim 2.4.5. □

Proof of Claim 2.4.6.

Suppose towards a contradiction that there exists a p.p.t. adversary $\mathcal{A}$ that distinguishes its view in $\mathsf{Hyb}_{3,j-1}$ and $\mathsf{Hyb}_{3,j,1}$ with a noticeable advantage ϵ. Then we show how to construct an adversary $\mathcal{B}$ using $\mathcal{A}$ that breaks the $\mathsf{SD}^{\mathbb{G}_2}_{\mathbf{B}_1 \to \mathbf{B}_1, \mathbf{B}_2}$ assumption with noticeable advantage ϵ. Suppose that the $\mathsf{SD}^{\mathbb{G}_2}_{\mathbf{B}_1 \to \mathbf{B}_1, \mathbf{B}_2}$ challenger is $\mathcal{C}$. Then, recall that $\mathcal{B}$ obtains $(\mathcal{D}, [\![\mathbf{h}]\!]_2)$ from $\mathcal{C}$ and $\mathcal{B}$'s goal is to distinguish whether $\mathbf{h} \in \mathsf{span}(\mathbf{B}_1)$ or $\mathbf{h} \in \mathsf{span}(\mathbf{B}_1, \mathbf{B}_2)$, where

$$
\begin{aligned}
&\mathcal{PG} := (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, q, g_1, g_2, g_T) \leftarrow \mathsf{PGGen}(1^\lambda),\\
&\mathbf{B}_1 \xleftarrow{\$} \mathbb{Z}_q^{(2k+1)\times k}, \mathbf{B}_2 \xleftarrow{\$} \mathbb{Z}_q^{(2k+1)\times k}, \mathbf{B}_3 \xleftarrow{\$} \mathbb{Z}_q^{(2k+1)\times 1},\\
&(\mathbf{B}_1^*, \mathbf{B}_2^*, \mathbf{B}_3^*)^T = (\mathbf{B}_1, \mathbf{B}_2, \mathbf{B}_3)^{-1},\\
&\mathcal{D} = (\mathcal{PG}, [\![\mathbf{B}_1]\!]_2, [\![\mathbf{B}_2]\!]_2, [\![\mathbf{B}_3]\!]_2, \mathsf{basis}(\mathbf{B}_1^*), \mathsf{basis}(\mathbf{B}_3^*), \mathsf{basis}(\mathbf{B}_1^*, \mathbf{B}_2^*)).
\end{aligned}
$$

$\mathcal{B}$ accomplishes this by invoking the adversary $\mathcal{A}$ for distinguishing $\mathsf{Hyb}_{3,j-1}$ and $\mathsf{Hyb}_{3,j,1}$. Towards this, $\mathcal{B}$ plays the role of $\mathcal{A}$'s challenger in its dinstinguishing game as follows: instead of running $\mathsf{GlobalSetup}^*$ on its own completely, it uses $[\![\mathbf{B}_1]\!]_2, [\![\mathbf{B}_2]\!]_2, [\![\mathbf{B}_3]\!]_2, \mathsf{basis}(\mathbf{B}_1^*), \mathsf{basis}(\mathbf{B}_3^*), \mathsf{basis}(\mathbf{B}_1^*, \mathbf{B}_2^*)$ obtained from $\mathcal{C}$ and samples $\mathbf{A}_1, \mathbf{A}_2, \mathbf{A}_1^*, \mathbf{A}_2^*$ on its own as in $\mathsf{GlobalSetup}^*$. Consequently, gp is identically distributed as in $\mathsf{GlobalSetup}^*$. While $\mathcal{B}$ does not know $\mathbf{B}_1, \mathbf{B}_2, \mathbf{B}_3, \mathbf{B}_1^*, \mathbf{B}_2^*, \mathbf{B}_3^*$ completely now, we show that the information obtained from $\mathcal{C}$ in this regard is sufficient for $\mathcal{B}$ to perfectly simulate the view for $\mathcal{A}$. This information shows up in two places in the game between $\mathcal{B}$ and $\mathcal{A}$:

- When computing challenge ciphertext, $\mathcal{B}$ needs to compute matrix $\mathbf{W}$ whose first row is $\gamma \mathbf{B}_3^{*T}$. Observe that since $\mathbf{B}_3^*$ is a vector, hence $\mathsf{basis}(\mathbf{B}_3^*) = \delta \mathbf{B}_3^*$ for some non-zero $\delta \in \mathbb{Z}_q$. Therefore, $\mathcal{B}$ can sample $\gamma' \xleftarrow{\$} \mathbb{Z}_q$ and set the first row of $\mathbf{W}$ to be $\gamma' \mathsf{basis}(\mathbf{B}_3^*)^T$. This perfectly simulates $\gamma \mathbf{B}_3^{*T}$ as it implicitly sets $\gamma = \gamma' \delta$ and γ is uniform random since γ' is uniform random.

- For computing H_1 on some input $\mathsf{GID}_{\mathsf{index}}$, $\mathcal{B}$ can use $[\![\mathbf{B}_1]\!]_2$, $[\![\mathbf{B}_3]\!]_2$ and $[\![\mathbf{h}]\!]_2$ obtained from $\mathcal{C}$ as follows:

$$
\mathbf{h}_{\mathsf{GID}_{\mathsf{index}}} \xleftarrow{\$} \mathbb{Z}_q^k, \mathsf{H}_1(\mathsf{GID}_{\mathsf{index}}) := \begin{cases} ([\![\mathbf{B}_1]\!]_2 \odot \mathbf{h}_{\mathsf{GID}_{\mathsf{index}}}) \cdot [\![\mathbf{B}_3]\!]_2 & , \text{if index} \leq j-1 \\ [\![\mathbf{h}]\!]_2 & , \text{if index} = j \\ [\![\mathbf{B}_1]\!]_2 \odot \mathbf{h}_{\mathsf{GID}_{\mathsf{index}}} & , \text{if index} > j \end{cases} .
$$

Therefore, it follows that when $\mathbf{h} \in \mathsf{span}(\mathbf{B}_1)$, $\mathcal{B}$ perfectly simulates $\mathsf{Hyb}_{3,j-1}$ to $\mathcal{A}$ and when $\mathbf{h} \in \mathsf{span}(\mathbf{B}_1, \mathbf{B}_2)$, $\mathcal{B}$ perfectly simulates $\mathsf{Hyb}_{3,j,1}$ to $\mathcal{A}$. Therefore, $\mathcal{B}$'s winning advantage against $\mathcal{C}$ is same as

$\mathcal{A}$'s winning advantage ϵ against $\mathcal{B}$. But this contradicts the assumption that such a p.p.t. adversary $\mathcal{B}$ cannot exist. This completes the proof of Claim 2.4.6. □

Proof of Claim 2.4.7. Observe that the only difference between $\mathsf{Hyb}_{3,j,1}$ and $\mathsf{Hyb}'_{3,j,1}$ is that in ciphertext component $c_{3,x}$ for all $x \in [n]$: $c_{3,x}$ contains $[\![\omega_x]\!]_1$, where ω_x is a secret share of $\gamma \mathbf{B}_3^{*T} \in \mathbb{Z}_q^{1\times(2k+1)}$ in $\mathsf{Hyb}_{3,j,1}$, but it is a secret share of $(\mathbf{B}_2^*\boldsymbol{\delta} + \gamma\mathbf{B}_3^*)^T$ in $\mathsf{Hyb}'_{3,j,1}$. Therefore, to prove that the hybrids are statistically indistinguishable, we will argue that $\mathbf{B}_2^*\boldsymbol{\delta}$ is information theoretically hidden to the adversary $\mathcal{A}$ in $\mathsf{Hyb}'_{3,j,1}$.

Suppose the challenge access policy $(\mathbf{M}, \rho)$ is defined over a set of authorities $U \subseteq \mathcal{AU}$, that is, $\rho : [n] \to U$. Recall from Section 2.3.6 that the game condition requires that $U_\mathcal{A} \cap U_\mathcal{B} = \emptyset$ and for each $\mathsf{GID} \in \mathcal{GID}$, it is required that $S \cup S_{\mathsf{GID}} \notin (\mathbf{M}, \rho)$, where

- $U_\mathcal{A} \subseteq U$ denotes the set of attribute authorities for which the adversary supplied the authority public keys $\{\mathsf{pk}_i\}$,
- $U_\mathcal{B}$ denote the set of attribute authorities for which the challenger created the master public key pairs in response to the authority setup query of the adversary so far,
- S denotes the subset of U containing the authorities in $U_\mathcal{A}$ plus the authorities for which the adversary made a master key query so far (in other words, S denotes the set of corrupt authorities),
- for each global identifier $\mathsf{GID} \in \mathcal{GID}$, S_{GID} denotes the subset of U containing authorities i such that the adversary queried a secret key for the pair (i, GID).

To show that $\mathbf{B}_2^*\boldsymbol{\delta}$ is information theoretically hidden from the adversary $\mathcal{A}$ in $\mathsf{Hyb}'_{3,j,1}$, we only need to rely on the second game condition and that too only for the jth GID, that is, we will use the fact that $S \cup S_{\mathsf{GID}_j} \notin (\mathbf{M}, \rho)$. Here, $S \cup S_{\mathsf{GID}_j} \notin (\mathbf{M}, \rho)$ is a shorthand for $(1, 0, \ldots, 0) \notin \mathsf{rowSpan}(\{\mathbf{M}_x\}_{\rho(x)\in S\cup S_{\mathsf{GID}_j}})$.

Note that the vectors $\mathbf{M}_x\mathbf{W}$ for all rows x of the challenge access matrix $\mathbf{M}$ labeled by corrupt authorities (that is, $\rho(x) \in S$) are information theoretically revealed to $\mathcal{A}$. However, by the game condition the subspace spanned by those rows does not include the vector $(1, 0, \ldots, 0)$. This means that there must exist some vector $\mathbf{u} \in \mathbb{Z}_q^\ell$ such that $\mathbf{u}$ is orthogonal to all these rows of $\mathbf{M}$ (that is $\mathbf{M}_x\mathbf{u} = 0$) but is not orthogonal to $(1, 0, \ldots, 0)$, that is, the first entry of $\mathbf{u}$ must be non-zero. We consider a basis $\mathbb{U}$ of $\mathbb{Z}_q^\ell$ involving the vector $\mathbf{u}$ and write $\mathbf{W} = \begin{pmatrix} (\mathbf{B}_2^*\boldsymbol{\delta} + \gamma\mathbf{B}_3^*)^T \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} = \widetilde{\mathbf{U}} + \mathbf{u}\mathbf{b}^T$ for some $\mathbf{b} \in \mathbb{Z}_q^{2k+1}$ and some $\widetilde{\mathbf{U}} \in \mathbb{Z}_q^{\ell\times(2k+1)}$ such that each column of $\widetilde{\mathbf{U}}$ lies in the column span of $\mathbb{U} \setminus \mathbf{u}$. Hence, $\widetilde{\mathbf{U}}$ reveals no information about $\mathbf{b}$. Now since the first entry of $\mathbf{u}$ is non-zero, it follows that the first row of $\mathbf{W}$, that is, $(\mathbf{B}_2^*\boldsymbol{\delta} + \gamma\mathbf{B}_3^*)^T$, depends on $\mathbf{b}$. But $\mathbf{M}_x\mathbf{W}$ for all the corrupted rows of $\mathbf{M}$ contains no information about $\mathbf{b}$ since $\mathbf{u}$ is orthogonal to all these rows. Thus, it follows that these rows do not leak information about $(\mathbf{B}_2^*\boldsymbol{\delta} + \gamma\mathbf{B}_3^*)^T$.

Therefore, the only possible way for $\mathcal{A}$ to get information about $\mathbf{B}_2^*\boldsymbol{\delta}$ is through the ciphertext components $c_{3,x}$ corresponding to the uncorrupted rows of $\mathbf{M}$. However, for each such row x, $\mathcal{A}$ can only recover $\mathbf{c}_x^T$, $\mathbf{M}_x\mathbf{W} + \mathbf{c}_x^T\mathbf{U}_{\rho(x)}$ information theoretically. For the uncorrupted rows x, we analyze two cases:

- x **such that** $\rho(x) \in S_{\mathsf{GID}_j}$**:** These are uncorrupted rows of $\mathbf{M}$ labeled by authorities who issued key for GID_j. From the game condition, it follows that $(1, 0, \ldots, 0) \notin \mathsf{rowSpan}(\{\mathbf{M}_x\}_{\rho(x)\in S_{\mathsf{GID}_j}})$. Therefore, using the same argument as for the analysis for rows corresponding to corrupt authorities above, it follows that $\mathbf{M}_x\mathbf{W}$ contains no information about $(\mathbf{B}_2^*\boldsymbol{\delta} + \gamma\mathbf{B}_3^*)^T$. Hence, $\mathbf{c}_x^T$, $\mathbf{M}_x\mathbf{W} + \mathbf{c}_x^T\mathbf{U}_{\rho(x)}$ reveals no information about $(\mathbf{B}_2^*\boldsymbol{\delta} + \gamma\mathbf{B}_3^*)^T$ to the adversary $\mathcal{A}$.

- x **such that** $\rho(x) \in U \setminus (S \cup S_{\mathsf{GID}_j})$**:** These are uncorrupted rows of $\mathbf{M}$ labeled by authorities who are neither corrupt nor issued key for GID_j. The game condition does not apply for these rows and the analysis requires more care as described next. Without loss of generality, we can compute $\mathbf{U}_{\rho(x)} := \mathbf{U}_{\rho(x),1}\mathbf{B}_1^{*T} + \mathbf{U}_{\rho(x),2}\mathbf{B}_2^{*T} + \mathbf{U}_{\rho(x),3}\mathbf{B}_3^{*T}$, where $\mathbf{U}_{\rho(x),1} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times k}$, $\mathbf{U}_{\rho(x),2} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times k}$, and $\mathbf{U}_{\rho(x),3} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times 1}$. Let the first entry of $\mathbf{M}_x$ be m_x, that is, $\mathbf{M}_x = (m_x, \ldots)$. Then, observe that we can write

$$
\begin{aligned}
&\mathbf{M}_x \begin{pmatrix} (\mathbf{B}_2^*\boldsymbol{\delta} + \gamma\mathbf{B}_3^*)^T \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + \mathbf{c}_x^T\mathbf{U}_{\rho(x)} \\
&= \mathbf{M}_x \begin{pmatrix} \gamma\mathbf{B}_3^{*T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + (m_x, \ldots) \begin{pmatrix} (\mathbf{B}_2^*\boldsymbol{\delta})^T \\ \mathbf{0} \end{pmatrix} + \mathbf{c}_x^T\mathbf{U}_{\rho(x)} \\
&= \mathbf{M}_x \begin{pmatrix} \gamma\mathbf{B}_3^{*T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + m_x(\mathbf{B}_2^*\boldsymbol{\delta})^T + \mathbf{c}_x^T\mathbf{U}_{\rho(x)} \\
&= \mathbf{M}_x \begin{pmatrix} \gamma\mathbf{B}_3^{*T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + (m_x\boldsymbol{\delta}^T + \mathbf{c}_x^T\mathbf{U}_{\rho(x),2})\mathbf{B}_2^{*T} + \mathbf{c}_x^T(\mathbf{U}_{\rho(x),1}\mathbf{B}_1^{*T} + \mathbf{U}_{\rho(x),3}\mathbf{B}_3^{*T}) \\
&= \mathbf{M}_x \begin{pmatrix} \gamma\mathbf{B}_3^{*T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + (\mathbf{c}_x^T\mathbf{U}'_{\rho(x),2})\mathbf{B}_2^{*T} + \mathbf{c}_x^T(\mathbf{U}_{\rho(x),1}\mathbf{B}_1^{*T} + \mathbf{U}_{\rho(x),3}\mathbf{B}_3^{*T})
\end{aligned}
$$

  where we can write $\mathbf{U}'_{\rho(x),2} = \mathbf{U}_{\rho(x),2} + \Delta$ such that $m_x\boldsymbol{\delta}^T = \mathbf{c}_x^T\Delta$. Therefore, to complete the proof, it suffices to argue that $\mathbf{U}_{\rho(x),2}$ and $\mathbf{U}'_{\rho(x),2}$ are identically distributed. We show this next.

  Observe that since ρ is injective, hence it follows that $\mathbf{U}_{\rho(x)}$ is a fresh random matrix and the only other place it appears is in secret keys $\mathsf{sk}_{\rho(x),\mathsf{GID}_{\mathsf{index}}}$. We argue that $\mathsf{sk}_{\rho(x),\mathsf{GID}_{\mathsf{index}}}$ information theoretically reveals no information about $\mathbf{U}_{\rho(x),2}$ to the adversary $\mathcal{A}$ and hence $\mathbf{U}_{\rho(x),2}$ and $\mathbf{U}'_{\rho(x),2}$ are identically distributed. To see this, observe that the $\mathbf{U}_{\rho(x)}$-dependent term of $\mathsf{sk}_{\rho(x),\mathsf{GID}_{\mathsf{index}}}$ is

of the form $\mathbf{U}_{\rho(x)} \odot \mathsf{H}_1(\mathsf{GID}_{\mathsf{index}})$, where $\mathsf{H}_1(\mathsf{GID}_{\mathsf{index}})$ is of the following form depending on index: $[\![\mathbf{B}_1\mathbf{h}_{\mathsf{GID}_{\mathsf{index}}} + \mathbf{B}_3]\!]_2$ if $\mathsf{index} \leq j-1$, $[\![\mathbf{B}_1\mathbf{h}_{\mathsf{GID}_{\mathsf{index}}} + \mathbf{B}_2\mathbf{h}'_{\mathsf{GID}_{\mathsf{index}}}]\!]_2$ if $\mathsf{index} = j$, and $[\![\mathbf{B}_1\mathbf{h}_{\mathsf{GID}_{\mathsf{index}}}]\!]_2$ if $\mathsf{index} > j$. We analyze the three cases separately:

- **Case 1:** $\mathsf{index} < j$**.** Observe that the $\mathbf{U}_{\rho(x)}$-dependent term of $\mathsf{sk}_{\rho(x),\mathsf{GID}_{\mathsf{index}}}$ information theoretically reveals $\mathbf{U}_{\rho(x),1}\mathbf{h}_{\mathsf{GID}_{\mathsf{index}}} + \mathbf{U}_{\rho(x),3}$ since $\mathbf{B}_2^{*T}\mathbf{B}_1 = \mathbf{0}$ and $\mathbf{B}_2^{*T}\mathbf{B}_3 = \mathbf{0}$.
- **Case 2:** $\mathsf{index} = j$**.** This case requires no analysis since adversary $\mathcal{A}$ never sees secret keys $\mathsf{sk}_{\rho(x),\mathsf{GID}_j}$. This is due to the definition of set S_{GID_j} and the fact that we are only considering x such that $\rho(x) \in U \setminus (S \cup S_{\mathsf{GID}_j})$.
- **Case 3:** $\mathsf{index} > j$**.** Observe that the $\mathbf{U}_{\rho(x)}$-dependent term of $\mathsf{sk}_{\rho(x),\mathsf{GID}_{\mathsf{index}}}$ information theoretically reveals $\mathbf{U}_{\rho(x),1}\mathbf{h}_{\mathsf{GID}_{\mathsf{index}}}$ since $\mathbf{B}_2^{*T}\mathbf{B}_1 = \mathbf{0}$.

Hence, it follows that $\mathsf{sk}_{\rho(x),\mathsf{GID}_{\mathsf{index}}}$ information theoretically reveals no information about $\mathbf{U}_{\rho(x),2}$, so $\mathbf{U}_{\rho(x)}$ and $\mathbf{U}'_{\rho(x)}$ are identically distributed. Thus, for all x such that $\rho(x) \in U \setminus (S \cup S_{\mathsf{GID}_j})$, ciphertext components $c_{3,x}$ reveal no information about $\mathbf{B}_2^*\boldsymbol{\delta}$ to the adversary $\mathcal{A}$.

To complete the proof, we argue that substituting $\mathbf{U}_{\rho(x),2}$ with $\mathbf{U}'_{\rho(x),2}$ (as described above) for all rows x of matrix $\mathbf{M}$ for which the challenger sampled the authority keys (that is, uncorrupted rows plus the rows for which the adversary queried the master secret key) allows us to move from $\mathsf{Hyb}'_{3,j,1}$ to $\mathsf{Hyb}_{3,j,1}$. We have already argued that this substitution does not change the distribution of the secret keys and ciphertext obtained by the adversary $\mathcal{A}$ for the uncorrupted rows of $\mathbf{M}$. For the case of rows x of $\mathbf{M}$ for which the adversary queried the master secret key, the adversary additionally learns $\mathbf{U}'_{\rho(x),2}$ and $\mathbf{M}_x \begin{pmatrix} \gamma\mathbf{B}_3^{*T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix}$ in $\mathsf{Hyb}_{3,j,1}$ and $\mathbf{U}_{\rho(x),2}$ and $\mathbf{M}_x \begin{pmatrix} (\mathbf{B}_2^*\boldsymbol{\delta} + \gamma\mathbf{B}_3^*)^T \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix}$ in $\mathsf{Hyb}'_{3,j,1}$ and we argue that this does not help the adversary $\mathcal{A}$ to distinguish between $\mathsf{Hyb}_{3,j,1}$ and $\mathsf{Hyb}'_{3,j,1}$. This is because $\mathbf{U}_{\rho(x),2}$ and $\mathbf{U}'_{\rho(x),2}$ are identically distributed and $\mathbf{M}_x \begin{pmatrix} \gamma\mathbf{B}_3^{*T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} = \mathbf{M}_x \begin{pmatrix} (\mathbf{B}_2^*\boldsymbol{\delta} + \gamma\mathbf{B}_3^*)^T \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix}$ due to the game condition.

From the case analysis above, it follows that $\mathsf{Hyb}_{3,j,1}$ and $\mathsf{Hyb}'_{3,j,1}$ are statistically indistinguishable. This completes the proof of Claim 2.4.7.

□

Proof of Claim 2.4.8. Similar to proof of Claim 2.4.6. □

Proof of Claim 2.4.9. Similar to the proof of Claim 2.4.5. □

Proof of Claim 2.4.10. Similar to proof of Claim 2.4.6. □

Proof of Claim 2.4.13. This holds since Hyb_5 contains no information about msg_0 and msg_1. □

2.5 MA-ABE for NC^1 with Multi-Use Security

Our construction of MA-ABE in Section 2.4 only supprts single-use of attributes since its security proof relies on ρ being injective. In this section, we present our construction of decentralized MA-ABE for NC^1 with multi-use security. To prove security, we rely on the Core 1-ABE construction of [KW19]. We first recall Core 1-ABE in Section 2.5.1 and then present our construction in Section 2.5.2.

2.5.1 Core 1-ABE

Definition 2.5.1 (Core 1-ABE Games $\mathsf{G}_{\mathcal{A}}^{\text{1-ABE},0}$, $\mathsf{G}_{\mathcal{A}}^{\text{1-ABE},1}$ [KW19])**.** *For a stateful adversary $\mathcal{A}$, for $b \in \{0,1\}$, define the game $\mathsf{G}_{\mathcal{A}}^{\text{1-ABE},b}$ as*

$$\begin{aligned}
&\mathbf{u}_i \leftarrow \mathsf{CPA.Setup}(1^\lambda),\\
&(\mu^{(0)}, \mu^{(1)}) \xleftarrow{\$} \mathbb{Z}_q,\\
&b' \leftarrow \mathcal{A}^{\mathcal{O}_X(\cdot),\mathcal{O}_E(\cdot,\cdot),\mathcal{O}_F(\cdot)}(\mu^{(0)}),\\
&\textbf{ret } b'
\end{aligned}$$

where $\mathcal{O}_F(f) = \mathsf{ct} := \{\mathsf{sk}'_f = \{\mu_j\}_{\rho'(j)=0} \cup \{\mathsf{CPA.Enc}(\mathbf{u}_{\rho'(j)}, \mu_j)\}_{\rho'(j)\neq 0}\}$, where $(\{\mu_j\}, \rho') \leftarrow \mathsf{LSSS.Share}(f, \mu^{(b)})$, and $\mathcal{O}_X(x) := (\mathsf{ct}'_x = \{\mathbf{u}_i\}_{x_i=1})$, and $\mathcal{O}_E(i,m) := \mathsf{CPA.Enc}(\mathbf{u}_i, m)$, with the restriction that (i) only one query is made to each $\mathcal{O}_F(\cdot)$ and $\mathcal{O}_X(\cdot)$, and (ii) the queries f and x to $\mathcal{O}_F(\cdot,\cdot)$ and $\mathcal{O}_X(\cdot)$ respectively satisfy $f(x) = 0$.

To be clear, the b in the game $\mathsf{G}_{\mathcal{A}}^{\text{1-ABE},b}$ affects only the implementation of the $\mathcal{O}_F(\cdot)$ oracle (where $\mu^{(b)}$ is shared), and $\mathcal{A}$ is given $\mu^{(0)}$ as input in *both* games.

The CPA-secure symmetric encryption scheme in [KW19] is constructed as follows:

$\mathsf{CPA.Setup}(1^\lambda)$:

1 : Run $\mathbb{G} \leftarrow \mathcal{G}(1^\lambda)$. Sample $\mathbf{u} \xleftarrow{\$} \mathbb{Z}_q^k$. Output $\mathsf{sk} := \mathbf{u}$.

$\mathsf{CPA.Enc}(\mathsf{sk}, [\![m]\!])$:

1 : Sample $\mathbf{r} \xleftarrow{\$} \mathbb{Z}_q^k$. Output $\mathsf{ct} := (\mathsf{ct}_0 = [\![\mathbf{r}]\!], \mathsf{ct}_1 = [\![m + \mathbf{u}^T\mathbf{r}]\!])$.

$\mathsf{CPA.Dec}(\mathsf{sk}, \mathsf{ct})$:

1 : Output $\mathsf{ct}_1 / (\mathsf{sk}^T \odot \mathsf{ct}_0)$.

Figure 2.3: CPA-secure symmetric encryption scheme in [KW19]

Lemma 2.5.2 (Core 1-ABE security [KW19])**.** *For the* Core 1-ABE *component of Definition 2.5.1 implemented with the CPA-secure symmetric encryption scheme* $(\mathsf{CPA.Setup}, \mathsf{CPA.Enc}, \mathsf{CPA.Dec})$ *from Figure 2.3, the following is a negligible function in λ when the policy class is NC^1:*

$$|\Pr[\mathsf{G}_{\mathcal{A}}^{\text{1-ABE},0}(\lambda) = 1] - \Pr[\mathsf{G}_{\mathcal{A}}^{\text{1-ABE},1}(\lambda) = 1]|$$

2.5.2 Our Construction

The construction is same as the one in Section 2.4 except that the dimensions of some of the matrices are changed as follows:

- $\mathsf{GlobalSetup}(1^\lambda)$: $\mathbf{A}_1 \xleftarrow{\$} \mathbb{Z}_q^{k\times 2k}$.
- $\mathsf{AuthSetup}(\mathsf{gp}, i)$: $\mathbf{V}_i, \mathbf{U}_i \xleftarrow{\$} \mathbb{Z}_q^{2k\times(2k+1)}$. While this increases the size of msk_i, we note that the size of pk_i remains unchanged.
- $\mathsf{KGen}(\mathsf{gp}, \mathsf{msk}_i, \mathsf{GID})$: Change in size of msk_i affects the size of $sk_{i,\mathsf{GID}}$ as follows: $sk_{i,\mathsf{GID}} := [\![\mathbf{V}_i\mathbf{k} + \mathbf{U}_i\mathbf{h}_{\mathsf{GID}}]\!]_2 \in \mathbb{G}_2^{2k}$.
- $\mathsf{Enc}(\mathsf{gp}, \mathsf{msg}, (\mathbf{M}, \rho), \{\mathsf{pk}_{\rho(i)}\}_{i\in[n]})$: Change in size of $\mathbf{A}_1$ affects the size of $c_{1,x}$ as follows: $c_{1,x} := [\![\mathbf{s}_x^T\mathbf{A}_1]\!]_1 \in \mathbb{G}_1^{1\times 2k}$.

Theorem 2.5.3. *The MA-ABE construction in Figure 2.2 amended with dimension changes specified in Section 2.5.2 supports* NC^1 *circuits and is fully adaptively secure (Definition 2.3.6) if all of the following hold true.*

- *game condition holds.*
- *k-MDDH assumption holds in groups* $\mathbb{G}_1$ *and* $\mathbb{G}_2$ *(Assumption 2.3.8).*
- $\mathsf{SD}^{\mathbb{G}_2}_{\mathbf{B}_1\to\mathbf{B}_1,\mathbf{B}_2}$ *assumption holds (Assumption 2.3.9).*
- $\mathsf{SD}^{\mathbb{G}_2}_{\mathbf{B}_2\to\mathbf{B}_2,\mathbf{B}_3}$ *assumption holds (Assumption 2.3.9).*

The high-level idea of the proof is same as in Section 2.4.2 except that we replace the statistical indistinguishability steps (these relied on the injectivity of ρ earlier) with computational indistinguishability steps using the security of Core 1-ABE of [KW19]. In Section 2.5.1, we first formally define the Core 1-ABE and then state the security of the Core 1-ABE construction of [KW19]. Then, in Section 2.5.3, we prove Theorem 2.5.3 via a sequence of hybrid games.

2.5.3 Proof of Theorem 2.5.3

We prove full adaptive security along with multi-use support (that is, ρ is not required to be injective) of the MA-ABE scheme for NC^1 presented in Figure 2.2 along with the amended dimension changes specified in Section 2.5.2. We prove this via a sequence of hybrid games. Suppose the adversary makes q number of queries to the random oracle H_1. Then, the hybrid games are as follows:

$$\mathsf{Hyb}_{\mathsf{Real}}, \mathsf{Hyb}'_{\mathsf{Real}}, \mathsf{Hyb}_1, \mathsf{Hyb}_2, \{\mathsf{Hyb}_{3,j,1}, \mathsf{Hyb}'_{3,j,1}, \mathsf{Hyb}_{3,j,2}, \mathsf{Hyb}'_{3,j,2}, \mathsf{Hyb}_{3,j,3}\}_{j\in[q]}, \mathsf{Hyb}_4, \mathsf{Hyb}_5.$$

The hybrid descriptions are exactly the same as in Section 2.4.2. The main difference is that each of the statistical indistinguishability steps are replaced by computational indistinguishability, where instead of relying on the injectivity of ρ, we will reduce the indistinguishability to the security of the Core 1-ABE construction of [KW19]. In more detail, we will use the security of the Core 1-ABE to prove the indistinguishability of the following pairs of distributions:

- Hyb_1 and Hyb_2,
- $\mathsf{Hyb}_{3,j,1}$ and $\mathsf{Hyb}'_{3,j,1}$ for $j \in [q]$,
- $\mathsf{Hyb}_{3,j,2}$ and $\mathsf{Hyb}'_{3,j,2}$ for $j \in [q]$,
- $\mathsf{Hyb}_{3,q}$ and Hyb_4.

Note that the security of Core 1-ABE is not an additional assumption required since we already assumed k-MDDH.

Before proceeding to prove the above transitions, we note that the change of parameter dimensions in the construction slightly affects the parameter dimensions in $\mathsf{GlobalSetup}^*$ algorithm used from hybrid $\mathsf{Hyb}'_{\mathsf{Real}}$ onwards as follows. We describe $\mathsf{GlobalSetup}^*$ next and highlight the changes of dimensions.

$\mathsf{GlobalSetup}^*$ runs the same computation as $\mathsf{GlobalSetup}$ to compute gp and additionally also computes the following:

$$\mathbf{A}_2 \xleftarrow{\$} \mathbb{Z}_q^{\boxed{k} \times \boxed{2k}}, \mathbf{B}_1, \mathbf{B}_2 \xleftarrow{\$} \mathbb{Z}_q^{(2k+1)\times k}, \mathbf{B}_3 \xleftarrow{\$} \mathbb{Z}_q^{(2k+1)\times 1}$$

$$\left(\mathbf{A}_1^*, \mathbf{A}_2^*\right) = \begin{pmatrix}\mathbf{A}_1\\ \mathbf{A}_2\end{pmatrix}^{-1}, \left(\mathbf{B}_1^*, \mathbf{B}_2^*, \mathbf{B}_3^*\right) = \left(\left(\mathbf{B}_1, \mathbf{B}_2, \mathbf{B}_3\right)^{-1}\right)^T.$$

Let $\mathbf{A} = \begin{pmatrix}\mathbf{A}_1\\ \mathbf{A}_2\end{pmatrix}$, $\mathbf{A}^* = \left(\mathbf{A}_1^*, \mathbf{A}_2^*\right)$, $\mathbf{B} = \left(\mathbf{B}_1, \mathbf{B}_2, \mathbf{B}_3\right)$, $\mathbf{B}^* = \left(\mathbf{B}_1^*, \mathbf{B}_2^*, \mathbf{B}_3^*\right)$. Then, $\mathsf{st} = (\mathbf{A}, \mathbf{A}^*, \mathbf{B}, \mathbf{B}^*)$.

Previously, the dimensions of $\mathbf{A}_1^*$ were $(k+1) \times k$, now they are $\boxed{2k} \times k$. Previously, the dimensions of $\mathbf{A}_2^*$ were $(k+1) \times 1$, now they are $\boxed{2k} \times \boxed{k}$. Observe that $\forall i, j \in \{1,2\}$: $\mathbf{A}_i \mathbf{A}_j^* = \mathbf{I}$ if $i = j$, and $\mathbf{0}$ if $i \neq j$. Similarly, $\forall i, j \in \{1,2,3\}$: $\mathbf{B}_i^T \mathbf{B}_j^* = \mathbf{I}$ if $i = j$, and $\mathbf{0}$ if $i \neq j$.

Next we prove the aforementioned transitioned through a series of claims.

Claim 2.5.4. *If the game condition holds and* Core 1-ABE *is secure, then,* Hyb_1 *and* Hyb_2 *are computationally indistinguishable.*

Proof. Observe that the only difference between Hyb_1 and Hyb_2 is that in ciphertext component $c_{3,x}$ for all $x \in [n]$: $c_{3,x}$ contains $[\![\omega_x]\!]_1$, where ω_x is a secret share of $\mathbf{0}^T \in \mathbb{Z}_q^{1\times(2k+1)}$ in Hyb_1, but it is a secret share of

$\gamma \mathbf{B}_3^{*T}$ in Hyb_2. Therefore, to prove that the hybrids are computationally indistinguishable, we will reduce the indistinguishability to the security of Core 1-ABE.

Suppose that there exists an adversary $\mathcal{A}$ that can distinguish between Hyb_1 and Hyb_2 with non-negligible probability. We will construct an adversary $\mathcal{B}$ that can break the security of Core 1-ABE with non-negligible probability. Suppose $\mathcal{C}$ is the challenger for the Core 1-ABE security game. $\mathcal{B}$ will run $\mathcal{A}$ and simulate the hybrid games.

We describe the reduction $\mathcal{B}$ next. $\mathcal{C}$ samples two messages $\mu^{(0)}, \mu^{(1)} \xleftarrow{\$} \mathbb{Z}_q$, samples a challenge bit $\beta \xleftarrow{\$} \{0,1\}$ and provides $\mu^{(0)}$ to $\mathcal{B}$. $\mathcal{B}$ proceeds as in Hyb_1 except that when it obtains the challenge query $(\mathsf{msg}_0, \mathsf{msg}_1, (\mathbf{M}, \rho), \{\mathsf{pk}_i\}_{i \in U_\mathcal{A}})$ from $\mathcal{A}$, $\mathcal{B}$ makes oracle query for $\mathcal{C}.\mathcal{O}_F((\mathbf{M}, \rho))$. $\mathcal{C}$ computes secret shares of $\mu^{(\beta)}$ as $\{\mu_x\} \leftarrow \mathsf{LSSS.Share}((\mathbf{M}, \rho), \mu^{(\beta)})$ and responds with $(\{\mu_x\}_{\rho(x)=0}, \{\mathsf{CPA.Enc}(\mathbf{u}_{\rho(x)}, \mu_x)\}_{\rho(x)\neq 0})$, where $\mathsf{CPA.Enc}(\mathbf{u}_{\rho(x)}, \mu_x) = ([\![\mathbf{r}_x]\!]_1, [\![\mu_x + \mathbf{r}_x^T \mathbf{u}_{\rho(x)}]\!]_1)$. The way $\mathcal{B}$ embeds this information obtained from $\mathcal{C}$ in its response to $\mathcal{A}$ is as follows: $\mathcal{B}$ samples $\widetilde{\mathbf{c}}_x \xleftarrow{\$} \mathbb{Z}_q^k$ and sets $[\![\mathbf{c}_x]\!]_1 := [\![\mathbf{A}_1^T \widetilde{\mathbf{c}}_x]\!]_1 \cdot (\mathbf{A}_2^T \odot [\![\mathbf{r}_x]\!]_1)$. $\mathcal{B}$ computes $c_0, c_{1,x}, c_{2,x}$ as in Hyb_1. For $c_{3,x}$, $\mathcal{B}$ computes secret shares of $-\mu^{(0)} \mathbf{B}_3^{*T}$ as $\{\widetilde{\omega_x}\} \leftarrow \mathsf{LSSS.Share}((\mathbf{M}, \rho), -\mu^{(0)} \mathbf{B}_3^{*T})$ and then computes $c_{3,x}$ as follows:

$$c_{3,x} := \begin{cases} [\![\widetilde{\omega_x}]\!]_1 \cdot (\mathbf{s}_x^T \odot \mathsf{pk}_{\rho(x),1}) \cdot ([\![\mu_x + \mathbf{r}_x^T \mathbf{u}_{\rho(x)}]\!]_1 \odot \mathbf{B}_3^{*T}) & \text{, if } x \in U_\mathcal{A} \text{ and } \rho(x) \neq 0 \\ [\![\widetilde{\omega_x}]\!]_1 \cdot (\mathbf{s}_x^T \odot \mathsf{pk}_{\rho(x),1}) \cdot ([\![1]\!]_1 \odot \mu_x \mathbf{B}_3^{*T}) & \text{, if } x \in U_\mathcal{A} \text{ and } \rho(x) = 0 \\ [\![\widetilde{\omega_x}]\!]_1 \cdot ([\![\mathbf{c}_x^T]\!]_1 \odot \widetilde{\mathbf{U}_{\rho(x)}}) \cdot ([\![\mu_x + \mathbf{r}_x^T \mathbf{u}_{\rho(x)}]\!]_1 \odot \mathbf{B}_3^{*T}) & \text{, if } x \in \overline{U_\mathcal{A}} \text{ and } \rho(x) \neq 0 \\ [\![\widetilde{\omega_x}]\!]_1 \cdot ([\![\mathbf{c}_x^T]\!]_1 \odot \widetilde{\mathbf{U}_{\rho(x)}}) \cdot ([\![1]\!]_1 \odot \mu_x \mathbf{B}_3^{*T}) & \text{, if } x \in \overline{U_\mathcal{A}} \text{ and } \rho(x) = 0 \end{cases},$$

where for oracle $\mathsf{AuthSetup}(i)$, $\mathcal{B}$ samples master secret key as $\mathsf{msk}_i = (\mathbf{V}_i \xleftarrow{\$} \mathbb{Z}_q^{(2k)\times(2k+1)}, \widetilde{\mathbf{U}_i} \xleftarrow{\$} \mathbb{Z}_q^{(2k)\times(2k+1)})$. $\mathcal{B}$ programs oracles $\mathsf{H}_1(\mathsf{GID})$, $\mathsf{KGen}(i, \mathsf{GID})$, $\mathsf{Corrupt}(i)$ as in Hyb_1. $\mathcal{B}$ implicitly sets

$$\mathbf{U}_i = \widetilde{\mathbf{U}_i} + \mathbf{A}_2^* \mathbf{u}_i \mathbf{B}_3^{*T} \text{ ($\mathcal{B}$ does not know } \mathbf{u}_i \text{ that is chosen by } \mathcal{C}).$$

Observe that when we change $\widetilde{\mathbf{U}_i}$ to $\mathbf{U}_i$, only pk_i, $\mathsf{sk}_{i,\mathsf{GID}}$ and ct are changed. We anaylze the effect of this change next.

- pk_i **remains unchanged:** Since $\mathbf{A}_1 \mathbf{A}_2^* = \mathbf{0}$, it follows that $\mathsf{pk}_{i,1} = [\![\mathbf{A}_1 \mathbf{U}_i]\!]_1 = [\![\mathbf{A}_1 \widetilde{\mathbf{U}_i} + \mathbf{A}_1 \mathbf{A}_2^* \mathbf{u}_i \mathbf{B}_3^{*T}]\!]_1 = [\![\mathbf{A}_1 \widetilde{\mathbf{U}_i}]\!]_1$.
- $\mathsf{sk}_{i,\mathsf{GID}}$ **remains unchanged for all** GID**:** Since $\mathbf{B}_3^{*T} \mathbf{B}_1 = \mathbf{0}$, it follows that $\mathsf{sk}_{i,\mathsf{GID}} = [\![\mathbf{V}_i \mathbf{k} + \mathbf{U}_i \mathbf{B}_1 \mathbf{h}_{\mathsf{GID}}]\!]_2 = [\![\mathbf{V}_i \mathbf{k} + \widetilde{\mathbf{U}_i} \mathbf{B}_1 \mathbf{h}_{\mathsf{GID}} + \mathbf{A}_2^* \mathbf{u}_i \mathbf{B}_3^{*T} \mathbf{B}_1 \mathbf{h}_{\mathsf{GID}}]\!]_2 = [\![\mathbf{V}_i \mathbf{k} + \widetilde{\mathbf{U}_i} \mathbf{B}_1 \mathbf{h}_{\mathsf{GID}}]\!]_2$.
- For the challenge ciphertext ct, observe that the only change is in the $c_{3,x}$ component where $x \in \overline{U_\mathcal{A}}$.

For x such that $\rho(x) \neq 0$, we have:

$$\begin{aligned}
c_{3,x} &= [\![\widetilde{\omega_x}]\!]_1 \cdot ([\![\mathbf{c}_x^T]\!]_1 \odot \widetilde{\mathbf{U}_{\rho(x)}}) \cdot ([\![\mu_x + \mathbf{r}_x^T \mathbf{u}_{\rho(x)}]\!]_1 \odot \mathbf{B}_3^{*T}) \\
&= [\![\widetilde{\omega_x} + \mathbf{c}_x^T \widetilde{\mathbf{U}_{\rho(x)}} + (\mu_x + \mathbf{r}_x^T \mathbf{u}_{\rho(x)}) \mathbf{B}_3^{*T}]\!]_1 \\
&= [\![(\widetilde{\omega_x} + \mu_x \mathbf{B}_3^{*T}) + \mathbf{c}_x^T \widetilde{\mathbf{U}_{\rho(x)}} + \mathbf{r}_x^T \mathbf{u}_{\rho(x)} \mathbf{B}_3^{*T}]\!]_1 \\
&= [\![(\widetilde{\omega_x} + \mu_x \mathbf{B}_3^{*T}) + \mathbf{c}_x^T \mathbf{U}_{\rho(x)} - \mathbf{c}_x^T \mathbf{A}_2^* \mathbf{u}_{\rho(x)} \mathbf{B}_3^{*T} + \mathbf{r}_x^T \mathbf{u}_{\rho(x)} \mathbf{B}_3^{*T}]\!]_1 \\
&= [\![(\widetilde{\omega_x} + \mu_x \mathbf{B}_3^{*T}) + \mathbf{c}_x^T \mathbf{U}_{\rho(x)} - (\mathbf{A}_1^T \widetilde{\mathbf{c}_x} + \mathbf{A}_2^T \mathbf{r}_x)^T \mathbf{A}_2^* \mathbf{u}_{\rho(x)} \mathbf{B}_3^{*T} + \mathbf{r}_x^T \mathbf{u}_{\rho(x)} \mathbf{B}_3^{*T}]\!]_1 \\
&= [\![(\widetilde{\omega_x} + \mu_x \mathbf{B}_3^{*T}) + \mathbf{c}_x^T \mathbf{U}_{\rho(x)}]\!]_1.
\end{aligned}$$

For x such that $\rho(x) = 0$, we have that $\mathbf{u}_{\rho(x)} = \mathbf{u}_0 = \mathbf{0}$. So, $\mathbf{U}_{\rho(x)} = \widetilde{\mathbf{U}_{\rho(x)}}$. Thus, we have:

$$\begin{aligned}
c_{3,x} &= [\![\widetilde{\omega_x}]\!]_1 \cdot ([\![\mathbf{c}_x^T]\!]_1 \odot \widetilde{\mathbf{U}_{\rho(x)}}) \cdot ([\![1]\!]_1 \odot \mu_x \mathbf{B}_3^{*T}) \\
&= [\![\widetilde{\omega_x} + \mathbf{c}_x^T \widetilde{\mathbf{U}_{\rho(x)}} + \mu_x \mathbf{B}_3^{*T}]\!]_1 \\
&= [\![(\widetilde{\omega_x} + \mu_x \mathbf{B}_3^{*T}) + \mathbf{c}_x^T \mathbf{U}_{\rho(x)}]\!]_1.
\end{aligned}$$

Recall that $\widetilde{\omega_x}$ is a secret share of $-\mu^{(0)} \mathbf{B}_3^{*T}$ and μ_x is a secret share of $\mu^{(\beta)}$. Then, from linearity of the LSSS secret sharing scheme, it follows that $(\widetilde{\omega_x} + \mu_x \mathbf{B}_3^{*T})$ is a secret share of $-\mu^{(0)} \mathbf{B}_3^{*T} + \mu^{(\beta)} \mathbf{B}_3^{*T} = (\mu^{(\beta)} - \mu^{(0)}) \mathbf{B}_3^{*T}$.

Observe that if $\mathcal{A}$ satisfies the game condition, then $\mathcal{B}$ is admissible in its game with $\mathcal{C}$. Observe that when $\mathcal{C}$ chooses $\beta = 0$, $(\widetilde{\omega_x} + \mu_x \mathbf{B}_3^{*T})$ are secret shares of $\mathbf{0}^T$ and thus $\mathcal{B}$ perfectly simulates Hyb_1. Similarly, when $\mathcal{C}$ chooses $\beta = 1$, $(\widetilde{\omega_x} + \mu_x \mathbf{B}_3^{*T})$ are secret shares of $(\mu^{(1)} - \mu^{(0)}) \mathbf{B}_3^{*T}$ and thus $\mathcal{B}$ perfectly simulates Hyb_2 where γ is set implicitly as $\gamma = \mu^{(1)} - \mu^{(0)}$. Thus, if $\mathcal{A}$ can distinguish between Hyb_1 and Hyb_2 non-negligible probability, then $\mathcal{B}$ can break the security of Core 1-ABE with non-negligible probability.

□

Claim 2.5.5. *If the game condition holds and* Core 1-ABE *is secure, then,* $\mathsf{Hyb}_{3,j,1}$ *and* $\mathsf{Hyb}'_{3,j,1}$ *are computationally indistinguishable.*

Proof. Observe that the only difference between $\mathsf{Hyb}_{3,j,1}$ and $\mathsf{Hyb}'_{3,j,1}$ is that in ciphertext component $c_{3,x}$ for all $x \in [n]$: $c_{3,x}$ contains $[\![\omega_x]\!]_1$, where ω_x is a secret share of $\gamma \mathbf{B}_3^{*T} \in \mathbb{Z}_q^{1 \times (2k+1)}$ in $\mathsf{Hyb}_{3,j,1}$, but it is a secret share of $(\mathbf{B}_2^* \delta + \gamma \mathbf{B}_3^*)^T$ in $\mathsf{Hyb}'_{3,j,1}$. Therefore, to prove that the hybrids are computationally indistinguishable, we will reduce the indistinguishability to the security of Core 1-ABE.

Suppose that there exists an adversary $\mathcal{A}$ that can distinguish between $\mathsf{Hyb}_{3,j,1}$ and $\mathsf{Hyb}'_{3,j,1}$ with non-negligible probability. We will construct an adversary $\mathcal{B}$ that can break the security of Core 1-ABE with

non-negligible probability. Suppose $\mathcal{C}$ is the challenger for the Core 1-ABE security game. $\mathcal{B}$ will run $\mathcal{A}$ and simulate the hybrid games.

We describe the reduction $\mathcal{B}$ next. $\mathcal{C}$ samples two messages $\mu^{(0)}, \mu^{(1)} \xleftarrow{\$} \mathbb{Z}_q$, samples a challenge bit $\beta \xleftarrow{\$} \{0,1\}$ and provides $\mu^{(0)}$ to $\mathcal{B}$. $\mathcal{B}$ proceeds as in $\mathsf{Hyb}_{3,j,1}$ except that when it obtains the challenge query $(\mathsf{msg}_0, \mathsf{msg}_1, (\mathbf{M}, \rho), \{\mathsf{pk}_i\}_{i \in U_\mathcal{A}})$ from $\mathcal{A}$, $\mathcal{B}$ makes oracle query for $\mathcal{C}.\mathcal{O}_F((\mathbf{M}, \rho))$. $\mathcal{C}$ computes secret shares of $\mu^{(\beta)}$ as $\{\mu_x\} \leftarrow \mathsf{LSSS.Share}((\mathbf{M}, \rho), \mu^{(\beta)})$ and responds with $(\{\mu_x\}_{\rho(x)=0}, \{\mathsf{CPA.Enc}(\mathbf{u}_{\rho(x)}, \mu_x)\}_{\rho(x)\neq 0})$, where $\mathsf{CPA.Enc}(\mathbf{u}_{\rho(x)}, \mu_x) = ([\![\mathbf{r}_x]\!]_1, [\![\mu_x + \mathbf{r}_x^T \mathbf{u}_{\rho(x)}]\!]_1)$. The way $\mathcal{B}$ embeds this information obtained from $\mathcal{C}$ in its response to $\mathcal{A}$ is as follows: $\mathcal{B}$ samples $\widetilde{\mathbf{c}}_x \xleftarrow{\$} \mathbb{Z}_q^k$ and sets $[\![\mathbf{c}_x]\!]_1 := [\![\mathbf{A}_1^T \widetilde{\mathbf{c}}_x]\!]_1 \cdot (\mathbf{A}_2^T \odot [\![\mathbf{r}_x]\!]_1)$. $\mathcal{B}$ computes $c_0, c_{1,x}, c_{2,x}$ as in $\mathsf{Hyb}_{3,j,1}$. For $c_{3,x}$, $\mathcal{B}$ samples $\gamma \xleftarrow{\$} \mathbb{Z}_q$, $\delta_0 \xleftarrow{\$} \mathbb{Z}_q^k$ and computes secret shares of $(-\mu^{(0)} \mathbf{B}_2^* \delta_0 + \gamma \mathbf{B}_3^*)^T$ as $\{\widetilde{\omega_x}\} \leftarrow \mathsf{LSSS.Share}((\mathbf{M}, \rho), (-\mu^{(0)} \mathbf{B}_2^* \delta_0 + \gamma \mathbf{B}_3^*)^T)$ and then computes $c_{3,x}$ as follows:

$$c_{3,x} := \begin{cases} [\![\widetilde{\omega_x}]\!]_1 \cdot (\mathbf{s}_x^T \odot \mathsf{pk}_{\rho(x),1}) \cdot ([\![\mu_x + \mathbf{r}_x^T \mathbf{u}_{\rho(x)}]\!]_1 \odot {\delta_0}^T \mathbf{B}_2^{*T}) & \text{, if } x \in U_\mathcal{A} \text{ and } \rho(x) \neq 0 \\ [\![\widetilde{\omega_x}]\!]_1 \cdot (\mathbf{s}_x^T \odot \mathsf{pk}_{\rho(x),1}) \cdot ([\![1]\!]_1 \odot \mu_x {\delta_0}^T \mathbf{B}_2^{*T}) & \text{, if } x \in U_\mathcal{A} \text{ and } \rho(x) = 0 \\ [\![\widetilde{\omega_x}]\!]_1 \cdot ([\![\mathbf{c}_x^T]\!]_1 \odot \widetilde{\mathbf{U}_{\rho(x)}}) \cdot ([\![\mu_x + \mathbf{r}_x^T \mathbf{u}_{\rho(x)}]\!]_1 \odot {\delta_0}^T \mathbf{B}_2^{*T}) & \text{, if } x \in \overline{U_\mathcal{A}} \text{ and } \rho(x) \neq 0 \\ [\![\widetilde{\omega_x}]\!]_1 \cdot ([\![\mathbf{c}_x^T]\!]_1 \odot \widetilde{\mathbf{U}_{\rho(x)}}) \cdot ([\![1]\!]_1 \odot \mu_x {\delta_0}^T \mathbf{B}_2^{*T}) & \text{, if } x \in \overline{U_\mathcal{A}} \text{ and } \rho(x) = 0 \end{cases},$$

where for oracle $\mathsf{AuthSetup}(i)$, $\mathcal{B}$ samples master secret key as $\mathsf{msk}_i = (\mathbf{V}_i \xleftarrow{\$} \mathbb{Z}_q^{(2k)\times(2k+1)}, \widetilde{\mathbf{U}_i} \xleftarrow{\$} \mathbb{Z}_q^{(2k)\times(2k+1)})$. $\mathcal{B}$ programs oracles $\mathsf{H}_1(\mathsf{GID})$, $\mathsf{KGen}(i, \mathsf{GID})$, $\mathsf{Corrupt}(i)$ as in $\mathsf{Hyb}_{3,j,1}$. $\mathcal{B}$ implicitly sets

$$\mathbf{U}_i = \widetilde{\mathbf{U}_i} + \mathbf{A}_2^* \mathbf{u}_i {\delta_0}^T \mathbf{B}_2^{*T} \text{ ($\mathcal{B}$ does not know } \mathbf{u}_i \text{ that is chosen by } \mathcal{C}).$$

Observe that when we change $\widetilde{\mathbf{U}_i}$ to $\mathbf{U}_i$, only pk_i, $\mathsf{sk}_{i,\mathsf{GID}}$ and ct are changed. We anaylze the effect of this change next.

- pk_i **remains unchanged:** Since $\mathbf{A}_1 \mathbf{A}_2^* = \mathbf{0}$, it follows that $\mathsf{pk}_{i,1} = [\![\mathbf{A}_1 \mathbf{U}_i]\!]_1 = [\![\mathbf{A}_1 \widetilde{\mathbf{U}_i} + \mathbf{A}_1 \mathbf{A}_2^* \mathbf{u}_i {\delta_0}^T \mathbf{B}_2^{*T}]\!]_1 = [\![\mathbf{A}_1 \widetilde{\mathbf{U}_i}]\!]_1$.

- $\mathsf{sk}_{i,\mathsf{GID}_{\mathsf{index}}}$ **remains unchanged for all** $\mathsf{GID}_{\mathsf{index}}$ **when** $\mathsf{index} \neq j$**. For** $\mathsf{index} = j$**,** $\mathsf{sk}_{i,\mathsf{GID}_{\mathsf{index}}}$ **changes but $\mathcal{B}$ can simulate the change using oracle query** $\mathcal{C}.\mathcal{O}_X(\{i\})$**.** Recall that $\mathsf{sk}_{i,\mathsf{GID}_{\mathsf{index}}}$ is of the form $[\![\mathbf{V}_i \mathbf{k}]\!]_2 \cdot (\mathbf{U}_i \odot \mathsf{H}_1(\mathsf{GID}_{\mathsf{index}}))$, where $\mathsf{H}_1(\mathsf{GID}_{\mathsf{index}})$ is of the following form depending on index:

$$\mathsf{H}_1(\mathsf{GID}_{\mathsf{index}}) = \begin{cases} [\![\mathbf{B}_1 \mathbf{h}_{\mathsf{GID}_{\mathsf{index}}} + \mathbf{B}_3]\!]_2 & \text{, if } \mathsf{index} \leq j-1 \\ [\![\mathbf{B}_1 \mathbf{h}_{\mathsf{GID}_{\mathsf{index}}} + \mathbf{B}_2 \mathbf{h}'_{\mathsf{GID}_{\mathsf{index}}}]\!]_2 & \text{, if } \mathsf{index} = j \\ [\![\mathbf{B}_1 \mathbf{h}_{\mathsf{GID}_{\mathsf{index}}}]\!]_2 & \text{, if } \mathsf{index} > j \end{cases}.$$

We analyze the three cases separately:

– **Case 1:** $\mathsf{index} < j-1$. $\mathsf{sk}_{i,\mathsf{GID}_{\mathsf{index}}}$ **remains unchanged.** Since $\mathbf{B}_2^{*T}\mathbf{B}_1 = \mathbf{0}, \mathbf{B}_2^{*T}\mathbf{B}_3 = \mathbf{0}$, it follows that

$$\begin{aligned}\mathsf{sk}_{i,\mathsf{GID}_{\mathsf{index}}} &= [\![\mathbf{V}_i\mathbf{k} + \mathbf{U}_i(\mathbf{B}_1\mathbf{h}_{\mathsf{GID}_{\mathsf{index}}} + \mathbf{B}_3)]\!]_2 \\ &= [\![\mathbf{V}_i\mathbf{k} + (\widetilde{\mathbf{U}_i} + \mathbf{A}_2^*\mathbf{u}_i{\delta_0}^T\mathbf{B}_2^{*T})(\mathbf{B}_1\mathbf{h}_{\mathsf{GID}_{\mathsf{index}}} + \mathbf{B}_3)]\!]_2 \\ &= [\![\mathbf{V}_i\mathbf{k} + \widetilde{\mathbf{U}_i}(\mathbf{B}_1\mathbf{h}_{\mathsf{GID}_{\mathsf{index}}} + \mathbf{B}_3)]\!]_2.\end{aligned}$$

– **Case 2:** $\mathsf{index} = j$. $\mathsf{sk}_{i,\mathsf{GID}_{\mathsf{index}}}$ **is simulated with the help of oracle query** $\mathcal{C}.\mathcal{O}_X(\{i\})$**.** Since $\mathbf{B}_2^{*T}\mathbf{B}_1 = \mathbf{0}, \mathbf{B}_2^{*T}\mathbf{B}_2 = \mathbf{I}$, it follows that

$$\begin{aligned}\mathsf{sk}_{i,\mathsf{GID}_{\mathsf{index}}} &= [\![\mathbf{V}_i\mathbf{k} + \mathbf{U}_i(\mathbf{B}_1\mathbf{h}_{\mathsf{GID}_{\mathsf{index}}} + \mathbf{B}_2\mathbf{h}'_{\mathsf{GID}_{\mathsf{index}}})]\!]_2 \\ &= [\![\mathbf{V}_i\mathbf{k} + (\widetilde{\mathbf{U}_i} + \mathbf{A}_2^*\mathbf{u}_i{\delta_0}^T\mathbf{B}_2^{*T})(\mathbf{B}_1\mathbf{h}_{\mathsf{GID}_{\mathsf{index}}} + \mathbf{B}_2\mathbf{h}'_{\mathsf{GID}_{\mathsf{index}}})]\!]_2 \\ &= [\![\mathbf{V}_i\mathbf{k} + \widetilde{\mathbf{U}_i}(\mathbf{B}_1\mathbf{h}_{\mathsf{GID}_{\mathsf{index}}} + \mathbf{B}_2\mathbf{h}'_{\mathsf{GID}_{\mathsf{index}}}) + \mathbf{A}_2^*\mathbf{u}_i{\delta_0}^T\mathbf{h}'_{\mathsf{GID}_{\mathsf{index}}}]\!]_2.\end{aligned}$$

So, to compute $\mathsf{sk}_{i,\mathsf{GID}_{\mathsf{index}}}$, $\mathcal{B}$ needs to know $\mathbf{u}_i$. For this, $\mathcal{B}$ queries the oracle $\mathcal{C}.\mathcal{O}_X(\{i\})$ and obtains $\mathbf{u}_i$ as a response. Using $\mathbf{u}_i$, $\mathcal{B}$ can compute $\mathbf{U}_i$ and using this, it can properly simulate $\mathsf{sk}_{i,\mathsf{GID}_{\mathsf{index}}}$.

– **Case 3:** $\mathsf{index} > j$. $\mathsf{sk}_{i,\mathsf{GID}_{\mathsf{index}}}$ **remains unchanged.** Since $\mathbf{B}_2^{*T}\mathbf{B}_1 = \mathbf{0}$, it follows that

$$\begin{aligned}\mathsf{sk}_{i,\mathsf{GID}_{\mathsf{index}}} &= [\![\mathbf{V}_i\mathbf{k} + \mathbf{U}_i\mathbf{B}_1\mathbf{h}_{\mathsf{GID}_{\mathsf{index}}}]\!]_2 \\ &= [\![\mathbf{V}_i\mathbf{k} + (\widetilde{\mathbf{U}_i} + \mathbf{A}_2^*\mathbf{u}_i{\delta_0}^T\mathbf{B}_2^{*T})\mathbf{B}_1\mathbf{h}_{\mathsf{GID}_{\mathsf{index}}}]\!]_2 \\ &= [\![\mathbf{V}_i\mathbf{k} + \widetilde{\mathbf{U}_i}\mathbf{B}_1\mathbf{h}_{\mathsf{GID}_{\mathsf{index}}}]\!]_2.\end{aligned}$$

- For the challenge ciphertext ct, observe that the only change is in the $c_{3,x}$ component where $x \in \overline{U_{\mathcal{A}}}$. For x such that $\rho(x) \neq 0$, we have:

$$\begin{aligned}c_{3,x} &= [\![\widetilde{\omega_x}]\!]_1 \cdot ([\![\mathbf{c}_x^T]\!]_1 \odot \widetilde{\mathbf{U}_{\rho(x)}}) \cdot ([\![\mu_x + \mathbf{r}_x^T\mathbf{u}_{\rho(x)}]\!]_1 \odot {\delta_0}^T\mathbf{B}_2^{*T}) \\ &= [\![\widetilde{\omega_x} + \mathbf{c}_x^T\widetilde{\mathbf{U}_{\rho(x)}} + (\mu_x + \mathbf{r}_x^T\mathbf{u}_{\rho(x)}){\delta_0}^T\mathbf{B}_2^{*T}]\!]_1 \\ &= [\![(\widetilde{\omega_x} + \mu_x{\delta_0}^T\mathbf{B}_2^{*T}) + \mathbf{c}_x^T\widetilde{\mathbf{U}_{\rho(x)}} + \mathbf{r}_x^T\mathbf{u}_{\rho(x)}{\delta_0}^T\mathbf{B}_2^{*T}]\!]_1 \\ &= [\![(\widetilde{\omega_x} + \mu_x{\delta_0}^T\mathbf{B}_2^{*T}) + \mathbf{c}_x^T\mathbf{U}_{\rho(x)} - \mathbf{c}_x^T\mathbf{A}_2^*\mathbf{u}_{\rho(x)}{\delta_0}^T\mathbf{B}_2^{*T} + \mathbf{r}_x^T\mathbf{u}_{\rho(x)}{\delta_0}^T\mathbf{B}_2^{*T}]\!]_1 \\ &= [\![(\widetilde{\omega_x} + \mu_x{\delta_0}^T\mathbf{B}_2^{*T}) + \mathbf{c}_x^T\mathbf{U}_{\rho(x)} - (\mathbf{A}_1^T\widetilde{\mathbf{c}_x} + \mathbf{A}_2^T\mathbf{r}_x)^T\mathbf{A}_2^*\mathbf{u}_{\rho(x)}{\delta_0}^T\mathbf{B}_2^{*T} + \mathbf{r}_x^T\mathbf{u}_{\rho(x)}{\delta_0}^T\mathbf{B}_2^{*T}]\!]_1 \\ &= [\![(\widetilde{\omega_x} + \mu_x{\delta_0}^T\mathbf{B}_2^{*T}) + \mathbf{c}_x^T\mathbf{U}_{\rho(x)}]\!]_1.\end{aligned}$$

For x such that $\rho(x) = 0$, we have that $\mathbf{u}_{\rho(x)} = \mathbf{u}_0 = \mathbf{0}$. So, $\mathbf{U}_{\rho(x)} = \widetilde{\mathbf{U}_{\rho(x)}}$. Thus, we have:

$$\begin{aligned} c_{3,x} &= [\![\widetilde{\omega_x}]\!]_1 \cdot ([\![\mathbf{c}_x^T]\!]_1 \odot \widetilde{\mathbf{U}_{\rho(x)}}) \cdot ([\![1]\!]_1 \odot \mu_x {\boldsymbol{\delta}_0}^T \mathbf{B}_2^{*T}) \\ &= [\![\widetilde{\omega_x} + \mathbf{c}_x^T \widetilde{\mathbf{U}_{\rho(x)}} + \mu_x {\boldsymbol{\delta}_0}^T \mathbf{B}_2^{*T}]\!]_1 \\ &= [\![(\widetilde{\omega_x} + \mu_x {\boldsymbol{\delta}_0}^T \mathbf{B}_2^{*T}) + \mathbf{c}_x^T \mathbf{U}_{\rho(x)}]\!]_1. \end{aligned}$$

Recall that $\widetilde{\omega_x}$ is a secret share of $(-\mu^{(0)}\mathbf{B}_2^*\boldsymbol{\delta}_0 + \gamma\mathbf{B}_3^*)^T$ and μ_x is a secret share of $\mu^{(\beta)}$. Then, from linearity of the LSSS secret sharing scheme, it follows that $(\widetilde{\omega_x} + \mu_x {\boldsymbol{\delta}_0}^T \mathbf{B}_2^{*T})$ is a secret share of $(-\mu^{(0)}\mathbf{B}_2^*\boldsymbol{\delta}_0 + \gamma\mathbf{B}_3^*)^T + \mu^{(\beta)}{\boldsymbol{\delta}_0}^T\mathbf{B}_2^{*T} = (\mu^{(\beta)} - \mu^{(0)}){\boldsymbol{\delta}_0}^T\mathbf{B}_2^{*T} + \gamma\mathbf{B}_3^{*T}$.

Observe that if $\mathcal{A}$ satisfies the game condition, then $\mathcal{B}$ is admissible in its game with $\mathcal{C}$. Observe that when $\mathcal{C}$ chooses $\beta = 0$, $(\widetilde{\omega_x} + \mu_x \mathbf{B}_3^{*T})$ are secret shares of $\gamma\mathbf{B}_3^{*T}$ and thus $\mathcal{B}$ perfectly simulates $\mathsf{Hyb}_{3,j,1}$. Similarly, when $\mathcal{C}$ chooses $\beta = 1$, $(\widetilde{\omega_x} + \mu_x \mathbf{B}_3^{*T})$ are secret shares of $(\mu^{(\beta)} - \mu^{(0)}){\boldsymbol{\delta}_0}^T\mathbf{B}_2^{*T} + \gamma\mathbf{B}_3^{*T}$ and thus $\mathcal{B}$ perfectly simulates $\mathsf{Hyb}'_{3,j,1}$ where $\boldsymbol{\delta}$ is set implicitly as $\boldsymbol{\delta} = (\mu^{(1)} - \mu^{(0)})\boldsymbol{\delta}_0$. Thus, if $\mathcal{A}$ can distinguish between $\mathsf{Hyb}_{3,j,1}$ and $\mathsf{Hyb}'_{3,j,1}$ non-negligible probability, then $\mathcal{B}$ can break the security of Core 1-ABE with non-negligible probability.

□

Claim 2.5.6. *If the game condition holds and* Core 1-ABE *is secure, then,* $\mathsf{Hyb}_{3,j,2}$ *and* $\mathsf{Hyb}'_{3,j,2}$ *are computationally indistinguishable.*

Proof. Similar to the proof of Claim 2.5.4. □

Claim 2.5.7. *If the game condition holds and* Core 1-ABE *is secure, then,* $\mathsf{Hyb}_{3,q}$ *and* Hyb_4 *are computationally indistinguishable.*

Proof. Observe that the only difference between $\mathsf{Hyb}_{3,q}$ and Hyb_4 is in ciphertext components $c_{2,x}$ for all $x \in [n]$. Observe that $c_{2,x}$ contains $[\![\lambda_x]\!]_1$, where λ_x is a secret share of $\mathbf{t}^T \in \mathbb{Z}_q^{1\times(2k+1)}$ in $\mathsf{Hyb}_{3,q}$, but it is a secret share of $(\mathbf{t} + \tau\mathbf{B}_3^*)^T$ in Hyb_4. Therefore, to prove that the hybrids are computationally indistinguishable, we will reduce the indistinguishability to the security of Core 1-ABE.

Suppose that there exists an adversary $\mathcal{A}$ that can distinguish between $\mathsf{Hyb}_{3,q}$ and Hyb_4 with non-negligible probability. We will construct an adversary $\mathcal{B}$ that can break the security of Core 1-ABE with non-negligible probability. Suppose $\mathcal{C}$ is the challenger for the Core 1-ABE security game. $\mathcal{B}$ will run $\mathcal{A}$ and simulate the hybrid games.

We describe the reduction $\mathcal{B}$ next. $\mathcal{C}$ samples two messages $\mu^{(0)}, \mu^{(1)} \xleftarrow{\$} \mathbb{Z}_q$, samples a challenge bit $\beta \xleftarrow{\$} \{0,1\}$ and provides $\mu^{(0)}$ to $\mathcal{B}$. $\mathcal{B}$ proceeds as in $\mathsf{Hyb}_{3,q}$ except that when it obtains the challenge query

$(\mathsf{msg}_0, \mathsf{msg}_1, (\mathbf{M}, \rho), \{\mathsf{pk}_i\}_{i \in U_\mathcal{A}})$ from $\mathcal{A}$, $\mathcal{B}$ makes oracle query for $\mathcal{C}.\mathcal{O}_F((\mathbf{M}, \rho))$. $\mathcal{C}$ computes secret shares of $\mu^{(\beta)}$ as $\{\mu_x\} \leftarrow \mathsf{LSSS.Share}((\mathbf{M}, \rho), \mu^{(\beta)})$ and responds with $(\{\mu_x\}_{\rho(x)=0}, \{\mathsf{CPA.Enc}(\mathbf{u}_{\rho(x)}, \mu_x)\}_{\rho(x)\neq 0})$, where $\mathsf{CPA.Enc}(\mathbf{u}_{\rho(x)}, \mu_x) = ([\![\mathbf{r}_x]\!]_1, [\![\mu_x + \mathbf{r}_x^T \mathbf{u}_{\rho(x)}]\!]_1)$. The way $\mathcal{B}$ embeds this information obtained from $\mathcal{C}$ in its response to $\mathcal{A}$ is as follows: $\mathcal{B}$ samples $\widetilde{\mathbf{c}_x} \xleftarrow{\$} \mathbb{Z}_q^k$ and sets $[\![\mathbf{c}_x]\!]_1 := [\![\mathbf{A}_1^T \widetilde{\mathbf{c}_x}]\!]_1 \cdot (\mathbf{A}_2^T \odot [\![\mathbf{r}_x]\!]_1)$. $\mathcal{B}$ computes $c_0, c_{1,x}$ as in $\mathsf{Hyb}_{3,q}$. For $c_{2,x}$, $\mathcal{B}$ computes secret shares of $(\mathbf{t} - \mu^{(0)} \mathbf{B}_3^*)^T$ as $\{\widetilde{\lambda_x}\} \leftarrow \mathsf{LSSS.Share}((\mathbf{M}, \rho), (\mathbf{t} - \mu^{(0)} \mathbf{B}_3^*)^T)$ and then computes $c_{2,x}$ as follows:

$$c_{2,x} := \begin{cases} [\![\widetilde{\lambda_x}]\!]_1 \cdot (\mathbf{s}_x^T \odot \mathsf{pk}_{\rho(x),0}) \cdot ([\![\mu_x + \mathbf{r}_x^T \mathbf{u}_{\rho(x)}]\!]_1 \odot \mathbf{B}_3^{*T}) & \text{, if } x \in U_\mathcal{A} \text{ and } \rho(x) \neq 0 \\ [\![\widetilde{\lambda_x}]\!]_1 \cdot (\mathbf{s}_x^T \odot \mathsf{pk}_{\rho(x),0}) \cdot ([\![1]\!]_1 \odot \mu_x \mathbf{B}_3^{*T}) & \text{, if } x \in U_\mathcal{A} \text{ and } \rho(x) = 0 \\ [\![\widetilde{\lambda_x}]\!]_1 \cdot ([\![\mathbf{c}_x^T]\!]_1 \odot \widetilde{\mathbf{V}_{\rho(x)}}) \cdot ([\![\mu_x + \mathbf{r}_x^T \mathbf{u}_{\rho(x)}]\!]_1 \odot \mathbf{B}_3^{*T}) & \text{, if } x \in \overline{U_\mathcal{A}} \text{ and } \rho(x) \neq 0 \\ [\![\widetilde{\lambda_x}]\!]_1 \cdot ([\![\mathbf{c}_x^T]\!]_1 \odot \widetilde{\mathbf{V}_{\rho(x)}}) \cdot ([\![1]\!]_1 \odot \mu_x \mathbf{B}_3^{*T}) & \text{, if } x \in \overline{U_\mathcal{A}} \text{ and } \rho(x) = 0 \end{cases},$$

where for oracle $\mathsf{AuthSetup}(i)$, $\mathcal{B}$ samples master secret key as $\mathsf{msk}_i = (\widetilde{\mathbf{V}}_i \xleftarrow{\$} \mathbb{Z}_q^{(2k)\times(2k+1)}, \widetilde{\mathbf{U}}_i \xleftarrow{\$} \mathbb{Z}_q^{(2k)\times(2k+1)})$. Since the vector $\mathbf{k}$ in glpbal parameters gp is uniform random, $\mathcal{B}$ can equivalently compute it as $\mathbf{k} := \mathbf{B}_1 \mathbf{k}_1 + \mathbf{B}_2 \mathbf{k}_2 + k_3 \mathbf{B}_3$ for uniform random $\mathbf{k}_1 \xleftarrow{\$} \mathbb{Z}_q^k$, $\mathbf{k}_2 \xleftarrow{\$} \mathbb{Z}_q^k$, $k_3 \xleftarrow{\$} \mathbb{Z}_q$. For $c_{3,x}$, $\mathcal{B}$ computes secret shares of $\gamma' \mathbf{B}_3^{*T}$ as $\{\widetilde{\omega_x}\} \leftarrow \mathsf{LSSS.Share}((\mathbf{M}, \rho), \gamma' \mathbf{B}_3^{*T})$, where $\gamma' \xleftarrow{\$} \mathbb{Z}_q$ and then computes $c_{3,x}$ as follows:

$$c_{3,x} := \begin{cases} [\![\widetilde{\omega_x}]\!]_1 \cdot (\mathbf{s}_x^T \odot \mathsf{pk}_{\rho(x),1}) / ([\![\mu_x + \mathbf{r}_x^T \mathbf{u}_{\rho(x)}]\!]_1 \odot k_3 \mathbf{B}_3^{*T}) & \text{, if } x \in U_\mathcal{A} \text{ and } \rho(x) \neq 0 \\ [\![\widetilde{\omega_x}]\!]_1 \cdot (\mathbf{s}_x^T \odot \mathsf{pk}_{\rho(x),1}) / ([\![1]\!]_1 \odot \mu_x k_3 \mathbf{B}_3^{*T}) & \text{, if } x \in U_\mathcal{A} \text{ and } \rho(x) = 0 \\ [\![\widetilde{\omega_x}]\!]_1 \cdot ([\![\mathbf{c}_x^T]\!]_1 \odot \widetilde{\mathbf{U}_{\rho(x)}}) / ([\![\mu_x + \mathbf{r}_x^T \mathbf{u}_{\rho(x)}]\!]_1 \odot k_3 \mathbf{B}_3^{*T}) & \text{, if } x \in \overline{U_\mathcal{A}} \text{ and } \rho(x) \neq 0 \\ [\![\widetilde{\omega_x}]\!]_1 \cdot ([\![\mathbf{c}_x^T]\!]_1 \odot \widetilde{\mathbf{U}_{\rho(x)}}) / ([\![1]\!]_1 \odot \mu_x k_3 \mathbf{B}_3^{*T}) & \text{, if } x \in \overline{U_\mathcal{A}} \text{ and } \rho(x) = 0 \end{cases},$$

$\mathcal{B}$ programs oracles $\mathsf{H}_1(\mathsf{GID})$, $\mathsf{KGen}(i, \mathsf{GID})$, $\mathsf{Corrupt}(i)$ as in $\mathsf{Hyb}_{3,q}$. $\mathcal{B}$ implicitly sets

$$\mathbf{V}_i = \widetilde{\mathbf{V}_i} + \mathbf{A}_2^* \mathbf{u}_i \mathbf{B}_3^{*T},\ \mathbf{U}_i = \widetilde{\mathbf{U}_i} - k_3 \mathbf{A}_2^* \mathbf{u}_i \mathbf{B}_3^{*T} \text{ ($\mathcal{B}$ does not know } \mathbf{u}_i \text{ that is chosen by } \mathcal{C}).$$

Observe that when we change $(\widetilde{\mathbf{V}_i}, \widetilde{\mathbf{U}_i})$ to $(\mathbf{V}_i, \mathbf{U}_i)$, only pk_i, $\mathsf{sk}_{i,\mathsf{GID}}$ and ct are changed. We anaylze the effect of this change next.

- pk_i **remains unchanged:** Since $\mathbf{A}_1 \mathbf{A}_2^* = \mathbf{0}$, it follows that $\mathsf{pk}_{i,0} = [\![\mathbf{A}_1 \mathbf{V}_i]\!]_1 = [\![\mathbf{A}_1 \widetilde{\mathbf{V}_i} + \mathbf{A}_1 \mathbf{A}_2^* \mathbf{u}_i \mathbf{B}_3^{*T}]\!]_1 = [\![\mathbf{A}_1 \widetilde{\mathbf{V}_i}]\!]_1$. Similarly, $\mathsf{pk}_{i,1} = [\![\mathbf{A}_1 \mathbf{U}_i]\!]_1 = [\![\mathbf{A}_1 \widetilde{\mathbf{U}_i} - k_3 \mathbf{A}_1 \mathbf{A}_2^* \mathbf{u}_i \mathbf{B}_3^{*T}]\!]_1 = [\![\mathbf{A}_1 \widetilde{\mathbf{U}_i}]\!]_1$.

- $\mathsf{sk}_{i,\mathsf{GID}}$ **remains unchanged for all** GID**:** Since $\mathbf{B}_3^{*T}\mathbf{B}_1 = \mathbf{0}, \mathbf{B}_3^{*T}\mathbf{B}_2 = \mathbf{0}, \mathbf{B}_3^{*T}\mathbf{B}_3 = \mathbf{I}$, it follows that

$$\begin{aligned}
\mathsf{sk}_{i,\mathsf{GID}} &= [\![\mathbf{V}_i\mathbf{k} + \mathbf{U}_i(\mathbf{B}_1\mathbf{h}_{\mathsf{GID}} + \mathbf{B}_3)]\!]_2 \\
&= [\![(\widetilde{\mathbf{V}_i} + \mathbf{A}_2^*\mathbf{u}_i\mathbf{B}_3^{*T})\mathbf{k} + (\widetilde{\mathbf{U}_i} - k_3\mathbf{A}_2^*\mathbf{u}_i\mathbf{B}_3^{*T})(\mathbf{B}_1\mathbf{h}_{\mathsf{GID}} + \mathbf{B}_3)]\!]_2 \\
&= [\![\widetilde{\mathbf{V}_i}\mathbf{k} + \mathbf{A}_2^*\mathbf{u}_i\mathbf{B}_3^{*T}\mathbf{k} + \widetilde{\mathbf{U}_i}(\mathbf{B}_1\mathbf{h}_{\mathsf{GID}} + \mathbf{B}_3) - k_3\mathbf{A}_2^*\mathbf{u}_i]\!]_2 \\
&= [\![\widetilde{\mathbf{V}_i}\mathbf{k} + \mathbf{A}_2^*\mathbf{u}_i\mathbf{B}_3^{*T}(\mathbf{B}_1\mathbf{k}_1 + \mathbf{B}_2\mathbf{k}_2 + k_3\mathbf{B}_3) + \widetilde{\mathbf{U}_i}(\mathbf{B}_1\mathbf{h}_{\mathsf{GID}} + \mathbf{B}_3) - k_3\mathbf{A}_2^*\mathbf{u}_i]\!]_2 \\
&= [\![\widetilde{\mathbf{V}_i}\mathbf{k} + \widetilde{\mathbf{U}_i}(\mathbf{B}_1\mathbf{h}_{\mathsf{GID}} + \mathbf{B}_3)]\!]_2.
\end{aligned}$$

- For the challenge ciphertext ct, observe that the only change is in the $c_{2,x}, c_{3,x}$ components where $x \in \overline{U_{\mathcal{A}}}$. For x such that $\rho(x) \neq 0$, we have:

$$\begin{aligned}
c_{2,x} &= [\![\widetilde{\lambda_x}]\!]_1 \cdot ([\![\mathbf{c}_x^T]\!]_1 \odot \widetilde{\mathbf{V}_{\rho(x)}}) \cdot ([\![\mu_x + \mathbf{r}_x^T\mathbf{u}_{\rho(x)}]\!]_1 \odot \mathbf{B}_3^{*T}) \\
&= [\![\widetilde{\lambda_x} + \mathbf{c}_x^T\widetilde{\mathbf{V}_{\rho(x)}} + (\mu_x + \mathbf{r}_x^T\mathbf{u}_{\rho(x)})\mathbf{B}_3^{*T}]\!]_1 \\
&= [\![(\widetilde{\lambda_x} + \mu_x\mathbf{B}_3^{*T}) + \mathbf{c}_x^T\widetilde{\mathbf{V}_{\rho(x)}} + \mathbf{r}_x^T\mathbf{u}_{\rho(x)}\mathbf{B}_3^{*T}]\!]_1 \\
&= [\![(\widetilde{\lambda_x} + \mu_x\mathbf{B}_3^{*T}) + \mathbf{c}_x^T\mathbf{V}_{\rho(x)} - \mathbf{c}_x^T\mathbf{A}_2^*\mathbf{u}_{\rho(x)}\mathbf{B}_3^{*T} + \mathbf{r}_x^T\mathbf{u}_{\rho(x)}\mathbf{B}_3^{*T}]\!]_1 \\
&= [\![(\widetilde{\lambda_x} + \mu_x\mathbf{B}_3^{*T}) + \mathbf{c}_x^T\mathbf{V}_{\rho(x)} - (\mathbf{A}_1^T\widetilde{\mathbf{c}_x} + \mathbf{A}_2^T\mathbf{r}_x)^T\mathbf{A}_2^*\mathbf{u}_{\rho(x)}\mathbf{B}_3^{*T} + \mathbf{r}_x^T\mathbf{u}_{\rho(x)}\mathbf{B}_3^{*T}]\!]_1 \\
&= [\![(\widetilde{\lambda_x} + \mu_x\mathbf{B}_3^{*T}) + \mathbf{c}_x^T\mathbf{V}_{\rho(x)}]\!]_1,
\end{aligned}$$

$$\begin{aligned}
c_{3,x} &= [\![\widetilde{\omega_x}]\!]_1 \cdot ([\![\mathbf{c}_x^T]\!]_1 \odot \widetilde{\mathbf{U}_{\rho(x)}}) / ([\![\mu_x + \mathbf{r}_x^T\mathbf{u}_{\rho(x)}]\!]_1 \odot k_3\mathbf{B}_3^{*T}) \\
&= [\![\widetilde{\omega_x} + \mathbf{c}_x^T\widetilde{\mathbf{U}_{\rho(x)}} - (\mu_x + \mathbf{r}_x^T\mathbf{u}_{\rho(x)})k_3\mathbf{B}_3^{*T}]\!]_1 \\
&= [\![(\widetilde{\omega_x} - \mu_x k_3\mathbf{B}_3^{*T}) + \mathbf{c}_x^T\widetilde{\mathbf{U}_{\rho(x)}} - k_3\mathbf{r}_x^T\mathbf{u}_{\rho(x)}\mathbf{B}_3^{*T}]\!]_1 \\
&= [\![(\widetilde{\omega_x} - \mu_x k_3\mathbf{B}_3^{*T}) + \mathbf{c}_x^T\mathbf{U}_{\rho(x)} + k_3\mathbf{c}_x^T\mathbf{A}_2^*\mathbf{u}_{\rho(x)}\mathbf{B}_3^{*T} - k_3\mathbf{r}_x^T\mathbf{u}_{\rho(x)}\mathbf{B}_3^{*T}]\!]_1 \\
&= [\![(\widetilde{\omega_x} - \mu_x k_3\mathbf{B}_3^{*T}) + \mathbf{c}_x^T\mathbf{U}_{\rho(x)} + k_3(\mathbf{A}_1^T\widetilde{\mathbf{c}_x} + \mathbf{A}_2^T\mathbf{r}_x)^T\mathbf{A}_2^*\mathbf{u}_{\rho(x)}\mathbf{B}_3^{*T} - k_3\mathbf{r}_x^T\mathbf{u}_{\rho(x)}\mathbf{B}_3^{*T}]\!]_1 \\
&= [\![(\widetilde{\omega_x} - \mu_x k_3\mathbf{B}_3^{*T}) + \mathbf{c}_x^T\mathbf{U}_{\rho(x)}]\!]_1.
\end{aligned}$$

  For x such that $\rho(x) = 0$, we have that $\mathbf{u}_{\rho(x)} = \mathbf{u}_0 = \mathbf{0}$. So, $\mathbf{V}_{\rho(x)} = \widetilde{\mathbf{V}_{\rho(x)}}$ and $\mathbf{U}_{\rho(x)} = \widetilde{\mathbf{U}_{\rho(x)}}$. Thus, we have:

$$\begin{aligned}
c_{2,x} &= [\![\widetilde{\lambda_x}]\!]_1 \cdot ([\![\mathbf{c}_x^T]\!]_1 \odot \widetilde{\mathbf{V}_{\rho(x)}}) \cdot ([\![1]\!]_1 \odot \mu_x\mathbf{B}_3^{*T}) \\
&= [\![\widetilde{\lambda_x} + \mathbf{c}_x^T\widetilde{\mathbf{V}_{\rho(x)}} + \mu_x\mathbf{B}_3^{*T}]\!]_1 \\
&= [\![(\widetilde{\lambda_x} + \mu_x\mathbf{B}_3^{*T}) + \mathbf{c}_x^T\mathbf{V}_{\rho(x)}]\!]_1.
\end{aligned}$$

$$
\begin{aligned}
c_{3,x} &= [\![\widetilde{\omega_x}]\!]_1 \cdot ([\![\mathbf{c}_x^T]\!]_1 \odot \widetilde{\mathbf{U}_{\rho(x)}}) / ([\![1]\!]_1 \odot \mu_x k_3 \mathbf{B}_3^{*T}) \\
&= [\![\widetilde{\omega_x} + \mathbf{c}_x^T \widetilde{\mathbf{U}_{\rho(x)}} - \mu_x k_3 \mathbf{B}_3^{*T}]\!]_1 \\
&= [\![(\widetilde{\omega_x} - \mu_x k_3 \mathbf{B}_3^{*T}) + \mathbf{c}_x^T \mathbf{U}_{\rho(x)}]\!]_1
\end{aligned}
$$

Recall that $\widetilde{\lambda_x}$ is a secret share of $(\mathbf{t} - \mu^{(0)}\mathbf{B}_3^*)^T$ and μ_x is a secret share of $\mu^{(\beta)}$. Then, from linearity of the LSSS secret sharing scheme, it follows that $(\widetilde{\lambda_x} + \mu_x \mathbf{B}_3^{*T})$ is a secret share of $(\mathbf{t} - \mu^{(0)}\mathbf{B}_3^*)^T + \mu^{(\beta)}\mathbf{B}_3^{*T} = \mathbf{t}^T + (\mu^{(\beta)} - \mu^{(0)})\mathbf{B}_3^{*T}$. Recall that $\widetilde{\omega_x}$ is a secret share of $\gamma' \mathbf{B}_3^{*T}$ and μ_x is a secret share of $\mu^{(\beta)}$. Then, from linearity of the LSSS secret sharing scheme, it follows that $(\widetilde{\omega_x} + \mu_x k_3 \mathbf{B}_3^{*T})$ is a secret share of $\gamma' \mathbf{B}_3^{*T} + \mu^{(\beta)} k_3 \mathbf{B}_3^{*T} = (\gamma' + k_3 \mu^{(\beta)})\mathbf{B}_3^{*T}$.

Observe that if $\mathcal{A}$ satisfies the game condition, then $\mathcal{B}$ is admissible in its game with $\mathcal{C}$. $\mathcal{B}$ implicitly sets $\gamma = \gamma' + k_3 \mu^{(\beta)}$ in $\mathcal{A}$'s view. Since γ' and k_3 are chosen to be uniform random, the distribution of γ is also uniform random for both $\beta \in \{0,1\}$. Observe that when $\mathcal{C}$ chooses $\beta = 0$, $(\widetilde{\lambda_x} + \mu_x \mathbf{B}_3^{*T})$ are secret shares of $\mathbf{t}^T$ and thus $\mathcal{B}$ perfectly simulates $\mathsf{Hyb}_{3,q}$. Similarly, when $\mathcal{C}$ chooses $\beta = 1$, $(\widetilde{\lambda_x} + \mu_x \mathbf{B}_3^{*T})$ are secret shares of $\mathbf{t}^T + (\mu^{(1)} - \mu^{(0)})\mathbf{B}_3^{*T}$ and thus $\mathcal{B}$ perfectly simulates Hyb_4 where τ is set implicitly as $\tau = \mu^{(1)} - \mu^{(0)}$. Thus, if $\mathcal{A}$ can distinguish between $\mathsf{Hyb}_{3,q}$ and Hyb_4 non-negligible probability, then $\mathcal{B}$ can break the security of Core 1-ABE with non-negligible probability.

□

Thus, from Claims 2.4.2 to 2.4.4, 2.4.6, 2.4.8, 2.4.10, 2.4.12, 2.4.13 and 2.5.4 to 2.5.7 and hybrid argument, it follows that Theorem 2.5.3 holds.

2.6 MA-ABE for ASP from prime-order groups

In this section, we present two MA-ABE constructions for arithmetic span programs (ASPs). Looking ahead, there seem to be fundamental barriers towards building MA-ABE for ASPs satisfying full adaptive security as discussed in Remark 2.6.1 in Section 2.6.1. Thus, we consider two weakenings of full adaptive security called Type 1 and Type 2 restrictions defined in Definitions 2.6.2 and 2.6.3. The first construction (Section 2.6.2) generalizes the MA-ABE construction for monotone BSPs presented in Figure 2.2 and satisfies full adaptive security with Type 1 restriction. The second construction (Section 2.6.4) is a compiler that transforms any MA-ABE scheme for monotone BSPs satisfying full adaptive security with Type 1 restriction into an MA-ABE scheme for ASPs satisfying full adaptive security with Type 2 restriction.

2.6.1 Definition

In case of MA-ABE for ASPs, each attribute will not only be associated with an authority, but also denotes a value in $\mathbb{Z}_q$. So, we modify the definition of MA-ABE presented in Section 2.3.6 to incorporate this generalization. We note that this modification is just syntactic and natural when considering MA-ABE for ASPs.

Let $\mathcal{AU}$ denote the authority/attribute universe and let each authority control one attribute. We use authority and attribute interchangably. Suppose that an ASP is denoted by a $(\mathbf{M}, \mathbf{N}, \rho)$ of matrices $\mathbf{M}, \mathbf{N} \in \mathbb{Z}_q^{n \times \ell}$ and a labeling function $\rho : [n] \to U$, where $U \subseteq \mathcal{AU}$ denotes a subset of the attribute universe.

The syntax of KGen algorithm is then as follows.

- $\mathsf{sk}_{i,\mathsf{GID},z_i} \leftarrow \mathsf{KGen}(\mathsf{gp}, \mathsf{msk}_i, \mathsf{GID}, z_i)$: The key generation algorithm takes as input the global parameters gp, a master secret key msk_i of an authority $i \in \mathcal{AU}$ a user's global identifier $\mathsf{GID} \in \mathcal{GID}$, and an attribute value $z_i \in \mathbb{Z}_q$. It outputs a secret key $\mathsf{sk}_{i,\mathsf{GID},z_i}$ for the user.

Correctness. An MA-ABE scheme for ASP is said to be correct if for every $\lambda \in \mathbb{N}$, $\mathsf{msg} \in \mathbb{M}$, $\mathsf{GID} \in \mathcal{GID}$, every ASP $(\mathbf{M}, \mathbf{N}, \rho)$ defined on a set $U \subseteq \mathcal{AU}$ of attributes, and every set of attribute values $S = \{z_{\rho(x)}\}_{x \in S_x}$ (where $S_x \subseteq [n]$ denotes a subset of row indices) which satisfy the access structure (i.e., $(1, 0, \ldots, 0) \in span(\{z_{\rho(x)}\mathbf{M}_x + \mathbf{N}_x\}_{x \in S_x})$), it holds that

$$\Pr\left[\begin{array}{c} \mathsf{gp} \leftarrow \mathsf{GlobalSetup}(1^\lambda), \\ \forall i \in U : (\mathsf{pk}_i, \mathsf{msk}_i) \leftarrow \mathsf{AuthSetup}(\mathsf{gp}, i) \\ \mathsf{msg}' = \mathsf{msg} : \forall x \in S_x : \mathsf{sk}_{\rho(x),\mathsf{GID},z_{\rho(x)}} \leftarrow \mathsf{KGen}(\mathsf{gp}, \mathsf{msk}_{\rho(x)}, \mathsf{GID}, z_{\rho(x)}) \\ \mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{gp}, \mathsf{msg}, \mathbb{A}, \{\mathsf{pk}_i\}_{i \in U}) \\ \mathsf{msg}' \leftarrow \mathsf{Dec}(\mathsf{gp}, \mathsf{ct}, \{z_{\rho(x)}, \mathsf{sk}_{\rho(x),\mathsf{GID},z_{\rho(x)}}\}_{x \in S_x}) \end{array}\right] = 1.$$

Full Adaptive Security. The security game remains the same as in Section 2.3.6 except that for keygen oracle query $\mathsf{KGen}(i, \mathsf{GID}, z_i)$, the reponse is $\mathsf{sk}_{i,\mathsf{GID},z_i} \leftarrow \mathsf{KGen}(\mathsf{gp}, \mathsf{msk}_i, \mathsf{GID}, z_i)$.

Remark 2.6.1. *Building MA-ABE for ASP satisfying full adaptive security seems to have fundamental barriers as pointed out in Section 4.1. In a little more detail, the fundamental barrier seems that if the challenge policy $(\mathbf{M}, \mathbf{N}, \rho)$ involves a corrupt authority controlling an attribute corresponding to some x-th rows $\mathbf{M}_x, \mathbf{N}_x$, then it can issue more than one attribute values $z_{\rho(x)}$ and $z'_{\rho(x)}$ that are not equal. Then, observe that the span of $z_{\rho(x)}\mathbf{M}_x + \mathbf{N}_x$ and $z'_{\rho(x)}\mathbf{M}_x + \mathbf{N}_x$ contains the vector $\mathbf{M}_x$. Then consider a scenarios where $(1, 0, \ldots, 0)$ is not in span of either of these two vectors and hence each secret key on its own would be insufficient to decrypt, but suppose that $(1, 0, \ldots, 0)$ is in*

span of $\mathbf{M}_x$. Then, collusion of the two keys should allow to decrypt. This is a fundamental barrier towards building MA-ABE for ASPs with full adaptive security.

In light of Remark 2.6.1, we consider two weakenings of full adaptive security defined as follows.

Definition 2.6.2 (Full Adaptive Security with Type 1 Restriction)**.** *An MA-ABE scheme for ASPs is said to be full adaptive secure with type 1 restriction if it satisfies full adaptive security subject to following additional admissibility criteria:*

- *no attribute authority appearing in a ciphertext is corrupted (that is, set S defined in Section 2.3.6 and Figure 2.1 is empty),*
- *the adversary queries at most one key per authority and user id pair (i, GID).*

Suppose the challenge access policy $(\mathbf{M}, \mathbf{N}, \rho)$ is defined over a set of attributes $U \subseteq \mathcal{AU}$, that is, $\rho : [n] \to U$. Then the game condition from Section 2.3.6 combined with the additional admissibility criteria described above simplifies to the following requirements:

- *for each $\mathsf{GID} \in \mathcal{GID}$, it is required that $S_{\mathsf{GID}} \notin (\mathbf{M}, \mathbf{N}, \rho)$,*
- *for each $\mathsf{GID} \in \mathcal{GID}$, for each $i \in S_{\mathsf{GID}}$, it is required that $|S_{i,\mathsf{GID}}| = 1$,*

where S_{GID} and $S_{i,\mathsf{GID}}$ are defined as follows:

- *for each global identifier $\mathsf{GID} \in \mathcal{GID}$, S_{GID} denotes the subset of U containing attributes i such that the adversary queried a secret key for the tuple $(i, \mathsf{GID}, \cdot)$.*
- *for each global identifier $\mathsf{GID} \in \mathcal{GID}$, for each attribute $i \in S_{\mathsf{GID}}$, $S_{i,\mathsf{GID}}$ denotes the set of attribute values z_i such that the adversary queried a secret key for the tuple (i, GID, z_i).*

We remark that the adversary is still allowed to make corrupt authorities adaptively subject to the above additional admissibility criteria.

Definition 2.6.3 (Full Adaptive Security with Type 2 Restriction)**.** *An MA-ABE scheme for ASPs is said to be full adaptive secure with type 2 restriction if it satisfies full adaptive security subject to following additional admissibility criteria:*

- *decryption requires keys from all authorities appearing in a ciphertext,*
- *either the adversary corrupts no authority appearing in the challenge ciphertext policy or for each GID queried, there exists an honest authority appearing in the challenge ciphertext policy who did not issue any secret key,*
- *the adversary queries at most one key per authority and user id pair (i, GID).*

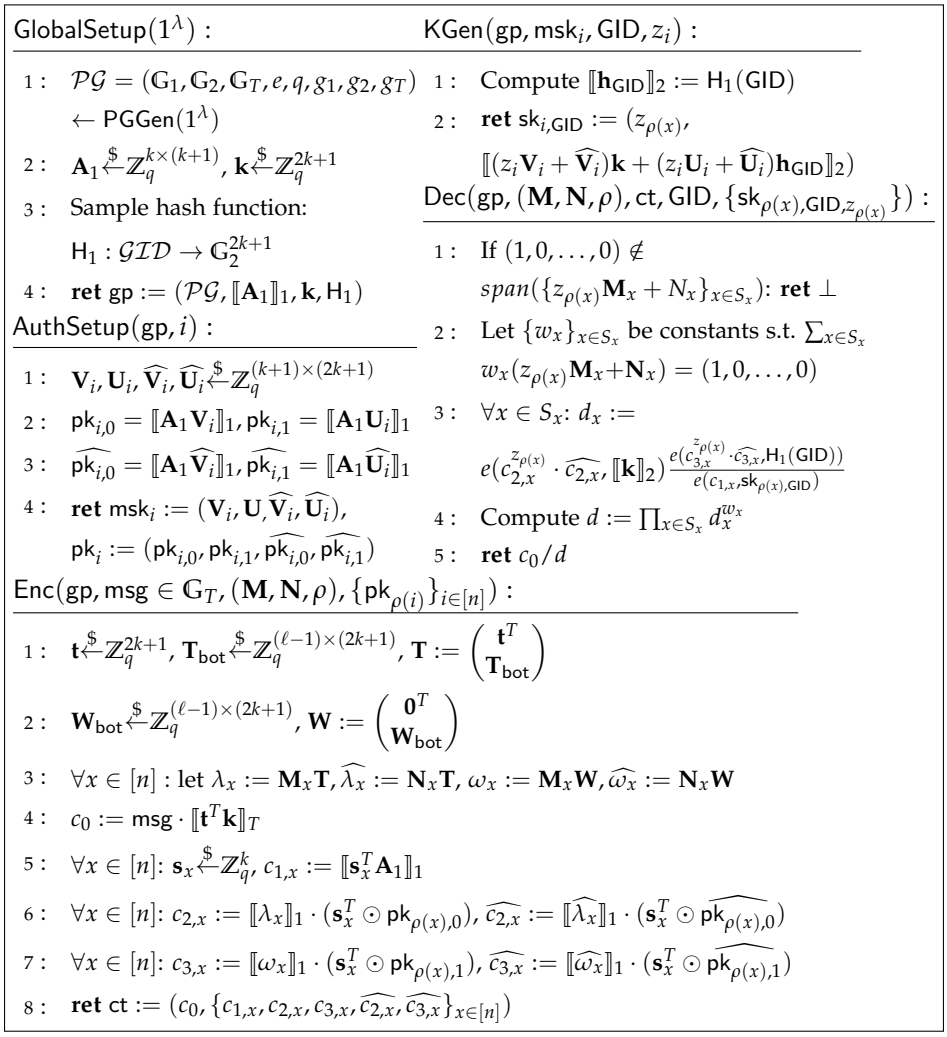


Figure 2.4: Construction: MA-ABE scheme for ASP from prime-order groups

2.6.2 Construction

Let $\mathcal{AU}$ denote the authority universe and let each authority control an attribute that can be assigned values in $\mathbb{Z}_q$. Let $\mathcal{GID}$ denote the universe of global identifiers of the users. We construct MA-ABE for access policies specified by an arithmetic span program (ASP) denoted by a tuple $(\mathbf{M}, \mathbf{N}, \rho)$ of matrices $\mathbf{M}, \mathbf{N} \in \mathbb{Z}_q^{n\times\ell}$ and a labeling function $\rho : [n] \to U$, where $U \subseteq \mathcal{AU}$ denotes a subset of attributes. Our construction is as in Figure 2.4. We prove full adaptive security with Type 1 restriction of the MA-ABE scheme for ASP presented in Figure 2.4 subject to the one-use restriction, that is, ρ is injective. We prove Theorem 2.6.4 in Section 2.6.3.

Correctness. For the MA-ABE for ASP construction in Figure 2.4, observe that d_x can be simplified as follows:

$$\begin{aligned}
d_x &= e(c_{2,x}^{z_{\rho(x)}} \cdot \widehat{c_{2,x}}, [\![\mathbf{k}]\!]_2) \cdot \frac{e(c_{3,x}^{z_{\rho(x)}} \cdot \widehat{c_{3,x}}, \mathsf{H}_1(\mathsf{GID}))}{e(c_{1,x}, \mathsf{sk}_{\rho(x),\mathsf{GID},z_{\rho(x)}})} \\
&= [\![((z_{\rho(x)}\lambda_x + \widehat{\lambda_x}) + \mathbf{s}_x^T\mathbf{A}(z_{\rho(x)}\mathbf{V}_{\rho(x)} + \widehat{\mathbf{V}_{\rho(x)}}))\mathbf{k}]\!]_T \\
&\quad \cdot \frac{[\![((z_{\rho(x)}\omega_x + \widehat{\omega_x}) + \mathbf{s}_x^T\mathbf{A}(z_{\rho(x)}\mathbf{U}_{\rho(x)} + \widehat{\mathbf{U}_{\rho(x)}}))\mathbf{h}_{\mathsf{GID}}]\!]_T}{[\![(\mathbf{s}_x^T\mathbf{A})((z_{\rho(x)}\mathbf{V}_{\rho(x)} + \widehat{\mathbf{V}_{\rho(x)}})\mathbf{k} + (z_{\rho(x)}\mathbf{U}_{\rho(x)} + \widehat{\mathbf{U}_{\rho(x)}})\mathbf{h}_{\mathsf{GID}})]\!]_T} \\
&= [\![(z_{\rho(x)}\lambda_x + \widehat{\lambda_x})\mathbf{k} + (z_{\rho(x)}\omega_x + \widehat{\omega_x})\mathbf{h}_{\mathsf{GID}}]\!]_T \\
&= [\![(z_{\rho(x)}\mathbf{M}_x + \mathbf{N}_x)(\mathbf{T}\mathbf{k} + \mathbf{W}\mathbf{h}_{\mathsf{GID}})]\!]_T.
\end{aligned}$$

Then, d can be simplified as follows:

$$\begin{aligned}
d &= \prod_{x \in S_x} d_x^{w_x} \\
&= [\![\sum_{x \in S_x} w_x(z_{\rho(x)}\mathbf{M}_x + \mathbf{N}_x)(\mathbf{T}\mathbf{k} + \mathbf{W}\mathbf{h}_{\mathsf{GID}})]\!]_T \\
&= [\![(1, 0, \ldots, 0)(\mathbf{T}\mathbf{k} + \mathbf{W}\mathbf{h}_{\mathsf{GID}})]\!]_T \\
&= [\![\mathbf{t}^T\mathbf{k} + \mathbf{0}^T\mathbf{h}_{\mathsf{GID}}]\!]_T \qquad \text{(because first rows of } \mathbf{T} \text{ and } \mathbf{W} \text{ are } \mathbf{t}^T \text{ and } \mathbf{0}^T\text{)} \\
&= [\![\mathbf{t}^T\mathbf{k}]\!]_T.
\end{aligned}$$

Thus, it follows that $c_0/d = \mathsf{msg}$. Thus, correctness holds.

Theorem 2.6.4. *The MA-ABE construction for ASP in Figure 2.4 is fully adaptively secure with Type 1 restriction (Definition 2.6.2) if all of the following hold true.*

- *game condition holds and ρ is injective.*
- *k-MDDH assumption holds in groups $\mathbb{G}_1$ and $\mathbb{G}_2$ (Assumption 2.3.8).*
- $\mathsf{SD}^{\mathbb{G}_2}_{\mathbf{B}_1 \to \mathbf{B}_1,\mathbf{B}_2}$ *assumption holds (Assumption 2.3.9).*
- $\mathsf{SD}^{\mathbb{G}_2}_{\mathbf{B}_2 \to \mathbf{B}_2,\mathbf{B}_3}$ *assumption holds (Assumption 2.3.9).*

2.6.3 Proof of Theorem 2.6.4

We prove this via a sequence of hybrid games. This sequence of hybrids closely follows those for the monotone BSP case presented in Theorem 2.4.1 and Section 2.4.2.

Suppose the adversary makes q number of queries to the random oracle H_1. Then, the hybrid games are as follows:

$$\mathsf{Hyb}_{\mathsf{Real}}, \mathsf{Hyb}'_{\mathsf{Real}}, \mathsf{Hyb}_1, \mathsf{Hyb}_2, \{\mathsf{Hyb}_{3,j,1}, \mathsf{Hyb}'_{3,j,1}, \mathsf{Hyb}_{3,j,2}, \mathsf{Hyb}'_{3,j,2}, \mathsf{Hyb}_{3,j,3}\}_{j\in[q]}, \mathsf{Hyb}_4, \mathsf{Hyb}_5.$$

Hybrid $\mathsf{Hyb}_{\mathsf{Real}}$**.** This is the real-world game $\mathsf{MA\text{-}ABE}_{\mathcal{A}}^{\mathsf{fully\text{-}adaptive}}$ with the additional admissibility criteria for full adaptive security with Type 1 restriction as defined in Definition 2.6.2.

Hybrid $\mathsf{Hyb}'_{\mathsf{Real}}$**.** This is $\mathsf{Hyb}_{\mathsf{Real}}$ except that the challenger computes $\boxed{(\mathsf{gp}, \mathsf{st}) \leftarrow \mathsf{GlobalSetup}^*(1^\lambda)}$ and provides gp to the adversary. Here, $\mathsf{GlobalSetup}^*$ runs the same computation as GlobalSetup to compute gp and additionally also computes the following:

$$\mathbf{A}_2 \xleftarrow{\$} \mathbb{Z}_q^{1\times(k+1)}, \mathbf{B}_1, \mathbf{B}_2 \xleftarrow{\$} \mathbb{Z}_q^{(2k+1)\times k}, \mathbf{B}_3 \xleftarrow{\$} \mathbb{Z}_q^{(2k+1)\times 1}$$

$$\left(\mathbf{A}_1^*, \mathbf{A}_2^*\right) = \begin{pmatrix}\mathbf{A}_1\\ \mathbf{A}_2\end{pmatrix}^{-1}, \left(\mathbf{B}_1^*, \mathbf{B}_2^*, \mathbf{B}_3^*\right) = \left(\left(\mathbf{B}_1, \mathbf{B}_2, \mathbf{B}_3\right)^{-1}\right)^T.$$

Let $\mathbf{A} = \begin{pmatrix}\mathbf{A}_1\\ \mathbf{A}_2\end{pmatrix}$, $\mathbf{A}^* = \left(\mathbf{A}_1^*, \mathbf{A}_2^*\right)$, $\mathbf{B} = \left(\mathbf{B}_1, \mathbf{B}_2, \mathbf{B}_3\right)$, $\mathbf{B}^* = \left(\mathbf{B}_1^*, \mathbf{B}_2^*, \mathbf{B}_3^*\right)$. Then, $\mathsf{st} = (\mathbf{A}, \mathbf{A}^*, \mathbf{B}, \mathbf{B}^*)$.

Observe that $\forall i, j \in \{1,2\}$: $\mathbf{A}_i\mathbf{A}_j^* = \mathbf{I}$ if $i = j$, and $\mathbf{0}$ if $i \neq j$. Similarly, $\forall i, j \in \{1,2,3\}$: $\mathbf{B}_i^T\mathbf{B}_j^* = \mathbf{I}$ if $i = j$, and $\mathbf{0}$ if $i \neq j$.

Claim 2.6.5. *Hybrids* $\mathsf{Hyb}_{\mathsf{Real}}$ *and* $\mathsf{Hyb}'_{\mathsf{Real}}$ *are identically distributed.*

Proof. $\mathsf{Hyb}_{\mathsf{Real}}$ and $\mathsf{Hyb}'_{\mathsf{Real}}$ are identically distributed because the distribution of gp in GlobalSetup and $\mathsf{GlobalSetup}^*$ is the same. □

Hybrid Hyb_0**.** This is same as $\mathsf{Hyb}_{\mathsf{Real}}$ except that the hash function H_1 is programmed to output all hash values in $\mathsf{span}(\mathbf{B}_1)$ as follows: on input GID, sample $\mathbf{h}_{\mathsf{GID}} \xleftarrow{\$} \boxed{\mathbb{Z}_q^k}$ and output $\mathsf{H}_1(\mathsf{GID}) = [\![\boxed{\mathbf{B}_1\mathbf{h}_{\mathsf{GID}}}]\!]_2$.

Claim 2.6.6. *If k-MDDH holds in group* $\mathbb{G}_2$*, then* $\mathsf{Hyb}'_{\mathsf{Real}}$ *and* Hyb_0 *are computationally indistinguishable.*

Proof. Similar to proof of Claim 2.4.3. □

Hybrid Hyb_1**.** This is same as $\mathsf{Hyb}'_{\mathsf{Real}}$ except that the ciphertext is changed to semi-functional form. Before presenting the semi-functional form, we note that the normal form of challenge ciphertext can be simply written as follows since no attribute authority appearing in the ciphetext can be corrupted (Definition 2.6.2):

$\mathsf{ct} := (c_0, \{c_{1,x}, c_{2,x}, c_{3,x}, \widehat{c_{2,x}}, \widehat{c_{3,x}}\}_{x\in[n]})$, where $c_0 := \mathsf{msg}_b \cdot [\![\mathbf{t}^T\mathbf{k}]\!]_T$, $\forall x \in [n]$:

$$
\begin{aligned}
c_{1,x} &:= [\![\mathbf{c}_x^T]\!]_1 := [\![\mathbf{s}_x^T\mathbf{A}_1]\!]_1,\\
c_{2,x} &:= [\![\lambda_x]\!]_1 \cdot \boxed{(\mathbf{c}_x^T \odot [\![\mathbf{V}_{\rho(x)}]\!]_1)},\\
c_{3,x} &:= [\![\omega_x]\!]_1 \cdot \boxed{(\mathbf{c}_x^T \odot [\![\mathbf{U}_{\rho(x)}]\!]_1)},\\
\widehat{c_{2,x}} &:= [\![\widehat{\lambda_x}]\!]_1 \cdot \boxed{(\mathbf{c}_x^T \odot [\![\widehat{\mathbf{V}_{\rho(x)}}]\!]_1)},\\
\widehat{c_{3,x}} &:= [\![\widehat{\omega_x}]\!]_1 \cdot \boxed{(\mathbf{c}_x^T \odot [\![\widehat{\mathbf{U}_{\rho(x)}}]\!]_1)}.
\end{aligned}
$$

Given this notation, the ciphertext in Hyb_1 is same as the normal ciphertext except that for all $x \in \overline{U_{\mathcal{A}}}$: $\boxed{c_{1,x} := [\![\mathbf{c}_x^T]\!]_1}$, where $\boxed{\mathbf{c}_x \xleftarrow{\$} \mathbb{Z}_q^{k+1}}$. We call this semi-functional ciphertext. Observe that in total this means that semi-functional ciphertext changes $c_{1,x}, c_{2,x}, c_{3,x}, \widehat{c_{2,x}}, \widehat{c_{3,x}}$ for $x \in [n]$ when compared to normal ciphertext.

Claim 2.6.7. *If k-MDDH holds in group $\mathbb{G}_1$, then Hyb_0 and Hyb_1 are computationally indistinguishable.*

Proof. Similar to the proof of Claim 2.4.4 □

Hybrid Hyb_2. This is same as Hyb_1 except that the ciphertext structure is changed as follows: for all $x \in [n]$, $c_{3,x}, \widehat{c_{3,x}}$ are computed using $\omega_x := \mathbf{M}_x\boxed{\mathbf{W}}$, $\widehat{\omega_x} := \mathbf{N}_x\boxed{\mathbf{W}}$, where $\mathbf{W}$ is same as in Hyb_1 except that the first row is changed from $\mathbf{0}^T$ to $\gamma\mathbf{B}_3^{*T}$, where $\gamma \xleftarrow{\$} \mathbb{Z}_q$, that is, $\mathbf{W} = \begin{pmatrix} \boxed{\gamma\mathbf{B}_3^{*T}} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix}$.

Claim 2.6.8. *If the game condition holds and ρ is injective, then, Hyb_1 and Hyb_2 are statistically indistinguishable.*

Proof. Observe that the only difference between Hyb_1 and Hyb_2 is that in ciphertext components $c_{3,x}, \widehat{c_{3,x}}$ for all $x \in [n]$: $c_{3,x}, \widehat{c_{3,x}}$ contain $[\![\omega_x]\!]_1, [\![\widehat{\omega_x}]\!]_1$ respectively, where $\omega_x, \widehat{\omega_x}$ are secret share of $\mathbf{0}^T \in \mathbb{Z}_q^{1\times(2k+1)}$ in Hyb_1, but are secret share of $\gamma\mathbf{B}_3^{*T}$ in Hyb_2. Therefore, to prove that the hybrids are statistically indistinguishable, we will argue that $\gamma\mathbf{B}_3^{*T}$ is information theoretically hidden from the adversary $\mathcal{A}$ in Hyb_2.

Since the adversary is not allowed to corrupt any attribute authorities appearing in the challenge ciphertext, the only possible way for $\mathcal{A}$ to get information about $\gamma\mathbf{B}_3^{*T}$ is through the ciphertext components $c_{3,x}, \widehat{c_{3,x}}$ corresponding to all the rows $x \in [n]$ of $\mathbf{N}$. However, for each such row x, $\mathcal{A}$ can only recover $\mathbf{c}_x^T$, $\mathbf{M}_x\mathbf{W} + \mathbf{c}_x^T\mathbf{U}_{\rho(x)}$, $\mathbf{N}_x\mathbf{W} + \mathbf{c}_x^T\widehat{\mathbf{U}_{\rho(x)}}$ information theoretically. Without loss of generality, we can compute $\mathbf{U}_{\rho(x)} := \mathbf{U}_{\rho(x),1}\mathbf{B}_1^{*T} + \mathbf{U}_{\rho(x),2}\mathbf{B}_2^{*T} + \mathbf{U}_{\rho(x),3}\mathbf{B}_3^{*T}$, where $\mathbf{U}_{\rho(x),1} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times k}$, $\mathbf{U}_{\rho(x),2} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times k}$, and $\mathbf{U}_{\rho(x),3} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times 1}$. Similarly, we can compute $\widehat{\mathbf{U}_{\rho(x)}} := \widehat{\mathbf{U}_{\rho(x),1}}\mathbf{B}_1^{*T} + \widehat{\mathbf{U}_{\rho(x),2}}\mathbf{B}_2^{*T} + \widehat{\mathbf{U}_{\rho(x),3}}\mathbf{B}_3^{*T}$, where

$\widehat{\mathbf{U}_{\rho(x),1}} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times k}$, $\widehat{\mathbf{U}_{\rho(x),2}} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times k}$, and $\widehat{\mathbf{U}_{\rho(x),3}} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times 1}$. Let the first entry of $\mathbf{M}_x$ be m_x, that is, $\mathbf{M}_x = (m_x, \ldots)$. Let the first entry of $\mathbf{N}_x$ be n_x, that is, $\mathbf{N}_x = (n_x, \ldots)$. Then, observe that we can write

$$
\begin{aligned}
&\mathbf{N}_x \begin{pmatrix} \gamma \mathbf{B}_3^{*T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + \mathbf{c}_x^T \widehat{\mathbf{U}_{\rho(x)}} \\
&= \mathbf{N}_x \begin{pmatrix} \mathbf{0} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + \mathbf{N}_x \begin{pmatrix} \gamma \mathbf{B}_3^{*T} \\ \mathbf{0} \end{pmatrix} + \mathbf{c}_x^T \widehat{\mathbf{U}_{\rho(x)}} \\
&= \mathbf{N}_x \begin{pmatrix} \mathbf{0} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + (n_x, \ldots) \begin{pmatrix} \gamma \mathbf{B}_3^{*T} \\ \mathbf{0} \end{pmatrix} + \mathbf{c}_x^T \widehat{\mathbf{U}_{\rho(x)}} \\
&= \mathbf{N}_x \begin{pmatrix} \mathbf{0} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + n_x \gamma \mathbf{B}_3^{*T} + \mathbf{c}_x^T \widehat{\mathbf{U}_{\rho(x)}} \\
&= \mathbf{N}_x \begin{pmatrix} \mathbf{0} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + (n_x \gamma + \mathbf{c}_x^T \widehat{\mathbf{U}_{\rho(x),3}}) \mathbf{B}_3^{*T} + \mathbf{c}_x^T \widehat{\mathbf{U}_{\rho(x),1}} \mathbf{B}_1^{*T} + \mathbf{c}_x^T \widehat{\mathbf{U}_{\rho(x),2}} \mathbf{B}_2^{*T} \\
&\equiv \mathbf{N}_x \begin{pmatrix} \mathbf{0} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + (\mathbf{c}_x^T \widehat{\mathbf{U}'_{\rho(x),3}}) \mathbf{B}_3^{*T} + \mathbf{c}_x^T \widehat{\mathbf{U}_{\rho(x),1}} \mathbf{B}_1^{*T} + \mathbf{c}_x^T \widehat{\mathbf{U}_{\rho(x),2}} \mathbf{B}_2^{*T}
\end{aligned}
$$

where we can write $\widehat{\mathbf{U}'_{\rho(x),3}} = \widehat{\mathbf{U}_{\rho(x),3}} + \widehat{\Delta}$ such that $n_x \gamma = \mathbf{c}_x^T \widehat{\Delta}$. Similarly, we can write

$$
\begin{aligned}
&\mathbf{M}_x \begin{pmatrix} \gamma \mathbf{B}_3^{*T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + \mathbf{c}_x^T \mathbf{U}_{\rho(x)} \\
&\equiv \mathbf{M}_x \begin{pmatrix} \mathbf{0} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + (\mathbf{c}_x^T \mathbf{U}'_{\rho(x),3}) \mathbf{B}_3^{*T} + \mathbf{c}_x^T \mathbf{U}_{\rho(x),1} \mathbf{B}_1^{*T} + \mathbf{c}_x^T \mathbf{U}_{\rho(x),2} \mathbf{B}_2^{*T}
\end{aligned}
$$

where we can write $\mathbf{U}'_{\rho(x),3} = \mathbf{U}_{\rho(x),3} + \Delta$ such that $m_x \gamma = \mathbf{c}_x^T \Delta$. Therefore, to complete the proof, it suffices to argue that $(\mathbf{U}_{\rho(x),3}, \widehat{\mathbf{U}_{\rho(x),3}})$ and $(\mathbf{U}'_{\rho(x),3}, \widehat{\mathbf{U}'_{\rho(x),3}})$ are identically distributed. We show this next.

Observe that since ρ is injective, hence it follows that $\mathbf{U}_{\rho(x)}, \widehat{\mathbf{U}_{\rho(x)}}$ are fresh random matrices and the only other place they appear are in the secret key $\mathsf{sk}_{\rho(x),\mathsf{GID},z_{\rho(x)}}$. Due to the game condition that $|S_{\rho(x),\mathsf{GID}}| = 1$, there can be only one such secret key per attribute $\rho(x)$. We argue that $\mathsf{sk}_{\rho(x),\mathsf{GID},z_{\rho(x)}}$ information theoretically leaks no information about $(\mathbf{U}_{\rho(x),3}, \widehat{\mathbf{U}_{\rho(x),3}})$ and hence $(\mathbf{U}_{\rho(x),3}, \widehat{\mathbf{U}_{\rho(x),3}})$ and $(\mathbf{U}'_{\rho(x),3}, \widehat{\mathbf{U}'_{\rho(x),3}})$ are identically distributed. To see this, observe that $\mathsf{sk}_{\rho(x),\mathsf{GID},z_{\rho(x)}}$ information theoretically reveals $(z_{\rho(x)} \cdot \mathbf{V}_{\rho(x)} + \widehat{\mathbf{V}_{\rho(x)}})\mathbf{k} + (z_{\rho(x)} \cdot \mathbf{U}_{\rho(x)} + \widehat{\mathbf{U}_{\rho(x)}})\mathbf{B}_1 \mathbf{h}_{\mathsf{GID}}$, where $\mathbf{U}_{\rho(x)} \mathbf{B}_1 \mathbf{h}_{\mathsf{GID}} = \mathbf{U}_{\rho(x),1} \mathbf{h}_{\mathsf{GID}}$ and $\widehat{\mathbf{U}_{\rho(x)}} \mathbf{B}_1 \mathbf{h}_{\mathsf{GID}} = \widehat{\mathbf{U}_{\rho(x),1}} \mathbf{h}_{\mathsf{GID}}$ since $\mathbf{B}_1^{*T} \mathbf{B}_1 = \mathbf{I}$, $\mathbf{B}_2^{*T} \mathbf{B}_1 = \mathbf{0}$ and $\mathbf{B}_3^{*T} \mathbf{B}_1 = \mathbf{0}$, thus no information about $(\mathbf{U}_{\rho(x),3}, \widehat{\mathbf{U}_{\rho(x),3}})$ is revealed.

In conclusion, substituting $(\mathbf{U}_{\rho(x),3}, \widehat{\mathbf{U}_{\rho(x),3}})$ with $(\mathbf{U}'_{\rho(x),3}, \widehat{\mathbf{U}'_{\rho(x),3}})$ (as described above) for all rows $x \in [n]$ of matrices $\mathbf{M}, \mathbf{N}$ allows us to move from Hyb_2 to Hyb_1.

Therefore, it follows that Hyb_1 and Hyb_2 are statistically indistinguishable. This completes the proof of Claim 2.6.8. □

Hybrid $\mathsf{Hyb}_{3,j-1}$ for $j \in [q+1]$. This hybrid is same as Hyb_2 except that for the ith global identifier GID_i for $i \leq j-1$, the challenger programs the output $\mathsf{H}_1(\mathsf{GID}_i)$ of the random oracle H_1 as $\mathsf{H}_1(\mathsf{GID}_i) = [\![\boxed{\mathbf{B}_1\mathbf{h}_{\mathsf{GID}_i} + \mathbf{B}_3}]\!]_2$, where $\mathbf{h}_{\mathsf{GID}_i} \xleftarrow{\$} \mathbb{Z}_q^k$, while for $i > j-1$, it programs the output $\mathsf{H}_1(\mathsf{GID}_i)$ of the random oracle H_1 as $\mathsf{H}_1(\mathsf{GID}_i) = [\![\mathbf{B}_1\mathbf{h}_{\mathsf{GID}_i}]\!]_2$ as earlier.

Observe that $\mathsf{Hyb}_{3,0}$ is same as Hyb_2. We introduce a sequnce of intermediate hybrids $\mathsf{Hyb}_{3,j,1}$, $\mathsf{Hyb}'_{3,j,1}$, $\mathsf{Hyb}_{3,j,2}$, $\mathsf{Hyb}'_{3,j,2}$, $\mathsf{Hyb}_{3,j,3}$ between $\mathsf{Hyb}_{3,j-1}$ and $\mathsf{Hyb}_{3,j}$ for all $j \in [q]$ as defined below.

Hybrid $\mathsf{Hyb}_{3,j,1}$ for $j \in [q]$. This hybrid is same as $\mathsf{Hyb}_{3,j-1}$ except that for the jth global identifier GID_j, the challenger programs the output $\mathsf{H}_1(\mathsf{GID}_j)$ of the random oracle H_1 as $\mathsf{H}_1(\mathsf{GID}_j) = [\![\boxed{\mathbf{B}_1\mathbf{h}_{\mathsf{GID}_j} + \mathbf{B}_2\mathbf{h}'_{\mathsf{GID}_j}}]\!]_2$, where $\mathbf{h}_{\mathsf{GID}_j}, \mathbf{h}'_{\mathsf{GID}_j} \xleftarrow{\$} \mathbb{Z}_q^k$.

Claim 2.6.9. *If $\mathsf{SD}^{\mathbb{G}_2}_{\mathbf{B}_1 \to \mathbf{B}_1, \mathbf{B}_2}$ assumption holds (Assumption 2.3.9), then, hybrids $\mathsf{Hyb}_{3,j-1}$ and $\mathsf{Hyb}_{3,j,1}$ are computationally indistinguishable for all $j \in [q]$.*

Proof. Similar to proof of Claim 2.4.6. □

Hybrid $\mathsf{Hyb}'_{3,j,1}$. This is same as $\mathsf{Hyb}_{3,j,1}$ except that the ciphertext structure is changed as follows: for all $x \in [n]$, $c_{3,x}, \widehat{c_{3,x}}$ are computed using $\omega_x := \mathbf{M}_x \boxed{\mathbf{W}}$, $\widehat{\omega_x} := \mathbf{N}_x \boxed{\mathbf{W}}$, where $\mathbf{W}$ is same as in Hyb_2 except that the first row is changed from $\gamma\mathbf{B}_3^{*T}$ to $(\mathbf{B}_2^*\delta + \gamma\mathbf{B}_3^*)^T$, where $\gamma \xleftarrow{\$} \mathbb{Z}_q$ and $\delta \xleftarrow{\$} \mathbb{Z}_q^k$ that is, $\mathbf{W} = \begin{pmatrix} \boxed{(\mathbf{B}_2^*\delta + \gamma\mathbf{B}_3^*)^T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix}$.

Claim 2.6.10. *If the game condition holds and ρ is injective, then, $\mathsf{Hyb}_{3,j,1}$ and $\mathsf{Hyb}'_{3,j,1}$ are statistically indistinguishable.*

Proof. Observe that the only difference between $\mathsf{Hyb}_{3,j,1}$ and $\mathsf{Hyb}'_{3,j,1}$ is that in ciphertext components $c_{3,x}, \widehat{c_{3,x}}$ for all $x \in [n]$: $c_{3,x}, \widehat{c_{3,x}}$ contain $[\![\omega_x]\!]_1, [\![\widehat{\omega_x}]\!]_1$ respectively, where $\omega_x, \widehat{\omega_x}$ are secret share of $\gamma\mathbf{B}_3^{*T} \in \mathbb{Z}_q^{1\times(2k+1)}$ in $\mathsf{Hyb}_{3,j,1}$, but it is a secret share of $(\mathbf{B}_2^*\delta + \gamma\mathbf{B}_3^*)^T$ in $\mathsf{Hyb}'_{3,j,1}$. Therefore, to prove that the hybrids are statistically indistinguishable, we will argue that $\mathbf{B}_2^*\delta$ is information theoretically hidden to the adversary $\mathcal{A}$ in $\mathsf{Hyb}'_{3,j,1}$.

Suppose the challenge access policy $(\mathbf{M}, \mathbf{N}, \rho)$ is defined over a set of attributes $U \subseteq \mathcal{AU}$, that is, $\rho : [n] \to U$. Recall from Definition 2.6.2 that the game condition requires that:

- for each $\mathsf{GID} \in \mathcal{GID}$, it is required that $S_{\mathsf{GID}} \notin (\mathbf{M}, \mathbf{N}, \rho)$,
- for each $\mathsf{GID} \in \mathcal{GID}$, for each $i \in S_{\mathsf{GID}}$, it is required that $|S_{i,\mathsf{GID}}| = 1$,

where S_{GID} and $S_{i,\mathsf{GID}}$ are defined as follows:

- for each global identifier $\mathsf{GID} \in \mathcal{GID}$, S_{GID} denotes the subset of U containing attributes i such that the adversary queried a secret key for the tuple $(i, \mathsf{GID}, \cdot)$.
- for each global identifier $\mathsf{GID} \in \mathcal{GID}$, for each attribute $i \in S_{\mathsf{GID}}$, $S_{i,\mathsf{GID}}$ denotes the set of attribute values z_i such that the adversary queried a secret key for the tuple (i, GID, z_i).

To show that $\mathbf{B}_2^* \delta$ is information theoretically hidden from the adversary $\mathcal{A}$ in $\mathsf{Hyb}'_{3,j,1}$, we only need to rely on the two game conditions for the jth GID, that is, we will use the fact that $S_{\mathsf{GID}_j} \notin (\mathbf{M}, \mathbf{N}, \rho)$ and for each $i \in S_{\mathsf{GID}_j}$, $|S_{i,\mathsf{GID}_j}| = 1$. Here, $S_{\mathsf{GID}_j} \notin (\mathbf{M}, \mathbf{N}, \rho)$ is a shorthand for $(1, 0, \ldots, 0) \notin \mathsf{rowSpan}(\{z_{\rho(x)}\mathbf{M}_x + \mathbf{N}_x\}_{\rho(x) \in S_{\mathsf{GID}_j}})$.

Since the adversary is not allowed to corrupt any attribute authorities appearing in the challenge ciphertext, the only possible way for $\mathcal{A}$ to get information about $\mathbf{B}_2^* \delta$ is through the ciphertext components $c_{3,x}, \widehat{c_{3,x}}$ corresponding to all the rows $x \in [n]$ of $\mathbf{M}, \mathbf{N}$. However, for each such row x, $\mathcal{A}$ can only recover $\mathbf{c}_x^T$, $\mathbf{M}_x\mathbf{W} + \mathbf{c}_x^T\mathbf{U}_{\rho(x)}$, $\mathbf{N}_x\mathbf{W} + \mathbf{c}_x^T\widehat{\mathbf{U}_{\rho(x)}}$ information theoretically. We analyze two cases:

- x **such that** $\rho(x) \in S_{\mathsf{GID}_j}$**:** These are the rows of $\mathbf{M}, \mathbf{N}$ labeled by authorities who issued key for GID_j. From the game condition, it follows that $(1, 0, \ldots, 0) \notin \mathsf{rowSpan}(\{z_{\rho(x)}\mathbf{M}_x + \mathbf{N}_x\}_{\rho(x) \in S_{\mathsf{GID}_j}})$. Therefore, it follows that $\mathbf{M}_x\mathbf{W}$ and $\mathbf{N}_x\mathbf{W}$ contain no information about $(\mathbf{B}_2^* \delta + \gamma \mathbf{B}_3^*)^T$. Hence, $\mathbf{c}_x^T$, $\mathbf{M}_x\mathbf{W} + \mathbf{c}_x^T\mathbf{U}_{\rho(x)}$, $\mathbf{N}_x\mathbf{W} + \mathbf{c}_x^T\widehat{\mathbf{U}_{\rho(x)}}$ reveal no information about $(\mathbf{B}_2^* \delta + \gamma \mathbf{B}_3^*)^T$ to the adversary $\mathcal{A}$.
- x **such that** $\rho(x) \in U \setminus S_{\mathsf{GID}_j}$**:** These are the rows of $\mathbf{M}, \mathbf{N}$ labeled by authorities who are neither corrupt nor issued key for GID_j. The game condition does not apply for these rows and the analysis requires more care as described next. Without loss of generality, we can compute $\mathbf{U}_{\rho(x)} := \mathbf{U}_{\rho(x),1}\mathbf{B}_1^{*T} + \mathbf{U}_{\rho(x),2}\mathbf{B}_2^{*T} + \mathbf{U}_{\rho(x),3}\mathbf{B}_3^{*T}$, where $\mathbf{U}_{\rho(x),1} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times k}, \mathbf{U}_{\rho(x),2} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times k}$, and $\mathbf{U}_{\rho(x),3} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times 1}$. Similarly, we can compute $\widehat{\mathbf{U}_{\rho(x)}} := \widehat{\mathbf{U}_{\rho(x),1}}\mathbf{B}_1^{*T} + \widehat{\mathbf{U}_{\rho(x),2}}\mathbf{B}_2^{*T} + \widehat{\mathbf{U}_{\rho(x),3}}\mathbf{B}_3^{*T}$, where $\widehat{\mathbf{U}_{\rho(x),1}} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times k}, \widehat{\mathbf{U}_{\rho(x),2}} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times k}$, and $\widehat{\mathbf{U}_{\rho(x),3}} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times 1}$. Let the first entry of $\mathbf{M}_x$ be m_x, that is, $\mathbf{M}_x = (m_x, \ldots)$. Let the first entry of

$\mathbf{N}_x$ be n_x, that is, $\mathbf{N}_x = (n_x, \ldots)$. Then, observe that we can write

$$\begin{aligned}
&\mathbf{N}_x \begin{pmatrix} (\mathbf{B}_2^* \delta + \gamma \mathbf{B}_3^*)^T \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + \mathbf{c}_x^T \widehat{\mathbf{U}_{\rho(x)}} \\
&= \mathbf{N}_x \begin{pmatrix} \gamma \mathbf{B}_3^{*T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + \mathbf{N}_x \begin{pmatrix} (\mathbf{B}_2^* \delta)^T \\ \mathbf{0} \end{pmatrix} + \mathbf{c}_x^T \widehat{\mathbf{U}_{\rho(x)}} \\
&= \mathbf{N}_x \begin{pmatrix} \gamma \mathbf{B}_3^{*T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + (n_x, \ldots) \begin{pmatrix} (\mathbf{B}_2^* \delta)^T \\ \mathbf{0} \end{pmatrix} + \mathbf{c}_x^T \widehat{\mathbf{U}_{\rho(x)}} \\
&= \mathbf{N}_x \begin{pmatrix} \gamma \mathbf{B}_3^{*T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + n_x (\mathbf{B}_2^* \delta)^T + \mathbf{c}_x^T \widehat{\mathbf{U}_{\rho(x)}} \\
&= \mathbf{N}_x \begin{pmatrix} \gamma \mathbf{B}_3^{*T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + (n_x \delta^T + \mathbf{c}_x^T \widehat{\mathbf{U}_{\rho(x),2}}) \mathbf{B}_2^{*T} + \mathbf{c}_x^T \widehat{\mathbf{U}_{\rho(x),1}} \mathbf{B}_1^{*T} + \mathbf{c}_x^T \widehat{\mathbf{U}_{\rho(x),3}} \mathbf{B}_3^{*T} \\
&\equiv \mathbf{N}_x \begin{pmatrix} \gamma \mathbf{B}_3^{*T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + (\mathbf{c}_x^T \widehat{\mathbf{U}'_{\rho(x),2}}) \mathbf{B}_2^{*T} + \mathbf{c}_x^T \widehat{\mathbf{U}_{\rho(x),1}} \mathbf{B}_1^{*T} + \mathbf{c}_x^T \widehat{\mathbf{U}_{\rho(x),3}} \mathbf{B}_3^{*T}
\end{aligned}$$

where we can write $\widehat{\mathbf{U}'_{\rho(x),2}} = \widehat{\mathbf{U}_{\rho(x),2}} + \widehat{\Delta}$ such that $n_x \delta^T = \mathbf{c}_x^T \widehat{\Delta}$. Similarly, we can write

$$\begin{aligned}
&\mathbf{M}_x \begin{pmatrix} (\mathbf{B}_2^* \delta + \gamma \mathbf{B}_3^*)^T \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + \mathbf{c}_x^T \mathbf{U}_{\rho(x)} \\
&\equiv \mathbf{N}_x \begin{pmatrix} \gamma \mathbf{B}_3^{*T} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix} + (\mathbf{c}_x^T \mathbf{U}'_{\rho(x),2}) \mathbf{B}_2^{*T} + \mathbf{c}_x^T \mathbf{U}_{\rho(x),1} \mathbf{B}_1^{*T} + \mathbf{c}_x^T \mathbf{U}_{\rho(x),3} \mathbf{B}_3^{*T}
\end{aligned}$$

where we can write $\mathbf{U}'_{\rho(x),2} = \mathbf{U}_{\rho(x),2} + \Delta$ such that $m_x \delta^T = \mathbf{c}_x^T \Delta$. Therefore, to complete the proof, it suffices to argue that $(\mathbf{U}_{\rho(x),2}, \widehat{\mathbf{U}_{\rho(x),2}})$ and $(\mathbf{U}'_{\rho(x),2}, \widehat{\mathbf{U}'_{\rho(x),2}})$ are identically distributed. We show this next.

Observe that since ρ is injective, hence it follows that $\mathbf{U}_{\rho(x)}, \widehat{\mathbf{U}_{\rho(x)}}$ are fresh random matrix and the only other place it appears are in secret key $\mathsf{sk}_{\rho(x),\mathsf{GID}_{\mathsf{index}}}$. Due to the game condition that $|S_{\rho(x),\mathsf{GID}}| = 1$, there can be only one such secret key per attribute $\rho(x)$. We argue that $\mathsf{sk}_{\rho(x),\mathsf{GID},z_{\rho(x)}}$ information theoretically leaks no information about $(\mathbf{U}_{\rho(x)}, \widehat{\mathbf{U}_{\rho(x)}})$ and hence $(\mathbf{U}_{\rho(x)}, \widehat{\mathbf{U}_{\rho(x)}})$ and $(\mathbf{U}'_{\rho(x)}, \widehat{\mathbf{U}'_{\rho(x)}})$ are identically distributed. To see this, observe that the $\mathbf{U}_{\rho(x)}, \widehat{\mathbf{U}_{\rho(x)}}$-dependent term of $\mathsf{sk}_{\rho(x),\mathsf{GID}_{\mathsf{index}},z_{\rho(x)}}$ is of the form $(z_{\rho(x)} \mathbf{U}_{\rho(x)} + \widehat{\mathbf{U}_{\rho(x)}}) \odot \mathsf{H}_1(\mathsf{GID}_{\mathsf{index}})$, where $\mathsf{H}_1(\mathsf{GID}_{\mathsf{index}})$ is of the

following form depending on index:

$$\mathsf{H}_1(\mathsf{GID}_{\mathsf{index}}) = \begin{cases} [\![\mathbf{B}_1\mathbf{h}_{\mathsf{GID}_{\mathsf{index}}} + \mathbf{B}_3]\!]_2 & \text{, if index} \leq j-1 \\ [\![\mathbf{B}_1\mathbf{h}_{\mathsf{GID}_{\mathsf{index}}} + \mathbf{B}_2\mathbf{h}'_{\mathsf{GID}_{\mathsf{index}}}]\!]_2 & \text{, if index} = j \\ [\![\mathbf{B}_1\mathbf{h}_{\mathsf{GID}_{\mathsf{index}}}]\!]_2 & \text{, if index} > j \end{cases}.$$

We analyze the three cases separately:

- **Case 1:** $\mathsf{index} < j$**.** Observe that the $\mathbf{U}_{\rho(x)}, \widehat{\mathbf{U}_{\rho(x)}}$-dependent term of $\mathsf{sk}_{\rho(x),\mathsf{GID}_{\mathsf{index}},z_{\rho(x)}}$ information theoretically reveals

  $$(z_{\rho(x)}\mathbf{U}_{\rho(x),1} + \widehat{\mathbf{U}_{\rho(x),1}})\mathbf{h}_{\mathsf{GID}_{\mathsf{index}}} + z_{\rho(x)}\mathbf{U}_{\rho(x),3} + \widehat{\mathbf{U}_{\rho(x),3}}$$

  since $\mathbf{B}_2^{*T}\mathbf{B}_1 = \mathbf{0}$ and $\mathbf{B}_2^{*T}\mathbf{B}_3 = \mathbf{0}$.

- **Case 2:** $\mathsf{index} = j$**.** This case requires no analysis since adversary $\mathcal{A}$ never sees secret keys $\mathsf{sk}_{\rho(x),\mathsf{GID}_j}$. This is due to the definition of set S_{GID_j} and the fact that we are only considering x such that $\rho(x) \in U \setminus S_{\mathsf{GID}_j}$.

- **Case 3:** $\mathsf{index} > j$**.** Observe that the $\mathbf{U}_{\rho(x)}, \widehat{\mathbf{U}_{\rho(x)}}$-dependent term of $\mathsf{sk}_{\rho(x),\mathsf{GID}_{\mathsf{index}},z_{\rho(x)}}$ information theoretically reveals

  $$(z_{\rho(x)}\mathbf{U}_{\rho(x),1} + \widehat{\mathbf{U}_{\rho(x),1}})\mathbf{h}_{\mathsf{GID}_{\mathsf{index}}}$$

  since $\mathbf{B}_2^{*T}\mathbf{B}_1 = \mathbf{0}$.

Hence, it follows that $\mathsf{sk}_{\rho(x),\mathsf{GID}_{\mathsf{index}},z_{\rho(x)}}$ information theoretically reveals no information about $(\mathbf{U}_{\rho(x),2}, \widehat{\mathbf{U}_{\rho(x),2}})$, so $(\mathbf{U}_{\rho(x),2}, \widehat{\mathbf{U}_{\rho(x),2}})$ and $(\mathbf{U}'_{\rho(x),2}, \widehat{\mathbf{U}'_{\rho(x),2}})$ are identically distributed. Thus, for all x such that $\rho(x) \in U \setminus S_{\mathsf{GID}_j}$, ciphertext components $c_{2,x}, \widehat{c_{2,x}}$ reveal no information about $\mathbf{B}_2^*\boldsymbol{\delta}$ to the adversary $\mathcal{A}$.

In conclusion, substituting $(\mathbf{U}_{\rho(x),2}, \widehat{\mathbf{U}_{\rho(x),2}})$ with $(\mathbf{U}'_{\rho(x),2}, \widehat{\mathbf{U}'_{\rho(x),2}})$ (as described above) for all rows $x \in [n]$ of matrices $\mathbf{M}, \mathbf{N}$ allows us to move from $\mathsf{Hyb}'_{3,j,1}$ to $\mathsf{Hyb}_{3,j,1}$.

Therefore, it follows that $\mathsf{Hyb}_{3,j,1}$ and $\mathsf{Hyb}'_{3,j,1}$ are statistically indistinguishable. This completes the proof of Claim 2.6.10.

□

Hybrid $\mathsf{Hyb}_{3,j,2}$**.** This is same as $\mathsf{Hyb}'_{3,j,1}$ except that for the jth global identifier GID_j, the challenger programs the output $\mathsf{H}_1(\mathsf{GID}_j)$ of the random oracle H_1 as $\mathsf{H}_1(\mathsf{GID}_j) = [\![\boxed{\mathbf{B}_1\mathbf{h}_{\mathsf{GID}_j} + \mathbf{B}_2\mathbf{h}'_{\mathsf{GID}_j} + \mathbf{B}_3}]\!]_2$, where $\mathbf{h}_{\mathsf{GID}_j}, \mathbf{h}'_{\mathsf{GID}_j} \xleftarrow{\$} \mathbb{Z}_q^k$.

Claim 2.6.11. *If* $\mathsf{SD}^{\mathbb{G}_2}_{\mathbf{B}_2\to\mathbf{B}_2,\mathbf{B}_3}$ *assumption holds (Assumption 2.3.9), then, hybrids* $\mathsf{Hyb}'_{3,j,1}$ *and* $\mathsf{Hyb}_{3,j,2}$ *are computationally indistinguishable for all* $j \in [q]$.

Proof. Similar to proof of Claim 2.4.8. □

Hybrid $\mathsf{Hyb}'_{3,j,2}$**.** This is same as $\mathsf{Hyb}_{3,j,2}$ except that the ciphertext structure is changed as follows: the first row of matrix $\mathbf{W}$ is changed from $(\mathbf{B}^*_2\delta + \gamma\mathbf{B}^*_3)^T$ to $\gamma\mathbf{B}^{*T}_3$, where and $\gamma \xleftarrow{\$} \mathbb{Z}_q$, that is, $\mathbf{W} = \begin{pmatrix} \boxed{\gamma\mathbf{B}^{*T}_3} \\ \mathbf{W}_{\mathsf{bot}} \end{pmatrix}$.

Claim 2.6.12. *If the game condition holds and* ρ *is injective, then,* $\mathsf{Hyb}_{3,j,2}$ *and* $\mathsf{Hyb}'_{3,j,2}$ *are statistically indistinguishable.*

Proof. Similar to the proof of Claim 2.6.8. □

Claim 2.6.13. *If* $\mathsf{SD}^{\mathbb{G}_2}_{\mathbf{B}_1\to\mathbf{B}_1,\mathbf{B}_2}$ *assumption holds (Assumption 2.3.9), then, hybrids* $\mathsf{Hyb}'_{3,j,2}$ *and* $\mathsf{Hyb}_{3,j}$ *are computationally indistinguishable for all* $j \in [q]$.

Proof. Similar to proof of Claim 2.6.9. □

Hybrid Hyb_4**.** This is same as hybrid $\mathsf{Hyb}_{3,q}$ except that the ciphertext structure is changed as follows: for all $x \in [n]$, $c_{2,x}, \widehat{c_{2,x}}$ are computed using $\lambda_x := \mathbf{M}_x\boxed{\mathbf{T}}$, $\widehat{\lambda_x} := \mathbf{N}_x\boxed{\mathbf{T}}$, where $\mathbf{T}$ is same as $\mathbf{T}$ except that the first row is changed from $\mathbf{t}^T$ to $(\mathbf{t} + \tau\mathbf{B}^*_3)^T$, where $\tau \xleftarrow{\$} \mathbb{Z}_q$, that is $\mathbf{T} = \begin{pmatrix} \boxed{(\mathbf{t} + \tau\mathbf{B}^*_3)^T} \\ \mathbf{T}_{\mathsf{bot}} \end{pmatrix}$. We note that c_0 remains unchanged, that is, the masking term is still $\mathbf{t}^T\mathbf{k}$.

Claim 2.6.14. *If the game condition holds and* ρ *is injective, then,* $\mathsf{Hyb}_{3,q}$ *and* Hyb_4 *are statistically indistinguishable.*

Proof. Observe that the only difference between $\mathsf{Hyb}_{3,q}$ and Hyb_4 is in ciphertext components $c_{2,x}, \widehat{c_{2,x}}$ for all $x \in [n]$. Observe that $c_{2,x}, \widehat{c_{2,x}}$ contains $\lambda_x, [\![\widehat{\lambda_x}]\!]_1$ respectively, where $\lambda_x, \widehat{\lambda_x}$ are secret share of $\mathbf{t}^T \in \mathbb{Z}_q^{1\times(2k+1)}$ in $\mathsf{Hyb}_{3,q}$, but are secret share of $(\mathbf{t} + \tau\mathbf{B}^*_3)^T$ in Hyb_4. Therefore, to prove that the hybrids are statistically indistinguishable, we will argue that $\tau\mathbf{B}^{*T}_3$ is information theoretically hidden to the adversary $\mathcal{A}$ in Hyb_4.

Since the adversary is not allowed to corrupt any attribute authorities appearing in the challenge ciphertext, the only possible way for $\mathcal{A}$ to get information about $\tau\mathbf{B}^{*T}_3$ is through the ciphertext components $c_{2,x}, \widehat{c_{2,x}}$ corresponding to all the rows $x \in [n]$ of $\mathbf{N}$. However, for each such row x, $\mathcal{A}$ can only recover $\mathbf{c}^T_x$, $\mathbf{M}_x\mathbf{T} + \mathbf{c}^T_x\mathbf{V}_{\rho(x)}$, $\mathbf{N}_x\mathbf{T} + \mathbf{c}^T_x\widehat{\mathbf{V}_{\rho(x)}}$ information theoretically. Without loss of generality, we can compute $\mathbf{V}_{\rho(x)} := \mathbf{V}_{\rho(x),1}\mathbf{B}^{*T}_1 + \mathbf{V}_{\rho(x),2}\mathbf{B}^{*T}_2 + \mathbf{V}_{\rho(x),3}\mathbf{B}^{*T}_3$, where $\mathbf{V}_{\rho(x),1} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times k}$, $\mathbf{V}_{\rho(x),2} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times k}$, and $\mathbf{V}_{\rho(x),3} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times 1}$. Similarly, we can compute $\widehat{\mathbf{V}_{\rho(x)}} := \widehat{\mathbf{V}_{\rho(x),1}}\mathbf{B}^{*T}_1 + \widehat{\mathbf{V}_{\rho(x),2}}\mathbf{B}^{*T}_2 + \widehat{\mathbf{V}_{\rho(x),3}}\mathbf{B}^{*T}_3$, where

$\widehat{\mathbf{V}_{\rho(x),1}} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times k}$, $\widehat{\mathbf{V}_{\rho(x),2}} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times k}$, and $\widehat{\mathbf{V}_{\rho(x),3}} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times 1}$. Let the first entry of $\mathbf{M}_x$ be m_x, that is, $\mathbf{M}_x = (m_x, \ldots)$. Let the first entry of $\mathbf{N}_x$ be n_x, that is, $\mathbf{N}_x = (n_x, \ldots)$. Then, observe that we can write

$$
\begin{aligned}
&\mathbf{N}_x \begin{pmatrix} (\mathbf{t} + \tau \mathbf{B}_3^*)^T \\ \mathbf{T}_{\mathsf{bot}} \end{pmatrix} + \mathbf{c}_x^T \widehat{\mathbf{V}_{\rho(x)}} \\
&= \mathbf{N}_x \begin{pmatrix} \mathbf{t}^T \\ \mathbf{T}_{\mathsf{bot}} \end{pmatrix} + \mathbf{N}_x \begin{pmatrix} \tau \mathbf{B}_3^{*T} \\ \mathbf{0} \end{pmatrix} + \mathbf{c}_x^T \widehat{\mathbf{V}_{\rho(x)}} \\
&= \mathbf{N}_x \begin{pmatrix} \mathbf{t}^T \\ \mathbf{T}_{\mathsf{bot}} \end{pmatrix} + (n_x, \ldots) \begin{pmatrix} \tau \mathbf{B}_3^{*T} \\ \mathbf{0} \end{pmatrix} + \mathbf{c}_x^T \widehat{\mathbf{V}_{\rho(x)}} \\
&= \mathbf{N}_x \begin{pmatrix} \mathbf{t}^T \\ \mathbf{T}_{\mathsf{bot}} \end{pmatrix} + n_x \tau \mathbf{B}_3^{*T} + \mathbf{c}_x^T \widehat{\mathbf{V}_{\rho(x)}} \\
&= \mathbf{N}_x \begin{pmatrix} \mathbf{t}^T \\ \mathbf{T}_{\mathsf{bot}} \end{pmatrix} + (n_x \tau + \mathbf{c}_x^T \widehat{\mathbf{V}_{\rho(x),3}}) \mathbf{B}_3^{*T} + \mathbf{c}_x^T \widehat{\mathbf{V}_{\rho(x),1}} \mathbf{B}_1^{*T} + \mathbf{c}_x^T \widehat{\mathbf{V}_{\rho(x),2}} \mathbf{B}_2^{*T} \\
&\equiv \mathbf{N}_x \begin{pmatrix} \mathbf{t}^T \\ \mathbf{T}_{\mathsf{bot}} \end{pmatrix} + (\mathbf{c}_x^T \widehat{\mathbf{V}'_{\rho(x),3}}) \mathbf{B}_3^{*T} + \mathbf{c}_x^T \widehat{\mathbf{V}_{\rho(x),1}} \mathbf{B}_1^{*T} + \mathbf{c}_x^T \widehat{\mathbf{V}_{\rho(x),2}} \mathbf{B}_2^{*T}
\end{aligned}
$$

where we can write $\widehat{\mathbf{V}'_{\rho(x),3}} = \widehat{\mathbf{V}_{\rho(x),3}} + \widehat{\Delta}$ such that $n_x \tau = \mathbf{c}_x^T \widehat{\Delta}$. Similarly, we can write

$$
\begin{aligned}
&\mathbf{M}_x \begin{pmatrix} (\mathbf{t} + \tau \mathbf{B}_3^*)^T \\ \mathbf{T}_{\mathsf{bot}} \end{pmatrix} + \mathbf{c}_x^T \mathbf{V}_{\rho(x)} \\
&\equiv \mathbf{M}_x \begin{pmatrix} \mathbf{t}^T \\ \mathbf{T}_{\mathsf{bot}} \end{pmatrix} + (\mathbf{c}_x^T \mathbf{V}'_{\rho(x),3}) \mathbf{B}_3^{*T} + \mathbf{c}_x^T \mathbf{V}_{\rho(x),1} \mathbf{B}_1^{*T} + \mathbf{c}_x^T \mathbf{V}_{\rho(x),2} \mathbf{B}_2^{*T}
\end{aligned}
$$

where we can write $\mathbf{V}'_{\rho(x),3} = \mathbf{V}_{\rho(x),3} + \Delta$ such that $m_x \tau = \mathbf{c}_x^T \Delta$. Therefore, to complete the proof, it suffices to argue that $(\mathbf{V}_{\rho(x),3}, \widehat{\mathbf{V}_{\rho(x),3}})$ and $(\mathbf{V}'_{\rho(x),3}, \widehat{\mathbf{V}'_{\rho(x),3}})$ are identically distributed. We show this next.

Observe that since ρ is injective, hence it follows that $\mathbf{V}_{\rho(x)}, \widehat{\mathbf{V}_{\rho(x)}}$ are fresh random matrices and the only other place they appear is in secret key $\mathsf{sk}_{\rho(x),\mathsf{GID},z_{\rho(x)}}$. Due to the game condition that $|S_{\rho(x),\mathsf{GID}}| = 1$, there can be only one such secret key per attribute $\rho(x)$. Specifically, $\mathsf{sk}_{\rho(x),\mathsf{GID}}$ information theoretically reveals $(z_{\rho(x)} \mathbf{V}_{\rho(x)} + \widehat{\mathbf{V}_{\rho(x)}}) \mathbf{k} + (z_{\rho(x)} \mathbf{U}_{\rho(x)} + \widehat{\mathbf{U}_{\rho(x)}})(\mathbf{B}_1 \mathbf{h}_{\mathsf{GID}} + \mathbf{B}_3)$.

Since $\mathbf{k}$ is uniform random, we can equivalently write it as $\mathbf{k} := \mathbf{B}_1 \mathbf{k}_1 + \mathbf{B}_2 \mathbf{k}_2 + k_3 \mathbf{B}_3$ for uniform random $\mathbf{k}_1 \xleftarrow{\$} \mathbb{Z}_q^k$, $\mathbf{k}_2 \xleftarrow{\$} \mathbb{Z}_q^k$, $k_3 \xleftarrow{\$} \mathbb{Z}_q$. Further, we can write $\mathbf{U}_{\rho(x)} := \mathbf{U}_{\rho(x),1} \mathbf{B}_1^{*T} + \mathbf{U}_{\rho(x),2} \mathbf{B}_2^{*T} + \mathbf{U}_{\rho(x),3} \mathbf{B}_3^{*T}$, where $\mathbf{U}_{\rho(x),1} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times k}$, $\mathbf{U}_{\rho(x),2} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times k}$, and $\mathbf{U}_{\rho(x),3} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times 1}$. Similarly, we can write $\widehat{\mathbf{U}_{\rho(x)}} := \widehat{\mathbf{U}_{\rho(x),1}} \mathbf{B}_1^{*T} + \widehat{\mathbf{U}_{\rho(x),2}} \mathbf{B}_2^{*T} + \widehat{\mathbf{U}_{\rho(x),3}} \mathbf{B}_3^{*T}$, where $\widehat{\mathbf{U}_{\rho(x),1}} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times k}$, $\widehat{\mathbf{U}_{\rho(x),2}} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times k}$, and $\widehat{\mathbf{U}_{\rho(x),3}} \xleftarrow{\$} \mathbb{Z}_q^{(k+1)\times 1}$. Then observe

that we can write

$$
\begin{aligned}
& z_{\rho(x)} \left(\mathbf{V}_{\rho(x)}\mathbf{k} + \mathbf{U}_{\rho(x)}\mathbf{B}_1\mathbf{h}_{\mathsf{GID}} + \mathbf{U}_{\rho(x)}\mathbf{B}_3 \right) \\
& \quad + \widehat{\mathbf{V}_{\rho(x)}}\mathbf{k} + \widehat{\mathbf{U}_{\rho(x)}}\mathbf{B}_1\mathbf{h}_{\mathsf{GID}} + \widehat{\mathbf{U}_{\rho(x)}}\mathbf{B}_3 \\
& = z_{\rho(x)} \left(\mathbf{V}_{\rho(x),1}\mathbf{k}_1 + \mathbf{V}_{\rho(x),2}\mathbf{k}_2 + k_3\mathbf{V}_{\rho(x),3} + \mathbf{U}_{\rho(x),1}\mathbf{h}_{\mathsf{GID}} + \mathbf{U}_{\rho(x),3} \right) \\
& \quad + \widehat{\mathbf{V}_{\rho(x),1}}\mathbf{k}_1 + \widehat{\mathbf{V}_{\rho(x),2}}\mathbf{k}_2 + k_3\widehat{\mathbf{V}_{\rho(x),3}} + \widehat{\mathbf{U}_{\rho(x),1}}\mathbf{h}_{\mathsf{GID}} + \widehat{\mathbf{U}_{\rho(x),3}} \\
& = z_{\rho(x)} \left(\mathbf{V}_{\rho(x),1}\mathbf{k}_1 + \mathbf{V}_{\rho(x),2}\mathbf{k}_2 + k_3\boxed{\mathbf{V}'_{\rho(x),3}} + \mathbf{U}_{\rho(x),1}\mathbf{h}_{\mathsf{GID}} + \boxed{\mathbf{U}_{\rho(x),3}} \right) \\
& \quad + \widehat{\mathbf{V}_{\rho(x),1}}\mathbf{k}_1 + \widehat{\mathbf{V}_{\rho(x),2}}\mathbf{k}_2 + k_3\boxed{\widehat{\mathbf{V}'_{\rho(x),3}}} + \widehat{\mathbf{U}_{\rho(x),1}}\mathbf{h}_{\mathsf{GID}} + \boxed{\widehat{\mathbf{U}'_{\rho(x),3}}}
\end{aligned}
$$

where we can write $\mathbf{V}'_{\rho(x),3} = \mathbf{V}_{\rho(x),3} + \Delta$, $\mathbf{U}'_{\rho(x),3} = \mathbf{U}_{\rho(x),3} - k_3\Delta$, $\widehat{\mathbf{V}'_{\rho(x),3}} = \widehat{\mathbf{V}_{\rho(x),3}} + \widehat{\Delta}$, and $\widehat{\mathbf{U}'_{\rho(x),3}} = \widehat{\mathbf{U}_{\rho(x),3}} - k_3\widehat{\Delta}$, where $\Delta, \widehat{\Delta}$ are as defined above, that is, choose Δ such that $m_x\tau = \mathbf{c}_x^T\Delta$ and $\widehat{\Delta}$ such that $n_x\tau = \mathbf{c}_x^T\widehat{\Delta}$. Therefore, $(\mathbf{V}_{\rho(x),3}, \widehat{\mathbf{V}_{\rho(x),3}})$ and $(\mathbf{V}'_{\rho(x),3}, \widehat{\mathbf{V}'_{\rho(x),3}})$ are identically distributed as long as $(\mathbf{U}_{\rho(x),3}, \widehat{\mathbf{U}_{\rho(x),3}})$ and $(\mathbf{U}'_{\rho(x),3}, \widehat{\mathbf{U}'_{\rho(x),3}})$ are identically distributed. We show this next. Observe that since ρ is injective, hence it follows that $\mathbf{U}_{\rho(x)}, \widehat{\mathbf{U}_{\rho(x)}}$ are fresh random matrix and other than $\mathsf{sk}_{\rho(x),\mathsf{GID},z_{\rho(x)}}$, the only other place they appears are in ciphertext components $c_{3,x}, \widehat{c_{3,x}}$. For ciphertext components $c_{3,x}, \widehat{c_{3,x}}$ corresponding to all the rows of $\mathbf{M}, \mathbf{N}$, $\mathcal{A}$ can recover $\mathbf{M}_x\mathbf{W} + \mathbf{c}_x^T\mathbf{U}_{\rho(x)}$, $\mathbf{N}_x\mathbf{W} + \mathbf{c}_x^T\widehat{\mathbf{U}_{\rho(x)}}$ information

theoretically. Observe that we can write

$$
\begin{aligned}
&\mathbf{N}_x\mathbf{W} + \mathbf{c}_x^T\widehat{\mathbf{U}_{\rho(x)}} \\
&\quad= \mathbf{N}_x\begin{pmatrix}\gamma\mathbf{B}_3^{*T}\\ \mathbf{W}_{\mathsf{bot}}\end{pmatrix} + \mathbf{c}_x^T\widehat{\mathbf{U}_{\rho(x)}} \\
&\quad= \mathbf{N}_x\begin{pmatrix}\gamma\mathbf{B}_3^{*T}\\ \mathbf{W}_{\mathsf{bot}}\end{pmatrix} + \mathbf{c}_x^T\widehat{\mathbf{U}_{\rho(x),1}}\mathbf{B}_1^{*T} + \mathbf{c}_x^T\widehat{\mathbf{U}_{\rho(x),2}}\mathbf{B}_2^{*T} + \mathbf{c}_x^T\widehat{\mathbf{U}_{\rho(x),3}}\mathbf{B}_3^{*T} \\
&\quad= \mathbf{N}_x\begin{pmatrix}\gamma\mathbf{B}_3^{*T}\\ \mathbf{W}_{\mathsf{bot}}\end{pmatrix} + \mathbf{c}_x^T(k_3\widehat{\Delta})\mathbf{B}_3^{*T} + \mathbf{c}_x^T\widehat{\mathbf{U}_{\rho(x),1}}\mathbf{B}_1^{*T} + \mathbf{c}_x^T\widehat{\mathbf{U}_{\rho(x),2}}\mathbf{B}_2^{*T} + \mathbf{c}_x^T\widehat{\mathbf{U}'_{\rho(x),3}}\mathbf{B}_3^{*T} \\
&\quad= \mathbf{N}_x\begin{pmatrix}\gamma\mathbf{B}_3^{*T}\\ \mathbf{W}_{\mathsf{bot}}\end{pmatrix} + n_x k_3\tau\mathbf{B}_3^{*T} + \mathbf{c}_x^T\widehat{\mathbf{U}_{\rho(x),1}}\mathbf{B}_1^{*T} + \mathbf{c}_x^T\widehat{\mathbf{U}_{\rho(x),2}}\mathbf{B}_2^{*T} + \mathbf{c}_x^T\widehat{\mathbf{U}'_{\rho(x),3}}\mathbf{B}_3^{*T} \\
&\quad= \mathbf{N}_x\begin{pmatrix}\gamma\mathbf{B}_3^{*T}\\ \mathbf{W}_{\mathsf{bot}}\end{pmatrix} + \mathbf{N}_x\begin{pmatrix}k_3\tau\mathbf{B}_3^{*T}\\ \mathbf{0}\end{pmatrix} + \mathbf{c}_x^T\widehat{\mathbf{U}_{\rho(x),1}}\mathbf{B}_1^{*T} + \mathbf{c}_x^T\widehat{\mathbf{U}_{\rho(x),2}}\mathbf{B}_2^{*T} + \mathbf{c}_x^T\widehat{\mathbf{U}'_{\rho(x),3}}\mathbf{B}_3^{*T} \\
&\quad= \mathbf{N}_x\begin{pmatrix}(\gamma + k_3\tau)\mathbf{B}_3^{*T}\\ \mathbf{W}_{\mathsf{bot}}\end{pmatrix} + \mathbf{c}_x^T\widehat{\mathbf{U}'_{\rho(x)}} \\
&\quad= \mathbf{N}_x\begin{pmatrix}\gamma'\mathbf{B}_3^{*T}\\ \mathbf{W}_{\mathsf{bot}}\end{pmatrix} + \mathbf{c}_x^T\widehat{\mathbf{U}'_{\rho(x)}}
\end{aligned}
$$

where we can write $\gamma' = \gamma + k_3\tau$. Similarly, we can write

$$\mathbf{M}_x\mathbf{W} + \mathbf{c}_x^T\mathbf{U}_{\rho(x)} = \mathbf{M}_x\begin{pmatrix}\gamma'\mathbf{B}_3^{*T}\\ \mathbf{W}_{\mathsf{bot}}\end{pmatrix} + \mathbf{c}_x^T\mathbf{U}'_{\rho(x)}.$$

Therefore to complete this part of the proof, it suffices to argue that γ and γ' are identically distributed. This holds true because γ, k_3, τ are uniform random, that is, $\gamma, k_3, \tau \xleftarrow{\$} \mathbb{Z}_q$.

In conclusion, substituting $(\mathbf{V}_{\rho(x),3}, \mathbf{U}_{\rho(x),3}, \widehat{\mathbf{V}_{\rho(x),3}}, \widehat{\mathbf{U}_{\rho(x),3}}, \gamma)$ with $(\mathbf{V}'_{\rho(x),3}, \mathbf{U}'_{\rho(x),3}, \widehat{\mathbf{V}'_{\rho(x),3}}, \widehat{\mathbf{U}'_{\rho(x),3}}, \gamma')$ (as described above) for all rows $x \in [n]$ of matrices $\mathbf{M}, \mathbf{N}$ allows us to move from Hyb_4 to $\mathsf{Hyb}_{3,q}$.

Therefore, it follows that $\mathsf{Hyb}_{3,q}$ and Hyb_4 are statistically indistinguishable. This completes the proof of Claim 2.6.14. □

Hybrid Hyb_5**.** This is same as Hyb_4 except that the ciphertext is changed to an encryption of a random value, that is, c_0 is changed from $c_0 := \mathsf{msg}_b \cdot [\![\mathbf{t}^T\mathbf{k}]\!]_T$ to $c_0 := \zeta \cdot [\![\mathbf{t}^T\mathbf{k}]\!]_T$, where $\zeta \xleftarrow{\$} \mathbb{G}_T$.

Claim 2.6.15. Hyb_4 *and* Hyb_5 *are statistically indistinguishable.*

Proof. Similar to proof of Claim 2.4.12 □

Claim 2.6.16. *In* Hyb_5*, adversary* $\mathcal{A}$*'s winning advantage is* 0*.*

Proof. This holds because Hyb_5 contains no information about challenge messages msg_0 and msg_1. □

Thus, from Claims 2.6.5 to 2.6.16 and hybrid argument, it follows that Theorem 2.6.4 holds.

2.6.4 Compiler for MA-ABE for ASP: boosting security

We now show how to generically modify our MA-ABE for ASP construction in Section 2.6 to achieve the security model of Cini et al. [CLW25]. However, it would satisfy the same weak functionality as [CLW25]: decryptors must have keys from all authorities in ciphertext policy in order to decrypt. Nevertheless, unlike [CLW25], our approach does not require a very-selective security model or bounds on number of authorities, and it supports fully adaptive queries, including corruption.

The modified construction is obtained via a compiler consisting of two layers: an inner layer of MA-ABE for ASP with full adaptive security with Type 1 restriction (such as in Figure 2.4), and an outer layer of MA-ABE for conjunctions (implied by MA-ABE for monotone BSP, such as in Figure 2.2) with full adaptive security. In the new scheme, to encrypt a message under an ASP policy, we encrypt the message with inner scheme, then encrypt the result under policy "conjunction of all authorities in set U" using outer scheme, where U is set of authorities appearing in the ASP policy.

$\mathsf{GlobalSetup}$, $\mathsf{AuthSetup}$, KGen are run for both layers, with outputs concatenated. Decryption uses all keys from set U to first decrypt outer ciphertext, recovering the inner one, which is then decrypted using the ASP scheme. We present the formal construction in Figure 2.5.

Theorem 2.6.17. *Suppose we have the following two MA-ABE schemes:*

- inner *denoting an MA-ABE for ASP that is fully adaptively secure with Type 1 restriction (Definition 2.6.2),*
- outer *denoting an MA-ABE for conjunctions that is fully adaptively secure (Definition 2.3.6).*

Then, MA-ABE scheme for ASP presented in Figure 2.5 is fully adaptive secure with Type 2 restriction (Definition 2.6.3).

The security proof proceeds by guessing at the outset whether the adversary's challenge will avoid corrupted authorities. If yes, we reduce the security to that of inner ASP scheme; if no, to the conjunction scheme. A wrong guess leads to abort, incurring a security loss of $\frac{1}{2}$. Thus, this construction offers a clean way to match [CLW25]'s protection against corrupt authorities while retaining fully adaptivity of our techniques. We provide the proof sketch in Section 2.6.5.

$\mathsf{GlobalSetup}(1^\lambda)$:

1 : $\mathsf{igp} \leftarrow \mathsf{inner.GlobalSetup}(1^\lambda)$
2 : $\mathsf{ogp} \leftarrow \mathsf{outer.GlobalSetup}(1^\lambda)$
3 : **ret** $\mathsf{gp} := (\mathsf{igp}, \mathsf{ogp})$

$\mathsf{AuthSetup}(\mathsf{gp}, i)$:

1 : $(\mathsf{imsk}_i, \mathsf{ipk}_i) \leftarrow \mathsf{inner.AuthSetup}(\mathsf{igp}, i)$
2 : $(\mathsf{omsk}_i, \mathsf{opk}_i) \leftarrow \mathsf{outer.AuthSetup}(\mathsf{ogp}, i)$
3 : **ret** $\mathsf{msk}_i := (\mathsf{imsk}_i, \mathsf{omsk}_i), \mathsf{pk}_i := (\mathsf{ipk}_i, \mathsf{opk}_i)$

$\mathsf{KGen}(\mathsf{gp}, \mathsf{msk}_i, \mathsf{GID}, z_i)$:

1 : $\mathsf{isk}_{i,\mathsf{GID},z_i} \leftarrow \mathsf{inner.KGen}(\mathsf{igp}, \mathsf{imsk}_i, \mathsf{GID}, z_i)$
2 : $\mathsf{osk}_{i,\mathsf{GID}} \leftarrow \mathsf{outer.KGen}(\mathsf{ogp}, \mathsf{omsk}_i, \mathsf{GID})$
3 : **ret** $\mathsf{sk}_{i,\mathsf{GID},z_i} := (\mathsf{isk}_{i,\mathsf{GID},z_i}, \mathsf{osk}_{i,\mathsf{GID}})$

$\mathsf{Enc}(\mathsf{gp}, \mathsf{msg} \in \mathbb{G}_T, (\mathbf{M}, \mathbf{N}, \rho), \{\mathsf{pk}_{\rho(i)}\}_{i\in[n]})$:

1 : $\mathsf{ict} \leftarrow \mathsf{inner.Enc}(\mathsf{igp}, \mathsf{msg}, (\mathbf{M}, \mathbf{N}, \rho), \{\mathsf{ipk}_{\rho(i)}\}_{i\in[n]})$
2 : Let $U = \{\rho(i) : i \in [n]\}$ be the set of authorities appearing in the ASP policy
3 : Let $P_U =$ "conjunction of all authorities in set U" be a policy
4 : $\mathsf{oct} \leftarrow \mathsf{outer.Enc}(\mathsf{ogp}, \mathsf{ict}, P_U, \{\mathsf{opk}_i\}_{i\in U})$
5 : **ret** $\mathsf{ct} := \mathsf{oct}$

$\mathsf{Dec}(\mathsf{gp}, (\mathbf{M}, \mathbf{N}, \rho), \mathsf{ct}, \mathsf{GID}, \{\mathsf{sk}_{\rho(i),\mathsf{GID},z_{\rho(i)}}\}_{i\in[n]})$:

1 : Let $U = \{\rho(i) : i \in [n]\}$
2 : Let $P_U =$ "conjunction of all authorities in set U" be a policy
3 : $\mathsf{omsg} \leftarrow \mathsf{outer.Dec}(\mathsf{ogp}, P_U, \mathsf{ct}, \mathsf{GID}, \{\mathsf{osk}_{\rho(i),\mathsf{GID},z_{\rho(i)}}\}_{\rho(i)\in U})$
4 : $\mathsf{imsg} \leftarrow \mathsf{inner.Dec}(\mathsf{igp}, (\mathbf{M}, \mathbf{N}, \rho), \mathsf{omsg}, \mathsf{GID}, \{\mathsf{isk}_{\rho(i),\mathsf{GID},z_{\rho(i)}}\})$
5 : **ret** imsg

Figure 2.5: Compiler Construction: MA-ABE for ASP

2.6.5 Proof Sketch of Theorem 2.6.17

Consider hybrids H_0^b, H_1^b for $b \in \{0,1\}$ as follows. H_0^b is real-world encrypting msg_b. H_1^b is same as H_0^b except that the game samples uniform random $\beta \in \{0,1\}$ at the beginning, where $\beta = 1$ indicates a guess that event A will occur and $\beta = 0$ indicates a guess that event A will not occur, where event A is: the adversary will satisfy the condition "NO corrupt authority appears in challenge policy" at the end of the game. The rest of the game proceeds same as H_0^b except that the game aborts if the guess turns out to be wrong, that is, one of two things occur: "$\beta = 1$ and not event A" or "$\beta = 0$ and event A". Next, we make two claims.

Claim 2.6.18. *If there exists a p.p.t. adversary with distinguishing advantage ϵ in distinguishing between its view in H_0^0 and H_0^1, then, there exists a p.p.t. adversary with distinguishing advantage $\epsilon/2$ in distinguishing between its view in H_1^0 and H_1^1.*

Claim 2.6.19. *If the inner scheme MA-ABE for ASP is weak adaptive secure and if the outer scheme MA-ABE for conjunction is fully adaptive secure, then, for any p.p.t. adversary, the distinguishing advantage in distinguishing between its view in H_1^0 and H_1^1 is negligible.*

From Claims 2.6.18 and 2.6.19, Theorem 2.6.17 follows. All that remains to prove are Claims 2.6.18 and 2.6.19.

Claim 2.6.18 follows from the fact that $\beta \in \{0,1\}$ is sampled uniformly at random in H_1^b for $b \in \{0,1\}$.

To see Claim 2.6.19, suppose there exists an adversary that distinguishes hybrids H_1^0 and H_1^1 with non-negligible probability, then, we describe a reduction algorithm R that breaks either the weak adaptive security of MA-ABE for ASP or breaks fully adaptive security of MA-ABE for conjunction. The reduction R works as follows.

At the beginning, R samples uniform random $\beta \in \{0,1\}$.

If $\beta = 1$, reduction R simulates MA-ABE for conjunction on its own and talks to challenger C_{ASP} for the weak adaptive security game of MA-ABE for ASP. GlobalSetup and oracles AuthSetup, KGen, Corrupt are handled in the natural way. For challenge query $(m_0, m_1, (\mathbf{M}, \mathbf{N}, \rho))$, it forwards the query to the challenger C_{ASP} and obtains ciphertext ict encrypting message m_b. Then it re-encrypts ict with respect to policy "conjunction of all authorities in set U" to obtain ciphertext oct, where U is the set of authorities appearing in policy $(\mathbf{M}, \mathbf{N}, \rho)$. It returns oct to the adversary. Eventually the adversary outputs a guess bit b'. At this point if it is the case that adversary *did not* corrupt any authority appearing in the challenge ciphertext policy, then it forwards b' to the challenger C_{ASP}. Else it sends a random bit $\widetilde{b}$ to the challenger.

If $\beta = 0$, reduction R simulates inner MA-ABE for ASP with weak adaptive security on its own and talks to challenger $C_{conjunction}$ for the fully adaptive security game of outer MA-ABE for conjunction. GlobalSetup and oracles AuthSetup, KGen, Corrupt are handled in the natural way. For challenge query $(m_0, m_1, (\mathbf{M}, \mathbf{N}, \rho))$, it locally computes ciphertexts ict_0 and ict_1 encrypting m_0 and m_1 respectively and sends challenge query $(\mathsf{ict}_0, \mathsf{ict}_1,$ "conjunction of all authorities in set U") to the challenger $C_{conjunction}$ and obtains ciphertext oct encrypting ict_b, where U is the set of authorities appearing in policy $(\mathbf{M}, \mathbf{N}, \rho)$. It returns oct to the adversary. Eventually the adversary outputs a guess bit b'. At this point if it is the case that adversary *did* corrupt any authority appearing in the challenge ciphertext policy, then it forwards b' to the challenger $C_{conjunction}$. Else it sends a random bit $\widetilde{b}$ to the challenger.

Observe that when $\beta = 0$, if the adversary is admissible and the guess β is correct, then, the reduction is admissible in its game against challenger C_{ASP}. Further, when $\beta = 1$, if the adversary is admissible and the guess β is correct, then, it must be the case that there exists an honest authority appearing in the challenge ciphertext policy who did not issue any secret key. Thus the conjunction policy is not satisfied and hence the reduction is admissible in its game against challenger $C_{conjunction}$. Hence, in both cases $\beta = 0$ and $\beta = 1$, the reduction perfectly simulates the views H_1^b to the adversary, where b is as chosen by the appropriate challenger (C_{ASP} or $C_{conjunction}$). Thus, it follows that if there exists an adversary that distinguishes hybrids H_1^0 and H_1^1 with non-negligible probability, then, the reduction algorithm R that breaks either the weak

Table 2.2: Communication Efficiency: Comparison of fully adaptively secure decentralized MA-ABE in prime order groups. All schemes are based on k-MDDH assumption in prime-order asymmetric pairing groups, so we need $k \geq 1$. n denotes the number of rows in the policy matrix $\mathbf{M}$ (and also matrix $\mathbf{N}$ in case of ASP). In ciphertext

Scheme	$\lvert\mathsf{msk}_i\rvert$	$\lvert\mathsf{pk}_i\rvert$	$\lvert sk_{i,\mathsf{GID}}\rvert$	$\lvert\mathsf{ct}\rvert$	Many-use?
DKW23 [DKW23]	$18k^2\lvert\mathbb{Z}_q\rvert$	$6k^2\lvert\mathbb{G}_1\rvert$	$6k\lvert\mathbb{G}_2\rvert$	$12nk\lvert\mathbb{G}_1\rvert$	No
Ours Section 2.4	$(4k^2+6k+2)\lvert\mathbb{Z}_q\rvert$	$(4k^2+2k)\lvert\mathbb{G}_1\rvert$	$(k+1)\lvert\mathbb{G}_2\rvert$	$n(5k+3)\lvert\mathbb{G}_1\rvert$	No
Ours[4] Section 2.6	$(8k^2+12k+4)\lvert\mathbb{Z}_q\rvert$	$(8k^2+4k)\lvert\mathbb{G}_1\rvert$	$(k+1)\lvert\mathbb{G}_2\rvert$	$n(9k+5)\lvert\mathbb{G}_1\rvert$	No
CCG+23 [CCG+23]	$(12k^2+6k)\lvert\mathbb{Z}_q\rvert$	$6k^2\lvert\mathbb{G}_1\rvert$	$(4k+2)\lvert\mathbb{G}_2\rvert$	$n(10k+2)\lvert\mathbb{G}_1\rvert$	Yes
Ours Section 2.5	$(8k^2+4k)\lvert\mathbb{Z}_q\rvert$	$(4k^2+2k)\lvert\mathbb{G}_1\rvert$	$2k\lvert\mathbb{G}_2\rvert$	$n(6k+2)\lvert\mathbb{G}_1\rvert$	Yes

adaptive security of MA-ABE for ASP or breaks fully adaptive security of MA-ABE for conjunction. This completes the proof of Claim 2.6.19 and hence the proof of Theorem 2.6.17.

2.7 Efficiency Analysis: Detailed

Notations for communication costs. We denote the size of the group $\mathbb{G}_1$ as $|\mathbb{G}_1|$, the size of the group $\mathbb{G}_2$ as $|\mathbb{G}_2|$, and the size of the field $\mathbb{Z}_q$ as $|\mathbb{Z}_q|$. We denote the size of the master secret key msk_i as $|\mathsf{msk}_i|$, the size of the public key pk_i as $|\mathsf{pk}_i|$, the size of the secret key $sk_{i,\mathsf{GID}}$ as $|sk_{i,\mathsf{GID}}|$, and the size of the ciphertext ct as $|\mathsf{ct}|$. We denote the number of rows in the policy matrix $\mathbf{M}$ as n, and the number of columns in the policy matrix $\mathbf{M}$ as ℓ. We denote the set of attributes with respect to which decryption is performed as S.

Notations for computation costs. For measuring computation costs, we note that there are three different types of operations, each with different costs: group operation, exponentiation, pairing. Typically group operation is faster than the other two by 3 orders of magnitude (See Table 2.4), so we can ignore it in the comparison. We denote the number of exponentiations as #exponentiations, and the number of pairings as #pairings. For #exponentiations, $(\mathsf{count})_i$ denotes the count for group G_i where $i \in \{1, 2, T\}$.

Asymptotic efficiency of our schemes is summarized in Table 2.2 and Table 2.3. We note that the communication and computation costs in Table 2.1 are obtained by substituting $k = 1$ in Tables 2.2 and 2.3.

Lastly, we present estimates of concrete computation costs in Table 2.4. To estimate concrete computation costs, we use the BLS12-381 elliptic curve as a representative for instantiating prime-order pairing groups. For the BLS12-381 elliptic curve, we use the microbenchmarks from [Tom22]. The numbers are for BLS12-381 curve implemented in Filecoin's blstrs [fil] Rust wrapper around the popular blst [sup] library. These microbenchmarks were run on a 10-core 2021 Apple M1 Max. The computation costs in Table 2.4 are obtained by multiplying the number of operations in Table 2.1b with the corresponding microbenchmark numbers in Table 2.5.

[4]Our MA-ABE for ASP satisfies weak adaptive security as defined in Definition 2.6.2

Table 2.3: Computation Efficiency: Comparison of fully adaptively secure decentralized MA-ABE in prime order groups. All schemes are based on k-MDDH assumption in prime-order asymmetric pairing groups, so we need $k \geq 1$. n and ℓ denote the number of rows and columns in the policy matrix $\mathbf{M}$ (and also matrix $\mathbf{N}$ in case of ASP) respectively. S denotes the set of attributes with respect to which decryption is performed. For each algorithm, we specify a tuple of three values: #group ops, #exponentiations, #pairings. For #group ops and #exponentiations, $(\text{count})_i$ denotes the count for group G_i where $i \in \{1, 2, T\}$.

Scheme	AuthSetup	KGen	Enc	Dec
DKW23 [DKW23]	$(18k^3)_1$, $(18k^3)_1$, 0	$(18k^2+3k)_2$, $(18k^2)_2$, 0	$(12nk^2+6nk\ell+3k^2+6nk)_1$, $(12nk^2+6nk\ell+3k^2+6(\ell-1)k)_1+(3k)_T$, $3k$	$(3k\|S\|)_2+((12k+4)\|S\|)_T$, $(\|S\|)_T$, $12k\|S\|$
Ours Section 2.4	$(4k^3+6k^2+2k)_1$, $(4k^3+6k^2+2k)_1$, 0	$(2k^2+3k+2)_2$, $(2k^2+3k+2)_2$, 0	$(n(5k^2+7k+2))_1+(1)_T$, $(n(5k^2+7k+2))_1+(1)_T$, 0	$((2k+1)\|S\|)_1+((3k+5)\|S\|+1)_T$, $((2k+1)\|S\|)_1+(\|S\|)_T$, $(3k+3)\|S\|$
Ours Section 2.6	$(8k^3+12k^2+4k)_1$, $(8k^3+12k^2+4k)_1$, 0	$(2k^2+3k+2)_2$, $(2k^2+3k+2)_2$, 0	$(n(9k^2+13k+4))_1+(1)_T$, $(n(9k^2+13k+4))_1+(1)_T$, 0	$((2k+1)\|S\|)_1+((7k+7)\|S\|+1)_T$, $((6k+3)\|S\|)_1+(\|S\|)_T$, $(3k+3)\|S\|$
CCG+23 [CCG+23]	$(12k^3+6k^2)_1$, $(12k^3+6k^2)_1$, 0	$(12k^2+9k)_2$, $(12k^2+6k)_2$, 0	$(n(10k^2+8k))_1$, $(n(10k^2+8k))_1+(3k)_T$, $3k$	$(3k\|S\|)_2+((10k+6)\|S\|)_T$, $(\|S\|)_T$, $(10k+2)\|S\|$
Ours Section 2.5	$(8k^3+4k^2)_1$, $(8k^3+4k^2)_1$, 0	$(4k^2+2k+1)_2$, $(4k^2+2k+1)_2$, 0	$(n(6k^2+6k+2))_1+(1)_T$, $(n(6k^2+6k+2))_1+(1)_T$, 0	$((2k+1)\|S\|)_1+((4k+4)\|S\|+1)_T$, $((2k+1)\|S\|)_1+(\|S\|)_T$, $(4k+2)\|S\|$

Table 2.4: Computation Efficiency over BLS12-381 elliptic curve: Comparison of fully adaptively secure decentralized MA-ABE in prime order groups. All schemes are instantiated from k-MDDH assumption in prime-order pairing groups, where $k = 1$. n and ℓ denote the number of rows and columns in the policy matrix $\mathbf{M}$ (and also matrix $\mathbf{N}$ in case of ASP) respectively. S denotes the set of attributes with respect to which decryption is performed.

Scheme	AuthSetup	KGen	Enc	Dec
DKW23 [DKW23]	1.296ms	2.448ms	$(0.432n\ell + 0.864n + 0.432\ell + 2.742)$ms	$(6.332\|S\|)$ms
Ours Section 2.4	0.864ms	0.952ms	$(1.008n + 0.5)$ms	$(3.632\|S\|)$ms
Ours Section 2.6	1.728ms	0.952ms	$(1.876n + 0.5)$ms	$(4.064\|S\|)$ms
CCG+23 [CCG+23]	1.728ms	2.856ms	$(1.296n + 4.458)$ms	$(6.332\|S\|)$ms
Ours Section 2.5	0.826ms	0.952ms	$(1.008n + 0.5)$ms	$(3.632\|S\|)$ms

Table 2.5: Computation time of various operations: We recall the computation time of various operations on the BLS12-381 elliptic curve from [Tom22]. The numbers are BLS12-381 curve implemented in Filecoin's blstrs [fil] Rust wrapper around the popular blst [sup] library. These microbenchmarks were run on a 10-core 2021 Apple M1 Max.

Operation	$\mathbb{G}_1$	$\mathbb{G}_2$	$\mathbb{G}_T$
group op	565ns	1484ns	1617ns
exponentiation	72¯s	136¯s	500¯s
pairing	486¯s		

Chapter 3

Fair Exchange

In this chapter, we explore the applications of functional encryption and homomorphic encryption in constructing fair exchange protocols.

Consider a scenario where a buyer wants to pay a seller in exchange for data, but neither party trusts the other to fulfill their side of the bargain. Fairness requires that either both parties receive what they expect, or neither does. To address these challenges, the fundamental problem of fair exchange has been extensively studied in cryptography, and unfortunately, it suffers from impossibility results in two-party settings without a trusted third party. Recently, blockchains have been used as a trusted third party to enable fair exchange in a decentralized manner. In more advanced scenarios, such as a buyer paying to train an AI model on a proprietary dataset (such as, patient records), the exchanged items are not simple digital assets with an "all-or-nothing" flavor, but rather insights derived from sensitive data. Crucially, in addition to fairness, the seller also has a privacy requirement: it wants to ensure that the buyer only learns a specific function evaluation on the data (such as, fine-tuned model parameters) without revealing the entire dataset. While smart contract-based approaches can enforce fairness and enable such advanced functionality, they suffer from inefficiency, high transaction costs, and lack of privacy for sensitive data. Moreover, they are incompatible with systems that do not support complex smart contracts. Thus, there is a need for efficient cryptographic protocols that can achieve fair exchange with privacy, and compatibility with systems that do not support complex smart contracts.

We define a new primitive called *Functional Adaptor Signatures (FAS)*, and present protocols using it that meet all of the above requirements. Notably, the protocol involves posting only a single signature on-chain for the payment, thus reducing on-chain costs to the bare minimum. We construct FAS in two different ways: (a) using functional encryption [VST24a], and (b) using homomorphic encryption [VGTS26]. The FE-based construction supports linear functions, while the HE-based construction supports arbitrary

polynomial functions. We additionally define the notion of *Functional Fair Exchange (FFE)*, which captures the requirements of fair exchange with privacy, and show how to construct FFE using FAS. Our FFE formalization generalizes the FAS-based fair-exchange protocols, providing a standalone target for future work to explore alternative constructions or different trade-offs, such as improved efficiency at the cost of more interaction.

As the lead author of [VST24a], I was responsible for all aspects of the work, including defining FAS, designing the FE-based FAS constructions, proving its security, instantiating FE-based FAS from groups of prime-order and lattices, implmeneting the group-based instantiation. As the co-lead author of [VGTS26], I was responsible for all aspects of the work except the BGN-based instantiation and its implementation, and the implementation of SNARK-based strawman FAS construction. In particular, I defined FFE, designed the FAS-based FFE construction, designed the HE-based FAS construction, proved its security, designed the TFHE-based FAS instantiation and its implementation and optimizations, and designing the linear HE (exponential El-Gamal) based FAS instantiation and its implementation and optimizations. I present my contributions in detail in the rest of this chapter.

3.1 Introduction

The shift from centralized Web2 to decentralized blockchain-based Web3 solutions has transformed digital goods trading. Smart contracts have been pivotal in this transition, and they facilitate seamless transactions between sellers and buyers. Imagine a seller offering a solution to a computational puzzle or a problem for sale. Typical smart contracts let the seller specify the terms of their digital offering, referred to as an *advertisement*, creating a transparent and accessible mechanism for buyers. The buyers lock coins to the contract as an expression of interest to buy the solution. Now the seller publishes the solution in the form of a transaction on the blockchain that invokes the contract. The contract executes a validation procedure to verify the solution's correctness, and if successful, transfers the locked coins to the seller. If the seller fails to respond with a correct solution, the contract refunds the buyer after the expiry of a timeout. This template underpins various digital trading scenarios including Hash Timelock Contracts [PD], Bridges [bri], NFT sales [WN22], smart contract-based offline services [cha], and e-commerce [KRA+22].

Despite the advantages of smart contracts, their current implementation faces significant challenges:

- *Cost and Efficiency*: Sellers often incur hefty fees for posting advertisements on smart contracts and verifying secret solutions, resulting in higher transaction costs. Ethereum measures these costs in gas, and many popular smart contract applications have significantly higher gas costs compared to regular user-to-user payment transactions [liv].

- *Privacy*: Openly disclosing the secret solution compromises privacy. Encrypting values on the contract to address this necessitates more expensive and intricate cryptographic tools, further exacerbating costs. Further, the approach affects the fungibility[1] of the system as pointed out in many previous works [EFH+21, TBM+20, TM21, TMMS22, HLTW24].

- *Compatibility*: Finally, the method relies on *complex* smart contracts, rendering itself incompatible with prominent blockchains like Bitcoin. This poor adaptability hampers scalability in the broader Web3 ecosystem.

Adaptor Signatures. As the community delves deeper into enhancing the efficiency and simplicity of smart contracts, cryptographic tools like adaptor signatures [MMSS+19, EEE20a, AEE+21, EFH+21, TMMS22, HLTW24, QPM+23] have emerged as promising solutions. Adaptor signatures help model a blockchain-based fair exchange between a buyer for its signature σ (on a transaction) and an NP witness x that is known to the seller. More formally, the seller first samples an NP statement X along with its witness x. The statement X binds the seller to its witness which it publishes to advertise its intent to sell x.[2] Now, adaptor signatures offer four algorithms $\mathsf{PreSign}, \mathsf{PreVerify}, \mathsf{Adapt}, \mathsf{Ext}$, that work as follows. The buyer using the pre-sign algorithm $\mathsf{PreSign}$ first generates a pre-signature $\tilde{\sigma}$ on the payment transaction m w.r.t. the buyer's signing key sk and seller's statement X. The seller verifies the validity of $\tilde{\sigma}$ using $\mathsf{PreVerify}$ algorithm. Then, using the Adapt algorithm, the seller adapts $\tilde{\sigma}$ into a full signature σ on m using its witness x. To redeem the transaction m, the seller posts σ on the blockchain. Central to the efficacy of adaptor signatures is their unique property that allows the buyer, with access to the pre-signature $\tilde{\sigma}$ and the posted signature σ, to efficiently *extract* the secret solution x using the algorithm Ext, thus completing the fair exchange. This innovative approach addresses the drawbacks of traditional smart contracts:

- *Improved efficiency and privacy:* Adaptor signatures require only a single transaction and signature to be posted on the blockchain, reducing transaction costs and enhancing efficiency. Moreover, with buyers locally extracting secrets, transactions resemble regular payments, thereby improving the privacy and fungibility of coins in the system.

- *Enhanced compatibility:* Signature verification, a universally supported operation, ensures compatibility across all blockchain systems, making adaptor signatures-based digital sales compatible with the broader Web3 landscape.

[1] A blockchain system is said to be fungible if all units/coins in the system have the same value, independent of their history.
[2] Typically, this step is not explicitly stated in the formalization of adaptor signatures. We adopt this extension to aid our functional adaptor signature formalization.

Looking ahead: Granular and functional sale of digital information. However, while adaptor signatures offer a promising solution for digital transactions, they come with certain limitations. They operate on an all-or-nothing basis, revealing the entire secret upon successful transactions or learning nothing in case the seller aborts. This lack of granularity contrasts with the concept of *functional encryption (FE)* [BSW11], where recipients can decrypt and learn specific functions applied to the encrypted message instead of the complete message. Exploring granularity not only enables more nuanced and functional sales of digital information but also ensures the privacy of the seller's data. For instance, the seller holding medical records might desire to safeguard specific functions of the records, adhering to medical data privacy regulations like the Health Insurance Portability and Accountability Act (HIPAA) in the United States. The privacy requirement here is that the buyer learns only the specific function for which the payment is made, ensuring compliance with legal frameworks. Below we expand on this intuition and give two illustrative examples of real-world scenarios where functional sales of digital information can be critical.

Application 1: Medical information. In this setting the seller holds a medical database and is authorized to sell functions of the database by the patients or appropriate authorities. For instance, the buyer who might be an insurance firm, can query some aggregate/statistics of the demography in the database. If it's a research organization, more complex functions like correlations of pre-existing medical conditions and cancer can be queried. The seller has the financial incentive to provide the information, which, in the long run, can contribute greatly to medical advancements.

Application 2: Fine-Tuning ML Models. A buyer holds a pre-trained ML model—say, a *Stochastic Gradient Descent* (SGD)-based breast cancer classifier [Ama93]—and seeks to fine-tune it using a high-quality, sensitive dataset owned by a seller (example, a medical institution). The model must be refined using the seller's data, without exposing that data to the buyer. Similar needs arise in private inference, image classification, encrypted search, and collaborative training tasks.

Therefore, we ask the following question,

Can we facilitate the functional sale of digital information using adaptor signatures?

More concretely, let the seller hold secret data x, and the buyer wants access to a function f. *Can we expand the functionality of adaptor signatures such that, at the end of the exchange, the seller obtains a payment transaction and a signature from the buyer, and the buyer learns $f(x)$, and not the entirety of x?*

Our works [VST24a, VGTS26] answers this question in the affirmative. [VST24a] focuses on the restricted function class of linear functions (where f is a linear function), arguing that they are already widely applicable. We show that practical constructions using lightweight cryptographic tools can be achieved for linear functions, with some insights into the challenges of achieving a more expressive class of

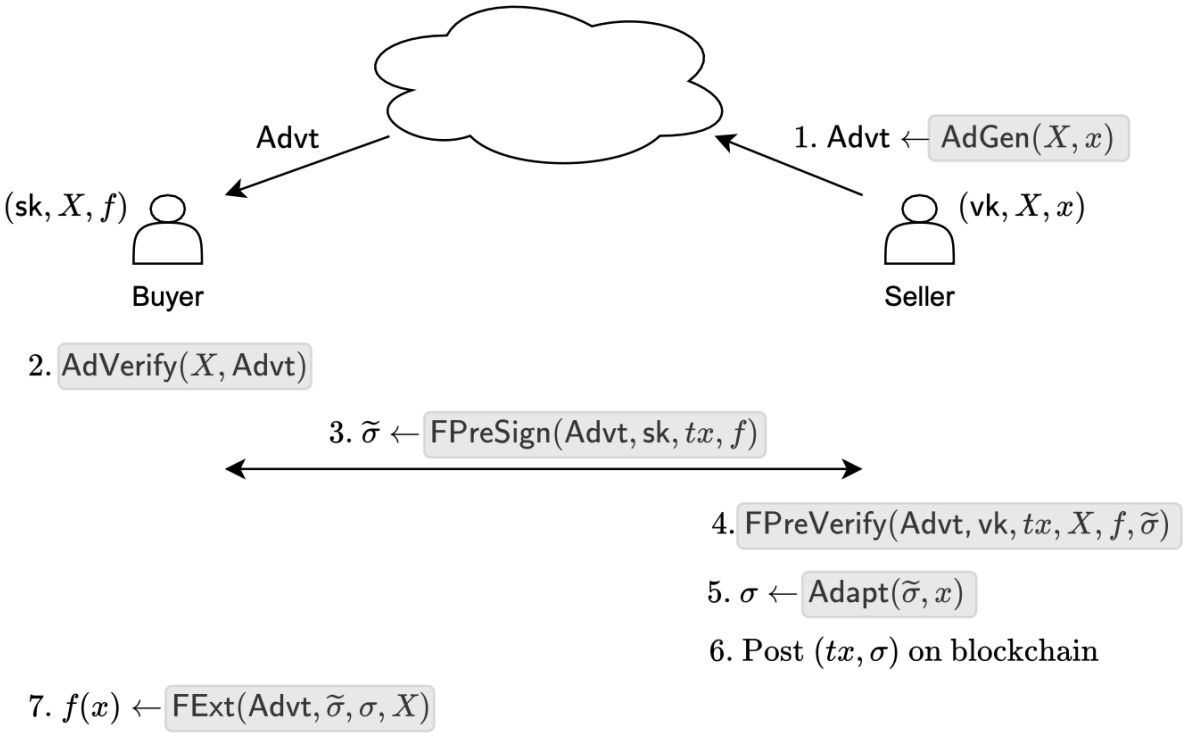


Figure 3.1: Functional payments via simplified FAS interface.

functions. [VGTS26] addresses the ultimate goal of supporting sale of general functions by presenting constructions using fully homomorphic encryption (FHE). Further, for linear functions, we improve the efficiency compared to [VST24a] by a significant margin using linear homomorphic encryption. We discuss the contributions of both works in the next two sections.

3.1.1 Our Contributions in [VST24a]

Below we summarize the contributions of this work.

New cryptographic primitive - Functional Adaptor Signatures. To answer the above question affirmatively, we formally introduce a novel cryptographic primitive called *functional adaptor signatures (FAS)* (in Section 3.5). It is similar in functionality to standard adaptor signatures, except that it is additionally parameterized by a family of functions $\mathcal{F}$. This addition introduces new interfaces to FAS; we describe these below in the context of a digital trade for ease of understanding. A pictorial description is given in Figure 3.1.

The seller of the trade generates an advertisement ad using the FAS interface AdGen, that embeds the statement X and the corresponding secret witness x, where $(X, x) \in R$ for some NP relation R. Anyone can publicly verify the well-formedness of ad using the AdVerify algorithm. The buyer executes the functional

pre-sign algorithm $\mathsf{FPreSign}_2$ to generate a pre-signature $\widetilde{\sigma}$ on the message m (the payment transaction, denoted as tx in Figure 3.1) for the statement X and a function f. $\mathsf{FPreVerify}_2$ algorithm helps the seller to verify if $\widetilde{\sigma}$ is valid with respect to the message m, statement X and function f. If the pre-signature is valid, the seller using the Adapt algorithm and the witness x can adapt $\widetilde{\sigma}$ into a signature σ on the message m. The seller may now post the transaction and the signature on the blockchain to get paid. Finally, the buyer can efficiently extract the value $y = f(x)$ using the functional extract algorithm FExt given access to ad, $\widetilde{\sigma}$, and σ. With the above outline of FAS, we can see that we gain the same advantages of adaptor signatures over smart contracts for digital trading, while also achieving the desired granularity.

Definitional framework. Recent works [DOY22, GSST24] have highlighted the inherent complexities in designing the security framework for even standard adaptor signatures, with many subtle nuances that require a lot of care. In this work, we overcome these challenges and present a comprehensive definitional framework for the security of FAS (in Section 3.5). We establish the formal foundations of FAS, constituting a substantial contribution of this work.

Broadly speaking, for a secure FAS, we are required to upgrade the security definitions of standard adaptor signatures to handle additional constraints posed by the functional aspects of the primitive. Moreover, we introduce additional novel security properties for a secure FAS to satisfy, among which *witness privacy* is a prominent one. This property captures the leakage a malicious buyer learns about x beyond $f(x)$. To formalize this, we present three different notions of witness privacy for FAS, akin to the different privacy guarantees of cryptographic proofs. To be more specific, we present (1) *witness hiding*, where the malicious buyer after learning $f(x)$ cannot output the secret witness x, (2) *witness indistinguishable*, where the malicious buyer learns $f(x)$ and cannot distinguish between two witnesses $x = x_0$ and $x = x_1$ (for the statement X) as long as $f(x_0) = f(x_1)$, and (3) *zero-knowledge*, where the malicious buyer learns no other information about x, other than $f(x)$.

Efficient constructions. From a practical perspective, we give several efficient constructions of FAS for the class of linear functions (like aggregates, statistics, kernel computation in ML, etc.) that are compatible with standard signatures like Schnorr and ECDSA. Notably, these signature schemes are widely used in blockchain systems for transaction verification, thus making our constructions ready for deployment in current systems. Towards a post-quantum instantiation of FAS, when the buyer has linearly independent function queries, we also present a FAS construction compatible with a variant of the lattice-based signature scheme of Lyubashevsky [Lyu12]. Later in Section 3.2.4, we discuss how the linear-independency requirement can be relaxed with minor trade-offs in efficiency.

Most importantly, the key conceptual contribution of our work is the surprising connection between two

seemingly unrelated cryptographic tools, namely, functional encryption [BSW11] that has been extensively studied for over a decade and a relatively new primitive, adaptor signatures. Concretely, we construct FAS for a family of inner-product functions $\mathcal{F}_{\mathsf{IP}}$ using three lightweight building blocks:

(i) a functional encryption scheme IPFE for $\mathcal{F}_{\mathsf{IP}}$,

(ii) an adaptor signature scheme AS w.r.t. the digital signature scheme DS and some hard relation R_{IPFE} (related to IPFE),

(iii) NIZK argument system for NP.

A nice feature of our construction is that we show it satisfies the zero-knowledge style witness privacy assuming only selective, IND-security of IPFE and zero-knowledge of NIZK.

We show two ways to instantiate the construction:

- **Prime-order groups (Section 3.7).** Using a varaint of the DDH-based IPFE scheme by Abdalla et al. [ABDP15], we get R_{IPFE} is the discrete log relation. So, we use Schnorr adaptor signature [AEE+21] for AS.

- **Lattices (Section 3.8).** Using a variant of the LWE-based IPFE scheme by Agrawal et al. [ALS16], we get R_{IPFE} is the short integer solution (SIS) relation. So, we use a variant of the post-quantum adaptor signature [EEE20a] for AS.

Throughout the chapter, for conceptual clarity, we abstract out the relation R to be any NP relation, and make black-box use of a NIZK [PS19] for relation R. However, we emphasize that in both cases, the NIZK can be efficiently instantiated depending on the exact application (i.e., for a concrete relation R) and plugged in without affecting other components. Lastly, for a weaker witness privacy notion of witness-indistinguishability, we present a round-optimal construction (Sections 3.2.5 and 3.11).

Performance evaluation. To assess the practicality of our FAS, we provide an open-source implementation [VST24b] of our prime-order group-based instantiation satisfying zero-knowledge style witness privacy in Python. The details of our performance evaluation are given in Section 3.9. We also perform a series of benchmarks on our FAS scheme for a wide range of parameter settings. Our results show that our scheme is practical for a wide variety of real-world scenarios. For instance, on a MacBook Pro, for 300KB size witness, the time taken for pre-signing is 0.344 seconds, pre-verification is 0.424 seconds, adapt is 0.035 seconds, functional-extract is 1.025 seconds. We share more details of our findings in Section 3.9.

Applicability of Linear Functions. Linear functions although restricted, allow us to capture many scenarios including learning statistical information (like mean, average, weighted linear functions, select entries) of

records in a medical database, employee records of an organization, weather data, or ecological records of endangered species. Linear functions also allow a buyer to measure the proximity of a vector he holds with the secret vector of the seller. Such proximity measures are quite fundamental in Machine Learning where they are referred to as computing the kernel of two vectors or graphs in the case of [ACR17].

3.1.2 Our Contributions in [VGTS26]

In this work, we make a substantial leap in the capabilities of functional adaptor signatures by introducing the *first* constructions that support *non-linear* and expressive function classes. We present *three* new secure FAS constructions: one that significantly improves efficiency for linear functions, and two that extend FAS to handle degree-2 and higher computations for the first time. These constructions follow from a general framework that combines *homomorphic encryption* (HE) with Schnorr adaptor signatures in a technically novel and modular way, expanding the design space of FAS beyond prior work. To demonstrate deployability, we undertake a significant implementation effort that required around 10,000 LoC across several cryptography libraries including PBC [Lyn] and Zama's suite of *fully homomorphic encryption* (FHE) libraries [tfh,cona,conb]. Finally, we benchmark our constructions across *five* application settings – database analytics, fine-tuning SGD classifier, ML inference through second-order factorization machines, and high-degree multivariate polynomial evaluation. Specifically, we have the following contributions:

Contribution 1. We propose a novel construction paradigm that builds FAS for any function class F while relying on HE for the same class, Schnorr adaptor signatures, and a new *cut-and-choose* style sigma protocol that proves plaintext equality across HE ciphertexts and Elgamal PKE ciphertexts.[3] This framework, described in Section 3.12, represents our core conceptual contribution, offering a simpler and more intuitive alternative to the *functional encryption* (FE)-based framework proposed in [VST24a]. By building on extensively implemented HE schemes, our paradigm benefits from practical advancements in HE, addressing the limitations of FE constructions, which remain less efficient and less mature for comparable function classes.

Contribution 2. Instantiating the HE scheme in our template with existing *fully* homomorphic encryption schemes [BGV11,GSW13] gives us a concrete scheme for FAS for *general functions* (see Section 3.13). To enable ML applications, we instead directly work with Zama's suite of FHE libraries that implement the Torus-FHE scheme [CGGI20]. However, this choice entailed significant implementation effort due to Zama's limited exposure to low-level FHE kernels. Specifically, for seller efficiency, our FAS construction requires accessing the random coins behind homomorphically evaluated ciphertexts, which unfortunately

[3]We require a natural randomness recovery property from our HE scheme, satisfied by several existing schemes: the randomness of a homomorphically evaluated ciphertext can be efficiently derived from the function and the randomness of the input ciphertexts. We formalize this via an algorithm REval.

is inaccessible in the current implementation of TFHE-rs library [tfh]. We implement this feature (8000+ LoC), which may be of independent interest. All of our code is available at [VGTS25].

As benchmark applications for general functions, we focus exclusively on ML-based applications. Firstly, we consider the setting where a buyer wants to learn high-degree polynomial evaluations of seller's secret input (Section 3.13.1). For a host of ML inference tasks that use transformers and neural network models, the underlying non-linear computations such as softmax and GeLU are often approximated through high-degree multivariate polynomials: degree 4 approximation of softmax [CZK+23], and degree 3 approximation of GeLU [ZYH+24, LHG+23, DLZ+23]. Given the prevalence of such high-degree polynomials in ML inference tasks, we believe polynomial evaluation is a representative application for FAS. Secondly, we demonstrate (in Section 3.13.1) the applicability of our FHE-based FAS scheme for fine-tuning a low-quality SGD classifier (Application #2 in Section 4.1).

Contribution 3. We explore the simpler case of degree-2 functions, which frequently arise in real-world applications, particularly in medical analytics and machine learning. For instance, analytics queries in medical settings can be encoded as weighted standard deviation functions, where the buyer specifies weights $w_1, \ldots, w_\ell$ for the seller's database $\vec{x}$, and computes the standard deviation of weighted samples $w_i \cdot \vec{x}[i]'s$. Similarly, inference using second-order factorization machines [Ren10]—a key method in machine learning to predict first- and second-order feature correlations—can also be cast as degree-2 functions.

To enable these applications, we optimize our construction by leveraging the Boneh-Goh-Nissim [BGN05] HE scheme, which supports degree-2 homomorphism. Additionally, we develop a *new* zero-knowledge proof of plaintext equality for Elgamal and BGN ciphertexts, generalizing the work of Chase et al. [COPZ22] to composite-order groups used in the BGN scheme. These optimizations target practical implementations, particularly for medical analytics (example, weighted standard deviation computations) and secure sales of second-order factorization machine inferences. We skip this contribution in this thesis and refer the reader to [VGTS26] for details.

Contribution 4. To further demonstrate the power of our new HE-based template, we revisit the case of linear functions. In addition to rigid algebraic structural requirements between *functional keys* of the underlying functional encryption, the functional extraction procedure in [VST24a] involves an inefficient discrete-logarithm computation. In contrast, our construction outperforms the constructions in [VST24a] by up to 10x in total run time (as shown in Figure 3.3) for the same application benchmark when instantiated with a linearly homomorphic encryption scheme, where the decryption operation only involves group multiplication (and not the expensive discrete-logarithm computation).

Contribution 5. As an additional conceptual contribution, we formalize the notion of *functional fair exchange* as a cryptographic interactive protocol involving the seller, the buyer, and a wallet-based token

system (Section 3.14). We rigorously define *buyer-fairness* and *seller-fairness* using the simulation-based framework. Our definitions are strictly more general in that they capture functional exchanges, compared to the ones in [BGKS25, TSZ+24] which only handle simple all-or-nothing payment-data exchanges. While our FAS constructions imply functional fair exchange (as we demonstrate), we deliberately decouple this abstraction from FAS to define a concrete and standalone target. This separation opens the door to future protocols that achieve functional fair exchange directly, rather than through FAS machinery.

3.2 Technical Overview 1: [VST24a]

In this overview, we present our approach to fair payments for learning linear functions of a seller's witness.

To describe our techniques, we use the following setup: let p and ℓ be integers where p is prime and L be an NP language containing statements X with witness $\mathbf{x} = (x_1, \ldots, x_\ell) \in \mathbb{Z}_p^\ell$. Our goal is to support linear functions over $\mathbf{x}$. Throughout this chapter, we represent such linear functions by vectors $\mathbf{y} = (y_1, \ldots, y_\ell) \in \mathbb{Z}_p^\ell$ and the corresponding evaluation on $\mathbf{x}$ by the inner-product of the vectors $\mathbf{x}$ and $\mathbf{y}$ modulo p. That is, $f_{\mathbf{y}}(\mathbf{x}) = \langle \mathbf{x}, \mathbf{y} \rangle = \sum_{i=1}^{\ell} x_i \cdot y_i \mod p$. [4]

3.2.1 Functional Adaptor Signatures for Fair Functional Payments

We have a fair exchange scenario where the seller has a witness $\mathbf{x}$ which is a vector in $\mathbb{Z}_p^\ell$, and only desires to sell some linear functions $\mathbf{y}$ on input $\mathbf{x}$. To enable such an exchange FAS, like adaptor signatures, involves algorithms like $\mathsf{FPreSign}_2$ and $\mathsf{FPreVerify}_2$, Adapt, and FExt. However, the key distinction lies in the fact that $\mathsf{FPreSign}_2$ and $\mathsf{FPreVerify}_2$ are defined w.r.t. a function $\mathbf{y}$ from the buyer, while the functional extraction algorithm FExt, returns $f_{\mathbf{y}}(\mathbf{x})$ instead of $\mathbf{x}$ itself, ensuring the buyer learns only the function's result.

Security of FAS

We consider two cases, where the adversary can corrupt either the seller or the buyer. Similar to standard adaptor signatures, when the seller is corrupt, FAS must mainly guarantee (1) *unforgeability*, where an adversary who doesn't know the witness cannot forge honest buyers' signatures, and (2) *witness extractability*, where the adversary cannot publish a buyer's signature such that when the buyer tries to extract using FExt, it gets $z' \neq f_{\mathbf{y}}(\mathbf{x})$. [5] Crucially, in the case of FAS, the adversary is allowed to get many functional pre-signatures for any choice of function $\mathbf{y}$.

[4] While our lattice-based instantiation is for computing inner products $\mod p$, the group-based instantiation is for computing integer inner products with bounded outputs, i.e., $f_{\mathbf{y}}(\mathbf{x}) \in \{0, \ldots, B\}$ for some apriori fixed bound $B \ll p$.

[5] We also require *advertisement soundness* and *pre-signature validity* properties against a corrupt seller, and defer their discussion to Section 3.5.

On the other hand, when the buyer is corrupt, similar to standard adaptor signatures, FAS must guarantee *pre-signature adaptability*, where if the adversary outputs a valid functional pre-signature, then adapting it using a valid witness $\mathbf{x}$ should result in a valid signature under the buyer's key. Most importantly, FAS must guarantee protection of the seller's witness $\mathbf{x}$ which we capture in the notion called *witness privacy*.

In more detail, FAS satisfies *zero-knowledge* witness privacy if a malicious buyer learns no more information about $\mathbf{x}$ than $f_{\mathbf{y}}(\mathbf{x})$ after the interaction. To formally define this in Section 3.5, we carefully borrow formalism from the related privacy notion of zero-knowledge for cryptographic proof systems [GMR89]. Informally, for every malicious buyer (that can sample its signing key), we require the existence of an efficient p.p.t. simulator Sim that can simulate the buyer's view only using $f_{\mathbf{y}}(\mathbf{x})$. Philosophically, the existence of such an efficient simulator ensures that whatever the buyer learned from interacting with the seller, it could have on its own from $f_{\mathbf{y}}(\mathbf{x})$ by running in polynomial time. While our notion shares similarities and is inspired by cryptographic proof systems, we need additional care to correctly model the fact that the buyer is allowed to ask for multiple functions which, moreover, can be adaptively chosen. To gain a comprehensive understanding of witness privacy, we also introduce two natural relaxed variants referred to as *witness hiding* and *witness indistinguishability* in Definitions 3.5.11 and 3.5.12, and study their relation with zero-knowledge witness privacy . Interestingly, in Section 3.2.5, we show that the relaxation to witness indistinguishability enables a round optimal construction.

For the rest of this overview, we will focus on building an efficient FAS centered around the witness privacy guarantee given it is the unique property of FAS compared to standard adaptor signatures. Moreover, our focus will be on the strongest notion of zero-knowledge witness privacy.

Strawman Construction of FAS

To build FAS, one can combine Schnorr adaptor signature (discussed above) along with a general-purpose non-interactive zero-knowledge (NIZK) proof. We present this strawman solution below and highlight a key efficiency challenge that serves as the starting point of our construction.

Recall the protocol flow in Figure 3.1. We describe how $\mathsf{ad}, \widetilde{\sigma}, \sigma$ are computed and the functional extraction process. In the advertisement phase, the seller publishes $\mathsf{ad} = (\mathsf{ct}, \pi)$, where ct is a semantically-secure public-key encryption of the witness $\mathbf{x}$ and NIZK proof π that certifies ct encrypts a witness for the statement X. In the exchange phase, the pre-signature $\widetilde{\sigma}$ is computed interactively among the buyer and the seller as follows. (i) The buyer sends f to the seller. (ii) The seller computes a secret-key encryption ct_z of the requested evaluation $z = f(\mathbf{x})$ using a fresh secret-key k. Then, the seller encodes k in $\mathbb{Z}_p$ and *engages with the buyer in an adaptor execution to sell the witness for the discrete logarithm of group element K, where*

$K = g^k$ *for some generator* g *of the group* $\mathbb{G}$. That is, the seller sends $(\mathsf{ct}_z, K, \pi_z)$ to the buyer, where NIZK proof π_z certifies ct_z encrypts $f(\mathbf{x})$ using the key k where $\mathbf{x}$ is encrypted in ct. The buyer treats K as the Schnorr adaptor statement and computes a pre-signature $\widetilde{\sigma}$ on the transaction message m if and only if the NIZK proof π_z verifies. On obtaining $\widetilde{\sigma}$, the seller adapts it into a valid signature σ on m using the key k as the witness. For functional extraction, the buyer runs Ext algorithm of the Schnorr adaptor signature to extract k from $(\widetilde{\sigma}, \sigma)$ and then decrypts ct_z using it to recover the function evaluation z.

The desired privacy of FAS is attained by relying on the zero-knowledge property of both NIZK proofs and the semantic security of ct. That is, the seller side of interaction can be efficiently simulated just from the evaluation $z = f(\mathbf{x})$. The security against a malicious seller relies on the security of the underlying adaptor signature scheme and the soundness of the NIZK protocol.

While this approach enables fair functional payments, it requires that the seller compute NIZK proofs for every $\mathsf{FPreSign}_2$ interaction with a buyer. The seller's computation in computing these proofs (and their size) grows *polynomially* in the witness size $\mathbf{x}$ and is proportional to the complexity of the function f. While the proof sizes can be reduced by relying on general-purpose ZK-SNARKs [BCCT13], the proving cost growing super-linearly in $|\mathbf{x}|$ will become prohibitively expensive even for moderately sized databases $\mathbf{x}$. This cost also affects the scalability of the solution at the application layer. For instance, if the seller is dealing with several thousand buyers each with a different function, the seller will be computationally strained, leading to delays and a potential Denial of Service (DoS) attack vector via malicious buyers. A more efficient solution would allow (a) the seller's computation to grow linearly in the size of $\mathbf{x}$ and (b) the seller's communication to be proportional to the size of the evaluation z which would be significantly smaller than $\mathbf{x}$ itself. As we explain below, we achieve these efficiency targets without relying on NIZKs, which ensures we use underlying cryptographic primitives in a *black-box* manner.

3.2.2 Our Techniques: Functional Encryption + Adaptor Signatures

The blueprint of our FAS construction is to combine adaptor signatures with another cryptographic tool called functional encryption (FE). FE was introduced as an enrichment of any (public-key) encryption scheme that allows for fine-grained decryption. In particular, in addition to algorithms $(\mathsf{Setup}, \mathsf{Enc}, \mathsf{Dec})$ which are typical to any encryption scheme, a FE scheme for a function class $\mathcal{F}$ provides an additional functional key-generation algorithm KGen. The algorithm takes as input the master secret-key msk and function $f \in \mathcal{F}$, and outputs a functional secret-key sk_f such that decrypting any encryption of $\mathbf{x}$ (w.r.t. msk) with sk_f reveals $f(\mathbf{x})$ and no more. As discussed earlier, this notion of strong privacy where no information about $\mathbf{x}$ is leaked other than $f(\mathbf{x})$ is formalized using the *simulation* paradigm.

For simplicity, we will henceforth refer to FE schemes satisfying this privacy property as *simulation-secure*. Numerous works have built FE schemes for rich classes of functions from a variety of hardness assumptions. More specifically, for the case of linear functions, we know of group-based schemes for prime-order groups [ABDP15, ALS16, Wee16, ALMT20] as well as for unknown-order groups [ALS16, ALMT20]; and post-quantum constructions [ABDP15, ALS16, ALMT20] are known from the hardness of the LWE problem.

Our construction template in more detail. In the quest to remove NIZKs from $\mathsf{FPreSign}_2$, we modify the above steps of the fair exchange as follows: (a) The AdGen remains identical except that ct is computed using the FE scheme's encryption algorithm, (b) In the $\mathsf{FPreSign}_2$ interaction, the seller instead computes the functional secret-key sk_f and engages with the buyer in an adaptor execution to sell sk_f, (c) the Adapt algorithm remains the same, and (d) the FExt algorithm extracts the functional secret-key sk_f and decrypts ct directly to return $f(\mathbf{x})$. If the FE scheme satisfies simulation-security and the NIZK (in the advertisement) is zero-knowledge, then seller's interaction can be simulated only via $f(\mathbf{x})$, which ensures the desired seller privacy.

An astute reader might ask why we use an encryption of $\mathbf{x}$ in the advertisement and not a one-way function of $\mathbf{x}$, similar to how statement X and witness x have a discrete log relation in standard adaptor signature constructions. A crucial advantage of our construction template is that since $\mathbf{x}$ is encrypted under semantically secure encryption, we can let witness $\mathbf{x}$ be of low-entropy like user databases, which are the main applications we are targeting for FAS. A similar setting would be insecure when using one-way functions instead of encryption since $\mathbf{x}$ is not a high-entropy witness. Moreover, achieving zero-knowledge witness privacy seems hopeless when using one-way functions for setting up X.

Challenges. Unfortunately the above approach doesn't quite work as-is and runs into two main challenges:

1. *Correctness of the Adaptor Statement*: Suppose that one could encode sk_f as an integer x_f in $\mathbb{Z}_p$ (for an appropriate p), and then use the Schnorr adaptor signature for the statement $X_f = g^{x_f}$. Still, for fairness, the buyer needs to gain confidence in the validity of X_f (example, learning the incorrect functional key $\tilde{x}$ will disallow the buyer from learning the correct evaluation) during $\mathsf{FPreSign}_2$. Augmenting the seller's message with a NIZK proof that attests to the correctness of X_f (i.e., discrete-logarithm of X_f is indeed the correct sk_f) would be sufficient. But the use of NIZKs was exactly what we were trying to avoid, and hence it seems we are back to square one.

2. *Compatibility with Adaptor Signatures*: Recall that known adaptor signature schemes only facilitate the sale of witnesses for special algebraic languages. For the above approach to work, we need a *simulation-secure* FE scheme where the functional key-generation algorithm exhibits the required structural compatibility.

For example, to rely on Schnorr adaptor signatures that allow selling discrete-logarithms $x \in \mathbb{Z}_p$ of prime-order group elements $X = g^x$, we need a FE scheme where the functional secret-key sk_f w.r.t. any linear function f is such that $\mathsf{sk}_f \in \mathbb{Z}_p$. In fact, [ABDP15] presents such an IPFE scheme, but the scheme satisfies only a weaker notion of IND-security (and not simulation security) which is insufficient for the template to work.

Our Technique 1: Augmenting IPFE with PubKGen **algorithm.** To avoid using NIZKs during $\mathsf{FPreSign}_2$, we observe that the IPFE schemes for computing inner product of vectors of length ℓ are obtained by starting with ℓ instances of some PKE scheme and a functional secret key is essentially a linear combination of the secret keys of the ℓ PKE instances (as pointed out in [ABDP15]). Combining this observation with the fact that the public key and secret key of the underlying PKE satisfy some algebraic relation, one can envision that a similar linear combination of the public keys of the PKE schemes might give us a commitment to the functional secret key that can be directly used as the statement X_f. Crucially, if X_f can be deterministically computed using IPFE's master public key and the function description f, then such an algorithm would be public and the buyer can compute X_f without any interaction with the seller, thus avoiding NIZKs during $\mathsf{FPreSign}_2$ altogether. We formalize this vision by augmenting IPFE with a public deterministic algorithm PubKGen. The requirement that the output of PubKGen, i.e., X_f must be a commitment to the output of KGen, i.e., sk_f, is formalized via a *compliance property* (Definition 3.4.12). Informally, R_{IPFE}-compliance says that for a given relation R_{IPFE}, it must be the case that $(X_f, \mathsf{sk}_f) \in R_{\mathsf{IPFE}}$.

For instance, if we set the relation R_{IPFE} to be the discrete-log relation, then the key structure becomes compatible with the hard relation of Schnorr adaptor signatures. As an example, the IND-secure IPFE scheme of [ABDP15] can be shown to have such a key structure. In the IND-secure IPFE scheme of [ABDP15], the master secret key is $\mathbf{s} = (s_1, \ldots, s_\ell) \in \mathbb{Z}_p^\ell$, the master public key is $g^{\mathbf{s}} = (g^{s_1}, \ldots, g^{s_\ell})$, the functional secret key for function $\mathbf{y} = (y_1, \ldots, y_\ell)$ is $\mathsf{sk}_{\mathbf{y}} = \sum_{i \in [\ell]} s_i y_i \bmod p$. Then, PubKGen can be defined to output $\mathsf{pk}_{\mathbf{y}} = \prod_{i \in [\ell]} (g^{s_i})^{y_i}$. Consequently, we can observe that $\mathsf{pk}_{\mathbf{y}} = g^{\mathsf{sk}_{\mathbf{y}}}$; thus $(\mathsf{pk}_{\mathbf{y}}, \mathsf{sk}_{\mathbf{y}})$ satisfy the discrete log relation. Defining PubKGen in this way makes it compliant with the Schnorr adaptor signatures: if the buyer wants to learn function $\mathbf{y}$ evaluated on the seller's witness, he can locally compute $\mathsf{pk}_{\mathbf{y}}$ and use it as the adaptor statement. Upon receiving a pre-signature from the buyer, the seller can locally compute $\mathsf{sk}_{\mathbf{y}}$ and use it to adapt the pre-signature. Eventually, the buyer can extract $\mathsf{sk}_{\mathbf{y}}$, thus enabling functional decryption. Crucially notice that because of deterministic nature of PubKGen, the pre-signing becomes non-interactive. But as noted earlier, IND-secure IPFE isn't sufficient for the FAS template to work, so we are not done yet.

Our Technique 2: Simulation-Secure Compatible IPFE. To resolve the compatibility with known adaptor

signatures, we rely on the IND-secure IPFE to simulation-secure IPFE compiler of [ALMT20]. More specifically, their compiler lifts any IND-secure IPFE to achieve simulation-security without changing the structure of the functional secret keys. This is achieved by increasing the length of function vectors from ℓ to 2ℓ, where the first ℓ slots encode the function as before, and the extra ℓ slots encode some random coins. These random coins are used by the IPFE simulator to argue simulation security. To preserve correctness, the length of the message vector to be encrypted is also increased from ℓ to 2ℓ, where the first ℓ slots encode the message as before, and the extra ℓ slots are set to zero. A small caveat is that the compiler results in a stateful functional secret key generation algorithm. We make it stateless which allows us to make an optimization where we increase the vector lengths from ℓ to only $\ell + 1$. Instantiating their compiler with the DDH-based IND-secure IPFE scheme of [ABDP15] results in a simulation-secure FE that is compatible with Schnorr adaptor signatures. We also extend this to the post-quantum setting by using a simplified variant of the LWE-based simulation-secure IPFE scheme of [ALMT20] [6] that is compatible with the Dilithium adaptor signature [EEE20b] instantiated on unstructured lattices.

While the above technique addresses the compatibility w.r.t. known adaptor signatures, proving *zero-knowledge* witness privacy requires more care and we will introduce another technique for it. Let us first explain the challenge with proving *zero-knowledge*. Even when starting with a base IPFE scheme (that is IND-secure) with a deterministic functional key-generation algorithm, the IPFE compiler introduces randomness in the functional key-generation algorithm[7]. This randomization is crucial for the IPFE simulator's correctness and essential to work with the base FE scheme that is only IND-secure. In the context of FAS, during $\mathsf{FPreSign}_2$, if the seller could share the randomness used as part of the generating sk_f, then the buyer can mimic the same computation using the seller's randomness and determine the validity of the sk_f embedded in X_f. While this approach makes pre-signing interactive yet it would certainly avoid the use of general-purpose NIZKs as seen in the template above because the check is *canonical* now. Unfortunately, exposing the seller's randomness would compromise simulation security of the compiled IPFE scheme as the IPFE simulator would then get stuck in the analysis. Consequently, we can't show *zero-knowledge* of FAS. Note that an approach to determine validity of X_f without having the seller share the random coins could be to use NIZKs, but as said before, we want to avoid it.

Our Technique 3: Interactive pre-signing without NIZKs. To address the *zero-knowledge* of FAS, we open

[6] Making the functional secret key generation algorithm stateless restricts the functionality in the LWE setting. In particular, such IPFE can only handle linearly independent function queries. We show how to slightly modify our FAS construction to ensure that linearly dependent function queries from different buyers can be augmented with some extra slots that always guarantee linear independence of function queries to the underlying IPFE. See Section 3.8.4 for more details.

[7] DDH-based IPFE scheme of [ALS16] has been shown to be simulation-secure in [ALMT20]. Interestingly, it does not introduce randomness in functional key-generation and for proving simulation-security, the simulator relies on the fact that there can be many master secret keys corresponding to a master public key. But unfortunately, it does not satisfy the IPFE compliance property as its master keys do not satisfy the discrete log relation. Hence, it is incompatible with Schnorr adaptor signatures. Despite many attempts, we could not make the two compatible.

up the IPFE compiler of [ALMT20], that is, make non-black-box use of the IPFE compiler and reveal only a part of the randomness of the functional key-generation algorithm. By carefully choosing the randomness revealed by the seller, we can guarantee the validity of the adaptor statement X_f to the buyer (without expensive NIZKs) and also allow flexibility to the IPFE simulator to successfully finish the simulation (See Remarks 3.2.1 and 3.2.2 for more details).

We emphasize that the non-black-box use of the techniques from [ALMT20] is a significant technical part of this work. We elaborate on these aspects of our construction in more detail in Section 3.2.3.

3.2.3 Our Construction

Suppose that the relation R is such that for any statement X and witness $\mathbf{x}$ satisfying $(X, \mathbf{x}) \in R$, it is the case that $\mathbf{x} \in \mathbb{Z}_p^\ell$. Further, suppose that inner-product functions are of the form $\mathbf{y} \in \mathbb{Z}_p^\ell$. Then, our FAS construction is obtained by employing the abovementioned techniques. We remind the reader of the protocol flow in Figure 3.1.

- Setup: Run by a trusted third party, it samples common reference string crs for NIZK and public parameters pp' for IPFE.
- AdGen: The seller samples $(\mathsf{mpk}, \mathsf{msk})$ corresponding to IND-secure IPFE and random coins $\mathbf{t}$ used by the non-black-box IPFE compiler to upgrade from IND-secure IPFE to simulation-secure IPFE. It computes the elongated vector $\widetilde{\mathbf{x}}$ used by the IPFE compiler, encrypts $\widetilde{\mathbf{x}}$ to obtain ct and computes a NIZK proof π certifying that ct encrypts a witness corresponding to X.
- AdVerify: The buyer verifies the NIZK proof π.
- Interactive pre-signing: For the sake of notational simplicity, we denote the 3-round interactive functional pre-signing via three algorithms: $\mathsf{FPreSign}_1$, $\mathsf{FPreVerify}_1$, $\mathsf{FPreSign}_2$.
  - First round: the buyer sends the function $\mathbf{y}$ to the seller.
  - Second round: the seller runs the $\mathsf{FPreSign}_1$ algorithm and sends auxiliary value $\mathsf{aux}_\mathbf{y}$ along with a proof $\pi_\mathbf{y}$ validating authenticity of $\mathsf{aux}_\mathbf{y}$ to the buyer. Specifically, the seller uses the random coins $\mathbf{t}$ of the IPFE compiler to compute $f_\mathbf{y}(\mathbf{t})$. Then, it sets the elongated vector $\widetilde{\mathbf{y}} = (\mathbf{y}^T, f_\mathbf{y}(\mathbf{t}))^T$ and generates $\mathsf{pk}_\mathbf{y}$ for it. It sends $(\mathsf{aux}_\mathbf{y}, \pi_\mathbf{y}) := (\mathsf{pk}_\mathbf{y}, f_\mathbf{y}(\mathbf{t}))$.
  - Third round: the buyer verifies the auxiliary value via $\mathsf{FPreVerify}_1$ algorithm, i.e., it creates $\widetilde{\mathbf{y}} = (\mathbf{y}^T, \pi_\mathbf{y})^T$ and checks if $\mathsf{aux}_\mathbf{y}$ matches the output of $\mathsf{IPFE.PubKGen}(\mathsf{mpk}, \widetilde{\mathbf{y}})$. If it verifies, then, it computes pre-signature $\widetilde{\sigma}$ for the adaptor statement $\mathsf{aux}_\mathbf{y}$ and sends it to the seller.

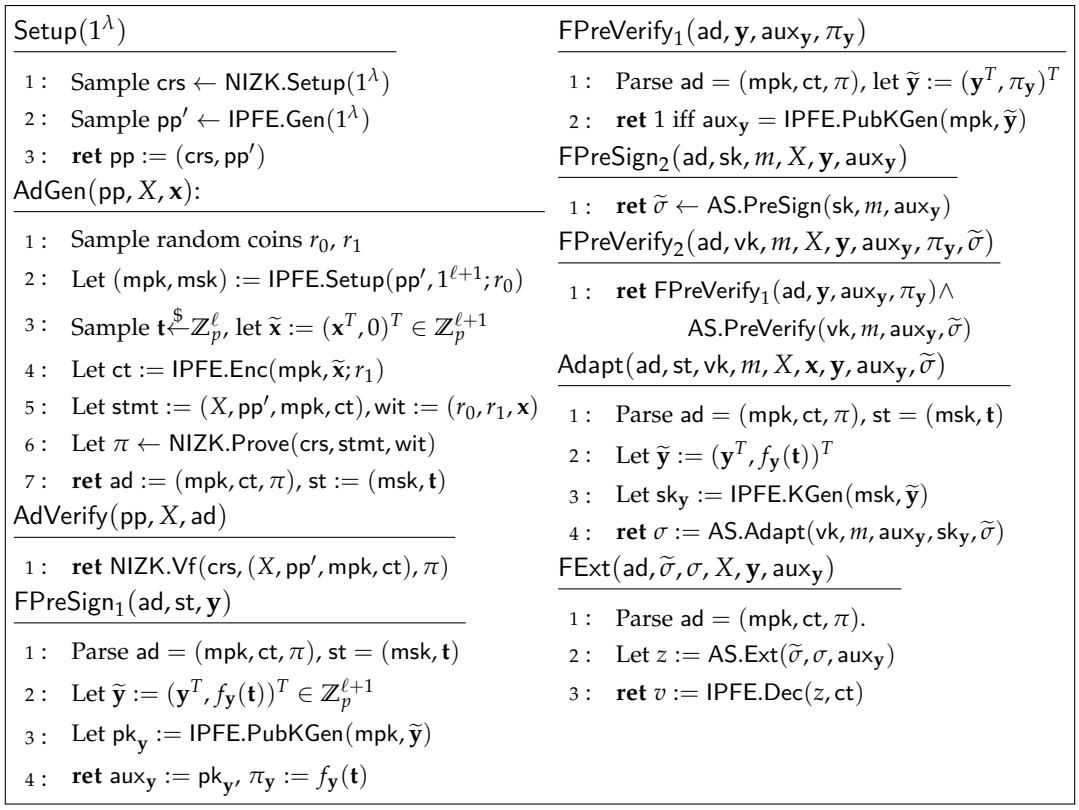


Figure 3.2: Construction: Functional Adaptor Signatures. We note that the syntax here is slightly different from the one in Definition 3.5.1 for ease of presentation. In particular, the function input is now represented via the vector $\mathbf{y}$. $(\mathsf{aux}_{\mathbf{y}}, \pi_{\mathbf{y}})$ denote $\widetilde{\sigma}_1$, and $\widetilde{\sigma}$ simply denotes $\widetilde{\sigma}_2$ in the figure for brevity. FPreSign$_1$ does not need X, x as inputs, FPreVerify$_1$ does not need X as input, FPreSign$_2$ does not need $\pi_{\mathbf{y}}$ as input, Adapt does not need $\pi_{\mathbf{y}}$ as input, and FExt does not need vk, $\pi_{\mathbf{y}}$ as inputs. So, those inputs are omitted for brevity.

- FPreVerify$_2$: The seller verfies that $\widetilde{\sigma}$ corresponds to $\mathbf{y}$, i.e., it verifies that $\widetilde{\sigma}$ corresponds to $\mathsf{aux}_{\mathbf{y}}$ and $\mathsf{aux}_{\mathbf{y}}$ corresponds to $\mathbf{y}$.

- Adapt: The seller computes IPFE functional key $\mathsf{sk}_{\mathbf{y}}$ for function $\widetilde{\mathbf{y}}$. Since $\mathsf{sk}_{\mathbf{y}}$ is a witness to $\mathsf{pk}_{\mathbf{y}}$, it uses $\mathsf{sk}_{\mathbf{y}}$ to adapt $\widetilde{\sigma}$ into σ.

- FExt: The buyer extracts the IPFE functional key $\mathsf{sk}_{\mathbf{y}}$ from $(\widetilde{\sigma}, \sigma)$ and uses it to decrypt ct and recover $f_{\mathbf{y}}(\mathbf{x})$.

Our formal construction is as in Figure 3.2. We defer the security theorem to Section 3.6. We alluded to in 'Our Technique 3' on how to avoid NIZKs in interactive pre-signing by making non-blackbox use of the IPFE compiler. Having described the full details of FAS construction, we now elucidate on it in Remarks 3.2.1 and 3.2.2.

Remark 3.2.1 (Simulatability with leakage $f_{\mathbf{y}}(\mathbf{t})$ on $\mathbf{t}$)**.** *Note that while proving zero-knowledge of FAS, the FAS simulator would still have to reveal $f_{\mathbf{y}}(\mathbf{t})$. This helps us avoid NIZKs in the interactive pre-signing as discussed before.*

Also note that FAS simulator would use the IPFE simulator internally, and $f_{\mathbf{y}}(\mathbf{t})$ is a leakage on the random coins $\mathbf{t}$ of the IPFE (compiler's) simulator. Crucially, this does not break simulation security of IPFE. This is because the IPFE compiler of [ALMT20] — and hence the IPFE simulator too – inherently leaks $f_{\mathbf{y}}(\mathbf{t})$. The latter is because IPFE *decryption implicitly takes $\widetilde{\mathbf{y}}$ as input and hence, the decrypter needs to know $f_{\mathbf{y}}(\mathbf{t})$ to ensure decryption correctness. Hence, from an honest seller's perspective, the* FAS *zero-knowledge simulation won't be affected by giving away $f_{\mathbf{y}}(\mathbf{t})$ as part of $\pi_{\mathbf{y}}$.*

Remark 3.2.2 (Purpose of $\pi_{\mathbf{y}}$)**.** *Note that $\pi_{\mathbf{y}}$ enables verifying that $\mathsf{aux}_{\mathbf{y}}$ is functional public key corresponding to $\widetilde{\mathbf{y}} = (\mathbf{y}^T, \pi_{\mathbf{y}})^T$. Regarding $\widetilde{\mathbf{y}}$, the buyer knows $\mathbf{y}$ but it does not know whether the last slot of $\widetilde{\mathbf{y}}$ chosen by the seller is well-formed, i.e., is $\pi_{\mathbf{y}} = f_{\mathbf{y}}(\mathbf{t})$? In security against a malicious seller, we argue that we do not need to guarantee this. In particular, it does not affect an honest buyer whether $\pi_{\mathbf{y}} = f_{\mathbf{y}}(\mathbf{t})$ holds or not. Suppose that the malicous seller chooses arbitrary value* val *and sends $(\mathsf{aux}_{\mathbf{y}}, \pi_{\mathbf{y}}) = (\mathsf{pk}_{\mathbf{y}}, \mathsf{val})$, where $\mathsf{pk}_{\mathbf{y}} = \mathsf{IPFE.PubKGen}(\mathsf{mpk}, \widetilde{\mathbf{y}})$ and $\widetilde{\mathbf{y}} = (\mathbf{y}^T, \mathsf{val})^T$. This would certainly ensure that $\mathsf{FPreVerify}_1$ passes and the buyer continues the protocol with the seller. Crucially though, we argue that if the honest buyer does end up making the payment to the seller, then, the honest buyer indeed learns $f_{\mathbf{y}}(\mathbf{x})$ at the end of the protocol disregard of what* val *was chosen by the malicious seller. This is because* advertisement soundness *would guarantee the honest buyer that* ct *encrypts a vector $\widetilde{\mathbf{x}}$ of the form $\widetilde{\mathbf{x}} = (\mathbf{x}^T, 0)^T$, where $(X, \mathbf{x}) \in R$. Further, witness extractability would guarantee that the honest buyer extracts a functional secret key $\mathsf{sk}_{\mathbf{y}}$ corresponding to $\widetilde{\mathbf{y}} = (\mathbf{y}^T, \mathsf{val})^T$. Thus, IPFE correctness would guarantee that the honest buyer recovers $f_{\widetilde{\mathbf{y}}}(\widetilde{\mathbf{x}})$ which is same as $f_{\mathbf{y}}(\mathbf{x})$ since the last slot of $\widetilde{\mathbf{x}}$ is zero.*

3.2.4 Instantiations

In Section 3.7, we provide an instantiation from prime order groups for the function class of inner products over integers with output values polynomially bounded by $B \ll p$. For example, if each entry of the witness/function vector is bounded by 10^3, and the vectors have 10^6 entries, then, the inner product value would be bounded by 10^{12}, so we can set $B = 10^{12}$. We explain below application scenarios where such restrictions are reasonable. We also present an instantiation from lattices in Section 3.8 that removes such restrictions. Specifically, it works for the function class of inner products modulo prime integer p. Further, it is post-quantum secure.

Prime-Order Groups based Instantiation.

To instantiate our FAS construction in Figure 3.2 from prime order groups, we instantiate the AS as Schnorr adaptor signature scheme [EFH+21] w.r.t. the hard relation R_{DL}, where R_{DL} is the discrete-log relation

in prime order groups (see Definition 3.4.3). We then set $R_{\mathsf{IPFE}} = R_{\mathsf{DL}}$ and show that a varaiant of the selective, IND-secure IPFE scheme by Abdalla et al. [ABDP15] satisfies R_{DL}-compliance.

Small plaintext space: reason and application scope. This limitation is because the function output here is encoded in the exponent and functional extraction of FAS instantiation involves a discrete log computation step. For values in the exponent bounded by B, this incurs a running time of $O(\sqrt{B})$. This is efficient only if bound B is a polynomial. We argue that this is sufficient for several realistic application scenarios. We benchmarked computation for datasets having a maximum of 10^6 entries, resulting in a 32 MB dataset. For example, the breast cancer dataset on Kaggle [kag] has 20000 entries, total size 50 KB, sum of entries $\approx 10^6$. Assuming each entry of a function is at most 10^4, we get $B = 10^{10}$. Thus, the benchmarks in Table 3.2 for $(\ell = 10^4, B = 10^{10})$ fit closest for this dataset. Other scenarios occur when the secret database is a company's employee record with their age, years of service with the employer, retirement contribution, etc. Such values while sensitive, are typically bounded (example, by $B = 10^{10}$) and the buyer may wish to learn statistical/aggregate information, like sum, weighted mean or average, etc. whose result is also in the bounded plaintext space. The buyer could be a research organization that is studying the workforce in companies or factories, or it could be a recruitment recommendation agency that tries to match clients with potential employers with appropriate aggregate requirements of the client.

Lattice based Instantiation.

We instantiate the AS as the lattice-based adaptor signature scheme by Esgin et al. [EEE20a] w.r.t. the inhomogenous short integer solution relation R_{ISIS} (see Definition 3.4.17). Our instantiation is over the ring of integers $\mathcal{R} = \mathbb{Z}$. Consequently, its security follows from plain SIS and LWE assumptions. Further, we show that a variant of the IND-secure IPFE scheme by Agrawal et al. [ALS16, Section 4.2] satisfies R_{ISIS}-compliance. [8]

For technical reasons related to IPFE, the resulting FAS construction can only support linearly independent function request across all buyers (See Remark 3.8.2 for details). This could be quite restrictive since the scheme cannot support same function requests from different buyers. In Section 3.8.4, we show how to modify the FAS construction to remove this restriction.

[8] Note that in lattice-based adaptor signature scheme by Esgin et al. [EEE20a], the extracted witness does not satisfy R_{ISIS} but it satisfies an extended relation R'_{ISIS} such that $R_{\mathsf{ISIS}} \subset R'_{\mathsf{ISIS}}$. This is due to the rejection sampling step involved in these signatures. Using such signatures along with IPFE thus becomes technically more challenging as the extracted witness acts as the IPFE decryption key. So, we additionally require a decryption robustness property from IPFE (Definition 3.4.13).

3.2.5 Towards Non-Interactive Pre-Signing

One of the main features of adaptor signatures that enable scalability is the non-interactive nature of the pre-signature generation algorithm. More specifically, the buyer in an adaptor signature execution can generate a pre-signature just from the seller's advertisement and, more importantly, without further interaction with the seller. Since the exchange of pre-signatures is done off-chain and hence the latency doesn't impact the blockchain eco-system, a non-interactive pre-signing phase is preferable.

We relaxed pre-signing to be interactive to achieve strong witness privacy for FAS, namely, the malicious buyer learns no information about $\mathbf{x}$ other than $f(\mathbf{x})$. This relaxation is not new to our work and was already introduced in [QPM+23] to achieve the advanced feature of *blindness* for adaptor signatures. Despite numerous attempts, the interactive nature of pre-signing seems necessary for this notion of strong privacy. An immediate challenge towards non-interactive pre-signing would be to build a simulation-secure IPFE scheme where the buyer can itself generate $\mathsf{pk}_f \leftarrow \mathsf{IPFE.PubKGen}(\mathsf{mpk}, f)$ such that the seller can generate the corresponding functional secret-key $\mathsf{sk}_f \leftarrow \mathsf{IPFE.KGen}(\mathsf{msk}, f)$. Unfortunately, the randomized nature of the key-generation algorithm of simulation-secure scheme is a major hurdle to enabling this.

We explore whether a non-interactive pre-signing can be achieved for FAS for relaxed privacy definitions. Towards this, we study FAS that only satisfies *witness indistinguishability*. The construction of such a FAS follows from our construction template by replacing the simulation-secure IPFE with an appropriate IND-secure IPFE. To understand the intuition, we refer the reader to the IPFE compliance example discussed in our technique 1 in Section 3.2.2. We provide more details on this construction in Section 3.11.

3.3 Technical Overview 2: [VGTS26]

This section outlines our techniques, starting with the following setup: Let $\vec{x}$ be a ℓ-dimensional secret integer vector that the seller holds and the buyer wishes to purchase the output of an integer-valued function $f(\vec{x})$. Here we assume $\vec{x}$ satisfies a publicly known (NP) relation $R(X, \vec{x})$ for some public value X. Our goal is to design a protocol where the buyer learns $f(\vec{x})$ if and only if the seller receives payment, and neither party can cheat the other.

Strawman Construction of FAS. In theory, the strawman protocol in previous section can support the sale of arbitrary functions using general-purpose NIZKs. However, in practice, the above strawman protocol is constrained in the exchange phase, by the inefficiency of the NIZK proof computation by the seller. To quantify this drawback, we implemented a significantly simpler variant of the exchange NIZK by using the Groth16 SNARK proof system in the Arkworks library [ac22].[9] In Figure 3.3, we plot the total time

[9]For completeness, we compute the Groth16 proof for the following simpler statement: $\exists \vec{x}$ such that some public hash value h

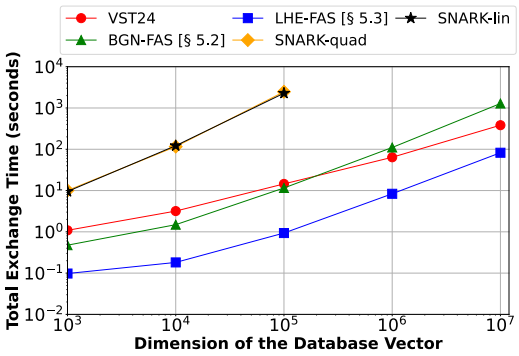


Figure 3.3: Runtime comparison for VST24 and our LHE-FAS (weighted sums), our BGN-FAS (weighted standard deviation), and SNARK-based strawmans over databases of varying dimensions with weights $\leq$ 1000.

for the exchange phase for such strawman protocols for the sale of linear and degree-2 functions over varying database sizes (Similar comparison for high-degree functions can be found in Section 3.13.1). Here the weights refer to the specific weighted computation (like weighted standard deviation). Notice that computing the SNARK proof incurs significant overhead for the seller. While optimized NIZKs may be possible for specific functions, we foresee that this method incurs significant overhead on the seller for general functions.

Prior work of Vanjani, Soni, and Thyagarajan [VST24a], henceforth VST24, overcome this challenge by replacing the advertisement PKE with a public-key *functional encryption (FE) scheme* [BSW11], and then re-purposing the online phase to let the buyer buy a "functional" secret-key w.r.t. its function f instead of a regular PKE decryption key (in the strawman approach) through the Schnorr adaptors. However, their construction requires certain algebraic properties from the underlying FE scheme and adaptor signatures, and is efficient only for functions for which efficient FE schemes are known. This is why they restrict themselves to functions where f is *linear* in its input. Although intensely pursued, the state of progress for an efficient FE scheme for richer functions is underwhelming. Returning to Figure 3.3, we see that the total time for our HE-based constructions (LHE-FAS and BGN-FAS) for the same applications is at least *two orders of magnitude faster* than the SNARK strawman protocols. Moreover, our LHE-FAS scheme is about 10x better than VST24 for most database sizes. We defer the details of LHE-FAS and BGN-FAS to the full version due to space constraints.

3.3.1 Our FAS Template: HE + Adaptors

Given the limitations of FE-based approaches, our key insight is to replace FE with (fully) homomorphic encryption ((F)HE), for which more practical constructions are available. Unlike FE, which allows adaptive

equals $H(\vec{x})$ and the claimed value $y = f(\vec{x})$, where H is SNARK-friendly hash function such as Poseidon.

decryption of $f(\vec{x})$ from an encryption of $\vec{x}$ without the master secret key, HE enables anyone to compute an encryption of $f(\vec{x})$ from the ciphertext of $\vec{x}$. However, HE lacks a mechanism for decryption without the secret key, necessitating a new protocol to build FAS from HE.

The high-level idea for our HE-based FAS construction is as follows: in the advertisement phase, the seller computes an HE encryption of $\vec{x}$. The buyer homomorphically evaluates this ciphertext with respect to a function f. To reveal the result, the seller "opens" the evaluated ciphertext, enabling the buyer to learn $f(\vec{x})$. Translating this idea into a practical FAS scheme requires addressing several technical challenges, which we detail after presenting the protocol's structure. We define the advertisement and exchange phases as follows:

- **Advertisement Phase:** The seller generates a fresh HE key pair $(\mathsf{pk}_{\mathsf{HE}}, \mathsf{sk}_{\mathsf{HE}})$ and publishes $\mathsf{CT}_{\mathsf{ad}} \leftarrow\$ \mathsf{HE.Enc}(\mathsf{pk}_{\mathsf{HE}}, \vec{x})$ along with $\mathsf{pk}_{\mathsf{HE}}$. A NIZK proof is provided to certify that $\mathsf{CT}_{\mathsf{ad}}$ encrypts a valid witness for X and that $\mathsf{pk}_{\mathsf{HE}}$ is well-formed.[10] This phase is performed once and can be published off-chain, such as on the seller's website, for reuse across all sales.
- **Exchange Phase:** The buyer computes CT_y, an HE encryption of $y = f(\vec{x})$, by homomorphically evaluating $\mathsf{CT}_{\mathsf{ad}}$ with respect to f. This computation, performed offline, requires no seller interaction. Once the seller receives f, it computes $y = f(\vec{x})$ [11] and encrypts it using the Elgamal PKE scheme with a fresh key pair $(K = g^k, k)$. Along with the Elgamal ciphertext ct_y, the seller provides a NIZK proof π that ct_y and CT_y (that was computed from the advertisement) encrypt the same plaintext, referred to as *Proof of Plaintext Equality (PoPE)*. The buyer verifies π, and if valid, subsequently purchases the Elgamal secret key k using the Schnorr adaptor signature. The buyer then decrypts the Elgamal ciphertext ct_y to retrieve y.

This protocol supports FAS for all functions f when the HE scheme is a fully homomorphic encryption scheme [GSW13, BGV11]. Its security (see Section 3.5 for definitions) follows from the properties of the underlying tools, including the semantic security of HE and Elgamal schemes, and the soundness of NIZKs.

3.3.2 Efficiency Improvements

While this template is secure, it lacks practicality in its current form. We identify three key challenges and opportunities to enhance efficiency for both parties:

- **Seller's Computation of CT_y and Randomness:** To compute the NIZK proof in the exchange phase, the seller must generate CT_y and its associated randomness r_y which serves as a witness in the NIZK proof.

[10] Ensuring $\mathsf{pk}_{\mathsf{HE}}$ is well-formed becomes crucial for the soundness of proofs in the exchange phase.

[11] Proofs over homomorphically encrypted values [ACGSV24, GGW24] are orthogonal to our work, but potentially useful for improving seller efficiency in FAS. With efficiently verifiable FHE, the buyer could also prove that its ciphertext correctly encodes f(x), eliminating the need for the seller to compute the function output.

Doing so naively via HEval is computationally expensive, particularly for resource-constrained sellers, as in some applications. Moreover, this would not aid the seller with computing r_y.

- **Complexity of General-Purpose NIZKs:** Although the exchange-phase NIZK is independent of the function f's complexity (unlike the Strawman), general-purpose NIZKs add significant overhead when modeling the algebraic constraints of the HE scheme. This can also lead to non-black-box use of the HE scheme, introducing further inefficiencies.
- **Buyer's Cost of Decrypting Elgamal PKE:** The buyer's decryption process involves computing discrete logarithms (due to the decryption of standard Elgamal PKE), which are computationally expensive (as VST24 observed) and require strict bounds on the function's output, limiting general applicability.

We address these challenges to construct a practical scheme, as detailed below.

(a) Efficient Computation of CT_y and Randomness. Ensuring the seller and buyer agree on the same CT_y is critical for a valid statement in the PoPE NIZK. This issue can be mitigated using HE schemes that support a *deterministic* HEval procedure, a feature common to most existing HE schemes [GSW13, CGGI20, BGV11]. However, if both parties independently compute CT_y via HEval, it leads to redundant work, reducing the overall efficiency. The buyer's homomorphic computation remains essential because offloading it to the seller would require proving the correctness of f-sized computations in the exchange NIZK proof, resulting in significant performance degradation compared to the original template. To improve efficiency, we leverage a sibling algorithm REval, implicit in most HE schemes. REval computes the randomness r_y for CT_y as a function of the randomness from the input ciphertext $\mathsf{CT}_{\mathsf{ad}}$, mimicking HEval without operating on the ciphertexts. This makes REval significantly faster than HEval. Once the seller computes r_y, CT_y is obtained using $\mathsf{HE.Enc}(\mathsf{pk}_{\mathsf{HE}}, y; r_y)$. Looking ahead, while such an REval algorithm is implicit in the TFHE and LHE schemes we use, for the case of degree-2 functions, we introduce a novel variant of the Boneh-Goh-Nissim (BGN) encryption scheme that supports deterministic HEval and efficient REval algorithms.

Modulo the computation of output y, can the seller's computation be made independent of the complexity of f? While this does not result in asymptotic improvements, this is highly relevant for practical efficiency. We show that this is indeed possible in HE schemes that support homomorphism and, additionally, the bootstrapping functionality. In this work, we use the TFHE scheme [CGGI20], and rely on an insight regarding the underlying structure of its HEval algorithm: key switching, one of the steps involved in bootstrapping that transforms a ciphertext $\mathsf{ct}_{2,f}$ encrypting message $f(m)$ under a key pk_2 into a ciphertext $\mathsf{ct}_{1,f}$ under another key pk_1. This is accomplished by using a key switching key ksk, which in itself is a FHE ciphertext encrypting secret key sk_2 (corresponding to pk_2) under pk_1. Key switching here is roughly

a linear computation as follows:

$$\mathsf{ct}_{1,f} := \langle \mathsf{ksk}, \mathsf{ct}_{2,f} \rangle,$$

where ksk is treated a vector of ciphertexts and $\mathsf{ct}_{2,f}$ is treated as a vector of known constants. Thus, the entire computation is simply a linear homomorphic evaluation on the ciphertext ksk. Crucially, after this transformation, the random coins $r_{1,f}$ underlying the new ciphertext $\mathsf{ct}_{1,f}$ only depend on the randomness r_{ksk} in the key switching key ksk, and not on the randomness of the original ciphertext $\mathsf{ct}_{2,f}$. Suppose r_{ksk} is the randomness underlying ciphertext ksk. Then $r_{1,f}$ can be simply recovered as $r_{1,f} := \langle r_{\mathsf{ksk}}, \mathsf{ct}_{2,f} \rangle$. This allows us to build an REval algorithm that takes a ciphertext $\mathsf{ct}_{2,f}$ computed using HEval, and key switches it into another one $\mathsf{ct}_{1,f}$ whose randomness $r_{1,f}$ is easy to compute as above. This computation is significantly cheaper than HEval itself, and is linear in the size of the key switching key and independent of the complexity of f. We refer to Section 3.13.1 for more details.

(b) Black-Box Efficient PoPE Protocol. The exchange phase NIZK in our protocol proves a simpler statement than in the strawman protocol. Specifically, the seller must demonstrate that an HE ciphertext and an Elgamal ciphertext encrypt the same plaintext value. This statement is independent of the function f, making it significantly more efficient for the seller, even with a general-purpose NIZK such as Groth16. However, this approach involves non-black-box use of the HE scheme. To address this, we propose a modular *cut-and-choose* based Σ-PoPE protocol[12] (Section 3.12.2) that makes black-box use of the HE scheme, and is of independent interest.

(c) Efficient Decryption. In the template construction, the buyer performs an Elgamal PKE decryption to obtain the result. Similar to VST24, this involves costly discrete-logarithm computations, adding a significant bottleneck, especially when combined with the buyer's HEval computation. We optimize this by slightly modifying the protocol. In the exchange phase, the seller encrypts the output y in a bitwise fashion. For an n-bit binary representation $y = y_1 || \ldots || y_n$, each bit y_i is encrypted using the same Elgamal public key K. The i-th ciphertext is given by $\mathsf{ct}_i = (g^{r_i}, K^{r_i} \cdot g^{y_i})$.

With this change, the buyer computes the result y by performing n parallel group multiplications, a far simpler operation than a discrete-logarithm computation. This modification necessitates minor updates to the PoPE protocol: the proof now addresses bitwise encryptions $(\mathsf{ct}_1, \ldots, \mathsf{ct}_n)$, rather than a single ciphertext ct_y. However, by leveraging the homomorphic properties of the Elgamal PKE scheme, the new PoPE statement can be efficiently reduced to the original statement with a single ciphertext ct_y. See Section 3.12 for details.

[12]This protocol can be made non-interactive using the Fiat-Shamir transform.

3.3.3 Defining Functional Fair Exchange

We define *Functional Fair Exchange* (FFE) as a generalization of the prior work on fair exchange protocols [BGKS25,TSZ+24], extending beyond the traditional all-or-nothing paradigm to the setting of functional sales. While FAS corresponds to a two-round instance of FFE,[13] we model FFE more generally as an interactive R-round protocol involving a seller, a buyer, and a wallet-based token system that abstracts the role of the blockchain.

FFE guarantees fairness for both parties. Buyer fairness is relatively straightforward: the buyer should not lose tokens without obtaining the correct output. Seller fairness is more delicate – it requires that a prematurely aborting buyer *learns nothing*, whereas a buyer who completes the protocol *learns only the intended output*. To capture this, we introduce a novel notion of *round-by-round zero-knowledge*, analogous to round-by-round soundness in interactive proofs [Hol19]. This guarantees that transcripts from aborted executions are simulatable without access to the output, while transcripts from non-aborting or complete executions can be simulated only given the output y. See Definition 3.14.1 for formal details.

3.4 Preliminaries

We recall the cryptographic tools needed in this work.

Notations. We denote by $x \leftarrow S$ the experiment of sampling x from a probability distribution S. We denote by $[A(\cdot)]$ the range of an algorithm $A(\cdot)$. If $p(\cdot,\cdot)$ denotes a predicate, then $\Pr[p(y,z) : x \leftarrow S, (y,z) \leftarrow A(x)]$ is the probability that the predicate $p(y,z)$ is true after the ordered sequence of events $x \leftarrow S$ followed by $(y,z) \leftarrow A(x)$.

We denote scalars by lower-case alphabets such as x, vectors by bold-face lower-case alphabets such as $\mathbf{x}$, matrices by bold-face upper-case alphabets such as $\mathbf{X}$. $\mathbf{x} = (x_1, x_2)^T$ denotes a column-vector with elements x_1 and x_2. $\mathbf{x}^T$ denotes a row-vector. Suppose $\mathbb{G}$ is a cyclic group of prime-order p and with generator g. For a scalar $x \in \mathbb{Z}_p$, g^x denotes its group encoding. For a vector $\mathbf{x} = (x_1, \ldots, x_n)^T \in \mathbb{Z}_p^n$, $g^{\mathbf{x}}$ denotes its group encoding $(g^{x_1}, \ldots, g^{x_n})^T$. Similarly, for a matrix $\mathbf{X}$, $g^{\mathbf{X}}$ denotes its group encoding. Given group encoding of a vector $g^{\mathbf{x}}$ and a vector $\mathbf{y}$, we can compute $g^{\mathbf{x}^T\mathbf{y}}$ efficiently as $g^{\mathbf{x}^T\mathbf{y}} = \Pi_{i\in[n]}(g^{x_i})^{y_i}$.

The security parameter is denoted by λ. We will denote negligible functions by ϵ, and $\epsilon(\lambda)$ denotes a function that is negligible in the security parameter λ. We say that a function $f : \mathbb{N} \to \mathbb{R}$ is negligible in λ if for every $k \in \mathbb{N}$, there exists a $\lambda_0 \in \mathbb{N}$ such that for every $\lambda \geq \lambda_0$, we have that $|f(\lambda)| \leq 1/\lambda^k$. Sometimes, we will use negl to also denote negligible functions.

[13] Here, a "round" refers to a message from the seller followed by a response from the buyer.

Linear Functions. We denote linear functions as inner product computation. Let $\mathcal{F}_{\mathsf{IP},\ell}$ denote the family of inner products of vectors of length ℓ. For a prime p, in this work we study two restrictions of inner product computations as follows:

- *Inner products modulo p.* The function class $\mathcal{F}_{\mathsf{IP},\ell,p} = \{f_{\mathbf{y}} : \mathbf{y} \in \mathbb{Z}_p^\ell\}$, where $f_{\mathbf{y}} : \mathbb{Z}_p^\ell \to \mathbb{Z}_p$ is defined as $f_{\mathbf{y}}(\mathbf{x}) = \mathbf{x}^T\mathbf{y} \bmod p$.
- *Inner products with output values polynomially bounded by $B \ll p$.* The function class $\mathcal{F}_{\mathsf{IP},\ell,p,B} = \{f_{\mathbf{y}} : \mathbf{y} \in \mathbb{Z}_p^\ell\}$, where $f_{\mathbf{y}} : \mathbb{Z}_p^\ell \to \{0,\ldots,B\}$ is defined as $f_{\mathbf{y}}(\mathbf{x}) = \mathbf{x}^T\mathbf{y} \in \{0,\ldots,B\}$.

Often we will use $\mathbf{y}$ instead of $f_{\mathbf{y}}$ to denote the function.

Digital Signature. A digital signature scheme $\mathsf{DS} := (\mathsf{KGen}, \mathsf{Sign}, \mathsf{Vf})$ has a key generation algorithm $(\mathsf{vk}, \mathsf{sk}) \leftarrow \mathsf{KGen}(1^\lambda)$ that outputs a verification-signing key pair. Using signing key sk we can compute signatures on a message m by running $\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$, which can be publicly verified using the corresponding verification key vk by running $\mathsf{Vf}(\mathsf{vk}, m, \sigma)$. We require the digital signature scheme to satisfy strong existential unforgeability [GMR88].

3.4.1 Hard Relation

We recall the notion of a hard relation R with statement/witness pairs (X, x). We denote by L_R the associated language defined as $L_R := \{\ X \mid \exists\ x, (X, x) \in R\ \}$.

Definition 3.4.1 (Hardness of R)**.** *We say that a relation R is hard, if (i) there exists a p.p.t. sampling algorithm $\mathsf{GenR}(1^\lambda)$ that outputs a statement/witness pair $(X, x) \in R$, (ii) the relation is poly-time decidable, (iii) for every non-uniform PPT adversary $\mathcal{A}$, there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$,*

$$\Pr[\mathsf{G}_{\mathcal{A},R}(1^\lambda) = 1] \leq \mathsf{negl}(\lambda)\ , \tag{3.1}$$

where the game $\mathsf{G}_{\mathcal{A},R}(1^\lambda)$ is defined as follows:

Game $\mathsf{G}_{\mathcal{A},R}(1^\lambda)$	
1 :	$(X, x) \leftarrow \mathsf{GenR}(1^\lambda)$
2 :	$(x') \leftarrow \mathcal{A}(1^\lambda, X)$
3 :	*If* $(X, x') \in R$*:* **ret** 1
4 :	**ret** 0

Definition 3.4.2 (Hardness of R w.r.t. function class $\mathcal{F}$)**.** *We say that a relation R is $\mathcal{F}$-hard, if (i) there exists a p.p.t. sampling algorithm $\mathsf{GenR}(1^\lambda)$ that outputs a statement/witness pair $(X, x) \in R$, (ii) for every non-uniform PPT adversary $\mathcal{A}$ there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$,*

$$\Pr[\mathsf{G}_{\mathcal{A},R,\mathcal{F}}(1^\lambda) = 1] \leq \mathsf{negl}(\lambda)\ , \tag{3.2}$$

where the game $\mathsf{G}_{\mathcal{A},R,\mathcal{F}}(1^\lambda)$ is defined as follows:

Game $\mathsf{G}_{\mathcal{A},R,\mathcal{F}}(1^\lambda)$

1 : $(X, x) \leftarrow \mathsf{GenR}(1^\lambda)$

2 : $(f, z) \leftarrow \mathcal{A}(1^\lambda, X)$

3 : *If* $(f \in \mathcal{F}) \wedge (z \in \{f(x') : \exists\ x'$ *such that* $(X, x') \in R\})$: **ret** 1

4 : **ret** 0

We note that in the above definition we allow the adversary $\mathcal{A}$ to choose f adaptively as this hardness assumption will later be used in proving unforgeability of our functional adaptor signatures construction where the adversary is allowed to choose the challenge function adaptively. This is necessary as functional adaptor signatures are non-trivial to build only in such a setting and become trivial if the challenge function is selectively chosen upfront by the adversary. Further note that we let the adversary $\mathcal{A}$ win even if $\mathcal{A}$ outputs a function evaluation of a witness x' of X that is different from x.

Definition 3.4.3 (Hard Relation R_{DL})**.** *The discrete log language L_{DL} is defined with respect to a group G with generator g and order p as $L_{\mathsf{DL}} := \{\ X \mid \exists\ x \in \mathbb{Z}_p, X = g^x\ \}$ with the corresponding hard relation $R_{\mathsf{DL}} = \{\ (X, x) \mid X = g^x\ \}$.*

3.4.2 Adaptor Signatures

Adaptor Signatures were first formally defined in [AEE+21]. [EEE20a] generalized the syntax in [AEE+21] to enable constructing adaptor signatures from lattices. More concretely, [AEE+21] defined adaptor signatures w.r.t. a digital signature scheme and a hard relation R. [EEE20a] generalized it to be w.r.t. two hard relations R and R' such that such that $R \subseteq R'$. For the group-based constructions, typically we have $R = R'$, but for the lattice based constructions, typically we have $R \neq R'$. In this work, one of our constructions will be from lattices, hence, we follow this generalized syntax.

In a different vein, [DOY22, GSST24] have strengthened the security properties of adaptor signatures, with notions like *unforgeability*, *pre-signature extractability*, and *witness extractability*. [DOY22] further added *unique extractability* and *unlinkability* as security properties and unified *unforgeability* and *witness extractability*

under a single security property called *extractability*. [GSST24] further added *pre-verify soundness* as a security property. We note that not all security properties are needed always. Within the scope of this work, we will use adaptor signatures as a building block for the construction of FAS and we only need *pre-signature adaptability* and *witness extractability* security properties of adaptor signatures. Intuitively, pre-signature adaptability ensures that given a valid pre-signature and a witness for the statement, one can always adapt the pre-signature into a valid signature. Witness extractability requires that given an honestly generated pre-signature and a valid signature, one should be able to extract a witness. We define these formally below.

Definition 3.4.4. *An adaptor signature scheme* $\mathsf{AS}_{\mathsf{DS},R,R'} := (\mathsf{PreSign}, \mathsf{PreVerify}, \mathsf{Adapt}, \mathsf{Ext})$ *is defined with respect to a signature scheme* $\mathsf{DS} = (\mathsf{KGen}, \mathsf{Sign}, \mathsf{Vf})$ *and hard relations* R *and* R' *such that* $R \subseteq R'$*. Here,* R *constitutes the relation for the statement-witness pairs generated by* GenR *and* R' *is an extended relation that defines the relation for* extracted *witnesses. The interfaces are described below.*

- $\widetilde{\sigma} \leftarrow \mathsf{PreSign}(\mathsf{sk}, m, X)$*: The pre-signing algorithm takes as input a signing key* sk*, a message* m*, and a statement* X *for the language* L_R*, and outputs a pre-signature* $\widetilde{\sigma}$ *(we sometimes also refer to this as a partial signature).*
- $0/1 \leftarrow \mathsf{PreVerify}(\mathsf{vk}, m, X, \widetilde{\sigma})$*: The pre-signature verification algorithm takes as input a verification key* vk*, a message* m*, a statement* X *for the language* L_R*, and a pre-signature* $\widetilde{\sigma}$*, and outputs* $0/1$ *signifying whether* $\widetilde{\sigma}$ *is correctly generated.*
- $\sigma := \mathsf{Adapt}(\mathsf{vk}, m, X, x, \widetilde{\sigma})$*: The adapt algorithm transforms a pre-signature* $\widetilde{\sigma}$ *into a valid signature* σ *given the witness* x *for the instance* X *of the language* L_R*.*
- $x := \mathsf{Ext}(\widetilde{\sigma}, \sigma, X)$*: The extract algorithm takes as input a pre-signature* $\widetilde{\sigma}$*, a signature* σ*, and an instance* X*, and outputs a witness* x' *such that* $(X, x') \in R'$*, or* $\perp$*.*

Note that an adaptor signature scheme $\mathsf{AS}_{\mathsf{DS},R,R'}$ also inherits $\mathsf{KGen}, \mathsf{Sign}, \mathsf{Vf}$ algorithms from the signature scheme DS.

Typically, an adaptor signature is run between a buyer and a seller. The buyer runs $\mathsf{KGen}, \mathsf{Sign}$, $\mathsf{PreSign}, \mathsf{Ext}$ algorithms, the seller runs Adapt algorithm, and anyone can run $\mathsf{PreVerify}, \mathsf{Vf}$ algorithms.

Remark 3.4.5. *Looking ahead, in the discrete-log based instantiation, we will have* $R' = R$*, but in the lattice-based instantiation, we will have* $R' \neq R$ *and the reason for this extension is the* knowledge/soundness gap *inherent in* efficient *lattice-based zero-knowledge proofs.*

Definition 3.4.6 (Correctness)**.** *An adaptor signature scheme* AS *satisfies correctness if for every* $n \in \mathbb{N}$*, every message* $m \in \{0,1\}^*$*, and every statement/witness pair* $(X, x) \in R$*, the following holds:*

$$\Pr\left[\begin{array}{l} \mathsf{PreVerify}(\mathsf{vk}, m, X, \widetilde{\sigma}) = 1 \\ \quad \wedge\ \mathsf{Vf}(\mathsf{vk}, m, \sigma) = 1 \\ \quad \wedge\ (X, x') \in R' \end{array} : \begin{array}{l} (\mathsf{sk}, \mathsf{vk}) \leftarrow \mathsf{KGen}(1^\lambda) \\ \widetilde{\sigma} \leftarrow \mathsf{PreSign}(\mathsf{sk}, m, X) \\ \sigma := \mathsf{Adapt}(\mathsf{vk}, m, X, x, \widetilde{\sigma}) \\ x' := \mathsf{Ext}(\widetilde{\sigma}, \sigma, X) \end{array}\right] = 1$$

In terms of security, we want witness extractability against a malicious seller and weak pre-signature adaptability against a malicious seller. We explain these next.

Witness extractability requires that for an honestly generated pre-signature and a valid signature, one should be able to extract a witness. Intuitively, it tries to capture that an interaction between an honest buyer (i.e., generates valid pre-signature) and a malicious seller trying to get paid (i.e., adapt pre-signature into a valid signature) without revealing a witness (i.e., extraction failure) should be unlikely to occur.

Definition 3.4.7 (Witness Extractability)**.** *An adaptor signature scheme* AS *is witness extractable if for every PPT adversary* $\mathcal{A}$*, there exists a negligible function* negl *such that for all* $\lambda \in \mathbb{N}$*,*

$$\Pr[\mathsf{aWitExt}_{\mathcal{A},\mathsf{AS}}(1^\lambda) = 1] \leq \mathsf{negl}(\lambda),$$

where the experiment $\mathsf{aWitExt}_{\mathcal{A},\mathsf{AS}}$ *is defined as in Figure 3.4.*

Experiment $\mathsf{aWitExt}_{\mathcal{A},\mathsf{AS}}$.

1 : $\mathcal{Q} := \emptyset$
2 : $(\mathsf{sk}, \mathsf{vk}) \leftarrow \mathsf{KGen}(1^\lambda)$
3 : $(m, X) \leftarrow \mathcal{A}^{\mathcal{O}_S(\cdot), \mathcal{O}_{pS}(\cdot,\cdot)}(\mathsf{vk})$
4 : $\widetilde{\sigma} \leftarrow \mathsf{PreSign}(\mathsf{sk}, m, X)$
5 : $\sigma \leftarrow \mathcal{A}^{\mathcal{O}_S(\cdot), \mathcal{O}_{pS}(\cdot,\cdot)}(\widetilde{\sigma})$
6 : $x' := \mathsf{Ext}(\widetilde{\sigma}, \sigma, X)$
7 : **ret** $((m \notin \mathcal{Q}) \wedge ((X, x') \notin R') \wedge \mathsf{Vf}(\mathsf{vk}, m, \sigma))$

Oracle $\mathcal{O}_S(m)$

1 : $\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$
2 : $\mathcal{Q} := \mathcal{Q} \vee \{m\}$
3 : **ret** σ

Oracle $\mathcal{O}_{pS}(m, X)$

1 : $\widetilde{\sigma} \leftarrow \mathsf{PreSign}(\mathsf{sk}, m, X)$
2 : $\mathcal{Q} := \mathcal{Q} \vee \{m\}$
3 : **ret** $\widetilde{\sigma}$

Figure 3.4: Witness Extractability experiment of adaptor signatures

Note that, in the above witness extractability definition, the adversary's winning condition is restricted to the extracted witness not being in R'. Since $R \subseteq R'$, $(X, x') \notin R'$ implies $(X, x') \notin R$. Therefore, it is sufficient to ensure that R' is a hard relation, which itself implies that R is also a hard relation. As a result, in our security assumptions, we make sure that R' is a hard relation.

Weak pre-signature adaptability requires that given a valid pre-signature and a witness for the instance, one can always adapt the pre-signature into a valid signature. Intuitively, this tries to capture that it should be impossible for a malicious buyer to learn the witness without doing the payment.

Definition 3.4.8 (Weak Pre-signature Adaptability)**.** *An adaptor signature scheme* AS *satisfies weak pre-signature adaptability if for any* $\lambda \in \mathbb{N}$*, any message* $m \in \{0,1\}^*$*, any key pair* $(\mathsf{sk}, \mathsf{vk}) \leftarrow \mathsf{KGen}(1^\lambda)$*, any statement/witness pair* $(X, x) \in R$*, and any pre-signature* $\widetilde{\sigma} \in \{0,1\}^*$ *with* $\mathsf{PreVerify}(\mathsf{vk}, m, X, \widetilde{\sigma}) = 1$*, we have:*

$$\Pr[\mathsf{Vf}(\mathsf{vk}, m, \mathsf{Adapt}(\mathsf{vk}, m, X, x, \widetilde{\sigma})) = 1] = 1.$$

Note that the above pre-signature adaptability is called *weak* because only statement-witness pairs satisfying R are guaranteed to be adaptable, and not those satisfying R'. Therefore, weak pre-signature adaptability does not guarantee, for example, that an *extracted* witness can be used to adapt a pre-signature successfully. This is however guaranteed whenever $R = R'$.

We have several constructions of adaptor signatures compatible with ECDSA [AEE+21], lattice-based signatures [EEE20a, ACL+22], and dichotomic signature schemes [GSST24] which is a recently introduced abstraction that captures many popular schemes like Schnorr, CL, BBS, etc. The group-based schemes are constructed w.r.t. the discrete logarithm NP language in the respective groups, while the lattice-based schemes are constructed w.r.t. the short integer solution NP language.

3.4.3 Inner Product Functional Encryption

We will need a inner-product functional encryption scheme (IPFE). Let ℓ be an integer. We define IPFE for computing inner products of vectors of length ℓ. Let the message space be $\mathcal{M} \subseteq \mathbb{Z}^\ell$ and function space $\mathcal{F}_{\mathsf{IP},\ell}$. Messages are denoted by $\mathbf{x} \in \mathcal{M}$, functions are denoted by $\mathbf{y} \in \mathcal{F}_{\mathsf{IP},\ell}$ and the function evaluation is denoted by $f_{\mathbf{y}}(\mathbf{x})$ (all three of them are parameterized by the prime $p \in \mathsf{pp}$ as output by Gen below). Formally, IPFE consists of the following algorithms:

- $\mathsf{pp} \leftarrow \mathsf{Gen}(1^\lambda)$: it is a *randomized* algorithm that takes in a security parameter λ and samples public parameters pp. We assume that pp contains a prime number p.
- $(\mathsf{mpk}, \mathsf{msk}) \leftarrow \mathsf{Setup}(\mathsf{pp}, \ell)$: it is a *randomized* algorithm that takes in the public parameters pp and the vector length ℓ, outputs a master public key mpk and a master secret key msk.
- $\mathsf{sk}_{\mathbf{y}} := \mathsf{KGen}(\mathsf{msk}, \mathbf{y} \in \mathcal{F}_{\mathsf{IP},\ell})$: it is a *deterministic* algorithm that takes in the master secret key msk, and a vector $\mathbf{y} \in \mathcal{F}_{\mathsf{IP},\ell}$, outputs a functional secret key $\mathsf{sk}_{\mathbf{y}}$.
- $\mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{mpk}, \mathbf{x} \in \mathcal{M})$: it is a *randomized* algorithm that takes in the master public key mpk, a plaintext vector $\mathbf{x} \in \mathcal{M}$, and outputs a ciphertext ct.
- $v := \mathsf{Dec}(\mathsf{sk}_{\mathbf{y}}, \mathsf{ct})$: it is a *deterministic* algorithm that takes in the functional secret key $\mathsf{sk}_{\mathbf{y}}$ and a ciphertext ct, and outputs a decrypted outcome v.

Further, we augment the IPFE scheme with an additional algorithm $\mathsf{PubKGen}$ defined as follows.

- $\mathsf{pk}_{\mathbf{y}} := \mathsf{PubKGen}(\mathsf{mpk}, \mathbf{y} \in \mathcal{F}_{\mathsf{IP},\ell})$: it is a *deterministic* algorithm that takes in the master public key mpk, and a vector $\mathbf{y} \in \mathcal{F}_{\mathsf{IP},\ell}$, outputs a functional public key $\mathsf{pk}_{\mathbf{y}}$.

Next, we describe the correctness of IPFE.

Definition 3.4.9 (IPFE Correctness)**.** *For any $\lambda \in \mathbb{N}$, let $\mathsf{pp} \leftarrow \mathsf{Gen}(1^\lambda)$, then for any $\ell \in \mathbb{N}, \mathbf{x} \in \mathcal{M}, \mathbf{y} \in \mathcal{F}_{\mathsf{IP},\ell}$, the following holds:*

$$\Pr\left[v = f_{\mathbf{y}}(\mathbf{x}) : \begin{array}{l} (\mathsf{mpk}, \mathsf{msk}) \leftarrow \mathsf{Setup}(\mathsf{pp}, \ell), \\ \mathsf{sk}_{\mathbf{y}} = \mathsf{KGen}(\mathsf{msk}, \mathbf{y}), \\ \mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{mpk}, \mathbf{x}), \\ v = \mathsf{Dec}(\mathsf{sk}_{\mathbf{y}}, \mathsf{ct}) \end{array} \right] = 1.$$

Next, we describe the security of IPFE.

Definition 3.4.10 (Selective, IND-security)**.** *We say that the IPFE scheme satisfies selective, IND-security, iff for any non-uniform p.p.t. admissible adversary $\mathcal{A}$, there exists a negligible function negl such that for all $\lambda \in \mathbb{N}, \ell \in \mathbb{N}$,*

$$|\Pr[\mathsf{IPFE\text{-}IND\text{-}SEL}^0_{\mathcal{A},\mathsf{IPFE}}(1^\lambda, \ell) = 1] - \Pr[\mathsf{IPFE\text{-}IND\text{-}SEL}^1_{\mathcal{A},\mathsf{IPFE}}(1^\lambda, \ell) = 1]| \leq \mathsf{negl}(\lambda),$$

where experiments $\mathsf{IPFE\text{-}IND\text{-}SEL}^b_{\mathcal{A},\mathsf{IPFE}}(1^\lambda)$ for $b \in \{0,1\}$ are described in Figure 3.5. Here, an adversary $\mathcal{A}$ is said to be admissible iff the following holds with probability 1: for any $\mathbf{y}$ submitted in a $\mathcal{O}_{\mathsf{KGen}}$ oracle query, it must be that $f_{\mathbf{y}}(\mathbf{x}^*_0) = f_{\mathbf{y}}(\mathbf{x}^*_1)$.

Experiment $\mathsf{IPFE\text{-}IND\text{-}SEL}^b_{\mathcal{A},\mathsf{IPFE}}(1^\lambda, \ell)$	Oracle $\mathcal{O}_{\mathsf{KGen}}(\mathbf{y})$
1 : $\mathsf{pp} \leftarrow \mathsf{Gen}(1^\lambda)$	1 : **ret** $\mathsf{sk}_{\mathbf{y}} = \mathsf{KGen}(\mathsf{msk}, \mathbf{y})$
2 : $(\mathbf{x}^*_0, \mathbf{x}^*_1) \leftarrow \mathcal{A}(\mathsf{pp})$	
3 : $(\mathsf{mpk}, \mathsf{msk}) \leftarrow \mathsf{Setup}(\mathsf{pp}, \ell)$, $\mathsf{ct}^* \leftarrow \mathsf{Enc}(\mathsf{mpk}, \mathbf{x}^*_b)$	
4 : $b' \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{KGen}}(\cdot)}(\mathsf{mpk}, \mathsf{ct}^*)$	
5 : **ret** b'	

Figure 3.5: Selective, IND-security experiment of Inner Product Functional Encryption

The above IND-security level is called selective because it requires the adversary to commit to the challenge message pair even before it sees mpk. Stronger security models exist such as (i) semi-adaptive, IND-security where the adversary can commit to the challenge message pair after seeing mpk but before gaining access to the oracle $\mathcal{O}_{\mathsf{KGen}}$ and (ii) adaptive, IND-security where the adversary can commit to the challenge message pair after seeing mpk and the power to make queries to the oracle $\mathcal{O}_{\mathsf{KGen}}$ before and

after committing to the challenge message pair. But, we only consider selective, IND-security as it suffices for the scope of this work. We also note that any adaptive, IND-secure IPFE is also selective, IND-secure.

Remark 3.4.11. *We note that* $\mathsf{pk}_{\mathbf{y}}$ *does not enable decryption by any means, so it will not change the correctness requirement. It will also not affect security of IPFE in any way. To see this, note that* $\mathsf{PubKGen}$ *is deterministic. So, an adversary can always compute* $\mathsf{pk}_{\mathbf{y}}$ *on its own. Given that* KGen *is also deterministic, one way to think of* $\mathsf{pk}_{\mathbf{y}}$ *is as a statistically binding and computationally hiding commitment to* $\mathsf{sk}_{\mathbf{y}}$*. Typically,* mpk *also has this property w.r.t.* msk *and it is always implicit in the security definitions of IPFE. We make this explicit below for* $\mathsf{pk}_{\mathbf{y}}$ *and* $\mathsf{sk}_{\mathbf{y}}$ *because our functional adaptor signatures will rely on it. We formalize this compliance requirement below.*

Definition 3.4.12 (R_{IPFE}-Compliant IPFE)**.** *For a hard relation* R_{IPFE}*, we say that an IPFE scheme is* R_{IPFE} *compliant if for any* $\lambda \in \mathbb{N}$*, for any* $\mathsf{pp} \leftarrow \mathsf{IPFE.Gen}(1^\lambda)$*, for any* $(\mathsf{mpk}, \mathsf{msk}) \leftarrow \mathsf{IPFE.Setup}(\mathsf{pp}, 1^\ell)$*, for any* $\mathbf{y} \in \mathcal{F}_{\mathsf{IP},\ell}$*, let* $\mathsf{pk}_{\mathbf{y}} := \mathsf{PubKGen}(\mathsf{mpk}, \mathbf{y})$*, and* $\mathsf{sk}_{\mathbf{y}} := \mathsf{KGen}(\mathsf{msk}, \mathbf{y})$*. Then, it must be the case that* $(\mathsf{pk}_{\mathbf{y}}, \mathsf{sk}_{\mathbf{y}}) \in R_{\mathsf{IPFE}}$*.*

Next, we describe an additional correctness property of IPFE that is tailored specifically to make IPFE compatible with adaptor signatures when the two are used in tandem in our functional adaptor signature construction. Looking ahead, we need this property because in our functional adaptor signature construction, we will use a standard adaptor signature to sell functional secret keys. Recall that adaptor signatures are defined w.r.t. two relations R and R'. In this case, R will be R_{IPFE} as defined in Definition 3.4.12 and let us denote the corresponding extended relation R' by R'_{IPFE}. Then, the AS.Ext algorithm of adaptor signature scheme AS will extract a functional secret key $\mathsf{sk}'_{\mathbf{y}}$ as the witness for the adaptor statement $\mathsf{pk}_{\mathbf{y}}$. While $\mathsf{sk}'_{\mathbf{y}}$ will be a witness of R'_{IPFE}, it may not be a witness of the base relation R_{IPFE} since $R_{\mathsf{IPFE}} \subseteq R'_{\mathsf{IPFE}}$, and we still want that $\mathsf{sk}'_{\mathbf{y}}$ should enable meaningful decryption. We formalize this robustness requirement below.

Definition 3.4.13 (R'_{IPFE}-Robust IPFE)**.** *Let* R_{IPFE} *be the hard relation as defined in Definition 3.4.12. Let* R'_{IPFE} *such that* $R_{\mathsf{IPFE}} \subseteq R'_{\mathsf{IPFE}}$ *be an an extended relation of* R_{IPFE}*. We say that an IPFE scheme is* R'_{IPFE} *robust if IPFE correctness holds even w.r.t. a functional secret key satisfying* R'_{IPFE}*. More formally, for any* $\lambda \in \mathbb{N}$*, let* $\mathsf{pp} \leftarrow \mathsf{IPFE.Gen}(1^\lambda)$*, then for any* $\ell \in \mathbb{N}$*,* $\mathbf{x} \in \mathcal{M}, \mathbf{y} \in \mathcal{F}_{\mathsf{IP},\ell}$*, the following holds:*

$$\Pr\left[\begin{array}{l} v = f_{\mathbf{y}}(\mathbf{x}) : \\ \\ \\ \\ \\ \end{array}\begin{array}{l} (\mathsf{mpk}, \mathsf{msk}) \leftarrow \mathsf{Setup}(\mathsf{pp}, \ell), \\ \mathsf{pk}_{\mathbf{y}} = \mathsf{PubKGen}(\mathsf{msk}, \mathbf{y}), \\ \mathsf{sk}'_{\mathbf{y}} \text{ s.t. } (\mathsf{pk}_{\mathbf{y}}, \mathsf{sk}'_{\mathbf{y}}) \in R'_{\mathsf{IPFE}}, \\ \mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{mpk}, \mathbf{x}), \\ v = \mathsf{Dec}(\mathsf{sk}'_{\mathbf{y}}, \mathsf{ct}) \end{array}\right] = 1.$$

Remark 3.4.14. *One can think of decryption correctness (Definition 3.4.9) as being implicitly defined with respect to* $\mathsf{sk}_{\mathbf{y}}$ *that is witness to* $\mathsf{pk}_{\mathbf{y}}$ *for the relation* R_{IPFE}*. And one can think of the* R'_{IPFE}*-robustness as decryption correctness with respect to* $\mathsf{sk}'_{\mathbf{y}}$ *that is witness to* $\mathsf{pk}_{\mathbf{y}}$ *for the relation* R'_{IPFE}*.*

Abdalla et al. [ABDP15] constructed selective, IND-secure IPFE schemes from DDH and LWE. Subsequently, Agrawal et al. [ALS16] constructed adaptive, IND-secure IPFE schemes from DDH, DCR, and LWE. Within the scope of this work, we will use the following two IPFE schemes:

- The DDH based IPFE by Abdalla et al. [ABDP15]. This scheme computes inner products with output values polynomially bounded by $B \ll p$. Specifically, the message space is $\mathcal{M} = \mathbb{Z}_p^\ell$ and the function class is $\mathcal{F}_{\mathsf{IP},\ell,p,B} = \{f_{\mathbf{y}} : \mathbf{y} \in \mathbb{Z}_p^\ell\} \subseteq \mathcal{F}_{\mathsf{IP},\ell}$, where $f_{\mathbf{y}} : \mathbb{Z}_p^\ell \to \{0,\ldots,B\}$ is defined as $f_{\mathbf{y}}(\mathbf{x}) = \mathbf{x}^T\mathbf{y} \in \{0,\ldots,B\}$.
- The LWE based IPFE by Agrawal et al. [ALS16, Section 4.2]. This scheme computes inner products modulo p. Specifically, the message space is $\mathcal{M} = \mathbb{Z}_p^\ell$ and the function class is $\mathcal{F}_{\mathsf{IP},\ell,p} = \{f_{\mathbf{y}} : \mathbf{y} \in \mathbb{Z}_p^\ell\} \subseteq \mathcal{F}_{\mathsf{IP},\ell}$, where $f_{\mathbf{y}} : \mathbb{Z}_p^\ell \to \mathbb{Z}_p$ is defined as $f_{\mathbf{y}}(\mathbf{x}) = \mathbf{x}^T\mathbf{y} \bmod p \in \mathbb{Z}_p$.

3.4.4 Lattice Preliminaries

We denote by $\mathcal{S}_c = \{\mathbf{x} \in \mathbb{Z}_q^n : ||\mathbf{x}||_\infty \leq c\}$ the set of vectors in $\mathbb{Z}_q^n$ whose maximum absolute coefficient is at most $c \in \mathbb{Z}^+$.

Definition 3.4.15 ($\mathsf{SIS}_{n,m,q,\beta}$)**.** *Let* $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{n\times m}$*. Given* $\mathbf{A}$*,* SIS *problem with parameters* $m > 1$ *and* $0 < \beta < q$ *asks to find a short non-zero* $\mathbf{v} \in \mathbb{Z}^m$ *such that* $\mathbf{Av} = 0 \bmod q$ *and* $||\mathbf{v}||_\infty \leq \beta$*.*

Next, we define SIS in Hermite Normal Form (HNF).

Definition 3.4.16 ($\mathsf{HNF\text{-}SIS}_{n,m,q,\beta}$)**.** *Let* $\mathbf{A}' \xleftarrow{\$} \mathbb{Z}_q^{n\times(m-1)}$ *and* $\mathbf{A} = (\mathbf{1}||\mathbf{A}') \in \mathbb{Z}_q^{n\times m}$*. Given* $\mathbf{A}$*,* HNF-SIS *problem with parameters* $m > 1$ *and* $0 < \beta < q$ *asks to find a short non-zero* $\mathbf{v} \in \mathbb{Z}^m$ *such that* $\mathbf{Av} = 0 \bmod q$ *and* $||\mathbf{v}||_\infty \leq \beta$*.*

Definition 3.4.17 ($\mathsf{ISIS}_{n,m,q,\beta}$)**.** *It is the inhomogeneous version of* HNF-SIS*. Let* $\mathbf{A}' \xleftarrow{\$} \mathbb{Z}_q^{n\times(m-1)}$ *and* $\mathbf{A} = (\mathbf{1}||\mathbf{A}') \in \mathbb{Z}_q^{n\times m}$ *and let* $\mathbf{u} \xleftarrow{\$} \mathbb{Z}^m$ *s.t.* $||\mathbf{u}||_\infty \leq \beta$*. Let* $\mathbf{b} = \mathbf{Au} \in \mathbb{Z}_q^n$*. Given* $(\mathbf{A},\mathbf{b})$*, the problem with parameters* $m > 1$ *and* $0 < \beta < q$ *asks to find a short non-zero* $\mathbf{v} \in \mathbb{Z}^m$ *such that* $\mathbf{Av} = \mathbf{b} \bmod q$ *and* $||\mathbf{v}||_\infty \leq \beta$*.*

Definition 3.4.18 ($\mathsf{LWE}_{n,m,q,\Psi}$)**.** LWE *problem with parameters* $n, m > 0$ *and distribution* Ψ *over* $\mathbb{Z}_q$ *asks to distinguish between the following two cases with non-negligible advantage: 1)* $(\mathbf{A},\mathbf{b}) \xleftarrow{\$} \mathbb{Z}_q^{m\times n} \times \mathbb{Z}_q^m$*, and 2)* $(\mathbf{A}, \mathbf{As}+\mathbf{e})$ *for* $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{m\times n}$*, a secret* $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^n$ *and an error vector* $\mathbf{e} \leftarrow \Psi^m$*.*

Definition 3.4.19 (Multi-hint extended-LWE). *Let q, m, t be integers, α be a real and τ be a distribution over $\mathbb{Z}^{t\times m}$, all of them functions of a parameter n. The multi-hint extended-LWE problem $\mathsf{mheLWE}_{n,m,q,\alpha,t,\tau}$ is to distinguish between the distributions of the tuples*

$$(\mathbf{A}, \mathbf{b}, \mathbf{Z}, \mathbf{Ze}) \text{ and } (\mathbf{A}, \mathbf{As} + \mathbf{e}, \mathbf{Z}, \mathbf{Ze}),$$

where $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{m\times n}$, $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^n$, $\mathbf{b} \xleftarrow{\$} \mathbb{Z}_q^m$, $\mathbf{e} \leftarrow D_{\mathbb{Z},\alpha q}^m$, $\mathbf{Z} \leftarrow \tau$.

Lemma 3.4.20 ([ALS16], Theorem 4). *Let $n \geq 100$, $q \geq 2$, $t < n$ and m with $m = \Omega(n \log n)$ and $m \leq n^{O(1)}$. There exists $\xi \leq O(n^4 m^2 \log^{5/2} n)$ and a distribution τ over $\mathbb{Z}^{t\times m}$ such that the following statements hold:*

- *There is a reduction from $\mathsf{LWE}_{n-t,m,q,D_{\mathbb{Z},\alpha q}}$ to $\mathsf{mheLWE}_{n,m,q,\alpha,t,\tau}$.*
- *It is possible to sample from τ in polynomial time in n.*
- *Each entry of matrix τ is an independent discrete Gaussian $\tau_{i,j} = D_{\mathbb{Z},\sigma_{i,j},c_{i,j}}$ for some $c_{i,j} \in \{0,1\}$ and $\sigma_{i,j} \geq \Omega(\sqrt{mn \log m})$.*
- *With probability at least $1 - n^{-\omega(1)}$, all rows from a sample from τ have norms at most ξ.*

Definition 3.4.21 (Discrete Gaussian distribution). *A discrete gaussian distribution $D_{\Lambda,\sqrt{\Sigma},\mathbf{c}}$, for $\mathbf{c} \in \mathbb{R}^n$, Σ a positive semi-definite matrix in $\mathbb{R}^{n\times n}$, and a lattice $\Lambda \subset \mathbb{Z}^n$, is a distribution with values in Λ and probabilities*

$$\Pr[X = \mathbf{x}] \propto \exp\left(-\frac{1}{2}(\mathbf{x} - \mathbf{c})^T \Sigma^+ (\mathbf{x} - \mathbf{c})\right).$$

Note that Σ^+ denotes the pseudoinverse of a matrix. If $\Lambda = \mathbb{Z}^n$, we shall just write $D_{\sqrt{\Sigma},\mathbf{c}}$. Further, if $\mathbf{c} = 0$, then we shall just write $D_{\sqrt{\Sigma}}$, and if $\sqrt{\Sigma} = \sigma I_n$ for $\sigma \in \mathbb{R}^+$ and I_n an identity matrix, we shall just write D_σ.

3.4.5 Public Key Encryption

Definition 3.4.22 (Public Key Encryption Scheme). *A public key encryption scheme $\mathsf{PKE} = (\mathsf{KGen}, \mathsf{Enc}, \mathsf{Dec})$ is a tuple of three algorithms KGen, Enc, Dec, defined as follows:*

(i) $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KGen}(1^\lambda)$*: The key generation algorithm KGen takes as input the security parameter, and outputs a key pair $(\mathsf{pk}, \mathsf{sk})$, where pk is called the public key, and sk is called the secret key.*

(ii) $\mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{pk}, m)$*: The encryption algorithm takes as input a public key pk and a message m, and outputs a ciphertext ct, which is an encryption of m under the public key pk.*

(iii) $m \leftarrow \mathsf{Dec}(\mathsf{sk}, \mathsf{ct})$*: The decryption algorithm takes as input a secret key sk and a ciphertext ct, and outputs the plaintext m.*

Definition 3.4.23 (Semantic Security for PKE). *A public key encryption scheme is semantically secure if for all p.p.t. adversaries $\mathcal{A}$, there exists a negligible function ϵ such that for all $\lambda \in \mathbb{N}$, we have:*

$$\mathsf{Adv}^{\mathsf{PKE}}_{\mathcal{A}} = \left| \Pr\left[G_0^{\mathcal{A}}(1^\lambda) = 1\right] - \Pr\left[G_1^{\mathcal{A}}(1^\lambda) = 1\right] \right| \leq \epsilon(\lambda),$$

where games G_0 and G_1 are defined as in Figure 3.6.

Game $G_b^{\mathcal{A}}(1^\lambda)$

1 : $(\mathsf{pk}, \mathsf{sk}) \leftarrow\!\$\ \mathsf{KGen}(1^\lambda)$
2 : $m_0, m_1 \leftarrow \mathcal{A}(\mathsf{pk})$
3 : $\mathsf{ct}_b \leftarrow \mathsf{PKE.Enc}(\mathsf{pk}, m_b)$
4 : $b' \leftarrow \mathcal{A}(\mathsf{ct}_b)$
5 : return b'

Figure 3.6: Semantic Security Game G_b

El Gamal Encryption Scheme. An encryption scheme that will be useful to us later is the ElGamal encryption scheme. We provide the details of the multiplicative version of this encryption scheme in Definition 3.4.24.

Definition 3.4.24 (Exponential ElGamal Encryption). *Let $\mathbb{G}$ be a cyclic group with prime order p where the* DL *problem is hard, and let g be a generator of $\mathbb{G}$. Then we denote the multiplicative ElGamal Encryption Scheme by* $\mathsf{ElGamal} := (\mathsf{KGen}, \mathsf{Enc}, \mathsf{Dec})$*, and is defined in Figure 3.7.*

$\mathsf{KGen}(1^\lambda)$:

1 : $\mathsf{sk} \leftarrow\!\$\ \mathbb{Z}_p$
2 : $\mathsf{pk} := g^{\mathsf{sk}}$
3 : return $(\mathsf{pk}, \mathsf{sk})$

$\mathsf{Enc}(\mathsf{pk}, m \in \mathbb{Z}_p)$:

1 : $\beta \leftarrow\!\$\ \mathbb{Z}_p$
2 : $\mathsf{ct}_1 := g^\beta$
3 : $\mathsf{ct}_2 := g^m \cdot \mathsf{pk}^\beta$
4 : return $\mathsf{ct} = (\mathsf{ct}_1, \mathsf{ct}_2)$

$\mathsf{Dec}(\mathsf{sk}, \mathsf{ct})$:

1 : Parse ct as $(\mathsf{ct}_1, \mathsf{ct}_2)$
2 : $\mathsf{ct}' := \mathsf{ct}_2 \cdot (\mathsf{ct}_1)^{-\mathsf{sk}}$
3 : $m := \mathsf{dlog}_g(\mathsf{ct}')$
4 : return m

Figure 3.7: Exponential ElGamal Encryption Algorithm Definitions

Theorem 3.4.25 (ElGamal Semantic Security). *The ElGamal* PKE *scheme is semantically secure.*

It will be useful later to encrypt messages $m \in \mathbb{Z}_p$ bitwise using ElGamal. That is, we decompose m into its bit representation, and encrypt each bit separately using exponential ElGamal. We will call this bitwise ElGamal encryption.

3.4.6 Non-Interactive Zero Knowledge (NIZK) Arguments

Definition 3.4.26 (Non-Interactive Zero Knowledge (NIZK) Argument)**.** *A* NIZK *argument for a language* L_{R}*, where* R *is a relation between statement/witness pairs, in the common reference string (CRS) model is defined by the three algorithms:*

(i) $\mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda)$*: The setup algorithm takes the security parameter* λ *as input, and outputs a common reference string* crs *which is then released publicly. The* crs *will be used to prove statements non-interactively.*

(ii) $\pi \leftarrow \mathsf{Prove}(\mathsf{crs}, \mathsf{stmt}, \mathsf{wit})$*: The* Prove *algorithm takes as input the common reference string* crs*, a statement* stmt*, and a witness* wit*, and outputs a corresponding argument* π*.*

(iii) $b \in \{0,1\} \leftarrow \mathsf{Verify}(\mathsf{crs}, \mathsf{stmt}, \pi)$*: The verification algorithm takes as input the common reference string* crs*, a statement* stmt*, and an argument* π*, and outputs a bit* b *indicating the validity of the argument.*

In order for a NIZK *argument system to be secure, it must satisfy three properties:* completeness *(Definition 3.4.27),* adaptive soundness *(Definition 3.4.28), and* zero-knowledge *(Definition 3.4.29).*

Definition 3.4.27 (Completeness)**.** *A* NIZK *argument for a language* L_{R} *satisfies* completeness *if for all statement/witness pairs* $(\mathsf{stmt}, \mathsf{wit}) \in \mathsf{R}$*, we have that if* $\mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda)$ *and* $\pi \leftarrow \mathsf{Prove}(\mathsf{crs}, \mathsf{stmt}, \mathsf{wit})$*, then:*

$$\Pr[\mathsf{Verify}(\mathsf{crs}, \mathsf{stmt}, \pi) = 1] = 1$$

Definition 3.4.28 (Adaptive Soundness)**.** *A* NIZK *argument for a language* L_{R} *satisfies* adaptive soundness *if for all p.p.t. provers* $\mathcal{P}$*, there exists a negligible function* ϵ *such that for all* $\lambda \in \mathbb{N}$*, we have that if* $\mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda)$ *and* $(\mathsf{stmt}, \pi) \leftarrow \mathcal{P}(\mathsf{crs})$*, then:*

$$\Pr[\mathsf{stmt} \notin L_{\mathsf{R}} \wedge \mathsf{Verify}(\mathsf{crs}, \mathsf{stmt}, \pi) = 1] \leq \epsilon(\lambda)$$

Definition 3.4.29 (Zero Knowledge)**.** *A* NIZK *argument for a language* L_{R} *satisfies* zero knowledge *if there exists a p.p.t. simulator* $\mathsf{Sim} = (\mathsf{Setup}^*, \mathsf{Prove}^*)$ *and there exists a negligible function* ϵ *such that for all p.p.t. distinguishers* $\mathcal{D}$*,* $\lambda \in \mathbb{N}$*, and statement/witness pairs* $(\mathsf{stmt}, \mathsf{wit}) \in \mathsf{R}$*, we have that if* $\mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda)$*,* $\pi \leftarrow \mathsf{Prove}(\mathsf{crs}, \mathsf{stmt}, \mathsf{wit})$*,* $(\mathsf{crs}^*, \mathsf{td}) \leftarrow \mathsf{Setup}^*(1^\lambda)$*, and* $\pi^* \leftarrow \mathsf{Prove}^*(\mathsf{crs}^*, \mathsf{stmt}, \mathsf{td})$*, then:*

$$\left| \Pr[\mathcal{D}(\mathsf{crs}, \pi) = 1] - \Pr[\mathcal{D}(\mathsf{crs}^*, \pi^*) = 1] \right| \leq \epsilon(\lambda)$$

where td *is a trapdoor that the simulator* Sim *uses to generate the argument* π^* *for the statement* stmt *without knowing the corresponding witness* wit.

Later in Section 3.12, for the zero knowledge proof of equality, we will need to prove in zero knowledge that a given ElGamal encryption ct encrypts a bit $b \in \{0,1\}$. Boneh and Shoup provide a zero knowledge protocol for this (Example 20.3 [BS17]), and it relies on the Chaum-Pedersen protocol that proves a given triple is a Diffie-Hellman triple. For completeness, we include the Chaum-Pedersen protocol here (considering the version in Boneh-Shoup Section 19.5.2 [BS17]).

Definition 3.4.30 (Chaum-Pedersen Protocol)**.** *Let* $\mathbb{G}$ *be a cyclic group of prime order* p *with some generator* $g \in \mathbb{G}$*. For some* $\alpha, \beta, \gamma \in \mathbb{Z}_p$*, we say that* $(g^\alpha, g^\beta, g^\gamma)$ *is a DH-triple if* $\gamma = \alpha\beta$*. Similarly, we can then say a triple* $(u, v, w) \in \mathbb{G}^3$ *is a DH-triple if and only if there exists a* $\beta \in \mathbb{Z}_p$ *such that* $v = g^\beta$*, and* $w = u^\beta$*. The relation for the Chaum-Pedersen Protocol is then:*

$$\mathsf{R}_{\mathsf{cp}} = \{(\beta, (u, v, w)) \in \mathbb{Z}_p \times \mathbb{G}^3 \mid v = g^\beta \wedge w = u^\beta\}$$

The three rounds of the sigma protocol Π_{cp} *are as follows:*

1. *The prover samples* $\beta_t \leftarrow\!\!\$\ \mathbb{Z}_p$*, and sets* $v_t \leftarrow g^{\beta_t}$ *and* $w_t \leftarrow u^{\beta_t}$*, sending* (v_t, w_t) *to the verifier.*
2. *The verifier samples a challenge* $c \leftarrow\!\!\$\ \mathcal{C}$*, and sends* c *to the prover.*
3. *The prover calculates* $\beta_z = \beta_t + \beta \cdot c$*, and sends* β_z *to the verifier.*

The verifier then does two checks to verify that (u, v, w) *is a DH-triple:*

(1) $g^{\beta_z} = v_t \cdot v^c$

(2) $u^{\beta_z} = u_t \cdot u^c$

Boneh and Shoup also provide a proof for the special soundness of the Π_{cp} protocol. The idea is that they can construct an extractor so that when given two accepting conversations $((v_t, w_t), c, \beta_z)$ and $((v_t, w_t), c', \beta'_z)$ where $c \neq c'$, a witness β can be extracted, where:

$$\beta = \Delta\beta / \Delta c,\ \Delta\beta = \beta_z - \beta'_z, \text{ and } \Delta c = c - c'$$

For completeness, we include a sketch of the protocol to prove that an ElGamal encryption ct encrypts a bit. As stated before, it utilizes the Chaum-Pedersen protocol.

Definition 3.4.31 (Encrypted ElGamal Bits Relation)**.** *Let* $\mathbb{G}$ *be a cyclic group of prime order* p *with some generator* $g \in \mathbb{G}$*. Suppose we have a bit ElGamal encryption* $(v, e) = (g^\beta, g^b u^\beta)$*. We can show that* (v, e) *is a valid ElGamal encryption of a bit by proving the following relation:*

$$\mathsf{R}_{\mathsf{egb}} = \left\{ ((b,\beta),(u,v,e)) \;\middle|\; \begin{array}{c} v = g^{\beta} \wedge e = u^{\beta} g^{b} \\ \wedge\ b \in \{0,1\} \end{array} \right\}$$

The protocol for the above relation relies on the observation that either (u, v, e) or $(u, v, e/g)$ is a DH-triple. So we can simply apply the Chaum-Pedersen protcol to both tuples (u, v, e) and $(u, v, e/g)$, and show that one of them is a valid DH-triple. This approach is enabled by the OR-proof construction found in Boneh-Shoup (Section 19.7.2 [BS17]). Throughout the chapter, we will denote this sigma protocol as Π_{egb}.

3.4.7 Homomorphic Encryption

Definition 3.4.32 (Homomorphic Property of Encryption Schemes)**.** *A public-key encryption scheme is said to be* homomorphic *with respect to some operation $*$ if for every message m_1, m_2:*

$$\mathsf{Enc}(m_1) * \mathsf{Enc}(m_2) = \mathsf{Enc}(m_1 * m_2)$$

Definition 3.4.33 (Homomorphic Encryption Scheme)**.** *A Homomorphic Encryption Scheme is denoted* $\mathsf{HE} := (\mathsf{KGen}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{Eval})$*, where the algorithms are defined as follows (note that here we assume the method uses asymmetric keys):*

(i) $(\mathsf{pk}, \mathsf{sk}) \leftarrow\!\!\$\ \mathsf{KGen}(1^{\lambda})$*: The key generation algorithm* $\mathsf{KGen}(1^{\lambda})$ *takes as input the security parameter, and outputs a public/secret key pair* $(\mathsf{pk}, \mathsf{sk})$*.*

(ii) $\mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{pk}, m)$*: The encryption algorithm* Enc *takes as input the public key* pk*, and a message m, and outputs the encryption* ct *of m under the key* pk*.*

(iii) $m' \leftarrow \mathsf{Dec}(\mathsf{sk}, \mathsf{ct})$*: The decryption algorithm* Dec *takes as input the secret key* sk*, and a ciphertext* ct*, and outputs the plaintext m' from the decryption of* ct*.*

(iv) $\widehat{ct} \leftarrow \mathsf{Eval}(\mathsf{pk}, \mathsf{ct}, f)$*: The evaluation algorithm* Eval *takes as input the public key* pk*, a ciphertext* ct*, and a function f, and outputs the result of* $\widehat{\mathsf{ct}} = f(\mathsf{ct})$*. Note that the function f should preserve the structure of the encryption, and* $\widehat{\mathsf{ct}}$ *can be decrypted so that the value $f(m)$ is extracted, where m is the corresponding plaintext to* ct*.*

Remark. Different HE schemes have different levels of homomorphism, and it is important to identify the differences between them. We further classify HE schemes into three main groups, as defined in Definition 3.4.34. Note that for each classification, the encryption schemes still have the same structure as detailed in Definition 3.4.33, but the way that the Eval algorithm is implemented depends on which classification the scheme falls into.

Definition 3.4.34 (Classification of Homomorphic Encryption Schemes)**.** *We classify homomorphic encryption schemes into three main groups:*

*(i) Partially Homomorphic Encryption (*PHE*): An encryption scheme that is homomorphic with respect to* one operation*, and can perform the operation an* unlimited *number of times.*

*(ii) Somewhat Homomorphic Encryption (*SWHE*): An encryption scheme that is homomorphic with respect to* more than one operation*, but can only perform the operations a* limited *number of times.*

*(iii) Fully Homomorphic Encryption (*FHE*): An encryption scheme that is homomorphic with respect to an* unlimited number of operations*, and can perform the operations an* unlimited *number of times.*

An example of SWHE would be the BGN encryption scheme [BGN05]. It is homomorphic with respect to addition and multiplication, but can only do one multiplication.

3.5 Definition of FAS

A functional adaptor signature scheme shares similar syntax to an adaptor signature scheme with few additional interfaces and parameters. Specifically, the interfaces are $\mathsf{FAS} := (\mathsf{Setup}, \mathsf{AdGen}, \mathsf{AdVerify}, \mathsf{FPreSign}_1, \mathsf{FPreVerify}_1, \mathsf{FPreSign}, \mathsf{FPreVerify}, \mathsf{Adapt}, \mathsf{FExt})$. We recall that $(\mathsf{FPreSign}_1, \mathsf{FPreVerify}_1, \mathsf{FPreSign})$ essentially denote the 3-round interactive pre-signing process. We describe the formal definition below.

Definition 3.5.1 (Functional Adaptor Signature)**.** *A FAS scheme* FAS *is defined w.r.t. a signature scheme* DS *, a hard relation* R*, and a family of functions $\mathcal{F}$, and consists of algorithms* Setup*,* AdGen*,* AdVerify*,* FPreSign*,* FPreVerify*,* Adapt*,* FExt *defined below:*

- $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$*: The setup algorithm outputs public parameters* pp *on input security parameter* 1^λ*.*
- $(\mathsf{ad}, \mathsf{st}) \leftarrow \mathsf{AdGen}(\mathsf{pp}, Y, y)$*: The advertisement generation algorithm on inputs* pp*, a statement Y for the language L_R, and a witness y, outputs a public advertisement* ad *and a secret state* st*.*
- $b \leftarrow \mathsf{AdVerify}(\mathsf{pp}, \mathsf{ad}, Y)$*: The advertisement verification algorithm on input the tuple* $(\mathsf{pp}, \mathsf{ad}, Y)$*, outputs bit* $b \in \{0,1\}$ *checking validity of* ad*.*
- $(\widetilde{\sigma}_1, \widetilde{\sigma}_2) \leftarrow \mathsf{FPreSign}$*, defined as* $\mathsf{FPreSign} = (\mathsf{FPreSign}_1, \mathsf{FPreSign}_2)$ *models a two-round protocol where the seller runs* $\mathsf{FPreSign}_1(\mathsf{pp}, \mathsf{ad}, \mathsf{st}, Y, y, f)$ *to generate $\widetilde{\sigma}_1$ for a function f, which is then used by the buyer to generate $\widetilde{\sigma}_2$ by running* $\mathsf{FPreSign}_2(\mathsf{pp}, \mathsf{ad}, \mathsf{sk}, m, Y, f, \widetilde{\sigma}_1)$*.*
- $(b_1, b_2) \leftarrow \mathsf{FPreVerify}$*, defined as* $\mathsf{FPreVerify} = (\mathsf{FPreVerify}_1, \mathsf{FPreVerify}_2)$ *where buyer runs* $\mathsf{FPreVerify}_1(\mathsf{pp}, \mathsf{ad}, Y, f, \widetilde{\sigma}_1)$ *to check $\widetilde{\sigma}_1$'s validity and seller runs* $\mathsf{FPreVerify}_2(\mathsf{pp}, \mathsf{ad}, \mathsf{vk}, m, Y, f, \widetilde{\sigma} := (\widetilde{\sigma}_1, \widetilde{\sigma}_2))$ *to check $\widetilde{\sigma}_2$'s validity.*

- $\sigma := \mathsf{Adapt}(\mathsf{ad}, \mathsf{st}, \mathsf{vk}, m, Y, y, f, \widetilde{\sigma} := (\widetilde{\sigma}_1, \widetilde{\sigma}_2))$: *The adapt algorithm outputs a valid signature* σ *on inputs* ad, st, *a verification key* vk, m, *a statement/witness pair* (Y, y), *a function* f, *and a pre-signature* $\widetilde{\sigma}$.
- $z := \mathsf{FExt}(\mathsf{ad}, \mathsf{vk}, Y, f, \widetilde{\sigma} := (\widetilde{\sigma}_1, \widetilde{\sigma}_2), \sigma)$: *The functional extract algorithm takes as input* ad, vk, *statement* Y, *a function* f *in family* $\mathcal{F}$, *a pre-signature* $\widetilde{\sigma}$, *and a signature* σ, *and outputs a value* z.

In a typical functional payment application, we will have a seller who holds a witness to the statement and wants to sell a function evaluation on it, and a buyer who wants to buy a function evaluation of the witness. We refer the reader to Figure 3.1 for the protocol flow.

In an honest run, FExt must return $z = f(y)$. Informally, security of FAS is formalized in two cases:

(1) For corrupt sellers, FAS must mainly guarantee (1) *advertisement binding*, where an adversary who comes up with (Y, ad) such that AdVerify passes must be bound to a unique witness y, (2) *unforgeability*, where an adversary who doesn't know the witness cannot forge honest buyers' signatures, and (3) *witness extractability*, where the adversary cannot publish a buyer's signature such that when the buyer tries to extract using FExt, it gets $z' \neq f(y)$. [14] Crucially, in the case of FAS, the adversary is allowed to get many functional pre-signatures for any choice of function f.

(2) For corrupt buyers, FAS must guarantee *pre-signature adaptability*, where if the adversary outputs a valid functional pre-signature, then adapting it using a valid witness y should result in a valid signature under the buyer's key. Most importantly, FAS must guarantee protection of the seller's witness y, which we capture in the notion called *witness privacy*. Witness privacy can be modeled in three different ways, and the strongest notion among them is *zero-knowledge*. In a little more detail, FAS satisfies *zero-knowledge* witness privacy if a malicious buyer learns no more information about y than $f(y)$ after a complete interaction, or learns nothing at all if it pre-maturely aborts.

Alternate Notations. Sometimes, for brevity, we drop pp from the inputs of various algorithms when it is clear from the context. If an algorithm does not use some of its inputs, we omit those inputs for brevity.

Next, we present the formal correctness and security definitions for FAS.

Definition 3.5.2 (Correctness)**.** *A functional adaptor signature scheme* FAS *satisfies correctness if for every* $\lambda \in \mathbb{N}$, *every message* $m \in \{0,1\}^*$, *every statement/witness pair* $(Y, y) \in \mathsf{R}$, *and every function* $f \in \mathcal{F}$, *suppose that:* $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$, $(\mathsf{ad}, \mathsf{st}) \leftarrow \mathsf{AdGen}(\mathsf{pp}, Y, y)$, $(\mathsf{sk}, \mathsf{vk}) \leftarrow \mathsf{KGen}(1^\lambda)$, $\widetilde{\sigma}_1 \leftarrow \mathsf{FPreSign}_1(\mathsf{pp}, \mathsf{ad}, \mathsf{st}, Y, y, f)$, $\widetilde{\sigma}_2 \leftarrow \mathsf{FPreSign}_2(\mathsf{pp}, \mathsf{ad}, \mathsf{sk}, m, Y, f, \widetilde{\sigma}_1)$, $\sigma := \mathsf{Adapt}(\mathsf{ad}, \mathsf{st}, \mathsf{vk}, m, Y, y, f, \widetilde{\sigma} := (\widetilde{\sigma}_1, \widetilde{\sigma}_2))$, *and* $z := \mathsf{FExt}(\mathsf{ad}, \mathsf{vk}, Y, f, \widetilde{\sigma} := (\widetilde{\sigma}_1, \widetilde{\sigma}_2), \sigma)$. *Then we have that:*

[14] We also require *advertisement soundness* and *pre-signature validity* properties against a corrupt seller. We defer this discussion, along with formal security definitions, to the full version.

$$\Pr\left[\begin{array}{c}\mathsf{AdVerify}(\mathsf{pp},\mathsf{ad},Y)=1\\ \wedge\ \mathsf{FPreVerify}_1(\mathsf{pp},\mathsf{ad},Y,f,\widetilde{\sigma}_1)=1\\ \wedge\ \mathsf{FPreVerify}_2(\mathsf{pp},\mathsf{ad},\mathsf{vk},m,Y,f,\widetilde{\sigma})=1\\ \wedge\mathsf{Vrfy}(\mathsf{vk},m,\sigma)=1\wedge z=f(y)\end{array}\right]=1$$

In terms of security, we define the following properties against malicous seller and buyer:

- **Malicious seller:** advertisement soundness, advertisement binding, pre-signature validity, unforgeability, witness extractability.
- **Malicious buyer:** pre-signature adaptability, witness privacy.

Looking ahead, we will consider three notions of witness privacy: zero-knowledge, witness indistinguishability, and witness hiding. Among these, zero-knowledge is the strongest. Based on this, we define two overall security levels of functional adaptor signatures below.

Definition 3.5.3 (Strongly-secure Functional Adaptor Signature Scheme)**.** *A* FAS *scheme is* strongly-secure *if it is advertisement sound, advertisement binding, pre-signature valid, unforgeable, witness extractable, pre-signature adaptable and zero-knowledge.*

Definition 3.5.4 (Weakly-secure Functional Adaptor Signature Scheme)**.** *A* FAS *scheme is* weakly-secure *if it is advertisement sound, advertisement binding, pre-signature valid, unforgeable, witness extractable, pre-signature adaptable and witness indistinguishable.*

We describe all the security properties next.

Advertisement Soundness. The property requires that given pp, it should be infeasible for a p.p.t. malicious seller to find statement $Y \notin L_R$ and advertisement ad such that ad is accepted by AdVerify.

Definition 3.5.5 (Advertisement Soundness)**.** *A functional adaptor signature scheme* FAS *satisfies advertisement soundness if there exists a negligible function* negl *such that for every p.p.t. malicious seller $\mathcal{A}$, for all $\lambda \in \mathbb{N}$, for all NP languages L_R, if* $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$*, and* $(Y, \mathsf{ad}) \leftarrow \mathcal{A}(\mathsf{pp})$*, then we have that:*

$$\Pr[(Y \notin L_R) \wedge (\mathsf{AdVerify}(\mathsf{pp},\mathsf{ad},Y)=1)] \leq \mathsf{negl}(\lambda)$$

Advertisement Binding. The property requires that given pp, if a p.p.t. malicious seller $\mathcal{A}$ outputs (Y, ad) such that $\mathsf{AdVerify}(\mathsf{pp}, Y, \mathsf{ad}) = 1$, then with overwhelming probability there exists exactly one unique pair (y, r) such that $\mathsf{AdGen}(\mathsf{pp}, Y, y; r) = (\mathsf{ad}, \cdot)$.

Definition 3.5.6 (Advertisement Binding)**.** *A functional adaptor signature scheme* FAS *satisfies advertisement binding if there exists a negligible function* negl *such that for every p.p.t. malicious seller* $\mathcal{A}$*, for all* $\lambda \in \mathbb{N}$*, if* $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$*,* $(Y, \mathsf{ad}) \leftarrow \mathcal{A}(\mathsf{pp})$*, such that* $\mathsf{AdVerify}(\mathsf{pp}, Y, \mathsf{ad}) = 1$*, then both the following hold:*

$$\Pr\left[\begin{array}{c} \exists (y, r), (y', r') \; s.t. \\ y \neq y' \wedge \\ \mathsf{AdGen}(\mathsf{pp}, Y, y; r) = (\mathsf{ad}, \cdot) \wedge \\ \mathsf{AdGen}(\mathsf{pp}, Y, y'; r') = (\mathsf{ad}, \cdot) \end{array}\right] \leq \mathsf{negl}(\lambda) \; ,$$

$$\Pr\left[\begin{array}{c} \nexists (y, r) \; s.t. \\ \mathsf{AdGen}(\mathsf{pp}, Y, y; r) = (\mathsf{ad}, \cdot) \end{array}\right] \leq \mathsf{negl}(\lambda) \; .$$

Informally, both above equations ensure that every (Y, ad) *(even maliciously chosen) has exactly one tuple* (y, r) *that results in* ad *via* AdGen*. Such a tuple can be computed (possibly inefficiently) by brute-force search algorithm which we refer to as the extractor* $\mathcal{E}$*.*

Pre-signature validity. This property says that given a valid $\widetilde{\sigma}_1$ value, one can always compute a valid pre-signature $\widetilde{\sigma}_2$ using it. Intuitively, this tries to capture that it should be impossible for a malicious seller to give out a malformed $\widetilde{\sigma}_1$ value for the function f and yet learn an honestly generated pre-signature $\widetilde{\sigma}_2$ for f that is valid.

Definition 3.5.7 (Pre-signature Validity)**.** *A functional adaptor signature scheme* FAS *satisfies pre-signature validity if for any* $\lambda \in \mathbb{N}$*, any* $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$*, any* X, ad *such that* $\mathsf{AdVerify}(\mathsf{pp}, \mathsf{ad}, X) = 1$*, any message* $m \in \{0,1\}^*$*, any function class* $\mathcal{F}$*, any function* $f \in \mathcal{F}$*, any* $\widetilde{\sigma}_1$*, any key pair* $(\mathsf{sk}, \mathsf{vk}) \leftarrow \mathsf{KGen}(1^\lambda)$*, any pre-signature* $\widetilde{\sigma}_2 \leftarrow \mathsf{FPreSign}_2(\mathsf{pp}, \mathsf{ad}, \mathsf{sk}, m, Y, f, \widetilde{\sigma}_1)$*, we have that if* $\mathsf{FPreVerify}_1(\mathsf{pp}, \mathsf{ad}, Y, f, \widetilde{\sigma}_1) = 1$*, then,*

$$\Pr[\mathsf{FPreVerify}_2(\mathsf{pp}, \mathsf{ad}, \mathsf{vk}, m, Y, f, \widetilde{\sigma} = (\widetilde{\sigma}_1, \widetilde{\sigma}_2)) = 1] = 1.$$

Unforgeability. This is similar to the unforgeability security property typically defined for adaptor signatures [AEE+21]. The property requires that even when the adversary (malicious seller) is given access to pre-signatures with respect to functions f of its choice, it should not be able to forge signatures if it does not know the witness y^* for the challenge statement Y^* in the first place. To capture this essence, the experiment (See Figure 3.8) samples (Y^*, y^*) at random. Beyond this, the adversary has all the powers of a (malicious) seller starting with generating ad. A curious reader might ask if the adversary doesn't know y^*, then, wouldn't it typically fail to pass the AdVerify check in step 5 in Figure 3.8 and thus rendering the experiment after that pointless? The answer is no, because *advertisement soundness* is for statements not in the language L_R, but here $Y^* \in L_R$ as the experiment chooses it.

Experiment $\mathsf{fUnforge}_{\mathcal{A},\mathsf{FAS}}(1^\lambda)$

1 : $Q := \emptyset, \mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$
2 : $(\mathsf{sk}, \mathsf{vk}) \leftarrow \mathsf{AS.KGen}(1^\lambda)$
3 : $(Y^*, y^*) \leftarrow \mathsf{GenR}(1^\lambda)$
4 : $(\mathsf{ad}^*, m^*, f^*, \widetilde{\sigma}_1^*) \leftarrow \mathcal{A}^{\mathcal{O}_s, \mathcal{O}_{\mathsf{fps}_2}}(\mathsf{pp}, \mathsf{vk}, Y^*)$
5 : if $\big(\mathsf{AdVerify}(\mathsf{pp}, \mathsf{ad}^*, Y^*) = 0 \vee (f^* \notin \mathcal{F})$
$\vee\ \mathsf{FPreVerify}_1(\mathsf{pp}, \mathsf{ad}^*, Y^*, f^*, \widetilde{\sigma}_1^*) = 0\big)$:
return $\perp$
6 : $\widetilde{\sigma}_2^* \leftarrow \mathsf{FPreSign}_2(\mathsf{pp}, \mathsf{ad}^*, \mathsf{sk}, m^*, Y^*, f^*, \widetilde{\sigma}_1^*)$
7 : $\sigma^* \leftarrow \mathcal{A}^{\mathcal{O}_s, \mathcal{O}_{\mathsf{fps}_2}}(\widetilde{\sigma}_2^*)$
8 : return $(m^* \notin Q) \wedge \mathsf{Vrfy}(\mathsf{vk}, m^*, \sigma^*) = 1$

Oracle $\mathcal{O}_s(m)$

1 : $\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$
2 : $Q := Q \cup \{m\}$
3 : return σ

Oracle $\mathcal{O}_{\mathsf{fps}_2}(\mathsf{ad}, m, Y, f, \widetilde{\sigma}_1)$

1 : if $(\mathsf{FPreVerify}_1(\mathsf{pp}, \mathsf{ad}, Y, f, \widetilde{\sigma}_1) = 0)$:
return $\perp$
2 : $\widetilde{\sigma}_2 \leftarrow \mathsf{FPreSign}_2(\mathsf{pp}, \mathsf{ad}, \mathsf{sk}, m, Y, f, \widetilde{\sigma}_1)$
3 : $Q := Q \cup \{m\}$
4 : return $\widetilde{\sigma}_2$

Figure 3.8: Unforgeability experiment for Functional Adaptor Signature Schemes

Definition 3.5.8 (Unforgeability)**.** *A functional adaptor signature scheme* FAS *satisfies unforgeability if for every stateful p.p.t. adversary $\mathcal{A}$, there exists a negligible function ϵ such that for all $\lambda \in \mathbb{N}$:*

$$\Pr\big[\mathsf{fUnforge}_{\mathcal{A},\mathsf{FAS}}(1^\lambda) = 1\big] \leq \epsilon(\lambda),$$

where $\mathsf{fUnforge}_{\mathcal{A},\mathsf{FAS}}$ *is defined in Figure 3.8.*

Witness extractability. Here we require that for any function $f \in \mathcal{F}$, for any honestly generated pre-signature w.r.t. f and a valid signature, one should be able to extract the function evaluation f on a witness. Intuitively, it tries to capture the interaction between an honest buyer (i.e., generates valid pre-signature) and a malicious seller trying to get paid (i.e., adapt pre-signature into a valid signature) without revealing function evaluation f on a witness (i.e., extraction failure) should be unlikely to occur. This is similar to the witness extractability security property of adaptor signatures (See Definition 3.4.7) except that in the formal witness extractability experiment Figure 3.9, in step 8, computing the set Z is inefficient.

Definition 3.5.9 (Witness Extractability)**.** *A functional adaptor signature scheme* FAS *satisfies witness extractability if for every p.p.t. adversary* $\mathcal{A}$*, there exists a negligible function* ϵ *such that for all* $\lambda \in \mathbb{N}$*:*

$$\Pr\left[\mathsf{fWitExt}_{\mathcal{A},\mathsf{FAS}}(1^\lambda) = 1\right] \leq \epsilon(\lambda),$$

where $\mathsf{fWitExt}_{\mathcal{A},\mathsf{FAS}}$ *is defined in Figure 3.9.*

Experiment $\mathsf{fWitExt}_{\mathcal{A},\mathsf{FAS}}(1^\lambda)$

$Q := \emptyset, \mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$
$(\mathsf{sk}, \mathsf{vk}) \leftarrow \mathsf{AS.KGen}(1^\lambda)$
$(\mathsf{ad}^*, m^*, Y^*, f^*, \widetilde{\sigma}_1^*) \leftarrow \mathcal{A}^{\mathcal{O}_s, \mathcal{O}_{fps_2}}(\mathsf{pp}, \mathsf{vk})$
if $\big(\mathsf{AdVerify}(\mathsf{pp}, \mathsf{ad}^*, Y^*) = 0 \vee (f^* \notin \mathcal{F})$
  $\vee\ \mathsf{FPreVerify}_1(\mathsf{pp}, \mathsf{ad}^*, Y^*, f^*, \widetilde{\sigma}_1^*) = 0\big)$:
  return $\perp$
$\widetilde{\sigma}_2^* \leftarrow \mathsf{FPreSign}_2(\mathsf{pp}, \mathsf{ad}^*, \mathsf{sk}, m^*, Y^*, f^*, \widetilde{\sigma}_1^*)$
$\sigma^* \leftarrow \mathcal{A}^{\mathcal{O}_s, \mathcal{O}_{fps_2}}(\widetilde{\sigma}_2^*)$
$z := \mathsf{FExt}(\mathsf{ad}^*, \mathsf{vk}, Y^*, f^*, (\widetilde{\sigma}_1^*, \widetilde{\sigma}_2^*), \sigma^*)$
$Z = \{f^*(y) : \exists\ y\ s.t.\ (Y^*, y) \in R\}$
return $(m^* \notin Q) \wedge \mathsf{Vrfy}(\mathsf{vk}, m^*, \sigma^*) = 1$
  $\wedge\ (z \notin Z)$

Oracle $\mathcal{O}_s(m)$

$\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$
$Q := Q \cup \{m\}$
return σ

Oracle $\mathcal{O}_{fps_2}(\mathsf{ad}, m, Y, f, \widetilde{\sigma}_1)$

if $(\mathsf{FPreVerify}_1(\mathsf{pp}, \mathsf{ad}, Y, f, \widetilde{\sigma}_1) = 0)$:
  return $\perp$
$\widetilde{\sigma}_2 \leftarrow \mathsf{FPreSign}_2(\mathsf{pp}, \mathsf{ad}, \mathsf{sk}, m, Y, f, \widetilde{\sigma}_1)$
$Q := Q \cup \{m\}$
return $\widetilde{\sigma}_2$

Figure 3.9: Witness Extractability experiment for Functional Adaptor Signature Schemes

Pre-signature adaptability. This property says that given a valid pre-signature and a witness for the instance, one can always adapt the pre-signature into a valid signature. Intuitively, this tries to capture that it should be impossible for a malicious buyer to learn a function evaluation on a witness without making the payment. This is similar to the pre-signature adaptability security property of adaptor signatures (See Definition 3.4.8).

Definition 3.5.10 (Pre-Signature Adaptability)**.** *A functional adaptor signature scheme* FAS *satisfies pre-signature adaptability if for every* $\lambda \in \mathbb{N}$*, and any* $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$*, any statement/witness pair* $(Y, y) \in \mathsf{R}$*,*

any $(\mathsf{ad}, \mathsf{st}) \leftarrow \mathsf{AdGen}(\mathsf{pp}, \mathsf{ad}, Y)$, *any message* $m \in \{0,1\}^*$, *any family of functions* $\mathcal{F}$, *any function* $f \in \mathcal{F}$, *any* $\widetilde{\sigma}_1 \leftarrow \mathsf{FPreSign}_1(\mathsf{pp}, \mathsf{ad}, Y, f, \widetilde{\sigma}_1)$, *any key pair* $(\mathsf{sk}, \mathsf{vk}) \leftarrow \mathsf{KGen}(1^\lambda)$, *and any pre-signature* $\widetilde{\sigma}_2$ *with* $\mathsf{FPreVerify}_2(\mathsf{pp}, \mathsf{ad}, \mathsf{vk}, m, Y, f, \widetilde{\sigma} = (\widetilde{\sigma}_1, \widetilde{\sigma}_2)) = 1$, *we have that:*

$$\Pr[\mathsf{Vrfy}(\mathsf{vk}, m, \mathsf{Adapt}(\mathsf{ad}, \mathsf{st}, \mathsf{vk}, m, Y, y, f, \widetilde{\sigma}))] = 1.$$

Witness privacy. Here we want that in an interaction between a malicious buyer wanting to learn function evaluation f on the witness and an honest seller using witness y to adapt the pre-signature into a valid signature, at the end the buyer should not be able to extract any information about y beyond $f(y)$. There are different ways to capture this formally. We consider various notions of witness privacy akin to those in functional encryption literature and zero-knowledge proofs literature as follows.

- **Zero-Knowledge:** for every $(Y, y) \in R$, there exists a p.p.t. simulator Sim such that the adversary should not be able to guess whether it is interacting with a real-world challenger that is allowed to use the witness y or the simulator Sim that is only allowed access to $f(y)$.
- **Witness Indistinguishability:** given two valid witnesses y_0 and y_1 for a statement $Y \in L_R$, the adversary should not be able to guess which witness was used to adapt a pre-signature as long as $f(y_0) = f(y_1)$. This notion is meaningful only for languages that have at least two witnesses.
- **Witness Hiding:** for every $Y \in L_R$, given valid pre-signature w.r.t. function f and valid signature adapted from it, the adversary should not be able to guess a witness y s.t. $(Y, y) \in R$.

Zero-Knowledge is the strongest among all in the sense that it implies the other two. All three are meaningful for non-unique witness languages. But for unique witness languages, witness indistinguishability is not meaningful. Further, witness-indistinguishability and witness-hiding are incomparable. Next, we describe all three formally.

Definition 3.5.11 (Witness Indistinguishability)**.** *A functional adaptor signature scheme* FAS *is* faWI*-secure or simply witness indistinguishable, if for every stateful p.p.t. adversary* $\mathcal{A}$*, there exists a negligible function* negl *such that for all* $\lambda \in \mathbb{N}$*,*

$$|\Pr[\mathsf{faWI}^0_{\mathcal{A},\mathsf{FAS}}(1^\lambda) = 1] - \Pr[\mathsf{faWI}^1_{\mathcal{A},\mathsf{FAS}}(1^\lambda) = 1]| \leq \mathsf{negl}(\lambda),$$

where the experiments $\mathsf{faWI}^b_{\mathcal{A},\mathsf{FAS}}$ *for* $b \in \{0,1\}$ *are defined as in Figure 3.10.*

Experiment $\mathsf{faWI}^{b}_{\mathcal{A},\mathsf{FAS}}$

1 : $\mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda)$
2 : $(\mathsf{vk}, Y, y_0, y_1) \leftarrow \mathcal{A}(\mathsf{crs})$
3 : If $(((Y, y_0) \notin R) \vee ((Y, y_1) \notin R))$: **ret** 0
4 : $(\mathsf{ad}, \mathsf{st}) \leftarrow \mathsf{AdGen}(\mathsf{crs}, Y, y_b)$
5 : $(m, f, \widetilde{\sigma} = (\widetilde{\sigma}_1, \widetilde{\sigma}_2)) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{FPreSign}_1}(\cdot)}(\mathsf{ad})$
6 : If $\mathsf{FPreVerify}_2(\mathsf{ad}, \mathsf{vk}, m, Y, f, \widetilde{\sigma} = (\widetilde{\sigma}_1, \widetilde{\sigma}_2)) = 0$: **ret** 0
7 : If $f(y_0) \neq f(y_1)$: **ret** 0
8 : $\sigma := \mathsf{Adapt}(\mathsf{ad}, \mathsf{st}, \mathsf{vk}, m, Y, y, f, \widetilde{\sigma} = (\widetilde{\sigma}_1, \widetilde{\sigma}_2))$
9 : $b' \leftarrow \mathcal{A}(\sigma)$
10 : **ret** b'

Oracle $\mathcal{O}_{\mathsf{FPreSign}_1}(f)$

1 : **ret** $\mathsf{FPreSign}_1(\mathsf{pp}, \mathsf{ad}, \mathsf{st}, Y, y, f)$

Figure 3.10: Witness Indistinguishability experiment of FAS

Definition 3.5.12 (Witness Hiding). *A functional adaptor signature scheme* FAS *is* faWH*-secure or simply witness hiding, if for every stateful p.p.t. adversary* $\mathcal{A}$*, there exists a negligible function* negl *such that for all* $\lambda \in \mathbb{N}$*,*

$$\Pr[\mathsf{faWH}_{\mathcal{A},\mathsf{FAS}}(1^\lambda) = 1] \leq \mathsf{negl}(\lambda),$$

where the experiment $\mathsf{faWH}_{\mathcal{A},\mathsf{FAS}}$ *is defined as in Figure 3.11.*

Experiment $\mathsf{faWH}_{\mathcal{A},\mathsf{FAS}}$

1 : $\mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda)$, $(Y, y) \leftarrow \mathsf{GenR}(1^\lambda)$, $(\mathsf{ad}, \mathsf{st}) \leftarrow \mathsf{AdGen}(\mathsf{crs}, Y, y)$
2 : $(\mathsf{vk}, m, f, \widetilde{\sigma} = (\widetilde{\sigma}_1, \widetilde{\sigma}_2)) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{FPreSign}_1}(\cdot)}(\mathsf{crs}, \mathsf{ad}, Y)$
3 : If $\mathsf{FPreVerify}_2(\mathsf{ad}, \mathsf{vk}, m, Y, f, \widetilde{\sigma} = (\widetilde{\sigma}_1, \widetilde{\sigma}_2)) = 0$: **ret** 0
4 : $\sigma := \mathsf{Adapt}(\mathsf{ad}, \mathsf{st}, \mathsf{vk}, m, Y, y, f, \widetilde{\sigma} = (\widetilde{\sigma}_1, \widetilde{\sigma}_2))$
5 : $y' \leftarrow \mathcal{A}(\sigma)$
6 : **ret** 1 iff $(Y, y') \in R$

Oracle $\mathcal{O}_{\mathsf{FPreSign}_1}(f)$

1 : **ret** $\mathsf{FPreSign}_1(\mathsf{pp}, \mathsf{ad}, \mathsf{st}, Y, y, f)$

Figure 3.11: Witness Hiding experiment of FAS

Next, we define zero-knowledge of FAS. This property is defined with respect to a malicious buyer and is defined using a simulator that simulates view for the buyer using only the information about the witness y that the malicious buyer gets to learn anyway. We consider two cases here: (i) malicious buyer does not abort. In this case, the buyer must learn only $f(y)$ about y and the simulator will have access to this information only. (ii) the malicious buyer aborts the protocol before providing $\widetilde{\sigma}_2$ to the honest seller. In this case, the buyer must learn nothing about y and the simulator will also have no information about y. We define the two cases separately below.

Experiment $\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)$

$\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$
$(\mathsf{ad}, \mathsf{st}) \leftarrow \mathsf{AdGen}(\mathsf{pp}, Y, y)$
$(m, f, \mathsf{vk}) \leftarrow \mathcal{A}(\mathsf{pp}, \mathsf{ad}, Y)$
while $\mathcal{A}$ is not done making queries:
  $\widetilde{\sigma}_1 \leftarrow \mathsf{FPreSign}_1(\mathsf{pp}, \mathsf{ad}, \mathsf{st}, Y, y, f)$
  $\widetilde{\sigma}_2 \leftarrow \mathcal{A}(\widetilde{\sigma}_1)$
  if $(\mathsf{FPreVerify}_2(\mathsf{pp}, \mathsf{ad}, \mathsf{vk}, m, Y, f, (\widetilde{\sigma}_1, \widetilde{\sigma}_2)) = 0)$:
    return $\perp$
  $\sigma := \mathsf{Adapt}(\mathsf{ad}, \mathsf{st}, \mathsf{vk}, m, Y, y, f, (\widetilde{\sigma}_1, \widetilde{\sigma}_2))$
  $(m, f) \leftarrow \mathcal{A}(\sigma)$
return view of $\mathcal{A}$

Experiment $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)$

$\mathsf{pp} \leftarrow \mathsf{Setup}^*(1^\lambda)$
$(\mathsf{ad}, \mathsf{st}) \leftarrow \mathsf{AdGen}^*(\mathsf{pp}, Y)$
$(m, f, \mathsf{vk}) \leftarrow \mathcal{A}(\mathsf{pp}, \mathsf{ad}, Y)$
while $\mathcal{A}$ is not done making queries:
  $\widetilde{\sigma}_1 \leftarrow \mathsf{FPreSign}^*_1(\mathsf{pp}, \mathsf{ad}, \mathsf{st}, Y, f(y), f)$
  $\widetilde{\sigma}_2 \leftarrow \mathcal{A}(\widetilde{\sigma}_1)$
  if $(\mathsf{FPreVerify}_2(\mathsf{pp}, \mathsf{ad}, \mathsf{vk}, m, Y, f, (\widetilde{\sigma}_1, \widetilde{\sigma}_2)) = 0)$:
    return $\perp$
  $\sigma := \mathsf{Adapt}^*(\mathsf{ad}, \mathsf{st}, \mathsf{vk}, m, Y, f(y), f, (\widetilde{\sigma}_1, \widetilde{\sigma}_2))$
  $(m, f) \leftarrow \mathcal{A}(\sigma)$
return view of $\mathcal{A}$

Figure 3.12: Zero Knowledge (non-abort) experiments for FAS

Definition 3.5.13 (Zero Knowledge (non-abort))**.** *A* FAS *scheme satisfies zero knowledge (non-abort) if there exists a negligible function ϵ such that for every stateful p.p.t. malicious buyer $\mathcal{A}$, for all $\lambda \in \mathbb{N}$, for all $(Y, y) \in \mathsf{R}$: there exists a stateful p.p.t. simulator $\mathsf{Sim} = (\mathsf{Setup}^*, \mathsf{AdGen}^*, \mathsf{FPreSign}^*_1, \mathsf{Adapt}^*)$ such that the following holds for any p.p.t. distinguisher $\mathcal{D}$,*

$$\left| \Pr\left[\mathcal{D}\left(\mathsf{V}_{\mathcal{A}}\right) = 1 : \mathsf{V}_{\mathcal{A}} \leftarrow \mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right] \right.$$
$$\left. - \Pr\left[\mathcal{D}\left(\mathsf{V}_{\mathcal{A}}\right) = 1 : \mathsf{V}_{\mathcal{A}} \leftarrow \mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right] \right|$$
$$\leq \epsilon(\lambda)$$

where $\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}$ *and* $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$ *are defined as in Figure 3.12 and* $\mathsf{V}_{\mathcal{A}}$ *is defined as the view of* $\mathcal{A}$ *in the two games respectively.*

Experiment $\mathsf{fZKReal}^{\mathsf{a}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)$

$\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$
$(\mathsf{ad}, \mathsf{st}) \leftarrow \mathsf{AdGen}(\mathsf{pp}, Y, y)$
$(m, f, \mathsf{vk}) \leftarrow \mathcal{A}(\mathsf{pp}, \mathsf{ad}, Y)$
while $\mathcal{A}$ is not done making queries:
  $\widetilde{\sigma}_1 \leftarrow \mathsf{FPreSign}_1(\mathsf{pp}, \mathsf{ad}, \mathsf{st}, Y, y, f)$
  $(m, f) \leftarrow \mathcal{A}(\widetilde{\sigma}_1)$
return view of $\mathcal{A}$

Experiment $\mathsf{fZKIdeal}^{\mathsf{a},\mathsf{Sim}^{\mathsf{a}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)$

$\mathsf{pp} \leftarrow \mathsf{Setup}^{*,\mathsf{a}}(1^\lambda)$
$(\mathsf{ad}, \mathsf{st}) \leftarrow \mathsf{AdGen}^{*,\mathsf{a}}(\mathsf{pp}, Y)$
$(m, f, \mathsf{vk}) \leftarrow \mathcal{A}(\mathsf{pp}, \mathsf{ad}, Y)$
while $\mathcal{A}$ is not done making queries:
  $\widetilde{\sigma}_1 \leftarrow \mathsf{FPreSign}_1^{*,\mathsf{a}}(\mathsf{pp}, \mathsf{ad}, \mathsf{st}, Y, f)$
  $(m, f) \leftarrow \mathcal{A}(\widetilde{\sigma}_1)$
return view of $\mathcal{A}$

Figure 3.13: Zero Knowledge (abort) experiments for FAS

Definition 3.5.14 (Zero Knowledge (abort))**.** *A* FAS *scheme satisfies zero knowledge (abort) if there exists a negligible function ϵ such that for every stateful p.p.t. malicious buyer $\mathcal{A}$, for all $\lambda \in \mathbb{N}$, for all $(Y, y) \in \mathsf{R}$: there exists a stateful p.p.t. simulator $\mathsf{Sim}^{\mathsf{a}} = (\mathsf{Setup}^{*,\mathsf{a}}, \mathsf{AdGen}^{*,\mathsf{a}}, \mathsf{FPreSign}_1^{*,\mathsf{a}})$ such that the following holds for any p.p.t. distinguisher $\mathcal{D}$,*

$$\left| \Pr\left[\mathcal{D}\left(\mathsf{V}_{\mathcal{A}}\right) = 1 : \mathsf{V}_{\mathcal{A}} \leftarrow \mathsf{fZKReal}^{\mathsf{a}}{}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right] - \Pr\left[\mathcal{D}\left(\mathsf{V}_{\mathcal{A}}\right) = 1 : \mathsf{V}_{\mathcal{A}} \leftarrow \mathsf{fZKIdeal}^{\mathsf{a},\mathsf{Sim}^{\mathsf{a}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right] \right| \leq \epsilon(\lambda)$$

where $\mathsf{fZKReal}^{\mathsf{a}}_{\mathcal{A},\mathsf{FAS}}$ *(defined as in Figure 3.13) is real-world game where the adversary $\mathcal{A}$ aborts after receiving $\widetilde{\sigma}_1$ and* $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$ *(defined as in Figure 3.13) is ideal-world game where the simulator simulates the adversary $\mathcal{A}$'s view upto receiving $\widetilde{\sigma}_1$ and $\mathsf{V}_{\mathcal{A}}$ is defined as the view of $\mathcal{A}$ in the two games respectively.*

Remark 3.5.15. *Crucially, notice that* $\mathsf{FPreSign}_1^*$ *in* fZKIdeal *game gets $f(y)$ as input, whereas* $\mathsf{FPreSign}_1^{*,\mathsf{a}}$ *in* $\mathsf{fZKIdeal}^{\mathsf{a}}$ *game does not.*

Organization. From Section 3.6 to Section 3.11, we present the FE-based FAS construction and its security proof. This is based on [VST24a]. From Section 3.12 to Section 3.17, we present the HE-based FAS construction and its security proof. This is based on [VGTS26].

3.6 Strongly-Secure FAS Construction from FE

In this section, we build a generic construction of FAS w.r.t. digital signature scheme DS, any NP relation R with statement/witness pairs $(X, \mathbf{x}) \in R$ such that $\mathbf{x} \in \mathcal{M}$ for some set $\mathcal{M} \subseteq \mathbb{Z}^\ell$, and function family $\mathcal{F}_{\mathsf{IP},\ell}$ for computing inner products of vectors of length ℓ. Our generic construction of FAS is presented in Figure 3.2 [15] and it uses the following building blocks:

- A selective, IND-secure IPFE satisfying R_{IPFE}-compliance and R'_{IPFE}-robustness (Definitions 3.4.10, 3.4.12 and 3.4.13) w.r.t. hard relations R_{IPFE} and R'_{IPFE} such that $R_{\mathsf{IPFE}} \subseteq R'_{\mathsf{IPFE}}$. The message space of IPFE is $\mathcal{M}' \subseteq \mathbb{Z}^{\ell+1}$ and the function family is $\mathcal{F}_{\mathsf{IP},\ell+1}$.

- An adaptor signature scheme AS w.r.t. a digital signature scheme DS, and hard relations R_{IPFE} and R'_{IPFE} that satisfies witness extractability (Definition 3.4.7) and weak pre-signature adaptability (Definition 3.4.8) security properties.

- A non-interactive zero-knowledge argument system NIZK in the common reference string model for the NP language L_{NIZK}:

$$L_{\mathsf{NIZK}} := \left\{ \begin{array}{c} (X, \mathsf{pp}', \mathsf{mpk}, \mathsf{ct}) : \\ \exists (r_0, r_1, \mathbf{x}) \text{ such that } \mathsf{pp}' \in [\mathsf{IPFE.Gen}(1^\lambda)], \\ (\mathsf{mpk}, \mathsf{msk}) = \mathsf{IPFE.Setup}(\mathsf{pp}', 1^{\ell+1}; r_0), \\ (X, \mathbf{x}) \in R, \mathsf{ct} = \mathsf{IPFE.Enc}(\mathsf{mpk}, (\mathbf{x}^T, 0)^T; r_1) \end{array} \right\}.$$

We sketch the security proof of unforgeability and zero-knowledge witness privacy below. Correctness and formal proofs are deferred to Section 3.10.

Lemma 3.6.1. *Suppose NIZK satisfies correctness,* AS *satisfies correctness, and* IPFE *satisfies* R_{IPFE}*-compliance and* R'_{IPFE}*-robustness. Then, the* FAS *construction in Figure 3.2 satisfies correctness.*

Theorem 3.6.2. *Let* $\mathcal{F}_{\mathsf{IP},\ell}$ *be the family of inner products functions of vectors of length* ℓ*. Let* R *be any NP relation with statement/witness pairs* $(X, \mathbf{x})$ *such that* $\mathbf{x} \in \mathcal{M}$ *for some set* $\mathcal{M} \subseteq \mathbb{Z}^\ell$*. Suppose that*

- $\mathcal{M}$ *is an additive group,* R *is* $\mathcal{F}_{\mathsf{IP},\ell}$*-hard (Definition 3.4.2),*

- NIZK *is a secure NIZK argument system (Definition 3.4.26),*

- AS *is an adaptor signature scheme w.r.t. digital signature scheme* DS *and hard relations* $R_{\mathsf{IPFE}}, R'_{\mathsf{IPFE}}$ *satisfying weak pre-signature adaptability (Definition 3.4.8), witness extractability (Definition 3.4.7),*

[15] Figure 3.2 is presented with $\mathcal{M} = \mathbb{Z}_p^\ell$, but the scheme generalizes to $\mathcal{M} \subseteq \mathbb{Z}^\ell$ as well. Further, $(\mathsf{aux}_{\mathbf{y}}, \pi_{\mathbf{y}})$ denote $\tilde{\sigma}_1$, and $\tilde{\sigma}$ simply denotes $\tilde{\sigma}_2$ in the figure for brevity.

Games G_0, G_1, G_2

1 : $\mathcal{Q} := \emptyset$, $\mathsf{crs} \leftarrow \mathsf{NIZK.Setup}(1^\lambda)$, $\mathsf{pp}' \leftarrow \mathsf{IPFE.Gen}(1^\lambda)$
2 : $\mathsf{pp} := (\mathsf{crs}, \mathsf{pp}')$, $(\mathsf{sk}, \mathsf{vk}) \leftarrow \mathsf{KGen}(\mathsf{pp}')$, $(X^*, \mathbf{x}^*) \leftarrow \mathsf{GenR}(1^\lambda)$
3 : $(\mathsf{ad}, m^*, \mathbf{y}^*, \mathsf{aux}^*_{\mathbf{y}}, \pi^*_{\mathbf{y}}) \leftarrow \mathcal{A}^{\mathcal{O}_S(\cdot)}(\mathsf{pp}, \mathsf{vk}, X^*)$
4 : Parse $\mathsf{ad} = (\mathsf{mpk}, \mathsf{ct}, \pi)$, let $\mathsf{stmt} := (X^*, \mathsf{pp}', \mathsf{mpk}, \mathsf{ct})$
5 : $\widetilde{\mathbf{y}^*} := (\mathbf{y}^{*T}, \pi^*_{\mathbf{y}})^T$, $\mathsf{pk}_{\mathbf{y}^*} := \mathsf{IPFE.PubKGen}(\mathsf{mpk}, \widetilde{\mathbf{y}^*})$
6 : $\mathsf{Bad}_1 := \mathsf{false}, \mathsf{Bad}_2 := \mathsf{false}$
7 : If $\mathsf{NIZK.Vf}(\mathsf{crs}, \mathsf{stmt}, \pi) = 0 \ \vee \ \mathbf{y}^* \notin \mathcal{F}_{\mathsf{IP},\ell} \ \vee \ \mathsf{pk}_{\mathbf{y}^*} \neq \mathsf{aux}^*_{\mathbf{y}}$:
  ret 0
8 : If $\mathsf{stmt} \notin L_{NIZK}$: $\mathsf{Bad}_1 := \mathsf{true}$, **ret** 0
9 : $\widetilde{\sigma}^* \leftarrow \mathsf{AS.PreSign}(\mathsf{sk}, m^*, \mathsf{aux}^*_{\mathbf{y}})$, $\sigma^* \leftarrow \mathcal{A}^{\mathcal{O}_S(\cdot), \mathcal{O}_{\mathsf{fpS}}(\cdot,\cdot,\cdot,\cdot,\cdot)}(\widetilde{\sigma}^*)$
10 : $z = \mathsf{AS.Ext}(\widetilde{\sigma}^*, \sigma^*, \mathsf{aux}^*_{\mathbf{y}})$
11 : If $(m^* \notin \mathcal{Q}) \wedge \mathsf{Vf}(\mathsf{vk}, m^*, \sigma^*) \wedge ((\mathsf{aux}^*_{\mathbf{y}}, z) \notin R'_{\mathsf{IPFE}})$:
12 : $\mathsf{Bad}_2 := \mathsf{true}$, **ret** 0
13 : **ret** $((m^* \notin \mathcal{Q}) \wedge \mathsf{Vf}(\mathsf{vk}, m^*, \sigma^*))$

Oracle $\mathcal{O}_S(m)$

1 : $\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$, $\mathcal{Q} := \mathcal{Q} \vee \{m\}$, **ret** σ

Oracle $\mathcal{O}_{\mathsf{fpS}}(m, X, \mathbf{y}, \mathsf{aux}_{\mathbf{y}}, \pi_{\mathbf{y}})$

1 : If $\mathsf{FPreVerify}_1(\mathsf{ad}, \mathbf{y}, \mathsf{aux}_{\mathbf{y}}, \pi_{\mathbf{y}}) = 0$: **ret** $\perp$
2 : $\widetilde{\sigma} \leftarrow \mathsf{FPreSign}_2(\mathsf{ad}, \mathsf{sk}, m, X, \mathbf{y}, \mathsf{aux}_{\mathbf{y}})$, $\mathcal{Q} := \mathcal{Q} \vee \{m\}$, **ret** $\widetilde{\sigma}$

Figure 3.14: Unforgeability Proof: Games G_0, G_1, G_2

- IPFE *is a selective, IND-secure IPFE scheme (Definition 3.4.10) for function family* $\mathcal{F}_{\mathsf{IP},\ell+1}$ *satisfying* R_{IPFE}*-compliance (Definition 3.4.12),* R'_{IPFE}*-robustness (Definition 3.4.13).*

Then, the construction in Figure 3.2 is a strongly-secure (Definition 3.5.3) functional adaptor signature scheme w.r.t. digital signature scheme DS*, NP relation* R*, and family of inner product functions* $\mathcal{F}_{\mathsf{IP},\ell}$*.*

Proof sketch of Unforgeability. A natural idea for proving unforgeability of FAS would be to somehow reduce it to unforgeability of AS. But, this requires non-blackbox use of the underlying AS, and thus results in a complex proof. We avoid that altogether and present a counter-intuitive yet elegant way of proving unforgeability of FAS by reducing it to witness extractability of AS. Our proof technique does not need to open up the blackbox of AS as evident in the sequence of games below.

In a little more detail, to show that $\Pr[G_0(1^\lambda) = 1] \leq \mathsf{negl}(\lambda)$, we consider a sequence of games G_0, G_1, G_2 as decribed in Figure 3.14.

- Game G_0 is the original game $\mathsf{faEUF\text{-}CMA}_{\mathcal{A},\mathsf{FAS}}$, where the adversary $\mathcal{A}$ has to come up with a valid forgery on a message m^* of his choice, while having access to functional pre-sign oracle $\mathcal{O}_{\mathsf{fpS}}$ and sign oracle $\mathcal{O}_S$. Here, variables $\mathsf{Bad}_1, \mathsf{Bad}_2$ do not affect the game and are used only to aide the analysis below.

- Game G_1 is same as G_0, except that before computing the pre-signature, the game checks if the NIZK statement $(X^*, \mathsf{pp}', \mathsf{mpk}, \mathsf{ct})$ is in the language L_{NIZK}. If no, the game sets the flag $\mathsf{Bad}_1 = \mathsf{true}$ and returns 0. Observe that G_0 and G_1 are identical until G_1 checks the condition for setting Bad_1. Hence,

$$|\Pr[G_0(1^\lambda) = 1] - \Pr[G_1(1^\lambda) = 1]| \leq \Pr[\mathsf{Bad}_1 \text{ in } G_1].$$

  Thus, for proving $|\Pr[G_0(1^\lambda) = 1] - \Pr[G_1(1^\lambda) = 1]| \leq \mathsf{negl}(\lambda)$, it suffices to show that in game G_1, $\Pr[\mathsf{Bad}_1] \leq \mathsf{negl}(\lambda)$. $\mathsf{Bad}_1 = \mathsf{true}$ implies $\mathsf{stmt} \notin L_{NIZK}$ and $\mathsf{NIZK.Vf}(\mathsf{crs}, \mathsf{stmt}, \pi) = 1$. Thus, the probability bound follows from the adaptive soundness of NIZK argument system.

- Game G_2 is same as G_1, except that when $\mathcal{A}$ outputs the forgery σ^*, the game extracts the witness z of the underlying adaptor signature scheme for the statement $\mathsf{pk}_{\mathbf{y}^*}$ and checks if the NIZK statement $(\mathsf{aux}^*_{\mathbf{y}}, z)$ satisfies R'_{IPFE}. If no, the game sets the flag $\mathsf{Bad}_2 = \mathsf{true}$ and returns 0. Observe that G_0 and G_1 are identical until G_1 checks the condition for setting Bad_1. Hence,

$$|\Pr[G_1(1^\lambda) = 1] - \Pr[G_2(1^\lambda) = 1]| \leq \Pr[\mathsf{Bad}_2 \text{ in } G_2].$$

  Thus, for proving $|\Pr[G_1(1^\lambda) = 1] - \Pr[G_2(1^\lambda) = 1]| \leq \mathsf{negl}(\lambda)$, it suffices to show that in game G_2, $\Pr[\mathsf{Bad}_2] \leq \mathsf{negl}(\lambda)$. This follows from the witness extractability of the underlying adaptor signature scheme AS. In a little more detail, we can show that if an adversary $\mathcal{A}$ successfully causes G_2 to set Bad_2, then, we can use it to come up with a reduction $\mathcal{B}$ that breaks the witness extractability of the underlying adaptor signature scheme. This is because $\mathsf{Bad}_2 = \mathsf{true}$ implies $(m^* \notin \mathcal{Q}) \wedge ((\mathsf{aux}^*_{\mathbf{y}}, z) \notin R'_{\mathsf{IPFE}}) \wedge \mathsf{Vf}(\mathsf{vk}, m^*, \sigma^*)$, i.e., the winning condition for $\mathcal{B}$.

- Lastly, we can show that $\Pr[G_2(1^\lambda) = 1] \leq \mathsf{negl}(\lambda)$ assuming $\mathcal{F}_{\mathsf{IP},\ell}$-hardness of relation R and R'_{IPFE}-robustness of the IPFE scheme. Essentially, we show that if $\mathcal{A}$ wins G_2, then, we can build a reduction $\mathcal{B}$ that breaks the $\mathcal{F}_{\mathsf{IP},\ell}$-hardness of relation R. $\mathcal{B}$ outputs $(\mathbf{y}^*, v)$, where $v = \mathsf{IPFE.Dec}(z, \mathsf{ct})$. To win, $\mathcal{B}$'s output must satisfy $(\mathbf{y}^* \in \mathcal{F}_{\mathsf{IP},\ell}) \wedge (v \in \{f_{\mathbf{y}^*}(\mathbf{x}) : \exists \mathbf{x}\ s.t.\ (X, \mathbf{x}) \in R\})$. If $\mathcal{A}$ wins, then $\mathbf{y}^* \in \mathcal{F}_{\mathsf{IP},\ell}$ and $\mathsf{pk}_{\mathbf{y}^*} = \mathsf{aux}^*_{\mathbf{y}}$. Next, $\mathsf{Bad}_1 = \mathsf{false}$ implies that $\mathsf{stmt} \in L_{NIZK}$, where $\mathsf{stmt} = (X^*, \mathsf{pp}', \mathsf{mpk}, \mathsf{ct})$. Hence, it follows that ct encrypts some vector $\widetilde{\mathbf{x}} = (\mathbf{x}^T, 0)^T \in \mathcal{M}' \subseteq \mathbb{Z}^{\ell+1}$ under mpk such that $(X^*, \mathbf{x}) \in R$. Next, $\mathsf{Bad}_2 = \mathsf{false}$ implies that $(\mathsf{aux}^*_{\mathbf{y}}, z) \in R'_{\mathsf{IPFE}}$. As $\mathsf{pk}_{\mathbf{y}^*} = \mathsf{aux}^*_{\mathbf{y}}$ and IPFE satisfies R'_{IPFE}-robustness, it follows that $v = f_{\widetilde{\mathbf{y}^*}}(\widetilde{\mathbf{x}})$. As $\widetilde{\mathbf{y}^*} = (\mathbf{y}^{*T}, \pi^*_{\mathbf{y}})^T$ and $\widetilde{\mathbf{x}} = (\mathbf{x}^T, 0)^T$, we get that $f_{\widetilde{\mathbf{y}^*}}(\widetilde{\mathbf{x}}) = f_{\mathbf{y}^*}(\mathbf{x})$. Hence, we conclude that $v \in \{f_{\mathbf{y}^*}(\mathbf{x}) : \exists \mathbf{x}\ s.t.\ (X, \mathbf{x}) \in R\}$. Thus, $\mathcal{B}$ wins its game. This completes the proof.

Proof sketch of Zero-Knowledge. We first describe the stateful simulator $\mathsf{Sim} = (\mathsf{Setup}^*, \mathsf{AdGen}^*, \mathsf{FPreSign_1}^*, \mathsf{Adapt}^*)$. Let the NIZK simulator be $\mathsf{NIZK.Sim} = (\mathsf{NIZK.Setup}^*, \mathsf{NIZK.Prove}^*)$. Then, the simulator Sim is as follows.

- $\mathsf{Setup}^*(1^\lambda)$: same as Setup except $(\mathsf{crs}, \mathsf{td}) \leftarrow \mathsf{NIZK.Setup}^*(1^\lambda)$ and trapdoor td is stored as internal state by Sim.
- $\mathsf{AdGen}^*(\mathsf{pp}, X)$: same as AdGen except $\widetilde{\mathbf{x}} := (-\mathbf{t}^T, 1)^T \in \mathbb{Z}_p^{\ell+1}$ and $\pi \leftarrow \mathsf{NIZK.Prove}^*(\mathsf{crs}, \mathsf{td}, (X, \mathsf{pp}', \mathsf{mpk}, \mathsf{ct}))$.
- $\mathsf{FPreSign_1}^*(\mathsf{ad}, \mathbf{y}, f_{\mathbf{y}}(\mathbf{x}))$: same as $\mathsf{FPreSign_1}$ except $\mathsf{pk_y}$ is computed for $\widetilde{\mathbf{y}} := (\mathbf{y}^T, f_{\mathbf{y}}(\mathbf{t}) + f_{\mathbf{y}}(\mathbf{x}))^T \in \mathcal{F}_{\mathsf{IP},\ell+1}$ and thus $\pi_{\mathbf{y}} := f_{\mathbf{y}}(\mathbf{t}) + f_{\mathbf{y}}(\mathbf{x})$ as well.
- $\mathsf{Adapt}^*(\mathsf{ad}, \mathsf{vk}, m, X, \mathbf{y}, \mathsf{aux_y}, \widetilde{\sigma}, f_{\mathbf{y}}(\mathbf{x}))$: same as Adapt except $\mathsf{sk_y}$ for $\widetilde{\mathbf{y}} := (\mathbf{y}^T, f_{\mathbf{y}}(\mathbf{t}) + f_{\mathbf{y}}(\mathbf{x}))^T$ is used.

To see that adversary's views in real and ideal world (Figure 3.12) are indistinguishable, consider a sequence of games G_0, G_1, G_2, G_3.

- Game G_0 corresponds to the real-world experiment where Setup, AdGen, $\mathsf{FPreSign_1}$, Adapt are used.
- Game G_1 is same as G_0, except the NIZK is switched to simulation mode, i.e., we change Setup to perform $(\mathsf{crs}, \mathsf{td}) \leftarrow \mathsf{NIZK.Setup}^*(1^\lambda)$ and change AdGen to perform $\pi \leftarrow \mathsf{NIZK.Prove}^*(\mathsf{crs}, \mathsf{td}, (X, \mathsf{pp}', \mathsf{mpk}, \mathsf{ct}))$. It follows from the zero-knowledge property of NIZK that $G_0 \approx_c G_1$.
- Game G_2 is same as G_1 except $\widetilde{\mathbf{y}}$ used in $\mathsf{FPreSign_1}$ and Adapt is switched from $\widetilde{\mathbf{y}} := (\mathbf{y}^T, f_{\mathbf{y}}(\mathbf{t}))^T$ to $\widetilde{\mathbf{y}} := (\mathbf{y}^T, f_{\mathbf{y}}(\mathbf{t}) + f_{\mathbf{y}}(\mathbf{x}))^T$. Since f is a linear function, it follows that $f_{\mathbf{y}}(\mathbf{t}) + f_{\mathbf{y}}(\mathbf{x}) = f_{\mathbf{y}}(\mathbf{t} + \mathbf{x})$, thus, one can view the transition to G_2 as a change of variables $\mathbf{t} \to \mathbf{t} + \mathbf{x}$. Since $\mathbf{t}$ is uniform random, it follows that G_1 and G_2 are identically distributed.
- Game G_3 is same as G_2 except that in AdGen, ct encrypts $\widetilde{\mathbf{x}} := (-\mathbf{t}^T, 1)^T$ instead of $\widetilde{\mathbf{x}} := (\mathbf{x}^T, 0)^T$. Observe that this does not change the inner product value $\widetilde{\mathbf{x}}^T\widetilde{\mathbf{y}}$. Thus, $G_2 \approx_c G_3$ follows from IND-security of the IPFE scheme.
- Lastly observe that adversary's view in G_3 is same as that in the ideal-world experiment. This complete the proof.

We emphasize that in G_3, even the underlying IPFE only uses the information $f_{\mathbf{y}}(\mathbf{x})$ about $\mathbf{x}$. Thus, going from G_1 to G_3 is essentially building an IND-security to simulation-security IPFE compiler. The random coins of this compiler are $\mathbf{t}$. Crucially, G_2 and G_3 explicitly use $f_{\mathbf{y}}(\mathbf{t})$, which is a leakage on these

random coins and for this reason the IPFE compiler is used in a non-blackbox way. We refer the reader back to Remark 3.2.1 on why $f_{\mathbf{y}}(\mathbf{t})$ is an acceptable leakage on the IPFE simulator's random coins $\mathbf{t}$.

3.7 FAS from Groups of Prime-Order

We provide instantiations of the FAS construction in Figure 3.2 from prime order groups. For this, it suffices to instantiate the building blocks IPFE and AS from prime order groups, while ensuring the two are compatible with each other. We describe these next. We can instantiate the NIZK for the language L_{NIZK} depending on the concrete relation R. That is, given R and the above instantiation of IPFE, we can pick the most efficient NIZK for L_{NIZK}. Since this is not the main contribution of this work, we will assume R to be a general NP relation and rely on the existence of NIZK for general NP [JJ21, PS19] and not delve into its details here.

More concretely, we instantiate the AS as Schnorr adaptor signature scheme [AEE+21] and thus, we have $R_{\mathsf{IPFE}} = R'_{\mathsf{IPFE}} = R_{\mathsf{DL}}$ as defined in Definition 3.4.3. Further, we show that the selective, IND-secure IPFE scheme by Abdalla et al. [ABDP15] satisfies R_{DL}-compliance and R_{DL}-robustness when appropriately augmented with a PubKGen algorithm.

3.7.1 Schnorr Adaptor Signature

First, we recall the Schnorr Signature Scheme Sch [Sch90]. Suppose $\mathsf{pp} := (\mathbb{G}, p, g) \leftarrow \mathsf{IPFE.Gen}(1^\lambda)$ as described in Figure 3.15, where $\mathbb{G}$ is a cyclic group of prime order p and g is a generator of $\mathbb{G}$. Suppose $H : \{0,1\}^* \to \mathbb{Z}_p$ is a hash function modeled as a random oracle. Then Sch is described below:

- $(\mathsf{vk}, \mathsf{sk}) \leftarrow \mathsf{Sch.KGen}(\mathsf{pp})$: choose $\delta \leftarrow \mathbb{Z}_p$ and set $\mathsf{sk} := \delta$ and $\mathsf{vk} := g^\delta$.
- $\sigma \leftarrow \mathsf{Sch.Sign}(\mathsf{sk}, m)$: sample a randomness $r \leftarrow \mathbb{Z}_p$ to compute $h := H(\mathsf{vk}||g^r||m), s := r + h\delta$ and output $\sigma := (h, s)$.
- $0/1 \leftarrow \mathsf{Sch.Vf}(\mathsf{vk}, m, \sigma)$: parse $\sigma := (h, s)$ and then compute $R := g^s/\mathsf{vk}^h$ and if $h = H(\mathsf{vk}||R||m)$ output 1, otherwise output 0.

Next, we describe the Adaptor Signature scheme AS with respect to the digital signature scheme Sch and hard relations $R = R' = R_{\mathsf{DL}}$, where R_{DL} is the discrete log relation. Let L_{DL} be the corresponding language. Then, AS is described below:

- $\widetilde{\sigma} \leftarrow \mathsf{AS.PreSign}(\mathsf{sk}, m, X \in L_{\mathsf{DL}})$: choose $r \xleftarrow{\$} \mathbb{Z}_p$ and compute $h := H(\mathsf{vk}||g^r \cdot X||m)$ and $\widetilde{s} := r + h\delta$. Set $\widetilde{\sigma} := (h, \widetilde{s})$.

$\mathsf{IPFE.Gen}(1^\lambda)$

1 : $\mathsf{pp} := (\mathbb{G}, p, g) \leftarrow \mathsf{GGen}(1^\lambda)$
2 : **ret** pp

$\mathsf{IPFE.Setup}(\mathsf{pp}, 1^\ell)$

1 : $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_p^\ell$, **ret** $\mathsf{msk} := \mathbf{s}, \mathsf{mpk} := g^{\mathbf{s}}$

$\mathsf{IPFE.Enc}(\mathsf{mpk}, \mathbf{x} \in \mathbb{Z}_p^\ell)$

1 : $r \xleftarrow{\$} \mathbb{Z}_p$, $\mathsf{ct}_0 := g^r$, $\mathsf{ct}_1 := g^{\mathbf{x}} \cdot \mathsf{mpk}^r$
2 : **ret** $\mathsf{ct} := (\mathsf{ct}_0, \mathsf{ct}_1)$

$\mathsf{IPFE.KGen}(\mathsf{msk}, \mathbf{y} \in \mathbb{Z}_p^\ell)$

1 : **ret** $\mathsf{sk}_\mathbf{y} := \mathbf{s}^T\mathbf{y} \in \mathbb{Z}_p$

$\mathsf{IPFE.PubKGen}(\mathsf{mpk}, \mathbf{y} \in \mathbb{Z}_p^\ell)$

1 : Parse $\mathsf{mpk} = (k_1, \ldots, k_\ell)$
2 : Parse $\mathbf{y} = (y_1, \ldots, y_\ell)$
3 : **ret** $\mathsf{pk}_\mathbf{y} := \prod_{i\in[\ell]} k_i^{y_i}$

$\mathsf{IPFE.Dec}(\mathsf{sk}_\mathbf{y}, \mathsf{ct})$

1 : Parse $\mathsf{ct}_1 = (c_1, \ldots, c_\ell)$
2 : Parse $\mathbf{y} = (y_1, \ldots, y_\ell)$
3 : $d := \prod_{i\in[\ell]} c_i^{y_i} / \mathsf{ct}_0^{\mathsf{sk}_\mathbf{y}}$
4 : **ret** $v := \mathsf{DLog}_g(d)$

Figure 3.15: Abdalla et al. [ABDP15] IPFE scheme augmented with PubKGen algorithm

- $0/1 \leftarrow \mathsf{AS.PreVerify}(\mathsf{vk}, m, X, \widetilde{\sigma})$: parse $\widetilde{\sigma} := (h, \widetilde{s})$ and compute $R := g^s / \mathsf{vk}^h$. If $h = H(\mathsf{vk}||R \cdot X||m)$ **ret** 1, else **ret** 0.
- $\sigma \leftarrow \mathsf{AS.Adapt}(\mathsf{vk}, m, X, x, \widetilde{\sigma})$: parse $\widetilde{\sigma} = (h, \widetilde{s})$ and compute $s := \widetilde{s} + x$. **ret** $\sigma := (h, s)$
- $x' \leftarrow \mathsf{AS.Ext}(\widetilde{\sigma}, \sigma, X)$: parse $\widetilde{\sigma} = (h, \widetilde{s})$ and $\sigma = (h, s)$. Compute $x' := s - \widetilde{s}$ and if $(X, x') \notin R_{\mathsf{DL}}$, **ret** $\perp$, else **ret** x'.

Lemma 3.7.1 ([AEE+21]). *The adaptor signature scheme in Section 3.7.1 satisfies witness extractability (Definition 3.4.7) and weak pre-signature adaptability (Definition 3.4.8).*

3.7.2 IPFE from Groups of Prime-Order

We first recall the IPFE scheme by Abdalla et al. [ABDP15]. Then, we show that it satisfies the additional compliance and robustness properties needed by our FAS scheme.

Suppose p is a λ-bit prime number and suppose we want to compute inner products of vectors of length ℓ. The Abdalla et al. [ABDP15] IPFE scheme computes inner products with output values polynomially bounded by $B \ll p$. Specifically, the message space is $\mathcal{M} = \mathbb{Z}_p^\ell$ and the function class is $\mathcal{F}_{\mathsf{IP},\ell,p,B} = \{f_\mathbf{y} : \mathbf{y} \in \mathbb{Z}_p^\ell\} \subseteq \mathcal{F}_{\mathsf{IP},\ell}$, where $f_\mathbf{y} : \mathbb{Z}_p^\ell \to \{0, \ldots, B\}$ is defined as $f_\mathbf{y}(\mathbf{x}) = \mathbf{x}^T\mathbf{y} \in \{0, \ldots, B\}$. The scheme by Abdalla et al. [ABDP15] augmented with PubKGen algorithm is as in Figure 3.15. For $d \in \mathbb{G}$, we denote the algorithm for computing the discrete log of d with respect to base $g \in \mathbb{G}$ by $\mathsf{DLog}_g(d)$. As long as we are promised that the discrete log is bounded by a polynomial B, the DLog algorithm simply enumerates all the possibilities to find the discrete log. The running time is polynomial because of the bound B.

Lemma 3.7.2 ([ABDP15]). *Suppose the DDH assumption holds. Then, the IPFE scheme in Figure 3.15 is selective, IND-secure (Definition 3.4.10).*

Next, we show the IPFE scheme augmented with the above $\mathsf{PubKGen}$ algorithm satisfies R_{DL}-compliance and R_{DL}-robustness.

Lemma 3.7.3. *The IPFE scheme in Figure 3.15 is R_{DL}-compliant.*

Proof. For any $\lambda \in \mathbb{N}$, for any $\mathsf{pp} \leftarrow \mathsf{IPFE.Gen}(1^\lambda)$ as implemented in Figure 3.15, for any $(\mathsf{mpk}, \mathsf{msk}) \leftarrow \mathsf{IPFE.Setup}(\mathsf{pp}, 1^\ell)$ as implemented in Figure 3.15, for any $\mathbf{y} \in \mathcal{F}_{\mathsf{IP},\ell,p,B}$, let $\mathsf{pk}_{\mathbf{y}} := \mathsf{PubKGen}(\mathsf{mpk}, \mathbf{y})$ as implemented in Figure 3.15, and $\mathsf{sk}_{\mathbf{y}} := \mathsf{KGen}(\mathsf{msk}, \mathbf{y})$ as implemented in Figure 3.15. Suppose here $\mathsf{msk} = \mathbf{s} \in \mathbb{Z}_p^\ell$. Then, $\mathsf{sk}_{\mathbf{y}} = \mathbf{s}^T\mathbf{y} \in \mathbb{Z}_p$ and $\mathsf{pk}_{\mathbf{y}} = g^{\mathbf{s}^T\mathbf{y}} \in \mathbb{G}$. Thus, clearly, $(\mathsf{pk}_{\mathbf{y}}, \mathsf{sk}_{\mathbf{y}}) \in R_{\mathsf{DL}}$. □

Lemma 3.7.4. *The IPFE scheme in Figure 3.15 is R_{DL}-robust.*

Proof. Observe that in the IPFE scheme in Figure 3.15, the base relation R and the extended relation R' are the same: $R = R' = R_{\mathsf{DL}}$. Further, R_{DL} is a unique witness relation. Thus, R_{DL}-robustness follows trivially from the correctness of the IPFE scheme. □

3.7.3 FAS Construction

Instantiating FAS construction in Section 3.6 with IPFE from Figure 3.15 and AS from Section 3.7.1, we obtain the following corollary.

Corollary 3.7.5. *Let p be a λ-bit prime number and let ℓ be an integer. Let $\mathcal{M} = \mathbb{Z}_p^\ell$ be an additive group. let $\mathcal{F}_{\mathsf{IP},\ell,\mathsf{p},\mathsf{B}}$ be the function family for computing inner products of vectors in $\mathbb{Z}_p^\ell$ such that the output value is bounded by some polynomial $B \ll p$. Let R be any NP relation with statement/witness pairs $(X, \mathbf{x})$ such that $\mathbf{x} \in \mathbb{Z}_p^\ell$. Suppose that*

- *R is $\mathcal{F}_{\mathsf{IP},\ell,p,B}$-hard (Definition 3.4.2),*
- *NIZK is a secure NIZK argument system (Definition 3.4.26),*
- *AS construction in Section 3.7.1 is an adaptor signature scheme w.r.t. digital signature scheme Sch and hard relation R_{DL} that satisfies weak pre-signature adaptability and witness extractability (Lemma 3.7.1),*
- *IPFE construction in Figure 3.15 is a selective, IND-secure IPFE scheme (Lemma 3.7.2) for function family $\mathcal{F}_{\mathsf{IP},\ell+1}$ that is R_{IPFE}-compliant (Lemma 3.7.3) and R_{IPFE}-robust (Lemma 3.7.4).*

Then, the functional adaptor signature scheme w.r.t. Schnorr signature scheme Sch, NP relation R, and family of inner product functions $\mathcal{F}_{\mathsf{IP},\ell,p,B}$ constructed in Figure 3.2 is strongly-secure (Definition 3.5.3).

The proof of the above corollary is immediate from the proof of the Theorem 3.6.2 concerning the generic construction, and Lemmas 3.7.2 to 3.7.4 that show that the IPFE scheme in Figure 3.15 has the required properties.

3.8 FAS from lattices

In this section, we provide instantiations of the FAS construction in Figure 3.2 from lattices. For this, it suffices to instantiate the building blocks IPFE and AS from lattices, while ensuring the two are compatible with each other. We describe these next.

More specifically, we instantiate the AS as the lattice-based adaptor signature scheme by Esgin et al. [EEE20a]. This scheme was built using cyclotomic ring $\mathcal{R} = \mathbb{Z}[x]/(x^d + 1)$ of degree $d = 256$ under Module-SIS and Module-LWE assumptions. Here, we set the degree to be $d = 1$, thus giving us the ring of integers $\mathcal{R} = \mathbb{Z}$. Consequently, security follows from plain SIS and LWE assumptions. One can look at the resulting AS to be w.r.t. a digital scheme Lyu′ that is somewhere in between Lyubashevsky's signature scheme [Lyu12] and Dilithium [DKL+18] instantiated with unstructured lattices. Further, AS is w.r.t. hard relations R_{ISIS} and R'_{ISIS} such that $R_{\mathsf{ISIS}} \subset R'_{\mathsf{ISIS}}$.

Further, we show that the IND-secure IPFE scheme by Agrawal et al. [ALS16, Section 4.2] satisfies R_{ISIS}-compliance and R'_{ISIS}-robustness when appropriately augmented with a PubKGen algorithm. We describe the instantiations of these two building blocks in the subsequent sections.

3.8.1 Lattice-based Adaptor Signature

First, we recall the lattice-based signature scheme Lyu′, which can be seen as a variant of Lyubashevsky's signature scheme [Lyu12] where the vector $\mathbf{y}$ sampled during signing comes from a uniform distribution over a small range instead of discrete gaussian distribution. It can also be seen as a variant of Dilithium signature scheme [DKL+18] instantiated over unstructured lattices.

Suppose $\mathsf{pp} := (n, m, p, q, k, \alpha) \leftarrow \mathsf{IPFE.Gen}(1^\lambda, p)$ as described in Figure 3.18, where p is a prime number, $n = \lambda$, $q = p^k$ and $k \in \mathbb{Z}$, $m \in \mathbb{Z}$, real $\alpha \in (0, 1)$ are chosen as described in the "Parameter choices" in Section 3.8.2. Let $\mathcal{H} : \{0,1\}^* \to \mathcal{C}$ be a family of hash functions (modelled as a random oracle), where $\mathcal{C} = \{\mathbf{c} \in \mathbb{Z}^h : ||\mathbf{c}||_1 = \kappa \wedge ||\mathbf{c}||_\infty = 1\}$. Let γ be the maximum absolute coefficient of the masking randomness $\mathbf{y}$ used in the Sign algorithm below. Then, the digital signature scheme Lyu′ is as in Figure 3.16.

$\mathsf{Lyu}'.\mathsf{KGen}(1^\lambda, \mathsf{pp})$

1 : Sample $\mathbf{A}' \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$
2 : Let $\mathbf{A} := (I_n || \mathbf{A}') \in \mathbb{Z}_q^{n \times (n+m)}$
3 : Sample $\mathbf{R} \xleftarrow{\$} \mathcal{S}_1^{(n+m) \times h}$
4 : Let $\mathbf{T} := \mathbf{AR} \bmod q \in \mathbb{Z}_q^{n \times h}$
5 : **ret** $\mathsf{vk} := (\mathbf{A}, \mathbf{T}), \mathsf{sk} := \mathbf{R}$

$\mathsf{Lyu}'.\mathsf{Sign}(\mathsf{sk}, \mu \in \{0,1\}^*)$

1 : Sample $\mathbf{y} \xleftarrow{\$} \mathcal{S}_\gamma^{n+m}$
2 : Let $\mathbf{w} := \mathbf{Ay}$
3 : Let $\mathbf{c} := \mathbb{H}(\mathsf{vk} \,||\, \mathbf{w} \,||\, \mu) \in \mathbb{Z}^h$
4 : Let $\mathbf{z} := \mathbf{y} + \mathbf{Rc}$
5 : If $||\mathbf{z}||_\infty > \gamma - \kappa$: restart
6 : **ret** $\sigma := (\mathbf{c}, \mathbf{z})$

$\mathsf{Lyu}'.\mathsf{Vf}(\mathsf{vk}, \mu, \sigma)$

1 : Parse $\sigma = (\mathbf{c}, \mathbf{z})$
2 : If $||\mathbf{z}||_\infty > \gamma - \kappa$: **ret** 0
3 : Let $\mathbf{w}' := \mathbf{Az} - \mathbf{Tc}$
4 : If $\mathbf{c} \neq \mathbb{H}(\mathsf{vk} \,||\, \mathbf{w}' \,||\, \mu)$: **ret** 0
5 : **ret** 1

Figure 3.16: Digital signature scheme Lyu' from SIS and LWE

Next, we describe the Adaptor Signature scheme AS with respect to the digital signature scheme Lyu' and hard relations $R_{\mathsf{ISIS}} := \mathsf{ISIS}_{n,n+m,q,\beta_0}$ and $R'_{\mathsf{ISIS}} := \mathsf{ISIS}_{n,n+m,q,\beta_1}$ such that $R_{\mathsf{ISIS}} \subseteq R'_{\mathsf{ISIS}}$. Here, $\beta_0 := \ell p B_\tau$ and $\beta_1 := 2(\gamma - \kappa) - \beta_0$ such that $\beta_1 > \beta_0$ and $\gamma - \kappa - \beta_0 > 0$. Let L_{ISIS} be the language corresponding to R_{ISIS} and R'_{ISIS}. Notice that $L_{\mathsf{ISIS}} \subseteq \mathbb{Z}_q^n$. Hence, for R_{ISIS}, statements are of the form $(\mathbf{A}, X) \in \mathbb{Z}_q^{n \times (n+m)} \times \mathbb{Z}_q^n$ (same for $'_{\mathsf{ISIS}}$) and witnesses are of the form $\mathbf{x} \in \mathbb{Z}^{n+m}$ such that $||\mathbf{x}||_\infty \leq \beta_0$ ($||\mathbf{x}||_\infty \leq \beta_1$ for R'_{ISIS}). As $\mathbf{A}$ is also part of the verification key, henceforth we drop it from the statement. We next describe the AS construction in Figure 3.17.

$\mathsf{AS.PreSign}(\mathsf{sk}, \mu \in \{0,1\}^*, X \in L_{\mathsf{ISIS}})$

1 : Sample $\mathbf{y} \xleftarrow{\$} \mathcal{S}_\gamma^{n+m}$, let $\mathbf{w} := \mathbf{Ay}$
2 : Let $\mathbf{c} := \mathbb{H}(\mathsf{vk} \,||\, \mathbf{w} + X \,||\, \mu) \in \mathbb{Z}^h$
3 : Let $\tilde{\mathbf{z}} := \mathbf{y} + \mathbf{Rc}$
4 : If $||\tilde{\mathbf{z}}||_\infty > \gamma - \kappa - \beta_0$: restart
5 : **ret** $\tilde{\sigma} := (\mathbf{c}, \tilde{\mathbf{z}})$

$\mathsf{AS.Adapt}(\mathsf{vk}, \mu, X, x, \tilde{\sigma})$

1 : If $\mathsf{PreVerify}(\mathsf{vk}, \mu, X, \tilde{\sigma}) = 0$: **ret** $\perp$
2 : Parse $\tilde{\sigma} = (\mathbf{c}, \tilde{\mathbf{z}})$
3 : $\mathbf{z} := \tilde{\mathbf{z}} + x$
4 : **ret** $\sigma := (\mathbf{c}, \mathbf{z})$

$\mathsf{AS.PreVerify}(\mathsf{vk}, \mu, X, \tilde{\sigma})$

1 : Parse $\tilde{\sigma} = (\mathbf{c}, \tilde{\mathbf{z}})$
2 : If $||\tilde{\mathbf{z}}||_\infty > \gamma - \kappa - \beta_0$: **ret** 0
3 : Let $\mathbf{w}' := \mathbf{A}\tilde{\mathbf{z}} - \mathbf{Tc}$
4 : If $\mathbf{c} \neq \mathbb{H}(\mathsf{vk} \,||\, \mathbf{w}' \,||\, \mu)$: **ret** 0
5 : **ret** 1

$\mathsf{AS.Ext}(\tilde{\sigma}, \sigma, X)$

1 : Parse $\tilde{\sigma} = (\mathbf{c}, \tilde{\mathbf{z}}), \sigma = (\mathbf{c}, \mathbf{z})$
2 : $x' := \mathbf{z} - \tilde{\mathbf{z}}$
3 : If $X \neq \mathbf{A}x' \bmod q$: **ret** $\perp$
4 : **ret** x'

Figure 3.17: AS w.r.t. Lyu' signature scheme and hard relations R_{ISIS} and R'_{ISIS}

Parameter choices and comparison. Our construction can be seen as similar to [EEE20a] but with degree 1 instead of 256. Our construction is different from [EEE20a] in the following aspects though: [EEE20a] chooses $\beta_0 = 1$, whereas we choose $\beta_0 = \ell p B_\tau$, where parameters ℓ, p, B_τ are chosen based on the specification in the IPFE construction in Figure 3.18. Further, [EEE20a] sets $\beta_1 = 2(\gamma - \kappa)$ whereas we set it a stricter bound on it $\beta_1 = 2(\gamma - \kappa) - \beta_0$ as it is sufficient for correctness and security. Further, note that for the choice of β_1, the two constraints $\beta_1 > \beta_0$ and $\gamma - \kappa - \beta_0 > 0$ are equivalent. Lastly, we note that

just like in [EEE20a], we can set $\gamma = 2\kappa$ to ensure that the average number of restarts in Sign and PreSign is about $e < 3$, where κ is the ℓ_1 norm of values in the range set $\mathcal{C}$ of the hash family $\mathcal{H}$. Lastly, in [EEE20a], the hash outputs belong to some polynomial ring $\mathcal{R}_q$, whereas we choose the range of hash outputs to be $\mathbb{Z}_q^h$ for some integer h.

Lemma 3.8.1 ([EEE20a]). *The adaptor signature scheme in Figure 3.17 satisfies witness extractability (Definition 3.4.7) and weak pre-signature adaptability (Definition 3.4.8).*

3.8.2 IPFE from Lattices

We first recall the IPFE scheme by Agrawal et al. [ALS16]. Then, we show that it satisfies the additional compliance and robustness properties needed by our FAS construction.

Suppose p is a prime number and suppose we want to compute inner products of vectors of length ℓ. Let the set of messages be $\mathcal{M} = \mathbb{Z}_p^\ell$ and the function class be $\mathcal{F}_{\mathsf{IP},\ell,p} = \{f_{\mathbf{y}} : \mathbf{y} \in \mathbb{Z}_p^\ell\}$, where $f_{\mathbf{y}} : \mathbb{Z}_p^\ell \to \mathbb{Z}_p$ is defined as $f_{\mathbf{y}}(\mathbf{x}) = \mathbf{x}^T\mathbf{y} \bmod p$. Our IPFE scheme augmented with PubKGen algorithm is as in Figure 3.18.

Remark 3.8.2. *We note that the original construction needs* KGen *to be stateful in order to support key generation queries that are linearly dependent modulo p. This is problematic for us as the corresponding* PubKGen *will also have to be stateful then, but that defeats the purpose of this algorithm being public. So, in order to remedy the situation, we assume that all* KGen *queries to* IPFE *are going to be linearly independent. Naively instantiating our* FAS *construction (Figure 3.2) will result in* FAS *only supporting linearly independent functions, which is acceptable if it involves a single buyer but in the most general case we would want to support multiple buyers who are agnostic of each other. Specifically, a buyer may engage in* FAS *protocol to learn a function evaluation of the secret that is linearly dependent on something learnt by other some other buyers. To handle such a scenario, we show in Section 3.8.4 how to assign unique IDs to each buyer and make the underlying* IPFE *key generation queries linearly independent.*

$\mathsf{IPFE.Gen}(1^\lambda, p)$

1 : Let $n = \lambda$, integers m, k, real $\alpha \in (0,1)$ as defined below
2 : Let $q = p^k$, **ret** $\mathsf{pp} = (n, m, p, q, k, \alpha)$

$\mathsf{IPFE.Setup}(\mathsf{pp}, 1^\ell)$

1 : Sample $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{m \times n}$, $\mathbf{Z} \leftarrow \tau$ where distribution τ over $\mathbb{Z}^{\ell \times m}$ is as defined below
2 : Let $\mathbf{U} := \mathbf{ZA} \in \mathbb{Z}_q^{\ell \times n}$, **ret** $\mathsf{mpk} := (\mathbf{A}, \mathbf{U}), \mathsf{msk} := \mathbf{Z}$

$\mathsf{IPFE.Enc}(\mathsf{mpk}, \mathbf{x} \in \mathbb{Z}_p^\ell)$

1 : Sample $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^n$, $\mathbf{e}_0 \leftarrow D_{\mathbb{Z},\alpha q}^m$, $\mathbf{e}_1 \leftarrow D_{\mathbb{Z},\alpha q}^\ell$
2 : Let $\mathsf{ct}_0 := \mathbf{As} + \mathbf{e}_0 \in \mathbb{Z}_q^m$, $\mathsf{ct}_1 := \mathbf{Us} + \mathbf{e}_1 + p^{k-1}\mathbf{x} \in \mathbb{Z}_q^\ell$
3 : **ret** $\mathsf{ct} := (\mathsf{ct}_0, \mathsf{ct}_1)$

$\mathsf{IPFE.KGen}(\mathsf{msk}, \mathbf{y} \in \mathbb{Z}_p^\ell)$

1 : **ret** $\mathsf{sk}_\mathbf{y} := \mathbf{Z}^T\mathbf{y} \in \mathbb{Z}^m$

$\mathsf{IPFE.PubKGen}(\mathsf{mpk}, \mathbf{y} \in \mathbb{Z}_p^\ell)$

1 : **ret** $\mathsf{pk}_\mathbf{y} := \mathbf{U}^T\mathbf{y} \in \mathbb{Z}_q^n$

$\mathsf{IPFE.Dec}(\mathsf{sk}_\mathbf{y}, \mathsf{ct})$

1 : Let $d := \mathsf{ct}_1^T\mathbf{y} - \mathsf{ct}_0^T\mathsf{sk}_\mathbf{y} \bmod q$
2 : **ret** $v \in \mathbb{Z}_p$ that minimizes $| p^{k-1}v - d |$

Figure 3.18: Agrawal et al. [ALS16] IPFE scheme augmented with PubKGen algorithm. Note that KGen here is stateless.

Parameter choices. Let B_τ be such that with probability at most $n^{-\omega(1)}$, each row of sample from τ has ℓ_2-norm at least B_τ. Then, the parameter constraints for correctness and security are as follows.

- $\alpha^{-1} \geq \ell^2 p^3 B_\tau \omega(\sqrt{\log n})$, $q \geq \alpha^{-1}\omega(\sqrt{\log n})$,
- $\tau = D_{\mathbb{Z},\sigma_1}^{\ell \times m/2} \times (D_{\mathbb{Z}^{m/2},\sigma_2,\delta_1} \times \ldots \times D_{\mathbb{Z}^{m/2},\sigma_2,\delta_\ell})$, where $\delta_i \in Z^\ell$ denotes the i-th canonical vector, and the standard deviation parameters satisfy $\sigma_1 = \Theta(\sqrt{n \log m}\max(\sqrt{m}, K'))$ and $\sigma_2 = \Theta(n^{7/2}m1/2\max(m, K'^2)\log^{5/2} m)$, with $K' = (\sqrt{\ell}p)^\ell$.

Further, R'_{ISIS}-robustness (Lemma 3.8.5) will require the constraint $\alpha^{-1} \geq 4p\omega(\sqrt{\log n})(\ell p + m\beta_1)$, where β_1 is as chosen below.

Lemma 3.8.3 ([ALS16]). *Suppose $\ell \leq n^{O(1)}$, $m \geq 4n\log_2(q)$ and q, α, τ are as described above. Suppose $\mathsf{mheLWE}_{q,\alpha,m,\ell,\tau}$ assumption (Definition 3.4.19) holds. Then, the IPFE scheme in Figure 3.18 is selective, IND-secure (Definition 3.4.10).*

Next, we show the IPFE scheme augmented with the above PubKGen algorithm satisfies R_{ISIS}-compliance and R'_{ISIS}-robustness, where $R_{\mathsf{ISIS}} = \mathsf{ISIS}_{n,m,q,\beta_0}$ and $R'_{\mathsf{ISIS}} = \mathsf{ISIS}_{n,m,q,\beta_1}$, where $\beta_0 = \ell p B_\tau$ and $\beta_1 = 2(\gamma - \kappa) - \beta_0$ s.t. $\beta_1 > \beta_0$ and $\gamma - \kappa - \beta_0 > 0$.

Lemma 3.8.4. *The IPFE scheme in Figure 3.18 is R_{ISIS}-compliant, where $R_{\mathsf{ISIS}} = \mathsf{ISIS}_{n,m,q,\beta_0}$ and $\beta_0 = \ell p B_\tau$.*

Proof. For any $\lambda \in \mathbb{N}$, for any $\mathsf{pp} \leftarrow \mathsf{IPFE.Gen}(1^\lambda)$ as implemented in Figure 3.18, for any $(\mathsf{mpk}, \mathsf{msk}) \leftarrow \mathsf{IPFE.Setup}(\mathsf{pp}, 1^\ell)$ as implemented in Figure 3.18, for any $\mathbf{y} \in \mathcal{F}_{\mathsf{IP},\ell,p}$, let $\mathsf{pk}_\mathbf{y} := \mathsf{PubKGen}(\mathsf{mpk}, \mathbf{y})$ as implemented in Figure 3.18, and $\mathsf{sk}_\mathbf{y} := \mathsf{KGen}(\mathsf{msk}, \mathbf{y})$ as implemented in Figure 3.18. Suppose here

$\mathsf{mpk} = (\mathbf{A}, \mathbf{U})$ and $\mathsf{msk} = \mathbf{Z}$. Then, $\mathsf{sk}_{\mathbf{y}} = \mathbf{Z}^T\mathbf{y} \in \mathbb{Z}^m$ and $\mathsf{pk}_{\mathbf{y}} = \mathbf{U}^T\mathbf{y} \bmod q = \mathbf{A}^T\mathsf{sk}_{\mathbf{y}} \bmod q$. Further, note that $||\mathsf{sk}_{\mathbf{y}}||_\infty \leq \ell \cdot ||\mathbf{Z}||_\infty \cdot ||\mathbf{y}||_\infty \leq \ell \cdot ||\mathbf{Z}||_2 \cdot ||\mathbf{y}||_\infty \leq \ell \cdot B_\tau \cdot p = \beta_0$. Hence, $(\mathsf{pk}_{\mathbf{y}}, \mathsf{sk}_{\mathbf{y}}) \in R_{\mathsf{ISIS}}$. □

Lemma 3.8.5. *If $\alpha^{-1} \geq 4p\omega(\sqrt{\log n})(\ell p + m\beta_1)$, then, the IPFE scheme in Figure 3.18 is R'_{ISIS}-robust, where $R'_{\mathsf{ISIS}} = \mathsf{ISIS}_{n,m,q,\beta_1}$ and $\beta_1 = 2(\gamma - \kappa) - \beta_0$ such that $\beta_1 > \beta_0$.*

Proof. For any $\lambda \in \mathbb{N}$ let $\mathsf{pp} \leftarrow \mathsf{IPFE.Gen}(1^\lambda)$ as implemented in Figure 3.18, for any $\ell \in \mathbb{N}$, $\mathbf{x} \in \mathbb{Z}_p^\ell, \mathbf{y} \in \mathcal{F}_{\mathsf{IP},\ell,p}$, let $(\mathsf{mpk}, \mathsf{msk}) \leftarrow \mathsf{Setup}(\mathsf{pp}, \ell)$ as implemented in Figure 3.18, let $\mathsf{pk}_{\mathbf{y}} = \mathsf{PubKGen}(\mathsf{msk}, \mathbf{y})$ as implemented in Figure 3.18, let $\mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{mpk}, \mathbf{x})$ as implemented in Figure 3.18. Then, for any $\mathsf{sk}'_{\mathbf{y}}$ such that $(\mathsf{pk}_{\mathbf{y}}, \mathsf{sk}'_{\mathbf{y}}) \in R'_{\mathsf{ISIS}}$, we want that $\mathsf{Dec}(\mathsf{sk}'_{\mathbf{y}}, \mathsf{ct})$ outputs $\mathbf{x}^T\mathbf{y} \bmod q$. Observe that the value d computed by Dec implemented in Figure 3.18 simplifies to $d = p^{k-1}\mathbf{x}^T\mathbf{y} + (\mathbf{e}_1^T\mathbf{y} - \mathbf{e}_0^T\mathsf{sk}'_{\mathbf{y}}) \bmod q$. Observe that

$$\begin{aligned}|\mathbf{e}_1^T\mathbf{y} - \mathbf{e}_0^T\mathsf{sk}_{\mathbf{y}}| &\leq \ell p\alpha q\omega(\sqrt{\log n}) + m\alpha q\omega(\sqrt{\log n})||\mathsf{sk}'_{\mathbf{y}}||_\infty \\ &= \alpha q\omega(\sqrt{\log n})(\ell p + m\beta_1).\end{aligned}$$

For decryption correctness to hold, we need that $|\mathbf{e}_1^T\mathbf{y} - \mathbf{e}_0^T\mathsf{sk}_{\mathbf{y}}| \leq q/4p$. Hence it suffices to have $\alpha^{-1} \geq 4p\omega(\sqrt{\log n})(\ell p + m\beta_1)$. □

3.8.3 FAS Construction

Instantiating FAS construction in Section 3.6 with IPFE from Figure 3.18 and AS from Figure 3.17, we obtain the following corollary.

Corollary 3.8.6. *Let p be a λ-bit prime number and let ℓ be an integer. Let $\mathcal{M} = \mathbb{Z}_p^\ell$ be an additive group. Let $\mathcal{F}_{\mathsf{IP},\ell,p}$ be the function family for computing inner products of vectors in $\mathbb{Z}_p^\ell$. Let R be any NP relation with statement/witness pairs $(X, \mathbf{x})$ such that $\mathbf{x} \in \mathbb{Z}_p^\ell$. Let $R_{\mathsf{ISIS}} = \mathsf{ISIS}_{n,m,q,\beta_0}$ and $R'_{\mathsf{ISIS}} = \mathsf{ISIS}_{n,m,q,\beta_1}$, where $\beta_0 = \ell p B_\tau$ and $\beta_1 = 2(\gamma - \kappa) - \beta_0$ s.t. $\beta_1 > \beta_0$ and $\gamma - \kappa - \beta_0 > 0$. Suppose that*

- *All function queries $\mathbf{y}$ to $\mathsf{FPreSign}_1$ are linearly independent modulo p,*
- *R is $\mathcal{F}_{\mathsf{IP},\ell,p}$-hard (Definition 3.4.2),*
- *NIZK is a secure NIZK argument system (Definition 3.4.26),*
- *AS construction in Figure 3.17 is an adaptor signature scheme w.r.t. digital signature scheme Lyu' and hard relations R_{ISIS} and R'_{ISIS} that satisfies weak pre-signature adaptability and witness extractability (Lemma 3.8.1),*
- *IPFE construction in Figure 3.15 is a selective, IND-secure IPFE scheme (Lemma 3.8.3) for function family $\mathcal{F}_{\mathsf{IP},\ell+1}$ that is R_{ISIS}-compliant (Lemma 3.8.4) and R'_{ISIS}-robust (Lemma 3.8.5).*

Then, the functional adaptor signature scheme w.r.t. Lyubashevsky signature scheme Lyu'*, NP relation* R*, and family of inner product functions* $\mathcal{F}_{\mathsf{IP},\ell,p}$ *constructed in Figure 3.2 is strongly-secure (Definition 3.5.3).*

The proof of the above corollary is immediate from the proof of the Theorem 3.6.2 concerning the generic construction, and Lemmas 3.8.3 to 3.8.5 that show that the IPFE scheme in Figure 3.18 has the required properties.

Remark 3.8.7. *Note that the linear independence modulo* p *restriction for all* $\mathsf{FPreSign}_1$ *queries in this lattice-based instantiation comes due to the same restriction on the underlying IPFE scheme as discussed in Remark 3.8.2. In practice,* $\mathsf{FPreSign}_1$ *queries can be sent by any buyer to the seller. On one hand it's reasonable to assume that all the queries by a single buyer are going to be linearly independent modulo* p *because by linearity of the functionality, the buyer can locally compute the function evaluation for some function* $\mathbf{y}$ *that is linear combination of the functions* $\mathbf{y}_i$*'s it previously queried as follows: if* $\mathbf{y} = \sum_i k_i \mathbf{y}_i \bmod q$*, then,* $f_{\mathbf{y}}(\mathbf{x})$ *can be computed as* $f_{\mathbf{y}}(\mathbf{x}) := \sum_i k_i f_{\mathbf{y}_i}(\mathbf{x})$*. On the other hand, it's unreasonable to assume that all buyers are working in coordination and make sure that their queries are linearly independent. In Section 3.8.4, we show how to remove this restriction on FAS.*

3.8.4 Modifying FAS Construction in Figure 3.2.

There is a simple way to ensure that even if $\mathsf{FPreSign}_1$ queries across multiple buyers are linearly dependent modulo p, the requests to the underlying $\mathsf{IPFE.PubKGen}$ are always linearly independent. Assume a polynomial bound k on the number of buyers. Then, we add k additional slots to the underlying IPFE resulting in a total of $\ell_k + 1$ slots and modify the FAS construction as follows:

- In AdGen on input $\mathbf{x} \in \mathbb{Z}_p^\ell$, the vector $\widetilde{\mathbf{x}} := (\mathbf{x}^T, \mathbf{0}^T)^T \in \mathbb{Z}_p^{\ell+k+1}$ is encrypted using $\mathsf{IPFE.Enc}$.
- Further, the seller maintains an internal map of size k from buyer verification keys vk's to a unique unit vector assigned to them. In other words, i-th buyer with verification key denoted by vk_i is assigned i-th unit vector $\mathbf{e}_i \in \{0,1\}^k$.
- At time of $\mathsf{FPreSign}_1$ query for vector $\mathbf{y} \in \mathbb{Z}_p^\ell$ by i-th buyer, the seller uses $\widetilde{\mathbf{y}} := (\mathbf{y}^T, f_{\mathbf{y}}(\mathbf{t}), \mathbf{e}_i^T)^T \in \mathbb{Z}_p^{\ell+k+1}$ to compute $\mathsf{aux}_{\mathbf{y}}$ and sets $\pi_{\mathbf{y}} := (f_{\mathbf{y}}(\mathbf{t}), i)$.
- In $\mathsf{FPreVerify}_1$, the buyer can recompute $\widetilde{\mathbf{y}}$ on its own using $\pi_{\mathbf{y}} = (f_{\mathbf{y}}(\mathbf{t}), i)$ and return 1 iff $\mathsf{aux}_{\mathbf{y}} = \mathsf{IPFE.PubKGen}(\mathsf{mpk}, \widetilde{\mathbf{y}})$.
- At the time of running Adapt, the seller looks up vk in its internal map to find the index i assigned to it. Then, it can recompute the same $\widetilde{\mathbf{y}}$ that it used during $\mathsf{FPreSign}_1$ query $\mathbf{y}$ by buyer vk. This will ensure that the adapted signature passes the signature verification.

Note that, the k extra slots as used above do not affect correctness as the function evaluation for a function $\mathbf{y}$ is always $\widetilde{\mathbf{x}}^T\widetilde{\mathbf{y}} = \mathbf{x}^T\mathbf{y} \bmod q$. Further note that these slots remain the same in all experiments for all security properties.

Remark 3.8.8 (Linear Independence)**.** *Observe that with the above modification, even if two buyers i and j requested for the same function $\mathbf{y} \in \mathbb{Z}_p^\ell$, $\mathsf{FPreSign}_1$ will use linearly independent queries $\widetilde{\mathbf{y}}_i$ and $\widetilde{\mathbf{y}}_j$ to the underlying $\mathsf{IPFE.PubKGen}$, where $\widetilde{\mathbf{y}}_i := (\mathbf{y}^T, f_{\mathbf{y}}(\mathbf{t}), \mathbf{e}_i^T)^T \in \mathbb{Z}_p^{\ell+k+1}$ and $\widetilde{\mathbf{y}}_j := (\mathbf{y}^T, f_{\mathbf{y}}(\mathbf{t}), \mathbf{e}_j^T)^T \in \mathbb{Z}_p^{\ell+k+1}$. These two are linearly independent because for $i \neq j$, the unit vectors $\mathbf{e}_i$ and $\mathbf{e}_j$ are linearly independent of each other.*

Remark 3.8.9 (Communication overhead)**.** *Note that in the IPFE construction in Figure 3.18, the size of $\mathsf{pk}_{\mathbf{y}}$ and $\mathsf{sk}_{\mathbf{y}}$ are independent of the vector length parameter ℓ. Hence, change from $\ell + 1$ to $\ell + k + 1$ does not increase the size $\mathsf{aux}_{\mathbf{y}}$. The size of $\pi_{\mathbf{y}}$ increases by $\log k$ bits, but that can also optimized to be sent only once per buyer and not for every $\mathsf{FPreSign}_1$ query. The main communication overhead is in the ciphertext ct communicated as part of the advertisement ad. Now $\mathsf{ct} \in \mathbb{Z}_q^{m+\ell_k+1}$ instead of $\mathsf{ct} \in \mathbb{Z}_q^{m+\ell+1}$. But fortunately, ad needs to be communicated only once.*

3.9 Performance Evaluation of FE-based FAS

Our implementation aims to (i) show that FAS can replace smart contracts for efficient functional sales, and (ii) benchmark the computational costs for each functional sale. We provide an open-source implementation [VST24b] of our prime-order group-based strongly-secure FAS in Python. We also perform a series of benchmarks on our implementation for a wide range of parameters. Our results show that our scheme is practical for variety of real-world scenarios.

We measured the costs on a Apple MacBook Pro with M2 chip, 16GB memory, 8 cores (4 cores @3.49 GHz and 4 cores @2.42 GHz). In typical applications, a seller wants to sell multiple functions of the same witness. The (application dependent) advertisement – containing NIZK proof – is just a one-time cost, while other processes are done once per sale. So, we benchmark computation costs of all algorithms except AdGen and AdVerify.

We use the Secp256k1 elliptic curve that is used by Bitcoin [sec] and other cryptocurrencies. We use the curve's python implementation from hanabi1224 [han]. For Schnorr signatures, we use a modified version of BIP-340 reference implementation [bip]. We implement the Schnorr adaptor signatures [AEE+21] on top of it. For compliance purpose, we implement the IPFE scheme in Figure 3.15 on the Secp256k1 curve. Finally, using these adaptor signatures and IPFE schemes, we implement our functional adaptor signature construction. We do not implement the NIZKs as they are needed only for AdGen and AdVerify which we do not implement here.

Table 3.1: Communication efficiency of FAS for $\ell = 10^6$. Last column denotes if a secure channel for buyer/seller is used.

Variable	Size	On-chain?	Secure channel?
$\mathsf{ad} = (\mathsf{IPFE.mpk}, \mathsf{IPFE.ct})$	128 MB	no[16]	no
Function $\mathbf{y}$	32 MB	no	yes
$(\mathsf{aux}_{\mathbf{y}}, \pi_{\mathbf{y}})$	96 bytes	no	yes
Pre-signature	64 bytes	no	yes
Adapted-signature	64 bytes	yes	no

Optimizations. We make following implementation optimizations.

- For vectors of length ℓ, we parallelize IPFE.Setup, IPFE.Enc.
- Recall that IPFE.Dec computes $\mathsf{ct}_1^T\mathbf{y} - \mathsf{ct}_0^T\mathsf{sk}_{\mathbf{y}}$ (See Figure 3.15). Here, ct_1 and $\mathbf{y}$ are vectors of length ℓ and $\mathsf{ct}_1^T\mathbf{y}$ is computed as $\prod_{i\in\ell} c_i^{y_i}$, where $\mathsf{ct}_1 = (c_1, \ldots, c_\ell)$ is a vector of group elements and $\mathbf{y} = (y_1, \ldots, y_\ell)$ is a vector of scalars. Thus, computing $\mathsf{ct}_1^T\mathbf{y}$ is expensive as it involves ℓ group exponentiation operations. But, this computation can be done in an offline stage by the buyer as it does not require knowledge of $\mathsf{sk}_{\mathbf{y}}$ from the seller. This helps us make IPFE.Dec and thus, FAS.FExt very efficient in practice.
- Note that IPFE.PubKGen and IPFE.Dec involve computing a product of powers such as $\prod_{i\in\ell} k_i^{y_i}$ and $\prod_{i\in\ell} c_i^{y_i}$ respectively. We compute these via FastMult algorithm of [BGR98, Section 3.2].
- Note that IPFE.Dec involves a discrete log computation. For this task, we implement the baby-step giant-step algorithm [Sha71].
- Note that $\mathsf{FPreSign}_1$ involves computing $\mathsf{pk}_{\mathbf{y}} := \mathsf{IPFE.PubKGen}(\mathsf{mpk}, \widetilde{\mathbf{y}})$ which requires $\ell + 1$ group exponentiation operations. As $\mathsf{FPreSign}_1$ is run by the seller who knows $\mathsf{st} = (\mathsf{msk}, \mathbf{t})$, hence, we optimize this computation as $\mathsf{sk}_{\mathbf{y}} := \mathsf{IPFE.KGen}(\mathsf{msk}, \widetilde{\mathbf{y}})$, $\mathsf{pk}_{\mathbf{y}} := g^{\mathsf{sk}_{\mathbf{y}}}$. This resulting way only involves 1 group exponentiation operation and hence is very efficient.

Communication efficiency. In our implementation, each group element is 64 bytes elliptic curve point and each Z_p element is 32 bytes. In Table 3.1, we give bounds on communication cost for a witness x of dimension $\ell = 10^6$ (total size 32 MB) and also specify if the communication is done on- or off-chain. Using standard compression techniques outlined in BIP-340 [bip], advertisement size can be reduced to 66 MB by encoding 64 bytes elliptic curve points using 33 bytes. In practice, the size of $\mathsf{ad} = (\mathsf{IPFE.mpk}, \mathsf{IPFE.ct})$ can be amortized to 33 MB by reusing the same IPFE.mpk for all advertisements by a seller. The multi-message security of IPFE scheme should preserve FAS security and result in optimal amortized advertisement size of 1.03x the witness size.

[16] for example, ad can be published on a website.

Table 3.2: Computational efficiency of FAS.

Params		Running Times (seconds)					
ℓ	B	$\mathsf{FPreSign}_1$	$\mathsf{FPreVerify}_1$	$\mathsf{FPreSign}_2$	$\mathsf{FPreVerify}_2$	Adapt	FExt
1	10^6	0.003	0.003	0.008	0.008	0.013	0.082
10^2	10^6	0.003	0.300	0.013	0.418	0.021	0.142
10^2	10^8	0.003	0.332	0.014	0.477	0.023	0.722
10^4	10^8	0.011	0.323	0.010	0.424	0.035	1.025
10^4	10^{10}	0.011	0.369	0.009	0.352	0.022	9.952
10^5	10^{11}	0.092	2.067	0.009	2.101	0.111	30.38
10^6	10^{12}	0.879	19.08	0.008	18.41	0.871	95.07
10^7	10^{13}	9.191	223.4	0.011	217.9	10.22	317.8
$3 \cdot 10^7$	$3 \cdot 10^{13}$	32.03	766.6	0.011	740.4	40.27	452.2

Computational efficiency. We share the performance numbers in Table 3.2. Here, ℓ denotes the length of the witness vector. Each entry of the vector is 32 Bytes integer. Thus, $\ell = 10^6$ implies that the witness is of size 32MB. We note that the pre-dominant cost in the implementation is the group exponentiation operation which hasn't been optimized here. We note that for parameter ℓ, the number of group exponentiation operations in each algorithm are as follows: in $\mathsf{FPreSign}_1$, $\ell + 1$ in $\mathsf{FPreVerify}_1$, 1 in $\mathsf{FPreSign}_2$, $\ell + 2$ in $\mathsf{FPreVerify}_2$, 0 in Adapt, $\ell + 2$ in FExt. Asymptotically, $\mathsf{FPreVerify}_1$ and $\mathsf{FPreVerify}_2$ run in time $O(\ell)$ and FExt runs in time $O(\sqrt{\ell})$. But, concretely FExt is the slowest for practical scenarios as highlighted in Table 3.2. The last two rows of Table 3.2 do not highlight practical scenarios as the costs of these algorithms are higher than 100 seconds each, but we benchmark them to understand at what point do the concrete running times of $\mathsf{FPreVerify}_1$ and $\mathsf{FPreVerify}_2$ start becoming a bottleneck.

3.10 Proofs for Generic Construction of FAS

Here we present the formal proofs for the generic FAS construction from Section 3.6.

3.10.1 Correctness

First, we give the proof of correctness.

Proof of Lemma 3.6.1. For every $\lambda \in \mathbb{N}$, every message $m \in \{0,1\}^*$, every statement/witness pair $(X, \mathbf{x}) \in R$, and every function $\mathbf{y} \in \mathcal{F}_{\mathsf{IP},\ell}$, suppose $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda), (\mathsf{ad}, \mathsf{st}) \leftarrow \mathsf{AdGen}(\mathsf{pp}, X, \mathbf{x}), (\mathsf{sk}, \mathsf{vk}) \leftarrow \mathsf{KGen}(1^\lambda), (\mathsf{aux}_\mathbf{y}, \pi_\mathbf{y}) \leftarrow \mathsf{FPreSign}_1(\mathsf{ad}, \mathsf{st}, \mathbf{y}), \widetilde{\sigma} \leftarrow \mathsf{FPreSign}_2(\mathsf{ad}, \mathsf{sk}, m, X, \mathbf{y}, \mathsf{aux}_\mathbf{y}), \sigma := \mathsf{Adapt}(\mathsf{ad}, \mathsf{st}, \mathsf{vk}, m, X, \mathbf{x}, \mathbf{y}, \mathsf{aux}_\mathbf{y}\widetilde{\sigma}), z := \mathsf{FExt}(\mathsf{ad}, \widetilde{\sigma}, \sigma, X, \mathbf{y}, \mathsf{aux}_\mathbf{y})$. Then,

- From NIZK correctness, it follows that $\mathsf{AdVerify}(\mathsf{pp}, X, \mathsf{ad}) = 1$.
- For determinism of IPFE.PubKGen, it follows that $\mathsf{FPreVerify}_1(\mathsf{ad}, \mathbf{y}, \mathsf{aux}_\mathbf{y}, \pi_\mathbf{y}) = 1$.

- From correctness of AS, it follows that $\mathsf{FPreVerify}_2(\mathsf{ad}, \mathsf{vk}, m, X, \mathbf{y}, \mathsf{aux}_{\mathbf{y}}, \pi_{\mathbf{y}}, \widetilde{\sigma}) = 1$.
- Recall that $\widetilde{\sigma}$ is an AS pre-signature w.r.t. the statement $\mathsf{pk}_{\mathbf{y}}$. Further, recall that $\mathsf{sk}_{\mathbf{y}}$ is the witness computed by the Adapt algorithm and used by AS.Adapt to compute σ. From R_{IPFE}-compliance of IPFE, it follows that $(\mathsf{pk}_{\mathbf{y}}, \mathsf{sk}_{\mathbf{y}}) \in R_{\mathsf{IPFE}}$. Hence, by the correctness of AS, it follows that $\mathsf{Vf}(\mathsf{vk}, m, \sigma) = 1$.
- From AS correctness, it follows that $(\mathsf{pk}_{\mathbf{y}}, z) \in R'_{\mathsf{IPFE}}$, where z is the extracted witness in the FExt algorithm. Then, from R'_{IPFE}-robustness of IPFE, it follows that $v = f_{\mathbf{y}}(\mathbf{x})$, where v is output of FExt.

□

In the subsequent sub-sections, we prove the lemmas regarding advertisement soundness, unforgeability, pre-signature adaptability, witness extractability, zero-knowledge.

3.10.2 Advertisement soundness

Lemma 3.10.1. *Suppose NIZK satisfies adaptive soundness. Then, the functional adaptor signature construction in Figure 3.2 is advertisement sound.*

Proof. We show that if a p.p.t. adversary $\mathcal{A}$ breaks the advertisement soundness of our functional adaptor signature construction with non-negligible advantage, then, there exists a p.p.t. reduction $\mathcal{B}$ that can break the adaptive soundness of the underlying NIZK scheme with the same non-negligible advantage. Suppose the challenger for adaptive soundness of NIZK is $\mathcal{C}$. The reduction $\mathcal{B}$ is as follows.

- The reduction $\mathcal{B}$ obtains crs from the challenger $\mathcal{C}$, computes $\mathsf{pp}' \leftarrow \mathsf{IPFE.Gen}(1^\lambda)$, and sends $\mathsf{pp} = (\mathsf{crs}, \mathsf{pp}')$ to $\mathcal{A}$.
- The adversary $\mathcal{A}$ sends public advertisement ad and a statement X to the reduction $\mathcal{B}$. $\mathcal{B}$ parses $\mathsf{ad} = (\mathsf{mpk}, \mathsf{ct}, \pi)$ and sends the statement $(X, \mathsf{pp}, \mathsf{mpk}, \mathsf{ct})$ and proof π to the challenger $\mathcal{C}$.

Observe that if $\mathcal{A}$ breaks advertisement soundness, then, $X \notin L_R$ and $\mathsf{AdVerify}(\mathsf{pp}, X, \mathsf{ad}) = 1$. From the definition of L_{NIZK} it follows that if $X \notin L_R$, then, $(X, \mathsf{pp}, \mathsf{mpk}, \mathsf{ct}) \notin L_{\mathsf{NIZK}}$. Further, recall that the implementation of AdVerify simply involves a NIZK proof verification. Therefore, if $\mathsf{AdVerify}(\mathsf{pp}, X, \mathsf{ad}) = 1$, then, $\mathsf{NIZK.Vf}(\mathsf{crs}, (X, \mathsf{pp}, \mathsf{mpk}, \mathsf{ct}), \pi) = 1$. Hence, whenever $\mathcal{A}$ breaks advertisement soundness of FAS, $\mathcal{B}$ breaks adaptive soundness of NIZK.

□

3.10.3 Pre-Signature Validity

Lemma 3.10.2. *Suppose the adaptor signature scheme satisfies correctness (Definition 3.4.6). Then, the functional adaptor signature construction in Figure 3.2 satisfies pre-signature validity.*

Proof. For any $\lambda \in \mathbb{N}$, any $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$ as computed in Figure 3.2, any X, ad such that $\mathsf{AdVerify}(\mathsf{pp}, \mathsf{ad}, X) = 1$ as computed in Figure 3.2, any message $m \in \{0,1\}^*$, any function $\mathbf{y} \in \mathcal{F}_{\mathsf{IP}}, \ell$, any $(\mathsf{aux}_{\mathbf{y}}, \pi_{\mathbf{y}})$, any key pair $(\mathsf{sk}, \mathsf{vk}) \leftarrow \mathsf{KGen}(1^\lambda)$, any pre-signature $\widetilde{\sigma} \leftarrow \mathsf{FPreSign}_2(\mathsf{ad}, \mathsf{sk}, m, X, \mathbf{y}, \mathsf{aux}_{\mathbf{y}})$ as computed in Figure 3.2, suppose $\mathsf{FPreVerify}_1(\mathsf{ad}, \mathbf{y}, \mathsf{aux}_{\mathbf{y}}, \pi_{\mathbf{y}}) = 1$. Then, to prove that $\Pr[\mathsf{FPreVerify}_2(\mathsf{ad}, \mathsf{vk}, m, X, f, \mathsf{aux}_f, \pi_f, \widetilde{\sigma})] = 1$ (as computed in Figure 3.2), it suffices to show that $\Pr[\mathsf{AS.PreVerify}(\mathsf{vk}, m, \mathsf{aux}_{\mathbf{y}}, \widetilde{\sigma})] = 1$. This follows from the correctness of the adaptor signature scheme AS. □

3.10.4 Unforgeability

Lemma 3.10.3. *Suppose the NIZK argument system* NIZK *satisfies adaptive soundness ((Definition 3.4.26)), the Adaptor Signature scheme* AS *satisfies witness extractability (Definition 3.4.7), the relation R is a $\mathcal{F}_{\mathsf{IP},\ell}$-hard (Definition 3.4.2), and* IPFE *scheme satisfies correctness and R'_{IPFE}-robustness (Definition 3.4.13). Then, the functional adaptor signature instantiation above is* faEUF-CMA*-secure.*

Proof. We prove the lemma by defining a sequence of games.

Game G_0: It is the original game $\mathsf{faEUF\text{-}CMA}_{\mathcal{A},\mathsf{FAS}}$, where the adversary $\mathcal{A}$ has to come up with a valid forgery on a message m^* of his choice, while having access to functional pre-sign oracle $\mathcal{O}_{\mathsf{fpS}}$ and sign oracle $\mathcal{O}_S$. Game G_0 is formally defined in Figure 3.19.

Game G_1: same as game G_0, except that before computing the pre-signature, the game checks if the NIZK statement $(X^*, \mathsf{pp}', \mathsf{mpk}, \mathsf{ct})$ is in the language L_{NIZK}. If no, the game sets the flag $\mathsf{Bad}_1 = \mathsf{true}$. Game G_1 is formally defined in Figure 3.19.

Game G_2: same as game G_1, except that when $\mathcal{A}$ outputs the forgery σ^*, the game extracts the witness z of the underlying adaptor signature scheme for the statement $\mathsf{pk}_{\mathbf{y}^*}$ and checks if the NIZK statement $(\mathsf{aux}^*_{\mathbf{y}}, z)$ satisfies R'_{IPFE}. If no, the game sets the flag $\mathsf{Bad}_2 = \mathsf{true}$. Game G_2 is formally defined in Figure 3.19.

To prove the lemma, we need to show that

$$\Pr[G_0(1^\lambda) = 1] \leq \mathsf{negl}(\lambda).$$

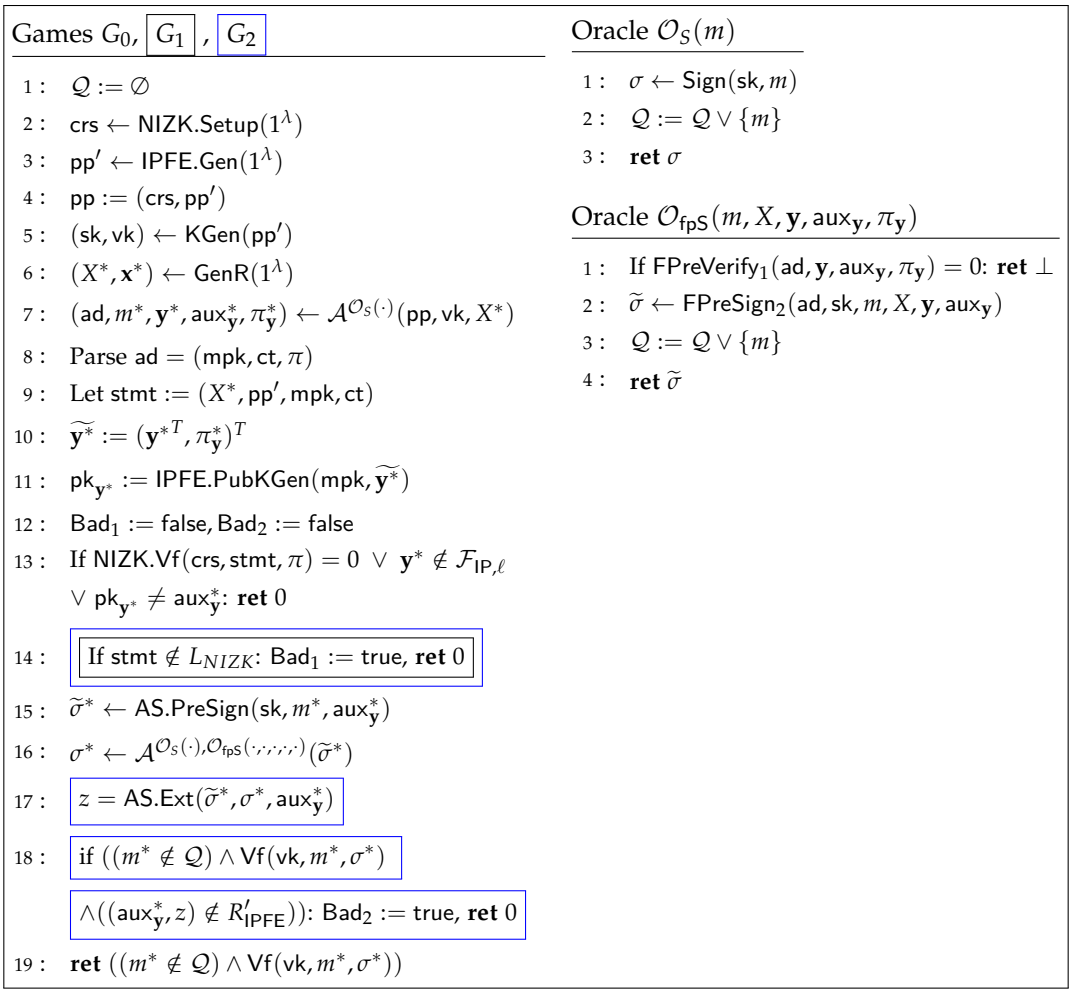


Figure 3.19: Unforgeability Proof: Games G_0, G_1, G_2

Note that by triangle inequality, it follows that

$$\begin{aligned}\Pr[G_0(1^\lambda) = 1] \leq\ & |\Pr[G_0(1^\lambda) = 1] - \Pr[G_1(1^\lambda) = 1]| \\ & + |\Pr[G_1(1^\lambda) = 1] - \Pr[G_2(1^\lambda) = 1]| \\ & + \Pr[G_2(1^\lambda) = 1].\end{aligned}$$

To complete the proof, we show in Claims 3.10.4 to 3.10.6 that each of the three terms on the right-hand-side are at most $\mathsf{negl}(\lambda)$. □

Claim 3.10.4. *If the NIZK argument system* NIZK *satisfies adaptive soundness (Definition 3.4.26), then,* $|\Pr[G_0(1^\lambda) = 1] - \Pr[G_1(1^\lambda) = 1]| \leq \mathsf{negl}(\lambda)$.

Proof. Observe that in game G_0, $\mathsf{Bad}_1 = \mathsf{false}$ always and game G_1 differs from it when the flag $\mathsf{Bad}_1 = \mathsf{true}$

is set. Both games are identical until G_1 checks the condition for setting Bad_1. Hence,

$$|\Pr[G_0(1^\lambda) = 1] - \Pr[G_1(1^\lambda) = 1]| \leq \Pr[\mathsf{Bad}_1 \text{ in } G_1].$$

Hence, it suffices to show that in game G_1, $\Pr[\mathsf{Bad}_1] \leq \mathsf{negl}(\lambda)$.

We prove the claim using a reduction to the adaptive soundness of the underlying NIZK scheme. More specifically, we show that if the adversary $\mathcal{A}$ causes game G_1 to set $\mathsf{Bad}_1 = \mathsf{true}$, then, we can use it to construct a reduction $\mathcal{B}$ that breaks the adaptive soundness of NIZK. Let $\mathcal{C}$ be the challenger for the NIZK adaptive soundness. Then, the reduction is as in Figure 3.20.

Reduction $\mathcal{B}$ for proof of Claim 3.10.4	Oracle $\mathcal{O}_S(m)$
1 : $\mathcal{Q} := \emptyset$	1 : $\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$
2 : $\mathsf{crs} \leftarrow \mathcal{C}(1^\lambda)$	2 : $\mathcal{Q} := \mathcal{Q} \vee \{m\}$
3 : $\mathsf{pp}' \leftarrow \mathsf{IPFE.Gen}(1^\lambda)$	3 : **ret** σ
4 : $\mathsf{pp} = (\mathsf{crs}, \mathsf{pp}')$	
5 : $(\mathsf{sk}, \mathsf{vk}) \leftarrow \mathsf{KGen}(\mathsf{pp}')$	
6 : $(X^*, \mathbf{x}^*) \leftarrow \mathsf{GenR}(1^\lambda)$	
7 : $(\mathsf{ad}, m^*, \mathbf{y}^*, \mathsf{aux}^*_{\mathbf{y}}, \pi^*_{\mathbf{y}}) \leftarrow \mathcal{A}^{\mathcal{O}_S(\cdot)}(\mathsf{pp}, \mathsf{vk}, X^*)$	
8 : Parse $\mathsf{ad} = (\mathsf{mpk}, \mathsf{ct}, \pi)$, let $\mathsf{stmt} := (X^*, \mathsf{pp}', \mathsf{mpk}, \mathsf{ct})$	
9 : $\widetilde{\mathbf{y}^*} := (\mathbf{y}^{*T}, \pi^*_{\mathbf{y}})^T$	
10 : $\mathsf{pk}_{\mathbf{y}^*} := \mathsf{IPFE.PubKGen}(\mathsf{mpk}, \widetilde{\mathbf{y}^*})$	
11 : $\mathsf{Bad}_1 = \mathsf{false}$	
12 : If $\mathsf{NIZK.Vf}(\mathsf{crs}, \mathsf{stmt}, \pi) = 0 \ \vee \ \mathbf{y}^* \notin \mathcal{F}_{\mathsf{IP},\ell} \ \vee \ \mathsf{pk}_{\mathbf{y}^*} \neq \mathsf{aux}^*_{\mathbf{y}}$: **abort**	
13 : If $\mathsf{stmt} \notin L_{NIZK}$: $\mathsf{Bad}_1 = \mathsf{true}$	
14 : **ret** (stmt, π) to $\mathcal{C}$	

Figure 3.20: Reduction $\mathcal{B}$ for proof of Claim 3.10.4

Observe that if $\mathcal{A}$ causes game G_1 to not abort, then, it must be the case that π is a valid proof for $\mathsf{stmt} = (X^*, \mathsf{pp}', \mathsf{mpk}, \mathsf{ct})$. Further, if $\mathcal{A}$ causes game G_1 to set $\mathsf{Bad}_1 = \mathsf{true}$, then, it must be the case that $\mathsf{stmt} \notin L_{NIZK}$. This means that in step 13, the reduction $\mathcal{B}$ successfully returns a valid proof π to $\mathcal{C}$ for a statement stmt not in the language L_{NIZK} and thus breaks the adaptive soundness of the NIZK. Thus, $\Pr[\mathsf{Bad}_1]$ is same as the probability that $\mathcal{B}$ breaks adaptive soundness of the NIZK. Hence, we can conclude that $\Pr[\mathsf{Bad}_1] \leq \mathsf{negl}(\lambda)$. □

Claim 3.10.5. *If the Adaptor Signature scheme* AS *satisfies witness extractability (Definition 3.4.7), then,* $|\Pr[G_1(1^\lambda) = 1] - \Pr[G_2(1^\lambda) = 1]| \leq \mathsf{negl}(\lambda)$.

Proof. Observe that in game G_1, $\mathsf{Bad}_2 = \mathsf{false}$ always and game G_2 differs from it when the flag $\mathsf{Bad}_2 = \mathsf{true}$

is set. Both games are identical until G_2 checks the condition for setting Bad_2. Hence,

$$|\Pr[G_1(1^\lambda) = 1] - \Pr[G_2(1^\lambda) = 1]| \leq \Pr[\mathsf{Bad}_2 \text{ in } G_2].$$

Then, it suffices to show that in game G_2, $\Pr[\mathsf{Bad}_2] \leq \mathsf{negl}(\lambda)$.

We prove the claim using a reduction to the witness extractability of the underlying AS scheme. More specifically, we show that if the adversary $\mathcal{A}$ causes Bad_2 in game G_2, then, we can use it to construct a reduction $\mathcal{B}$ that breaks the witness extractability of AS. For the AS witness extractability game, let $\mathcal{C}$ be the challenger and let $\mathsf{AS}.\mathcal{O}_S, \mathsf{AS}.\mathcal{O}_{pS}$ be the signing and pre-signing oracles that the reduction $\mathcal{B}$ has access to. Then, the reduction is as in Figure 3.21.

Reduction $\mathcal{B}$ for proof of Claim 3.10.5

1 : $\mathcal{Q} := \emptyset$
2 : $\mathsf{crs} \leftarrow \mathsf{NIZK.Setup}(1^\lambda)$
3 : $(\mathsf{pp}', \mathsf{vk}) \leftarrow \mathcal{C}(1^\lambda)$
4 : $\mathsf{pp} = (\mathsf{crs}, \mathsf{pp}')$
5 : $(X^*, \mathbf{x}^*) \leftarrow \mathsf{GenR}(1^\lambda)$
6 : $(\mathsf{ad}, m^*, \mathbf{y}^*, \mathsf{aux}^*_{\mathbf{y}}, \pi^*_{\mathbf{y}}) \leftarrow \mathcal{A}^{\mathcal{O}_S(\cdot)}(\mathsf{pp}, \mathsf{vk}, X^*)$
7 : Parse $\mathsf{ad} = (\mathsf{mpk}, \mathsf{ct}, \pi)$
8 : Let $\mathsf{stmt} := (X^*, \mathsf{pp}', \mathsf{mpk}, \mathsf{ct})$
9 : $\widetilde{\mathbf{y}^*} := (\mathbf{y}^{*T}, \pi^*_{\mathbf{y}})^T$
10 : $\mathsf{pk}_{\mathbf{y}^*} := \mathsf{IPFE.PubKGen}(\mathsf{mpk}, \widetilde{\mathbf{y}^*})$
11 : $\mathsf{Bad}_1 := \mathsf{false}, \mathsf{Bad}_2 := \mathsf{false}$
12 : If $\mathsf{NIZK.Vf}(\mathsf{crs}, \mathsf{stmt}, \pi) = 0 \ \vee \ \mathbf{y}^* \notin \mathcal{F}_{\mathsf{IP},\ell}$ $\vee \ \mathsf{pk}_{\mathbf{y}^*} \neq \mathsf{aux}^*_{\mathbf{y}}$: **abort**
13 : If $\mathsf{stmt} \notin L_{NIZK}$: $\mathsf{Bad}_1 := \mathsf{true}$, **abort**
14 : $\widetilde{\sigma}^* \leftarrow \mathcal{C}(m^*, \mathsf{aux}^*_{\mathbf{y}})$
15 : $\sigma^* \leftarrow \mathcal{A}^{\mathcal{O}_S(\cdot), \mathcal{O}_{\mathsf{fpS}}(\cdot,\cdot,\cdot,\cdot,\cdot)}(\widetilde{\sigma}^*)$
16 : $z = \mathsf{AS.Ext}(\widetilde{\sigma}^*, \sigma^*, \mathsf{aux}^*_{\mathbf{y}})$
17 : If $(m^* \notin \mathcal{Q}) \wedge \mathsf{Vf}(\mathsf{vk}, m^*, \sigma^*) \wedge ((\mathsf{aux}^*_{\mathbf{y}}, z) \notin R'_{\mathsf{IPFE}})$: $\mathsf{Bad}_2 = \mathsf{true}$, **ret** σ^* to $\mathcal{C}$
18 : Else: **abort**

Oracle $\mathcal{O}_S(m)$

1 : $\sigma \leftarrow \mathsf{AS}.\mathcal{O}_S(m)$
2 : $\mathcal{Q} := \mathcal{Q} \vee \{m\}$
3 : **ret** σ

Oracle $\mathcal{O}_{\mathsf{fpS}}(m, X, \mathbf{y}, \mathsf{aux}_{\mathbf{y}}, \pi_{\mathbf{y}})$

1 : If $\mathsf{FPreVerify}_1(\mathsf{ad}, \mathbf{y}, \mathsf{aux}_{\mathbf{y}}, \pi_{\mathbf{y}}) = 0$: **ret** $\perp$
2 : $\widetilde{\sigma} \leftarrow \mathsf{AS}.\mathcal{O}_{pS}(m, \mathsf{aux}_{\mathbf{y}})$
3 : $\mathcal{Q} := \mathcal{Q} \vee \{m\}$
4 : **ret** $\widetilde{\sigma}$

Figure 3.21: Reduction $\mathcal{B}$ for proof of Claim 3.10.5

Observe that the reduction $\mathcal{B}$ perfectly simulates the game G_2 to the adversary $\mathcal{A}$ as it appropriately forwards the signing and functional pre-signing queries to its the signing and pre-signing oracles of AS. Thus, to complete the proof it suffices to argue that if $\mathcal{B}$ returns σ^* to $\mathcal{C}$ in step 17, then, it breaks the witness extractability of AS. Suppose that the query set maintained by $\mathcal{C}$ is denoted by $\mathsf{AS}.\mathcal{Q}$. Recall then that to break the witness extractability of AS, the signature σ^* on message m^* must satisfy $(m^* \notin \mathsf{AS}.\mathcal{Q}) \wedge ((\mathsf{aux}^*_{\mathbf{y}}, z) \notin R'_{\mathsf{IPFE}}) \wedge \mathsf{Vf}(\mathsf{vk}, m^*, \sigma^*)$, where $z = \mathsf{AS.Ext}(\widetilde{\sigma}^*, \sigma^*, \mathsf{aux}^*_{\mathbf{y}})$. The last of the three conditions

holds because the same is present in the pre-condition for returning σ^* to $\mathcal{C}$. Hence, it remains to argue that the first two conditions also hold true. We note that $m^* \notin \mathcal{Q}$ implies $m^* \notin \mathsf{AS}.\mathcal{Q}$ as from the description of the reduction $\mathcal{B}$, it follows that $\mathcal{Q} = \mathsf{AS}.\mathcal{Q}$. Next, $(\mathsf{aux}^*_{\mathbf{y}}, z) \notin R'_{\mathsf{IPFE}}$ because $\mathsf{Bad}_2 = \mathsf{true}$. Therefore, if the adversary $\mathcal{A}$ causes Event_2 in game G_2, then, $\mathcal{B}$ that breaks the witness extractability of AS. Hence, it follows that $\Pr[\mathsf{Event}_2] \leq \mathsf{negl}(\lambda)$. □

Claim 3.10.6. *Suppose relation R is a $\mathcal{F}_{\mathsf{IP},\ell}$-hard (Definition 3.4.2), IPFE scheme satisfies R'_{IPFE}-robustness (Definition 3.4.13). Then, $\Pr[G_2(1^\lambda) = 1] \leq \mathsf{negl}(\lambda)$.*

Reduction $\mathcal{B}$ for proof of Claim 3.10.6

$\mathcal{Q} := \emptyset$
$\mathsf{crs} \leftarrow \mathsf{NIZK.Setup}(1^\lambda)$
$\mathsf{pp}' \leftarrow \mathsf{IPFE.Gen}(1^\lambda)$
$\mathsf{pp} := (\mathsf{crs}, \mathsf{pp}')$
$(\mathsf{sk}, \mathsf{vk}) \leftarrow \mathsf{KGen}(\mathsf{pp}')$
$X^* \leftarrow \mathcal{C}(1^\lambda)$
$(\mathsf{ad}, m^*, \mathbf{y}^*, \mathsf{aux}^*_{\mathbf{y}}, \pi^*_{\mathbf{y}}) \leftarrow \mathcal{A}^{\mathcal{O}_S(\cdot)}(\mathsf{pp}, \mathsf{vk}, X^*)$
Parse $\mathsf{ad} = (\mathsf{mpk}, \mathsf{ct}, \pi)$, let $\mathsf{stmt} := (X^*, \mathsf{pp}', \mathsf{mpk}, \mathsf{ct})$
$\widetilde{\mathbf{y}^*} := (\mathbf{y}^{*T}, \pi^*_{\mathbf{y}})^T$
$\mathsf{pk}_{\mathbf{y}^*} := \mathsf{IPFE.PubKGen}(\mathsf{mpk}, \widetilde{\mathbf{y}^*})$
$\mathsf{Bad}_1 := \mathsf{false}, \mathsf{Bad}_2 := \mathsf{false}$
If $\mathsf{NIZK.Vf}(\mathsf{crs}, \mathsf{stmt}, \pi) = 0 \ \vee \ \mathbf{y}^* \notin \mathcal{F}_{\mathsf{IP},\ell}$
$\vee \ \mathsf{pk}_{\mathbf{y}^*} \neq \mathsf{aux}^*_{\mathbf{y}}$: **abort**
If $\mathsf{stmt} \notin L_{NIZK}$: $\mathsf{Bad}_1 := \mathsf{true}$, **abort**
$\widetilde{\sigma}^* \leftarrow \mathsf{AS.PreSign}(\mathsf{sk}, m^*, \mathsf{aux}^*_{\mathbf{y}})$
$\sigma^* \leftarrow \mathcal{A}^{\mathcal{O}_S(\cdot), \mathcal{O}_{\mathsf{fpS}}(\cdot,\cdot,\cdot,\cdot,\cdot)}(\widetilde{\sigma}^*)$
$z = \mathsf{AS.Ext}(\widetilde{\sigma}^*, \sigma^*, \mathsf{aux}^*_{\mathbf{y}})$
If $(m^* \notin \mathcal{Q}) \wedge \mathsf{Vf}(\mathsf{vk}, m^*, \sigma^*) \wedge ((\mathsf{aux}^*_{\mathbf{y}}, z) \notin R'_{\mathsf{IPFE}})$:
  $\mathsf{Bad}_2 = \mathsf{true}$, **abort**
$v = \mathsf{IPFE.Dec}(z, \mathsf{ct})$
If $((m^* \notin \mathcal{Q}) \wedge \mathsf{Vf}(\mathsf{vk}, m^*, \sigma^*))$:
  ret $(\mathbf{y}^*, v)$
Else: abort game with $\mathcal{C}$

Oracle $\mathcal{O}_S(m)$

$\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$
$\mathcal{Q} := \mathcal{Q} \vee \{m\}$
ret σ

Oracle $\mathcal{O}_{\mathsf{fpS}}(m, X, \mathbf{y}, \mathsf{aux}_{\mathbf{y}}, \pi_{\mathbf{y}})$

If $\mathsf{FPreVerify}_1(\mathsf{ad}, \mathbf{y}, \mathsf{aux}_{\mathbf{y}}, \pi_{\mathbf{y}}) = 0$: **ret** $\perp$
$\widetilde{\sigma} \leftarrow \mathsf{FPreSign}_2(\mathsf{ad}, \mathsf{sk}, m, X, \mathbf{y}, \mathsf{aux}_{\mathbf{y}})$
$\mathcal{Q} := \mathcal{Q} \vee \{m\}$
ret $\widetilde{\sigma}$

Figure 3.22: Reduction $\mathcal{B}$ for proof of Claim 3.10.6

Proof. We prove the claim using a reduction to the $\mathcal{F}_{\mathsf{IP},\ell}$-hardness of the relation R. More specifically, we show that if an adversary $\mathcal{A}$ causes game G_2 to return 1, then, we can use it to construct a reduction $\mathcal{B}$ that breaks the $\mathcal{F}_{\mathsf{IP},\ell}$-hardness of the relation R. Let $\mathcal{C}$ be the challenger for the $\mathcal{F}_{\mathsf{IP},\ell}$-hardness of the relation R. The reduction $\mathcal{B}$ is as in Figure 3.22.

Observe that the reduction $\mathcal{B}$ perfectly simulates game G_2 to the adversary $\mathcal{A}$ as it obtains X^* from the challenger $\mathcal{C}$ who computes it as $(X^*, \cdot) \leftarrow \mathsf{GenR}(1^\lambda)$. Then, if $\mathcal{A}$ causes G_2 to return 1, it must be the case that the if condition in step 19 of the reduction must be true. In such a case, the reduction returns $(\mathbf{y}^*, v)$ to the challenger $\mathcal{C}$. To complete the proof, it remains to show that this results in $\mathcal{B}$ winning the game against $\mathcal{C}$.

To win the game against $\mathcal{C}$, the reduction $\mathcal{B}$'s output must satisfy $(\mathbf{y}^* \in \mathcal{F}_{\mathsf{IP},\ell}) \wedge (v \in \{f_{\mathbf{y}^*}(\mathbf{x}) : \exists \mathbf{x}\ s.t.\ (X, \mathbf{x}) \in R\})$. Note that since the reduction did not abort, it implies $\mathbf{y}^* \in \mathcal{F}_{\mathsf{IP},\ell}$ and $\mathsf{pk}_{\mathbf{y}^*} = \mathsf{aux}^*_{\mathbf{y}}$. Next, $\mathsf{Bad}_1 = \mathsf{false}$ implies that $\mathsf{stmt} \in L_{NIZK}$, where $\mathsf{stmt} = (X^*, \mathsf{pp}', \mathsf{mpk}, \mathsf{ct})$. Hence, it follows that ct encrypts some vector $\widetilde{\mathbf{x}} = (\mathbf{x}^T, 0)^T \in \mathcal{M}' \subseteq \mathbb{Z}^{\ell+1}$ under mpk such that $(X^*, \mathbf{x}) \in R$, where $(\mathsf{mpk}, \mathsf{msk}) \leftarrow \mathsf{IPFE.Setup}(\mathsf{pp}', 1^{\ell+1})$. Next, $\mathsf{Bad}_2 = \mathsf{false}$ implies that $(\mathsf{aux}^*_{\mathbf{y}}, z) \in R'_{\mathsf{IPFE}}$. As $\mathsf{pk}_{\mathbf{y}^*} = \mathsf{aux}^*_{\mathbf{y}}$ and IPFE satisfies R'_{IPFE}-robustness, it follows that $v = f_{\widetilde{\mathbf{y}^*}}(\widetilde{\mathbf{x}})$. As $\widetilde{\mathbf{y}^*} = ({\mathbf{y}^*}^T, \pi^*_{\mathbf{y}})^T$ and $\widetilde{\mathbf{x}} = (\mathbf{x}^T, 0)^T$, we get that $f_{\widetilde{\mathbf{y}^*}}(\widetilde{\mathbf{x}}) = f_{\mathbf{y}^*}(\mathbf{x})$. Hence, we conclude that $v \in \{f_{\mathbf{y}^*}(\mathbf{x}) : \exists \mathbf{x}\ s.t.\ (X, \mathbf{x}) \in R\}$. This completes the proof.

□

3.10.5 Witness Extractability

Lemma 3.10.7. *Suppose* AS *satisfies witness extractability,* NIZK *satisfies adaptive soundness, and* IPFE *satisfies* R'_{IPFE}*-robustness. Then, the functional adaptor signature construction in Figure 3.2 is* faWitExt*-secure.*

Proof. We prove the lemma by defining a sequence of games.

Game G_0**:** It is the original game $\mathsf{faWitExt}_{\mathcal{A},\mathsf{FAS}}$, where the adversary $\mathcal{A}$ who is given access to a pre-signature on a message, must come up with a full signature such that it does not reveal the function evaluation on a witness. The adversary also has access to functional pre-sign oracle $\mathcal{O}_{\mathsf{fpS}}$ and sign oracle $\mathcal{O}_S$. Game G_0 is formally defined in Figure 3.23.

Game G_1**:** same as game G_0, except that when $\mathcal{A}$ outputs a public advertisement ad, the game checks if the NIZK statement $(X^*, \mathsf{mpk}, \mathsf{ct})$ is in the language L_{NIZK}. If no, the game sets the flag $\mathsf{Bad}_1 = \mathsf{true}$. Game G_1 is formally defined in Figure 3.23.

To prove the lemma, we need to show that

$$\Pr[G_0(1^\lambda) = 1] \leq \mathsf{negl}(\lambda).$$

Note that by triangle inequality, it follows that

$$\Pr[G_0(1^\lambda) = 1] \leq |\Pr[G_0(1^\lambda) = 1] - \Pr[G_1(1^\lambda) = 1]| + \Pr[G_1(1^\lambda) = 1].$$

Games G_0, $\boxed{G_1}$

$\mathcal{Q} := \emptyset$
$\mathsf{crs} \leftarrow \mathsf{NIZK.Setup}(1^\lambda)$
$\mathsf{pp}' \leftarrow \mathsf{IPFE.Gen}(1^\lambda)$
$\mathsf{pp} := (\mathsf{crs}, \mathsf{pp}')$
$(\mathsf{sk}, \mathsf{vk}) \leftarrow \mathsf{KGen}(\mathsf{pp}')$
$(X^*, \mathsf{ad}, m^*, \mathbf{y}^*, \mathsf{aux}^*_{\mathbf{y}}, \pi^*_{\mathbf{y}}) \leftarrow \mathcal{A}^{\mathcal{O}_S(\cdot)}(\mathsf{pp}, \mathsf{vk})$
Parse $\mathsf{ad} = (\mathsf{mpk}, \mathsf{ct}, \pi)$, let $\mathsf{stmt} := (X^*, \mathsf{pp}', \mathsf{mpk}, \mathsf{ct})$
$\widetilde{\mathbf{y}^*} := ({\mathbf{y}^*}^T, \pi^*_{\mathbf{y}})^T$
$\mathsf{pk}_{\mathbf{y}^*} := \mathsf{IPFE.PubKGen}(\mathsf{mpk}, \widetilde{\mathbf{y}^*})$
$\mathsf{Bad}_0 := \mathsf{false}, \mathsf{Bad}_1 := \mathsf{false}, \mathsf{Bad}_2 := \mathsf{false}$
If $\mathsf{NIZK.Vf}(\mathsf{crs}, \mathsf{stmt}, \pi) = 0 \vee \mathbf{y}^* \notin \mathcal{F}_{\mathsf{IP},\ell} \vee \mathsf{pk}_{\mathbf{y}^*} \neq \mathsf{aux}^*_{\mathbf{y}}$:
  $\mathsf{Bad}_0 := \mathsf{true}$
$\boxed{\text{If } \mathsf{stmt} \notin L_{\mathsf{NIZK}}\text{: } \mathsf{Bad}_1 := \mathsf{true}}$
$\widetilde{\sigma}^* \leftarrow \mathsf{AS.PreSign}(\mathsf{sk}, m^*, \mathsf{aux}^*_{\mathbf{y}})$
$\sigma^* \leftarrow \mathcal{A}^{\mathcal{O}_S(\cdot), \mathcal{O}_{\mathsf{fpS}}(\cdot,\cdot,\cdot,\cdot,\cdot)}(\widetilde{\sigma}^*)$
$z := \mathsf{AS.Ext}(\widetilde{\sigma}^*, \sigma^*, \mathsf{aux}^*_{\mathbf{y}})$
$v := \mathsf{IPFE.Dec}(z, \mathsf{ct})$
If $v \in \{f_{\mathbf{y}^*}(\mathbf{x}) : \exists\ \mathbf{x}\ s.t.\ (X^*, \mathbf{x}) \in R\}$: $\mathsf{Bad}_2 = \mathsf{true}$
ret $((m^* \notin \mathcal{Q}) \wedge \mathsf{Vf}(\mathsf{vk}, m^*, \sigma^*) \wedge \neg\mathsf{Bad}_0 \wedge \neg\mathsf{Bad}_1 \wedge \neg\mathsf{Bad}_2)$

Oracle $\mathcal{O}_S(m)$

$\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$
$\mathcal{Q} := \mathcal{Q} \vee \{m\}$
ret σ

Oracle $\mathcal{O}_{\mathsf{fpS}}(m, X, \mathbf{y}, \mathsf{aux}_{\mathbf{y}}, \pi_{\mathbf{y}})$

If $\mathsf{FPreVerify}_1(\mathsf{ad}, \mathbf{y}, \mathsf{aux}_{\mathbf{y}}, \pi_{\mathbf{y}}) = 0$: **ret** $\perp$
$\widetilde{\sigma} \leftarrow \mathsf{FPreSign}_2(\mathsf{ad}, \mathsf{sk}, m, X, \mathbf{y}, \mathsf{aux}_{\mathbf{y}})$
$\mathcal{Q} := \mathcal{Q} \vee \{m\}$
ret $\widetilde{\sigma}$

Figure 3.23: Witness Extractability proof: Games G_0 and G_1

To complete the proof, we show in Claims 3.10.8 and 3.10.9 that each of the two terms on the right-hand-side are at most $\mathsf{negl}(\lambda)$. □

Claim 3.10.8. *If the NIZK argument system* NIZK *satisfies adaptive soundness, then,* $|\Pr[G_0(1^\lambda) = 1] - \Pr[G_1(1^\lambda) = 1]| \leq \mathsf{negl}(\lambda)$.

Proof. Similar to proof of Claim 3.10.4. □

Claim 3.10.9. *Suppose the Adaptor Signature scheme* AS *satisfies witness extractability and the IPFE scheme* IPFE *satisfies* R'_{IPFE}*-robustness. Then,* $\Pr[G_1(1^\lambda) = 1] \leq \mathsf{negl}(\lambda)$.

Proof. To prove $\Pr[G_1(1^\lambda) = 1] \leq \mathsf{negl}(\lambda)$, we show that if there exists a p.p.t. adversary $\mathcal{A}$ such that it wins game G_1 with non-negligible advantage, then, we can construct a p.p.t. reduction $\mathcal{B}$ that breaks the witness extractability of the underlying adaptor signature scheme AS. For the AS witness extractability game, let $\mathcal{C}$ be the challenger and let $\mathsf{AS}.\mathcal{O}_S, \mathsf{AS}.\mathcal{O}_{pS}$ be the signing and pre-signing oracles that the reduction $\mathcal{B}$ has access to. Then, the reduction is as in Figure 3.24.

Reduction B for proof of Claim 3.10.9

$\mathcal{Q} := \emptyset$
$\mathsf{crs} \leftarrow \mathsf{NIZK.Setup}(1^\lambda)$
$(\mathsf{pp}', \mathsf{vk}) \leftarrow \mathcal{C}(1^\lambda)$
$\mathsf{pp} := (\mathsf{crs}, \mathsf{pp}')$
$(X^*, \mathsf{ad}, m^*, \mathbf{y}^*, \mathsf{aux}^*_{\mathbf{y}}, \pi^*_{\mathbf{y}}) \leftarrow \mathcal{A}^{\mathcal{O}_S(\cdot)}(\mathsf{pp}, \mathsf{vk})$
Parse $\mathsf{ad} = (\mathsf{mpk}, \mathsf{ct}, \pi)$, let $\mathsf{stmt} := (X^*, \mathsf{pp}', \mathsf{mpk}, \mathsf{ct})$
$\widetilde{\mathbf{y}^*} := (\mathbf{y}^{*T}, \pi^*_{\mathbf{y}})^T$
$\mathsf{pk}_{\mathbf{y}^*} := \mathsf{IPFE.PubKGen}(\mathsf{mpk}, \widetilde{\mathbf{y}^*})$
$\mathsf{Bad}_0 := \mathsf{false}, \mathsf{Bad}_1 := \mathsf{false}, \mathsf{Bad}_2 := \mathsf{false}$
If $\mathsf{NIZK.Vf}(\mathsf{crs}, \mathsf{stmt}, \pi) = 0 \vee \mathbf{y}^* \notin \mathcal{F}_{\mathsf{IP},\ell} \vee \mathsf{pk}_{\mathbf{y}^*} \neq \mathsf{aux}^*_{\mathbf{y}}$: $\mathsf{Bad}_0 := \mathsf{true}$
If $\mathsf{stmt} \notin L_{\mathsf{NIZK}}$: $\mathsf{Bad}_1 = \mathsf{true}$
$\widetilde{\sigma}^* \leftarrow \mathcal{C}(m^*, \mathsf{aux}^*_{\mathbf{y}})$
$\sigma^* \leftarrow \mathcal{A}^{\mathcal{O}_S(\cdot), \mathcal{O}_{\mathsf{fpS}}(\cdot,\cdot,\cdot,\cdot,\cdot)}(\widetilde{\sigma}^*)$
$z := \mathsf{AS.Ext}(\widetilde{\sigma}^*, \sigma^*, \mathsf{aux}^*_{\mathbf{y}})$
$v := \mathsf{IPFE.Dec}(z, \mathsf{ct})$
If $v \in \{f_{\mathbf{y}^*}(\mathbf{x}) : \exists\ \mathbf{x}\ s.t.\ (X^*, \mathbf{x}) \in R\}$: $\mathsf{Bad}_2 = \mathsf{true}$
If $((m^* \notin \mathcal{Q}) \wedge \mathsf{Vf}(\mathsf{vk}, m^*, \sigma^*) \wedge \neg\mathsf{Bad}_0 \wedge \neg\mathsf{Bad}_1 \wedge \neg\mathsf{Bad}_2)$: **ret** σ^*
Else: abort game with $\mathcal{C}$

Oracle $\mathcal{O}_S(m)$

$\sigma \leftarrow \mathsf{AS}.\mathcal{O}_S(m)$
$\mathcal{Q} := \mathcal{Q} \vee \{m\}$
ret σ

Oracle $\mathcal{O}_{\mathsf{fpS}}(m, X, f)$

If $\mathsf{ad} = \perp$: **ret** $\perp$
$\mathsf{pk}_{\mathbf{y}} = \mathsf{IPFE.PubKGen}(\mathsf{mpk}, \mathbf{y})$
$\widetilde{\sigma} \leftarrow \mathsf{AS}.\mathcal{O}_{pS}(m, \mathsf{pk}_{\mathbf{y}})$
$\mathcal{Q} := \mathcal{Q} \vee \{m\}$
ret $\widetilde{\sigma}$

Figure 3.24: Reduction B for proof of Claim 3.10.9

From the description of reduction $\mathcal{B}$, we can observe that whenever the challenger $\mathcal{C}$ obtains the signature σ^* on message m^* obtained, it is the case that $((m^* \notin \mathcal{Q}) \wedge \mathsf{Vf}(\mathsf{vk}, m^*, \sigma^*) \wedge \neg\mathsf{Bad}_0 \wedge \neg\mathsf{Bad}_1 \wedge \neg\mathsf{Bad}_2)$. Suppose that the query set maintained by $\mathcal{C}$ is denoted by $\mathsf{AS}.\mathcal{Q}$. For $\mathcal{B}$ to succeed, the signature σ^* must satisfy $((m^* \notin \mathsf{AS}.\mathcal{Q}) \wedge \mathsf{Vf}(\mathsf{vk}, m^*, \sigma^*) \wedge ((\mathsf{pk}_{\mathbf{y}^*}, z) \notin R'_{\mathsf{IPFE}}))$. We note that $m^* \notin \mathcal{Q}$ implies $m^* \notin \mathsf{AS}.\mathcal{Q}$ as from the description of the reduction $\mathcal{B}$, it follows that $\mathcal{Q} = \mathsf{AS}.\mathcal{Q}$. Note that σ^* already satisfies the second condition and it suffices to show that if $\neg\mathsf{Bad}_0 \wedge \neg\mathsf{Bad}_1 \wedge \neg\mathsf{Bad}_2$, then, $(\mathsf{pk}_{\mathbf{y}^*}, z) \notin R'_{\mathsf{IPFE}}$.

Note that $\mathsf{Bad}_0 = \mathsf{false}$ implies $\mathsf{NIZK.Vf}(\mathsf{crs}, \mathsf{stmt}, \pi) = 1$ and $\mathbf{y}^* \in \mathcal{F}_{\mathsf{IP},\ell}$ and $\mathsf{aux}^*_{\mathbf{y}} = \mathsf{pk}_{\mathbf{y}^*}$. Next, $\mathsf{Bad}_1 = \mathsf{false}$ implies that $\mathsf{stmt} \in L_{\mathsf{NIZK}}$, where $\mathsf{stmt} = (X^*, \mathsf{pp}', \mathsf{mpk}, \mathsf{ct})$. Hence, it follows that ct encrypts some vector $\widetilde{\mathbf{x}^*} = (\mathbf{x}^{*T}, 0)^T \in \mathcal{M}' \subseteq \mathbb{Z}^{\ell+1}$ under mpk such that $(X^*, \mathbf{x}^*) \in R$, where $(\mathsf{mpk}, \mathsf{msk}) \leftarrow \mathsf{IPFE.Setup}(\mathsf{pp}', 1^\ell)$. Next, $\mathsf{Bad}_2 = \mathsf{false}$ implies that $v \notin \{f_{\mathbf{y}^*}(\mathbf{x}) : \exists\ \mathbf{x}\ s.t.\ (X^*, \mathbf{x}) \in R\}$, where v is computed as $v := \mathsf{IPFE.Dec}(z, \mathsf{ct})$. This implies $v \neq f_{\mathbf{y}^*}(\mathbf{x}^*)$. Note that for $\widetilde{\mathbf{y}^*} = (\mathbf{y}^{*T}, \pi^*_{\mathbf{y}})^T$, we have $f_{\widetilde{\mathbf{y}^*}}(\widetilde{\mathbf{x}^*}) = f_{\mathbf{y}^*}(\mathbf{x}^*)$. Hence, it follows that $v \neq f_{\widetilde{\mathbf{y}^*}}(\widetilde{\mathbf{x}^*})$.

To complete the proof, we argue that if $v \neq f_{\widetilde{\mathbf{y}^*}}(\widetilde{\mathbf{x}^*})$, then, $(\mathsf{pk}_{\mathbf{y}^*}, z) \notin R'_{\mathsf{IPFE}}$. This follows from R'_{IPFE}-robustness of IPFE. In particular, the contra-positive form of R'_{IPFE}-robustness says that if $v \neq f_{\widetilde{\mathbf{y}^*}}(\widetilde{\mathbf{x}^*})$, then either pp′ is not computed honestly, or mpk is not computed honestly or ct is not computed honestly

or $\mathsf{pk}_{\mathbf{y}^*}$ is not computed honestly or v is not computed honestly or $(\mathsf{pk}_{\mathbf{y}^*}, z) \notin R'_{\mathsf{IPFE}}$. Observe that the challenger $\mathcal{C}$ samples pp' honestly, the NIZK proof π attests that mpk and ct are computed honestly, the reduction $\mathcal{B}$ computes $\mathsf{pk}_{\mathbf{y}^*}$ and v honestly. Thus, it must be the case that $(\mathsf{pk}_{\mathbf{y}^*}, z) \notin R'_{\mathsf{IPFE}}$.

□

3.10.6 Pre-Signature Adaptability

Lemma 3.10.10. *Suppose* IPFE *satisfies* R_{IPFE}*-compliance (Definition 3.4.12) and suppose that* AS *satisfies weak pre-signature adaptability (Definition 3.4.8). Then, the functional adaptor signature construction in Figure 3.2 is pre-signature adaptable (Definition 3.5.10).*

Proof. For any $\lambda \in \mathbb{N}$, let $\mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda)$ be as computed in Figure 3.2. For any statement/witness pair $(X, \mathbf{x}) \in R$, let $(\mathsf{ad}, \mathsf{st}) \leftarrow \mathsf{AdGen}(1^\ell, \mathsf{crs}, X, \mathbf{x})$ be as computed in Figure 3.2. For any message $m \in \{0,1\}^*$, any function $\mathbf{y} \in \mathcal{F}_{IP,\mathbb{Z}_p^\ell}$, any $(\mathsf{aux}_{\mathbf{y}}, \pi_{\mathbf{y}}) \leftarrow \mathsf{FPreSign}_1(\mathsf{ad}, \mathsf{st}, \mathbf{y})$ as computed in Figure 3.2, any key pair $(\mathsf{sk}, \mathsf{vk}) \leftarrow \mathsf{KGen}(1^\lambda)$ as computed in Figure 3.2, any pre-signature $\widetilde{\sigma} \in \{0,1\}^*$ such that $\mathsf{FPreVerify}_2(\mathsf{ad}, \mathsf{vk}, m, X, \mathbf{y}, \mathsf{aux}_{\mathbf{y}}, \pi_{\mathbf{y}}, \widetilde{\sigma}) = 1$ as computed in Figure 3.2. This implies $\mathsf{AS.PreVerify}(\mathsf{vk}, m, \mathsf{aux}_{\mathbf{y}}, \widetilde{\sigma}) = 1$. By R_{IPFE}-compliance of the IPFE scheme, we know that $(\mathsf{aux}_{\mathbf{y}}, \mathsf{sk}_{\mathbf{y}}) \in R_{\mathsf{IPFE}}$, where $\mathsf{sk}_{\mathbf{y}}$ is as computed in the Adapt algorithm in Figure 3.2. Then, it follows by the weak pre-signature adaptability of AS that $\Pr[\mathsf{AS.Vf}(\mathsf{vk}, m, \sigma) = 1] = 1$, where $\sigma = \mathsf{AS.Adapt}(\mathsf{vk}, m, \mathsf{aux}_{\mathbf{y}}, \mathsf{sk}_{\mathbf{y}}, \widetilde{\sigma})$. Then, from the implementation of the Adapt algorithm of our functional adaptor signature scheme as in Figure 3.2, it follows that

$$\Pr[\mathsf{Vf}(\mathsf{vk}, m, \mathsf{Adapt}(\mathsf{ad}, \mathsf{st}, \mathsf{vk}, m, X, \mathbf{x}, \mathbf{y}, \mathsf{aux}_{\mathbf{y}}, \widetilde{\sigma})) = 1] = 1.$$

□

3.10.7 Zero-Knowledge

We prove lemmas regarding the non-abort and abort variants of zero-knowledge for the generic FE-based FAS construction.

Lemma 3.10.11. *Suppose* $\mathcal{M}$ *is an additive group,* NIZK *satisfies zero-knowledge (Definition 3.4.26) and* IPFE *satisfies selective, IND-security (Definition 3.4.10). Then, the functional adaptor signature construction in Figure 3.2 satisfies zero-knowledge (non-abort) as in Definition 3.5.13.*

Proof. To prove the lemma, we need to show that for every stateful p.p.t. adversary $\mathcal{A}$, there exists a stateful p.p.t. simulator $\mathsf{Sim} = (\mathsf{Setup}^*, \mathsf{AdGen}^*, \mathsf{FPreSign}_1{}^*, \mathsf{Adapt}^*)$ and there exists a negligible function negl such

that for all p.p.t. distinguishers $\mathcal{D}$, for all $\lambda \in \mathbb{N}$, for all $(X, \mathbf{x}) \in R$,

$$|\Pr[\mathcal{D}(\mathsf{faZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, X, \mathbf{x})) = 1] - \Pr[\mathcal{D}(\mathsf{faZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, X, \mathbf{x})) = 1]| \leq \mathsf{negl}(\lambda).$$

We first describe the stateful simulator $\mathsf{Sim} = (\mathsf{Setup}^*, \mathsf{AdGen}^*, \mathsf{FPreSign}_1{}^*, \mathsf{Adapt}^*)$. Let $\mathsf{NIZK.Sim} = (\mathsf{NIZK.Setup}^*, \mathsf{NIZK.Prove}^*)$ be the NIZK simulator. Then, the simulator Sim is as in Figure 3.25.

$\mathsf{Setup}^*(1^\lambda)$

1 : Let $(\mathsf{crs}, \mathsf{td}) \leftarrow \mathsf{NIZK.Setup}^*(1^\lambda)$
2 : Sample $\mathsf{pp}' \leftarrow \mathsf{IPFE.Gen}(1^\lambda)$
3 : Store internal state td, **ret** $\mathsf{pp} := (\mathsf{crs}, \mathsf{pp}')$

$\mathsf{AdGen}^*(\mathsf{pp}, X)$

1 : Sample $(\mathsf{mpk}, \mathsf{msk}) \leftarrow \mathsf{IPFE.Setup}(\mathsf{pp}', 1^{\ell+1})$
2 : Sample $\mathbf{t} \xleftarrow{\$} \mathcal{M}$
3 : Let $\widetilde{\mathbf{x}} := (-\mathbf{t}^T, 1)^T \in \mathcal{M}' \subseteq \mathbb{Z}^{\ell+1}$
4 : Let $\mathsf{ct} \leftarrow \mathsf{IPFE.Enc}(\mathsf{mpk}, \widetilde{\mathbf{x}})$
5 : Let $\pi \leftarrow \mathsf{NIZK.Prove}^*(\mathsf{crs}, \mathsf{td}, (X, \mathsf{pp}', \mathsf{mpk}, \mathsf{ct}))$
6 : Store internal state $\mathsf{st} := (\mathsf{msk}, \mathbf{t})$
7 : **ret** $\mathsf{ad} := (\mathsf{mpk}, \mathsf{ct}, \pi)$

$\mathsf{FPreSign}_1{}^*(\mathsf{ad}, \mathbf{y}, f_{\mathbf{y}}(\mathbf{x}))$

1 : Parse $\mathsf{ad} = (\mathsf{mpk}, \mathsf{ct}, \pi)$
2 : Let $\widetilde{\mathbf{y}} := (\mathbf{y}^T, f_{\mathbf{y}}(\mathbf{t}) + f_{\mathbf{y}}(\mathbf{x}))^T \in \mathcal{F}_{\mathsf{IP},\ell+1}$
3 : Let $\mathsf{pk}_{\mathbf{y}} := \mathsf{IPFE.PubKGen}(\mathsf{mpk}, \widetilde{\mathbf{y}})$
4 : **ret** $\mathsf{aux}_{\mathbf{y}} := \mathsf{pk}_{\mathbf{y}}$, $\pi_{\mathbf{y}} := f_{\mathbf{y}}(\mathbf{t}) + f_{\mathbf{y}}(\mathbf{x})$

$\mathsf{Adapt}^*(\mathsf{ad}, \mathsf{vk}, m, X, \mathbf{y}, \mathsf{aux}_{\mathbf{y}}, \widetilde{\sigma}, f_{\mathbf{y}}(\mathbf{x}))$

1 : Parse $\mathsf{ad} = (\mathsf{mpk}, \mathsf{ct}, \pi)$, and $\mathsf{st} = (\mathsf{msk}, \mathbf{t})$
2 : Let $\widetilde{\mathbf{y}} := (\mathbf{y}^T, f_{\mathbf{y}}(\mathbf{t}) + f_{\mathbf{y}}(\mathbf{x}))^T \in \mathcal{F}_{\mathsf{IP},\ell+1}$
3 : Let $\mathsf{sk}_{\mathbf{y}} := \mathsf{IPFE.KGen}(\mathsf{msk}, \widetilde{\mathbf{y}})$
4 : **ret** $\sigma := \mathsf{AS.Adapt}(\mathsf{vk}, m, \mathsf{aux}_{\mathbf{y}}, \mathsf{sk}_{\mathbf{y}}, \widetilde{\sigma})$

Figure 3.25: Zero-Knowledge Simulator Sim

Having described the Simulator Sim, the experiments $\mathsf{faZKReal}_{\mathcal{A},\mathsf{FAS}}$ and $\mathsf{faZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$ are as in Figure 3.26. Observe that $\mathsf{faZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$ is same as $\mathsf{faZKReal}_{\mathcal{A},\mathsf{FAS}}$ except that NIZK is switched to simulation mode and IPFE ciphertext ct encrypts $\widetilde{\mathbf{x}} := (-\mathbf{t}^T, 1)^T \in \mathcal{M}' \subseteq \mathbb{Z}^{\ell+1}$ and functional keys $\mathsf{pk}_{\mathbf{y}}$ and $\mathsf{sk}_{\mathbf{y}}$ correspond to $\widetilde{\mathbf{y}} := (\mathbf{y}^T, f_{\mathbf{y}}(\mathbf{t}) + f_{\mathbf{y}}(\mathbf{x}))^T \in \mathcal{F}_{\mathsf{IP},\ell+1}$. Observe that $f_{\mathbf{y}}(\mathbf{x}) = f_{\widetilde{\mathbf{y}}}(\widetilde{\mathbf{x}})$. To prove zero-knowledge, we introduce an intermediate hybrid experiment $\mathsf{Hyb}^{\mathsf{NIZK.Sim}}_{\mathcal{A},\mathsf{FAS}}$ which is same as $\mathsf{faZKReal}_{\mathcal{A},\mathsf{FAS}}$ except that only NIZK is switched to simulation mode. Note that by triangle inequality, it follows that

$$\begin{aligned}
&|\Pr[\mathcal{D}(\mathsf{faZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, X, \mathbf{x})) = 1] - \Pr[\mathcal{D}(\mathsf{faZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, X, \mathbf{x})) = 1]| \\
&\leq |\Pr[\mathcal{D}(\mathsf{faZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, X, \mathbf{x})) = 1] - \Pr[\mathcal{D}(\mathsf{Hyb}^{\mathsf{NIZK.Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, X, \mathbf{x})) = 1]| \\
&\quad + |\Pr[\mathcal{D}(\mathsf{Hyb}^{\mathsf{NIZK.Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, X, \mathbf{x})) = 1] - \Pr[\mathcal{D}(\mathsf{faZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, X, \mathbf{x})) = 1]|.
\end{aligned}$$

To complete the proof, we show in Claims 3.10.12 and 3.10.13 that each of the two terms on the right-hand-side are at most $\mathsf{negl}(\lambda)$. □

Claim 3.10.12. *Suppose* NIZK *satisfies zero-knowledge. Then, for every stateful p.p.t. adversary $\mathcal{A}$, for the p.p.t. simulator* NIZK.Sim, *there exists a negligible function* negl *such that for all p.p.t. distinguishers $\mathcal{D}$, for all $\lambda \in \mathbb{N}$, for all $(X, \mathbf{x}) \in R$,*

$$|\Pr[\mathcal{D}(\mathsf{faZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, X, \mathbf{x})) = 1] - \Pr[\mathcal{D}(\mathsf{Hyb}^{\mathsf{NIZK.Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, X, \mathbf{x})) = 1]| \leq \mathsf{negl}(\lambda).$$

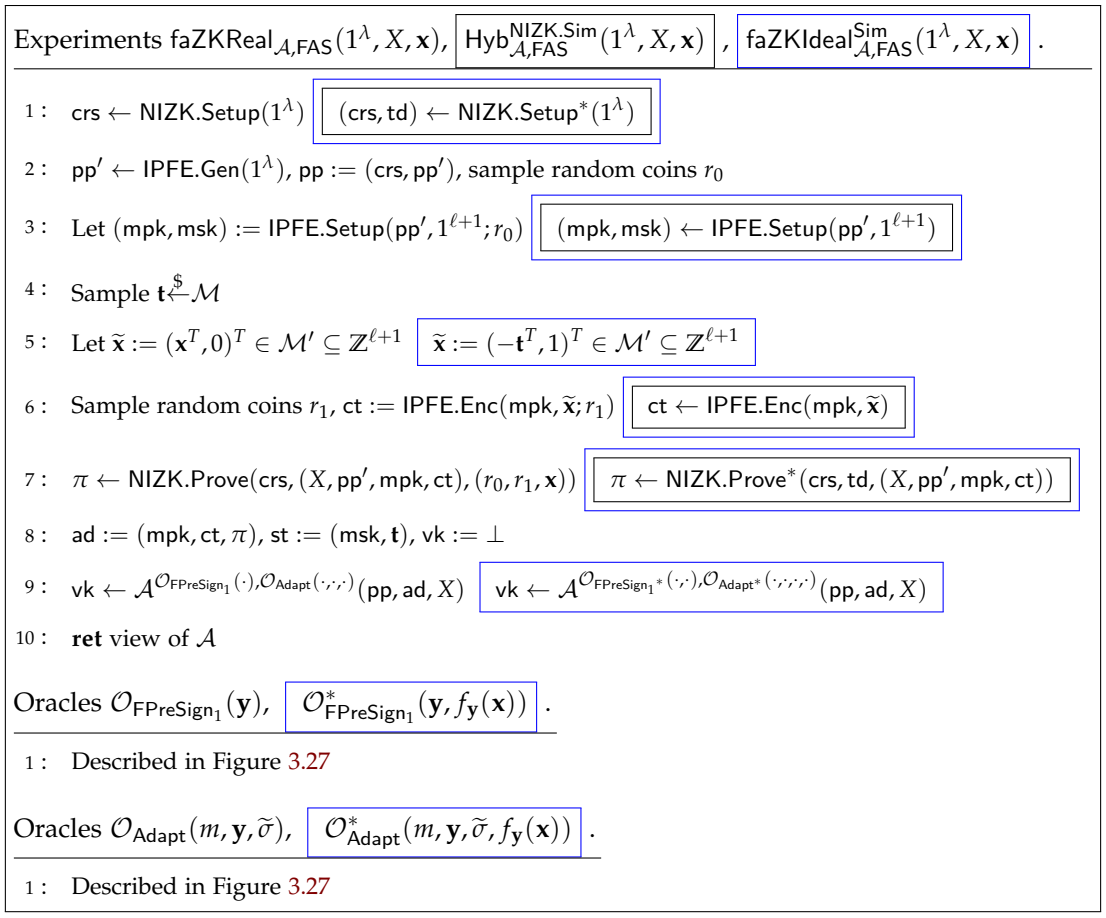


Figure 3.26: Zero-knowledge security of FAS: $\mathsf{faZKReal}_{\mathcal{A},\mathsf{FAS}}$ is the real world experiment, $\mathsf{Hyb}^{\mathsf{NIZK.Sim}}_{\mathcal{A},\mathsf{FAS}}$ is an intermediate hybrid experiment, $\mathsf{faZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$ is the ideal world experiment.

Proof. Suppose towards a contradiction that for the stateful p.p.t. adversary $\mathcal{A}$ that makes no queries to the oracles $\mathcal{O}_{\mathsf{FPreSign}_1}$ and $\mathcal{O}_{\mathsf{Adapt}}$, there exists a p.p.t. distinguisher $\mathcal{D}$ and a non-negligible value ϵ such that

$$|\Pr[\mathcal{D}(\mathsf{faZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, X, \mathbf{x})) = 1] - \Pr[\mathcal{D}(\mathsf{Hyb}^{\mathsf{NIZK.Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, X, \mathbf{x})) = 1]| = \epsilon.$$

Then, we build a p.p.t. distinguisher $\mathcal{D}_{\mathsf{NIZK}}$ using $\mathcal{D}$ which breaks the zero-knowledge of NIZK for the NP language L_{NIZK}. This should complete the proof.

Suppose the inputs to the distinguisher D_{NIZK} are the NIZK common reference string crs and a NIZK proof π for some statement $(X, \mathsf{pp}', \mathsf{mpk}, \mathsf{ct}) \in L_{\mathsf{NIZK}}$. Then, $\mathcal{D}_{\mathsf{NIZK}}$ is as follows. It sets $\mathsf{pp} = (\mathsf{crs}, \mathsf{pp}')$ and $\mathsf{ad} = (\mathsf{mpk}, \mathsf{ct}, \pi)$. As $\mathcal{A}$ makes no oracle queries, the view of the adversary $\mathcal{A}$ against the the zero-knowledge security of functional adaptor signatures is simply $(\mathsf{pp}, \mathsf{ad}, X)$. Then, $\mathcal{D}_{\mathsf{NIZK}}$ runs the

Oracles $\mathcal{O}_{\mathsf{FPreSign}_1}(\mathbf{y})$, $\boxed{\mathcal{O}^*_{\mathsf{FPreSign}_1}(\mathbf{y}, f_{\mathbf{y}}(\mathbf{x}))}$.

1 : Parse $\mathsf{ad} = (\mathsf{mpk}, \mathsf{ct}, \pi)$

2 : Compute $\widetilde{\mathbf{y}} := (\mathbf{y}^T, f_{\mathbf{y}}(\mathbf{t}))^T \in \mathcal{F}_{\mathsf{IP},\ell+1}$ $\boxed{\text{Compute } \widetilde{\mathbf{y}} := (\mathbf{y}^T, f_{\mathbf{y}}(\mathbf{t}) + f_{\mathbf{y}}(\mathbf{x}))^T \in \mathcal{F}_{\mathsf{IP},\ell+1}}$

3 : Compute $\mathsf{pk}_{\mathbf{y}} := \mathsf{IPFE.PubKGen}(\mathsf{mpk}, \widetilde{\mathbf{y}})$

4 : **ret** $\mathsf{aux}_{\mathbf{y}} := \mathsf{pk}_{\mathbf{y}}$, $\pi_{\mathbf{y}} := f_{\mathbf{y}}(\mathbf{t})$ $\boxed{\textbf{ret } \mathsf{aux}_{\mathbf{y}} := \mathsf{pk}_{\mathbf{y}}, \pi_{\mathbf{y}} := f_{\mathbf{y}}(\mathbf{t}) + f_{\mathbf{y}}(\mathbf{x})}$

Oracles $\mathcal{O}_{\mathsf{Adapt}}(m, \mathbf{y}, \widetilde{\sigma})$, $\boxed{\mathcal{O}^*_{\mathsf{Adapt}}(m, \mathbf{y}, \widetilde{\sigma}, f_{\mathbf{y}}(\mathbf{x}))}$.

1 : Parse $\mathsf{ad} = (\mathsf{mpk}, \mathsf{ct}, \pi)$, and $\mathsf{st} = (\mathsf{msk}, \mathbf{t})$. If $\mathsf{vk} = \perp$: **ret** $\perp$

2 : $(\mathsf{aux}_{\mathbf{y}}, \pi_{\mathbf{y}}) \leftarrow \mathcal{O}_{\mathsf{FPreSign}_1}(\mathbf{y})$ $\boxed{(\mathsf{aux}_{\mathbf{y}}, \pi_{\mathbf{y}}) \leftarrow \mathcal{O}^*_{\mathsf{FPreSign}_1}(\mathbf{y}, f_{\mathbf{y}}(\mathbf{x}))}$

3 : If $\mathsf{FPreVerify}_1(\mathsf{ad}, \mathbf{y}, \mathsf{aux}_{\mathbf{y}}, \pi_{\mathbf{y}}) = 0 \ \vee \ \mathsf{AS.PreVerify}(\mathsf{vk}, m, \mathsf{aux}_{\mathbf{y}}, \widetilde{\sigma}) = 0$: **ret** $\perp$

4 : Compute $\widetilde{\mathbf{y}} := (\mathbf{y}^T, f_{\mathbf{y}}(\mathbf{t}))^T \in \mathcal{F}_{\mathsf{IP},\ell+1}$ $\boxed{\text{Compute } \widetilde{\mathbf{y}} := (\mathbf{y}^T, f_{\mathbf{y}}(\mathbf{t}) + f_{\mathbf{y}}(\mathbf{x}))^T \in \mathcal{F}_{\mathsf{IP},\ell+1}}$

5 : Compute $\mathsf{sk}_{\mathbf{y}} := \mathsf{IPFE.KGen}(\mathsf{msk}, \widetilde{\mathbf{y}})$

6 : **ret** $\sigma := \mathsf{AS.Adapt}(\mathsf{vk}, m, \mathsf{aux}_{\mathbf{y}}, \mathsf{sk}_{\mathbf{y}}, \widetilde{\sigma})$

Figure 3.27: Oracle descriptions for experiments $\mathsf{faZKReal}_{\mathcal{A},\mathsf{FAS}}$, $\mathsf{Hyb}^{\mathsf{NIZK.Sim}}_{\mathcal{A},\mathsf{FAS}}$, $\mathsf{faZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$.

distinguisher $\mathcal{D}$ on inputs $(\mathsf{pp}, \mathsf{ad}, X)$ and returns whatever $\mathcal{D}$ returns. Then, observe that

$$\begin{aligned}
&\Pr\left[\begin{array}{c} \mathcal{D}_{\mathsf{NIZK}}(\mathsf{crs}, \pi) = 1 : \\ \mathsf{crs} \leftarrow \mathsf{NIZK.Setup}(1^\lambda), \\ \pi \leftarrow \mathsf{NIZK.Prove}(\mathsf{pp}, (X, \mathsf{pp}', \mathsf{mpk}, \mathsf{ct}), (r_0, r_1, \mathbf{x})) \end{array}\right] \\
&= \Pr\left[\begin{array}{c} \mathcal{D}(\mathsf{pp} = (\mathsf{crs}, \mathsf{pp}'), \mathsf{ad} = (\mathsf{mpk}, \mathsf{ct}, \pi), X) = 1 : \\ \mathsf{crs} \leftarrow \mathsf{NIZK.Setup}(1^\lambda), \\ \pi \leftarrow \mathsf{NIZK.Prove}(\mathsf{pp}, (X, \mathsf{pp}', \mathsf{mpk}, \mathsf{ct}), (r_0, r_1, \mathbf{x})) \end{array}\right] \\
&= \Pr\left[\mathcal{D}(\mathsf{faZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, X, \mathbf{x})) = 1\right].
\end{aligned}$$

Similarly, we get that

$$\begin{aligned}
&\Pr\left[\begin{array}{c} \mathcal{D}_{\mathsf{NIZK}}(\mathsf{crs}, \pi) = 1 : \\ (\mathsf{crs}, \mathsf{td}) \leftarrow \mathsf{NIZK.Setup}^*(1^\lambda) \\ \pi \leftarrow \mathsf{NIZK.Prove}^*(\mathsf{crs}, \mathsf{td}, \mathsf{stmt} = (X, \mathsf{pp}', \mathsf{mpk}, \mathsf{ct})) \end{array}\right] \\
&= \Pr\left[\begin{array}{c} \mathcal{D}(\mathsf{pp} = (\mathsf{crs}, \mathsf{pp}'), \mathsf{ad} = (\mathsf{mpk}, \mathsf{ct}, \pi), X) = 1 : \\ (\mathsf{crs}, \mathsf{td}) \leftarrow \mathsf{NIZK.Setup}^*(1^\lambda) \\ \pi \leftarrow \mathsf{NIZK.Prove}^*(\mathsf{crs}, \mathsf{td}, \mathsf{stmt} = (X, \mathsf{pp}', \mathsf{mpk}, \mathsf{ct})) \end{array}\right] \\
&= \Pr\left[\mathcal{D}(\mathsf{Hyb}^{\mathsf{NIZK.Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, X, \mathbf{x})) = 1\right].
\end{aligned}$$

This implies that

$$\begin{aligned}
&\left| \Pr\left[\begin{array}{c} \mathcal{D}_{\mathsf{NIZK}}(\mathsf{crs},\pi)=1: \\ \mathsf{crs}\leftarrow \mathsf{NIZK.Setup}(1^\lambda), \\ \pi\leftarrow \mathsf{NIZK.Prove}(\mathsf{pp},(X,\mathsf{pp}',\mathsf{mpk},\mathsf{ct}),(\mathbf{x},r))\end{array}\right] \right. \\
&\left. - \Pr\left[\begin{array}{c} \mathcal{D}_{\mathsf{NIZK}}(\mathsf{crs},\pi)=1: \\ (\mathsf{crs},\pi)\leftarrow \mathsf{NIZK.Sim}(1^\lambda,\mathsf{stmt}=(X,\mathsf{pp}',\mathsf{mpk},\mathsf{ct}))\end{array}\right] \right| \\
&= \left| \begin{array}{l} \Pr\left[\mathcal{D}(\mathsf{faZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda,X,\mathbf{x}))=1\right] \\ -\Pr\left[\mathcal{D}(\mathsf{Hyb}^{\mathsf{NIZK.Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda,X,\mathbf{x}))=1\right]\end{array}\right| \\
&= \epsilon.
\end{aligned}$$

As ϵ is non-negligible, hence, $\mathcal{D}_{\mathsf{NIZK}}$ breaks the zero-knowledge of the underlying NIZK scheme.

□

Claim 3.10.13. *Suppose $\mathcal{M}$ is an additive group and* IPFE *satisfies selective, IND-security. Then, for every stateful p.p.t. adversary $\mathcal{A}$, for the p.p.t. simulators* NIZK.Sim *and* Sim *(described in Figure 3.25), there exists a negligible function* negl *such that for all p.p.t. distinguishers $\mathcal{D}$, for all $\lambda\in\mathbb{N}$, for all $(X,\mathbf{x})\in R$,*

$$|\Pr[\mathcal{D}(\mathsf{Hyb}^{\mathsf{NIZK.Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda,X,\mathbf{x}))=1]-\Pr[\mathcal{D}(\mathsf{faZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda,X,\mathbf{x}))=1]|\leq \mathsf{negl}(\lambda).$$

Proof. Observe that experiments $\mathsf{Hyb}^{\mathsf{NIZK.Sim}}_{\mathcal{A},\mathsf{FAS}}$ and $\mathsf{faZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$ differ in the following two aspects:

- The vector $\widetilde{\mathbf{x}}$ that ciphertext ct encrypts: In $\mathsf{Hyb}^{\mathsf{NIZK.Sim}}_{\mathcal{A},\mathsf{FAS}}$, we have $\widetilde{\mathbf{x}}:=(\mathbf{x}^T,0)^T\in\mathcal{M}'\subseteq\mathbb{Z}^{\ell+1}$, and in $\mathsf{faZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$, we have $\widetilde{\mathbf{x}}:=(-\mathbf{t}^T,1)^T\in\mathcal{M}'\subseteq\mathbb{Z}^{\ell+1}$.
- The vector $\widetilde{\mathbf{y}}$ used for computing $\mathsf{pk}_{\mathbf{y}}$ and $\mathsf{sk}_{\mathbf{y}}$ for every $\mathbf{y}$ queried to $\mathsf{FPreSign}_1$ and Adapt oracles: In $\mathsf{Hyb}^{\mathsf{NIZK.Sim}}_{\mathcal{A},\mathsf{FAS}}$, we have $\widetilde{\mathbf{y}}:=(\mathbf{y}^T,f_{\mathbf{y}}(\mathbf{t}))^T\in\mathcal{F}_{\mathsf{IP},\ell+1}$, and in $\mathsf{faZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$, we have $\widetilde{\mathbf{y}}:=(\mathbf{y}^T,f_{\mathbf{y}}(\mathbf{t})+f_{\mathbf{y}}(\mathbf{x}))^T\in\mathcal{F}_{\mathsf{IP},\ell+1}$.

Consider an intermediate hybrid $\overline{\mathsf{Hyb}}^{\mathsf{NIZK.Sim}}_{\mathcal{A},\mathsf{FAS}}$ that uses $\widetilde{\mathbf{x}}$ as in $\mathsf{Hyb}^{\mathsf{NIZK.Sim}}_{\mathcal{A},\mathsf{FAS}}$ and $\widetilde{\mathbf{y}}$ as in $\mathsf{faZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$.

We argue that Hyb and $\overline{\mathsf{Hyb}}$ are identically distributed. The two differ in how $\widetilde{\mathbf{y}}$ is computed. Notice that by linearity, it follows that $f_{\mathbf{y}}(\mathbf{t})+f_{\mathbf{y}}(\mathbf{x})=f_{\mathbf{y}}(\mathbf{t}+\mathbf{x})$. Hence, one can view the change from Hyb to $\overline{\mathsf{Hyb}}$ as simply a change of variables $\widetilde{\mathbf{t}}=\mathbf{t}\rightarrow\widetilde{\mathbf{t}}=\mathbf{t}+\mathbf{x}$. As $\mathcal{M}$ is an additive group and $\mathbf{t},\mathbf{x}\in\mathcal{M}$, hence, $\mathbf{t}+\mathbf{x}\in\mathcal{M}$. As $\mathbf{t}\xleftarrow{\$}\mathcal{M}$, hence, it follows that $\mathbf{t}$ and $\mathbf{t}+\mathbf{x}$ are identically distributed. Therefore, Hyb and $\overline{\mathsf{Hyb}}$ are identically distributed. In other words,

$$|\Pr[\mathcal{D}(\mathsf{Hyb}^{\mathsf{NIZK.Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda,X,\mathbf{x}))=1]-\Pr[\mathcal{D}(\overline{\mathsf{Hyb}}^{\mathsf{NIZK.Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda,X,\mathbf{x}))=1]|=0.$$

To complete the proof, we argue that if IPFE is selective, IND-secure, then,

$$|\Pr[\mathcal{D}(\overline{\mathsf{Hyb}}^{\mathsf{NIZK.Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, X, \mathbf{x})) = 1] - \Pr[\mathcal{D}(\mathsf{faZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, X, \mathbf{x})) = 1]| \leq \mathsf{negl}(\lambda).$$

Suppose towards a contradiction that for any stateful p.p.t. adversary $\mathcal{A}$, there exists a there exists a p.p.t. distinguisher $\mathcal{D}$ and a non-negligible value ϵ such that

$$|\Pr[\mathcal{D}(\overline{\mathsf{Hyb}}^{\mathsf{NIZK.Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, X, \mathbf{x})) = 1] - \Pr[\mathcal{D}(\mathsf{faZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, X, \mathbf{x})) = 1]| = \epsilon.$$

Then, we build a p.p.t. reduction $\mathcal{B}$ using $\mathcal{D}$ that breaks the selective, IND-security of IPFE. This should complete the proof.

Reduction $\mathcal{B}(1^\lambda, X, \mathbf{x})$ for proof of Claim 3.10.13.

$\mathsf{pp}' \leftarrow \mathcal{C}(1^\lambda)$
Sample $\mathbf{t} \xleftarrow{\$} \mathcal{M}$
Let $\widetilde{\mathbf{x}_0} := (\mathbf{x}^T, 0)^T \in \mathcal{M}' \subseteq \mathbb{Z}^{\ell+1}$
Let $\widetilde{\mathbf{x}_1} := (-\mathbf{t}^T, 1)^T \in \mathcal{M}' \subseteq \mathbb{Z}^{\ell+1}$
$(\mathsf{mpk}, \mathsf{ct}) \leftarrow \mathcal{C}(\widetilde{\mathbf{x}_0}, \widetilde{\mathbf{x}_1})$
$(\mathsf{crs}, \mathsf{td}) \leftarrow \mathsf{NIZK.Setup}^*(1^\lambda)$
$\mathsf{pp} = (\mathsf{crs}, \mathsf{pp}')$
$\pi \leftarrow \mathsf{NIZK.Prove}^*(\mathsf{crs}, \mathsf{td}, (X, \mathsf{pp}', \mathsf{mpk}, \mathsf{ct}))$
$\mathsf{ad} = (\mathsf{mpk}, \mathsf{ct}, \pi)$, $\mathsf{st} = \mathbf{t}$
$\mathsf{vk} = \perp$
$\mathsf{vk} \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{FPreSign}_1{}^*}(\cdot,\cdot)\mathcal{O}_{\mathsf{Adapt}^*}(\cdot,\cdot,\cdot,\cdot)}(\mathsf{pp}, \mathsf{ad}, X)$
$b \leftarrow \mathcal{D}(\text{view of } \mathcal{A})$
ret bit b to $\mathcal{C}$

Oracle $\mathcal{O}^*_{\mathsf{FPreSign}_1}(\mathbf{y}, f_{\mathbf{y}}(\mathbf{x}))$.

Parse $\mathsf{ad} = (\mathsf{mpk}, \mathsf{ct}, \pi)$ and $\mathsf{st} = \mathbf{t}$
Compute $\widetilde{\mathbf{y}} := (\mathbf{y}^T, f_{\mathbf{y}}(\mathbf{t}) + f_{\mathbf{y}}(\mathbf{x}))^T \in \mathcal{F}_{\mathsf{IP},\ell+1}$
Compute $\mathsf{pk}_{\mathbf{y}} := \mathsf{IPFE.PubKGen}(\mathsf{mpk}, \widetilde{\mathbf{y}})$
ret $\mathsf{aux}_{\mathbf{y}} := \mathsf{pk}_{\mathbf{y}}$, $\pi_{\mathbf{y}} := f_{\mathbf{y}}(\mathbf{t}) + f_{\mathbf{y}}(\mathbf{x})$

Oracle $\mathcal{O}^*_{\mathsf{Adapt}}(m, \mathbf{y}, \widetilde{\sigma}, f_{\mathbf{y}}(\mathbf{x}))$.

Parse $\mathsf{ad} = (\mathsf{mpk}, \mathsf{ct}, \pi)$, and $\mathsf{st} = \mathbf{t}$
If $\mathsf{vk} = \perp$: **ret** $\perp$
$(\mathsf{aux}_{\mathbf{y}}, \pi_{\mathbf{y}}) \leftarrow \mathcal{O}^*_{\mathsf{FPreSign}_1}(\mathbf{y}, f_{\mathbf{y}}(\mathbf{x}))$
If $\mathsf{FPreVerify}_1(\mathsf{ad}, \mathbf{y}, \mathsf{aux}_{\mathbf{y}}, \pi_{\mathbf{y}}) = 0 \vee$ $\mathsf{AS.PreVerify}(\mathsf{vk}, m, \mathsf{aux}_{\mathbf{y}}, \widetilde{\sigma}) = 0$:
  ret $\perp$
Compute $\widetilde{\mathbf{y}} := (\mathbf{y}^T, f_{\mathbf{y}}(\mathbf{t}) + f_{\mathbf{y}}(\mathbf{x}))^T \in \mathcal{F}_{\mathsf{IP},\ell+1}$
Compute $\mathsf{sk}_{\mathbf{y}} \leftarrow \mathsf{IPFE}.\mathcal{O}_{\mathsf{KGen}}(\widetilde{\mathbf{y}})$
ret $\sigma := \mathsf{AS.Adapt}(\mathsf{vk}, m, \mathsf{aux}_{\mathbf{y}}, \mathsf{sk}_{\mathbf{y}}, \widetilde{\sigma})$

Figure 3.28: Reduction $\mathcal{B}$ for proof of Claim 3.10.13

Let the IPFE challenger be $\mathcal{C}$ and let its KGen oracle be denoted be $\mathsf{IPFE}.\mathcal{O}_{\mathsf{KGen}}$. The reduction $\mathcal{B}$ is as in Figure 3.28. Observe that if the challenger $\mathcal{C}$ chooses to encrypt $\widetilde{\mathbf{x}_0}$, then, $\mathcal{A}$'s view is same as in $\overline{\mathsf{Hyb}}$ and if it chooses to encrypt $\widetilde{\mathbf{x}_1}$, then $\mathcal{A}$'s view is same as in faZKIdeal. Thus, if $\mathcal{D}$ can distinguish the two views of $\mathcal{A}$ with a non-negligible distinguishing advantage ϵ against $\mathcal{B}$, then, $\mathcal{B}$ has the same non-negligible distinguishing advantage ϵ against $\mathcal{C}$. Thus, $\mathcal{B}$ breaks the selective, IND-security of IPFE.

□

Lemma 3.10.14. *Suppose $\mathcal{M}$ is an additive group,* NIZK *satisfies zero-knowledge (Definition 3.4.26) and* IPFE *satisfies selective, IND-security (Definition 3.4.10). Then, the functional adaptor signature construction in Figure 3.2 satisfies zero-knowledge (abort) as in Definition 3.5.14.*

Proof. The proof is similar to the proof of Lemma 3.10.11, but simpler. The simulator Sim is similar to before except that there is no Adapt^* algorithm and ${\mathsf{FPreSign}_1}^*$ does not get $f_{\mathbf{y}}(\mathbf{x})$ as input. The latter algorithm runs the same as before except it simply sets $\widetilde{\mathbf{y}} := (\mathbf{y}^T, f_{\mathbf{y}}(\mathbf{t}))^T \in \mathcal{F}_{\mathsf{IP},\ell+1}$ and $\pi_{\mathbf{y}} := f_{\mathbf{y}}(\mathbf{t})$. In other words, ${\mathsf{FPreSign}_1}^*$ is same as $\mathsf{FPreSign}_1$ in the real-world. To transition from the real experiment to the ideal experiment, we first use the zero-knowledge of NIZK (as in Claim 3.10.12) to move to the hybrid experiment $\mathsf{Hyb}^{\mathsf{NIZK.Sim}}_{\mathcal{A},\mathsf{FAS}}$. Then, we use the selective, IND-security of IPFE to move to the ideal experiment $\mathsf{faZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$. In the latter transition, only $\widetilde{\mathbf{x}}$ is changed from $(\mathbf{x}^T, 0)^T$ to $(-\mathbf{t}^T, 1)^T$ and this is ok since the adversary never obtains any IPFE decryption keys (because the adversary aborts). □

3.11 Weakly-Secure FAS Construction from FE

Recall that our main FAS construction was obtained by starting with a simulation-secure IPFE scheme. The construction was presented using an IND-secure IPFE and applying the IND-secure IPFE to simulation-secure IPFE compiler of [ALMT20] in a non-black-box way within our FAS construction. Here, we show that if (i) we do not apply the compiler and just use the IND-secure IPFE, (ii) use NIZK with adaptive zero-knowledge, then, the resulting FAS construction is weakly-secure. This means that it satisfies all security properties as before except that for witness privacy, it satisfies witness indistinguishability. We first define adaptive zero-knowledge of NIZK and then describe the FAS construction next.

Definition 3.11.1 (Adaptively Secure NIZK Argument System)**.** *An adaptively secure NIZK argument system must satisfy completeness and adaptive soundness as before. Additionally, it must satisfy* adaptive zero-knowledge *which requires that there exists a p.p.t. simulator* $\mathsf{Sim} = (\mathsf{Setup}^*, \mathsf{Prove}^*)$ *and there exists a negligible function* negl *such that for all p.p.t. distinguishers* $\mathcal{D}$*, for all* $\lambda \in \mathbb{N}$*, for all* $(\mathsf{stmt}, \mathsf{wit}) \in R$*,*

$$|\Pr[\mathcal{D}(\mathsf{crs}, \pi) = 1] - \Pr[\mathcal{D}(\mathsf{crs}^*, \pi^*) = 1]| \leq \mathsf{negl}(\lambda),$$

where $\mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda)$*,* $\pi \leftarrow \mathsf{Prove}(\mathsf{crs}, \mathsf{stmt}, \mathsf{wit})$*, and* $(\mathsf{crs}^*, \mathsf{td}) \leftarrow \mathsf{Setup}^*(1^\lambda)$*,* $\pi \leftarrow \mathsf{Prove}^*(\mathsf{crs}^*, \mathsf{td}, \mathsf{stmt})$*. Here,* td *is the trapdoor for the simulated* crs^* *that is used by* Prove^* *to generate an accepting proof for* stmt *without knowing the witness.*

Our generic construction of weakly-secure functional adaptor signatures will use the following building blocks:

- An inner product functional encryption scheme IPFE that satisfies selective, IND-security (Definition 3.4.10), R_{IPFE}-compliance (Definition 3.4.12) and R'_{IPFE}-robustness (Definition 3.4.13) w.r.t. hard relations R_{IPFE}, R'_{IPFE} such that $R_{\mathsf{IPFE}} \subseteq R'_{\mathsf{IPFE}}$. The message space of IPFE is $\mathcal{M}' \subseteq \mathbb{Z}^\ell$ and the function family is $\mathcal{F}_{\mathsf{IP},\ell}$.

- An adaptor signature scheme AS w.r.t. a digital signature scheme DS, and hard relations R_{IPFE} and R'_{IPFE} that satisfies witness extractability (Definition 3.4.7) and weak pre-signature adaptability (Definition 3.4.8) security properties.

- An adaptively secure NIZK (Definition 3.11.1) for the NP language L_{NIZK} defined as

$$L_{\mathsf{NIZK}} := \left\{ (X, \mathsf{pp}', \mathsf{mpk}, \mathsf{ct}) : \begin{array}{c} \exists (r_0, r_1, \mathbf{x}) \text{ such that} \\ \mathsf{pp}' \in [\mathsf{IPFE.Gen}(1^\lambda)], \\ (\mathsf{mpk}, \mathsf{msk}) = \mathsf{IPFE.Setup}(\mathsf{pp}', 1^{\ell+1}; r_0), \\ (X, \mathbf{x}) \in R, \\ \mathsf{ct} = \mathsf{IPFE.Enc}(\mathsf{mpk}, \mathbf{x}; r_1) \end{array} \right\}.$$

Our construction is as in Figure 3.29. Observe that it uses ℓ slots instead of $\ell + 1$ slots in the underlying IPFE, thus making the construction slightly more efficient than before. Further, compared to before, $\mathsf{FPreSign}_1$ and $\mathsf{FPreVerify}_1$ are redundant, thus, can be skipped in the protocol resulting in a non-interactive pre-signing stage. The construction can be instantiated from prime-order groups and lattices as before.

$\underline{\mathsf{Setup}(1^\lambda)}$

1 : Sample $\mathsf{crs} \leftarrow \mathsf{NIZK.Setup}(1^\lambda)$
2 : Sample $\mathsf{pp}' \leftarrow \mathsf{IPFE.Gen}(1^\lambda)$
3 : **ret** $\mathsf{pp} := (\mathsf{crs}, \mathsf{pp}')$

$\underline{\mathsf{AdGen}(\mathsf{pp}, X, \mathbf{x})}$:

1 : Sample random coins r_0, r_1
2 : Let $(\mathsf{mpk}, \mathsf{msk}) := \mathsf{IPFE.Setup}(\mathsf{pp}', 1^\ell; r_0)$
3 : Let $\mathsf{ct} := \mathsf{IPFE.Enc}(\mathsf{mpk}, \mathbf{x}; r_1)$
4 : Let $\pi \leftarrow \mathsf{NIZK.Prove}(\mathsf{crs}, (X, \mathsf{pp}', \mathsf{mpk}, \mathsf{ct}), (r_0, r_1, \mathbf{x}))$
5 : **ret** $\mathsf{ad} := (\mathsf{mpk}, \mathsf{ct}, \pi)$, and $\mathsf{st} := \mathsf{msk}$

$\underline{\mathsf{AdVerify}(\mathsf{pp}, X, \mathsf{ad})}$

1 : **ret** $\mathsf{NIZK.Vf}(\mathsf{crs}, (X, \mathsf{pp}', \mathsf{mpk}, \mathsf{ct}), \pi)$

$\underline{\mathsf{FPreSign}_1(\mathsf{ad}, \mathsf{st}, \mathbf{y})}$

1 : **ret** $\mathsf{aux}_\mathbf{y} = \bot, \pi_\mathbf{y} = \bot$

$\underline{\mathsf{FPreVerify}_1(\mathsf{ad}, \mathbf{y}, \mathsf{aux}_\mathbf{y}, \pi_\mathbf{y})}$

1 : **ret** 1

$\underline{\mathsf{FPreSign}_2(\mathsf{ad}, \mathsf{sk}, m, X, \mathbf{y}, \mathsf{aux}_\mathbf{y})}$

1 : Parse $\mathsf{ad} = (\mathsf{mpk}, \mathsf{ct}, \pi)$
2 : Let $\mathsf{pk}_\mathbf{y} := \mathsf{IPFE.PubKGen}(\mathsf{mpk}, \mathbf{y})$
3 : **ret** $\widetilde{\sigma} \leftarrow \mathsf{AS.PreSign}(\mathsf{sk}, m, \mathsf{pk}_\mathbf{y})$

$\underline{\mathsf{FPreVerify}_2(\mathsf{ad}, \mathsf{vk}, m, X, \mathbf{y}, \mathsf{aux}_\mathbf{y}, \pi_\mathbf{y}, \widetilde{\sigma})}$

1 : Parse $\mathsf{ad} = (\mathsf{mpk}, \mathsf{ct}, \pi)$
2 : Let $\mathsf{pk}_\mathbf{y} := \mathsf{IPFE.PubKGen}(\mathsf{mpk}, \mathbf{y})$
3 : **ret** $(\mathsf{FPreVerify}_1(\mathsf{ad}, \mathbf{y}, \mathsf{aux}_\mathbf{y}, \pi_\mathbf{y})) \wedge$
$(\mathsf{AS.PreVerify}(\mathsf{vk}, m, \mathsf{pk}_\mathbf{y}, \widetilde{\sigma}))$

$\underline{\mathsf{Adapt}(\mathsf{ad}, \mathsf{st}, \mathsf{vk}, m, X, \mathbf{x}, \mathbf{y}, \mathsf{aux}_\mathbf{y}, \widetilde{\sigma})}$

1 : Parse $\mathsf{ad} = (\mathsf{mpk}, \mathsf{ct}, \pi)$, and $\mathsf{st} = (\mathsf{msk}, \mathbf{t})$
2 : Let $\mathsf{pk}_\mathbf{y} := \mathsf{IPFE.PubKGen}(\mathsf{mpk}, \mathbf{y})$
3 : Let $\mathsf{sk}_\mathbf{y} := \mathsf{IPFE.KGen}(\mathsf{msk}, \mathbf{y})$
4 : **ret** $\sigma := \mathsf{AS.Adapt}(\mathsf{vk}, m, \mathsf{pk}_\mathbf{y}, \mathsf{sk}_\mathbf{y}, \widetilde{\sigma})$

$\underline{\mathsf{FExt}(\mathsf{ad}, \widetilde{\sigma}, \sigma, X, \mathbf{y}, \mathsf{aux}_\mathbf{y})}$

1 : Parse $\mathsf{ad} = (\mathsf{mpk}, \mathsf{ct}, \pi)$.
2 : Let $\mathsf{pk}_\mathbf{y} := \mathsf{IPFE.PubKGen}(\mathsf{mpk}, \mathbf{y})$
3 : Let $z := \mathsf{AS.Ext}(\widetilde{\sigma}, \sigma, \mathsf{pk}_\mathbf{y})$
4 : **ret** $v := \mathsf{IPFE.Dec}(z, \mathsf{ct})$

Figure 3.29: Construction: Weakly-secure FAS

Lemma 3.11.2. *Suppose NIZK satisfies correctness,* AS *satisfies correctness, and* IPFE *satisfies* R_{IPFE}*-compliance and* R'_{IPFE}*-robustness. Then, the functional adaptor signatures construction in Figure 3.29 satisfies correctness.*

Proof. Similar to the proof of Lemma 3.6.1. □

Theorem 3.11.3. *Let* $\mathcal{F}_{\mathsf{IP},\ell}$ *be the function family for computing inner products of vectors of length* ℓ*. Let* R *be any NP relation with statement/witness pairs* $(X, \mathbf{x})$ *such that* $\mathbf{x} \in \mathcal{M}$ *for some set* $\mathcal{M} \subseteq \mathbb{Z}^{\ell}$*. Suppose that*

- $\mathcal{M}$ *is an additive group,*
- R *is* $\mathcal{F}_{\mathsf{IP},\ell}$*-hard (Definition 3.4.2),*
- NIZK *is a secure NIZK argument system (Definition 3.4.26),*
- AS *is an adaptor signature scheme w.r.t. digital signature scheme* DS *and hard relations* $R_{\mathsf{IPFE}}, R'_{\mathsf{IPFE}}$ *that satisfies weak pre-signature adaptability (Definition 3.4.8) and witness extractability (Definition 3.4.7),*
- IPFE *is a selective, IND-secure IPFE scheme (Definition 3.4.10) for function family* $\mathcal{F}_{\mathsf{IP},\ell}$ *that is* R_{IPFE}*-compliant (Definition 3.4.12) and* R'_{IPFE}*-robust (Definition 3.4.13).*

Then, the functional adaptor signature scheme w.r.t. digital signature scheme DS*, NP relation* R*, and family of inner product functions* $\mathcal{F}_{\mathsf{IP},\ell}$ *constructed in Figure 3.29 is weakly-secure (Definition 3.5.4).*

Proof. Follows from Lemmas 3.10.1 to 3.10.3, 3.10.7, 3.10.10 and 3.11.4. □

3.11.1 Witness Indistinguishability

Lemma 3.11.4. *Suppose* NIZK *satisfies adaptive zero-knowledge and* IPFE *satisfies selective, IND-security. Then, the functional adaptor signature construction in Figure 3.2 is witness indistinguishable.*

Proof. We proof this through a sequence of games G_0^b and G_1^b for $b \in \{0, 1\}$.

Game G_0^b**:** This game corresponds to the original game $\mathsf{faWI}^b_{\mathcal{A},\mathsf{FAS}}$. Formally, the game is defined in Figure 3.30. For sake of simplicity, we drop the oracle $\mathcal{O}_{\mathsf{FPreSign}_1}$ as it is redundant in context of the construction.

Game G_1^b**:** This game is same as G_0^b except that NIZK simulator is used instead of NIZK prover. Formally, the game is defined in Figure 3.30.

To prove the lemma, we need to show that

$$|\Pr[G_0^0(1^\lambda) = 1] - \Pr[G_0^1(1^\lambda) = 1]| \leq \mathsf{negl}(\lambda),$$

Games G_0^b, $\boxed{G_1^b}$.

1 : $\mathsf{crs} \leftarrow \mathsf{NIZK.Setup}(1^\lambda)$ $\boxed{\mathsf{crs} \leftarrow \mathsf{NIZK.Setup}^*(1^\lambda)}$
2 : $\mathsf{pp}' \leftarrow \mathsf{IPFE.Gen}(1^\lambda)$
3 : $\mathsf{pp} = (\mathsf{crs}, \mathsf{pp}')$
4 : $(\mathsf{vk}, X, \mathbf{x}_0, \mathbf{x}_1) \leftarrow \mathcal{A}(\mathsf{pp})$
5 : if $(((X, \mathbf{x}_0) \notin R) \vee ((X, \mathbf{x}_1) \notin R))$: **ret**0
6 : Sample random coins r_0, r_1
7 : $(\mathsf{mpk}, \mathsf{msk}) := \mathsf{IPFE.Setup}(\mathsf{pp}', 1^\ell; r_0)$
8 : $\mathsf{ct} := \mathsf{IPFE.Enc}(\mathsf{mpk}, \mathbf{x}_b; r_1)$
9 : $\pi \leftarrow \mathsf{NIZK.Prove}(\mathsf{crs}, (X, \mathsf{pp}', \mathsf{mpk}, \mathsf{ct}), (r_0, r_1, \mathbf{x}_b))$
$\boxed{\pi \leftarrow \mathsf{NIZK.Prove}^*(\mathsf{crs}, (X, \mathsf{pp}', \mathsf{mpk}, \mathsf{ct}))}$
10 : $\mathsf{ad} := (\mathsf{mpk}, \mathsf{ct}, \pi)$, $\mathsf{st} := \mathsf{msk}$
11 : $(m, \mathbf{y}, \mathsf{aux}_\mathbf{y}, \pi_\mathbf{y}, \widetilde{\sigma}) \leftarrow \mathcal{A}(\mathsf{ad})$
12 : $\mathsf{pk}_\mathbf{y} := \mathsf{IPFE.PubKGen}(\mathsf{mpk}, \mathbf{y})$
13 : If $\mathsf{AS.PreVerify}(\mathsf{vk}, m, \mathsf{pk}_\mathbf{y}, \widetilde{\sigma}) = 0$: **ret** 0
14 : If $f_\mathbf{y}(\mathbf{x}_0) \neq f_\mathbf{y}(\mathbf{x}_1)$: **ret** 0
15 : $\mathsf{sk}_\mathbf{y} := \mathsf{IPFE.KGen}(\mathsf{msk}, \mathbf{y})$
16 : $\sigma = \mathsf{AS.Adapt}(\mathsf{vk}, m, \mathsf{pk}_\mathbf{y}, \mathsf{sk}_\mathbf{y}, \widetilde{\sigma})$
17 : $b' \leftarrow \mathcal{A}(\sigma)$
18 : **ret** b'

Figure 3.30: Witness Indistinguishability proof: Games G_0^b and G_1^b for $b \in \{0,1\}$

Note that by triangle inequality, it follows that

$$\begin{aligned}
&|\Pr[G_0^0(1^\lambda) = 1] - \Pr[G_0^1(1^\lambda) = 1]| \\
&\leq |\Pr[G_0^0(1^\lambda) = 1] - \Pr[G_1^0(1^\lambda) = 1]| \\
&\quad + |\Pr[G_1^0(1^\lambda) = 1] - \Pr[G_1^1(1^\lambda) = 1]| \\
&\quad + |\Pr[G_1^1(1^\lambda) = 1] - \Pr[G_0^1(1^\lambda) = 1]|.
\end{aligned}$$

To complete the proof, we show in Claim 3.11.5 that the first and third terms on the right-hand-side are $\leq \mathsf{negl}(\lambda)$ and in Claim 3.11.6 that the second term on the right-hand-side is $\leq \mathsf{negl}(\lambda)$.

□

Claim 3.11.5. *If* NIZK *satisfies adaptive zero-knowledge, then, there exists a negligible function* negl *such that for all* $\lambda \in \mathbb{N}$, *for all* $b \in \{0,1\}$,

$$|\Pr[G_0^b(1^\lambda) = 1] - \Pr[G_1^b(1^\lambda) = 1]| \leq \mathsf{negl}(\lambda).$$

Proof. We show that if there exists a p.p.t. adversary $\mathcal{A}$ can distinguish its view in games G_0^b and G_1^b, then, we can create a distinguisher $\mathcal{D}^b$ that breaks the adaptive zero-knowledge of the underlying NIZK. Let $\mathcal{C}$ be the adaptive zero-knowledge challenger for NIZK. $\mathcal{C}$ samples a random bit $\beta \xleftarrow{\$} \{0,1\}$ and if $\beta = 0$, it samples crs and π as in the real-world and if $\beta = 1$, it samples crs and π using the NIZK simulator algorithms Setup^* and Prove^* respectively. Note that as the adaptive zero-knowledge of the NIZK proof system holds for all valid statement-witness pairs, we will allow the distinguisher $\mathcal{D}^b$ to choose a valid statement-witness pair adaptively after seeing crs. The distinguisher $\mathcal{D}^b$ must output a bit at the end and it will use the adversary $\mathcal{A}$ for this task. The distinguisher $\mathcal{D}^b$ interacts with $\mathcal{A}$ and either presents it the view as in G_0^b or as in G_1^b and at the end $\mathcal{A}$ outputs its guess bit $\beta' \in \{0,1\}$. The distinguisher $\mathcal{D}^b$ is as in Figure 3.31.

Distinguisher $\mathcal{D}^b$ for proof of Claim 3.11.5

1 : $\mathsf{crs} \leftarrow \mathcal{C}(1^\lambda)$
2 : $\mathsf{pp}' \leftarrow \mathsf{IPFE.Gen}(1^\lambda)$
3 : $\mathsf{pp} = (\mathsf{crs}, \mathsf{pp}')$
4 : $(\mathsf{vk}, X, \mathbf{x}_0, \mathbf{x}_1) \leftarrow \mathcal{A}(\mathsf{pp})$
5 : if $(((X, \mathbf{x}_0) \notin R) \vee ((X, \mathbf{x}_1) \notin R))$: **ret**0
6 : Sample random coins r_0, r_1
7 : $(\mathsf{mpk}, \mathsf{msk}) := \mathsf{IPFE.Setup}(\mathsf{pp}', 1^\ell; r_0)$
8 : $\mathsf{ct} := \mathsf{IPFE.Enc}(\mathsf{mpk}, \mathbf{x}_b; r_1)$
9 : $\pi \leftarrow \mathcal{C}((X, \mathsf{pp}', \mathsf{mpk}, \mathsf{ct}), (r_0, r_1, \mathbf{x}_b))$
10 : $\mathsf{ad} := (\mathsf{mpk}, \mathsf{ct}, \pi)$, $\mathsf{st} := \mathsf{msk}$
11 : $(m, \mathbf{y}, \mathsf{aux}_\mathbf{y}, \pi_\mathbf{y}, \widetilde{\sigma}) \leftarrow \mathcal{A}(\mathsf{ad})$
12 : $\mathsf{pk}_\mathbf{y} := \mathsf{IPFE.PubKGen}(\mathsf{mpk}, \mathbf{y})$
13 : If $\mathsf{AS.PreVerify}(\mathsf{vk}, m, \mathsf{pk}_\mathbf{y}, \widetilde{\sigma}) = 0$: **ret** 0
14 : If $f_\mathbf{y}(\mathbf{x}_0) \neq f_\mathbf{y}(\mathbf{x}_1)$: **ret** 0
15 : $\mathsf{sk}_\mathbf{y} := \mathsf{IPFE.KGen}(\mathsf{msk}, \mathbf{y})$
16 : $\sigma = \mathsf{AS.Adapt}(\mathsf{vk}, m, \mathsf{pk}_\mathbf{y}, \mathsf{sk}_\mathbf{y}, \widetilde{\sigma})$
17 : $\beta' \leftarrow \mathcal{A}(\sigma)$
18 : **ret** β'

Figure 3.31: Distinguisher $\mathcal{D}^b$ for proof of Claim 3.11.5

Observe that if $\mathcal{C}$ chose bit β, then, the view observed by $\mathcal{A}$ is that of game G_β^b. Hence, it follows that

$$\begin{aligned} &|\Pr[\mathcal{D}^b(\mathsf{crs}, \pi) = 1 | \beta = 0] - \Pr[\mathcal{D}^b(\mathsf{crs}, \pi) = 1 | \beta = 1]| \\ &= |\Pr[G_0^b(1^\lambda) = 1] - \Pr[G_0^b(1^\lambda) = 1]|. \end{aligned}$$

Hence, if $\mathcal{A}$ has a noticeable distinguishing advantage ϵ, then, $\mathcal{D}^b$ also has a noticeable distinguishing advantage ϵ in its game with challenger $\mathcal{C}$. This completes the proof. □

Claim 3.11.6. *If* IPFE *satisfies selective, IND-security, then, there exists a negligible function* negl *such that for all* $\lambda \in \mathbb{N}$,

$$|\Pr[G_1^0(1^\lambda) = 1] - \Pr[G_1^1(1^\lambda) = 1]| \leq \mathsf{negl}(\lambda).$$

Proof. We show that if there exists a p.p.t. adversary $\mathcal{A}$ can distinguish its view in games G_1^0 and G_1^1, then, we can create a reduction $\mathcal{B}$ that breaks the selective, IND-security of the underlying IPFE. For the IPFE selective, IND-security game, let $\mathcal{C}$ be the challenger and let $\mathsf{IPFE}.\mathcal{O}_{\mathsf{KGen}}$ be the key generation oracle that the reduction $\mathcal{B}$ has access to. The reduction $\mathcal{B}$ is as in Figure 3.32.

Reduction $\mathcal{B}$ for proof of Claim 3.11.6

$\mathsf{crs} \leftarrow \mathsf{NIZK.Setup}^*(1^\lambda)$
$\mathsf{pp}' \leftarrow \mathcal{C}(1^\lambda)$
$\mathsf{pp} = (\mathsf{crs}, \mathsf{pp}')$
$(\mathsf{vk}, X, \mathbf{x}_0, \mathbf{x}_1) \leftarrow \mathcal{A}(\mathsf{pp})$
if $(((X, \mathbf{x}_0) \notin R) \vee ((X, \mathbf{x}_1) \notin R))$: **ret**0
$(\mathsf{mpk}, \mathsf{ct}) \leftarrow \mathcal{C}(\mathbf{x}_0, \mathbf{x}_1)$
$\pi \leftarrow \mathsf{NIZK.Prove}^*(\mathsf{crs}, (X, \mathsf{pp}', \mathsf{mpk}, \mathsf{ct}))$
$\mathsf{ad} := (\mathsf{mpk}, \mathsf{ct}, \pi)$, $\mathsf{st} := \mathsf{msk}$
$(m, \mathbf{y}, \mathsf{aux}_\mathbf{y}, \pi_\mathbf{y}, \widetilde{\sigma}) \leftarrow \mathcal{A}(\mathsf{ad})$
$\mathsf{pk}_\mathbf{y} := \mathsf{IPFE.PubKGen}(\mathsf{mpk}, \mathbf{y})$
If $\mathsf{AS.PreVerify}(\mathsf{vk}, m, \mathsf{pk}_\mathbf{y}, \widetilde{\sigma}) = 0$: **ret** 0
If $f_\mathbf{y}(\mathbf{x}_0) \neq f_\mathbf{y}(\mathbf{x}_1)$: **ret** 0
$\mathsf{sk}_\mathbf{y} = \mathsf{IPFE}.\mathcal{O}_{\mathsf{KGen}}(\mathbf{y})$
$\sigma = \mathsf{AS.Adapt}(\mathsf{vk}, m, \mathsf{pk}_\mathbf{y}, \mathsf{sk}_\mathbf{y}, \widetilde{\sigma})$
$b' \leftarrow \mathcal{A}(\sigma)$
ret b'

Figure 3.32: Reduction $\mathcal{B}$ for proof of Claim 3.11.6

Observe that when $\mathcal{C}$ plays game $\mathsf{IPFE\text{-}Expt\text{-}SEL}_\mathcal{A}^b(1^\lambda, n)$ with the reduction $\mathcal{B}$ for some $b \in \{0,1\}$, then, ct encrypts $\mathbf{x}_b$ and thus, $\mathcal{A}$'s view is same as G_1^b. Further, observe that $\mathcal{B}$ makes query to the key generation oracle only if $f_\mathbf{y}(\mathbf{x}_0) = f_\mathbf{y}(\mathbf{x}_1)$. Thus, $\mathcal{B}$ is an admissible adversary in the game with $\mathcal{C}$. Hence, it follows that for all $b \in \{0,1\}$,

$$\begin{aligned}\Pr[\mathsf{IPFE\text{-}Expt\text{-}SEL}_\mathcal{B}^b(1^\lambda, n) = 1] &= \Pr[b' = 1] \\ &= \Pr[G_1^b(1^\lambda) = 1].\end{aligned}$$

Therefore, we get that

$$\begin{aligned}&|\Pr[\mathsf{IPFE\text{-}Expt\text{-}SEL}^0_{\mathcal{A}}(1^\lambda, n) = 1] - \Pr[\mathsf{IPFE\text{-}Expt\text{-}SEL}^1_{\mathcal{A}}(1^\lambda, n) = 1]|\\ &= |\Pr[G^0_1(1^\lambda) = 1] - \Pr[G^1_1(1^\lambda) = 1]|.\end{aligned}$$

Hence, if $\mathcal{A}$ has a noticeable distinguishing advantage ϵ in its game with reduction $\mathcal{B}$, then, $\mathcal{B}$ also has a noticeable distinguishing advantage ϵ in its game with challenger $\mathcal{C}$. This completes the proof. □

3.12 FAS for General Functions from FHE

In this section, we build a FAS construction for general functions from FHE. Our FAS construction is defined w.r.t. the Schnorr Signature scheme, any hard relation R with statement/witness pairs (Y, y) where witness y is a vector of integers, and the function family $\mathcal{F}_{\mathsf{circuits}}$ of unbounded depth polynomial-size circuits. The construction uses the following building blocks and is detailed in Figure 3.34: (a) semantically-secure homomorphic encryption scheme FHE, (b) the Elgamal public-key encryption scheme EG, (c) two NIZK argument systems $\mathsf{NIZK}_{\mathsf{ad}}$ and $\mathsf{NIZK}_{\mathsf{eq}}$, and (d) the Schnorr Adaptor signature AS.

$\mathsf{NIZK}_{\mathsf{ad}}$ is used in the AdGen algorithm, which is run only once. So, we do not optimize it and rely on a general-purpose NIZK. In contrast, $\mathsf{NIZK}_{\mathsf{eq}}$ is invoked in the $\mathsf{FPreSign}_1$ algorithm, which is run by the seller *once* every functional sale, so it must be efficient. We present an optimized instantiation, with the remainder of this section organized as follows: Section 3.12.1 covers the building blocks, Section 3.12.2 presents the efficient $\mathsf{NIZK}_{\mathsf{eq}}$ instantiation, and Section 3.12.3 details the full construction.

3.12.1 Building Blocks

Public Key Encryption. A public key encryption scheme $\mathsf{PKE} = (\mathsf{KGen}, \mathsf{Enc}, \mathsf{Dec})$ is a tuple of three algorithms. KGen outputs a public key pk and a secret key sk. A message m is encrypted as $\mathsf{ct} \leftarrow\!\!\$\ \mathsf{Enc}(\mathsf{pk}, m)$, and the resulting ciphertext ct is decrypted as $m = \mathsf{Dec}(\mathsf{sk}, \mathsf{ct})$. We require *semantic security*, which ensures that ciphertexts reveal no information about the message.

ElGamal encryption. Given a cyclic group G of prime order p and a generator g, the keys are computed as $\mathsf{sk} \leftarrow\!\!\$\ \mathbb{Z}_p$ and $\mathsf{pk} = g^{\mathsf{sk}}$ where ciphertext $\mathsf{ct} = (g^r, \mathsf{pk}^r \cdot g^m)$ for a randomly chosen $r \xleftarrow{\$} \mathbb{Z}_p$, and decryption of a ciphertext $\mathsf{ct} = (\mathsf{ct}_0, \mathsf{ct}_1)$ outputs the discrete log of $\mathsf{ct}_1/\mathsf{ct}_0^{\mathsf{sk}}$ w.r.t. g. As decryption requires discrete-logarithm computation, the scheme is efficient only for small messages m.

Schnorr signature. The scheme Sch uses the same parameters $(\mathbb{G}, p, q)$ as ElGamal, with an added hash function $H : \{0,1\}^* \to \mathbb{Z}_p$ modeled as a random oracle, where $\mathsf{Sch.KGen}(1^\lambda)$ outputs signing key $\mathsf{sk} \xleftarrow{\$} \mathbb{Z}_p$

and verification key $\mathsf{vk} = g^{\mathsf{sk}}$. $\mathsf{Sch.Sign}(\mathsf{sk}, m)$ outputs a signature $\sigma := (h, s)$, where $h := H(\mathsf{vk}||g^r||m)$, $s := r + h \cdot \mathsf{sk}$ and $r \xleftarrow{\$} \mathbb{Z}_p$. $\mathsf{Sch.Vf}(\mathsf{vk}, m, \sigma)$ computes $R := g^s/\mathsf{vk}^h$, and outputs 1 iff $h = H(\mathsf{vk}||R||m)$.

Schnorr adaptor signature. The scheme AS is defined w.r.t. Schnorr scheme Sch and the discrete-logarithm relation R_{DL} on the group G. Let L_{DL} be the NP language corresponding to R_{DL}. Then, AS is as follows for $Y \in L_{DL}$: $\mathsf{AS.PreSign}(\mathsf{sk}, m, Y)$ outputs pre-signature $\widetilde{\sigma} := (h, \widetilde{s})$, where $h := H(\mathsf{vk}||(g^r \cdot Y)||m)$, $\widetilde{s} := r + h \cdot \mathsf{sk}$ and $r \xleftarrow{\$} \mathbb{Z}_p$. $\mathsf{AS.PreVerify}(\mathsf{vk}, m, Y, \widetilde{\sigma})$ computes $R := g^{\widetilde{s}}/\mathsf{vk}^h$, and outputs 1 iff $h = H(\mathsf{vk}||R \cdot Y||m)$. $\mathsf{AS.Adapt}(\mathsf{vk}, m, Y, y, \widetilde{\sigma})$ parses $\widetilde{\sigma} := (h, \widetilde{s})$ and outputs $\sigma := (h, s := \widetilde{s} + y)$. $\mathsf{AS.Ext}(\widetilde{\sigma}, \sigma, Y)$ computes $y := s - \widetilde{s}$ and outputs y.

Homomorphic Encryption (HE). Informally, a homomorphic encryption scheme additionally supports homomorphism over the ciphertext space. Linear homomorphism requires that for any two messages m_0 and m_1, we have that $\mathsf{Dec}(\mathsf{sk}, \mathsf{Enc}(\mathsf{pk}, m_0) * \mathsf{Enc}(\mathsf{pk}, m_1)) = m_0 + m_1$, where $+$ is the group operation of the message space and $*$ is the group operator of the ciphertext space. *Full homomorphism* generalizes this property to any function f, formalized via a public evaluation algorithm HEval that satisfies $\mathsf{Dec}(\mathsf{sk}, \mathsf{HEval}(\mathsf{pk}, \mathsf{Enc}(\mathsf{pk}, m), f)) = f(m)$ for all m and f.

Augmenting HE syntax with REval algorithm. We extend the syntax of HE schemes with (a possibly *private*) algorithm REval for efficiently recovering the randomness corresponding to a homomorphically evaluated ciphertext. Suppose $\mathsf{ct} := \mathsf{Enc}(\mathsf{pk}, m; r)$ and $\mathsf{ct}_f := \mathsf{HEval}(\mathsf{pk}, \mathsf{ct}, f)$. Then, we define $r_f := \mathsf{REval}(\mathsf{pk}, \mathsf{sk}, \mathsf{ct}, m, r, f, \mathsf{ct}_f)$. Correctness requires $\mathsf{Enc}(\mathsf{pk}, f(m); r_f) = \mathsf{ct}_f$. Since REval is not exposed to the adversary, it does not impact the semantic security of HE. For some schemes in this work (example, BGN and LHE), REval can recover r_f without $\mathsf{sk}, \mathsf{ct}_f$ as inputs, thus providing a more efficient alternative to compute ct_f than HEval (i.e., directly as $\mathsf{ct}_f = \mathsf{Enc}(pk, f(m); r_f)$).[17]

Remark 3.12.1. *While we explicitly include* sk *as input to* REval*, in most HE schemes—including FHE schemes like BGV [BGV11], GSW [GSW13], and TFHE [CGGI20], as well as low-degree schemes like BGN [BGN05]—r_f can be computed from* pk*, f, and the input randomness r by mimicking* HEval*. In fact, in our BGN-based instantiation for quadratic functions, the HE public-key is generated via a trusted setup, and the seller does not hold* sk *to compute* REval*. In contrast, in our TFHE-FAS, the seller selects* pk *and retains* sk*. However, due to complex bootstrapping operations,* REval *is as expensive as* HEval*. To address this, we leverage knowledge of* sk *to compute r_f more efficiently using techniques like key-switching. See Section 3.13.1 for details.*

Non-Interactive Zero-Knowledge (NIZK) Arguments. A NIZK for the NP language L_R consists of algorithms $(\mathsf{Setup}, \mathsf{Prove}, \mathsf{Vf})$ where $\mathsf{Setup}(1^\lambda)$ outputs the common reference string crs, $\mathsf{Prove}(\mathsf{crs}, \mathsf{stmt}, \mathsf{wit})$

[17] For soundness, we require that all HE ciphertexts ct, even those computed through HEval, satisfy binding w.r.t. well-formed pk - that is, $\forall (m, r), (m', r')$, if $\mathsf{ct} = \mathsf{Enc}(pk, m; r) = \mathsf{Enc}(pk, m'; r')$ then $m = m'$.

outputs a proof π that the statement $\mathsf{stmt} \in L_R$ using a witness wit, and $\mathsf{Vf}(\mathsf{crs}, \mathsf{stmt}, \pi)$ checks π's validity. We require the NIZK argument system to be (1) *complete*, where honestly generated proofs for $(\mathsf{stmt}, \mathsf{wit}) \in R$ verify successfully, (2) *zero-knowledge*, where a malicious verifier learn nothing more than stmt is in L_R, i.e., there exists an p.p.t. simulator Sim that simulates a malicious verifier's view without using the witness, and (3) *adaptive soundness*, where p.p.t. provers cannot generate convincing proofs of false statements that are chosen adaptively after receiving the crs. Such NIZK arguments are known from several assumptions [FLS90, CHK03, GOS06, PS19, CCH+19].

3.12.2 $\mathsf{NIZK}_{\mathsf{eq}}$ Instantiation

Our FAS construction in Figure 3.34 relies on a NIZK proof system $\mathsf{NIZK}_{\mathsf{eq}}$ to prove that two ciphertexts, $\mathsf{ct}_{\mathsf{FHE}}$ and $\mathsf{ct}_{\mathsf{EG}}$, encrypt the same message [18]. Here, FHE and EG are encryption schemes with message space $\mathbb{Z}_p$, where p is a constant (example, $p = 256$ in our implementation). Since this proof is computed once per functional sale, we propose a new cut-and-choose [CD00] based proof that is optimized for this relation. Unlike other SNARK-based approaches, our focus is on reducing prover time, rather than minimizing proof size. Since our FHE-based approach aims to minimize seller computation, we prefer proof systems with faster provers. Our results in Figure 3.37 justify the choice of the cut-and-choose approach.

More formally, NIZK language $L_{\mathsf{eq}}^{\mathsf{pk}_{\mathsf{FHE}}}$ is parameterized by FHE's public-key $\mathsf{pk}_{\mathsf{FHE}}$, and contains statements of the form $(\mathsf{ct}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{EG}})$ if there exists witness $(z, r, \bar{r})$ such that $\mathsf{ct}_{\mathsf{FHE}} = \mathsf{FHE.Enc}(\mathsf{pk}_{\mathsf{FHE}}, z; r)$, $\mathsf{ct}_{\mathsf{EG}} = \mathsf{EG.Enc}(\mathsf{pk}_{\mathsf{EG}}, z; \bar{r})$, and $\mathsf{pk}_{\mathsf{EG}} \in \mathbb{G}$.

Suppose $\mathbb{H} : \{0,1\}^* \to \{0,1\}^\lambda$ is a hash function. Then $\mathsf{NIZK}_{\mathsf{eq}}$ is as in Figure 3.33. We note that the prover here is extremely fast since it only has to perform 2λ FHE encryptions, 2λ El Gamal encryptions, 1 hash computation, and 3λ additions. Furthermore, the verifier is extremely fast since it only has to perform 1 hash computation, 2λ FHE encryptions, 2λ El Gamal encryptions, λ FHE ciphertext additions, λ El Gamal ciphertext additions, and 4λ equality checks. This is evidenced by our implementation numbers, as highlighted in Figure 3.37. [19] We defer the formal proofs to the full version.

3.12.3 FAS Construction

[18] We also need standard range proof for proving well-formedness of El Gamal encryption of bits. We defer its details to the full version.

[19] For reasonable security level, setting $\lambda = 40$ in $\mathsf{NIZK}_{\mathsf{eq}}$ suffices.

[21] In our FHE based instantiation of FAS (Section 3.13.1), the buyer supplies $(f, \widehat{\mathsf{ct}})$ after $\mathsf{AdVerify}$ at the beginning of exchange phase, and $\mathsf{FPreSign}_1(\mathsf{pp}, \mathsf{ad}, \mathsf{st}, Y, \vec{y}, (f, \widehat{\mathsf{ct}}))$ algorithm computes r_2 as $r_2 := \mathsf{FHE.REval}(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{sk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, \perp, r_1, f, \widehat{\mathsf{ct}})$. This significantly boosts efficiency of REval, as the seller does not need to recompute $\widehat{\mathsf{ct}}$ in REval internally.

$\mathsf{Setup}(1^\lambda)$:

1 : Sample $\mathsf{hk} \leftarrow \mathbb{H}.\mathsf{Gen}(1^\lambda)$

2 : **ret** $\mathsf{crs} := \mathsf{hk}$

$\mathsf{Prove}(\mathsf{crs}, \mathsf{stmt} = (\mathsf{ct}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{EG}}), \mathsf{wit} = (z, r, \bar{r}))$:

1 : For all $i \in [2\lambda]$: Sample messages $s_i \xleftarrow{\$} \mathbb{Z}_p$,
sample random coins r_i and $\overline{r_i}$,
compute $\mathsf{ct}_{\mathsf{FHE},i} := \mathsf{FHE.Enc}(\mathsf{pk}_{\mathsf{FHE}}, s_i; r_i)$,
and compute $\mathsf{ct}_{\mathsf{EG},i} := \mathsf{EG.Enc}(\mathsf{pk}_{\mathsf{EG}}, s_i; \overline{r_i})$.

2 : Compute $str := \mathbb{H}.\mathsf{H}(\mathsf{hk}, \mathsf{ct}_{\mathsf{FHE}} || \mathsf{ct}_{\mathsf{EG}} || \mathsf{ct}_{\mathsf{FHE},1} || \ldots || \mathsf{ct}_{\mathsf{FHE},2\lambda} || \mathsf{ct}_{\mathsf{EG},1} || \ldots || \mathsf{ct}_{\mathsf{EG},2\lambda}) \in \{0,1\}^\lambda$.

3 : Define set $I = \{2 \cdot i + str[i] : i \in [\lambda]\}$ of size λ.

4 : **ret** $\pi := (\{\mathsf{ct}_{\mathsf{FHE},i}, \mathsf{ct}_{\mathsf{EG},i}\}_{i\in[2\lambda]}, I, \{s_i, r_i, \overline{r_i}\}_{i\in I}, \{\widehat{s_i} := z + s_i, \widehat{r_i} := r + r_i, \widehat{\overline{r_i}} := \bar{r} + \overline{r_i}\}_{i\notin I})$.

$\mathsf{Vf}(\mathsf{crs}, \mathsf{stmt}, \pi)$:

1 : Parse $\mathsf{stmt} = (\mathsf{ct}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{EG}})$.

2 : Parse $\pi = (\{\mathsf{ct}_{\mathsf{FHE},i}, \mathsf{ct}_{\mathsf{EG},i}\}_{i\in[2\lambda]}, I, \{s_i, r_i, \overline{r_i}\}_{i\in I}, \{\widehat{s_i}, \widehat{r_i}, \widehat{\overline{r_i}}\}_{i\notin I})$.

3 : Compute $str := \mathbb{H}.\mathsf{H}(\mathsf{hk}, \mathsf{ct}_{\mathsf{FHE}} || \mathsf{ct}_{\mathsf{EG}} || \mathsf{ct}_{\mathsf{FHE},1} || \ldots || \mathsf{ct}_{\mathsf{FHE},2\lambda} || \mathsf{ct}_{\mathsf{EG},1} || \ldots || \mathsf{ct}_{\mathsf{EG},2\lambda}) \in \{0,1\}^\lambda$.

4 : **ret** 1 iff $I = \{2 \cdot i + str[i] : i \in [\lambda]\}$ and,
for all $i \in I$:
$\mathsf{ct}_{\mathsf{FHE},i} = \mathsf{FHE.Enc}(\mathsf{pk}_{\mathsf{FHE}}, s_i; r_i)$,
$\mathsf{ct}_{\mathsf{EG},i} = \mathsf{EG.Enc}(\mathsf{pk}_{\mathsf{EG}}, s_i; \overline{s_i})$,
and for all $i \notin I$:
$\mathsf{ct}_{\mathsf{FHE}} + \mathsf{ct}_{\mathsf{FHE},i} = \mathsf{FHE.Enc}(\mathsf{pk}_{\mathsf{FHE}}, \widehat{s_i}; \widehat{r_i})$,
$\mathsf{ct}_{\mathsf{EG}} \cdot \mathsf{ct}_{\mathsf{EG},i} = \mathsf{EG.Enc}(\mathsf{pk}_{\mathsf{EG}}, \widehat{s_i}; \widehat{\overline{r_i}})$.

Figure 3.33: Cut-and-choose based $\mathsf{NIZK}_{\mathsf{eq}}$ for proving plaintext equality across HE and ElGamal ciphertexts.

The proposed FAS construction is described in Figure 3.34. Note that we have two NIZK argument systems in the scheme, which we define as $\mathsf{NIZK}_{\mathsf{ad}} = (\mathsf{Setup}_{\mathsf{ad}}, \mathsf{Prove}_{\mathsf{ad}}, \mathsf{Verify}_{\mathsf{ad}})$ and $\mathsf{NIZK}_{\mathsf{eq}} = (\mathsf{Setup}_{\mathsf{eq}}, \mathsf{Prove}_{\mathsf{eq}}, \mathsf{Verify}_{\mathsf{eq}})$. The argument system $\mathsf{NIZK}_{\mathsf{ad}}$ is used for proving that the advertisement ad is correctly generated, and $\mathsf{NIZK}_{\mathsf{eq}}$ is used for proving that the two ciphertexts, ct and $\widehat{\mathsf{ct}}$, both encrypt the same value $z = f(y)$. The language L_{eq} is defined in Section 3.12.2 and L_{ad} is defined as follows: L_{ad} contains statements of the form $(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, Y)$ if there exists a witness (y, r_1, r_0) such that $(Y, y) \in R$ and $\mathsf{ct}_{\mathsf{ad}} := \mathsf{FHE.Enc}(\mathsf{pk}_{\mathsf{FHE}}, y; r_1)$ and $(\mathsf{pk}_{\mathsf{FHE}}, \cdot) := \mathsf{FHE.KGen}(1^\lambda; r_0)$. We assume that functions f output $(k+1)$-bit integers for some integer k. The security of our construction is captured in the following theorem, while the formal analysis is deferred to the full version.

Theorem 3.12.2 (Informal). *If the* NIZK *arguments for the languages* L_{ad} *and* L_{eq} *both satisfy zero-knowledge*

$\mathsf{Setup}(1^\lambda)$:

1 : $\mathsf{crs}_{\mathsf{ad}} \leftarrow \mathsf{Setup}_{\mathsf{ad}}(1^\lambda)$
2 : $\mathsf{crs}_{\mathsf{eq}} \leftarrow \mathsf{Setup}_{\mathsf{eq}}(1^\lambda)$
3 : **ret** $\mathsf{pp} := (\mathsf{crs}_{\mathsf{ad}}, \mathsf{crs}_{\mathsf{eq}})$

$\mathsf{AdGen}(\mathsf{pp}, Y, \vec{y})$:

1 : $(\mathsf{sk}_{\mathsf{FHE}}, \mathsf{pk}_{\mathsf{FHE}}) := \mathsf{FHE.KGen}(1^\lambda; r_0)$
2 : $\mathsf{ct}_{\mathsf{ad}} := \mathsf{FHE.Enc}(\mathsf{pk}_{\mathsf{FHE}}, \vec{y}; r_1)$
3 : $\mathsf{stmt}_{\mathsf{ad}} := (\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, Y)$
4 : $\mathsf{wit}_{\mathsf{ad}} := (\vec{y}, r_0, r_1)$
5 : $\pi_{\mathsf{ad}} \leftarrow \mathsf{Prove}_{\mathsf{ad}}(\mathsf{crs}_{\mathsf{ad}}, \mathsf{stmt}_{\mathsf{ad}}, \mathsf{wit}_{\mathsf{ad}})$
6 : $\mathsf{ad} := (\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, \pi_{\mathsf{ad}})$
7 : $\mathsf{st} := (\mathsf{sk}_{\mathsf{FHE}}, r_0, r_1)$
8 : **ret** $(\mathsf{ad}, \mathsf{st})$

$\mathsf{AdVerify}(\mathsf{pp}, \mathsf{ad}, Y)$:

1 : Parse ad as $(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, \pi_{\mathsf{ad}})$
2 : $\mathsf{stmt}_{\mathsf{ad}} := (\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, Y)$
3 : **ret** $\mathsf{Verify}_{\mathsf{ad}}(\mathsf{crs}_{\mathsf{ad}}, \mathsf{stmt}_{\mathsf{ad}}, \pi_{\mathsf{ad}})$

$\mathsf{FPreSign}_1(\mathsf{pp}, \mathsf{ad}, \mathsf{st}, Y, \vec{y}, f)$:

1 : Parse st as $(\mathsf{sk}_{\mathsf{FHE}}, r_0, r_1)$
2 : Parse ad as $(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, \pi_{\mathsf{ad}})$
3 : $r_2 := \mathsf{FHE.REval}(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{sk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, \vec{y}, r_1, f, \bot)$[21]
4 : $(\mathsf{sk}_{\mathsf{EG}}, \mathsf{pk}_{\mathsf{EG}}) \leftarrow \mathsf{EG.KGen}(1^\lambda)$
5 : $z := f(\vec{y})$
6 : $(b_0, \ldots, b_k) :=$ bit-decompose z
7 : $\forall j \in \{0, \ldots, k\}$:
$\mathsf{ct}_j := \mathsf{EG.Enc}(\mathsf{pk}_{\mathsf{EG}}, b_j; r_{3,j})$
8 : $\vec{r_3} := (r_{3,0}, \ldots, r_{3,k})$
9 : $\vec{\mathsf{ct}} := (\mathsf{ct}_0, \ldots, \mathsf{ct}_k)$
10 : $\widehat{\mathsf{ct}} := \mathsf{FHE.Enc}(\mathsf{pk}_{\mathsf{FHE}}, z; r_2)$
11 : $\mathsf{stmt}_{\mathsf{eq}} := (\mathsf{pk}_{\mathsf{EG}}, \vec{\mathsf{ct}}, \mathsf{pk}_{\mathsf{FHE}}, \widehat{\mathsf{ct}})$
12 : $\mathsf{wit}_{\mathsf{eq}} := (\mathsf{sk}_{\mathsf{FHE}}, z, r_2, \vec{r_3})$
13 : $\pi_{\mathsf{eq}} \leftarrow \mathsf{Prove}_{\mathsf{eq}}(\mathsf{crs}_{\mathsf{eq}}, \mathsf{stmt}_{\mathsf{eq}}, \mathsf{wit}_{\mathsf{eq}})$
14 : Update $\mathsf{st} = (\mathsf{sk}_{\mathsf{FHE}}, r_0, r_1, \mathsf{sk}_{\mathsf{EG}})$
15 : **ret** $\widetilde{\sigma}_1 := (\mathsf{pk}_{\mathsf{EG}}, \vec{\mathsf{ct}}, \pi_{\mathsf{eq}})$

$\mathsf{FPreVerify}_1(\mathsf{pp}, \mathsf{ad}, Y, f, \widetilde{\sigma}_1)$:

1 : Parse ad as $(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, \pi_{\mathsf{ad}})$
2 : Parse $\widetilde{\sigma}_1$ as $(\mathsf{pk}_{\mathsf{EG}}, \vec{\mathsf{ct}}, \pi_{\mathsf{eq}})$
3 : $\widehat{\mathsf{ct}} = \mathsf{FHE.Eval}(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, f)$
4 : $\mathsf{stmt}_{\mathsf{eq}} := (\mathsf{pk}_{\mathsf{EG}}, \vec{\mathsf{ct}}, \mathsf{pk}_{\mathsf{FHE}}, \widehat{\mathsf{ct}})$
5 : **ret** $\mathsf{Verify}_{\mathsf{eq}}(\mathsf{crs}_{\mathsf{eq}}, \mathsf{stmt}_{\mathsf{eq}}, \pi_{\mathsf{eq}})$

$\mathsf{FPreSign}_2(\mathsf{pp}, \mathsf{ad}, \mathsf{sk}, m, Y, f, \widetilde{\sigma}_1)$:

1 : Parse $\widetilde{\sigma}_1$ as $(\mathsf{pk}_{\mathsf{EG}}, \vec{\mathsf{ct}}, \pi_{\mathsf{eq}})$
2 : $\widetilde{\sigma}_2 \leftarrow \mathsf{AS.pSign}(\mathsf{sk}, m, \mathsf{pk}_{\mathsf{EG}})$
3 : **ret** $\widetilde{\sigma}_2$

$\mathsf{FPreVerify}_2(\mathsf{pp}, \mathsf{ad}, \mathsf{vk}, m, Y, f, \widetilde{\sigma})$:

1 : Parse $\widetilde{\sigma}$ as $(\widetilde{\sigma}_1, \widetilde{\sigma}_2)$
2 : Parse $\widetilde{\sigma}_1$ as $(\mathsf{pk}_{\mathsf{EG}}, \vec{\mathsf{ct}}, \pi_{\mathsf{eq}})$
3 : **ret** $\mathsf{AS.pVrfy}(\mathsf{vk}, m, \mathsf{pk}_{\mathsf{EG}}, \widetilde{\sigma}_2)$

$\mathsf{Adapt}(\mathsf{ad}, \mathsf{st}, \mathsf{vk}, m, Y, \vec{y}, f, \widetilde{\sigma})$:

1 : Parse st as $(\mathsf{sk}_{\mathsf{FHE}}, r_0, r_1, \mathsf{sk}_{\mathsf{EG}})$
2 : Parse $\widetilde{\sigma}$ as $(\widetilde{\sigma}_1, \widetilde{\sigma}_2)$
3 : $\sigma := \mathsf{AS.Adapt}(\mathsf{vk}, m, \mathsf{sk}_{\mathsf{EG}}, \widetilde{\sigma}_2)$
4 : **ret** σ

$\mathsf{FExt}(\mathsf{ad}, \mathsf{vk}, Y, f, \widetilde{\sigma}, \sigma)$:

1 : Parse $\widetilde{\sigma}$ as $(\widetilde{\sigma}_1, \widetilde{\sigma}_2)$
2 : Parse $\widetilde{\sigma}_1$ as $(\mathsf{pk}_{\mathsf{EG}}, \vec{\mathsf{ct}}, \pi_{\mathsf{eq}})$
3 : Parse $\vec{\mathsf{ct}}$ as $(\mathsf{ct}_0, \ldots, \mathsf{ct}_k)$
4 : $\mathsf{sk}'_{\mathsf{EG}} := \mathsf{AS.Ext}(\mathsf{vk}, \mathsf{pk}_{\mathsf{EG}}, \widetilde{\sigma}_2, \sigma)$
5 : $\forall i \in \{0, \ldots, k\}$:
$b_i := \mathsf{EG.Dec}(\mathsf{sk}'_{\mathsf{EG}}, \mathsf{ct}_i)$
6 : **ret** $z := \sum_{i=0}^{k} 2^i \cdot b_i$

Figure 3.34: Description of algorithms in our HE-based FAS construction.

and adaptive soundness, the encryption schemes FHE *and* EG *satisfy semantic security, the underlying* AS *scheme satisfies witness extractability, and relation* R *is* $\mathcal{F}$*-hard, then the proposed* FAS *construction in Figure 3.34 satisfies the properties of advertisement binding, unforgeability, witness extractability, pre-signature adaptability, and zero-knowledge witness privacy w.r.t.* $\mathcal{F}$.

3.13 Implementation and Evaluation of HE-based FAS

In this section, we present the details of our implementations and evaluations. We implemented all FAS constructions – referred to as FHE FAS, LHE FAS, and BGN FAS – and the SNARK-based strawman to evaluate real-world tradeoffs. All implementations are available at [VGTS25]. We comprehensively benchmark each scheme across various parameters, showing practical performance in real-world scenarios.

Communication Overhead. We note that across all our implementations, only a single Schnorr signature (64 bytes) is put on-chain.

3.13.1 FHE FAS

Our FHE FAS scheme is implemented partly in Python, and partly in Rust. We use the Secp256k1 elliptic curve that is used by Bitcoin [sec] and other cryptocurrencies. We use the curve's python implementation from hanabi1224 [han]. For Schnorr signatures, we use a modified version of BIP-340 reference implementation [bip]. We implement the Schnorr adaptor signatures [AEE+21] on top of it. These parts are borrowed from the open-source FAS implementation of [VST24b]. Further, we implement ElGamal encryption and $\mathsf{NIZK}_{\mathsf{eq}}$ parts related to ElGamal on top of the secp256k1 elliptic curve. For the FHE scheme, we use the TFHE scheme [CGGI20] implemented by Zama: this involves a suite of three libraries: TFHE-rs [tfh] is a Rust library that implements FHE, Concrete [cona] is a Python library that is a layer on top of TFHE-rs to convert high-level circuits into FHE friendly circuits. Further, ConcreteML [conb] is a python library that is a layer on top of Concrete that provides APIs for ML applications. Further, we implement the $\mathsf{NIZK}_{\mathsf{eq}}$ parts related to FHE in Rust. We do not benchmark the one-time $\mathsf{NIZK}_{\mathsf{ad}}$ (from the setup phase of the seller), as this is needed only for AdGen and AdVerify.

For high-degree polynomials and fine-tuning models settings, we use the Concrete and ConcreteML libraries, respectively. Before presenting performance evaluations, we present our core optimization of an efficient REval algorithm for the TFHE scheme [tfh, CGGI20].

Core Optimization: efficient REval

Recall that if the seller's advertisement contains $\mathsf{ct}_{\mathsf{ad}} := \mathsf{FHE.Enc}(\mathsf{pk}_{\mathsf{FHE}}, y; r)$, and the buyer wants to buy function f, then, the $\mathsf{NIZK}_{\mathsf{eq}}$'s witness contains r_f, which the seller must compute using REval before proceeding to compute π_{eq}. As noted in Remark 3.12.1, we want REval to be more efficient than HEval, and we achieve it by using the secret key $\mathsf{sk}_{\mathsf{FHE}}$ and the structure of HEval.

We first recall the TFHE [tfh, CGGI20] scheme in Figure 3.35 that internally relies on three encryption schemes: an LWE-based encryption scheme denoted LWE, a generalized LWE-based encryption scheme denoted GLWE, and a generalized GSW encryption scheme denoted GGSW.

The homomorphic evaluation HEval proceeds as follows: For linear functions f, it is essentially free by mimicking linear operations on the ciphertexts. For non-linear f, HEval uses a combination of programmable bootstrapping followed by key-switching in two steps, as follows: (1) ct_1 is transformed to a GLWE ciphertext $\mathsf{ct}_{2,f}$ under pk_2 encrypting message $f(m)$ by using the bootstrapping key bsk – a GGSW encryption of sk_1 under pk_2. (2) $\mathsf{ct}_{2,f}$ is transformed back to a LWE ciphertext $\mathsf{ct}_{1,f}$ under key pk_1 encrypting message $f(m)$ using the key-switching key ksk – an LWE encryption of the key sk_2 under pk_1. Since $\mathsf{ct}_{1,f}$ is encrypted under pk_1, it can be used again for further evaluations. See Figure 3.36 for details

$\mathsf{TFHE.KGen}(1^\lambda)$:

1 : Compute $(\mathsf{sk}_1, \mathsf{pk}_1) \leftarrow \mathsf{LWE.KGen}(1^\lambda)$
2 : Compute $(\mathsf{sk}_2, \mathsf{pk}_2) \leftarrow \mathsf{GLWE.KGen}(1^\lambda)$
3 : Compute $\mathsf{ksk} := \mathsf{LWE.Enc}(\mathsf{pk}_1, \mathsf{sk}_2; r_{\mathsf{ksk}})$
4 : Compute $\mathsf{bsk} \leftarrow \mathsf{GGSW.Enc}(\mathsf{sk}_2, \mathsf{sk}_1)$
5 : **ret** $\mathsf{pk}_{\mathsf{FHE}} := (\mathsf{pk}_1, \mathsf{pk}_2, \mathsf{ksk}, \mathsf{bsk})$,
$\mathsf{sk}_{\mathsf{FHE}} := (\mathsf{sk}_1, \mathsf{sk}_2, r_{\mathsf{ksk}}, \mathsf{pk}_{\mathsf{FHE}})$

$\mathsf{TFHE.Enc}(\mathsf{pk}_{\mathsf{FHE}}, m)$:

1 : Parse $\mathsf{pk}_{\mathsf{FHE}} = (\mathsf{pk}_1, \mathsf{pk}_2, \mathsf{ksk}, \mathsf{bsk})$
2 : Compute $\mathsf{ct}_2 := \mathsf{GLWE.Enc}(\mathsf{pk}_2, m; r)$
3 : Compute $\mathsf{ct}_1 := \mathsf{KeySwitch}(\mathsf{ksk}, \mathsf{ct}_2)$
4 : **ret** $\mathsf{ct} := (\mathsf{ct}_1, \mathsf{ct}_2)$

$\mathsf{TFHE.Dec}(\mathsf{sk}_{\mathsf{FHE}}, ct)$:

1 : Parse $\mathsf{sk}_{\mathsf{FHE}} = (\mathsf{sk}_1, \mathsf{sk}_2, r_{\mathsf{ksk}}, \mathsf{pk}_{\mathsf{FHE}})$
2 : Parse $\mathsf{ct} = (\mathsf{ct}_1, \mathsf{ct}_2)$
3 : **ret** $\mathsf{GLWE.Dec}(\mathsf{sk}_2, \mathsf{ct}_2)$

Figure 3.35: TFHE encryption scheme from [tfh, CGGI20].

on HEval's non-linear component.

$\mathsf{TFHE.HEval}(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}, f)$:

1 : Parse $\mathsf{pk}_{\mathsf{FHE}} = (\mathsf{pk}_1, \mathsf{pk}_2, \mathsf{ksk}, \mathsf{bsk})$
2 : Parse $\mathsf{ct} = (\mathsf{ct}_1, \mathsf{ct}_2)$
3 : Compute $\mathsf{ct}_{2,f} := \mathsf{Bootstrap}(\mathsf{bsk}, \mathsf{ct}_1, f)$
4 : Compute $\mathsf{ct}_{1,f} := \mathsf{KeySwitch}(\mathsf{ksk}, \mathsf{ct}_{2,f})$
5 : **ret** $\mathsf{ct}_f := (\mathsf{ct}_{1,f}, \mathsf{ct}_{2,f})$

$\mathsf{TFHE.REval}(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{sk}_{\mathsf{FHE}}, \mathsf{ct}, m, r, f, \mathsf{ct}_f)$:

1 : Parse $\mathsf{sk}_{\mathsf{FHE}} = (\mathsf{sk}_1, \mathsf{sk}_2, r_{\mathsf{ksk}}, \mathsf{pk}_{\mathsf{FHE}})$
2 : If $\mathsf{ct}_f = \perp : \mathsf{ct}_f := \mathsf{TFHE.HEval}(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}, f)$
3 : Parse $\mathsf{ct}_f = (\mathsf{ct}_{1,f}, \mathsf{ct}_{2,f})$
4 : Compute $r_{1,f} := \langle r_{\mathsf{ksk}}, \mathsf{ct}_{2,f} \rangle$
5 : **ret** $r_f := r_{1,f}$

Figure 3.36: TFHE.HEval and TFHE.REval for non-linear f in [tfh, CGGI20].

Defining REval **algorithm to compute** r_f**.** Suppose that $\mathsf{ct}_f = (\mathsf{ct}_{1,f}, \mathsf{ct}_{2,f})$ is the final evaluated ciphertext encrypting $f(m)$ where both $\mathsf{ct}_{i,f}$'s encrypt $f(m)$ with randomness, say $r_{i,f}$'s, respectively. Unfortunately, as decryption involves rounding operations which are lossy, we cannot exactly compute $r_{1,f}$ (or $r_{2,f}$) by decrypting $\mathsf{ct}_{1,f}$ (or $\mathsf{ct}_{2,f}$) using $\mathsf{sk}_{\mathsf{FHE}}$. One option is to track how the original randomness evolves through KeySwitch and Bootstrap, but this is as costly as HEval itself—especially since Bootstrap is complex.

Moreover, if f is a composition of k functions, tracking randomness takes $O(k)$ time, common in systems like ConcreteML.

Can we make REval **cheaper than** HEval **and independent of** f**?** We show this is possible by explicitly tracking $r_{1,f}$'s evolution through HEval, which motivates making ct_1 and $\mathsf{ct}_{1,f}$ explicit in our algorithms – despite them being the result of deterministic computations.

The randomness r_f from REval is used to construct the $\mathsf{NIZK}_{\mathsf{eq}}$ proof that ct_f and $\mathsf{ct}_{\mathsf{EG}}$ encrypt the same message. We reduce this proof to showing that $\mathsf{ct}_{1,f} := \mathsf{KeySwitch}(\mathsf{ksk}, \mathsf{ct}_{2,f})$ and $\mathsf{ct}_{\mathsf{EG}}$ encrypt the same message – making the proof more tractable.

Why this transformation? The key insight is that the *randomness in* $\mathsf{ct}_{1,f}$ *depends only on the randomness in* ksk*, not on* $r_{2,f}$ *from* $\mathsf{ct}_{2,f}$*!* This greatly simplifies REval: we only need to track how randomness r_{ksk} in ksk yields $r_{1,f}$, avoiding the need to trace $r_{2,f}$. This shift makes randomness tracking far more efficient.

The reason for this simplification lies in the structure of the KeySwitch operation. Recall that ksk is an LWE ciphertext: it encrypts the secret key sk_2 under the public key pk_1 with randomness r_{ksk}, that is, $\mathsf{ksk} := \mathsf{LWE.Enc}(\mathsf{pk}_1, \mathsf{sk}_2; r_{\mathsf{ksk}})$. The KeySwitch operation computes $\mathsf{ct}_{1,f}$ as an inner product of ksk and $\mathsf{ct}_{2,f}$: that is, $\mathsf{ct}_{1,f} := \langle \mathsf{ksk}, \mathsf{ct}_{2,f} \rangle$, where ksk is treated a vector of ciphertexts and $\mathsf{ct}_{2,f}$ is treated as a vector of known constants. Thus, the entire computation is simply a linear homomorphic evaluation on the ciphertext.

To track the randomness in $\mathsf{ct}_{1,f}$, the REval algorithm computes the inner product of the known constants $\mathsf{ct}_{2,f}$ with the randomness r_{ksk} from ksk as described in Figure 3.36. We emphasize that this is remarkably more efficient than HEval, and is independent of the description of non-linear functions f.

Applications

We demonstrate the practicality of our implementations via two main applications related to data analytics and ML fine-tuning.

High-degree polynomial evaluations. Suppose a seller holds a proprietary ML model, and the buyer wants to obtain an inference on some data using the seller's model. Currently, all business scenarios involve some trust assumption between the buyer and seller, such as pay first and get service later, or vice versa. In trustless scenarios, an FAS-based approach is warranted. Popular ML models involve non-linear functions (example, exp, softmax, ReLU, GeLU) that are not crypto-friendly, so private inference relies on polynomial approximations—example, degree-4 softmax [CZK+23], and degree-3 GeLU [ZYH+24, LHG+23, DLZ+23]. We support degree-3 and degree-4 polynomial evaluations and report performance in Figure 3.37.

Fine-tuning Machine Learning models. In a dual setting, a seller holds a proprietary dataset, and a buyer

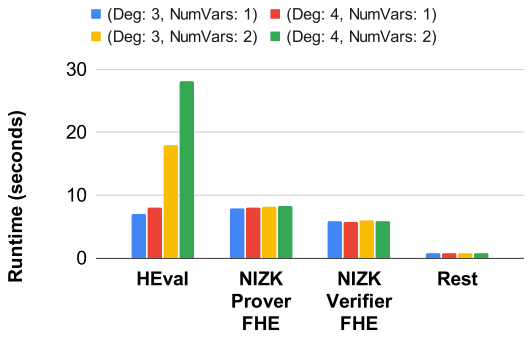


Figure 3.37: Runtime of FHE-FAS algorithms for polynomial evaluations. We plot times for HEval, NIZK Prover and Verifier. "Rest" includes $\mathsf{FPreSign}_2$, $\mathsf{FPreVerify}_2$, Adapt, and Vf.

with a pre-trained model wants to fine-tune it. Due to privacy laws like HIPAA, raw data (example, medical records) can't be sold, but function evaluations useful for training may be allowed. Our FAS protocol supports this setting. We demonstrate this by training an SGD classifier on the Iris dataset using a log-loss function, with results shown in Figure 3.38. As dataset size grows, homomorphic evaluation dominates the cost, while other FAS components remain constant.

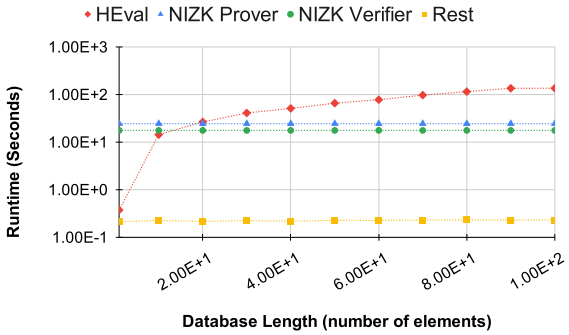


Figure 3.38: Runtime of FHE-FAS algorithms for ML model fine-tuning model task. See Figure 3.37's caption for descriptions of reported algorithms. NIZK times are estimates based on Figure 3.37.

Comparison. We compare the performance of our FHE-based FAS scheme with SNARK-based FAS scheme. Recall that our design goal for FAS is to keep the seller lightweight (critical for ML fine-tuning and scalability, as stated above). Accordingly, our FHE-FAS scheme places only a lightweight PoPE on the seller, independent of dataset size, while shifting FHE evaluation to the buyer. By contrast, the SNARK-based approach requires a seller-side prover whose cost grows linearly with dataset size. In the following evaluation, we compute $\sum_i a_i \cdot x_i^3$ where $i \in [N]$, $(x_1, \ldots, x_N)$ is the seller's dataset and $(a_1, \ldots, a_N)$ describes the buyer's function. All variables and co-efficients are random integers between $0 - 10$. Seller and buyer times are reported below.

N = 1000. SNARK approach: seller: 5.111s (SNARK Prover) , buyer: 0.007s (SNARK Verify), total 5.118s.

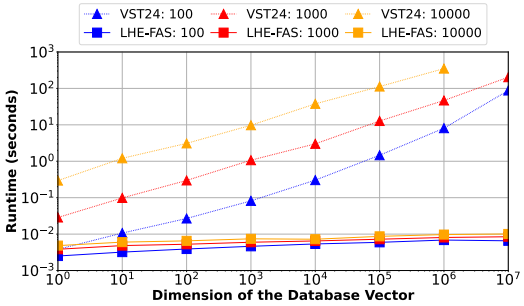


Figure 3.39: Runtime of FExt for VST24 and LHE-FAS for varying witness bounds. For example, "VST24: 100" means VST24 with a witness bound of 100.

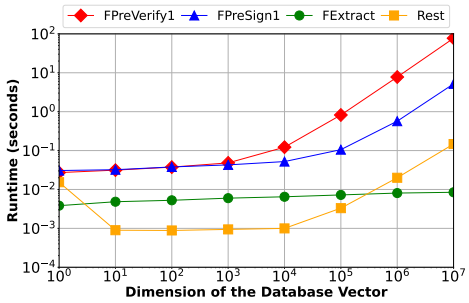


Figure 3.40: Runtime of LHE-FAS algorithms for different database sizes. We report times for $\mathsf{FPreVerify}_1$, $\mathsf{FPreSign}_1$, FExt, with "Rest" including total combined time for $\mathsf{FPreSign}_2$, $\mathsf{FPreVerify}_2$, Adapt, and Vf.

FHE approach: seller: 8.224s (0.150s FHE Decrypt, 8.074s NIZK Prover) , buyer: 70.862s (65.029s FHE Eval, 5.833s NIZK Verify), total 79.086s.

N = 10000. SNARK approach: seller: 41.355s (SNARK Prover) , buyer: 0.030s (SNARK Verify), total 41.385s. FHE approach: seller: 8.224s (0.150s FHE Decrypt, 8.074s NIZK Prover) , buyer: 754.453s (748.620s FHE Eval, 5.833s NIZK Verify), total 762.677s.

These results confirm our hypothesis: SNARKs favor the seller on small datasets ($N = 10^3$), while for larger ones ($N \geq 10^4$) FHE is superior since seller time stays $\sim 8s$. At $N = 10^4$, the FHE seller is $\sim 5\times$ faster though the overall protocol is $\sim 18\times$ slower. As N increases, the gap between our seller and SNARK seller becomes even more pronounced: example, at $N = 10^5$, we expect $50\times$ faster seller but an overall $20\times$ slower protocol[22].

3.13.2 LHE FAS

We implemented LHE FAS in Go, and the $\mathsf{NIZK}_{\mathsf{eq}}$ is implemented using the Schnorr PoK protocol. We benchmark running times by varying the length of the witness vector y from 1 to 10^6. Figure 3.40 indicates that the seller is very efficient with sub-5 second runtime under all scenarios. For the buyer, FExt is very

[22]Our FHE-based FAS implementation uses FHE in a black-box manner, and does not optimize it. The main bottleneck is FHE evaluation and recent line of works [JCT+25, ACJ24, KKC+23, SS24] try to improve its performance. Such advances in FHE will directly improve our FAS performance as well.

fast, as in Figure 3.39. Even so, our buyer exhibits some tradeoffs when compared with VST24. When each witness and function entry is bounded by 10^2, our buyer matches VST24 at vector length 10^6. At bounds of 10^3 and 10^4, our buyer is 2×–100× and 12×–100× faster, respectively.

These tradeoffs stem from our buyer performing 2 MSMs, while the buyer in [VST24a] performs 1 MSM and 1 discrete log. For vector length ℓ and bound B, MSM costs $O(\ell \log B)$, and discrete log costs $O(\sqrt{\ell} \cdot B)$. Moreover, our 2 MSMs can be parallelized, unlike the sequential MSM and discrete log in [VST24a]. With parallelism, our buyer outperforms VST24 in all cases.

3.14 Functional Fair Exchange (FFE)

We define Functional Fair Exchange (FFE) in an advertisement-based setting, and show that any FAS scheme implies an FFE protocol. Our definition is inspired by Della Monica et al. [DMVVZ25], and models FFE as a three-party protocol: Alice (seller) advertises an asset, Bob (buyer) seeks a function evaluation in exchange for payment, and T is a trusted third party (TTP).

Notations. For PPT algorithms $\mathsf{A}, \mathsf{B}, \mathsf{C}$, we write $[x, y, z] \leftarrow \mathsf{A}(a), \mathsf{B}(b), \mathsf{C}(c)$ to denote their interaction on common input d, private inputs a, b, c, and independent randomness, where x, y, z are their respective outputs. Inspired by [BK14, DMVVZ25], we define tokens with two properties: (i) exclusive ownership – only one user can hold a token at a time, and (ii) transferable and verifiable – tokens can be freely transferred, and recipients can instantly verify their validity. We assume that each user A owns a wallet W_A. $\mathcal{T}$ denotes the set of all possible tokens with the same value. $\mathsf{tk}[n] := \{\mathsf{tk}_1, \ldots, \mathsf{tk}_n\}$ denotes a set of n tokens such that $\mathsf{tk}_i \in \mathcal{T}$ for all $i \in [n]$. $\mathsf{tk}[n] \in \mathsf{W}_\mathsf{A}$ denotes that the tokens $\mathsf{tk}[n]$ are by W_A. Let $\mathsf{owner}(\mathsf{tk}[n], \mathsf{W})$ be a predicate that returns 1 if all tokens $\{\mathsf{tk}_i\}_{i \in [n]}$ are in wallet W, and 0 otherwise.

Definition 3.14.1. *A FFE protocol Π_{FFE} for function class $\mathcal{F}$ is a three-party protocol run by a p.p.t. seller A (with wallet W_A), a p.p.t. buyer B (with wallet W_B) and a trusted-third party T satisfying Completeness, Advertisement Binding, and Buyer- and Seller- Fairness with following algorithms:*

- $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$*: outputs random public parameters* $\mathsf{pp} \in \{0,1\}^*$ *on input a security parameter* λ*.*
- $(x_{\mathsf{prv}}, x_{\mathsf{pub}}) \leftarrow \mathsf{AdGen}(\mathsf{pp}, x, \mathsf{tk}_{\mathsf{ask}})$*: Seller* A *runs this randomized algorithm that takes as input public parameter* pp*, an asset* $x \in \{0,1\}^*$*, and* $\mathsf{tk}_{\mathsf{ask}} \in \mathbb{N}^+$*, and outputs an advertisement* x_{pub} *to buyer* B *and stores internal state* x_{prv}*.*
- $b = \mathsf{AdVerify}(\mathsf{pp}, x_{\mathsf{pub}})$*: Buyer* B *runs this deterministic algorithm that takes as input the public parameter* pp*, and advertisement* x_{pub} *and outputs a bit* $b \in \{0,1\}$*. If* $b = 0$*, buyer* B *aborts the protocol.*
- $[\alpha, \beta] \leftarrow [\mathsf{A}(x_{\mathsf{prv}}), \mathsf{B}(\mathsf{tk}[n]), \mathsf{T}](\mathsf{pp}, x_{\mathsf{pub}}, f)$*: the exchange phase is a three party protocol where* B *wants to learn* $f(x)$ *from* A *in exchange for tokens* $\mathsf{tk}[n]$ *and the trusted third-party* T *assists them in this protocol. Alice's private*

inputs are x_{prv} and Bob's private input are $\mathsf{tk}[n] \in \mathsf{W_B}$. Without loss of generality, we say the exchange phase is R rounds where, for all $i \in [R]$: the first message is from the seller A and second message is from the buyer B. After R rounds, there is just a single message from the seller A that terminates the protocol. At the end of the protocol, A's output is $\alpha \in \{0,1\}$ defined as the predicate evaluation $\alpha := \mathsf{owner}(\mathsf{tk}[n], \mathsf{W_A})$ and B's output is β.

Completeness. *Given an honest seller A with wallet $\mathsf{W_A}$ and an honest buyer B with wallet $\mathsf{W_B}$ such that $\mathsf{tk}[n] \in \mathsf{W_B}$, for any asset $x \in \{0,1\}^*$, any $\mathsf{tk_{ask}} \in \mathbb{N}^+$, and any $f \in \mathcal{F}$: suppose $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$, $(x_{\mathsf{prv}}, x_{\mathsf{pub}}) \leftarrow \mathsf{AdGen}(\mathsf{pp}, x, \mathsf{tk_{ask}})$, $[\alpha, \beta] \leftarrow [\mathsf{A}(x_{\mathsf{prv}}), \mathsf{B}(\mathsf{tk}[n]), \mathsf{T}](\mathsf{pp}, x_{\mathsf{pub}}, f)$, then $\Pr[\mathsf{AdVerify}(\mathsf{pp}, x_{\mathsf{pub}}) = 1 \wedge \alpha = 1 \wedge \beta = f(x)] = 1$.*

Advertisement Binding. *There exists a negligible function negl such that for all p.p.t. and stateful malicious seller A^*, for any $\lambda \in \mathbb{N}$ if $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$, $x_{\mathsf{pub}} \leftarrow \mathsf{A}^*(\mathsf{pp})$, such that $\mathsf{AdVerify}(\mathsf{pp}, x_{\mathsf{pub}}) = 1$, then both the following hold: Informally, both equations ensure that any (even malicious) x_{pub} has a unique tuple $(x, \mathsf{tk_{ask}}, r)$ mapping to it via AdGen. This tuple can be found via brute-force by an extractor $\mathcal{E}$ which we used below.*

$$\Pr\left[\begin{array}{c} \exists (x, \mathsf{tk_{ask}}, r), (x', \mathsf{tk'_{ask}}, r') \; s.t. \\ x \neq x' \wedge \\ \mathsf{AdGen}(\mathsf{pp}, x, \mathsf{tk_{ask}}; r) = (\cdot, x_{\mathsf{pub}}) \wedge \\ \mathsf{AdGen}(\mathsf{pp}, x', \mathsf{tk'_{ask}}; r') = (\cdot, x_{\mathsf{pub}}) \end{array}\right] \leq \mathsf{negl}(\lambda) \; ,$$

$$\Pr\left[\begin{array}{c} \nexists (x, \mathsf{tk_{ask}}, r) \; s.t. \\ \mathsf{AdGen}(\mathsf{pp}, x, \mathsf{tk_{ask}}; r) = (\cdot, x_{\mathsf{pub}}) \end{array}\right] \leq \mathsf{negl}(\lambda) \; .$$

Buyer Fairness. *There exists an extractor algorithm (possibly inefficient) $\mathbf{E}$ and there exists a negligible function negl such that for any honest buyer B with wallet $\mathsf{W_B}$ and $\mathsf{tk}[n] \in \mathsf{W_B}$, for all p.p.t. and stateful malicious seller A^*, for any $\lambda \in \mathbb{N}$, for any $f \in \mathcal{F}$, it holds that: if $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$, $x_{\mathsf{pub}} \leftarrow \mathsf{A}^*(\mathsf{pp})$ such that $\mathsf{AdVerify}(\mathsf{pp}, x_{\mathsf{pub}}) = 1$, $[\alpha, \beta] \leftarrow [\mathsf{A}^*, \mathsf{B}(\mathsf{tk}[n]), \mathsf{T}](\mathsf{pp}, x_{\mathsf{pub}}, f)$ then the following is $\leq \mathsf{negl}(\lambda)$:*

$$\Pr\left[\begin{array}{c} \exists i \in [n] \; s.t. \; \mathsf{tk}_i \notin \mathsf{W_B} \\ \wedge \; \beta \neq f(x^*) \end{array} : x^* := \mathbf{E}(\mathsf{pp}, x_{\mathsf{pub}})\right] .$$

Seller Fairness. *Suppose the exchange phase contains R rounds with seller and buyer speaking once in each round with the interaction concluding with a single message from the seller. There exists a negligible function negl such that for any honest seller A, asset $x \in \{0,1\}^*$ and $\mathsf{tk_{ask}} \in \mathbb{N}^+$, any p.p.t. malicious buyer B^* with wallet $\mathsf{W_B}$ such that $\mathsf{tk}[n] \in \mathsf{W_B}$ and $f \in \mathcal{F}$, seller fairness requires:*

- *there exists a p.p.t. stateful simulator $\mathsf{Sim} = (\mathsf{Sim}_0, \mathsf{Sim}_1, \mathsf{Sim}_2)$ such that for any p.p.t. distinguisher $\mathcal{D}$, the difference in $\Pr[\mathcal{D}(\mathsf{V_{B^*}}) = 1 : \mathsf{V_{B^*}} \leftarrow \mathsf{SFReal}_{\mathsf{B}^*,\mathsf{T}}(1^\lambda, x, \mathsf{tk_{ask}}, f)]$ and $\Pr[\mathcal{D}(\mathsf{V_{B^*}}) = 1 : \mathsf{V_{B^*}} \leftarrow$*

$\mathsf{SFReal}_{\mathsf{B}^*,\mathsf{T}}(1^\lambda, x, \mathsf{tk}_{\mathsf{ask}}, f)$:

1 : $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$
2 : $(x_{\mathsf{prv}}, x_{\mathsf{pub}}) \leftarrow \mathsf{AdGen}(\mathsf{pp}, x, \mathsf{tk}_{\mathsf{ask}})$
3 : $[\alpha, \beta] \leftarrow [\mathsf{A}(x_{\mathsf{prv}}), \mathsf{B}^*(\mathsf{tk}[n]), \mathsf{T}](\mathsf{pp}, x_{\mathsf{pub}}, f)$
4 : **ret** $\mathsf{V}_{\mathsf{B}^*}$

$\mathsf{SFIdeal}^{\mathsf{Sim}}_{\mathsf{B}^*,\mathsf{T}}(1^\lambda, x, \mathsf{tk}_{\mathsf{ask}}, f)$:

1 : $\mathsf{pp}^* \leftarrow \mathsf{Sim}_0(1^\lambda)$
2 : $x^*_{\mathsf{pub}} \leftarrow \mathsf{Sim}_1(\mathsf{pp}^*, \mathsf{tk}_{\mathsf{ask}})$
3 : $[\alpha, \beta] \leftarrow [\mathsf{Sim}_2(f(x)), \mathsf{B}^*(\mathsf{tk}[n]), \mathsf{T}](\mathsf{pp}^*, x^*_{\mathsf{pub}}, f)$
4 : **ret** $\mathsf{V}_{\mathsf{B}^*}$

$\mathsf{SFReal}^i_{\mathsf{B}^*,\mathsf{T}}(1^\lambda, x, \mathsf{tk}_{\mathsf{ask}}, f)$:

1 : $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$
2 : $(x_{\mathsf{prv}}, x_{\mathsf{pub}}) \leftarrow \mathsf{AdGen}(\mathsf{pp}, x, \mathsf{tk}_{\mathsf{ask}})$
3 : $[\alpha, \beta] \leftarrow [\mathsf{A}(x_{\mathsf{prv}}), \mathsf{B}^*(\mathsf{tk}[n]), \mathsf{T}](\mathsf{pp}, x_{\mathsf{pub}}, f)$
4 : **ret** $\mathsf{V}_{\mathsf{B}^*}$

$\mathsf{SFIdeal}^{i,\mathsf{Sim}^i}_{\mathsf{B}^*,\mathsf{T}}(1^\lambda, x, \mathsf{tk}_{\mathsf{ask}}, f)$:

1 : $\mathsf{pp}^* \leftarrow \mathsf{Sim}^i_0(1^\lambda)$
2 : $x^*_{\mathsf{pub}} \leftarrow \mathsf{Sim}^i_1(\mathsf{pp}^*, \mathsf{tk}_{\mathsf{ask}})$
3 : $[\alpha, \beta] \leftarrow [\mathsf{Sim}^i_2, \mathsf{B}^*(\mathsf{tk}[n]), \mathsf{T}](\mathsf{pp}^*, x^*_{\mathsf{pub}}, f)$
4 : **ret** $\mathsf{V}_{\mathsf{B}^*}$

Figure 3.41: SFReal, SFIdeal, SFReal^i and $\mathsf{SFIdeal}^i$ games for Seller Fairness.

$\mathsf{SFIdeal}^{\mathsf{Sim}}_{\mathsf{B}^*,\mathsf{T}}(1^\lambda, x, \mathsf{tk}_{\mathsf{ask}}, f)] \leq \mathsf{negl}(\lambda)$, *where* SFReal *and* SFIdeal *are games defined as in Figure 3.41 and* $\mathsf{V}_{\mathsf{B}^*}$ *is* B^* *view that includes all messages and its internal state.*

- *for all* $i \in [R]$, *there exists a p.p.t. stateful simulator* $\mathsf{Sim}^i = (\mathsf{Sim}^i_0, \mathsf{Sim}^i_1, \mathsf{Sim}^i_2)$ *such that for any p.p.t. distinguisher* $\mathcal{D}$, *the difference in* $\Pr[\mathcal{D}(\mathsf{V}_{\mathsf{B}^*}) = 1 : \mathsf{V}_{\mathsf{B}^*} \leftarrow \mathsf{SFReal}^i_{\mathsf{B}^*,\mathsf{T}}(1^\lambda, x, \mathsf{tk}_{\mathsf{ask}}, f)]$ *and* $\Pr[\mathcal{D}(\mathsf{V}_{\mathsf{B}^*}) = 1 : \mathsf{V}_{\mathsf{B}^*} \leftarrow \mathsf{SFIdeal}^{i,\mathsf{Sim}^i}_{\mathsf{B}^*,\mathsf{T}}(1^\lambda, x, \mathsf{tk}_{\mathsf{ask}}, f)]$ *is* $\leq \mathsf{negl}(\lambda)$, *where* SFReal^i *game (defined in Figure 3.41) is real-world game where the buyer* B^* *aborts after receiving* i^{th} *message of the exchange phase and* $\mathsf{SFIdeal}^i$ *game (defined in Figure 3.41) is ideal-world game where the simulator simulates the buyer* B^**'s view upto* i *rounds of the exchange phase and* $\mathsf{V}_{\mathsf{B}^*}$ *is defined as the view of* B^* *in the two games respectively.*

Crucially, Sim_2 in SFIdeal game gets $f(x)$ as input, whereas for all $i \in [R]$, Sim^i_2 in $\mathsf{SFIdeal}^i$ does not.

Remark 3.14.2. *In this MPC-like setting, the malicious buyer* B *has no private input and specifies* f*, after which the honest seller already knows* $f(x)$*. If* B *aborts, it learns nothing; otherwise, it learns* $f(x)$*. This differs from standard MPC, where the malicious party sees the output first and decides whether the honest party learns it—hence, requiring a different ideal functionality.*

3.14.1 FFE protocol from FAS

In this section, we formalize the FFE protocol obtained from FAS that was informally described in Section 4.1 and in [VST24a]. Let FAS be defined w.r.t. a digital signature scheme DS, a hard relation R, and a family of functions $\mathcal{F}$. Then, FFE for $\mathcal{F}$ is as follows, where the role of T is played by a blockchain and B owns a key pair $(\mathsf{sk}, \mathsf{vk}) \leftarrow \mathsf{DS.KGen}(1^\lambda)$ where vk is public.

- $\mathsf{Setup}(1^\lambda)$: **ret** $\mathsf{pp} \leftarrow \mathsf{FAS.Setup}(1^\lambda)$.
- $\mathsf{AdGen}(\mathsf{pp}, x, \mathsf{tk}_{\mathsf{ask}})$: A computes X ensuring $(X, x) \in R$, and computes $(\mathsf{ad}, \mathsf{st}) \leftarrow \mathsf{FAS.AdGen}(\mathsf{pp}, X, x)$ to send $x_{\mathsf{pub}} := (X, \mathsf{ad}, \mathsf{tk}_{\mathsf{ask}})$ to B and keep $x_{\mathsf{prv}} := (x, \mathsf{st})$ locally.
- $\mathsf{AdVerify}(\mathsf{pp}, x_{\mathsf{pub}})$ is run by B and it computes $\mathsf{FAS.AdVerify}(\mathsf{pp}, X, \mathsf{ad})$. If the output is 0, B aborts, else it continues to the exchange phase described next.
- $[\alpha, \beta] \leftarrow [\mathsf{A}(x_{\mathsf{prv}}), \mathsf{B}(\mathsf{tk}[n]), \mathsf{T}](\mathsf{pp}, x_{\mathsf{pub}}, f, \mathsf{vk})$:
  1. A computes and sends $\widetilde{\sigma_1}$ to B where $\widetilde{\sigma_1} \leftarrow\!\!\$\ \mathsf{FAS.FPreSign}_1(\mathsf{pp}, \mathsf{ad}, \mathsf{st}, X, x, f)$.
  2. B aborts if $\mathsf{FAS.FPreVerify}_1(\mathsf{pp}, \mathsf{ad}, X, f, \widetilde{\sigma_1}) = 0$. Else, B computes $\widetilde{\sigma_2} \leftarrow \mathsf{FPreSign}_2(\mathsf{pp}, \mathsf{ad}, \mathsf{sk}, m, X, f, \widetilde{\sigma_1})$ where $m :=$ "transfer $\mathsf{tk}[n]$ from $\mathsf{W_B}$ to $\mathsf{W_A}$" and $n := \mathsf{tk}_{\mathsf{ask}}$. B sends $\widetilde{\sigma_2}$ to A.
  3. A aborts if $\mathsf{FAS.FPreVerify}_2(\mathsf{pp}, \mathsf{ad}, \mathsf{vk}, m, X, f, \widetilde{\sigma} := (\widetilde{\sigma_1}, \widetilde{\sigma_2})) = 0$ or if the number of tokens n in the message m satisfy $n < \mathsf{tk}_{\mathsf{ask}}$. Else, A computes $\sigma \leftarrow \mathsf{FAS.Adapt}(\mathsf{ad}, \mathsf{st}, \mathsf{vk}, m, X, x, f, \widetilde{\sigma} := (\widetilde{\sigma_1}, \widetilde{\sigma_2}))$ and sends the signed transaction (m, σ) to the blockchain T.
     - T rejects the signed transaction (m, σ) if $\mathsf{DS.Vf}(\mathsf{vk}, m, \sigma) = 0$, else it accepts and transfers tokens $\mathsf{tk}[n]$ from $\mathsf{W_B}$ to $\mathsf{W_A}$.
     - When B sees signed transaction (m, σ) posted to the blockchain T, it computes $\beta := \mathsf{FAS.FExt}(\mathsf{ad}, \mathsf{vk}, X, f, \widetilde{\sigma} := (\widetilde{\sigma_1}, \widetilde{\sigma_2}), \sigma)$.

     The protocol ends at this point and A's output is $\alpha := \mathsf{owner}(\mathsf{tk}[n], \mathsf{W_A})$ and B's output is β.

Correctness. For honest A and B, the correctness of the FFE protocol follows from that of FAS.

Theorem 3.14.3. *The FFE protocol in Section 3.14.1 satisfies:*

- *advertisement soundness when* FAS *satisfies advertisement soundness.*
- *buyer fairness when* FAS *satisfies witness extractability and* DS *is unforgeable.*
- *seller fairness when* FAS *satisfies zero-knowledge (non-abort) and zero-knowledge (abort). , then the FFE protocol satisfies seller fairness.*

3.14.2 FAS security definitions: Informal

We will informally define the zero-knowledge property of FAS that we only alluded to in Section 3.5.

Definition 3.14.4 ((Informal) Zero Knowledge (non-abort)). *A* FAS *scheme satisfies zero knowledge (non-abort) if there exists a negligible function ϵ such that for every stateful p.p.t. malicious buyer $\mathcal{A}$, for all $\lambda \in \mathbb{N}$, for all $(Y, y) \in \mathsf{R}$: there exists a stateful p.p.t. simulator $\mathsf{Sim} = (\mathsf{Setup}^*, \mathsf{AdGen}^*, \mathsf{FPreSign}_1^*, \mathsf{Adapt}^*)$ such that for any p.p.t. distinguisher $\mathcal{D}$, the difference in $\Pr\left[\mathcal{D}\left(\mathsf{V}_{\mathcal{A}}\right) = 1 : \mathsf{V}_{\mathcal{A}} \leftarrow \mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right]$ and $\Pr\left[\mathcal{D}\left(\mathsf{V}_{\mathcal{A}}\right) = 1 : \mathsf{V}_{\mathcal{A}} \leftarrow \mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right]$ is $\leq \epsilon(\lambda)$, where $\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}$ and $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$ are described next and $\mathsf{V}_{\mathcal{A}}$ is defined as $\mathcal{A}$'s view in the two games.* $\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}$ *is the real-world experiment between an honest seller and a malicious buyer. The seller computes* $(\mathsf{pp}, \mathsf{ad}, Y)$ *using* $\mathsf{Setup}, \mathsf{AdGen}$ *and provides these to the seller. On obtaining* (m, f, vk) *from the buyer, the seller computes* $\widetilde{\sigma_1}$ *using* $\mathsf{FPreSign}_1$ *and sends it. On obtaining* $\widetilde{\sigma_2}$ *from the buyer, the seller computes* σ *using* Adapt *and posts it to the blockchain to get paid if* $\mathsf{FPreVerify}_2$ *passes, else it aborts protocol.* $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$ *is same as the real-world experiment except each of the 4 algorithms mentioned above (except* $\mathsf{FPreVerify}_2$*) are replaced by the simulation algorithms. Crucially,* $f(x)$ *is the only information about* x *that the simulation algorithms* $\mathsf{FPreSign}_1^*, \mathsf{Adapt}^*$ *get as input.*

Definition 3.14.5 ((Informal) Zero Knowledge (abort)). *A* FAS *scheme satisfies zero knowledge (abort) if there exists a negligible function ϵ such that for every stateful p.p.t. malicious buyer $\mathcal{A}$, for all $\lambda \in \mathbb{N}$, for all $(Y, y) \in \mathsf{R}$: there exists a stateful p.p.t. simulator $\mathsf{Sim}^{\mathsf{a}} = (\mathsf{Setup}^{*,\mathsf{a}}, \mathsf{AdGen}^{*,\mathsf{a}}, \mathsf{FPreSign}_1^{*,\mathsf{a}})$ such that for any p.p.t. distinguisher $\mathcal{D}$, the difference in $\Pr\left[\mathcal{D}\left(\mathsf{V}_{\mathcal{A}}\right) = 1 : \mathsf{V}_{\mathcal{A}} \leftarrow \mathsf{fZKReal}^{\mathsf{a}}{}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right]$ and $\Pr\left[\mathcal{D}\left(\mathsf{V}_{\mathcal{A}}\right) = 1 : \mathsf{V}_{\mathcal{A}} \leftarrow \mathsf{fZKIdeal}^{\mathsf{a},\mathsf{Sim}^{\mathsf{a}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right]$ is $\leq \epsilon(\lambda)$, where $\mathsf{fZKReal}^{\mathsf{a}}_{\mathcal{A},\mathsf{FAS}}$ is real-world game where the adversary $\mathcal{A}$ aborts after receiving $\widetilde{\sigma_1}$ and $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$ is ideal-world game where the simulator simulates the adversary $\mathcal{A}$'s view upto receiving $\widetilde{\sigma_1}$ and $\mathsf{V}_{\mathcal{A}}$ is defined as the view of $\mathcal{A}$ in the two games respectively.*

Specifically, $\mathsf{fZKReal}^{\mathsf{a}}_{\mathcal{A},\mathsf{FAS}}$ *is same as* $\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}$ *except the adversary* $\mathcal{A}$ *aborts after receiving* $\widetilde{\sigma_1}$. $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$ *is same as* $\mathsf{fZKReal}^{\mathsf{a}}_{\mathcal{A},\mathsf{FAS}}$ *except each algorithms* $\mathsf{Setup}, \mathsf{AdGen}, \mathsf{FPreSign}_1$ *are replaced by the respective simulation algorithms. Crucially, no information about* x *is provided to any simulation algorithm.*

3.14.3 Proof Sketch of Theorem 3.14.3

In this section, we discuss buyer- and seller-fairness. We note that advertisement binding of FFE follows from advertisement binding of FAS naturally.

Buyer Fairness. The FFE extractor algorithm $\mathcal{E}$ is essentially the same as the FAS extractor algorithm $\mathcal{E}$ associated with advertisement binding. Suppose its output is x^*. Then, it suffices to analyze all the cases when honest buyer's output $\beta \neq f(x^*)$. In each of the cases, we will argue that if $\exists i \in [n] s.t. \mathsf{tk}_i \notin \mathsf{W_B}$, then, we can break security of one of the underlying primitives. There are three cases here:

1. Case 1: B aborted because AdVerify failed and in this case $\beta = \perp$ canonically.

2. Case 2: B aborted because in the exchange phase $\mathsf{FAS.FPreVerify}_1(\mathsf{pp}, \mathsf{ad}, X, f, \widetilde{\sigma_1}) = 0$. In this case, $\beta = \perp$ canonically.
3. Case 3: B sent $\widetilde{\sigma_2}$ to A in step 2 of exchange phase and A posted σ on the blockchain T to get the tokens $\mathsf{tk}[n]$ and then B computed $\beta := \mathsf{FAS.FExt}(\mathsf{ad}, \mathsf{vk}, X, f, \widetilde{\sigma} := (\widetilde{\sigma_1}, \widetilde{\sigma_2}), \sigma)$, but it is the case that $\beta \neq f(x^*)$.

For cases 1 and 2, if $\exists i \in [n] s.t.\ \mathsf{tk}_i \notin \mathsf{W_B}$, then, we can design a simple reduction that breaks unforgeability of the digital signature scheme DS with respect to which FAS is defined. For case 3, if $\exists i \in [n] s.t. \mathsf{tk}_i \notin \mathsf{W_B}$, then, we can design a simple reduction that breaks the witness extractability of the FAS scheme.

Seller Fairness. Observe that the FAS-based FFE protocol has $R = 1$ number of rounds in the exchange phase. So, we need to show malicious buyer B^*'s view $\mathsf{V_{B^*}}$ (i) in SFReal and $\mathsf{SFIdeal}$ games is indistinguishable and (ii) in SFReal^1 and $\mathsf{SFIdeal}^2$ games is indistinguishable. We prove the former using the zero-knowledge (non-abort) property of FAS and the latter by using the zero-knowledge (abort) property of FAS.

1. We describe how to construct the simulator $\mathsf{Sim} = (\mathsf{Sim}_0, \mathsf{Sim}_1, \mathsf{Sim}_2)$ by using the FAS zero-knowledge (non-abort) simulator $\mathsf{FAS.Sim} = (\mathsf{FAS.Setup}^*, \mathsf{FAS.AdGen}^*, \mathsf{FAS.FPreSign}_1^*, \mathsf{FAS.Adapt}^*)$. $\mathsf{Sim}_0(1^\lambda)$ simply outputs $\mathsf{pp}^* \leftarrow \mathsf{FAS.Setup}^*(1^\lambda)$. $\mathsf{Sim}_1(\mathsf{pp}^*, \mathsf{tk_{ask}})$ samples $X \in_R$, runs $(\mathsf{ad}^*, \mathsf{st}^*) \leftarrow \mathsf{FAS.AdGen}^*(\mathsf{pp}^*, X)$ and outputs $x^*_{\mathsf{pub}} := (X, \mathsf{ad}^*, \mathsf{tk_{ask}})$. $\mathsf{Sim}_2(f(x))$ computes $\widetilde{\sigma_1^*} \leftarrow \mathsf{FAS.FPreSign}_1^*(\mathsf{pp}^*, \mathsf{ad}^*, \mathsf{st}^*, X, f(x), f)$ and sends $\widetilde{\sigma_1^*}$ to B^* in step 1 of exchange phase. In step 3 of exchange phase, it computes $\sigma := \mathsf{FAS.Adapt}^*(\mathsf{ad}^*, \mathsf{st}^*, \mathsf{vk}, m, X, f(x), f, (\widetilde{\sigma_1^*}, \widetilde{\sigma_2}))$ and posts σ to the blockchain T. From the description of the FFE simulator, we see that if FAS satisfies zero-knowledge (non-abort), then the FFE protocol satisfies seller fairness' first requirement.
2. We describe how to construct the simulator $\mathsf{Sim}^{\mathsf{a}} = (\mathsf{Sim}_0^{\mathsf{a}}, \mathsf{Sim}_1^{\mathsf{a}}, \mathsf{Sim}_2^{\mathsf{a}})$ by using the FAS zero-knowledge (non-abort) simulator $\mathsf{FAS.Sim} = (\mathsf{FAS.Setup}^{*,\mathsf{a}}, \mathsf{FAS.AdGen}^{*,\mathsf{a}}, \mathsf{FAS.FPreSign}_1^{*,\mathsf{a}})$. $\mathsf{Sim}_0^{\mathsf{a}}, \mathsf{Sim}_1^{\mathsf{a}}$ use $\mathsf{FAS.Setup}^{*,\mathsf{a}}, \mathsf{FAS.AdGen}^{*,\mathsf{a}}$ like in the previous case. $\mathsf{Sim}_2^{\mathsf{a}}$ computes $\widetilde{\sigma_1^*} \leftarrow \mathsf{FAS.FPreSign}_1^{*,\mathsf{a}}(\mathsf{pp}^*, \mathsf{ad}^*, \mathsf{st}^*, X, f)$ and sends $\widetilde{\sigma_1^*}$ to B^* in step 1 of the exchange. From the description of the FFE simulator, we see that if FAS satisfies zero-knowledge (abort), then the FFE protocol satisfies seller fairness' second requirement.

3.15 Proofs: FAS from FHE

Correctness. If everything is run honestly, then observe that $\mathsf{AdVerify}(\mathsf{pp}, Y, y) = 1$ due to completeness of $\mathsf{NIZK_{ad}}$, $\mathsf{FPreVerify}_1(\mathsf{pp}, \mathsf{ad}, m, Y, f, \widetilde{\sigma_1}) = 1$ due to completeness of $\mathsf{NIZK_{eq}}$, $\mathsf{FPreVerify}_2(\mathsf{pp}, \mathsf{ad}, \mathsf{vk}, m, Y, f, \widetilde{\sigma}) = 1$ due to correctness of the adaptor signature AS, and lastly $\mathsf{FExt}(\mathsf{ad}, \mathsf{vk}, Y, f, \widetilde{\sigma}, \sigma) = f(y)$ due to the correctness of the AS and the El Gamal encryption scheme EG.

Theorem 3.15.1 (Unforgeability for FAS Construction). *If the* NIZK *arguments for the languages* L_{ad} *and* L_{eq} *both satisfy adaptive soundness, the underlying* AS *scheme satisfies witness extractability, and the relation* R *is* $\mathcal{F}$*-hard, then the proposed* FAS *construction in Figure 3.34 satisfies the property of unforgeability as defined in Definition 3.5.8.*

Overview. In order to prove that the FAS construction is unforgeable, we will reduce it to the $\mathcal{F}$-hardness of the relation R. Consider the sequence of games G_0, G_1, G_2, G_3 in Figure 3.42.

Games G_0, G_1, G_2, G_3

1 : $Q := \emptyset, \mathsf{bad}_1 := \mathsf{false}, \mathsf{bad}_2 := \mathsf{false}, \mathsf{bad}_3 := \mathsf{false}$
2 : $\mathsf{crs}_{\mathsf{ad}} \leftarrow \mathsf{Setup}_{\mathsf{ad}}(1^\lambda)$
3 : $\mathsf{crs}_{\mathsf{eq}} \leftarrow \mathsf{Setup}_{\mathsf{eq}}(1^\lambda)$
4 : $\mathsf{pp} := (\mathsf{crs}_{\mathsf{ad}}, \mathsf{crs}_{\mathsf{eq}})$
5 : $(\mathsf{sk}, \mathsf{vk}) \leftarrow \mathsf{AS.KGen}(1^\lambda)$
6 : $(Y^*, y^*) \leftarrow \mathsf{GenR}(1^\lambda)$
7 : $(\mathsf{ad}^*, m^*, f^*, \widetilde{\sigma}_1^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{s}}, \mathcal{O}_{\mathsf{fps}_2}}(\mathsf{pp}, \mathsf{vk}, Y^*)$
8 : Parse ad^* as $(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, \pi_{\mathsf{ad}})$
9 : $\mathsf{stmt}_{\mathsf{ad}} := (\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, Y^*)$
10 : Parse $\widetilde{\sigma}_1^*$ as $(\mathsf{pk}_{\mathsf{EG}}, \vec{\mathsf{ct}}, \pi_{\mathsf{eq}})$
11 : $\widehat{\mathsf{ct}} := \mathsf{FHE.Eval}(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, f^*)$
12 : $\mathsf{stmt}_{\mathsf{eq}} := (\mathsf{pk}_{\mathsf{EG}}, \vec{\mathsf{ct}}, \mathsf{pk}_{\mathsf{FHE}}, \widehat{\mathsf{ct}})$
13 : if $\big(\mathsf{Verify}_{\mathsf{ad}}(\mathsf{crs}_{\mathsf{ad}}, \mathsf{stmt}_{\mathsf{ad}}, \pi_{\mathsf{ad}}) = 0 \vee (f^* \notin \mathcal{F})$
$\quad \vee \mathsf{Verify}_{\mathsf{eq}}(\mathsf{crs}_{\mathsf{eq}}, \mathsf{stmt}_{\mathsf{eq}}, \pi_{\mathsf{eq}}) = 0\big)$: **return** $\perp$
14 : $\widetilde{\sigma}_2^* \leftarrow \mathsf{AS.pSign}(\mathsf{sk}, m^*, \mathsf{pk}_{\mathsf{EG}})$
15 : $\sigma^* \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{s}}, \mathcal{O}_{\mathsf{fps}_2}}(\widetilde{\sigma}_2^*)$
16 : if $(\mathsf{stmt}_{\mathsf{ad}} \notin L_{\mathsf{ad}})$: $\mathsf{bad}_1 := \mathsf{true}$
17 : if $(\mathsf{stmt}_{\mathsf{eq}} \notin L_{\mathsf{eq}})$: $\mathsf{bad}_2 := \mathsf{true}$
18 : $\mathsf{sk}'_{\mathsf{EG}} := \mathsf{AS.Ext}(\mathsf{vk}, \mathsf{pk}_{\mathsf{EG}}, \widetilde{\sigma}_2^*, \sigma^*)$
19 : if $\big((m^* \notin Q) \wedge \mathsf{Vrfy}(\mathsf{vk}, m^*, \sigma^*) = 1 \wedge (\mathsf{pk}_{\mathsf{EG}}, \mathsf{sk}'_{\mathsf{EG}}) \notin \mathsf{R}_{\mathsf{EG}}\big)$
$\quad \mathsf{bad}_3 := \mathsf{true}$
20 : return $(m^* \notin Q) \wedge \mathsf{Vrfy}(\mathsf{vk}, m^*, \sigma^*) = 1$
$\wedge \neg\mathsf{bad}_1 \wedge \neg\mathsf{bad}_2 \wedge \neg\mathsf{bad}_3$

Oracle $\mathcal{O}_{\mathsf{s}}(m)$

1 : $\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$
2 : $Q := Q \cup \{m\}$
3 : return σ

Oracle $\mathcal{O}_{\mathsf{fps}_2}(\mathsf{ad}, m, Y, f, \widetilde{\sigma}_1)$

1 : Parse ad as $(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, \pi_{\mathsf{ad}})$
2 : Parse $\widetilde{\sigma}_1$ as $(\mathsf{pk}_{\mathsf{EG}}, \vec{\mathsf{ct}}, \pi_{\mathsf{eq}})$
3 : $\widehat{\mathsf{ct}} := \mathsf{FHE.Eval}(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, f)$
4 : $\mathsf{stmt}_{\mathsf{eq}} := (\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \mathsf{pk}_{\mathsf{FHE}}, \widehat{\mathsf{ct}})$
5 : if $(\mathsf{Verify}_{\mathsf{eq}}(\mathsf{crs}_{\mathsf{eq}}, \mathsf{stmt}_{\mathsf{eq}}, \pi_{\mathsf{eq}}) = 0)$:
$\quad$ **return** $\perp$
6 : $\widetilde{\sigma}_2 \leftarrow \mathsf{AS.pSign}(\mathsf{sk}, m, \mathsf{pk}_{\mathsf{EG}})$
7 : $Q := Q \cup \{m\}$
8 : return $\widetilde{\sigma}_2$

Figure 3.42: Unforgeability Proof: Games G_0, G_1, G_2

- Game G_0 is the original game $\mathsf{fUnforge}_{\mathcal{A},\mathsf{FAS}}$ for the FAS scheme we propose. In this game, the adversary $\mathcal{A}$ is tasked with creating a valid forgery σ^* for a message m^* of their choice. The adversary

has access to the signature and functional pre-signature oracles, which are denoted $\mathcal{O}_{\mathsf{s}}$ and $\mathcal{O}_{\mathsf{fps}_2}$, respectively.

- Game G_1 is identical to Game G_0, except that after line 15, the game additionally checks if $\mathsf{stmt}_{\mathsf{ad}} := (\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, Y^*)$ is in the language L_{ad}. If it is not in the language, then we set the flag $\mathsf{bad}_1 = \mathsf{true}$. The winning condition of G_1 is also different from G_0, as in order for $\mathcal{A}$ to win in G_1, it must also satisfy $\neg\mathsf{bad}_1$. That is, the flag bad_1 must be false.

- Game G_2 is identical to Game G_1, except that after line 16, the game additionally checks if $\mathsf{stmt}_{\mathsf{eq}} := (\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \mathsf{pk}_{\mathsf{FHE}}, \widehat{\mathsf{ct}})$ is in the language L_{eq}. If it is not in the language, then we set the flag $\mathsf{bad}_2 = \mathsf{true}$. The winning condition of G_2 is also different from G_1, as in order for $\mathcal{A}$ to win in G_2, it must also satisfy $\neg\mathsf{bad}_2$. That is, the flag bad_2 must be false.

- Game G_3 is identical to Game G_2, except that after line 17, the game additionally checks if the pair $(\mathsf{pk}_{\mathsf{EG}}, \mathsf{sk}'_{\mathsf{EG}}) \in \mathsf{R}_{\mathsf{EG}}$. If this is not a valid key pair, then we set the flag $\mathsf{bad}_3 = \mathsf{true}$. The winning condition of G_3 is also different from G_2, as in order for $\mathcal{A}$ to win in G_3, it must also satisfy $\neg\mathsf{bad}_3$. That is, the flag bad_3 must be false.

First, we will argue in Claims 3.15.2, 3.15.3, and 3.15.4 that the game hops from G_0 to G_3 are close. After, we will argue in Claim 3.15.5 that if there exists a stateful p.p.t. adversary $\mathcal{A}$ that wins G_3 with non-negligible probability, then $\mathcal{A}$ could be used to break the $\mathcal{F}$-hardness of the relation R. The formal proof for Theorem 3.15.1 can be found in Claim 3.15.6. □

Claim 3.15.2. *If the* NIZK *argument for language L_{ad} satisfies adaptive soundness, then for every stateful p.p.t. adversary $\mathcal{A}$, there exists a negligible function ϵ, such that for every $\lambda \in \mathbb{N}$:*

$$\left|\Pr[G_0 = 1] - \Pr[G_1 = 1]\right| \leq \epsilon(\lambda)$$

Proof. Assume towards a contradiction that $\left|\Pr[G_0 = 1] - \Pr[G_1 = 1]\right| > \epsilon(\lambda)$. Notice that since Games G_0 and G_1 are identical except for the check in G_1 that determines the flag bad_1, we have:

$$\begin{aligned}\Pr[\mathsf{bad}_1 = \mathsf{true}, \text{in } G_1] &\geq \left|\Pr[G_0 = 1] - \Pr[G_1 = 1]\right| \\ &> \epsilon(\lambda)\end{aligned} \tag{3.3}$$

We want to show that if this is the case, then there exists some p.p.t. prover $\mathcal{P}$ that uses $\mathcal{A}$ to break the adaptive soundness of the NIZK argument for the language L_{ad}. Recall that in order to win this game, the prover $\mathcal{P}$ needs to find a pair $(\mathsf{stmt}_{\mathsf{ad}}, \pi_{\mathsf{ad}})$ such that it satisfies two conditions: $(\mathsf{stmt}_{\mathsf{ad}} \notin$

Reduction $\mathcal{P}$ for the proof of Claim 3.15.2

$Q = \emptyset, \mathsf{bad}_1 := \mathsf{false}$
$\mathsf{crs}_{\mathsf{ad}} \leftarrow \mathcal{C}$
$\mathsf{crs}_{\mathsf{eq}} \leftarrow \mathsf{Setup}_{\mathsf{eq}}(1^\lambda)$
$\mathsf{pp} := (\mathsf{crs}_{\mathsf{ad}}, \mathsf{crs}_{\mathsf{eq}})$
$(\mathsf{sk}, \mathsf{vk}) \leftarrow \mathsf{AS.KGen}(1^\lambda)$
$(Y^*, y^*) \leftarrow \mathsf{GenR}(1^\lambda)$
$(\mathsf{ad}^*, m^*, f^*, \widetilde{\sigma}_1^*) \leftarrow \mathcal{A}^{\mathcal{O}_s, \mathcal{O}_{\mathsf{fps}_2}}(\mathsf{pp}, \mathsf{vk}, Y^*)$
Parse ad^* as $(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, \pi_{\mathsf{ad}})$
$\mathsf{stmt}_{\mathsf{ad}} := (\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, Y^*)$
if $(\mathsf{Verify}_{\mathsf{ad}}(\mathsf{crs}_{\mathsf{ad}}, \mathsf{stmt}_{\mathsf{ad}}, \pi_{\mathsf{ad}}) = 0)$: **return** $\perp$
return $(\mathsf{stmt}_{\mathsf{ad}}, \pi_{\mathsf{ad}})$ to $\mathcal{C}$

Oracle $\mathcal{O}_s(m)$

$\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$
$Q := Q \cup \{m\}$
return σ

Oracle $\mathcal{O}_{fps_2}(\mathsf{ad}, m, Y, f, \widetilde{\sigma}_1)$

Parse ad as $(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, \pi_{\mathsf{ad}})$
Parse $\widetilde{\sigma}_1$ as $(\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \pi_{\mathsf{eq}})$
$\widehat{\mathsf{ct}} := \mathsf{FHE.Eval}(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, f)$
$\mathsf{stmt}_{\mathsf{eq}} := (\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \mathsf{pk}_{\mathsf{FHE}}, \widehat{\mathsf{ct}})$
if $(\mathsf{Verify}_{\mathsf{eq}}(\mathsf{crs}_{\mathsf{eq}}, \mathsf{stmt}_{\mathsf{eq}}, \pi_{\mathsf{eq}}) = 0)$:
    return $\perp$
$\widetilde{\sigma}_2 \leftarrow \mathsf{AS.pSign}(\mathsf{sk}, m, \mathsf{pk}_{\mathsf{EG}})$
$Q := Q \cup \{m\}$
return $\widetilde{\sigma}_2$

Figure 3.43: Reduction for prover $\mathcal{P}$ playing the adaptive soundness game with challenger $\mathcal{C}$

$L_{\mathsf{ad}}) \wedge (\mathsf{Verify}_{\mathsf{ad}}(\mathsf{crs}_{\mathsf{ad}}, \mathsf{stmt}_{\mathsf{ad}}, \pi_{\mathsf{ad}}) = 1)$. So we have that $\Pr[\mathcal{P} \text{ wins}]$ is given by:

$$\begin{aligned} \Pr[\mathcal{P} \text{ wins}] = \Pr\big[&(\mathsf{stmt}_{\mathsf{ad}} \notin L_{\mathsf{ad}}) \\ &\wedge (\mathsf{Verify}_{\mathsf{ad}}(\mathsf{crs}_{\mathsf{ad}}, \mathsf{stmt}_{\mathsf{ad}}, \pi_{\mathsf{ad}}) = 1)\big] \end{aligned} \tag{3.4}$$

The reduction $\mathcal{P}$ in Figure 3.43 will simultaneously play the role of challenger to adversary $\mathcal{A}$ as in Game G_1, and the role of the prover in the adaptive soundness game for the NIZK argument for the language L_{ad}. Let $\mathcal{C}$ be the corresponding challenger for that adaptive soundness game. The procedure for the reduction is as follows:

1. $\mathcal{C}$ honestly samples a $\mathsf{crs}_{\mathsf{ad}}$, and sends it to $\mathcal{P}$.

2. $\mathcal{P}$ generates the public parameters pp (which includes the $\mathsf{crs}_{\mathsf{ad}}$ from $\mathcal{C}$), and also samples $(\mathsf{sk}, \mathsf{vk})$ for the AS scheme. It also generates an instance (Y^*, y^*) of the relation R. $\mathcal{P}$ then sends $(\mathsf{pp}, \mathsf{vk}, Y^*)$ to $\mathcal{A}$.

3. $\mathcal{A}$ performs queries to the signature and functional pre-signature oracles, $\mathcal{O}_s$ and $\mathcal{O}_{fps_2}$, which $\mathcal{P}$ answers.

4. After making queries, $\mathcal{A}$ then sends a response $(\mathsf{ad}^*, m^*, f^*, \widetilde{\sigma}_1^*)$ to $\mathcal{P}$.

5. $\mathcal{P}$ then parses ad^* as $(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, \pi_{\mathsf{ad}})$, and constructs $\mathsf{stmt}_{\mathsf{ad}} := (\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, Y^*)$. Then, $\mathcal{P}$ sends the response $(\mathsf{stmt}_{\mathsf{ad}}, \pi_{\mathsf{ad}})$ to $\mathcal{C}$.

Notice that during Game G_1, when $(\mathsf{stmt}_{\mathsf{ad}} \notin L_{\mathsf{ad}})$, we set the flag $\mathsf{bad}_1 := \mathsf{true}$ in line 16. But it must also have been the case that $\mathsf{Verify}_{\mathsf{ad}}(\mathsf{crs}_{\mathsf{ad}}, \mathsf{stmt}_{\mathsf{ad}}, \pi_{\mathsf{ad}}) = 1$, as otherwise, G_1 would have aborted in line 13 (and therefore it would not have reached line 16 to set the flag bad_1 to true). Therefore, we have that:

$$\begin{aligned} \Pr[\mathsf{bad}_1 = \mathsf{true}, \text{ in } G_1] = \Pr\big[&(\mathsf{stmt}_{\mathsf{ad}} \notin L_{\mathsf{ad}}) \\ &\wedge (\mathsf{Verify}_{\mathsf{ad}}(\mathsf{crs}_{\mathsf{ad}}, \mathsf{stmt}_{\mathsf{ad}}, \pi_{\mathsf{ad}}) = 1)\big] \end{aligned} \tag{3.5}$$

Notice that combining equations 3.3, 3.4 and 3.5 gives that:

$$\Pr[\mathcal{P} \text{ wins}] = \Pr[\mathsf{bad}_1 = \mathsf{true}, \text{ in } G_1] > \epsilon(\lambda), \tag{3.6}$$

where ϵ is a negligible function. Therefore, we have constructed a prover $\mathcal{P}$ that wins the adaptive soundness game of the NIZK argument for the language L_{ad} with non-negligible probability, a contradiction to our initial assumption of the claim. This concludes the proof. □

Claim 3.15.3. *If the* NIZK *argument for language* L_{eq} *satisfies adaptive soundness, then for every stateful p.p.t. adversary* $\mathcal{A}$*, there exists a negligible function* ϵ*, such that for every* $\lambda \in \mathbb{N}$*:*

$$\big|\Pr[G_1 = 1] - \Pr[G_2 = 1]\big| \leq \epsilon(\lambda)$$

Proof. Assume towards a contradiction that $\big|\Pr[G_1 = 1] - \Pr[G_2 = 1]\big| > \epsilon(\lambda)$. Notice that since Games G_1 and G_2 are identical except for the check that determines the flag bad_2, we have:

$$\begin{aligned} \Pr[\mathsf{bad}_2 = \mathsf{true}, \text{in } G_2] &\geq \big|\Pr[G_1 = 1] - \Pr[G_2 = 1]\big| \\ &> \epsilon(\lambda) \end{aligned} \tag{3.7}$$

We want to show that if this is the case, then there exists some p.p.t. prover $\mathcal{P}^*$ that uses $\mathcal{A}$ to break the adaptive soundness of the NIZK argument for the language L_{eq}. Recall that in order to win this

game, the prover $\mathcal{P}^*$ needs to find a pair $(\mathsf{stmt}_{\mathsf{eq}}, \pi_{\mathsf{eq}})$ such that it satisfies two conditions: $(\mathsf{stmt}_{\mathsf{eq}} \notin L_{\mathsf{eq}}) \wedge (\mathsf{Verify}_{\mathsf{eq}}(\mathsf{crs}_{\mathsf{eq}}, \mathsf{stmt}_{\mathsf{eq}}, \pi_{\mathsf{eq}}) = 1)$. So we have that $\Pr[\mathcal{P}^* \text{ wins}]$ is given by:

$$\begin{aligned}\Pr[\mathcal{P}^* \text{ wins}] = \Pr\big[&(\mathsf{stmt}_{\mathsf{eq}} \notin L_{\mathsf{eq}}) \\ &\wedge(\mathsf{Verify}_{\mathsf{eq}}(\mathsf{crs}_{\mathsf{eq}}, \mathsf{stmt}_{\mathsf{eq}}, \pi_{\mathsf{eq}}) = 1)\big]\end{aligned} \tag{3.8}$$

Notice that the winning condition of $\mathcal{P}^*$ is similar to that of the winning condition for $\mathcal{P}$ in Claim 3.15.2. Further, the hop from G_1 to G_2 is similar to that of the hop from G_0 to G_1. Therefore, the rest of this proof follows from the proof of Claim 3.15.2. This concludes the proof. □

Claim 3.15.4. *If the underlying adaptor signature scheme* AS *satisfies witness extractability, then for every stateful p.p.t. adversary $\mathcal{A}$, there exists a negligible function ϵ, such that for every $\lambda \in \mathbb{N}$:*

$$\big|\Pr[G_2 = 1] - \Pr[G_3 = 1]\big| \leq \epsilon(\lambda)$$

Proof. Assume towards a contradiction that $\big|\Pr[G_2 = 1] - \Pr[G_3 = 1]\big| > \epsilon(\lambda)$. Notice that since Games G_2 and G_3 are identical except for the check that determines the flag bad_3, we have:

$$\begin{aligned}\Pr[\mathsf{bad}_3 = \mathsf{true}, \text{in } G_3] &\geq \big|\Pr[G_2 = 1] - \Pr[G_3 = 1]\big| \\ &> \epsilon(\lambda)\end{aligned} \tag{3.9}$$

We want to show that if this is the case, then we can create some reduction $\mathcal{B}$ that uses $\mathcal{A}$ to break the witness extractability of the underlying AS scheme. Recall that in order to win the witness extractability game, $\mathcal{B}$ needs to find $(m^*, \mathsf{pk}_{\mathsf{EG}}, \sigma^*)$ such that all of the following conditions are met: $(m^* \notin Q) \wedge \mathsf{Vrfy}(\mathsf{vk}, m^*, \sigma^*) = 1 \wedge (\mathsf{pk}_{\mathsf{EG}}, \mathsf{AS.Ext}(\mathsf{vk}, \mathsf{pk}_{\mathsf{EG}}, \widetilde{\sigma}_2^*, \sigma^*)) \notin \mathsf{R}_{\mathsf{EG}}$. We will denote $\mathsf{sk}'_{\mathsf{EG}} = \mathsf{AS.Ext}(\mathsf{vk}, \mathsf{pk}_{\mathsf{EG}}, \widetilde{\sigma}_2^*, \sigma^*)$, so we get that:

$$\begin{aligned}\Pr[\mathcal{B} \text{ wins}] = \Pr\big[&(m^* \notin Q) \wedge \mathsf{Vrfy}(\mathsf{vk}, m^*, \sigma^*) = 1 \\ &\wedge(\mathsf{pk}_{\mathsf{EG}}, \mathsf{sk}'_{\mathsf{EG}}) \notin \mathsf{R}_{\mathsf{EG}}\big]\end{aligned} \tag{3.10}$$

The reduction $\mathcal{B}$ in Figure 3.44 will simultaneously play the role of challenger to adversary $\mathcal{A}$ as in Game G_3, and the role of the adversary in the witness extractability game for the underlying AS scheme. Let $\mathcal{C}$ be the corresponding challenger for the witness extractability game for the underlying AS scheme. The procedure for the reduction is as follows:

1. $\mathcal{C}$ honestly samples the signing and verification keys $(\mathsf{sk}, \mathsf{vk})$, and sends vk to $\mathcal{B}$.

2. $\mathcal{B}$ generates the public parameters pp. It also generates an instance (Y^*, y^*) of the relation R. $\mathcal{B}$ then sends $(\mathsf{pp}, \mathsf{vk}, Y^*)$ to $\mathcal{A}$.

Reduction $\mathcal{B}$ for the proof of Claim 3.15.4

$Q = \emptyset, \mathsf{bad}_1 := \mathsf{false}, \mathsf{bad}_2 := \mathsf{false}, \mathsf{bad}_3 := \mathsf{false}$
$\mathsf{crs}_{\mathsf{ad}} \leftarrow \mathsf{Setup}_{\mathsf{ad}}(1^\lambda),\ \mathsf{crs}_{\mathsf{eq}} \leftarrow \mathsf{Setup}_{\mathsf{eq}}(1^\lambda)$
$\mathsf{pp} := (\mathsf{crs}_{\mathsf{ad}}, \mathsf{crs}_{\mathsf{eq}})$
$\mathsf{vk} \leftarrow \mathcal{C}$
$(Y^*, y^*) \leftarrow \mathsf{GenR}(1^\lambda)$
$(\mathsf{ad}^*, m^*, f^*, \widetilde{\sigma}_1^*) \leftarrow \mathcal{A}^{\mathcal{O}_s, \mathcal{O}_{fps_2}}(\mathsf{pp}, \mathsf{vk}, Y^*)$
Parse ad^* as $(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, \pi_{\mathsf{ad}})$
$\mathsf{stmt}_{\mathsf{ad}} := (\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, Y^*)$
Parse $\widetilde{\sigma}_1^*$ as $(\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \pi_{\mathsf{eq}})$
$\widehat{\mathsf{ct}} := \mathsf{FHE.Eval}(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, f^*)$
$\mathsf{stmt}_{\mathsf{eq}} := (\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \mathsf{pk}_{\mathsf{FHE}}, \widehat{\mathsf{ct}})$
if $\big(\mathsf{Verify}_{\mathsf{ad}}(\mathsf{crs}_{\mathsf{ad}}, \mathsf{stmt}_{\mathsf{ad}}, \pi_{\mathsf{ad}}) = 0 \vee (f^* \notin \mathcal{F})$
$\quad \vee \mathsf{Verify}_{\mathsf{eq}}(\mathsf{crs}_{\mathsf{eq}}, \mathsf{stmt}_{\mathsf{eq}}, \pi_{\mathsf{eq}}) = 0\big)$: **return** $\perp$
$\widetilde{\sigma}_2^* \leftarrow \mathcal{C}(m^*, \mathsf{pk}_{\mathsf{EG}})$
$\sigma^* \leftarrow \mathcal{A}^{\mathcal{O}_s, \mathcal{O}_{fps_2}}(\widetilde{\sigma}_2^*)$
return $(m^*, \mathsf{pk}_{\mathsf{EG}}, \sigma^*)$ to $\mathcal{C}$

Oracle $\mathcal{O}_s(m)$

$\sigma \leftarrow \mathcal{C}(m)$
$Q := Q \cup \{m\}$
return σ

Oracle $\mathcal{O}_{fps_2}(\mathsf{ad}, m, Y, f, \widetilde{\sigma}_1)$

Parse ad as $(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, \pi_{\mathsf{ad}})$
Parse $\widetilde{\sigma}_1$ as $(\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \pi_{\mathsf{eq}})$
$\widehat{\mathsf{ct}} := \mathsf{FHE.Eval}(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, f)$
$\mathsf{stmt}_{\mathsf{eq}} := (\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \mathsf{pk}_{\mathsf{FHE}}, \widehat{\mathsf{ct}})$
if $(\mathsf{Verify}_{\mathsf{eq}}(\mathsf{crs}_{\mathsf{eq}}, \mathsf{stmt}_{\mathsf{eq}}, \pi_{\mathsf{eq}}) = 0)$:
$\quad$ **return** $\perp$
$\widetilde{\sigma}_2 \leftarrow \mathcal{C}(m, \mathsf{pk}_{\mathsf{EG}})$
$Q := Q \cup \{m\}$
return $\widetilde{\sigma}_2$

Figure 3.44: Reduction for $\mathcal{B}$ playing the witness extractability game for AS with challenger $\mathcal{C}$

3. $\mathcal{A}$ performs queries to the signature and functional pre-signature oracles, $\mathcal{O}_s$ and $\mathcal{O}_{fps_2}$. Note that $\mathcal{B}$ answers these queries by forwarding the queries to its challenger, $\mathcal{C}$. After making queries, $\mathcal{A}$ then sends a response $(\mathsf{ad}^*, m^*, f^*, \widetilde{\sigma}_1^*)$ to $\mathcal{B}$.

4. $\mathcal{B}$ then uses $\mathcal{C}$ to generate a honest pre-signature $\widetilde{\sigma}_2^*$ for $\mathcal{A}$'s chosen message m^* and statement $\mathsf{pk}_{\mathsf{EG}}$. $\mathcal{B}$ then forwards this pre-signature $\widetilde{\sigma}_2^*$ to $\mathcal{A}$.

5. After making more queries, $\mathcal{A}$ then sends a response σ^* to $\mathcal{B}$.

6. Then, $\mathcal{B}$ sends $(m^*, \mathsf{pk}_{\mathsf{EG}}, \sigma^*)$ to $\mathcal{C}$.

Now, we need to show that when the flag $\mathsf{bad}_3 = \mathsf{true}$ in G_3, the winning conditions for $\mathcal{B}$ are also met. We know that when the flag $\mathsf{bad}_3 = \mathsf{true}$, we have that the three following conditions are met: $(m^* \notin Q) \wedge \mathsf{Vrfy}(\mathsf{vk}, m^*, \sigma^*) = 1 \wedge (\mathsf{pk}_{\mathsf{EG}}, \mathsf{sk}'_{\mathsf{EG}}) \notin \mathsf{R}_{\mathsf{EG}}$. So we have that:

$$\begin{aligned} \Pr[\mathsf{bad}_3 = \mathsf{true}, \text{ in } G_3] = \Pr\big[(m^* \notin Q) \\ \wedge \mathsf{Vrfy}(\mathsf{vk}, m^*, \sigma^*) = 1 \wedge (\mathsf{pk}_{\mathsf{EG}}, \mathsf{sk}'_{\mathsf{EG}}) \notin \mathsf{R}_{\mathsf{EG}}\big] \end{aligned} \tag{3.11}$$

Notice that combining equations 3.9, 3.10 and 3.11 gives that:

$$\Pr[\mathcal{B} \text{ wins}] = \Pr[\mathsf{bad}_3 = \mathsf{true}, \text{ in } G_3] > \epsilon(\lambda), \tag{3.12}$$

where ϵ is a negligible function. Therefore, we have constructed a reduction $\mathcal{B}$ that wins the witness extractability game of the underlying AS scheme with non-negligible probability, a contradiction to our initial assumption of the claim. This concludes the proof.

□

Claim 3.15.5. *If the relation* R *is* $\mathcal{F}$*-hard, then for every stateful p.p.t. adversary* $\mathcal{A}$*, there exists a negligible function* ϵ*, such that for every* $\lambda \in \mathbb{N}$*, we have that:*

$$\Pr[G_3 = 1] \leq \epsilon(\lambda)$$

Proof. Assume towards a contradiction that $\Pr[G_3 = 1] > \epsilon(\lambda)$, that is, there exists some p.p.t. adversary $\mathcal{A}$ that wins Game G_3 with non-negligible probability. We want to show that if this is the case, then we can create some reduction $\mathcal{B}'$ that uses $\mathcal{A}$ to break the $\mathcal{F}$-hardness of the relation R. Recall that in order to win this game, given some statement $Y^* \in L_{\mathsf{R}}$, $\mathcal{B}'$ needs to produce a pair (f^*, z) such that $(f^* \in \mathcal{F}) \wedge \big(z \in \{f^*(y) \mid \exists y \text{ s.t } (Y^*, y) \in \mathsf{R}\}\big)$. So we get that:

$$\begin{aligned} \Pr[\mathcal{B}' \text{ wins}] = \Pr\big[(f^* \in \mathcal{F}) \\ \wedge \big(z \in \{f^*(y) \mid \exists y \text{ s.t } (Y^*, y) \in \mathsf{R}\}\big)\big] \end{aligned} \tag{3.13}$$

The reduction $\mathcal{B}'$ in Figure 3.45 will simultaneously play the role of challenger to adversary $\mathcal{A}$ as in Game G_3, and the role of adversary in the $\mathcal{F}$-hardness game for the relation R. Let $\mathcal{C}$ be the corresponding challenger for the $\mathcal{F}$-hardness game for the relation R. The procedure for the reduction is as follows:

1. $\mathcal{C}$ honestly samples an instance (Y^*, y^*) of the relation R. It then sends Y^* to $\mathcal{B}'$.

2. $\mathcal{B}'$ generates the public parameters pp. It also samples the signing and verification keys $(\mathsf{sk}, \mathsf{vk})$. $\mathcal{B}'$ then sends $(\mathsf{pp}, \mathsf{vk}, Y^*)$ to $\mathcal{A}$.

Reduction $\mathcal{B}'$ for the proof of Claim 3.15.5

$Q = \emptyset, \mathsf{bad}_1 := \mathsf{false}, \mathsf{bad}_2 := \mathsf{false}, \mathsf{bad}_3 := \mathsf{false}$
$\mathsf{crs}_{\mathsf{ad}} \leftarrow \mathsf{Setup}_{\mathsf{ad}}(1^\lambda)$
$\mathsf{crs}_{\mathsf{eq}} \leftarrow \mathsf{Setup}_{\mathsf{eq}}(1^\lambda)$
$\mathsf{pp} := (\mathsf{crs}_{\mathsf{ad}}, \mathsf{crs}_{\mathsf{eq}})$
$(\mathsf{sk}, \mathsf{vk}) \leftarrow \mathsf{AS.KGen}(1^\lambda)$
$Y^* \leftarrow \mathcal{C}$
$(\mathsf{ad}^*, m^*, f^*, \widetilde{\sigma}_1^*) \leftarrow \mathcal{A}^{\mathcal{O}_s, \mathcal{O}_{fps_2}}(\mathsf{pp}, \mathsf{vk}, Y^*)$
Parse ad^* as $(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, \pi_{\mathsf{ad}})$
$\mathsf{stmt}_{\mathsf{ad}} := (\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, Y^*)$
Parse $\widetilde{\sigma}_1^*$ as $(\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \pi_{\mathsf{eq}})$
$\widehat{\mathsf{ct}} := \mathsf{FHE.Eval}(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, f^*)$
$\mathsf{stmt}_{\mathsf{eq}} := (\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \mathsf{pk}_{\mathsf{FHE}}, \widehat{\mathsf{ct}})$
if $\big(\mathsf{Verify}_{\mathsf{ad}}(\mathsf{crs}_{\mathsf{ad}}, \mathsf{stmt}_{\mathsf{ad}}, \pi_{\mathsf{ad}}) = 0 \vee (f^* \notin \mathcal{F})$
$\quad \vee \mathsf{Verify}_{\mathsf{eq}}(\mathsf{crs}_{\mathsf{eq}}, \mathsf{stmt}_{\mathsf{eq}}, \pi_{\mathsf{eq}}) = 0\big)$: **return** $\perp$
$\widetilde{\sigma}_2 \leftarrow \mathsf{AS.pSign}(\mathsf{sk}, m^*, \mathsf{pk}_{\mathsf{EG}})$
$\sigma^* \leftarrow \mathcal{A}^{\mathcal{O}_s, \mathcal{O}_{fps_2}}(\widetilde{\sigma}_2^*)$
$\mathsf{sk}'_{\mathsf{EG}} := \mathsf{AS.Ext}(\mathsf{vk}, \mathsf{pk}_{\mathsf{EG}}, \widetilde{\sigma}_2^*, \sigma^*)$
$z := \mathsf{EG.Dec}(\mathsf{sk}'_{\mathsf{EG}}, \mathsf{ct})$
return (f^*, z) to $\mathcal{C}$

Oracle $\mathcal{O}_s(m)$

$\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$
$Q := Q \cup \{m\}$
return σ

Oracle $\mathcal{O}_{fps_2}(\mathsf{ad}, m, Y, f, \widetilde{\sigma}_1)$

Parse ad as $(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, \pi_{\mathsf{ad}})$
Parse $\widetilde{\sigma}_1$ as $(\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \pi_{\mathsf{eq}})$
$\widehat{\mathsf{ct}} := \mathsf{FHE.Eval}(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, f)$
$\mathsf{stmt}_{\mathsf{eq}} := (\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \mathsf{pk}_{\mathsf{FHE}}, \widehat{\mathsf{ct}})$
if $(\mathsf{Verify}_{\mathsf{eq}}(\mathsf{crs}_{\mathsf{eq}}, \mathsf{stmt}_{\mathsf{eq}}, \pi_{\mathsf{eq}}) = 0)$:
$\quad$ **return** $\perp$
$\widetilde{\sigma}_2 \leftarrow \mathsf{AS.pSign}(\mathsf{sk}, m, \mathsf{pk}_{\mathsf{EG}})$
$Q := Q \cup \{m\}$
return $\widetilde{\sigma}_2$

Figure 3.45: Reduction for $\mathcal{B}'$ which breaks the $\mathcal{F}$-hardness of the relation R.

3. $\mathcal{A}$ performs queries to the signature and functional pre-signature oracles, $\mathcal{O}_s$ and $\mathcal{O}_{fps_2}$. Note that $\mathcal{B}'$ answers these queries. After making queries, $\mathcal{A}$ then sends a response $(\mathsf{ad}^*, m^*, f^*, \widetilde{\sigma}_1^*)$ to $\mathcal{B}'$.

4. $\mathcal{B}'$ then generates an honest pre-signature $\widetilde{\sigma}_2^*$ for $\mathcal{A}$'s chosen message m^* and statement $\mathsf{pk}_{\mathsf{EG}}$. $\mathcal{B}'$ then sends this pre-signature $\widetilde{\sigma}_2^*$ to $\mathcal{A}$.

5. After making more queries, $\mathcal{A}$ then sends a response σ^* to $\mathcal{B}'$.

6. Then, $\mathcal{B}'$ extracts the key $\mathsf{sk}'_{\mathsf{EG}}$ from the signatures, and uses it to decrypt the ciphertext ct to obtain value z. $\mathcal{B}'$ then sends (f^*, z) to $\mathcal{C}$.

Now, we need to show that when $\mathcal{A}$ wins Game G_3, the winning conditions for $\mathcal{B}'$ are also met. When $\mathcal{A}$ wins Game G_3, we know that the following occur:

(1) $\big(\mathsf{Verify}_{\mathsf{ad}}(\mathsf{crs}_{\mathsf{ad}}, \mathsf{stmt}_{\mathsf{ad}}, \pi_{\mathsf{ad}}) = 1\big)$

$\wedge \big(\mathsf{Verify}_{\mathsf{eq}}(\mathsf{crs}_{\mathsf{eq}}, \mathsf{stmt}_{\mathsf{eq}}, \pi_{\mathsf{eq}}) = 1\big) \wedge \big(f^* \in \mathcal{F}\big)$

(2) $\big(\mathsf{bad}_1 = \mathsf{false}\big) \implies \mathsf{stmt}_{\mathsf{ad}} \in L_{\mathsf{ad}}$

(3) $\left(\mathsf{bad}_2 = \mathsf{false}\right) \implies \mathsf{stmt}_{\mathsf{eq}} \in L_{\mathsf{eq}}$

(4) $\left(\mathsf{bad}_3 = \mathsf{false}\right) \implies (\mathsf{pk}_{\mathsf{EG}}, \mathsf{sk}_{\mathsf{EG}}) \in \mathsf{R}_{\mathsf{EG}}$

From (1), we know that one of $\mathcal{B}$'s winning conditions, $f^* \in \mathcal{F}$, is satisfied when $\mathcal{A}$ wins G_3. Therefore, it remains to show that $\mathcal{A}$ winning G_3 implies $\mathcal{B}$'s last winning condition, which is that $\left(z \in \{f^*(y) \mid \exists y \text{ s.t } (Y^*, y) \in \mathsf{R}\}\right)$

Notice that when $\mathcal{A}$ wins G_3, we have that conditions $(2), (3)$ are satisfied, so we know that the statements $\mathsf{stmt}_{\mathsf{ad}}$ and $\mathsf{stmt}_{\mathsf{eq}}$ are valid statements for their corresponding languages. This fact is vital for the correctness of the analysis below.

When condition (1) is satisfied, we have that $\mathsf{Verify}_{\mathsf{ad}}(\mathsf{crs}_{\mathsf{ad}}, \mathsf{stmt}_{\mathsf{ad}}, \pi_{\mathsf{ad}}) = 1$, which means that $\exists (y, r_1, r_0)$ such that $(Y^*, y) \in \mathsf{R}$ and $\mathsf{ct}_{\mathsf{ad}} = \mathsf{FHE.Enc}(\mathsf{pk}_{\mathsf{FHE}}, y; r_1)$. We also know from condition (1) that $\mathsf{Verify}_{\mathsf{eq}}(\mathsf{crs}_{\mathsf{eq}}, \mathsf{stmt}_{\mathsf{eq}}, \pi_{\mathsf{eq}}) = 1$, which means that $\exists (\mathsf{sk}_{\mathsf{FHE}}, z, r_2, r_0)$ such that $\mathsf{ct} = \mathsf{EG.Enc}(\mathsf{pk}_{\mathsf{EG}}, z; r_2)$, and $\mathsf{FHE.Dec}(\mathsf{sk}_{\mathsf{FHE}}, \widehat{\mathsf{ct}}) = z$, where $\widehat{\mathsf{ct}}$ is the functional evaluation of $\mathsf{ct}_{\mathsf{ad}}$ (which we showed above was correctly generated). Since $\widehat{\mathsf{ct}}$ was correctly generated, we know that $z = f^*(y)$. Since condition (4) is satisfied, we have that $(\mathsf{pk}_{\mathsf{EG}}, \mathsf{sk}'_{\mathsf{EG}}) \in \mathsf{R}_{\mathsf{EG}}$, meaning that it is a valid key pair. Therefore, know that $z = \mathsf{EG.Dec}(\mathsf{sk}'_{\mathsf{EG}}, \mathsf{ct})$ is a correct decryption, which outputs $z = f^*(y)$. This gives that $z \in \{f^*(y) \mid \exists y \text{ s.t } (Y^*, y) \in \mathsf{R}\}$, which is the second winning condition for $\mathcal{B}'$. Therefore, we have shown that when $\mathcal{A}$ wins Game G_3, $\mathcal{B}'$ also wins its game. Since we assumed that $\Pr[G_3 = 1] > \epsilon(\lambda)$, we have that:

$$\Pr[\mathcal{B} \text{ wins}] \geq \Pr[G_3 = 1] > \epsilon(\lambda), \tag{3.14}$$

where ϵ is a negligible function. Therefore, we have constructed a reduction $\mathcal{B}'$ that wins the $\mathcal{F}$-hardness game of the relation R with non-negligible probability, a contradiction to our initial assumption of the claim. This concludes the proof.

□

Claim 3.15.6. *If the* NIZK *arguments for the languages L_{ad} and L_{eq} both satisfy adaptive soundness, the underlying* AS *scheme satisfies witness extractability, and the relation* R *is $\mathcal{F}$-hard, then for every stateful p.p.t. adversary $\mathcal{A}$, there exists a negligible function ϵ, such that for every $\lambda \in \mathbb{N}$: $\Pr\left[\mathsf{fUnforge}_{\mathcal{A},\mathsf{FAS}}(1^\lambda) = 1\right] \leq \epsilon(\lambda)$.*

Proof. Since Game G_0 (as defined in Figure 3.42) is the original game $\mathsf{fUnforge}_{\mathcal{A},\mathsf{FAS}}$ for the FAS scheme we propose, we have that $\Pr[G_0 = 1] = \Pr[\mathsf{fUnforge}_{\mathcal{A},\mathsf{FAS}}(1^\lambda) = 1]$. By triangle inequality,

$$\begin{aligned}\Pr[G_0 = 1] \leq &\left|\Pr[G_0 = 1] - \Pr[G_1 = 1]\right| \\ &+ \left|\Pr[G_1 = 1] - \Pr[G_2 = 1]\right| \\ &+ \left|\Pr[G_2 = 1] - \Pr[G_3 = 1]\right| + \Pr[G_3 = 1]\end{aligned}$$

For every stateful p.p.t. adversary $\mathcal{A}$, we have:

- Since the NIZK argument for the language L_{ad} satisfies adaptive soundness, Claim 3.15.2 guarantees that there exists some negligible function ϵ_1 such that for every $\lambda \in \mathbb{N}$, we have that $\left|\Pr[G_0 = 1] - \Pr[G_1 = 1]\right| \leq \epsilon_1(\lambda)$
- Since the NIZK argument for the language L_{eq} satisfies adaptive soundness, Claim 3.15.3 guarantees that there exists some negligible function ϵ_2 such that for every $\lambda \in \mathbb{N}$, we have that $\left|\Pr[G_1 = 1] - \Pr[G_2 = 1]\right| \leq \epsilon_2(\lambda)$
- Since the underlying adaptor signature scheme AS satisfies witness extractability, Claim 3.15.4 guarantees that there exists some negligible function ϵ_3 such that for every $\lambda \in \mathbb{N}$, we have that $\left|\Pr[G_2 = 1] - \Pr[G_3 = 1]\right| \leq \epsilon_3(\lambda)$
- Since the relation R is $\mathcal{F}$-hard, Claim 3.15.5 guarantees that there exists some negligible function ϵ_4 such that for every $\lambda \in \mathbb{N}$, we have that $\Pr[G_3 = 1] \leq \epsilon_4(\lambda)$.

Combining these results with the expression obtained above for $\Pr[G_0 = 1]$, we have that:

$$\Pr[G_0 = 1] \leq \epsilon_1(\lambda) + \epsilon_2(\lambda) + \epsilon_3(\lambda) + \epsilon_4(\lambda) = \epsilon(\lambda) \tag{3.15}$$

Notice that since $\epsilon_1(\lambda), \epsilon_2(\lambda), \epsilon_3(\lambda)$, and $\epsilon_4(\lambda)$ are all negligible functions, we have that the function $\epsilon(\lambda) = \epsilon_1(\lambda) + \epsilon_2(\lambda) + \epsilon_3(\lambda) + \epsilon_4(\lambda)$, for all $\lambda \in \mathbb{N}$, is also a negligible function. Note that since we can find an $\epsilon_1(\lambda), \epsilon_2(\lambda), \epsilon_3(\lambda)$, and $\epsilon_4(\lambda)$ for any stateful p.p.t. adversary $\mathcal{A}$, we can then find a negligible function $\epsilon(\lambda)$ for any stateful p.p.t. adversary $\mathcal{A}$ as well.

Therefore, we have shown that for every stateful p.p.t. adversary $\mathcal{A}$, there exists a negligible function ϵ, such that for every $\lambda \in \mathbb{N}$, we have that $\Pr[G_0 = 1] \leq \epsilon(\lambda)$. This means that for any p.p.t. adversary $\mathcal{A}$, we have that $\Pr\left[\mathsf{fUnforge}_{\mathcal{A},\mathsf{FAS}}(1^\lambda) = 1\right] \leq \epsilon(\lambda)$ as well. This concludes the proof of unforgeability for the proposed FAS construction.

□

Theorem 3.15.7 (Witness Extractability for FAS Construction)**.** *If the* NIZK *arguments for the languages* L_{ad} *and* L_{eq} *both satisfy adaptive soundness, and the underlying* AS *scheme satisfies witness extractability, then the proposed* FAS *construction in Figure 3.34 satisfies the property of witness extractability as defined in Definition 3.5.9.*

Overview. In order to prove that the FAS construction satisfies witness extractability, we will reduce it to the witness extractability of the underlying AS scheme. Consider the sequence of games G_0, G_1, G_2 in Figure 3.46.

- Game G_0 is the original game $\mathsf{fWitExt}_{\mathcal{A},\mathsf{FAS}}$ for the FAS scheme we propose. In this game, the adversary $\mathcal{A}$ is tasked with creating a signature σ^* for a message m^* of their choice, in such a way that the the function evaluation of the witness is unable to be extracted using the FExt algorithm. The adversary has access to the signature and functional pre-signature oracles, which are denoted $\mathcal{O}_s$ and $\mathcal{O}_{fps_2}$, respectively.
- Game G_1 is identical to Game G_0, except that after line 16, the game additionally checks if $\mathsf{stmt}_{\mathsf{ad}} := (\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, Y^*)$ is in the language L_{ad}. If it is not in the language, then we set the flag $\mathsf{bad}_1 = \mathsf{true}$. The winning condition of G_1 is also different from G_0, as in order for $\mathcal{A}$ to win in G_1, it must also satisfy $\neg\mathsf{bad}_1$. That is, the flag bad_1 must be false.
- Game G_2 is identical to Game G_1, except that after line 17, the game additionally checks if $\mathsf{stmt}_{\mathsf{eq}} := (\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \mathsf{pk}_{\mathsf{FHE}}, \widehat{\mathsf{ct}})$ is in the language L_{eq}. If it is not in the language, then we set the flag $\mathsf{bad}_2 = \mathsf{true}$. The winning condition of G_2 is also different from G_1, as in order for $\mathcal{A}$ to win in G_2, it must also satisfy $\neg\mathsf{bad}_2$. That is, the flag bad_2 must be false.

First, we will argue in Claims 3.15.8, and 3.15.9 that the game hops from G_0 to G_2 are close. After, we will argue in Claim 3.15.10 that if there exists a stateful p.p.t. adversary $\mathcal{A}$ that wins G_2 with non-negligible probability, then $\mathcal{A}$ could be used to break the witness extractability of the underlying AS scheme. The formal proof for Theorem 3.15.7 can be found in Claim 3.15.11. □

Claim 3.15.8. *If the* NIZK *argument for language L_{ad} satisfies adaptive soundness, then for every stateful p.p.t. adversary $\mathcal{A}$, there exists a negligible function ϵ, such that for every $\lambda \in \mathbb{N}$:*

$$\big|\Pr[G_0 = 1] - \Pr[G_1 = 1]\big| \leq \epsilon(\lambda)$$

Proof. Assume towards a contradiction that $\big|\Pr[G_0 = 1] - \Pr[G_1 = 1]\big| > \epsilon(\lambda)$. Notice that since Games G_0 and G_1 are identical except for the check in G_1 that determines the flag bad_1, we have:

$$\begin{aligned}\Pr[\mathsf{bad}_1 = \mathsf{true}, \text{in } G_1] &\geq \big|\Pr[G_0 = 1] - \Pr[G_1 = 1]\big| \\ &> \epsilon(\lambda)\end{aligned} \tag{3.16}$$

We want to show that if this is the case, then there exists some p.p.t. prover $\mathcal{P}$ that uses $\mathcal{A}$ to break the adaptive soundness of the NIZK argument for the language L_{ad}. Recall that in order to win this game, the prover $\mathcal{P}$ needs to find a pair $(\mathsf{stmt}_{\mathsf{ad}}, \pi_{\mathsf{ad}})$ such that it satisfies two conditions: $(\mathsf{stmt}_{\mathsf{ad}} \notin L_{\mathsf{ad}}) \wedge (\mathsf{Verify}_{\mathsf{ad}}(\mathsf{crs}_{\mathsf{ad}}, \mathsf{stmt}_{\mathsf{ad}}, \pi_{\mathsf{ad}}) = 1)$. So we have that $\Pr[\mathcal{P} \text{ wins}]$ is given by:

$$\begin{aligned}\Pr[\mathcal{P} \text{ wins}] = \Pr\big[&(\mathsf{stmt}_{\mathsf{ad}} \notin L_{\mathsf{ad}}) \\ &\wedge(\mathsf{Verify}_{\mathsf{ad}}(\mathsf{crs}_{\mathsf{ad}}, \mathsf{stmt}_{\mathsf{ad}}, \pi_{\mathsf{ad}}) = 1)\big]\end{aligned} \tag{3.17}$$

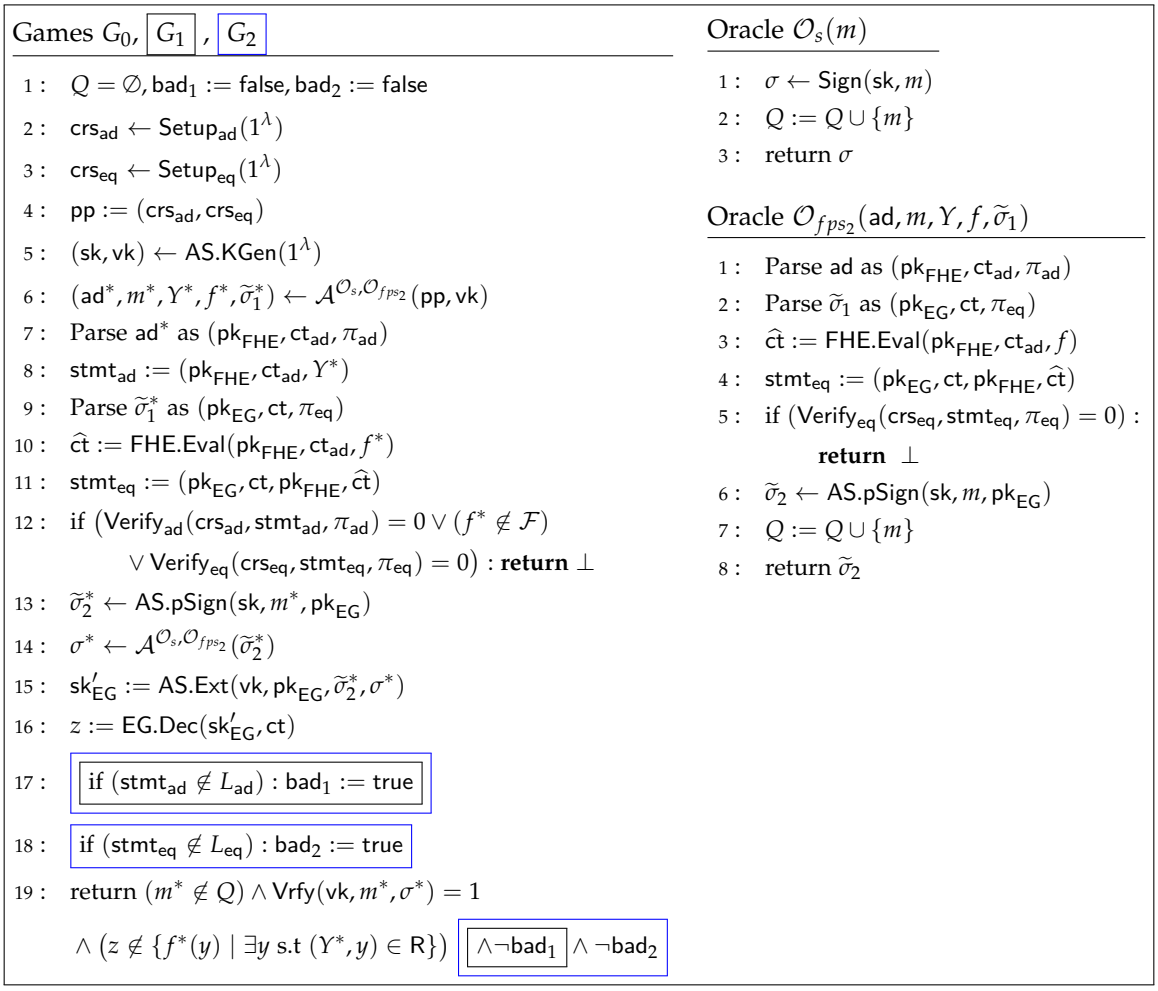


Figure 3.46: Witness Extractability Proof: Games G_0, G_1, G_2

The reduction $\mathcal{P}$ in Figure 3.47 will simultaneously play the role of challenger to adversary $\mathcal{A}$ as in Game G_1, and the role of the prover in the adaptive soundness game for the NIZK argument for the language L_{ad}. Let $\mathcal{C}$ be the corresponding challenger for that adaptive soundness game. The procedure for the reduction is as follows:

1. $\mathcal{C}$ honestly samples a $\mathsf{crs}_{\mathsf{ad}}$, and sends it to $\mathcal{P}$.

2. $\mathcal{P}$ generates the public parameters pp (which includes the $\mathsf{crs}_{\mathsf{ad}}$ from $\mathcal{C}$), and also samples $(\mathsf{sk}, \mathsf{vk})$ for the AS scheme. $\mathcal{P}$ then sends $(\mathsf{pp}, \mathsf{vk})$ to $\mathcal{A}$.

3. $\mathcal{A}$ performs queries to the signature and functional pre-signature oracles, $\mathcal{O}_s$ and $\mathcal{O}_{fps_2}$. Note that $\mathcal{P}$ answers these queries.

4. After making queries, $\mathcal{A}$ then sends a response $(\mathsf{ad}^*, m^*, Y^*, f^*, \widetilde{\sigma}_1^*)$ to $\mathcal{P}$.

Reduction $\mathcal{P}$ for the proof of Claim 3.15.8

1 : $Q = \emptyset, \mathsf{bad}_1 := \mathsf{false}$
2 : $\mathsf{crs}_{\mathsf{ad}} \leftarrow \mathcal{C}$
3 : $\mathsf{crs}_{\mathsf{eq}} \leftarrow \mathsf{Setup}_{\mathsf{eq}}(1^\lambda)$
4 : $\mathsf{pp} := (\mathsf{crs}_{\mathsf{ad}}, \mathsf{crs}_{\mathsf{eq}})$
5 : $(\mathsf{sk}, \mathsf{vk}) \leftarrow \mathsf{AS.KGen}(1^\lambda)$
6 : $(\mathsf{ad}^*, m^*, Y^*, f^*, \widetilde{\sigma}_1^*) \leftarrow \mathcal{A}^{\mathcal{O}_s, \mathcal{O}_{fps_2}}(\mathsf{pp}, \mathsf{vk})$
7 : Parse ad^* as $(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, \pi_{\mathsf{ad}})$
8 : $\mathsf{stmt}_{\mathsf{ad}} := (\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, Y^*)$
9 : if $(\mathsf{Verify}_{\mathsf{ad}}(\mathsf{crs}_{\mathsf{ad}}, \mathsf{stmt}_{\mathsf{ad}}, \pi_{\mathsf{ad}}) = 0)$: **return** $\perp$
10 : return $(\mathsf{stmt}_{\mathsf{ad}}, \pi_{\mathsf{ad}})$ to $\mathcal{C}$

Oracle $\mathcal{O}_s(m)$

1 : $\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$
2 : $Q := Q \cup \{m\}$
3 : return σ

Oracle $\mathcal{O}_{fps_2}(\mathsf{ad}, m, Y, f, \widetilde{\sigma}_1)$

1 : Parse ad as $(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, \pi_{\mathsf{ad}})$
2 : Parse $\widetilde{\sigma}_1$ as $(\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \pi_{\mathsf{eq}})$
3 : $\widehat{\mathsf{ct}} := \mathsf{FHE.Eval}(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, f)$
4 : $\mathsf{stmt}_{\mathsf{eq}} := (\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \mathsf{pk}_{\mathsf{FHE}}, \widehat{\mathsf{ct}})$
5 : if $(\mathsf{Verify}_{\mathsf{eq}}(\mathsf{crs}_{\mathsf{eq}}, \mathsf{stmt}_{\mathsf{eq}}, \pi_{\mathsf{eq}}) = 0)$: **return** $\perp$
6 : $\widetilde{\sigma}_2 \leftarrow \mathsf{AS.pSign}(\mathsf{sk}, m, \mathsf{pk}_{\mathsf{EG}})$
7 : $Q := Q \cup \{m\}$
8 : return $\widetilde{\sigma}_2$

Figure 3.47: Reduction for prover $\mathcal{P}$ playing the adaptive soundness game with challenger $\mathcal{C}$

5. $\mathcal{P}$ then parses ad^* as $(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, \pi_{\mathsf{ad}})$, and constructs $\mathsf{stmt}_{\mathsf{ad}} := (\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, Y^*)$. Then, $\mathcal{P}$ sends the response $(\mathsf{stmt}_{\mathsf{ad}}, \pi_{\mathsf{ad}})$ to $\mathcal{C}$.

Notice that during an instance of Game G_1, when $(\mathsf{stmt}_{\mathsf{ad}} \notin L_{\mathsf{ad}})$, we set the flag $\mathsf{bad}_1 = \mathsf{true}$ in line 17. But it must also have been the case that $\mathsf{Verify}_{\mathsf{ad}}(\mathsf{crs}_{\mathsf{ad}}, \mathsf{stmt}_{\mathsf{ad}}, \pi_{\mathsf{ad}}) = 1$, as otherwise, G_1 would have aborted in line 12 (and therefore it would not have reached line 17 to set the flag bad_1 to true). Therefore, we have that:

$$\begin{aligned}\Pr[\mathsf{bad}_1 = \mathsf{true}, \text{ in } G_1] = \Pr\big[(\mathsf{stmt}_{\mathsf{ad}} \notin L_{\mathsf{ad}}) \\ \wedge (\mathsf{Verify}_{\mathsf{ad}}(\mathsf{crs}_{\mathsf{ad}}, \mathsf{stmt}_{\mathsf{ad}}, \pi_{\mathsf{ad}}) = 1)\big]\end{aligned} \tag{3.18}$$

Notice that combining equations 3.16, 3.17 and 3.18 gives that:

$$\Pr[\mathcal{P} \text{ wins}] = \Pr[\mathsf{bad}_1 = \mathsf{true}, \text{ in } G_1] > \epsilon(\lambda), \tag{3.19}$$

where ϵ is a negligible function. Therefore, we have constructed a prover $\mathcal{P}$ that wins the adaptive soundness game of the NIZK argument for the language L_{ad} with non-negligible probability, a contradiction to our initial assumption of the claim. This concludes the proof. □

Claim 3.15.9. *If the* NIZK *argument for language L_{eq} satisfies adaptive soundness, then for every stateful p.p.t. adversary $\mathcal{A}$, there exists a negligible function ϵ, such that for every $\lambda \in \mathbb{N}$:*

$$\big|\Pr[G_1 = 1] - \Pr[G_2 = 1]\big| \leq \epsilon(\lambda)$$

Proof. Assume towards a contradiction that $\big|\Pr[G_1 = 1] - \Pr[G_2 = 1]\big| > \epsilon(\lambda)$. Notice that since Games G_1 and G_2 are identical except for the check that determines the flag bad_2, we have:

$$\begin{aligned}\Pr[\mathsf{bad}_2 = \mathsf{true}, \text{in } G_2] &\geq \big|\Pr[G_1 = 1] - \Pr[G_2 = 1]\big| \\ &> \epsilon(\lambda)\end{aligned} \tag{3.20}$$

We want to show that if this is the case, then there exists some p.p.t. prover $\mathcal{P}^*$ that uses $\mathcal{A}$ to break the adaptive soundness of the NIZK argument for the language L_{eq}. Recall that in order to win this game, the prover $\mathcal{P}^*$ needs to find a pair $(\mathsf{stmt}_{\mathsf{eq}}, \pi_{\mathsf{eq}})$ such that it satisfies two conditions: $(\mathsf{stmt}_{\mathsf{eq}} \notin L_{\mathsf{eq}}) \wedge (\mathsf{Verify}_{\mathsf{eq}}(\mathsf{crs}_{\mathsf{eq}}, \mathsf{stmt}_{\mathsf{eq}}, \pi_{\mathsf{eq}}) = 1)$. So we have that $\Pr[\mathcal{P}^* \text{ wins}]$ is given by:

$$\begin{aligned}\Pr[\mathcal{P}^* \text{ wins}] = \Pr\big[&(\mathsf{stmt}_{\mathsf{eq}} \notin L_{\mathsf{eq}}) \\ &\wedge(\mathsf{Verify}_{\mathsf{eq}}(\mathsf{crs}_{\mathsf{eq}}, \mathsf{stmt}_{\mathsf{eq}}, \pi_{\mathsf{eq}}) = 1)\big]\end{aligned} \tag{3.21}$$

Notice that the winning condition of $\mathcal{P}^*$ is similar to that of the winning condition for $\mathcal{P}$ in Claim 3.15.8. Further, the hop from G_1 to G_2 is similar to that of the hop from G_0 to G_1. Therefore, the rest of this proof follows from the proof of Claim 3.15.8. This concludes the proof. □

Claim 3.15.10. *If the underlying adaptor signature scheme* AS *satisfies witness extractability, then for every stateful p.p.t. adversary $\mathcal{A}$, there exists a negligible function ϵ, such that for every $\lambda \in \mathbb{N}$:*

$$\Pr[G_2 = 1] \leq \epsilon(\lambda)$$

Proof. Assume towards a contradiction that $\Pr[G_2 = 1] > \epsilon(\lambda)$ that is, there exists some p.p.t. adversary $\mathcal{A}$ that wins Game G_2 with non-negligible probability. We want to show that if this is the case, then we can construct a reduction $\mathcal{B}$ that uses the adversary $\mathcal{A}$ to break the witness extractability of the underlying AS scheme. Recall that in order for $\mathcal{B}$ to win its game, it needs to find $(m^*, \mathsf{pk}_{\mathsf{EG}}, \sigma^*)$ such that the following conditions are satisfied: $(m^* \notin Q) \wedge \mathsf{Vrfy}(\mathsf{vk}, m^*, \sigma^*) = 1 \wedge (\mathsf{pk}_{\mathsf{EG}}, \mathsf{AS.Ext}(\mathsf{vk}, \mathsf{pk}_{\mathsf{EG}}, \widetilde{\sigma}_2^*, \sigma^*)) \notin \mathsf{R}_{\mathsf{EG}}$. Let $\mathsf{sk}'_{\mathsf{EG}} = \mathsf{AS.Ext}(\mathsf{vk}, \mathsf{pk}_{\mathsf{EG}}, \widetilde{\sigma}_2^*, \sigma^*)$, so we get that:

$$\begin{aligned}\Pr[\mathcal{B} \text{ wins}] = \Pr\big[&(m^* \notin Q) \\ &\wedge \mathsf{Vrfy}(\mathsf{vk}, m^*, \sigma^*) = 1 \wedge (\mathsf{pk}_{\mathsf{EG}}, \mathsf{sk}'_{\mathsf{EG}}) \notin \mathsf{R}_{\mathsf{EG}}\big]\end{aligned} \tag{3.22}$$

The reduction $\mathcal{B}$ in Figure 3.48 will simultaneously play the role of challenger to adversary $\mathcal{A}$ as in Game G_2, and the role of the adversary in the witness extractability game for the underlying AS scheme. Let $\mathcal{C}$ be the corresponding challenger for the witness extractability game for the underlying AS scheme. The procedure for the reduction is as follows:

1. $\mathcal{C}$ honestly samples the signing and verification keys $(\mathsf{sk}, \mathsf{vk})$, and sends vk to $\mathcal{B}$.

Reduction $\mathcal{B}$ for the proof of Claim 3.15.10

$Q = \emptyset, \mathsf{bad}_1 := \mathsf{false}, \mathsf{bad}_2 := \mathsf{false}$
$\mathsf{crs}_{\mathsf{ad}} \leftarrow \mathsf{Setup}_{\mathsf{ad}}(1^\lambda)$
$\mathsf{crs}_{\mathsf{eq}} \leftarrow \mathsf{Setup}_{\mathsf{eq}}(1^\lambda)$
$\mathsf{pp} := (\mathsf{crs}_{\mathsf{ad}}, \mathsf{crs}_{\mathsf{eq}})$
$\mathsf{vk} \leftarrow \mathcal{C}$
$(\mathsf{ad}^*, m^*, Y^*, f^*, \widetilde{\sigma}_1^*) \leftarrow \mathcal{A}^{\mathcal{O}_s, \mathcal{O}_{fps_2}}(\mathsf{pp}, \mathsf{vk})$
Parse ad^* as $(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, \pi_{\mathsf{ad}})$
$\mathsf{stmt}_{\mathsf{ad}} := (\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, Y^*)$
Parse $\widetilde{\sigma}_1^*$ as $(\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \pi_{\mathsf{eq}})$
$\widehat{\mathsf{ct}} := \mathsf{FHE.Eval}(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, f^*)$
$\mathsf{stmt}_{\mathsf{eq}} := (\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \mathsf{pk}_{\mathsf{FHE}}, \widehat{\mathsf{ct}})$
if $\big(\mathsf{Verify}_{\mathsf{ad}}(\mathsf{crs}_{\mathsf{ad}}, \mathsf{stmt}_{\mathsf{ad}}, \pi_{\mathsf{ad}}) = 0 \vee (f^* \notin \mathcal{F})$
$\quad \vee \mathsf{Verify}_{\mathsf{eq}}(\mathsf{crs}_{\mathsf{eq}}, \mathsf{stmt}_{\mathsf{eq}}, \pi_{\mathsf{eq}}) = 0\big)$: **return** $\perp$
$\widetilde{\sigma}_2^* \leftarrow \mathcal{C}(m^*, \mathsf{pk}_{\mathsf{EG}})$
$\sigma^* \leftarrow \mathcal{A}^{\mathcal{O}_s, \mathcal{O}_{fps_2}}(\widetilde{\sigma}_2^*)$
return $(m^*, \mathsf{pk}_{\mathsf{EG}}, \sigma^*)$ to $\mathcal{C}$

Oracle $\mathcal{O}_s(m)$

$\sigma \leftarrow \mathcal{C}(m)$
$Q := Q \cup \{m\}$
return σ

Oracle $\mathcal{O}_{fps_2}(\mathsf{ad}, m, Y, f, \widetilde{\sigma}_1)$

Parse ad as $(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, \pi_{\mathsf{ad}})$
Parse $\widetilde{\sigma}_1$ as $(\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \pi_{\mathsf{eq}})$
$\widehat{\mathsf{ct}} := \mathsf{FHE.Eval}(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, f)$
$\mathsf{stmt}_{\mathsf{eq}} := (\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \mathsf{pk}_{\mathsf{FHE}}, \widehat{\mathsf{ct}})$
if $(\mathsf{Verify}_{\mathsf{eq}}(\mathsf{crs}_{\mathsf{eq}}, \mathsf{stmt}_{\mathsf{eq}}, \pi_{\mathsf{eq}}) = 0)$:
$\quad$ **return** $\perp$
$\widetilde{\sigma}_2 \leftarrow \mathcal{C}(m, \mathsf{pk}_{\mathsf{EG}})$
$Q := Q \cup \{m\}$
return $\widetilde{\sigma}_2$

Figure 3.48: Reduction for $\mathcal{B}$ playing the witness extractability game for AS with challenger $\mathcal{C}$

2. $\mathcal{B}$ generates the public parameters pp, and then sends $(\mathsf{pp}, \mathsf{vk})$ to $\mathcal{A}$.

3. $\mathcal{A}$ performs queries to the signature and functional pre-signature oracles, $\mathcal{O}_s$ and $\mathcal{O}_{fps_2}$. Note that $\mathcal{B}$ answers these queries by forwarding the queries to its challenger, $\mathcal{C}$. After making queries, $\mathcal{A}$ then sends a response $(\mathsf{ad}^*, m^*, Y^*, f^*, \widetilde{\sigma}_1^*)$ to $\mathcal{B}$.

4. $\mathcal{B}$ then uses $\mathcal{C}$ to generate a honest pre-signature $\widetilde{\sigma}_2^*$ for $\mathcal{A}$'s chosen message m^* and statement $\mathsf{pk}_{\mathsf{EG}}$. $\mathcal{B}$ then forwards this pre-signature $\widetilde{\sigma}_2^*$ to $\mathcal{A}$.

5. After making more queries, $\mathcal{A}$ then sends a response σ^* to $\mathcal{B}$.

6. Then, $\mathcal{B}$ sends $(m^*, \mathsf{pk}_{\mathsf{EG}}, \sigma^*)$ to $\mathcal{C}$.

Now, we need to show that when $\mathcal{A}$ wins Game G_2, the winning conditions for $\mathcal{B}$ are also met. When $\mathcal{A}$ wins Game G_2, we know that the following occur:

(1) $\big(\mathsf{Verify}_{\mathsf{ad}}(\mathsf{crs}_{\mathsf{ad}}, \mathsf{stmt}_{\mathsf{ad}}, \pi_{\mathsf{ad}}) = 1\big)$

$\wedge \big(\mathsf{Verify}_{\mathsf{eq}}(\mathsf{crs}_{\mathsf{eq}}, \mathsf{stmt}_{\mathsf{eq}}, \pi_{\mathsf{eq}}) = 1\big) \wedge \big(f^* \in \mathcal{F}\big)$

(2) $\big(\mathsf{bad}_1 = \mathsf{false}\big) \wedge \big(\mathsf{bad}_2 = \mathsf{false}\big)$

(3) $(m^* \notin Q) \wedge \mathsf{Vrfy}(\mathsf{vk}, m^*, \sigma^*) = 1$

$$\wedge\, z \notin \{f^*(y) \mid \exists y \text{ s.t } (Y^*, y) \in \mathsf{R}\}$$

From (3), we know that two of $\mathcal{B}$'s winning conditions, $(m^* \notin Q) \wedge \mathsf{Vrfy}(\mathsf{vk}, m^*, \sigma^*) = 1$, are satisfied when $\mathcal{A}$ wins G_2. Therefore, it remains to show that $\mathcal{A}$ winning G_2 implies $\mathcal{B}$'s last winning condition: $(\mathsf{pk}_{\mathsf{EG}}, \mathsf{AS.Ext}(\mathsf{vk}, \mathsf{pk}_{\mathsf{EG}}, \widetilde{\sigma}_2^*, \sigma^*)) \notin \mathsf{R}_{\mathsf{EG}}$. In this case, that is, $(\mathsf{pk}_{\mathsf{EG}}, \mathsf{sk}'_{\mathsf{EG}}) \notin \mathsf{R}_{\mathsf{EG}}$.

Suppose towards a contradiction that $(\mathsf{pk}_{\mathsf{EG}}, \mathsf{sk}'_{\mathsf{EG}}) \in \mathsf{R}_{\mathsf{EG}}$. Notice that when $\mathcal{A}$ wins G_2, we have that condition (2) is satisfied, so we know that the statements $\mathsf{stmt}_{\mathsf{ad}}$ and $\mathsf{stmt}_{\mathsf{eq}}$ are valid statements for their corresponding languages. This fact is vital for the correctness of the analysis below.

When condition (1) is satisfied, we have that $\mathsf{Verify}_{\mathsf{ad}}(\mathsf{crs}_{\mathsf{ad}}, \mathsf{stmt}_{\mathsf{ad}}, \pi_{\mathsf{ad}}) = 1$, which means that $\exists (y, r_1, r_0)$ such that $(Y^*, y) \in \mathsf{R}$ and $\mathsf{ct}_{\mathsf{ad}} = \mathsf{FHE.Enc}(\mathsf{pk}_{\mathsf{FHE}}, y; r_1)$. We also know from condition (1) that $\mathsf{Verify}_{\mathsf{eq}}(\mathsf{crs}_{\mathsf{eq}}, \mathsf{stmt}_{\mathsf{eq}}, \pi^*_{\mathsf{eq}}) = 1$, which means that $\exists (\mathsf{sk}_{\mathsf{FHE}}, z, r_2, r_0)$ such that $\mathsf{ct} = \mathsf{EG.Enc}(\mathsf{pk}_{\mathsf{EG}}, z; r_2)$, and $\mathsf{FHE.Dec}(\mathsf{sk}_{\mathsf{FHE}}, \widehat{\mathsf{ct}}) = z$, where $\widehat{\mathsf{ct}}$ is the functional evaluation of $\mathsf{ct}_{\mathsf{ad}}$ (which we showed above was correctly generated). Since $\widehat{\mathsf{ct}}$ was correctly generated, we know that $z = f^*(y)$, where $f^* \in \mathcal{F}$. Since $(\mathsf{pk}_{\mathsf{EG}}, \mathsf{sk}'_{\mathsf{EG}}) \in \mathsf{R}_{\mathsf{EG}}$, we know that $z = \mathsf{EG.Dec}(\mathsf{sk}'_{\mathsf{EG}}, \mathsf{ct})$ is a correct decryption, which outputs $z = f^*(y)$. This gives that $z \in \{f^*(y) \mid \exists y \text{ s.t } (Y^*, y) \in \mathsf{R}\}$, which contradicts one of the win conditions of G_2 (see condition (3)). Therefore, when $\mathcal{A}$ wins G_2, it must be the case that $(\mathsf{pk}_{\mathsf{EG}}, \mathsf{sk}'_{\mathsf{EG}}) \notin \mathsf{R}_{\mathsf{EG}}$.

This means that when $\mathcal{A}$ wins G_2, it generates $(m^*, \mathsf{pk}_{\mathsf{EG}}, \sigma^*)$ that satisfy the three conditions $(m^* \notin Q) \wedge \mathsf{Vrfy}(\mathsf{vk}, m^*, \sigma^*) = 1 \wedge (\mathsf{pk}_{\mathsf{EG}}, \mathsf{sk}'_{\mathsf{EG}}) \notin \mathsf{R}$. We then obtain the following result.

$$\begin{aligned} \Pr\big[(m^* \notin Q) \wedge \mathsf{Vrfy}(\mathsf{vk}, m^*, \sigma^*) = 1 \\ \wedge (\mathsf{pk}_{\mathsf{EG}}, \mathsf{sk}'_{\mathsf{EG}}) \notin \mathsf{R}_{\mathsf{EG}}\big] \geq \Pr[G_2 = 1] \end{aligned} \tag{3.23}$$

Notice that combining equations 3.22 and 3.23, along with the assumption that $\Pr[G_2 = 1] > \epsilon(\lambda)$:

$$\Pr[\mathcal{B} \text{ wins}] \geq \Pr[G_2 = 1] > \epsilon(\lambda) \tag{3.24}$$

where ϵ is a negligible function. Therefore, we have constructed an adversary $\mathcal{B}$ that wins the witness extractability game of the underlying AS scheme with non-negligible probability, a contradiction to our initial assumption of the claim. This concludes the proof. □

Claim 3.15.11. *If the* NIZK *arguments for the languages L_{ad} and L_{eq} both satisfy adaptive soundness, and the underlying* AS *scheme satisfies witness extractability, then for every stateful p.p.t. adversary $\mathcal{A}$, there exists a negligible function ϵ, such that for every $\lambda \in \mathbb{N}$:*

$$\Pr\Big[\mathsf{fWitExt}_{\mathcal{A},\mathsf{FAS}}(1^\lambda) = 1\Big] \leq \epsilon(\lambda)$$

Proof. Since Game G_0 (as defined in Figure 3.46) is the original game $\mathsf{fWitExt}_{\mathcal{A},\mathsf{FAS}}$ for the FAS scheme we propose, we have that $\Pr[G_0 = 1] = \Pr\big[\mathsf{fWitExt}_{\mathcal{A},\mathsf{FAS}}(1^\lambda) = 1\big]$. Notice that the triangle inequality gives

that:

$$\begin{aligned}\Pr[G_0 = 1] &\leq \big|\Pr[G_0 = 1] - \Pr[G_1 = 1]\big| + \Pr[G_1 = 1] \\ &\leq \big|\Pr[G_0 = 1] - \Pr[G_1 = 1]\big| \\ &\quad + \big|\Pr[G_1 = 1] - \Pr[G_2 = 1]\big| + \Pr[G_2 = 1]\end{aligned}$$

For every stateful p.p.t. adversary $\mathcal{A}$, we have the following:

- Since the NIZK argument for the language L_{ad} satisfies adaptive soundness, Claim 3.15.8 guarantees that there exists some negligible function ϵ_1 such that for every $\lambda \in \mathbb{N}$, we have that $\big|\Pr[G_0 = 1] - \Pr[G_1 = 1]\big| \leq \epsilon_1(\lambda)$
- Since the NIZK argument for the language L_{eq} satisfies adaptive soundness, Claim 3.15.9 guarantees that there exists some negligible function ϵ_2 such that for every $\lambda \in \mathbb{N}$, we have that $\big|\Pr[G_1 = 1] - \Pr[G_2 = 1]\big| \leq \epsilon_2(\lambda)$
- Since the underlying adaptor signature scheme AS satisfies witness extractability, Claim 3.15.10 guarantees that there exists some negligible function ϵ_3 such that for every $\lambda \in \mathbb{N}$, we have that $\Pr[G_2 = 1] \leq \epsilon_3(\lambda)$

Combining these results with the expression obtained above for $\Pr[G_0 = 1]$, we have that:

$$\Pr[G_0 = 1] \leq \epsilon_1(\lambda) + \epsilon_2(\lambda) + \epsilon_3(\lambda) = \epsilon(\lambda) \tag{3.25}$$

Notice that since $\epsilon_1(\lambda), \epsilon_2(\lambda)$, and $\epsilon_3(\lambda)$ are all negligible functions, we have that the function $\epsilon(\lambda) = \epsilon_1(\lambda) + \epsilon_2(\lambda) + \epsilon_3(\lambda)$, for all $\lambda \in \mathbb{N}$, is also a negligible function. Note that since we can find an $\epsilon_1(\lambda), \epsilon_2(\lambda)$, and $\epsilon_3(\lambda)$ for any stateful p.p.t. adversary $\mathcal{A}$, we can then find a negligible function $\epsilon(\lambda)$ for any stateful p.p.t. adversary $\mathcal{A}$ as well.

Therefore, we have shown that for every stateful p.p.t. adversary $\mathcal{A}$, there exists a negligible function ϵ, such that for every $\lambda \in \mathbb{N}$, we have that $\Pr[G_0 = 1] \leq \epsilon(\lambda)$. This means that for any stateful p.p.t. adversary $\mathcal{A}$, we have that $\Pr\big[\mathsf{fWitExt}_{\mathcal{A},\mathsf{FAS}}(1^\lambda) = 1\big] \leq \epsilon(\lambda)$ as well. This concludes the proof of witness extractability for the proposed FAS construction.

□

Theorem 3.15.12 (Zero Knowledge (non-abort) for FAS Construction)**.** *If the* NIZK *arguments for the languages* L_{ad} *and* L_{eq} *both satisfy zero knowledge, and the* FHE *encryption scheme satisfies semantic security, then the proposed* FAS *construction in Figure 3.34 satisfies zero knowledge (non-abort) as defined in Definition 3.14.4.*

Overview. In order to prove the zero knowledge (non-abort) of FAS, we need to show that for every stateful p.p.t. adversary $\mathcal{A}$, there exists a stateful p.p.t. simulator Sim and a negligible function ϵ, such that for all p.p.t. distinguishers $\mathcal{D}$, for all $\lambda \in \mathbb{N}$, and all $(Y, y) \in \mathsf{R}$:

$$\left| \Pr\left[\mathcal{D}\left(\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] - \Pr\left[\mathcal{D}\left(\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \right| \leq \epsilon(\lambda)$$

First, we need to construct the stateful p.p.t. simulator $\mathsf{Sim} = (\mathsf{Setup}^*, \mathsf{AdGen}^*, \mathsf{FPreSign}_1^*, \mathsf{Adapt}^*)$. Let $\mathsf{Sim}_{\mathsf{ad}} = (\mathsf{Setup}^*_{\mathsf{ad}}, \mathsf{Prove}^*_{\mathsf{ad}})$ be the simulator (as defined in Definition 3.4.29) for the NIZK argument for the language L_{ad}. Similarly, let $\mathsf{Sim}_{\mathsf{eq}} = (\mathsf{Setup}^*_{\mathsf{eq}}, \mathsf{Prove}^*_{\mathsf{eq}})$ be the simulator for the NIZK argument for the language L_{eq}. We then define our simulator Sim as in Figure 3.49.

Next, consider the sequence of games $\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}$, $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}$, $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}$, $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$ as described in Figure 3.50.

- The first game is $\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}$ for the FAS scheme we propose. In this game, the adversary $\mathcal{A}$ is interacting with a challenger who has access to the witness y. The adversary is able to run as many interactions as it wants, and we then we examine the view of $\mathcal{A}$.
- The game $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}$ is identical to the first game, except that it uses the simulator $\mathsf{Sim}_{\mathsf{ad}}$'s methods $(\mathsf{Setup}^*_{\mathsf{ad}}, \mathsf{Prove}^*_{\mathsf{ad}})$ to generate the proof of correctness for $\mathsf{ct}_{\mathsf{ad}}$. This involves generating the pair $(\mathsf{crs}^*_{\mathsf{ad}}, \mathsf{td}_{\mathsf{ad}})$ during setup, and using the trapdoor $\mathsf{td}_{\mathsf{ad}}$ instead of $\mathsf{wit}_{\mathsf{ad}}$ during the proof. Since we no longer need $\mathsf{wit}_{\mathsf{ad}}$, we set it to $\perp$, to indicate we no longer need to use y for the proof.
- The game $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}$ is identical to $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}$, except that it uses the simulator $\mathsf{Sim}_{\mathsf{eq}}$'s methods $(\mathsf{Setup}^*_{\mathsf{eq}}, \mathsf{Prove}^*_{\mathsf{eq}})$ to generate the proof of equality for ct and $\widehat{\mathsf{ct}}$. This involves generating the pair $(\mathsf{crs}^*_{\mathsf{eq}}, \mathsf{td}_{\mathsf{eq}})$ during setup, and using the trapdoor $\mathsf{td}_{\mathsf{eq}}$ instead of $\mathsf{wit}_{\mathsf{eq}}$ during the proof. Since we no longer need $\mathsf{wit}_{\mathsf{eq}}$, we set it to $\perp$ to indicate this.
- The last game is $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$ for the FAS scheme we propose. Note that this game is identical to $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}$ except that $\mathsf{ct}_{\mathsf{ad}}$ now encrypts 0, rather than y. In this game, the adversary $\mathcal{A}$ is interacting with a challenger who has access to only the function $f(y)$ of the witness y. The adversary is able to run as many interactions as it wants, and we then we examine the view of $\mathcal{A}$.

First, we will argue in Claim 3.15.14 that the hop from $\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}$ to $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}$ is close due to the zero knowledge property of the NIZK argument for the language L_{ad}. Similarly, we argue in Claim 3.15.15 that

$\mathsf{Setup}^*(1^\lambda)$

1 : $(\mathsf{crs}^*_{\mathsf{ad}}, \mathsf{td}_{\mathsf{ad}}) \leftarrow \mathsf{Setup}^*_{\mathsf{ad}}(1^\lambda)$
2 : $(\mathsf{crs}^*_{\mathsf{eq}}, \mathsf{td}_{\mathsf{eq}}) \leftarrow \mathsf{Setup}^*_{\mathsf{eq}}(1^\lambda)$
3 : $\mathsf{pp} := (\mathsf{crs}^*_{\mathsf{ad}}, \mathsf{crs}^*_{\mathsf{eq}})$
4 : return pp

$\mathsf{AdGen}^*(\mathsf{pp}, Y)$

1 : Parse pp as $(\mathsf{crs}^*_{\mathsf{ad}}, \mathsf{crs}^*_{\mathsf{eq}})$
2 : $(\mathsf{sk}_{\mathsf{FHE}}, \mathsf{pk}_{\mathsf{FHE}}) := \mathsf{FHE.KGen}(1^\lambda; r_0)$
3 : $\mathsf{ct}_{\mathsf{ad}} := \mathsf{FHE.Enc}(\mathsf{pk}_{\mathsf{FHE}}, 0; r_1)$
4 : $\mathsf{stmt}_{\mathsf{ad}} := (\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, Y)$
5 : $\pi^*_{\mathsf{ad}} \leftarrow \mathsf{Prove}^*_{\mathsf{ad}}(\mathsf{crs}^*_{\mathsf{ad}}, \mathsf{stmt}_{\mathsf{ad}}, \mathsf{td}_{\mathsf{ad}})$
6 : $\mathsf{ad} := (\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, \pi^*_{\mathsf{ad}})$
7 : $\mathsf{st} := (\mathsf{sk}_{\mathsf{FHE}})$
8 : return $(\mathsf{ad}, \mathsf{st})$

$\mathsf{FPreSign}^*_1(\mathsf{pp}, \mathsf{ad}, \mathsf{st}, Y, f(y), f)$

1 : Parse st as $(\mathsf{sk}_{\mathsf{FHE}})$
2 : Parse ad as $(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, \pi^*_{\mathsf{ad}})$
3 : $\widehat{\mathsf{ct}} := \mathsf{FHE.Eval}(f, \mathsf{ct}_{\mathsf{ad}})$
4 : $(\mathsf{sk}_{\mathsf{EG}}, \mathsf{pk}_{\mathsf{EG}}) \leftarrow \mathsf{ElGamal.KGen}(1^\lambda)$
5 : $z := f(y)$
6 : $\mathsf{ct} := \mathsf{ElGamal.Enc}(\mathsf{pk}_{\mathsf{EG}}, z; r_2)$
7 : $\mathsf{stmt}_{\mathsf{eq}} := (\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \mathsf{pk}_{\mathsf{FHE}}, \widehat{\mathsf{ct}})$
8 : $\pi^*_{\mathsf{eq}} \leftarrow \mathsf{Prove}^*_{\mathsf{eq}}(\mathsf{crs}^*_{\mathsf{eq}}, \mathsf{stmt}_{\mathsf{eq}}, \mathsf{td}_{\mathsf{eq}})$
9 : Update $\mathsf{st} = (\mathsf{sk}_{\mathsf{FHE}}, \mathsf{sk}_{\mathsf{EG}})$
10 : return $\widetilde{\sigma}_1 := (\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \pi^*_{\mathsf{eq}})$

$\mathsf{Adapt}^*(\mathsf{ad}, \mathsf{st}, \mathsf{vk}, m, Y, f(y), f, (\widetilde{\sigma}_1, \widetilde{\sigma}_2))$

1 : Parse st as $(\mathsf{sk}_{\mathsf{FHE}}, \mathsf{sk}_{\mathsf{EG}})$
2 : return $\sigma := \mathsf{AS.Adapt}(\mathsf{vk}, m, \mathsf{sk}_{\mathsf{EG}}, \widetilde{\sigma}_2)$

Figure 3.49: Simulator Sim for zero knowledge (non-abort) of FAS proof

the hop from $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}$ to $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}$ is close due to the zero knowledge property of the NIZK argument for the language L_{eq}. Finally, we prove in Claim 3.15.16 that the hop from $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}$ to $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$ is close due to the semantic security of the FHE encryption scheme. The formal proof for Theorem 3.15.12 can be found in Claim 3.15.17.

□

Remark 3.15.13. *To simply notation throughout the next few proofs, let E_{ad} be the event that:*

Games $\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)$, $\boxed{\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)}$, $\boxed{\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)}$, $\boxed{\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)}$

1 : $\mathsf{crs}_{\mathsf{ad}} \leftarrow \mathsf{Setup}_{\mathsf{ad}}(1^\lambda)$ $\boxed{(\mathsf{crs}^*_{\mathsf{ad}}, \mathsf{td}_{\mathsf{ad}}) \leftarrow \mathsf{Setup}^*_{\mathsf{ad}}(1^\lambda)}$

2 : $\mathsf{crs}_{\mathsf{eq}} \leftarrow \mathsf{Setup}_{\mathsf{eq}}(1^\lambda)$ $\boxed{(\mathsf{crs}^*_{\mathsf{eq}}, \mathsf{td}_{\mathsf{eq}}) \leftarrow \mathsf{Setup}^*_{\mathsf{eq}}(1^\lambda)}$

3 : $\mathsf{pp} := (\mathsf{crs}_{\mathsf{ad}}, \mathsf{crs}_{\mathsf{eq}})$ $\boxed{\mathsf{pp} := (\mathsf{crs}^*_{\mathsf{ad}}, \mathsf{crs}_{\mathsf{eq}})}$ $\boxed{\mathsf{pp} := (\mathsf{crs}^*_{\mathsf{ad}}, \mathsf{crs}^*_{\mathsf{eq}})}$

4 : $(\mathsf{sk}_{\mathsf{FHE}}, \mathsf{pk}_{\mathsf{FHE}}) := \mathsf{FHE.KGen}(1^\lambda; r_0)$

5 : $\mathsf{ct}_{\mathsf{ad}} := \mathsf{FHE.Enc}(\mathsf{pk}_{\mathsf{FHE}}, y; r_1)$ $\boxed{\mathsf{ct}_{\mathsf{ad}} := \mathsf{FHE.Enc}(\mathsf{pk}_{\mathsf{FHE}}, 0; r_1)}$

6 : $\mathsf{stmt}_{\mathsf{ad}} := (\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, Y)$

7 : $\mathsf{wit}_{\mathsf{ad}} := (y, r_1, r_0)$ $\boxed{\mathsf{wit}_{\mathsf{ad}} := \perp}$

8 : $\pi_{\mathsf{ad}} \leftarrow \mathsf{Prove}_{\mathsf{ad}}(\mathsf{crs}_{\mathsf{ad}}, \mathsf{stmt}_{\mathsf{ad}}, \mathsf{wit}_{\mathsf{ad}})$ $\boxed{\pi_{\mathsf{ad}} \leftarrow \mathsf{Prove}^*_{\mathsf{ad}}(\mathsf{crs}^*_{\mathsf{ad}}, \mathsf{stmt}_{\mathsf{ad}}, \mathsf{td}_{\mathsf{ad}})}$

9 : $\mathsf{ad} := (\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, \pi_{\mathsf{ad}})$

10 : $\mathsf{st} := (\mathsf{sk}_{\mathsf{FHE}})$

11 : $(m, f, \mathsf{vk}) \leftarrow \mathcal{A}(\mathsf{pp}, \mathsf{ad}, Y)$

12 : while $\mathcal{A}$ is not done making queries:

13 : $\quad \widehat{\mathsf{ct}} := \mathsf{FHE.Eval}(f, \mathsf{ct}_{\mathsf{ad}})$

14 : $\quad (\mathsf{sk}_{\mathsf{EG}}, \mathsf{pk}_{\mathsf{EG}}) \leftarrow \mathsf{ElGamal.KGen}(1^\lambda)$

15 : $\quad z := f(y)$

16 : $\quad \mathsf{ct} := \mathsf{ElGamal.Enc}(\mathsf{pk}_{\mathsf{EG}}, z; r_2)$

17 : $\quad \mathsf{stmt}_{\mathsf{eq}} := (\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \mathsf{pk}_{\mathsf{FHE}}, \widehat{\mathsf{ct}})$

18 : $\quad \mathsf{wit}_{\mathsf{eq}} := (\mathsf{sk}_{\mathsf{FHE}}, z, r_2, r_0)$ $\boxed{\mathsf{wit}_{\mathsf{eq}} := \perp}$

19 : $\quad \pi_{\mathsf{eq}} \leftarrow \mathsf{Prove}_{\mathsf{eq}}(\mathsf{crs}_{\mathsf{eq}}, \mathsf{stmt}_{\mathsf{eq}}, \mathsf{wit}_{\mathsf{eq}})$ $\boxed{\pi_{\mathsf{eq}} \leftarrow \mathsf{Prove}^*_{\mathsf{eq}}(\mathsf{crs}^*_{\mathsf{eq}}, \mathsf{stmt}_{\mathsf{eq}}, \mathsf{td}_{\mathsf{eq}})}$

20 : $\quad$ Update $\mathsf{st} := (\mathsf{sk}_{\mathsf{FHE}}, \mathsf{sk}_{\mathsf{EG}})$

21 : $\quad \widetilde{\sigma}_1 := (\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \pi_{\mathsf{eq}})$

22 : $\quad \widetilde{\sigma}_2 \leftarrow \mathcal{A}(\widetilde{\sigma}_1)$

23 : $\quad$ if $(\mathsf{AS.pVrfy}(\mathsf{vk}, m, \mathsf{pk}_{\mathsf{EG}}, \widetilde{\sigma}_2) = 0)$:

$\quad\quad$ **return** $\perp$

24 : $\quad \sigma := \mathsf{AS.Adapt}(\mathsf{vk}, m, \mathsf{sk}_{\mathsf{EG}}, \widetilde{\sigma}_2)$

25 : $\quad (m, f) \leftarrow \mathcal{A}(\sigma)$

26 : return view of $\mathcal{A}$

Figure 3.50: Zero Knowledge (non-abort) Proof: Games $\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}$, $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}$, $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}$, and $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$

$$\mathsf{crs}_{\mathsf{eq}} \leftarrow \mathsf{Setup}_{\mathsf{eq}}(1^\lambda),$$
$$\pi_{\mathsf{eq}} \leftarrow \mathsf{Prove}_{\mathsf{eq}}(\mathsf{crs}_{\mathsf{eq}}, \mathsf{stmt}_{\mathsf{eq}}, \mathsf{wit}_{\mathsf{eq}})$$

and let E^*_{ad} *be the event that:*

$$(\mathsf{crs}^*{}_{\mathsf{ad}}, \mathsf{td}_{\mathsf{ad}}) \leftarrow \mathsf{Setup}^*_{\mathsf{ad}}(1^\lambda),$$
$$\pi^*_{\mathsf{ad}} \leftarrow \mathsf{Prove}^*_{\mathsf{ad}}(\mathsf{crs}^*_{\mathsf{ad}}, \mathsf{stmt}_{\mathsf{ad}}, \mathsf{td}_{\mathsf{ad}})$$

Claim 3.15.14. *If the* NIZK *argument for the language* L_{ad} *satisfies zero knowledge, then for every stateful p.p.t. adversary* $\mathcal{A}$*, there exists a negligible function* ϵ*, such that for all p.p.t. distinguishers* $\mathcal{D}$*, for all* $\lambda \in \mathbb{N}$*, for all* $(Y, y) \in \mathsf{R}$*:*

$$\left| \Pr\left[\mathcal{D}\left(\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] - \Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \right| \leq \epsilon(\lambda)$$

Proof. Assume towards a contradiction that:

$$\left| \Pr\left[\mathcal{D}\left(\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] - \Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \right| > \epsilon(\lambda) \tag{3.26}$$

We want to show that if this is the case, then there exists a p.p.t. distinguisher $\mathcal{D}^*$ that uses $\mathcal{D}$ to break the zero knowledge of the NIZK argument for the language L_{ad}.

Recall that since the NIZK argument for the language L_{ad} satisfies zero knowledge, for any p.p.t. distinguisher $\mathcal{D}^*$, we have that for all statement/witness pairs $(\mathsf{stmt}_{\mathsf{ad}}, \mathsf{wit}_{\mathsf{ad}})$:

$$P_1 = \Pr\left[\mathcal{D}^*(\mathsf{crs}_{\mathsf{ad}}, \pi_{\mathsf{ad}}) = 1 : E_{\mathsf{ad}}\right],$$
$$P_2 = \Pr\left[\mathcal{D}^*(\mathsf{crs}^*_{\mathsf{ad}}, \pi^*_{\mathsf{ad}}) = 1 : E^*_{\mathsf{ad}}\right],$$
$$|P_1 - P_2| \leq \epsilon(\lambda) \tag{3.27}$$

Suppose that for some statement $\mathsf{stmt}_{\mathsf{ad}} = (\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, Y)$, the inputs to $\mathcal{D}^*$ are some common reference string $\mathsf{crs}_{\mathsf{ad}}$ and a proof π_{ad}. The distinguisher $\mathcal{D}^*$ can use this information to generate the public parameters $\mathsf{pp} := (\mathsf{crs}_{\mathsf{ad}}, \mathsf{crs}_{\mathsf{eq}})$, as well as the advertisement $\mathsf{ad} := (\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, \pi_{\mathsf{ad}})$. The distinguisher $\mathcal{D}^*$ then passes $(\mathsf{pp}, \mathsf{ad}, Y)$ to the adversary $\mathcal{A}$. Note that since the two games $\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}$ and $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}$ are the same during the while loop, the only differences in $\mathcal{A}$'s view are from the inputs received before the while loop. Therefore, it is sufficient to have $\mathcal{D}^*$ only simulate the inputs before the while loop.

After $\mathcal{D}^*$ has sent $(\mathsf{pp}, \mathsf{ad}, Y)$ to the adversary $\mathcal{A}$, it then runs the distinguisher $\mathcal{D}$ on $\mathcal{A}$. Then, $\mathcal{D}^*$ returns whatever $\mathcal{D}$ returns. We then have that:

$$\begin{aligned} P_1 =& \Pr\left[\mathcal{D}^*(\mathsf{crs}_{\mathsf{ad}}, \pi_{\mathsf{ad}}) = 1 : E_{\mathsf{ad}}\right] \\ =& \Pr\left[\mathcal{D}(\mathsf{pp}, \mathsf{ad}, Y) = 1 : E_{\mathsf{ad}}\right] \\ =& \Pr\left[\mathcal{D}\left(\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \end{aligned}$$

Similarly, we also get that:

$$\begin{aligned} P_2 =& \Pr\left[\mathcal{D}^*(\mathsf{crs}^*_{\mathsf{ad}}, \pi^*_{\mathsf{ad}}) = 1 : E^*_{\mathsf{ad}}\right] \\ =& \Pr\left[\mathcal{D}(\mathsf{pp}, \mathsf{ad}, Y) = 1 : E^*_{\mathsf{ad}}\right] \\ =& \Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \end{aligned}$$

We then have that:

$$\begin{aligned} |P_1 - P_2| = \Bigg| &\Pr\left[\mathcal{D}\left(\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \\ &- \Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \Bigg| \end{aligned} \tag{3.28}$$

Combining equations 3.26 and 3.28, we obtain that $|P_1 - P_2| > \epsilon(\lambda)$, a contradiction to equation 3.27. This concludes the proof.

□

Claim 3.15.15. *If the* NIZK *argument for the language L_{eq} satisfies zero knowledge, then for every stateful adversary $\mathcal{A}$, there exists a negligible function ϵ, such that for all p.p.t. distinguishers $\mathcal{D}$, for all $\lambda \in \mathbb{N}$, for all $(Y, y) \in \mathsf{R}$:*

$$\begin{aligned} \Bigg| &\Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \\ &- \Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad},\mathsf{eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \Bigg| \leq \epsilon(\lambda) \end{aligned}$$

Proof. Assume towards a contradiction that:

$$\begin{aligned} \Bigg| &\Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \\ &- \Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad},\mathsf{eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \Bigg| > \epsilon(\lambda) \end{aligned} \tag{3.29}$$

We want to show that if this is the case, then there exists a p.p.t. distinguisher $\mathcal{D}^*$ that uses $\mathcal{D}$ to break the zero knowledge of the NIZK argument for the language L_{eq}.

Recall that since the NIZK argument for the language L_{eq} satisfies zero knowledge, for any p.p.t. distinguisher $\mathcal{D}^*$, we have that for all statement/witness pairs $(\mathsf{stmt}_{\mathsf{eq}}, \mathsf{wit}_{\mathsf{eq}})$:

$$\begin{aligned} P_1 &= \Pr\left[\mathcal{D}^*(\mathsf{crs}_{\mathsf{eq}}, \pi_{\mathsf{eq}}) = 1 : E_{\mathsf{ad}}\right], \\ P_2 &= \Pr\left[\mathcal{D}^*(\mathsf{crs}^*_{\mathsf{eq}}, \pi^*_{\mathsf{eq}}) = 1 : E^*_{\mathsf{ad}}\right], \\ |P_1 - P_2| &\leq \epsilon(\lambda) \end{aligned} \tag{3.30}$$

Suppose that for some statement $\mathsf{stmt}_{\mathsf{eq}} = (\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \mathsf{pk}_{\mathsf{FHE}}, \widehat{\mathsf{ct}})$, the inputs to $\mathcal{D}^*$ are some common reference string $\mathsf{crs}_{\mathsf{eq}}$ and a proof π_{eq}. The distinguisher $\mathcal{D}^*$ can use this information to generate the information

$\widetilde{\sigma}_1 = (\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \pi_{\mathsf{eq}})$. The distinguisher can then pass the information $\widetilde{\sigma}_1 = (\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \pi_{\mathsf{eq}})$ to the adversary $\mathcal{A}$. Note that since the two games $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}$ and $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}$ are only different during the while loop, the only differences in $\mathcal{A}$'s view are from the inputs received during the while loop. The rest of the simulation proceeds as normal in each of the games.

After $\mathcal{D}^*$ has sent $\widetilde{\sigma}_1 = (\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \pi_{\mathsf{eq}})$ to the adversary $\mathcal{A}$, it then runs the distinguisher $\mathcal{D}$. Then, $\mathcal{D}^*$ returns whatever $\mathcal{D}$ returns. We then have that:

$$\begin{aligned} P_1 =& \Pr\left[\mathcal{D}^*(\mathsf{crs}_{\mathsf{eq}}, \pi_{\mathsf{eq}}) = 1 : E_{\mathsf{ad}}\right] \\ =& \Pr\left[\mathcal{D}(\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \pi_{\mathsf{eq}}) = 1 : E_{\mathsf{ad}}\right] \\ =& \Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \end{aligned}$$

Similarly, we also get that:

$$\begin{aligned} P_2 =& \Pr\left[\mathcal{D}^*(\mathsf{crs}^*_{\mathsf{eq}}, \pi^*_{\mathsf{eq}}) = 1 : E^*_{\mathsf{ad}}\right] \\ =& \Pr\left[\mathcal{D}(\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \pi_{\mathsf{eq}}) = 1 : E^*_{\mathsf{ad}}\right] \\ =& \Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \end{aligned}$$

We then have that:

$$\begin{aligned} |P_1 - P_2| = \Big| &\Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \\ &- \Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \Big| \end{aligned} \tag{3.31}$$

Combining equations 3.29 and 3.31, we obtain that $|P_1 - P_2| > \epsilon(\lambda)$, a contradiction to equation 3.30. This concludes the proof.

□

Claim 3.15.16. *If the* FHE *scheme satisfies semantic security, then for every stateful p.p.t. adversary $\mathcal{A}$, there exists a negligible function ϵ, such that for all p.p.t. distinguishers $\mathcal{D}$, for all $\lambda \in \mathbb{N}$, and for all $(Y, y) \in \mathsf{R}$:*

$$\begin{aligned} \Big| &\Pr\Big[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\Big] \\ &- \Pr\Big[\mathcal{D}\left(\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\Big] \Big| \leq \epsilon(\lambda) \end{aligned}$$

Proof. Assume towards a contradiction that for any stateful p.p.t. adversary $\mathcal{A}$, there exists a p.p.t. distinguisher $\mathcal{D}$ such that for all $\lambda \in \mathbb{N}$, and for all $(Y, y) \in \mathsf{R}$:

$$\begin{aligned} \Big| &\Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \\ &- \Pr\left[\mathcal{D}\left(\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \Big| > \epsilon(\lambda) \end{aligned} \tag{3.32}$$

Notice that the games $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}$ and $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$ are exactly the same, except for the way in which $\mathsf{ct}_{\mathsf{ad}}$ is generated. In $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}$, $\mathsf{ct}_{\mathsf{ad}}$ is a FHE encryption of the witness y, while in $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$, $\mathsf{ct}_{\mathsf{ad}}$ is a FHE encryption of 0. We want to show that if the p.p.t. distinguisher $\mathcal{D}$ exists (as we assumed above), then we can break the semantic security of the FHE scheme.

We will show that if such a distinguisher $\mathcal{D}$ exists, then we can create a reduction $\mathcal{B}$ that breaks the semantic security of the FHE encryption scheme. Let $\mathcal{C}$ be the challenger for the semantic security game for the FHE encryption scheme. $\mathcal{B}$ plays the role of the adversary in this semantic security game. $\mathcal{B}$ also serves as the challenger to adversary $\mathcal{A}$ playing the games $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}$ and $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$. The procedure for the reduction is as follows:

1. $\mathcal{C}$ honestly samples the FHE public parameters pp', and sends pp' to $\mathcal{B}$.

2. $\mathcal{B}$ completes the rest of the setup necessary to generate the public parameters pp for the game $\mathcal{A}$ is playing. In this case, this involves performing the setup for the NIZK arguments.

3. $\mathcal{B}$ then sends the two values $(0, y)$ to $\mathcal{C}$. $\mathcal{C}$ then responds with the FHE encryption ($\mathsf{ct}_{\mathsf{ad}}$) of one of them, along with the $\mathsf{pk}_{\mathsf{FHE}}$ used to generate the encryption. Note that if $\mathcal{C}$ chooses to encrypt 0, then $\mathcal{A}$'s view will be as in the game $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$. Similarly, if $\mathcal{C}$ chooses to encrypt y, then $\mathcal{A}$'s view will be as in the game $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}$.

4. $\mathcal{B}$ then generates a proof π_{ad} for $\mathsf{ct}_{\mathsf{ad}}$, and creates the advertisement $\mathsf{ad} := (\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, \pi_{\mathsf{ad}})$. Afterwards, $\mathcal{B}$ sends the information $(\mathsf{pp}, \mathsf{ad}, Y)$ to the adversary $\mathcal{A}$.

5. The adversary $\mathcal{A}$ is then able to query $\mathcal{B}$ as much as it wants.

6. After $\mathcal{A}$ is done making queries, $\mathcal{B}$ uses the distinguisher $\mathcal{D}$ on $\mathcal{A}$. Then, $\mathcal{B}$ simply forwards the output of $\mathcal{D}$ to the challenger $\mathcal{C}$.

Since the only difference between the two games $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}$ and $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$ is the way $\mathsf{ct}_{\mathsf{ad}}$ is encrypted, if $\mathcal{D}$ can distinguish between the games with some probability p, then that means that $\mathcal{D}$ can tell which value $\mathsf{ct}_{\mathsf{ad}}$ encrypts with the same probability p. Since $\mathcal{B}$ outputs the result that $\mathcal{D}$ outputs, we can then express the probability that $\mathcal{B}$ wins the semantic security game as follows:

$$\begin{aligned}\Pr[\mathcal{B}\text{ wins}] \geq \Big|&\Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \\ &-\Pr\left[\mathcal{D}\left(\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right]\Big|\end{aligned} \tag{3.33}$$

Combining equations 3.32 and 3.33, we have that $\Pr[\mathcal{B} \text{ wins}] > \epsilon(\lambda)$, where ϵ is a negligible function. Therefore, we have constructed an adversary $\mathcal{B}$ that breaks the semantic security of the FHE encryption scheme, which is a contradiction to the assumptions made in the claim. This concludes the proof.

□

Claim 3.15.17. *If the* NIZK *arguments for the languages* L_{ad} *and* L_{eq} *both satisfy zero knowledge, and the* FHE *encryption scheme satisfies semantic security, then for all stateful p.p.t. adversaries* $\mathcal{A}$*, there exists a negligible function* ϵ *such that for all p.p.t. distinguishers* $\mathcal{D}$*, for all* $\lambda \in \mathbb{N}$*, and for all* $(Y, y) \in \mathsf{R}$ *, we have that:*

$$\left| \Pr\left[\mathcal{D}\left(\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] - \Pr\left[\mathcal{D}\left(\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \right| \leq \epsilon(\lambda),$$

where Sim *is the simulator defined in Figure 3.49, and the two games* $\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)$ *and* $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)$ *are defined as in Figure 3.50.*

Proof. Using the triangle inequality multiple times gives that:

$$
\begin{aligned}
&\Pr[\mathcal{D}(\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)) = 1] \\
\leq\ &\Big|\Pr\left[\mathcal{D}\left(\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \\
&- \Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right]\Big| \\
&+ \Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \\
\leq\ &\Big|\Pr\left[\mathcal{D}\left(\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \\
&- \Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right]\Big| \\
&+ \Big|\Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \\
&- \Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right]\Big| \\
&+ \Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \\
\leq\ &\Big|\Pr\left[\mathcal{D}\left(\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \\
&- \Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right]\Big| \\
&+ \Big|\Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \\
&- \Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right]\Big| \\
&+ \Big|\Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \\
&- \Pr\left[\mathcal{D}\left(\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right]\Big| \\
&+ \Pr\left[\mathcal{D}\left(\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right]
\end{aligned}
$$

Since the NIZK argument for the language L_{ad} satisfies zero knowledge, then Claim 3.15.14 guarantees that for every stateful p.p.t. adversary $\mathcal{A}$, there exists a negligible function ϵ_1 such that for all p.p.t. distinguishers $\mathcal{D}$, for all $\lambda \in \mathbb{N}$, and for all $(Y, y) \in \mathsf{R}$:

$$
\begin{aligned}
&\Big|\Pr\left[\mathcal{D}\left(\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \\
-&\Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right]\Big| \leq \epsilon_1(\lambda)
\end{aligned}
\tag{3.34}
$$

Similarly, since the NIZK argument for the language L_{eq} satisfies zero knowledge, then Claim 3.15.15 guarantees that for every stateful p.p.t. adversary $\mathcal{A}$, there exists a negligible function ϵ_2 such that for all p.p.t. distinguishers $\mathcal{D}$, for all $\lambda \in \mathbb{N}$, and for all $(Y, y) \in \mathsf{R}$:

$$\begin{aligned} &\Big|\Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \\ &-\Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right]\Big| \leq \epsilon_2(\lambda) \end{aligned} \tag{3.35}$$

Lastly, since the FHE encryption scheme satisfies semantic security, Claim 3.15.16 guarantees that for every stateful p.p.t. adversary $\mathcal{A}$, there exists a negligible function ϵ_3 such that for all p.p.t. distinguishers $\mathcal{D}$, for all $\lambda \in \mathbb{N}$, and for all $(Y, y) \in \mathsf{R}$:

$$\begin{aligned} &\Big|\Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \\ &-\Pr\left[\mathcal{D}\left(\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right]\Big| \leq \epsilon_3(\lambda) \end{aligned} \tag{3.36}$$

Combining the equations 3.34, 3.35, and 3.36 with the inequality above that was derived from the triangle inequality, we obtain that:

$$\begin{aligned} &\Pr\left[\mathcal{D}\left(\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \leq \\ &\Pr\left[\mathcal{D}\left(\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] + \epsilon_1(\lambda) \\ &+\epsilon_2(\lambda) + \epsilon_3(\lambda) \end{aligned}$$

Notice that since $\epsilon_1(\lambda), \epsilon_2(\lambda)$, and $\epsilon_3(\lambda)$ are all negligible functions, we have that the function $\epsilon(\lambda) = \epsilon_1(\lambda) + \epsilon_2(\lambda) + \epsilon_3(\lambda)$, for all $\lambda \in \mathbb{N}$, is also a negligible function. Note that since for any stateful p.p.t. adversary $\mathcal{A}$, for any p.p.t. distinguisher $\mathcal{D}$, and for all $(Y, y) \in \mathsf{R}$, we can always find the functions $\epsilon_1(\lambda), \epsilon_2(\lambda)$, and $\epsilon_3(\lambda)$, we can then also always find the negligible function $\epsilon(\lambda)$ for the same constraints. The inequality then becomes:

$$\begin{aligned} &\Pr\left[\mathcal{D}\left(\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \leq \\ &\Pr\left[\mathcal{D}\left(\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] + \epsilon(\lambda), \end{aligned}$$

which implies that:

$$\begin{aligned} &\Big|\Pr\left[\mathcal{D}\left(\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \\ &-\Pr\left[\mathcal{D}\left(\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right]\Big| \leq \epsilon(\lambda) \end{aligned} \tag{3.37}$$

Therefore, we have shown that for all stateful p.p.t. adversaries $\mathcal{A}$, there exists a negligible function ϵ, such that for all p.p.t. distinguishers $\mathcal{D}$, for all $\lambda \in \mathbb{N}$, and for all $(Y, y) \in \mathsf{R}$, we have that equation 3.37 holds, which is what we wanted to show. This concludes the proof of zero knowledge (non-abort) for the FAS scheme.

□

Theorem 3.15.18 (Zero Knowledge (abort) for FAS Construction). *If the* NIZK *arguments for the languages* L_{ad} *and* L_{eq} *both satisfy zero knowledge, the* FHE *encryption scheme satisfies semantic security, and the* ElGamal *encryption scheme satisfies semantic security, then the proposed* FAS *construction in Figure 3.34 satisfies zero knowledge (abort) as defined in Definition 3.14.5.*

Overview. In order to prove the zero knowledge (abort) of FAS, we need to show that for every stateful p.p.t. adversary $\mathcal{A}$ that aborts before providing $\widetilde{\sigma_2}$ to the honest seller, there exists a stateful p.p.t. simulator Sim and a negligible function ϵ, such that for all p.p.t. distinguishers $\mathcal{D}$, for all $\lambda \in \mathbb{N}$, and all $(Y, y) \in \mathsf{R}$:

$$\left| \Pr\left[\mathcal{D}\left(\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] - \Pr\left[\mathcal{D}\left(\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \right| \leq \epsilon(\lambda)$$

First, we need to construct the stateful p.p.t. simulator $\mathsf{Sim} = (\mathsf{Setup}^*, \mathsf{AdGen}^*, \mathsf{FPreSign}_1^*)$. Let $\mathsf{Sim}_{\mathsf{ad}} = (\mathsf{Setup}^*_{\mathsf{ad}}, \mathsf{Prove}^*_{\mathsf{ad}})$ be the simulator (as defined in Definition 3.4.29) for the NIZK argument for the language L_{ad}. Similarly, let $\mathsf{Sim}_{\mathsf{eq}} = (\mathsf{Setup}^*_{\mathsf{eq}}, \mathsf{Prove}^*_{\mathsf{eq}})$ be the simulator for the NIZK argument for the language L_{eq}. We then define our simulator Sim as in Figure 3.51.

Next, consider the sequence of games $\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}$, $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}$, $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}$, $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$ as described in Figure 3.52.

- The first game is $\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}$ for the FAS scheme we propose. In this game, the adversary $\mathcal{A}$ is interacting with a challenger who has access to the witness y. The adversary is able to run as many interactions as it wants, and we then we examine the view of $\mathcal{A}$.

- The game $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}$ is identical to the first game, except that it uses the simulator $\mathsf{Sim}_{\mathsf{ad}}$'s methods $(\mathsf{Setup}^*_{\mathsf{ad}}, \mathsf{Prove}^*_{\mathsf{ad}})$ to generate the proof of correctness for $\mathsf{ct}_{\mathsf{ad}}$. This involves generating the pair $(\mathsf{crs}^*_{\mathsf{ad}}, \mathsf{td}_{\mathsf{ad}})$ during setup, and using the trapdoor $\mathsf{td}_{\mathsf{ad}}$ instead of $\mathsf{wit}_{\mathsf{ad}}$ during the proof. Since we no longer need $\mathsf{wit}_{\mathsf{ad}}$, we set it to $\perp$, to indicate we no longer need to use y for the proof.

- The game $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}$ is identical to $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}$, except that it uses the simulator $\mathsf{Sim}_{\mathsf{eq}}$'s methods $(\mathsf{Setup}^*_{\mathsf{eq}}, \mathsf{Prove}^*_{\mathsf{eq}})$ to generate the proof of equality for ct and $\widehat{\mathsf{ct}}$. This involves generating the pair $(\mathsf{crs}^*_{\mathsf{eq}}, \mathsf{td}_{\mathsf{eq}})$ during setup, and using the trapdoor $\mathsf{td}_{\mathsf{eq}}$ instead of $\mathsf{wit}_{\mathsf{eq}}$ during the proof. Since we no longer need $\mathsf{wit}_{\mathsf{eq}}$, we set it to $\perp$ to indicate this.

- The last game is $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$ for the FAS scheme we propose. Note that this game is identical to $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}$ except that $\mathsf{ct}_{\mathsf{ad}}$ and ct now both encrypt 0, rather than y and $f(y)$ respectively. In this

$\mathsf{Setup}^*(1^\lambda)$

1 : $(\mathsf{crs}^*_{\mathsf{ad}}, \mathsf{td}_{\mathsf{ad}}) \leftarrow \mathsf{Setup}^*_{\mathsf{ad}}(1^\lambda)$
2 : $(\mathsf{crs}^*_{\mathsf{eq}}, \mathsf{td}_{\mathsf{eq}}) \leftarrow \mathsf{Setup}^*_{\mathsf{eq}}(1^\lambda)$
3 : $\mathsf{pp} := (\mathsf{crs}^*_{\mathsf{ad}}, \mathsf{crs}^*_{\mathsf{eq}})$
4 : return pp

$\mathsf{AdGen}^*(\mathsf{pp}, Y)$

1 : Parse pp as $(\mathsf{crs}^*_{\mathsf{ad}}, \mathsf{crs}^*_{\mathsf{eq}})$
2 : $(\mathsf{sk}_{\mathsf{FHE}}, \mathsf{pk}_{\mathsf{FHE}}) := \mathsf{FHE.KGen}(1^\lambda; r_0)$
3 : $\mathsf{ct}_{\mathsf{ad}} := \mathsf{FHE.Enc}(\mathsf{pk}_{\mathsf{FHE}}, 0; r_1)$
4 : $\mathsf{stmt}_{\mathsf{ad}} := (\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, Y)$
5 : $\pi^*_{\mathsf{ad}} \leftarrow \mathsf{Prove}^*_{\mathsf{ad}}(\mathsf{crs}^*_{\mathsf{ad}}, \mathsf{stmt}_{\mathsf{ad}}, \mathsf{td}_{\mathsf{ad}})$
6 : $\mathsf{ad} := (\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, \pi^*_{\mathsf{ad}})$
7 : $\mathsf{st} := (\mathsf{sk}_{\mathsf{FHE}})$
8 : return $(\mathsf{ad}, \mathsf{st})$

$\mathsf{FPreSign}^*_1(\mathsf{pp}, \mathsf{ad}, \mathsf{st}, Y, f)$

1 : Parse st as $(\mathsf{sk}_{\mathsf{FHE}})$
2 : Parse ad as $(\mathsf{pk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{ad}}, \pi^*_{\mathsf{ad}})$
3 : $\widehat{\mathsf{ct}} := \mathsf{FHE.Eval}(f, \mathsf{ct}_{\mathsf{ad}})$
4 : $(\mathsf{sk}_{\mathsf{EG}}, \mathsf{pk}_{\mathsf{EG}}) \leftarrow \mathsf{ElGamal.KGen}(1^\lambda)$
5 : $z := 0$
6 : $\mathsf{ct} := \mathsf{ElGamal.Enc}(\mathsf{pk}_{\mathsf{EG}}, z; r_2)$
7 : $\mathsf{stmt}_{\mathsf{eq}} := (\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \mathsf{pk}_{\mathsf{FHE}}, \widehat{\mathsf{ct}})$
8 : $\pi^*_{\mathsf{eq}} \leftarrow \mathsf{Prove}^*_{\mathsf{eq}}(\mathsf{crs}^*_{\mathsf{eq}}, \mathsf{stmt}_{\mathsf{eq}}, \mathsf{td}_{\mathsf{eq}})$
9 : Update $\mathsf{st} = (\mathsf{sk}_{\mathsf{FHE}}, \mathsf{sk}_{\mathsf{EG}})$
10 : return $\widetilde{\sigma}_1 := (\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \pi^*_{\mathsf{eq}})$

Figure 3.51: Simulator Sim for zero knowledge (abort) of FAS proof

game, the adversary $\mathcal{A}$ is interacting with a challenger who has access to no information of the witness y. The adversary is able to run as many interactions as it wants, and we then we examine the view of $\mathcal{A}$.

First, we will argue in Claim 3.15.19 that the hop from $\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}$ to $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}$ is close due to the zero knowledge property of the NIZK argument for the language L_{ad}. Similarly, we argue in Claim 3.15.20 that the hop from $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}$ to $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}$ is close due to the zero knowledge property of the NIZK argument for the language L_{eq}. Finally, we prove in Claim 3.15.21 that the hop from $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}$ to $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$ is close due to the semantic security of the FHE encryption scheme and the semantic security of the ElGamal encryption scheme. The formal proof for Theorem 3.15.18 can be found in Claim 3.15.22.

□

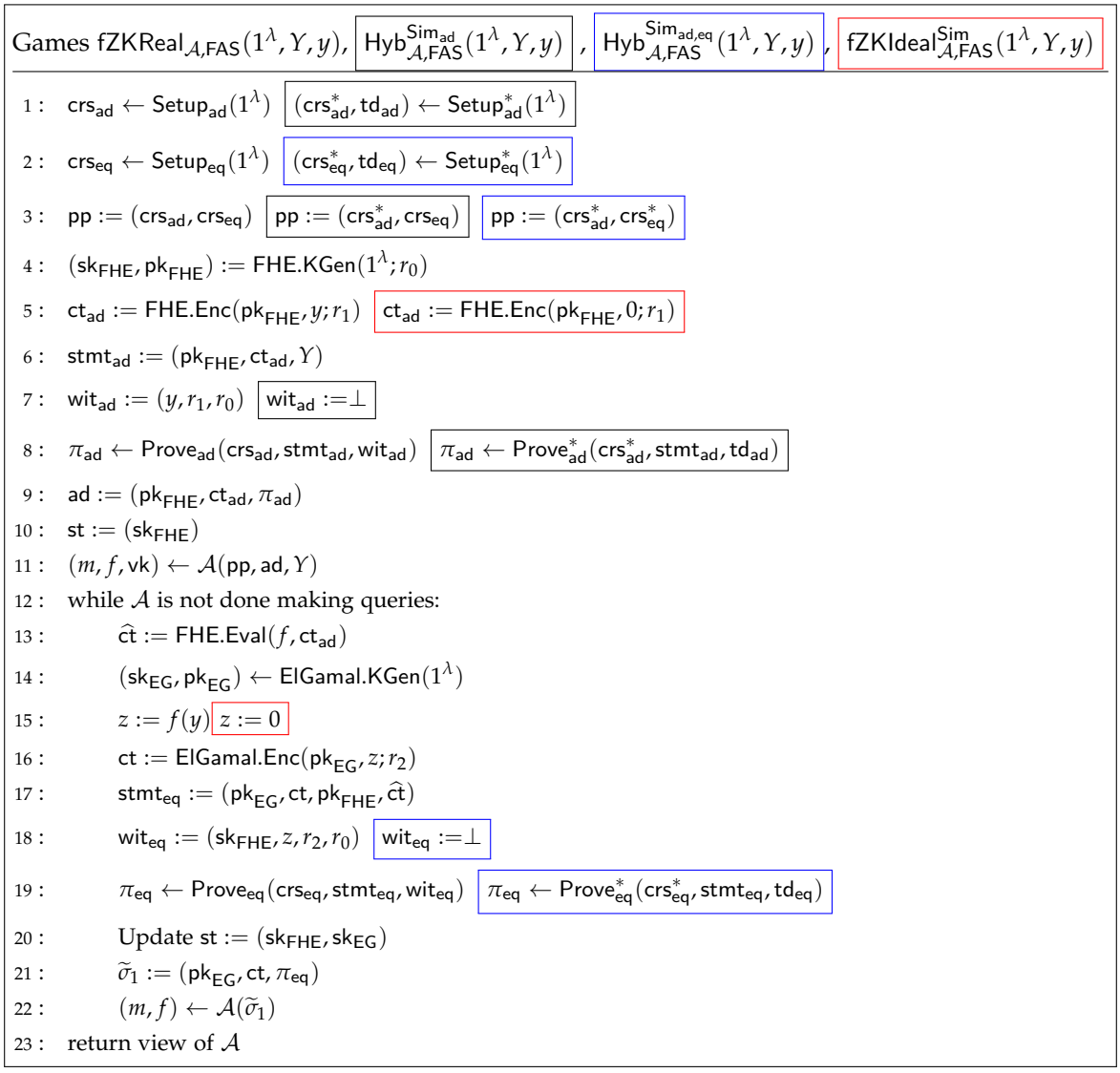


Figure 3.52: Zero Knowledge (abort) Proof: Games $\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}$, $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}$, $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}$, and $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$

Claim 3.15.19. *If the* NIZK *argument for the language L_{ad} satisfies zero knowledge, then for every stateful p.p.t. adversary $\mathcal{A}$ that aborts before providing $\widetilde{\sigma}_2$ to the honest seller, there exists a negligible function ϵ, such that for all p.p.t. distinguishers $\mathcal{D}$, for all $\lambda \in \mathbb{N}$, for all $(Y, y) \in \mathsf{R}$:*

$$\left| \Pr\left[\mathcal{D}\left(\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] - \Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \right| \leq \epsilon(\lambda)$$

Proof. Similar to proof of Claim 3.15.14. □

Claim 3.15.20. *If the* NIZK *argument for the language L_{eq} satisfies zero knowledge, then for every stateful adversary $\mathcal{A}$ that aborts before providing $\widetilde{\sigma}_2$ to the honest seller, there exists a negligible function ϵ, such that for all p.p.t.*

distinguishers $\mathcal{D}$, for all $\lambda \in \mathbb{N}$, for all $(Y, y) \in \mathsf{R}$:

$$\left| \Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] - \Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \right| \leq \epsilon(\lambda)$$

Proof. Similar to proof of Claim 3.15.15. □

Claim 3.15.21. *If the* FHE *scheme satisfies semantic security and the* ElGamal *encryption scheme satisfies semantic security, then for every stateful p.p.t. adversary $\mathcal{A}$ that aborts before providing $\widetilde{\sigma_2}$ to the honest seller, there exists a negligible function ϵ, such that for all p.p.t. distinguishers $\mathcal{D}$, for all $\lambda \in \mathbb{N}$, and for all $(Y, y) \in \mathsf{R}$:*

$$\left| \Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] - \Pr\left[\mathcal{D}\left(\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \right| \leq \epsilon(\lambda)$$

Proof. We prove by introducing an intermediate hybrid $\overline{\mathsf{Hyb}}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)$ that is same as $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)$ except that only the ciphertext $\mathsf{ct}_{\mathsf{ad}}$ is changed to encrypt 0. Observe that we can reduce the indistinguishability of hybrids $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)$ and $\overline{\mathsf{Hyb}}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)$ to the semantic security of the FHE encryption scheme. This proof is similar to the proof of Claim 3.15.16. Therefore, all that remains to show is that hybrids $\overline{\mathsf{Hyb}}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)$ and $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)$ are indistinguishable. Observe that $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)$ is same as $\overline{\mathsf{Hyb}}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)$ except that the ElGamal ciphertext ct encrypts 0. We reduce the indistinguishability of the two hybrids to the semantic security of the ElGamal encryption scheme next. Note that if the adversary makes Q number of queries (m_i, f_i), then, we need to change all the Q ElGamal ciphertexts ct_i encrypting $f_i(y)$ to encryptions of 0 and this will require Q number of hybrids, where in i^{th} hybrid we switch ct_i. For sake of simplicity, we present the proof below for $Q = 1$, that is, a single query. And the proof for Q queries follows similarly via a hybrid argument.

Assume towards a contradiction that for any stateful p.p.t. adversary $\mathcal{A}$, there exists a p.p.t. distinguisher $\mathcal{D}$ such that for all $\lambda \in \mathbb{N}$, and for all $(Y, y) \in \mathsf{R}$:

$$\left| \Pr\left[\mathcal{D}\left(\overline{\mathsf{Hyb}}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] - \Pr\left[\mathcal{D}\left(\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \right| > \epsilon(\lambda) \tag{3.38}$$

Notice that the games $\overline{\mathsf{Hyb}}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)$ and $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)$ are exactly the same, except for the way in which ElGamal ciphertext ct is generated. In $\overline{\mathsf{Hyb}}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)$, ct is an ElGamal encryption of $f(y)$, while in $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)$, ct is an ElGamal encryption of 0. We want to show that if the p.p.t.

distinguisher $\mathcal{D}$ exists (as we assumed above), then we can break the semantic security of the ElGamal encryption scheme.

We will show that if such a distinguisher $\mathcal{D}$ exists, then we can create a reduction $\mathcal{B}$ that breaks the semantic security of the ElGamal encryption scheme. Let $\mathcal{C}$ be the challenger for the semantic security game for the ElGamal encryption scheme. $\mathcal{B}$ plays the role of the adversary in this semantic security game. $\mathcal{B}$ also serves as the challenger to adversary $\mathcal{A}$ playing the games $\overline{\mathsf{Hyb}}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)$ and $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)$. The procedure for the reduction is as follows:

1. $\mathcal{C}$ honestly samples the ElGamal key pair $(\mathsf{sk}_{\mathsf{EG}}, \mathsf{pk}_{\mathsf{EG}})$, and sends $\mathsf{pk}_{\mathsf{EG}}$ to $\mathcal{B}$.

2. $\mathcal{B}$ completes the setup necessary to generate the public parameters pp for the game $\mathcal{A}$ is playing. In this case, this involves performing the setup for the NIZK arguments. Next, $\mathcal{B}$ samples the FHE key pair and computes $\mathsf{ct}_{\mathsf{ad}}$ as FHE encryption of 0. Next, $\mathcal{B}$ computes π_{ad} using the $\mathsf{NIZK}_{\mathsf{ad}}$ simulator, computes ad and sends $\mathsf{pp}, \mathsf{ad}, Y$ to the adversary ad. $\mathcal{B}$ obtains (m, f, vk) from $\mathcal{A}$. $\mathcal{B}$ computes $\widehat{\mathsf{ct}}$ as the FHE evaluation of f on $\mathsf{ct}_a dvt$.

3. $\mathcal{B}$ sends the challenge messages $(f(y), 0)$ to the challenger $\mathcal{C}$ and obtains a ciphertext ct from it. Based on this, it sets $\mathsf{stmt}_{\mathsf{eq}} := (\mathsf{pk}_{\mathsf{EG}}, \mathsf{ctpk}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{FHE}})$. Next, $\mathcal{B}$ computes π_{eq} using the $\mathsf{NIZK}_{\mathsf{eq}}$ simulator and sends $\widetilde{\sigma_1} := (\mathsf{pk}_{\mathsf{EG}}, \mathsf{ct}, \pi_{\mathsf{eq}})$ to the adversary $\mathcal{A}$.

4. At this point, the interaction with $\mathcal{A}$ ends since it does not return any $\widetilde{\sigma_2}$. So, $\mathcal{B}$ uses the distinguisher $\mathcal{D}$ on $\mathcal{A}$. Then, $\mathcal{B}$ simply forwards the output of $\mathcal{D}$ to the challenger $\mathcal{C}$.

Since the only difference between the two games $\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}$ and $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}$ is the way ct is encrypted, if $\mathcal{D}$ can distinguish between the games with some probability p, then that means that $\mathcal{D}$ can tell which value ct encrypts with the same probability p. Since $\mathcal{B}$ outputs the result that $\mathcal{D}$ outputs, we can then express the probability that $\mathcal{B}$ wins the semantic security game as follows:

$$\begin{aligned}\Pr[\mathcal{B}\text{ wins}] \geq \Bigg|&\Pr\left[\mathcal{D}\left(\overline{\mathsf{Hyb}}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \\ &-\Pr\left[\mathcal{D}\left(\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right]\Bigg|\end{aligned} \tag{3.39}$$

Combining equations 3.38 and 3.39, we have that $\Pr[\mathcal{B}\text{ wins}] > \epsilon(\lambda)$, where ϵ is a negligible function. Therefore, we have constructed an adversary $\mathcal{B}$ that breaks the semantic security of the ElGamal encryption scheme, which is a contradiction to the assumptions made in the claim. This concludes the proof.

□

Claim 3.15.22. *If the* NIZK *arguments for the languages* L_{ad} *and* L_{eq} *both satisfy zero knowledge, the* FHE *encryption scheme satisfies semantic security, and the* ElGamal *encryption scheme satisfies semantic security, then for all stateful p.p.t. adversaries* $\mathcal{A}$*, there exists a negligible function* ϵ *such that for all p.p.t. distinguishers* $\mathcal{D}$*, for all* $\lambda \in \mathbb{N}$*, and for all* $(Y, y) \in \mathsf{R}$ *, we have that:*

$$\Big|\Pr\Big[\mathcal{D}\left(\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\Big] - \Pr\Big[\mathcal{D}\left(\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\Big]\Big| \leq \epsilon(\lambda),$$

where Sim *is the simulator defined in Figure 3.51, and the two games* $\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)$ *and* $\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)$ *are defined as in Figure 3.52.*

Proof. Using the triangle inequality multiple times gives that:

$$\begin{aligned}
&\Pr[\mathcal{D}(\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)) = 1] \\
\leq\ &\Big|\Pr\Big[\mathcal{D}\left(\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\Big] \\
&\quad - \Pr\Big[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\Big]\Big| \\
&\quad + \Pr\Big[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\Big] \\
\leq\ &\Big|\Pr\Big[\mathcal{D}\left(\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\Big] \\
&\quad - \Pr\Big[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\Big]\Big| \\
&\quad + \Big|\Pr\Big[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\Big] \\
&\quad - \Pr\Big[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad},\mathsf{eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\Big]\Big| \\
&\quad + \Pr\Big[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad},\mathsf{eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\Big] \\
\leq\ &\Big|\Pr\Big[\mathcal{D}\left(\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\Big] \\
&\quad - \Pr\Big[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\Big]\Big| \\
&\quad + \Big|\Pr\Big[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\Big] \\
&\quad - \Pr\Big[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad},\mathsf{eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\Big]\Big| \\
&\quad + \Big|\Pr\Big[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad},\mathsf{eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\Big] \\
&\quad - \Pr\Big[\mathcal{D}\left(\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\Big]\Big| \\
&\quad + \Pr\Big[\mathcal{D}\left(\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\Big]
\end{aligned}$$

Since the NIZK argument for the language L_{ad} satisfies zero knowledge, then Claim 3.15.19 guarantees that for every stateful p.p.t. adversary $\mathcal{A}$ that aborts before providing $\widetilde{\sigma_2}$ to the honest seller, there exists a negligible function ϵ_1 such that for all p.p.t. distinguishers $\mathcal{D}$, for all $\lambda \in \mathbb{N}$, and for all $(Y, y) \in \mathsf{R}$:

$$\left| \Pr\left[\mathcal{D}\left(\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] - \Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \right| \leq \epsilon_1(\lambda) \tag{3.40}$$

Similarly, since the NIZK argument for the language L_{eq} satisfies zero knowledge, then Claim 3.15.20 guarantees that for every stateful p.p.t. adversary $\mathcal{A}$ that aborts before providing $\widetilde{\sigma_2}$ to the honest seller, there exists a negligible function ϵ_2 such that for all p.p.t. distinguishers $\mathcal{D}$, for all $\lambda \in \mathbb{N}$, and for all $(Y, y) \in \mathsf{R}$:

$$\left| \Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] - \Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \right| \leq \epsilon_2(\lambda) \tag{3.41}$$

Lastly, since the FHE and ElGamal encryption schemes satisfies semantic security, Claim 3.15.21 guarantees that for every stateful p.p.t. adversary $\mathcal{A}$ that aborts before providing $\widetilde{\sigma_2}$ to the honest seller, there exists a negligible function ϵ_3 such that for all p.p.t. distinguishers $\mathcal{D}$, for all $\lambda \in \mathbb{N}$, and for all $(Y, y) \in \mathsf{R}$:

$$\left| \Pr\left[\mathcal{D}\left(\mathsf{Hyb}^{\mathsf{Sim}_{\mathsf{ad,eq}}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] - \Pr\left[\mathcal{D}\left(\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \right| \leq \epsilon_3(\lambda) \tag{3.42}$$

Combining the equations 3.40, 3.41, and 3.42 with the inequality above that was derived from the triangle inequality, we obtain that:

$$\begin{aligned} &\Pr\left[\mathcal{D}\left(\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \leq \\ &\Pr\left[\mathcal{D}\left(\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] + \epsilon_1(\lambda) \\ &+\epsilon_2(\lambda) + \epsilon_3(\lambda) \end{aligned}$$

Notice that since $\epsilon_1(\lambda), \epsilon_2(\lambda)$, and $\epsilon_3(\lambda)$ are all negligible functions, we have that the function $\epsilon(\lambda) = \epsilon_1(\lambda) + \epsilon_2(\lambda) + \epsilon_3(\lambda)$, for all $\lambda \in \mathbb{N}$, is also a negligible function. Note that since for any stateful p.p.t. adversary $\mathcal{A}$, for any p.p.t. distinguisher $\mathcal{D}$, and for all $(Y, y) \in \mathsf{R}$, we can always find the functions $\epsilon_1(\lambda), \epsilon_2(\lambda)$, and $\epsilon_3(\lambda)$, we can then also always find the negligible function $\epsilon(\lambda)$ for the same constraints. The inequality then becomes:

$$\begin{aligned} &\Pr\left[\mathcal{D}\left(\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] \leq \\ &\Pr\left[\mathcal{D}\left(\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] + \epsilon(\lambda), \end{aligned}$$

which implies that:

$$\left|\Pr\left[\mathcal{D}\left(\mathsf{fZKReal}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right] - \Pr\left[\mathcal{D}\left(\mathsf{fZKIdeal}^{\mathsf{Sim}}_{\mathcal{A},\mathsf{FAS}}(1^\lambda, Y, y)\right) = 1\right]\right| \leq \epsilon(\lambda) \tag{3.43}$$

Therefore, we have shown that for all stateful p.p.t. adversaries $\mathcal{A}$, there exists a negligible function ϵ, such that for all p.p.t. distinguishers $\mathcal{D}$, for all $\lambda \in \mathbb{N}$, and for all $(Y, y) \in \mathsf{R}$, we have that equation 3.43 holds, which is what we wanted to show. This conlcudes the proof of zero knowledge (abort) for the FAS scheme.

□

3.16 Proofs: Cut-and-choose based $\mathsf{NIZK}_{\mathsf{eq}}$

Claim 3.16.1 (Completeness)**.** *If encryption schemes* FHE *and* EG *are linearly homomorphic, then* $\mathsf{NIZK}_{\mathsf{eq}}$ *in Figure 3.33 satisfies completeness.*

Proof. Follows in a straightforward way. □

Claim 3.16.2 (Adaptive Soundness)**.** $\mathsf{NIZK}_{\mathsf{eq}}$ *in Figure 3.33 satisfies adaptive soundness (Definition 3.4.28) in the random oracle model.*

Proof. We prove soundness via a sequence of two hybrid experiments Hyb_0 and Hyb_1. Hyb_0 corresponds to the original soundness game against an adversarial p.p.t. prover $\mathcal{P}^*$. Hyb_1 is same as Hyb_0 except the hash function $\mathbb{H}$ is replaced by a random function RF. Since we are in the random oracle model, Hyb_0 and Hyb_1 are identically distributed. Thus, it suffices to show that in Hyb_1, for all $\mathcal{P}^*$, there exists a negligible function ε such that for all $\lambda \in \mathbb{N}$, we have

$$\Pr\left[\mathsf{stmt} \notin L_{\mathsf{R}} \wedge \mathsf{Verify}(\mathsf{crs}, \mathsf{stmt}, \pi) = 1 \;:\; \begin{array}{c} \mathsf{crs} \leftarrow \mathsf{Setup}(\lambda), \\ (\mathsf{stmt}, \pi) \leftarrow \mathcal{P}(\mathsf{crs}) \end{array}\right] \leq \epsilon(\lambda).$$

We will show that for the negligible function $\epsilon(\lambda) = 1/2^\lambda$.

Suppose $\mathcal{P}^*$ on input crs returns (stmt, π), where $\mathsf{stmt} = (\mathsf{ct}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{EG}})$ and $\pi = (\{\mathsf{ct}_{\mathsf{FHE},i}, \mathsf{ct}_{\mathsf{EG},i}\}_{i\in[2\lambda]}, I, \{\widetilde{s}_i, \widetilde{r}_i, \widetilde{\overline{r}_i}\}_{i\in I}, \{\widehat{s}_i, \widehat{r}_i, \widehat{\overline{r}_i}\}_{i\notin I})$. Without loss of generality, suppose that $\mathsf{ct}_{\mathsf{FHE}} = \mathsf{FHE.Enc}(\mathsf{pk}_{\mathsf{FHE}}, z; r)$ and $\mathsf{ct}_{\mathsf{EG}} = \mathsf{EG.Enc}(\mathsf{pk}_{\mathsf{EG}}, \overline{z}; \overline{r})$ for some $z, \overline{z}, r$ and $\overline{r}$. Then, $\mathsf{stmt} \notin L_{\mathsf{eq}}^{\mathsf{pk}_{\mathsf{FHE}}, \mathsf{pk}_{\mathsf{EG}}}$ implies that $z \neq \overline{z}$. Further, $\mathsf{Verify}(\mathsf{crs}, \mathsf{stmt}, \pi) = 1$ implies all of the following:

- $I = \{2 \cdot i + str[i] : i \in [\lambda]\}$, where $str := \mathsf{RF}(\mathsf{ct}_{\mathsf{FHE}} || \mathsf{ct}_{\mathsf{EG}} || \mathsf{ct}_{\mathsf{FHE},1} || \ldots || \mathsf{ct}_{\mathsf{FHE},2\lambda} || \mathsf{ct}_{\mathsf{EG},1} || \ldots || \mathsf{ct}_{\mathsf{EG},2\lambda}) \in \{0,1\}^\lambda$.

- For all $i \in I$: $\widetilde{s_i} = s_i = \overline{s_i}$, $\widetilde{r_i} = r_i$ and, $\widetilde{\overline{r_i}} = \overline{r_i}$.
- For all $i \notin I$: $\widehat{s_i} = z + s_i = \overline{z} + \overline{s_i}$, $\widetilde{r_i} = r + r_i$ and, $\widetilde{\overline{r_i}} = \overline{r} + \overline{r_i}$. This implies that $\overline{s_i} = s_i + (z - \overline{z})$.

Observe that the cheating prover above can freely choose the variables $z, \overline{z}, r, \overline{r}, \{s_i, r_i, \overline{r_i}\}_{i \in [2\lambda]}$ and its task is reduced to guessing $str \in \{0,1\}^\lambda$ which determines the set I. It can then set $\{\overline{s_i} := s_i + (z - \overline{z})\}_{i \notin I}$. This fully determines the values of the input to the random function RF and thus its output $str' \in \{0,1\}^\lambda$ and the corresponding set I'. The prover wins if $I = I'$. This holds iff $str = str'$. Since RF is a random function, the probability of this event is $1/2^\lambda$. Thus, $\epsilon(\lambda) = 1/2^\lambda$. □

Claim 3.16.3 (Zero-Knowledge). $\mathsf{NIZK}_{\mathsf{eq}}$ *in Figure 3.33 satisfies zero-knowledge (Definition 3.4.29) in the programmable random oracle model.*

Proof. We start by describing the p.p.t. simulator $\mathsf{Sim} = (\mathsf{Setup}^*, \mathsf{Prove}^*)$. Let PRO be the programmable random oracle. Then, the simulator is as in Section 3.16.

$\mathsf{Setup}^*(1^\lambda)$:

1 : Sample $(\mathsf{hk}, \mathsf{td}) \leftarrow \mathsf{PRO.Gen}(1^\lambda)$
2 : **ret** $\mathsf{crs}^* := \mathsf{hk}$ and td

$\mathsf{Prove}^*(\mathsf{crs}^*, \mathsf{stmt} = (\mathsf{ct}_{\mathsf{FHE}}, \mathsf{ct}_{\mathsf{EG}}))$:

1 : Compute $str \xleftarrow{\$} \{0,1\}^\lambda$.
2 : Define set $I = \{2 \cdot i + str[i] : i \in [\lambda]\}$ of size λ.
3 : For all $i \in I$: Sample random messages $s_i \xleftarrow{\$} \mathbb{Z}_p$, random coins r_i and $\overline{r_i}$ and compute $\mathsf{ct}_{\mathsf{FHE},i} := \mathsf{FHE.Enc}(\mathsf{pk}_{\mathsf{FHE}}, s_i; r_i)$ and $\mathsf{ct}_{\mathsf{EG},i} := \mathsf{EG.Enc}(\mathsf{pk}_{\mathsf{EG}}, s_i; \overline{r_i})$.
4 : For all $i \notin I$: Sample random messages $\widehat{s_i} \xleftarrow{\$} \mathbb{Z}_p$, random coins $\widehat{r_i}$ and $\widehat{\overline{r_i}}$ and compute $\widehat{\mathsf{ct}_{\mathsf{FHE},i}} := \mathsf{FHE.Enc}(\mathsf{pk}_{\mathsf{FHE}}, \widehat{s_i}; \widehat{r_i})$, $\widehat{\mathsf{ct}_{\mathsf{EG},i}} := \mathsf{EG.Enc}(\mathsf{pk}_{\mathsf{EG}}, \widehat{s_i}; \widehat{\overline{r_i}})$, $\mathsf{ct}_{\mathsf{FHE},i} := \widehat{\mathsf{ct}_{\mathsf{FHE},i}} - \mathsf{ct}_{\mathsf{FHE}}$ and, $\mathsf{ct}_{\mathsf{EG},i} := \widehat{\mathsf{ct}_{\mathsf{EG},i}} / \mathsf{ct}_{\mathsf{EG}}$.
5 : Program PRO using td such that $\mathsf{PRO.H}(\mathsf{hk}, \mathsf{ct}_{\mathsf{FHE}} || \mathsf{ct}_{\mathsf{EG}} || \mathsf{ct}_{\mathsf{FHE},1} || \ldots || \mathsf{ct}_{\mathsf{FHE},2\lambda} || \mathsf{ct}_{\mathsf{EG},1} || \ldots || \mathsf{ct}_{\mathsf{EG},2\lambda}) := str$.
6 : **ret** $\pi := (\{\mathsf{ct}_{\mathsf{FHE},i}, \mathsf{ct}_{\mathsf{EG},i}\}_{i \in [2\lambda]}, I, \{s_i, r_i, \overline{r_i}\}_{i \in I}, \{\widehat{s_i}, \widehat{r_i}, \widehat{\overline{r_i}}\}_{i \notin I})$.

Having described the simulator Sim, we now show that there exists a negligible function ϵ such that for all p.p.t. distinguishers $\mathcal{D}$, $\lambda \in \mathbb{N}$, and statement/witness pairs $(\mathsf{stmt}, \mathsf{wit}) \in \mathsf{R}_{\mathsf{eq}}^{\mathsf{pk}_{\mathsf{FHE}}, \mathsf{pk}_{\mathsf{EG}}}$, we have that:

$$\left| \Pr[\mathcal{D}(\mathsf{crs}, \pi) = 1] - \Pr[\mathcal{D}(\mathsf{crs}^*, \pi^*) = 1] \right| \leq \epsilon(\lambda),$$

where $\mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda)$, $\pi \leftarrow \mathsf{Prove}(\mathsf{crs}, \mathsf{stmt}, \mathsf{wit})$,$\mathsf{crs}^*, \mathsf{td} \leftarrow \mathsf{Setup}^*(\lambda)$,$\pi^* \leftarrow \mathsf{Prove}^*(\mathsf{crs}^*, \mathsf{stmt}, \mathsf{td})$.

Let experiment $\mathsf{NIZKReal}_{\mathsf{NIZK}_{\mathsf{eq}}}(1^\lambda, \mathsf{stmt}, \mathsf{wit})$ return (crs, π), where $\mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda)$ and $\pi \leftarrow \mathsf{Prove}(\mathsf{crs}, \mathsf{stmt}, \mathsf{wit})$ and experiement $\mathsf{NIZKIdeal}^{\mathsf{Sim}}_{\mathsf{NIZK}_{\mathsf{eq}}}(1^\lambda, \mathsf{stmt})$ return (crs^*, π^*), where $(\mathsf{crs}^*, \mathsf{td}) \leftarrow \mathsf{Setup}^*(1^\lambda)$ and $\pi^* \leftarrow \mathsf{Prove}^*(\mathsf{crs}^*, \mathsf{td}, \mathsf{stmt})$. Then, it suffices to show

$$\left|\Pr\left[\mathcal{D}(\mathsf{NIZKReal}_{\mathsf{NIZK}_{\mathsf{eq}}}(1^\lambda,\mathsf{stmt},\mathsf{wit}))=1\right]-\Pr\left[\mathcal{D}(\mathsf{NIZKIdeal}^{\mathsf{Sim}}_{\mathsf{NIZK}_{\mathsf{eq}}}(1^\lambda,\mathsf{stmt}))=1\right]\right|\leq\epsilon(\lambda).$$

We show this through a sequence of hybrid experiments $\mathsf{Hyb}_0,\ldots,\mathsf{Hyb}_3$ that are as follows.

Experiment Hyb_0**:** This is same as $\mathsf{NIZKReal}_{\mathsf{NIZK}_{\mathsf{eq}}}(1^\lambda,\mathsf{stmt},\mathsf{wit})$.

Experiment Hyb_1**:** Same as Hyb_0 except that the hash function is switch from $\mathbb{H}$ to the programmable random oracle PRO.

Experiment Hyb_2**:** Same as Hyb_1 except that *str* is sampled uniformly at random and the random oracle is programmed using the trapdoor td such that $\mathsf{PRO.H}(\mathsf{hk},\mathsf{ct}_{\mathsf{FHE}}||\mathsf{ct}_{\mathsf{EG}}||\mathsf{ct}_{\mathsf{FHE},1}||\ldots||\mathsf{ct}_{\mathsf{FHE},2\lambda}||\mathsf{ct}_{\mathsf{EG},1}||\ldots||\mathsf{ct}_{\mathsf{EG},2\lambda}):=$ *str*.

Experiment Hyb_3**:** This is same as $\mathsf{NIZKIdeal}^{\mathsf{Sim}}_{\mathsf{NIZK}_{\mathsf{eq}}}(1^\lambda,\mathsf{stmt})$. In other words, this is same as Hyb_2 except that for all $i\notin I$, $\mathsf{ct}_{\mathsf{FHE},i}$ and $\mathsf{ct}_{\mathsf{EG},i}$ are computed as in $\mathsf{NIZKIdeal}^{\mathsf{Sim}}_{\mathsf{NIZK}_{\mathsf{eq}}}(1^\lambda,\mathsf{stmt})$.

Analysis. Observe that Hyb_0 and Hyb_1 are identically distributed since we are in the random oracle model. Further, due to programmability of the random oracle, Hyb_1 and Hyb_2 are identically distributed . Lastly, due to the correctness and linear homomorphism of FHE and EG encryption schemes, Hyb_2 and Hyb_3 are identically distributed. By a hybrid argument, it follows that Hyb_0 and Hyb_3 are identically distributed. Thus, for the distinguisher $\mathcal{D}$, we have

$$\left|\Pr\left[\mathcal{D}(\mathsf{NIZKReal}_{\mathsf{NIZK}_{\mathsf{eq}}}(1^\lambda,\mathsf{stmt},\mathsf{wit}))=1\right]-\Pr\left[\mathcal{D}(\mathsf{NIZKIdeal}^{\mathsf{Sim}}_{\mathsf{NIZK}_{\mathsf{eq}}}(1^\lambda,\mathsf{stmt}))=1\right]\right|=0.$$

This completes the proof. □

3.17 FAS for Linear Functions (Revisited)

In this section, we describe how to make our FAS construction for general functions even more efficient when restricted to linear functions only. Our efficient instantiation follows the template in Figure 3.34 with two modifications as follows.

Replacing FHE with El Gamal Encryption. Since we are restricted to the linear functions setting, it sufficies to replace the FHE scheme with a linearly homomorphic encryption (LHE) scheme. For this, we choose the El Gamal Encryption scheme with messages encoded in the exponent. This is linearly homomorphic since for any two messages $m_0,m_1\in\mathbb{Z}_p$, note that $\mathsf{ct}_0=(g^{r_0},\mathsf{pk}^{r_0}\cdot g^{m_0})$ and $\mathsf{ct}_1=(g^{r_1},\mathsf{pk}^{r_1}\cdot g^{m_1})$. As a consequence $\mathsf{ct}=((g^{r_0})\cdot(g^{r_1}),(\mathsf{pk}^{r_0}\cdot g^{m_0})\cdot(\mathsf{pk}^{r_1}\cdot g^{m_1}))$ is an encryption of m_0+m_1 under the randomness

$r_0 + r_1$. Furthermore, for sake of simplicity of $\mathsf{NIZK}_{\mathsf{eq}}$ as discussed in the next paragraph, we use the prime-order cyclic group based on the secp256k1 elliptic-curve for this El Gamal Encryption scheme.

$\mathsf{NIZK}_{\mathsf{eq}}$: Replacing cut-and-choose style proof with Schnorr-style proof. Recall that $\mathsf{NIZK}_{\mathsf{eq}}$ is a proof of plaintext equality. In particular, the language L_{eq} is about statements containing $\mathsf{ct}_{\mathsf{FHE}}$ which is an FHE evaluated ciphertexted and $\mathsf{ct}_{\mathsf{EG}}$ which is an El Gamal ciphertext. Since FHE and El Gamal encryption schemes were based on different mathematical structure, lattices and prime-order cyclic groups respectively, we needed to rely on cut-and-choose style proof. But observe that now we have replaced FHE with El Gamal encryption scheme as well. As a consequence the language L_{eq} is even more simplified and only concerns a single prime-order cyclic group. This allows us to use Schnorr-style proofs for the language. The main upside here is that the soundness error after 1 repetition is already negligible and we do not need to do any repetitions. This helps make the NIZK prover and verifier even more efficient in both computation and communication. In particular, communication just involves 7 group elements of group $\mathbb{G}$ and 6 scalars in $\mathbb{Z}_p$.

Efficiency. We note that the overall resulting FAS construction is far more efficient than that of Vanjani et al [VST24a]. Specifically, the seller does not have to do any expensive computation. Further, the buyer has to perform only 2 multi-scalar multiplication (MSM) operations that are involved in the homomorphic evaluation. In contrast, [VST24a] required seller to perform 2 MSMs and buyer to perform 1 MSM and 1 discrete log computation. Our implementation numbers reflect the same and we refer to Section 3.13.2 for more details.

$\mathsf{LHE.REval}(\mathsf{pk}, \mathsf{sk}, \mathsf{ct}, \vec{x}, \vec{r}, f, \mathsf{ct}_f)$:

1 : Parse f as $\vec{\alpha} = (\alpha_0, \alpha_1, \ldots, \alpha_\ell)$
2 : $r_f = \langle \vec{r}, \vec{\alpha} \rangle$
3 : **ret** r_f

Figure 3.53: Description of REval for our LHE scheme.

Chapter 4

Non-Interactive Anonymous Routing

Anonymous communication has been studied since the 1980s. Classical protocols such as DC-Nets and Tor are interactive and rely on threshold assumptions about honest routers; if the assumption is broken, no anonymity is left. To address these limitations, Shi and Wu [SW21] proposed the non-interactive anonymous routing (NIAR) model, which treats all routers as potentially malicious. They showed how to build NIAR using functional encryption (FE) [SW21, SV23], however, it required $O(n^2)$ router computation and proved security in presence of static corruptions only.

In our work [FSS+23], we introduce the first NIAR scheme with quasi-linear router computation and proved security in presence of adaptive corruptions. We achieved this by using indistinguishability obfuscation (iO), a primitive related to FE. Our approach employs a network of small obfuscated programs, and we developed new authentication techniques to reason about its security. Thus, our work advances the foundations of NIAR.

As the lead author of this work, I was responsible for designing our NIAR construction using SSU signatures and proving its security, including developing new techniques to handle adaptive corruptions.

4.1 Introduction

Anonymous communication systems allow users to communicate without revealing their identities and messages. The earliest design of an anonymous communication system goes back to Chaum [Cha81] who proposed the design of an encrypted email service that additionally hides the identities of the sender and the receiver. Since then, numerous approaches have been proposed to build anonymous routing schemes [Cha81, Cha88, Abe99, GRS99, SBS02, DMS04, ZZZR05, CGF10, BG12, CBM15, vdHLZZ15, TGL+17] – a key component of anonymous communication systems. These include mix-nets [Cha81, Abe99, BG12], the Dining Cryptographers' nets [Cha88, CGF10, APY20], onion routing [GRS99, DMS04, CL05, DS18],

multi-party-computation-based approaches [HEK12, AKTZ17, SA19], multi-server PIR-write [OS97, GIKM00, CBM15], as well as variants [ZZZR05, vdHLZZ15, TGL+17].

However, all of these routing schemes are *interactive*, where many servers or routers engage in an interactive protocol to achieve routing. The security relies on *threshold* type assumptions, example, majority or at least one of the routers must be honest. This is unsatisfactory since the threshold-based trust model increases the barrier of adoption, the interactivity leads to higher network latency, and finally, the schemes provide no guarantees when *all* routers may be malicious, or worse yet, colluding with a subset of the receivers and senders.

The recent work of Shi and Wu [SW21] was the first to study the feasibility of *non-interactive* anonymous routing (NIAR) with a *single, untrusted* router which can additionally collude with a subset of senders and receivers. The setting is as follows: there are n senders and n receivers, and each sender u wants to talk to a unique receiver $v = \pi(u)$ given by the routing permutation π. The NIAR scheme has a trusted setup that given a routing permutation π outputs encryption keys for senders, decryption keys for receivers, and a routing token for the router that secretly encrypts the routing permutation. In each time step, each sender uses its encryption key to encrypt a message. The router upon collecting all the n ciphertexts applies the routing token to permute them and convert them into n transformed ciphertexts, and delivers to each receiver a single transformed ciphertext. Each receiver learns their message by decrypting the received ciphertext with their key. The computation of the permuted ciphertexts can be viewed as the router *obliviously* applying the routing permutation π, without learning π.

NIAR was shown to have numerous applications in [SW21] including realizing a non-interactive anonymous shuffle (NIAS) where n senders send encryptions of their private messages to an entity called *shuffler* who, upon decryption, learns a permutation of the senders' messages, without learning the mapping between each message and the corresponding sender. A NIAS scheme can be used to instantiate the *shuffle model* adopted in a line of work on distributed, differentially private mechanisms [CSU+19, BBGN19b, GPV19, EFM+19, BEM+17, BBGN19a]. We can realize such a NIAS construction from NIAR by having the shuffler act on behalf of the NIAR router and all n receivers, as long as the underlying NIAR scheme provides meaningful security even when all the receivers collude with the router – termed as *receiver-insider* security by Shi and Wu [SW21].

Shi and Wu [SW21] give a NIAR scheme that satisfies receiver-insider security assuming the hardness of the decisional linear problem. Their scheme not only supports an *unbounded* number of time steps, but also has good efficiency features: each sender only needs to send $O_\lambda(1)$ bits per time step to encrypt a bit,[1] moreover, the sender and receiver keys are $O_\lambda(1)$ and the public parameters are $O_\lambda(n)$ in size. However,

[1] Throughout the chapter, we use $O_\lambda(\cdot)$ to hide $\mathsf{poly}(\lambda)$ multiplicative factors where λ denotes the security parameter.

Shi and Wu's scheme suffers from two main drawbacks.

- First, their token size and router computation per time step are both *quadratic* in the number of users n, that is, $O_\lambda(n^2)$. We also stress that the quadratic router computation drawback pertains not only to the work of Shi and Wu [SW21]. As Gordon et al. [GKLX22] pointed out, even in classical, *interactive* anonymous routing constructions [Cha81, Cha88, HEK12, SA19], the total router computation is typically $\Omega(nm)$ where n and m denote the number of clients and routers, respectively — therefore, in a peer-to-peer environment where the clients also act as routers, the total computation would be quadratic in n.
- Shi and Wu's construction is proven secure only under *static corruption*, i.e., the adversary must specify all corrupt senders and receivers upfront. This leaves open an interesting question whether we can construct a NIAR scheme that is secure against adaptive corruptions, i.e., when the adversary can dynamically decide which players to corrupt.

The status quo gives rise to the following natural questions:

1. *Can we have a NIAR scheme with* subquadratic *router computation?*
2. *Can we have a NIAR scheme secure against* adaptive *corruptions?*

4.1.1 Main Result

In this chapter, we construct a new NIAR scheme that simultaneously answers both of the above questions affirmatively. In particular, our new NIAR construction achieves $\widetilde{O}_\lambda(n)$ router computation per time step where $\widetilde{O}_\lambda(\cdot)$ hides both $\mathsf{poly}(\lambda, \log n)$ factors; moreover, it achieves security in the presence of adaptive corruptions. In terms of assumptions, we need the existence of indistinguishability obfuscator (iO) [GGH+13a, JLS21, GP20, WW20, BDGM20] and one-way functions.

Theorem 4.1.1 (Informal: NIAR with subquadratic router computation)**.** *Let λ be a security parameter. Let $n = n(\lambda)$ be the number of senders/receivers. Then, assuming the existence of indistinguishability obfuscator and one-way functions, there exists a NIAR scheme (in the receiver insider protection setting) that satisfies security under adaptive corruptions. Further, the asymptotical performance bounds are as follows:*

1. *the token size and router computation per time step is $\widetilde{O}_\lambda(n)$;*
2. *the per-sender communication and encryption time per bit of the message is $\widetilde{O}_\lambda(1)$;*
3. *each sender key is of length $\widetilde{O}_\lambda(1)$, each receiver key is of length $O_\lambda(1)$.*

Technical highlights. The above result is obtained through a sequence of stepping stones.

- *Techniques for reasoning about a network of obfuscated programs.* First, we show how to achieve the desired efficiency, but under a relaxed notion of security, that is, assuming static corruptions and a selective, single-challenge setting. To achieve this, we use a gate-by-gate obfuscation approach. Specifically, we break up one big circuit into a network of smaller circuits to obfuscate, through the use of a quasilinear-sized routing network. In this network, each smaller circuit is of size polylogarithmic in the number of senders, thus helping us meet our efficiency goals even after obfuscating each of the smaller circuits. We also devise new techniques for reasoning about a network of obfuscated programs. Specifically, we propose a new notion of a Somewhere Statistically Unforgeable (SSU) signature which may be of independent interest, and we show how to construct SSU signatures from either iO + one-way functions, or from fully homomorphic encryption. The construction of SSU signatures is not part of this thesis and is deferred to the full version of the paper [FSS+23].

- *New techniques for upgrading from selective and static security to fully adaptive security.* Next, we want to remove the static corruption and selective-single-challenge restrictions. What is interesting is that the *standard complexity leveraging techniques completely fail* in our context due to our efficiency requirements. Therefore, we devise various new techniques for upgrading the security of the scheme, which eventually gets us the final result. An important consequence of our techniques is that we only incur a *polynomial security loss* when performing the upgrades. A key insight in our upgrade is to consider the following single-inversion restriction on the adversary: it must submit two permutations separated by a single inversion in the two worlds, i.e., the two permutations are almost identical except for swapping the destinations of a pair of senders. We prove that security w.r.t. a single inversion is in fact equivalent to security w.r.t two arbitrary permutations.

  Along the way, we also explore the relationship of different definitions of NIAR security, and get several additional results (see Section 4.1.2) which may be of independent interest.

4.1.2 Additional Results

Adaptively secure NIAR with $O(n^2)$ router computation from standard assumptions. Our techniques for upgrading from selective/static to adaptive security can be of independent interest. For example, we can apply the same upgrade techniques to the previous NIAR scheme by Shi and Wu [SW21], which gives an adaptively secure NIAR scheme with $O_\lambda(n^2)$ router computation from standard assumptions.

Corollary 4.1.2 (Informal: quadratic computation NIAR scheme assuming bilinear maps)**.** *Assume standard bilinear map assumptions. There exists a NIAR scheme (in the receiver insider protection setting) that satisfies security under adaptive corruptions, and the asymptotical performance bounds are as follows:*

1. *the token size and router computation per time step is* $O_\lambda(n^2)$*;*
2. *the per-sender communication and encryption time per bit of the message is* $\widetilde{O}_\lambda(1)$*;*
3. *each sender key is of length* $\widetilde{O}_\lambda(1)$*, each receiver key is of length* $O_\lambda(1)$*.*

4.1.3 Related Work and Open Questions

Techniques for reasoning about a network of obfuscated programs. To the best of our knowledge, the only other works that used the gate-by-gate obfuscation technique are Jain and Jin [JJ22] and Canetti et al. [CLTV15]. However, our techniques are fundamentally different in nature from the previous works. With Jain and Jin's techniques, the evaluator will need to spend $\mathsf{poly}(n)$ time to evaluate each obfuscated gate whereas for our construction, each obfuscated gate takes only $\mathsf{poly}(\lambda, \log n)$ time to evaluate, which is important for our efficiency claims. Our network of iOs idea also differs fundamentally from Canetti et al. [CLTV15], which builds leveled fully-homomorphic encryption scheme from iO. In our setting, there are *multiple* encrypters some of whom may be *corrupt*, whereas in the setting of Canetti et al. [CLTV15], there is a *single* encrypter who is assumed to be *honest* — so their setting is a lot easier.

Another line of works [CHJV14, KLW15, BGL+15, AJS17, JJ22] constructs indistinguishability obfuscation for Turing machines and RAM programs. A natural question is whether obfuscating the routing network modelled as a Turing machine or RAM program can result in the required sub-quadratic routing efficiency. Unfortunately, prior approaches [CHJV14, KLW15, BGL+15, AJS17, JJ22] suffer from evaluation time that is polynomial in the input length — in the case of the routing network, it would result in $\mathsf{poly}(n)$ runtime. Therefore, we cannot directly use existing iO for Turing machines or RAMs as a blackbox to achieve the desired efficiency. This is also another way to see why our results are non-trivial.

Additional related work. The recent work of Bünz, Hu, Matsuo and Shi [BHMS21] made an attempt at answering the question. They could not fully achieve the above goal, but did suggest a scheme with $O(\lambda^{\frac{1}{\gamma}} \cdot n^{1+\gamma})$ router computation for any $\gamma \in (0, 1)$. Their scheme has two significant drawbacks. First, their subquadratic router computation comes at the price of relaxing the security definition to (ϵ, δ)-*differential privacy* [DMNS06]. In other words, their scheme ensures that the adversary's views are indistinguishable only for two *neighboring* routing permutations (whereas full security requires indistinguishability for any two routing permutations). Not only is differential privacy a significantly weaker security notion, it can

also lead to additional complications in terms of managing the privacy budget. Second, their $\mathsf{poly}(\lambda)$ dependency is not a fixed one — to improve the dependence on the parameter n, we want to choose an arbitrarily small γ, however, this would significantly blow up the polynomial dependence on the security parameter λ.

Comparison with concurrent work. We stress that in this chapter, we focus on constructing a NIAR scheme whose security is sufficient for instantiating a non-interactive anonymous shuffler. As mentioned earlier, the shuffler application is important in the context of distributed differentially private mechanisms in the so-called "shuffle model". For this application to work, we need the NIAR scheme to satisfy a notion of security called *receiver-insider protection*, that is, corrupt receivers (possibly colluding with the router and some corrupt senders) should not learn which honest senders have sent the message.

In comparison, the elegant concurrent work by Bunn, Kushilevitz, and Ostrovsky [BKO22] solves the *dual* problem as ours. Their syntax is the same as ours, but their security guarantees are for the *sender insider protection* setting, and are not sufficient for instantiating the shuffler application. In particular, in the sender insider protection setting, corrupt senders (possibly colluding with the router and some corrupt receivers) should not learn which honest receivers are receiving their messages.

From a technical standpoint, *sender insider protection* is akin to Private Information Retrieval (PIR) [CGKS95, CG97]. In fact, if we allow quadratic router computation, a NIAR scheme with a sender insider protection is implied by PIR which is known from standard, polynomial-strength assumptions. By contrast, PIR does not directly lead to NIAR with *receiver*-insider security (even if router computation efficiency is a non-concern). In fact, NIAR with receiver insider protection is technically akin to *multi-client functional encryption (MCFE) with function-hiding security*. Technically, a NIAR scheme with receiver-insider protection implies a function-hiding MCFE for the selection functionality with the bounded, upfront key queries[2]. So far, the only known way to achieve receiver-insider security (i.e., the work by Shi and Wu [SW21]) also uses function-hiding, multi-client functional encryption as a building block. For this reason, receiver insider protection is technically more challenging based on the existing knowledge and techniques.

Bunn et al. [BKO22] did not discuss the issue of adaptive corruption in the context of their primitive. Interestingly, our work's static-to-adaptive compiler can also be applied to their sender insider protection setting.

Finally, from a technical perspective, Bunn et al.'s main idea is to use a rate-1 PIR scheme where they reuse the clients' PIR queries at the router over multiple sessions. To cut the router computation to

[2]In this sense, our adaptive corruption result is also interesting in the context of function-hiding MCFE since how to get function-hiding MCFE under adaptive corruption was not known earlier. The recent work of Shi and Vanjani [SV23] showed a function-hiding MCFE scheme for inner-product computation under static corruption, relying on standard bilinear group assumptions.

quasi-linear, they also rely an oblivious routing network. In their paper, they construct and analyze a new oblivious routing network for this purpose. Alternatively, they can also directly use the same oblivious routing network that we use in this chapter, which is directly borrowed from the earlier algorithms literature [ACN+20, RS21].

Open questions. Our feasibility results naturally raise several open questions for future work. Can we achieve subquadratic router computation from standard assumptions without using indistinguishability obfuscation? Can we construct a scheme with good concrete performance? Can we strengthen the security of the scheme to get full insider protection (as defined by [SW21]) from standard assumptions?

4.2 Technical Roadmap

To get the above result, we had to go through multiple intermediate steps, where we first construct schemes with relaxed security notions and then gradually lift them to full security. In this process, we develop several interesting new techniques and building blocks that may be of independent interest. At a very high level, our blueprint and techniques are summarized below:

4.2.1 Single Selective Challenge and Static Corruptions

Our first step is to construct a NIAR scheme with quasilinear router computation, but we relax the definitions and only require security when the adversary has to upfront commit to 1) all corrupt senders and receivers and 2) a single challenge time step along with the corresponding challenge plaintext vectors. In this step, we encounter multiple challenges.

Definitional challenge for single, selective security. First, it turns out that even defining a meaningful selective notion of security (in the static corruption setting) is non-trivial, because it is unclear how the non-challenge rounds should behave. This definition should satisfy two goals: first, the non-challenge rounds should contain no information about the permutation. This is because our techniques below crucially rely on this. Second, the definition should generalize naturally to full security. We discuss these issues in more detail in Section 4.7.

Gate-by-gate obfuscation for efficiency. Next, we consider how to get a NIAR scheme with quasilinear router computation under the relaxed security. To start with, it is helpful to imagine an inefficient scheme where the router's token is an obfuscated circuit that encodes the entire permutation π as well as encryption and decryption keys. Now, upon receiving the n incoming ciphertexts, the obfuscated circuit decrypts

the incoming ciphertexts, applies the permutation π, and reencrypts the outcomes using each receiver's respective key. The intuition is that if we treat the obfuscation as a black box which completely hides its internals, then it should hide everything about π and the honest parties' plaintexts beyond what the corrupted parties are allowed to decrypt.

There are two problems with this approach. First, the seminal work of [BGI+01] showed that it is impossible to achieve an "virtual black-box" (VBB) obfuscation scheme that perfectly hides its internals. Second, even forgetting about the security analysis, all known obfuscation schemes have large polynomial blowup in the input size; there is no obfuscation scheme that even comes close to a quadratic blowup, let alone subquadratic. This clearly does not meet our efficiency requirement.

Our idea to solve this efficiency problem is to break up one big circuit into a network of smaller circuits to obfuscate, through the use of a quasilinear-sized routing network. In this routing network, each gate has only polylogarithmically sized inputs and outputs, and there are $O(n)$ such gates. Now, if we obfuscate each gate separately and create a network of obfuscated gates, then the total size of all obfuscated gates would be quasilinear.

It turns out that this idea would only work if the underlying routing network has a special "obliviousness" property, that is, a corrupt sender cannot infer from its own route the destinations of honest senders (see Definition 4.4.1). Fortunately, we were able to get such a routing network using known techniques from the oblivious sorting literature (although the notion of "obliviousness" there is of a different nature).

New techniques for reasoning about a network of obfuscated programs. We now turn to the challenges involved in reasoning about the security of "networked obfuscated programs". To solve these challenges, we develop techniques which we believe have potential to be useful in future applications. In particular, when the output of one obfuscated gate is fed into another, we want to ensure that the adversary does not tamper with the output in between. To achieve this, we would like to have each obfuscated gate authenticate its own outputs, and have the next obfuscated gate verify the authentication information before proceeding to the computation.

As mentioned before, it is well-known that VBB obfuscation is impossible, and it is only possible to achieve a much more restrictive notion called indistinguishability obfuscation (iO). iO achieves a much weaker notion of security: it only guarantees that obfuscations of two *functionally equivalent* programs are indistinguishable. As is evident from prior works, computationally secure primitives are generally incompatible with the functional equivalence requirements of iO. Therefore, we need to develop new iO-compatible techniques for authentication.

A new notion of somewhere-statistically-unforgeable signatures. To this end, one of our contributions is to introduce a new building block called a *Somewhere Statistically Unforgeable (SSU)* signature scheme. Informally, in an SSU signature scheme, there are three modes to sample the signing and verification keys: *normal mode*, *punctured mode*, and the *binding mode*.

- Normal mode: the signing and verification keys behave same as in standard digital signatures.
- Punctured mode: the signing key is *punctured* w.r.t. a set of points X but the verification key is normal. Intuitively, this means that *no* valid signature can be computed using the signing algorithm for points outside of the set X when using the *punctured* signing key.
- Binding mode: here, both the signing and verification keys are *binding* w.r.t. a sets of points X. Intuitively, this means that, with overwhelming probability, *no* valid signatures exist for points outside of the set X w.r.t. a randomly sampled *binding* verification key. In other words, this means that statistical unforgeability holds *somewhere* (points outside the set X) in the binding mode.

SSU signatures can be used to sign/verify tuples of the form (t, m), where t denotes a round and m denotes a message. For a fixed round t^* and message m^*, we specifically focus on a set X_{t^*,m^*} which contains pairs (t, m) as follows:

- For all $t \neq t^*$, $(t, m) \in X_{t^*,m^*}$ for all $m \in \{0,1\}^{\mathsf{len}}$.
- For $t = t^*$, there is a single pair $(t^*, m^*) \in X_{t^*,m^*}$, and for all $m' \neq m^*$, $(t^*, m') \notin X_{t^*,m^*}$.

Intuitively, we can use this restriction to restrict the behavior of the network of circuits during the challenge round t^*. Note that both the round t^* and the message m^* must be fixed when generating the keys of the signature scheme, hence (among other reasons) why the techniques here achieve a selective notion of security for the NIAR scheme.

For our network of iO proof to go through, we need the following important property from the SSU signature. We require these distributions to be computationally indistinguishable:

$$\begin{pmatrix} \textit{punctured signing key,} \\ \textit{normal verification key} \end{pmatrix} \equiv_c \begin{pmatrix} \textit{binding signing key,} \\ \textit{binding verification key} \end{pmatrix}$$

This property is critical when we perform a layer-by-layer hybrid argument in our proofs.

We stress that for technical reasons explained below, this property is important for our "networked obfuscated programs" techniques to work. This property also differentiates our SSU signature scheme from previous known puncturable signature schemes [HIJ+17, BSW16, GWZ22]. The main difference from

previous puncturable signature schemes is that we need the two verification keys to be indistinguishable even in the presence of some signing key, whereas the previous schemes required that the two verification keys be indistinguishable in absence of any signing key.

Theorem 4.2.1 (Informal: SSU signatures [FSS+23])**.** *Assuming the existence of one-way functions and indistinguishability obfuscation, or assuming leveled fully homomorphic encryption, there exists a somewhere statistically unforgeable signature scheme for the family of sets X_{t^*,m^*} defined above.*

Network of iOs: proof highlight. We sketch our proof outline, focusing on the part that makes use of the aforementioned property of our SSU signature. In our construction, a ciphertext encrypts the message as well as the route it should be sent along. Imprecisely speaking then, each gate does the following: decrypts the incoming ciphertexts and verifies the input message signature (to authenticate the message) and route signature (to authenticate the route); and if valid, it performs the routing, encrypts the output messages (along with the routing information), and uses an output signing key to sign them. Our proof goes through a sequence of hybrids sketched below[3].

- First, starting from the real-world experiment, through a sequence of hybrids, we hard-wire the route signatures on corrupt wires (which are shared across all time steps) — we defer the details of these hybrids to the subsequent technical sections so we can focus on the part of proof that uses the aforementioned property of our SSU signatures.
- Next, through a layer-by-layer hybrid sequence, we want to switch to a world in which for challenge time step t^*, honest and filler wires' ciphertexts and signatures (for both messages and routes) are hard-wired in the obfuscated gate and the obfuscated gate only accepts an incoming ciphertext that matches the hiredwired one. Except for the honest-to-corrupt wires in the last layers, which are hard-wired encryptions of the actual messages to be received by the corrupt receiver, for all other honest/filler wires, the hired-wired ciphertexts are encryptions of fillers. In this new world, the challenge ciphertexts for honest senders are also random encryptions of fillers; therefore, for the challenge time step t^*, the adversary's view contains no information about honest-to-honest and honest-to-corrupt routes, as well as honest-to-honest messages.

  As described below, this layer-by-layer hybrid is where we critically need the aforementioned security property from the SSU signatures.

[3]Our formal proof in the technical sections actually first proves single, selective-challenge, static security only for an adversary subject to the following restrictions: it must corrupt all receivers, and submit two permutations that differ by a single inversion. We prove that even this weaker version is sufficient for our upgrade all the way to full security under adaptive corruption.

- Assuming that layer i's input verification key is already switched to binding, we can then switch layer i's output signing key to a punctured signing key by using iO security (since the binding verification key already ensures that the punctured messages will never pass through);
- Next, we make the following replacement by relying on the security of the SSU signature scheme:

$$\begin{aligned} & (\text{punctured signing key: layer } i, \quad \text{normal verification key: layer } i+1) \\ \Longrightarrow \quad & (\text{binding signing key: layer } i, \quad \text{binding verification key: layer } i+1) \end{aligned}$$

- At this moment, by relying on iO security, we can hard-wire the ciphertexts and signatures for t^* on honest/filler wires, such that the obfuscated gate only accepts the input on the wire if it matches the hard-wired value.

4.2.2 Removing the Selective Challenge Restriction

Recall that the techniques above are able to achieve a limited notion of security, which we call "selective single-challenge" security. The selective notion requires that the adversary submit not just two permutations $\pi^{(0)}$ and $\pi^{(1)}$ upfront, but additionally commit to both a challenge round t^* and the set $\{x^{(0)}_{u,t^*}, x^{(1)}_{u,t^*}\}_{u\in\mathcal{H}_S}$ of challenge plaintexts to be used during round t^* for the honest senders $\mathcal{H}_S$, two for each honest sender. Recall that we use the SSU signature scheme above, and we puncture the signing and verification keys at round t^* and at the target plaintexts, which we hardcode in the obfuscated gates during the inner hybrids. This is essentially why we need this data upfront.

Standard complexity leveraging fails. The standard tool to achieve such a transformation is complexity leveraging. Namely, to run the adaptive-query single-challenge with the selective scheme, we guess the values t^* and $\{x^{(0)}_{u,t^*}, x^{(1)}_{u,t^*}\}_{u\in\mathcal{H}_S}$ at the beginning of the experiment. This incurs a security loss proportional to 2^α, where α is the number of bits needed to represent t^* and $\{x^{(0)}_{u,t^*}, x^{(1)}_{u,t^*}\}_{u\in\mathcal{H}_S}$. We stress that due to our efficiency requirements, complexity leveraging fails completely even if we are willing to accept the (sub-)exponential loss in the security reduction. More specifically, α can be as large as $O(n)$, which means that the selective-secure NIAR scheme would have to be $2^{O(n)}$-secure for the reduction to be meaningful. Thus we must adopt a security parameter greater than n in all the underlying primitives, including the iO scheme, resulting in $\mathsf{poly}(n)$ cost and thus defeating our efficiency goals. This problem seems inherent with our techniques, because as explained above, we hardcode information about each $x^{(b)}_{u,t^*}$ for all honest u in the obfuscated gates.

Removing the selective challenge restriction through equivalence to single-inversion security. To remove the selective challenge restriction, our key insight is to define a single-inversion notion of NIAR

security, and using single-inversion security as a stepping stone. In normal NIAR security (under static corruption), we want security to hold for two arbitrary admissible permutations. In single-inversion security, we consider two admissible permutations where only a pair of honest senders' destinations are swapped.

If we can prove equivalence to single-inversion security, then we can do the selective-query to adaptive-query upgrade for single-inversion security. In this case, a standard complexity leveraging argument has only polynomial security loss as explained below. Specifically, with single-inversion security, the reduction only needs to guess the challenge time step t^* and the challenge plaintexts of the two swapped honest users in the two worlds — without loss of generality, we can assume that each sender's plaintext is a single bit, since we can always get a multi-bit scheme by parallel composition of multiple single-bit schemes. Further, we assume that the reduction is given an upper bound on the p.p.t. adversary's running time. Therefore, the space of the guesses is polynomially bounded.

The remaining technicality is proving equivalence to single-inversion security. At first sight, it might be tempting to conclude that this is obvious, since given $\pi_{(0)}$ and $\pi_{(1)}$, we can always swap a pair of honest senders at a time to eventually transform $\pi_{(0)}$ to $\pi_{(1)}$. However, correctly implementing this idea is subtle. Specifically, we need any pair of adjacent hybrids to be not trivially distinguishable by the adversary, where the adversary is subject to the admissibility rules of the beginning and the end hybrids. We prove that given any beginning and end hybrids, we can indeed construct a sequence of intermediate hybrids, each time swapping a pair of honest senders' destinations and their messages, such that each pair of adjacent hybrids satisfy the aforementioned constraint.

4.2.3 Achieving Security for Adaptive Corruptions

So far, we have constructed a NIAR scheme that achieves security under static corruptions but adaptive queries. The last question remaining is how to upgrade the scheme to get security even under adaptive corruptions.

A first idea that comes to mind is to again attempt complexity leveraging. Again, unfortunately, due to our efficiency requirements, complexity leveraging completely does not work even if we are willing to suffer from (sub-)exponential losses in the reduction. Suppose that the reduction guesses which set of players the adversary will corrupt. Since there are 2^n possible guesses, for the parameters to work in the complexity leveraging, we must adopt a security parameter that is greater than n in the underlying iO scheme, which results in at least $\mathsf{poly}(n)$ blowup.

A new compiler for upgrading to adaptive corruptions. Instead of complexity leveraging, we construct a new compiler that compiles a NIAR scheme secure under static corruption to a NIAR scheme secure under adaptive corruption, with only polynomial loss in the security reduction.

The compiler is very simple: each sender will first encrypt their plaintext using a PRF that is secure against selective opening (which is implied by standard PRF security as shown by Abraham et al. [ACD+19]), before encrypting it using the NIAR scheme that is secure under static corruption. For proving that this construction secure against adaptive corruptions, we will go through the following key steps:

- First, suppose we want to prove single-inversion security when all the receivers are corrupt. Now, when the reduction receives the two permutations $\pi_{(0)}$ and $\pi_{(1)}$, it may assume that only the inverted pair of senders are honest. Therefore, in this case, security under adaptive corruption is the same as security under static corruption.
- Next, still assuming that all receivers are corrupt, we want to prove security under adaptive corruption for any two arbitrary permutations. For this step, we need to prove the equivalence of security under two arbitrary permutations and single-inversion security, but now for the scenario when the senders can be adaptively corrupted. The technicalities in this proof are similar to the counterpart for the static corruption case; however, the argument becomes somewhat more involved now that the senders can be adaptively corrupt.
- Finally, we show how to remove the assumption where the receivers must be all corrupted upfront, and allow the adversary to adaptively corrupt the receivers. This step will rely on the selective-opening security of the PRF which is implied by standard PRF security [ACD+19].

Organization of rest of the chapter. In Section 4.3, we define NIAR, and in Section 4.4, we present preliminaries. In Section 4.5, we define SSU signatures. In Section 4.6, we construct a NIAR scheme secure against a static and all-receiver-corrupting adversary and present the security proof in Sections 4.7 and 4.8. In Section 4.9, we present a compiler that transforms the above NIAR scheme to one with full security, i.e., removing the static and all-receiver-corrupting restrictions on the adversary.

4.3 Definitions for NIAR

In this section, we define the syntax and security for NIAR, focusing on the strongest definition of full security against adaptive corruptions.

4.3.1 Syntax

We begin with the syntax. A Non-Interactive Anonymous Router (NIAR) is a cryptographic scheme consisting of the following, possibly randomized algorithms:

- $(\{\mathsf{ek}_u\}_{u\in[n]}, \{\mathsf{rk}_u\}_{u\in[n]}, \mathsf{tk}) \leftarrow \mathsf{Setup}(1^\lambda, \mathsf{len}, n, \pi)$: the trusted Setup algorithm takes the security parameter 1^λ, the length of the messages len, the number of senders/receivers n, and a permuation π. The algorithm outputs a sender key for each sender denoted $\{\mathsf{ek}_u\}_{u\in[n]}$, a receiver key for each receiver denoted $\{\mathsf{rk}_u\}_{u\in[n]}$, and a token for the router denoted tk.

- $\mathsf{ct}_{u,t} \leftarrow \mathsf{Enc}(\mathsf{ek}_u, x_{u,t}, t)$: sender u uses its sender key ek_u to encrypt the message $x_{u,t} \in \{0,1\}^{\mathsf{len}}$ where t denotes the current time step. The Enc algorithm produces a ciphertext $\mathsf{ct}_{u,t}$.

- $(\mathsf{ct}'_{1,t}, \mathsf{ct}'_{2,t}, \ldots, \mathsf{ct}'_{n,t}) \leftarrow \mathbf{Rte}(\mathsf{tk}, \mathsf{ct}_{1,t}, \mathsf{ct}_{2,t}, \ldots, \mathsf{ct}_{n,t})$: the routing algorithm **Rte** takes pk and its token tk (which encodes some permutation π), and n ciphertexts received from the n senders denoted $\mathsf{ct}_{1,t}, \mathsf{ct}_{2,t}, \ldots, \mathsf{ct}_{n,t}$, and produces *transformed ciphertexts* $\mathsf{ct}'_{1,t}, \mathsf{ct}'_{2,t}, \ldots, \mathsf{ct}'_{n,t}$ where $\mathsf{ct}'_{u,t}$ is destined for the receiver $u \in [n]$.

- $x \leftarrow \mathsf{Dec}(\mathsf{rk}_u, \mathsf{ct}'_{u,t}, t)$: the decryption algorithm Dec takes a receiver key rk_u, a transformed ciphertext $\mathsf{ct}'_{u,t}$, a time step t, and outputs a message x.

Correctness of NIAR. Correctness requires that with probability 1, the following holds for any $\lambda, \mathsf{len} \in \mathbb{N}$, any $(x_1, x_2, \ldots, x_n) \in \{0,1\}^{\mathsf{len}\cdot n}$, and any t: let $(\{\mathsf{ek}_u\}_{u\in[n]}, \{\mathsf{rk}_u\}_{u\in[n]}, \mathsf{tk}) \leftarrow \mathsf{Setup}(1^\lambda, \mathsf{len}, n, \pi)$, let $\mathsf{ct}_{u,t} \leftarrow \mathsf{Enc}(\mathsf{ek}_u, x_u, t)$ for $u \in [n]$, let $(\mathsf{ct}'_{1,t}, \mathsf{ct}'_{2,t}, \ldots, \mathsf{ct}'_{n,t}) \leftarrow \mathbf{Rte}(\mathsf{tk}, \mathsf{ct}_{1,t}, \mathsf{ct}_{2,t}, \ldots, \mathsf{ct}_{n,t})$, and let $x'_u \leftarrow \mathsf{Dec}(\mathsf{rk}_u, \mathsf{ct}'_{u,t}, t)$ for $u \in [n]$; it must be that $x'_{\pi(u)} = x_u$ for every $u \in [n]$.

4.3.2 NIAR Security under Static Corruption

We review the full security notion under static corruption, defined first by Shi and Wu [SW21]. Henceforth, we use the notation $\mathcal{K}_R$ and $\mathcal{K}_S$ to denote the set of corrupt receivers and senders, respectively; we use $\mathcal{H}_S$ and $\mathcal{H}_R$ to denote the set of honest senders and honest receivers, respectively. Security is defined with respect to the following experiment, parametrized by a bit $b \in \{0,1\}$, and a ***stateful*** adversary $\mathcal{A}$.

Static corruption experiment $\mathsf{NIARStatic}^{b,\mathcal{A}}(1^\lambda)$**.**

1. $(n, \mathsf{len}, \mathcal{K}_S, \mathcal{K}_R, \pi^{(0)}, \pi^{(1)}) \leftarrow \mathcal{A}(1^\lambda)$.

2. $(\{\mathsf{ek}_u\}_{u\in[n]}, \{\mathsf{rk}_u\}_{u\in[n]}, \mathsf{tk}) \leftarrow \mathsf{Setup}(1^\lambda, \mathsf{len}, n, \pi^{(b)})$.

3. $\perp \leftarrow \mathcal{A}(\mathsf{tk}, \{\mathsf{ek}_u\}_{u\in\mathcal{K}_S}, \{\mathsf{rk}_u\}_{u\in\mathcal{K}_R})$.

4. For $t = 1, 2, \ldots$:

   - $\{x^{(0)}_{u,t}, x^{(1)}_{u,t}\}_{u\in\mathcal{H}_S} \leftarrow \mathcal{A}(\perp)$.
   - for $u \in \mathcal{H}_S$, $\mathsf{ct}_{u,t} \leftarrow \mathsf{Enc}(\mathsf{ek}_u, x^{(b)}_{u,t}, t)$
   - $\perp \leftarrow \mathcal{A}(\{\mathsf{CT}_{u,t}\}_{u\in\mathcal{H}_S})$.

5. At any time, $\mathcal{A}$ may halt and output an arbitrary function of its view. The experiment then also halts and returns the output of $\mathcal{A}$.

Admissibility. We say that $\mathcal{A}$ is *admissible* iff with probability 1, it guarantees that

1. For all $u \in \mathcal{K}_S$, $\pi^{(0)}(u) = \pi^{(1)}(u)$; and
2. For all rounds t, and for any $u \in \mathcal{K}_R \cap \pi^{(0)}(\mathcal{H}_S) = \mathcal{K}_R \cap \pi^{(1)}(\mathcal{H}_S)$, $x^{(0)}_{v_0,t} = x^{(1)}_{v_1,t}$ where for $b \in \{0,1\}$, $v_b := (\pi^{(b)})^{-1}(u)$. In other words, here we require that in the two alternate worlds $b = 0$ or 1, every corrupt receiver receiving from an honest sender must receive the same message.

Definition 4.3.1 (Security against static corruptions)**.** *We say that a NIAR scheme satisfies security against static corruption, iff for any non-uniform p.p.t. admissible $\mathcal{A}$, its views in the two experiments $\mathsf{NIARStatic}^{0,\mathcal{A}}(1^\lambda)$ and $\mathsf{NIARStatic}^{1,\mathcal{A}}(1^\lambda)$ are computationally indistinguishable.*

4.3.3 NIAR Full Security

In this section, we present a security notion for NIAR against a very strong adversary. In particular, we allow such an adversary to (a) adaptively corrupt the set of senders and receivers, and (b) adaptively ask for encryptions of chosen plaintext under the senders' keys that are not yet corrupted. Our security definition is a strict generalization of the "receiver-insider corruption" notion introduced by Shi and Wu [SW21] which captured only static corruptions of users.

We formalize our definition via the experiment $\mathsf{NIARFull}^{b,\mathcal{A}}$ which is parametrized by some challenge bit b and a ***stateful*** non-uniform p.p.t. adversary $\mathcal{A}$. At the beginning of the experiment, adversary $\mathcal{A}$ submits two challenge permutations $\pi^{(0)}$ and $\pi^{(1)}$ over $[n]$ for its choice of n. At any time in the experiment, the adversary can choose to corrupt a sender or receiver, and it will receive the secret key for the newly corrupted player. The adversary receives tk, and then in each time step, it can submit two plaintext vectors $\{x^{(0)}_{u,t}, x^{(1)}_{u,t}\}_{u\in\mathcal{H}_S}$ for the set of currently honest senders $\mathcal{H}_S$. The challenger will encrypt the plaintexts indexed by $b \in \{0,1\}$, and at the end of the experiment, the adversary's job is to distinguish which world b

the challenger is in. The adversary must be subject to a set of admissibility rules such that it cannot trivially distinguish which world it is in.

More formally, our full NIAR security game is defined as follows, where $\mathsf{Cor}(\cdot)$ is the following oracle: upon receiving a sender or receiver identity,

- return its corresponding secret key;
- in case the newly corrupted player is a sender, additionally return all the historical random coins consumed by the Enc algorithm during the previous time steps;
- update the honest sender set $\mathcal{H}_S$ and honest receiver set $\mathcal{H}_R$ accordingly.

NIAR full security experiment $\mathsf{NIARFull}^{b,\mathcal{A}}(1^\lambda)$**.**

1. $(n, \mathsf{len}, \pi^{(0)}, \pi^{(1)}) \leftarrow \mathcal{A}(1^\lambda)$.
2. $\mathcal{H}_S = [n]$, $\mathcal{H}_R = [n]$.
3. $(\{\mathsf{ek}_u\}_{u\in[n]}, \{\mathsf{rk}_u\}_{u\in[n]}, \mathsf{tk}) \leftarrow \mathsf{Setup}(1^\lambda, \mathsf{len}, n, \pi^{(b)})$.
4. For $t = 1, 2, \ldots$:
   - if $t = 1$: $\{x_{u,t}^{(0)}, x_{u,t}^{(1)}\}_{u\in\mathcal{H}_S} \leftarrow \mathcal{A}^{\mathsf{Cor}(\cdot)}(\mathsf{tk})$;
     else $\{x_{u,t}^{(0)}, x_{u,t}^{(1)}\}_{u\in\mathcal{H}_S} \leftarrow \mathcal{A}^{\mathsf{Cor}(\cdot)}(\{\mathsf{CT}_{u,t-1}\}_{u\in\mathcal{H}_S})$.
   - for all $u \in \mathcal{H}_S$, $\mathsf{CT}_{u,t} \leftarrow \mathsf{Enc}(\mathsf{ek}_u, x_{u,t}^{(b)}, t)$.
5. At any time, $\mathcal{A}$ may halt and output an arbitrary function of its view. The experiment then also halts and returns the output of $\mathcal{A}$.

In the above definition, if the adversary wants to specify an initially corrupt set, it can simply make calls to the corruption oracle $\mathsf{Cor}(\cdot)$ at the beginning of $t = 1$. Therefore, without loss of generality, we may assume that the initially corrupt set before the challenger calls **Setup** is empty.

Admissibility. We state some admissibility rules on the adversary to make sure that the adversary cannot trivially distinguish whether it is in world $b = 0$ or $b = 1$. Our admissibility rule corresponds to the "receiver-insider protection" version of Shi and Wu [SW21], which is sufficient for building a non-interactive anonymous shuffler. Basically, we assume that senders know their receivers but receivers do not know their senders. Therefore, if the adversary corrupts some senders, the adversary will know the corrupt senders' receivers. We remark that Shi and Wu [SW21] additionally described a "full insider protection" notion

where it is assumed that neither senders nor receivers know who they are paired with. Their "full insider protection" construction requires polynomial in n evaluation time and uses indistinguishable obfuscation and bilinear group assumptions [SW21]. It remains an open question how to reduce the evaluation time for the "full insider protection" version.

Henceforth, if a player remains honest at the end of the execution, we say that it is *eventually honest*; otherwise we say that it is *eventually corrupt*. We say that $\mathcal{A}$ is *admissible* iff with probability 1, the following holds where $\mathcal{H}_S$ and $\mathcal{H}_R$ refer to the eventually honest sender and receiver set, and define $\mathcal{K}_R = [n] \setminus \mathcal{H}_R, \mathcal{K}_S = [n] \setminus \mathcal{H}_S$ to be the eventually corrupt sender and receiver sets:

1. For all eventually corrupt senders $u \in \mathcal{K}_S$, $\pi^{(0)}(u) = \pi^{(1)}(u)$.
2. For any eventually corrupt sender $u \in \mathcal{K}_S$, for any t in which u was not corrupt yet, $x^{(0)}_{u,t} = x^{(1)}_{u,t}$. In other words, here we require that in the two alternate worlds $b = 0$ or $b = 1$, every eventually corrupt sender should be sending the same message in all rounds before it was corrupted.
3. For all rounds t, and for any $v \in \mathcal{K}_R \cap \pi^{(0)}(\mathcal{H}_S) = \mathcal{K}_R \cap \pi^{(1)}(\mathcal{H}_S)$, $x^{(0)}_{u_0,t} = x^{(1)}_{u_1,t}$ where for $b \in \{0,1\}$, $u_b := (\pi^{(b)})^{-1}(v)$. In other words, here we require that in the two alternate worlds $b = 0$ or 1, every eventually corrupt receiver receiving from an eventually honest sender must receive the same message in all rounds.

Definition 4.3.2 (NIAR full security)**.** *We say that a NIAR scheme is fully secure iff for any non-uniform p.p.t.* admissible $\mathcal{A}$*, its views in the two experiments* $\mathsf{NIARFull}^{0,\mathcal{A}}(1^\lambda)$ *and* $\mathsf{NIARFull}^{1,\mathcal{A}}(1^\lambda)$ *are computationally indistinguishable.*

4.4 Preliminaries

Whenever we refer to an adversary in the chapter henceforth, we implicitly mean it to be a *non-uniform* adversary.

4.4.1 Notations

We say that a function $\mathsf{negl} : \mathbb{N} \to \mathbb{R}$ is negligible, if for every constant $c > 0$ and for all sufficiently large $\lambda \in \mathbb{N}$ we have $\mathsf{negl}(\lambda) < \lambda^{-c}$. Two distribution ensembles $\{X^\lambda_0\}_\lambda$ and $\{X^\lambda_1\}_\lambda$ are computationally indistinguishable if for every p.p.t. adversary $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$, $|\Pr[x \leftarrow X^\lambda_0 : \mathcal{A}(x) = 0] - \Pr[x \leftarrow X^\lambda_1 : \mathcal{A}(x) = 0]| \leq \mathsf{negl}(\lambda)$. We use '_' to denote that a value is irrelevant. For instance, in $(a, _, c)$ the second value is irrelevant and can be anything. Often

times, we use a short hand $\{y_i : i \in [n]\}$ to denote an ordered sequence $(y_1, \ldots, y_n)$. For instance, $\{y_i : i \in [n]\} \leftarrow f(t, \{x_i : i \in [n]\})$ means $(y_1, \ldots, y_n) \leftarrow f(t, x_1, \ldots, x_n)$.

4.4.2 Routing Networks

Imagine we have a directed acyclic graph henceforth called a *routing network* with $2n$ sources and $2n$ destinations. Suppose we have n *producers*, each of whom assigned to a distinct source vertex. We also have n *consumers*, each of whom assigned to a distinct destination vertex. Suppose that producer $u \in [n]$ is assigned source vertex $2u - 1 \in [2n]$. Suppose that consumer $u \in [n]$ is assigned destination vertex $2u - 1 \in [2n]$. Now, each producer wants to route one product to a distinct consumer, and the desired mapping between the producers and consumers is called the routing permutation π. Then, producer u is mapped to consumer $\pi(u)$. In the routing network, this translates to routing producer u's product from source vertex $2u - 1$ to destination vertex $2\pi(u) - 1$. To avoid congestion, we want all n routes to be over *edge-disjoint* paths. Earlier works [ACN+20, RS21] have constructed such a routing network with the following properties.

- *Congestion-free routing.* There exists a randomized algorithm $\mathsf{AssignRoutes}(1^\lambda, n, \pi)$ that takes in the security parameter λ and the routing permutation π, and with $1 - \mathsf{negl}(\lambda)$ probability, outputs the following information for each producer $u \in [n]$: the path that producer u traverses to reach its consumer which is assigned to some destination vertex. Henceforth, the above information is called the *route* for producer u, often denoted rte_u. As mentioned, all producers' routes are edge-disjoint. We allow the $\mathsf{AssignRoutes}(1^\lambda, n, \pi)$ algorithm to have a negligibly small failure probability in which case it outputs $\perp$.

- *Layered construction.* The network is layered. We may imagine that the source and destination vertices form two special layers numbered 0 and L, respectively, and all other intermediate-layer vertices are henceforth called *gates*. Directed edges, henceforth called *wires*, exist only between adjacent layers ℓ and $\ell + 1$.

- *Efficiency.* Each gate has $W = O(\log^2 \lambda)$ incoming wires and W outgoing wires. The network has $O(\log n)$ layers, and each intermediate layer has $G = 2n/W$ gates. Each gate has $O(\log^2 \lambda)$ incoming wires and $O(\log^2 \lambda)$ outgoing wires. The number of wires between any two adjacent layers is exactly $2n$.

- *Obliviousness.* The network and the corresponding $\mathsf{AssignRoutes}(1^\lambda, n, \pi)$ algorithm satisfies a privacy property. Informally speaking, imagine that a subset of the producers are corrupt, and they can learn their routes to their respective destinations (including which source nodes the corrupt producers are

assigned to). We want that the choice of the corrupt producers' routes are independent of the honest producers' destinations. We will formally define this privacy property below.

Definition 4.4.1 (Obliviousness of a routing network)**.** *We say that a routing network satisfies obliviousness, iff there exists another simulated* $\mathsf{AssignRoutes}^*$ *algorithm and a negligible function* $\mathsf{negl}(\cdot)$*, such that for any two routing permutations* π_0 *and* π_1 *on* $[n]$*, let* $C(\pi_0, \pi_1)$ *be the set of senders that have the same destinations in* π_0 *and* π_1*, it holds that for either* $b \in \{0,1\}$*, the following two distributions have statistical distance at most* $\mathsf{negl}(\lambda)$*:*

- *Sample* $(\mathsf{rte}_1, \ldots, \mathsf{rte}_n) \leftarrow \mathsf{AssignRoutes}(1^\lambda, n, \pi_b)$*, and output* $(\{\mathsf{rte}_u\}_{u \in C(\pi_0,\pi_1)}, \{\mathsf{rte}_u\}_{u \in [n] \setminus C(\pi_0,\pi_1)})$*;*
- *Sample* $\left(\{\mathsf{rte}_u\}_{u \in C(\pi_0,\pi_1)}, \{\mathsf{rte}_u^\beta\}_{u \in [n] \setminus C(\pi_0,\pi_1), \beta \in \{0,1\}}\right) \leftarrow \mathsf{AssignRoutes}^*(1^\lambda, n, \pi_0, \pi_1)$*, and output* $(\{\mathsf{rte}_u\}_{u \in C(\pi_0,\pi_1)}, \{\mathsf{rte}_u^b\}_{u \in [n] \setminus C(\pi_0,\pi_1)})$.

The definition says that the simulated $\mathsf{AssignRoutes}^*$ algorithm takes in two routing permutations π_0 and π_1. For a sender $u \in C(\pi_0, \pi_1)$ with the same destinations in π_0 and π_1, $\mathsf{AssignRoutes}^*$ outputs a single route rte_u for u. For a sender $u \notin C(\pi_0, \pi_1)$ with different destinations in π_0 and π_1, $\mathsf{AssignRoutes}^*$ outputs two routes rte_u^0 and rte_u^1 for u. Not only so, for either $b \in \{0,1\}$, the union of the routes for senders' in $C(\pi_0, \pi_1)$, and the b-th set of routes $\{\mathsf{rte}_u^b\}_{u \notin C(\pi_0,\pi_1)}$ for everyone else output by the simulated $\mathsf{AssignRoutes}^*$ must be statistically indistinguishable from running the real-world $\mathsf{AssignRoutes}$ using permutation π_b.

Intuitively, in our NIAR scheme, only those who are in $C(\pi_0, \pi_1)$ can possibly be corrupt. Therefore, the definition decomposes the generation of the possibly-corrupt sender' routes from the remaining honest senders' destinations. In this sense, the possibly-corrupt senders' routes do not leak information about honest senders' destinations (beyond what is already leaked by the possibly-corrupt senders' destinations, and barring a negligibly small statistical difference).

Remark 4.4.2. *Our definition of obliviousness is not the same as the "data obliviousness" notion of Asharov et al. [ACN+20] and Ramachandran and Shi [RS21] — their notion requires that the access patterns of the routing algorithm not depend on the input data when executed on a RAM. On the other hand, our notion is closely related to a line of work called "oblivious routing" from the standard algorithms literature [RÖ2, HKLR05], where roughly speaking, we want that a player's route be independent of others' destinations.*

Interestingly, it turns out that we can obtain a routing network that satisfies Definition 4.4.1 using techniques from Asharov et al. [ACN+20] and Ramachandran and Shi [RS21]. Asharov et al. [ACN+20] and Ramachandran and Shi [RS21] propose a butterfly network where each gate has polylogarithmically many incoming and outgoing wires. We can compose two instances of their butterfly network back-to-back. The first instance is to route each input element (i.e., those on input wires $2u - 1$ *for* $u \in [n]$*) to a random and distinct wire in the output layer. The second instance will then route each element* u *to its correct destination, i.e.,* $2\pi(u) - 1$ *of the output layer.*

In our routing network, there are more wires in each layer than the number of producers or consumers. Therefore, some wires do not carry load. Henceforth in this chapter, we also call such wires that do not carry actual load *filler* wires.

4.4.3 Puncturable PRF

A puncturable pseudorandom function consists of the following algorithms:

- $\mathsf{sk} \leftarrow \mathsf{Gen}(1^\lambda, 1^\ell)$: takes in a security parameter 1^λ, the length 1^ℓ of the input messages where $\ell := \ell(\lambda)$ is a polynomial function in λ, and outputs a PRF key K.
- $\sigma \leftarrow \mathbf{Eval}(K, x)$: a *deterministic* function that takes in a PRF key K, a message $x \in \{0,1\}^\ell$, and outputs the evaluation outcome σ.
- $K^* \leftarrow \mathbf{Puncture}(K, S = \{x_1^*, \ldots, x_{|S|}^*\})$: takes in a PRF key K, the set of messages $S = \{x_1^*, \ldots, x_{|S|}^*\}$ that the PRF key needs to be punctured on, and outputs a punctured key K^*.
- $\sigma \leftarrow \mathbf{PEval}(K^*, x)$: a *deterministic* function that takes in a punctured PRF key K^*, and a message $x \in \{0,1\}^{\mathsf{len}}$, outputs the evaluation outcome σ.

Correctness. We say that a puncturable PRF scheme satisfies correctness if the punctured key preserves functionality when evaluated at unpunctured points. Formally, we require that for any λ, ℓ, any $S = \{x_1^*, \ldots, x_{|S|}^*\}$, any $x \in \{0,1\}^{\mathsf{len}}$ such that $x \notin S$,

$$\Pr\left[\begin{array}{l} K \leftarrow \mathsf{Gen}(1^\lambda, \ell), \\ K^* \leftarrow \mathbf{Puncture}(K, S) \end{array} : \mathbf{Eval}(K, x) = \mathbf{PEval}(K^*, x)\right] = 1$$

Security. We say that a puncturable PRF scheme is secure, if given a punctured key, the original PRF's evaluation outcomes at punctured points (i.e., points that the punctured key cannot evaluate) remain pseudorandom. Formally, consider the following experiment $\mathsf{ExptPPRF}^{\mathcal{A},b}(1^\lambda, 1^\ell)$ parametrized by a bit $b \in \{0,1\}$:

- The stateful adversary $\mathcal{A}$ sends a set S to the challenger. The challenger computes a PRF key $K \leftarrow \mathsf{Gen}(1^\lambda, 1^\ell)$. The challenger then computes the punctured key $K^* \leftarrow \mathbf{Puncture}(K, S)$ and sends it back to the adversary.
- The adversary $\mathcal{A}$ can adaptively make evaluation queries on messages $x_i \notin S$ to the challenger. For each such query, the challenger responds as follows. If $b = 0$, it sends $\mathbf{Eval}(K, x_i)$ to the adversary. If $b = 1$, it sends a uniformly random string to the adversary.

- $\mathcal{A}$ outputs a guess $b' \in \{0,1\}$, the experiment outputs b'.

Definition 4.4.3 (Puncturable PRF security)**.** *We say that a puncturable PRF scheme is secure iff for any ℓ polynomially bounded by λ, for any non-uniform p.p.t. adversary $\mathcal{A}$, its views in the two experiments* $\mathsf{ExptPPRF}^{\mathcal{A},b}(1^\lambda, 1^\ell)$ *and* $\mathsf{ExptPPRF}^{\mathcal{A},b}(1^\lambda, 1^\ell)$ *are computationally indistinguishable.*

Theorem 4.4.4 ([GGM86, BW13, BGI14, KPTZ13])**.** *If one-way functions exist, then for all efficiently computable $\ell(\lambda)$, there exists a secure puncturable PRF.*

Shorthand notations. Sometimes for we use $\mathsf{PRF}(K, \cdot)$ as a shorthand for $\mathsf{PRF}.\mathbf{Eval}(K, \cdot)$ and $\mathsf{PRF}(K^*, \cdot)$ as a shorthand for $\mathsf{PRF}.\mathbf{PEval}(K^*, \cdot)$.

4.4.4 Indistinguishability Obfuscation

The notion of indistinguishability obfuscation (iO) was first defined in [BGI+01]. Recent works have shown constructions from well-founded assumptions [JLS21, GP20, WW20, BDGM20]. We give the formal definition below, taken almost verbatim from Jain et al. [JLS21].

Definition 4.4.5 (Indistinguishability Obfuscator (iO))**.** *A uniform p.p.t. algorithm* iO *is called an indistinguishability obfuscator for polynomial-sized circuits if the following holds:*

- ***Completeness:*** *For every $\lambda \in \mathbb{N}$, every circuit C with input length n, every input $x \in \{0,1\}^n$, we have that*

$$\Pr\left[C'(x) = C(x) : C' \leftarrow \mathsf{iO}(1^\lambda, C)\right] = 1.$$

- ***Polynomial Security:*** *For any two ensembles $\{C_{0,\lambda}\}_\lambda$, $\{C_{1,\lambda}\}_\lambda$ of polynomial-sized circuits that have the same size, input length, and output length, and are functionally equivalent (i.e., $C_{0,\lambda}(x) = C_{1,\lambda}(x)$ for every λ and x), the distributions $\{\mathsf{iO}(1^\lambda, C_{0,\lambda})\}_\lambda$ and $\{\mathsf{iO}(1^\lambda, C_{1,\lambda})\}_\lambda$ are computationally indistinguishable.*

4.4.5 Selective Opening Security for PRFs

In our adaptive corruption scheme, we need a PRF that is secure against selective opening attacks. The definition of such a PRF was given in the work of Abraham et al. [ACD+19]. Moreover, they showed that the standard PRF security notion implies selective opening security except with a polynomial loss in the security failure probability.

Specifically, selective opening security is defined as follows, borrowing verbatim from Abraham et al. [ACD+19].

We consider a selective opening adversary that interacts with a challenger. The adversary can request to create new PRF instances, query existing instances with specified messages, selectively corrupt instances and obtain the secret keys of these instances, and finally, we would like to claim that for instances that have not been corrupt, the adversary is unable to distinguish the PRFs' evaluation outcomes on any future message from random values from an appropriate domain. More formally, we consider the following game between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$.

$\underline{\mathsf{PRFExpt}_b^{\mathcal{A}}(1^\lambda)}$: $\mathcal{A}(1^\lambda)$ can adaptively interact with $\mathcal{C}$ through the following queries:

- *Create instance.* The challenger $\mathcal{C}$ creates a new PRF instance by calling the honest PRF key generation algorithm $\mathsf{Gen}(1^\lambda)$. Henceforth, the instance will be assigned an index that corresponds to the number of "create instance" queries made so far. The i-th instance's secret key will be denoted sk_i.
- *Evaluate.* The adversary $\mathcal{A}$ specifies an index i that corresponds to an instance already created and a message m, and the challenger computes $r \leftarrow \mathsf{PRF}_{\mathsf{sk}_i}(m)$ and returns r to $\mathcal{A}$.
- *Corrupt.* The adversary $\mathcal{A}$ specifies an index i, and the challenger $\mathcal{C}$ returns sk_i to $\mathcal{A}$ (if the i-th instance has been created).
- *Challenge.* The adversary $\mathcal{A}$ specifies an index i^* that must have been created and a message m. If $b = 0$, the challenger returns a completely random string of appropriate length. If $b = 1$, the challenger computes $r \leftarrow \mathsf{PRF}_{\mathsf{sk}_{i^*}}(m)$ and returns r to $\mathcal{A}$.

We say that $\mathcal{A}$ is admissible iff with probability 1, every challenge tuple (i^*, m) it submits satisfies the following: 1) $\mathcal{A}$ does not make a corruption query on i^* throughout the game; and 2) $\mathcal{A}$ does not make any evaluation query on the tuple (i^*, m).

Definition 4.4.6 (Selective opening security of a PRF family)**.** *We say that a PRF scheme satisfies pseudorandomness under selective opening iff for any admissible non-uniform p.p.t. adversary $\mathcal{A}$, its views in $\mathsf{PRFExpt}_0^{\mathcal{A}}(1^\lambda)$ and $\mathsf{PRFExpt}_1^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable.*

4.5 Somewhere Statistically Unforgeable (SSU) Signatures

In this section, we define SSU signatures and provide the theorem statement for constructing it.

4.5.1 Definition

We consider an SSU signature scheme where the signing and verification algorithms both take a counter t (i.e., time step) in addition to the message x to be signed. We refer to t as the *round*. Specifically, an SSU signature scheme contains the following algorithms:

- $(\mathsf{sk}, \mathsf{vk}, \mathsf{pp}) \leftarrow \mathsf{Setup}(1^\lambda, \mathsf{tlen}, \mathsf{len})$: takes as input the security parameter 1^λ, the length of the round $\mathsf{tlen} \geq 0$, the length of the messages to be signed $\mathsf{len} > 0$, and outputs a signing key sk, a verification key vk, and a public parameter pp.

- $\sigma \leftarrow \mathsf{Sign}(\mathsf{pp}, \mathsf{sk}, t, x)$: a *deterministic* algorithm that takes as input a public paramter pp, a singing key sk, along with a round $t \in \{0,1\}^{\mathsf{tlen}}$ ($t = \perp$ in case $\mathsf{tlen} = 0$) and a message $x \in \{0,1\}^{\mathsf{len}}$ and outputs a signature σ for x w.r.t. t.

- $(0 \text{ or } 1) \leftarrow \mathsf{Vf}(\mathsf{pp}, \mathsf{vk}, t, x, \sigma)$: takes as input a public paramter pp, a verification key vk, a round t, a message x, and a signature σ, and outputs either 1 for accept or 0 for reject.

- $(\mathsf{sk}, \widetilde{\mathsf{sk}}, \mathsf{vk}, \mathsf{pp}) \leftarrow \textbf{PuncturedSetup}(1^\lambda, \mathsf{tlen}, \mathsf{len}, t^*, x^*)$: takes as input the security parameter 1^λ, the length of the round tlen, the length of the messages to be signed len, a round $t^* \in \{0,1\}^{\mathsf{tlen}}$ ($t^* = \perp$ in case $\mathsf{tlen} = 0$) and a message $x^* \in \{0,1\}^{\mathsf{len}}$, and outputs a signing key sk, a punctured signing key $\widetilde{\mathsf{sk}}$, a verification key vk, and a public paramter pp.

- $(\mathsf{sk}^*, \mathsf{vk}^*, \mathsf{pp}^*) \leftarrow \textbf{BindingSetup}(1^\lambda, \mathsf{tlen}, \mathsf{len}, t^*, x^*)$: takes as input the security parameter 1^λ, the length of the round tlen, the length of the messages to be signed len, a round $t^* \in \{0,1\}^{\mathsf{tlen}}$ ($t^* = \perp$ in case $\mathsf{tlen} = 0$) and message $x^* \in \{0,1\}^{\mathsf{len}}$, and outputs a binding signing key sk^*, a binding verification key vk^*, and a binding public paramter pp^*.

- $\sigma \leftarrow \mathsf{PSign}(\mathsf{pp}, \widetilde{\mathsf{sk}}, t, x)$: a *deterministic* algorithm that takes as input a public paramter pp, a punctured signing key $\widetilde{\mathsf{sk}}$ generated by **PuncturedSetup**, a round t and a message x, and outputs a signature σ for x w.r.t. t.

Correctness of SSU signature. An SSU signature is said to be correct iff the following holds,

- For all $\lambda, \mathsf{len}, \mathsf{tlen} \in \mathbb{N}$, $t \in \{0,1\}^{\mathsf{tlen}}$, $x \in \{0,1\}^{\mathsf{len}}$,

$$\Pr\left[\begin{array}{r} (\mathsf{sk}, \mathsf{vk}, \mathsf{pp}) \leftarrow \mathsf{Setup}(1^\lambda, \mathsf{tlen}, \mathsf{len}) \\ \sigma \leftarrow \mathsf{Sign}(\mathsf{pp}, \mathsf{sk}, t, x) \end{array} : \mathsf{Vf}(\mathsf{pp}, \mathsf{vk}, t, x, \sigma) = 1\right] = 1.$$

- For all $\lambda, \mathsf{len}, \mathsf{tlen} \in \mathbb{N}$, $t^*, t \in \{0,1\}^{\mathsf{tlen}}$, $x^*, x \in \{0,1\}^{\mathsf{len}}$ such that it is *not* the case that $t = t^*$ and $x \neq x^*$,

$$\Pr\left[\begin{array}{r}(\mathsf{sk}, \widetilde{\mathsf{sk}}, \mathsf{vk}, \mathsf{pp}) \leftarrow \mathbf{PuncturedSetup}(1^\lambda, \mathsf{tlen}, \mathsf{len}, t^*, x^*) : \\ \mathsf{Sign}(\mathsf{pp}, \mathsf{sk}, t, x) = \mathsf{PSign}(\mathsf{pp}, \widetilde{\mathsf{sk}}, t, x)\end{array}\right] = 1.$$

Definition 4.5.1 (Security for SSU Signatures)**.** *An SSU signature is said to be secure if it has the following properties:*

- **Identical distribution of normal keys output by** Setup **and PuncturedSetup.** *For any λ,* $\mathsf{len}, \mathsf{tlen} \in \mathbb{N}$*, any $t^* \in \{0,1\}^{\mathsf{tlen}}$, any $x^* \in \{0,1\}^{\mathsf{len}}$, we have the following where $\equiv$ denotes identical distribution:*

$$\begin{aligned}&\{(\mathsf{sk}, \mathsf{vk}, \mathsf{pp}) \leftarrow \mathsf{Setup}(1^\lambda, \mathsf{tlen}, \mathsf{len}) : \textit{ output } (\mathsf{sk}, \mathsf{vk}, \mathsf{pp})\} \\ \equiv&\{(\mathsf{sk}, \widetilde{\mathsf{sk}}, \mathsf{vk}, \mathsf{pp}) \leftarrow \mathbf{PuncturedSetup}(1^\lambda, \mathsf{tlen}, \mathsf{len}, t^*, x^*) : \textit{ output } (\mathsf{sk}, \mathsf{vk}, \mathsf{pp})\}\end{aligned}$$

- **Indistinguishability of punctured and binding setups.** *For any* len *and* tlen *that are polynomially bounded by λ, any $t^* \in \{0,1\}^{\mathsf{tlen}}$, any $x^* \in \{0,1\}^{\mathsf{len}}$, we have the following where $\approx$ denotes computational indistinguishability:*

$$\begin{aligned}&\{(\mathsf{sk}, \widetilde{\mathsf{sk}}, \mathsf{vk}, \mathsf{pp}) \leftarrow \mathbf{PuncturedSetup}(1^\lambda, \mathsf{tlen}, \mathsf{len}, t^*, x^*) : \textit{ output } (\widetilde{\mathsf{sk}}, \mathsf{vk}, \mathsf{pp})\} \\ \approx&\{(\mathsf{sk}^*, \mathsf{vk}^*, \mathsf{pp}^*) \leftarrow \mathbf{BindingSetup}(1^\lambda, \mathsf{tlen}, \mathsf{len}, t^*, x^*) : \textit{ output } (\mathsf{sk}^*, \mathsf{vk}^*, \mathsf{pp}^*)\}\end{aligned}$$

- ***Statistical unforgeability at*** (t^*, x^*)**.** *For any* $\mathsf{len}, \mathsf{tlen}$ *that are polynomially bounded in λ, there exists a negligible function* $\mathsf{negl}(\cdot)$*, such that for any $t^* \in \{0,1\}^{\mathsf{tlen}}$, $x^* \in \{0,1\}^{\mathsf{len}}$, for any λ,*

$$\Pr\left[\begin{array}{r}(\mathsf{sk}^*, \mathsf{vk}^*, \mathsf{pp}^*) \leftarrow \mathbf{BindingSetup}(1^\lambda, \mathsf{tlen}, \mathsf{len}, t^*, x^*) : \\ \exists\, (\sigma, x) \textit{ s.t. } x \neq x^* \ \wedge\ \mathsf{Vf}(\mathsf{pp}^*, \mathsf{vk}^*, t^*, x, \sigma) = 1\end{array}\right] \leq \mathsf{negl}(\lambda),$$

$$\Pr\left[\begin{array}{r}(\mathsf{sk}^*, \mathsf{vk}^*, \mathsf{pp}^*) \leftarrow \mathbf{BindingSetup}(1^\lambda, \mathsf{tlen}, \mathsf{len}, t^*, x^*) : \\ \exists\, \sigma \neq \mathsf{PSign}(\mathsf{pp}^*, \mathsf{sk}^*, t^*, x^*) \textit{ s.t. } \mathsf{Vf}(\mathsf{pp}^*, \mathsf{vk}^*, t^*, x^*, \sigma) = 1\end{array}\right] \leq \mathsf{negl}(\lambda).$$

Theorem 4.5.2 (Corollary C.2 of [FSS+23])**.** *Assuming the existence of one-way functions and indistinguishability obfuscation, or assuming leveled fully homomorphic encryption, there exists a somewhere statistically unforgeable signature scheme.*

4.6 NIAR for a Static and All-Receiver-Corrupting Adversary

In this section, we first introduce a basic NIAR scheme which we prove secure under an adversary who is restricted to make all corruption queries upfront, and moreover, it must always corrupt all receivers — we

call such an adversary a static, all-receiver-corrupting adversary. This is same as the adversary in the full security game in Definition 4.3.2 except with the aforementioned restrictions. For sake of completeness, we define this version of security in Definition 4.3.1.

Later in Section 4.9, we give a compiler that transforms the scheme in this section to one with full security, i.e., removing the static and all-receiver-corrupting restrictions on the adversary.

Notation. To describe our construction more formally, it will be helpful to introduce some notation for the routing network. Recall that a routing network for n senders and n receivers is a layered directed acyclic graph that has $O(\log n)$ layers numbered from $0, 1, \ldots, L$. Each sender $u \in [n]$ is assigned to the $(2u-1)$-th wire in the input layer (i.e., layer-0), and each receiver $v \in [n]$ is assigned to the $(2v-1)$-th wire in the output layer (i.e., layer-L). Let G be the number of gates contained in each of the $L-1$ intermediate layers. There are $(L-1) \cdot G$ gates overall, and we refer to the g-th gate in the ℓ-th layer by the tuple $(\ell, g) \in [L-1] \times [G]$. Let $W = O(\log^2 \lambda)$ be the number of incoming and outgoing wires in each gate. Overall, there are $L \times [2n]$ wires where we index the i-th wire in the ℓ-th layer by the tuple $(\ell, i) \in [L] \times [2n]$.[4] We refer to the W incoming wires of every gate (ℓ, g) by the set $\mathsf{Input}_{(\ell,g)} \subseteq [2n]$ and the W outgoing wires by the set $\mathsf{Output}_{(\ell,g)} \subseteq [2n]$. In other words, the wires coming into gate (ℓ, g) are the set $\{(\ell, w)\}_{w \in \mathsf{Input}_{(\ell,g)}}$, and the wires outgoing from gate (ℓ, g) are the set $\{(\ell+1, w)\}_{w \in \mathsf{Output}_{(\ell,g)}}$. Finally, recall that a route rte_u from sender u to receiver v is a sequence of wires $(j_1, \ldots, j_L)$ where j_ℓ is a wire in the ℓ-th layer for all $\ell \in [L]$. Based on the description of routing network, also recall that $j_1 = 2u - 1 \in [2n]$ and $j_L = 2v - 1 \in [2n]$.

4.6.1 Construction

Simplifying assumption. Throughout this section, we shall assume that the message length $\mathsf{len} = 1$. This assumption is without loss of generality, since we can always parallel-compose multiple NIAR schemes where $\mathsf{len} = 1$ to get a NIAR scheme for $\mathsf{len} > 1$.

We now describe our basic NIAR scheme in detail.

Keys associated with wires. In our construction, each wire (ℓ, i) in the routing network will have the following associated with it:

- A PRF key $k_{(\ell,i)}$, which will be used to encrypt and decrypt the (signed) message along with its routing information on the wire;

[4] To be more precise there are $c \cdot n$ wires in each layer for constant $c \geq 2$, but for simplicity we assume $c = 2$ as this is achieved by our proposed instantiation.

- A message signing key tuple $(\mathsf{mpp}_{(\ell,i)}, \mathsf{msk}_{(\ell,i)}, \mathsf{mvk}_{(\ell,i)})$, which will later be used by the corresponding sender or obfuscated gate to sign the message to be sent to the wire;
- A route signing key tuple $(\mathsf{rpp}_{(\ell,i)}, \mathsf{rsk}_{(\ell,i)}, \mathsf{rvk}_{(\ell,i)})$, which will be used by the Setup algorithm to sign the routes and by the obfuscated gates to verify the routes before performing the routing.

Hardcoded values. $\mathsf{Gate}_{(\ell,g)}$ has hardcoded the following values:

- For each wire $i \in \mathsf{Input}_{(\ell,g)}$ in layer ℓ: $k_{(\ell,i)}$, $\mathsf{mpp}_{(\ell,i)}$, $\mathsf{mvk}_{(\ell,i)}$, $\mathsf{rpp}_{(\ell,i)}$, $\mathsf{rvk}_{(\ell,i)}$.
- For each wire $i \in \mathsf{Output}_{(\ell,g)}$ in layer $\ell+1$: $k_{(\ell+1,i)}$, $\mathsf{mpp}_{(\ell+1,i)}$, $\mathsf{msk}_{(\ell+1,i)}$.

Procedure. $\mathsf{Gate}_{(\ell,g)}$ takes as input a round t and a set of ciphertexts $\{\mathsf{CT}_{(\ell,i)} : i \in \mathsf{Input}_{(\ell,g)}\}$ corresponding to the input wires. It computes as follows.

1. For each input wire $i \in \mathsf{Input}_{(\ell,g)}$:
   a) If $\ell = 1$ and i is even, continue to next i. // *Filler, ignored.*
   b) **Decrypt and authenticate the message/route:**
      i. Compute $y = \mathsf{CT}_{(\ell,i)} \oplus \mathsf{PRF}(k_{(\ell,i)}, t)$ and parse y as $(x, \overline{\mathsf{rte}}, \mathsf{msig})$.
      ii. Abort if $\mathsf{Sig.Vf}(\mathsf{mpp}_{(\ell,i)}, \mathsf{mvk}_{(\ell,i)}, t, (x, \overline{\mathsf{rte}}), \mathsf{msig}) = 0$.
      iii. If $x = \perp_{\mathsf{filler}}$ and $\overline{\mathsf{rte}} = \perp_{\mathsf{filler}}$, continue to the next i. // *Filler, ignored.*
      iv. Parse $\overline{\mathsf{rte}}$ as $(\mathsf{rte}, \mathsf{rsig})$, rte as $(j_1, \ldots, j_L)$, and rsig as $(\mathsf{rsig}_1, \ldots, \mathsf{rsig}_L)$. Abort if $j_\ell \neq i$ or the next hop $j_{\ell+1} \notin \mathsf{Output}_{(\ell,g)}$ or $\mathsf{Sig.Vf}(\mathsf{rpp}_{(\ell,i)}, \mathsf{rvk}_{(\ell,i)}, 1, \mathsf{rte}, \mathsf{rsig}_\ell) = 0$.
   c) **Prepare the output ciphertext** $\mathsf{CT}_{(\ell+1,j_{\ell+1})}$**:**
      i. For convenience, set $j = j_{\ell+1}$.
      ii. If $\mathsf{CT}_{(\ell+1,j)}$ has already been computed, then abort.
      iii. Else if $\ell + 1 < L$ (intermediate layer), first compute a new message signature $\mathsf{msig}' = \mathsf{Sig.Sign}(\mathsf{mpp}_{(\ell+1,j)}, \mathsf{msk}_{(\ell+1,j)}, t, (x, \overline{\mathsf{rte}}))$. Then, compute the ciphertext $\mathsf{CT}_{(\ell+1,j)} = (x, \overline{\mathsf{rte}}, \mathsf{msig}') \oplus \mathsf{PRF}(k_{(\ell+1,j)}, t)$.
      iv. Else if $\ell + 1 = L$ (output layer), compute the ciphertext $\mathsf{CT}_{(L,j)} = x \oplus \mathsf{PRF}(k_{(L,j)}, t)$.
2. For each $j \in \mathsf{Output}_{(\ell,g)}$ such that $\mathsf{CT}_{(\ell+1,j)}$ has not been computed yet, compute filler ciphertexts:
   a) Set $x = \perp_{\mathsf{filler}}$ and $\overline{\mathsf{rte}} = \perp_{\mathsf{filler}}$.
   b) Compute $\mathsf{msig}' = \mathsf{Sig.Sign}(\mathsf{mpp}_{(\ell+1,j)}, \mathsf{msk}_{(\ell+1,j)}, t, (x, \overline{\mathsf{rte}}))$.
   c) Compute $\mathsf{CT}_{(\ell+1,j)} = (x, \overline{\mathsf{rte}}, \mathsf{msig}') \oplus \mathsf{PRF}(k_{(\ell+1,j)}, t)$.
3. Output $\{\mathsf{CT}_{(\ell+1,i)} : i \in \mathsf{Output}_{(\ell,g)}\}$.

Figure 4.1: The circuit $\mathsf{Gate}_{(\ell,g)}$.

Circuit for each gate. We first describe the circuit for each gate to be obfuscated later in our construction. The circuit $\mathsf{Gate}_{(\ell,g)}$ denotes the g-th gate in the ℓ-th layer. It receives a ciphertext on each input wire and

decrypts it using a PRF key to obtain a tuple $(x, \overline{\mathsf{rte}}, \mathsf{msig})$, where x is a message, $\overline{\mathsf{rte}}$ is the authenticated route, and msig is a message signature. It verifies the message signature msig on the tuple $(x, \overline{\mathsf{rte}})$. Next, it performs route authentication and prepares the output ciphertext which varies depending on whether the wire is filler or not. A wire is filler if $x = \perp_{\mathsf{filler}}$ and $\overline{\mathsf{rte}} = \perp_{\mathsf{filler}}$. For a filler input wire, no route authentication is performed as there is no route associated with it. Computing output ciphertext for filler output wires is deferred to later as the circuit does not know which are filler output wires at the moment. For a non-filler input wire i, the circuit parses $\overline{\mathsf{rte}} = (\mathsf{rte}, \mathsf{rsig})$ and verifies that the route rte is valid using rsig. Then, it parses $\mathsf{rte} = (j_1, \ldots, j_L)$. If $j_\ell = i$, then it finds the corresponding non-filler output wire $j_{\ell+1}$ from the rte and computes a new message signature msig' and then a new ciphertext for the output wire in the natural manner. At the end, all output wires which do not have any ciphertext assigned to them are interpreted as filler wires and the circuit computes message signature and ciphertext similarly by setting $x = \perp_{\mathsf{filler}}$ and $\overline{\mathsf{rte}} = \perp_{\mathsf{filler}}$. In Figure 4.1 we describe the circuit formally and in more detail, where Sig is a SSU signature scheme defined in Section 4.5 and PRF is a puncturable PRF.

We next describe the Setup algorithm.

Setup Algorithm. Given a routing permutation π, the Setup algorithm first sets $\mathsf{tlen} = \log^2(\lambda)$. Then, it runs the AssignRoutes algorithm to sample a set of edge-disjoint routes $\{\mathsf{rte}_u\}_{u\in[n]}$ between each sender/receiver pair. Then, for every wire $(\ell, i) \in [L] \times [2n]$ in the routing network we sample (a) PRF key $k_{(\ell,i)}$ for encryption, (b) a signature key pair $(\mathsf{rsk}_{(\ell,i)}, \mathsf{rvk}_{(\ell,i)}, \mathsf{rpp}_{(\ell,i)})$ for signing routes, and (c) a signature key pair $(\mathsf{msk}_{(\ell,i)}, \mathsf{mvk}_{(\ell,i)}, \mathsf{mpp}_{(\ell,i)})$ for signing messages. Looking ahead, when proving security, the route signature keys for wires assigned to corrupt senders' routes, and the message signature keys for all other wires will be punctured to ensure "uniqueness of routes and plaintexts".

Given the above set of keys, consider a sender/receiver pair (u, v) with route $\mathsf{rte}_u = (j_1, \ldots, j_L)$. Then sender u's sender key ek_u and receiver v's decryption key rk_v are defined as follows, where rsig_ℓ is the signature on rte_u computed using the route public param $\mathsf{rpp}_{(\ell,j_\ell)}$ and route signing key $\mathsf{rsk}_{(\ell,j_\ell)}$.

$$\mathsf{ek}_u = \left(k_{(1,j_1)}, \mathsf{mpp}_{(1,j_1)}, \mathsf{msk}_{(1,j_1)}, \overline{\mathsf{rte}}_u = (\mathsf{rte}_u, \mathsf{rsig}_u = (\mathsf{rsig}_1, \ldots, \mathsf{rsig}_L))\right) \ , \mathsf{rk}_v = k_{(L,j_L)} \ .$$

Lastly, the routing token tk is then defined as follows, where the circuit $\mathsf{Gate}_{(\ell,g)}$ is as described in Figure 4.1.

$$\mathsf{tk} = \{\mathsf{iO}(\mathsf{Gate}_{(\ell,g)}) : (\ell, g) \times [L-1] \times [G]\} \ .$$

More formally, the Setup algorithm is as in Figure 4.2.

$\mathsf{Setup}(1^\lambda, \mathsf{len} = 1, n, \pi)$: on inputs the security parameter 1^λ, the individual message length $\mathsf{len} = 1$, the number of parties n, and the permutation π, Setup does the following:

1. Set $\mathsf{tlen} = \log^2(\lambda)$.
2. **Sampling Routes**: Run the AssignRoutes procedure (Section 4.4.2) on inputs $(1^\lambda, n, \pi)$. Abort if it outputs $\perp$. Else parse the output as a set of edge-disjoint routes $\{\mathsf{rte}_u\}_{u\in[n]}$ between each sender/receiver pair. Let $0, \ldots, L$ be the layers in the resulting network. Let G be the number of gates contained in each of the $L-1$ intermediate layers. Let W be the number of incoming and outgoing wires in each gate. Then, for all $u \in [n]$, the size of the string rte_u is $\mathsf{len}_{\mathsf{rte}} = L \cdot \log(2n)$.
3. **Sampling Wire Keys**: For each wire (ℓ, i) in $[L] \times [2n]$:
   a) Sample PRF key $k_{(\ell,i)} \leftarrow \mathsf{PRF.Gen}(1^\lambda)$ as the encryption key for this wire.
   b) To sign and verify routes of length $\mathsf{len}_{\mathsf{rte}}$, sample route signature keys
   $$(\mathsf{rsk}_{(\ell,i)}, \mathsf{rvk}_{(\ell,i)}, \mathsf{rpp}_{(\ell,i)}) \leftarrow \mathsf{Sig.Setup}(1^\lambda, 0, \mathsf{len}_{\mathsf{rte}}) .$$
   Suppose that the resulting route signatures will be of size $\mathsf{poly}_{\mathsf{rsig}}(\lambda)$ for some polynomials $\mathsf{poly}_{\mathsf{rsig}}$. Then, the messages signed will be of length $\mathsf{len}_m = \mathsf{tlen} + 1 + L \cdot \log(2n) + L \cdot \mathsf{poly}_{\mathsf{rsig}}(\lambda)$. To sign and verify messages of length len_m, sample message signature keys
   $$(\mathsf{msk}_{(\ell,i)}, \mathsf{mvk}_{(\ell,i)}, \mathsf{mpp}_{(\ell,i)}) \leftarrow \mathsf{Sig.Setup}(1^\lambda, \mathsf{tlen}, \mathsf{len}_m) .$$
4. **Signing Routes**: For each sender $u \in [n]$ do the following:
   a) Parse $\mathsf{rte}_u = (j_1, \ldots, j_L)$. Sign rte_u using route signing keys for each wire along rte_u, that is, for $\ell \in [L]$ compute $\mathsf{rsig}_\ell = \mathsf{Sig.Sign}(\mathsf{rpp}_{(\ell,j_\ell)}, \mathsf{rsk}_{(\ell,j_\ell)}, 1, \mathsf{rte})$.
   b) Set $\overline{\mathsf{rte}}_u = (\mathsf{rte}_u, \mathsf{rsig}_u = (\mathsf{rsig}_1, \ldots, \mathsf{rsig}_L))$.
5. **Setting Routing Token**:
   a) For each merge-split gate (ℓ, g) in $[L-1] \times [G]$, compute an indistinguishability obfuscation $\overline{\mathsf{Gate}}_{(\ell,g)} \leftarrow \mathsf{iO}(1^\lambda, \mathsf{Gate}_{(\ell,g)})$ of the circuit $\mathsf{Gate}_{(\ell,g)}$ described in Figure 4.1.
   b) Set $\mathsf{tk} = \{\overline{\mathsf{Gate}}_{(\ell,g)} : \ell \in [L-1], g \in [G]\}$.
6. **Setting Sender Keys**: For each $u \in [n]$, set $\mathsf{ek}_u = (k_{(1,j_1)}, \mathsf{mpp}_{(1,j_1)}, \mathsf{msk}_{(1,j_1)}, \overline{\mathsf{rte}}_u)$.
7. **Setting Receiver Keys**: For each $v \in [n]$, set $\mathsf{rk}_v = k_{(L,2v-1)}$.
8. Output $(\{\mathsf{ek}_u\}_{u\in[n]}, \{\mathsf{rk}_u\}_{u\in[n]}, \mathsf{tk})$.

Figure 4.2: The Setup algorithm

Next, we describe how encryption, routing and decryption work.

Encryption Algorithm. For a sender u to send a message x to its receiver for time step t, the sender first computes a message signature msig for the tuple $(x, \overline{\mathsf{rte}}_u)$ for round t, and encrypts the tuple $(x, \overline{\mathsf{rte}}_u, \mathsf{msig})$ using its PRF key.

$\mathsf{Enc}(\mathsf{ek}_u, x_u, t)$ on input user u's encryption key ek_u and plaintext x_u and the round t, does the following:

1. Parse ek_u as $(k, \mathsf{mpp}, \mathsf{msk}, \overline{\mathsf{rte}}_u)$.
2. Compute the message signature $\mathsf{msig} = \mathsf{Sig.Sign}(\mathsf{mpp}, \mathsf{msk}, t, (x_u, \overline{\mathsf{rte}}_u))$.
3. Compute the ciphertext $\mathsf{CT}_u = (x_u, \overline{\mathsf{rte}}_u, \mathsf{msig}) \oplus \mathsf{PRF}(k, t)$.
4. Output CT_u.

Routing Algorithm. The router receives a routing token tk from the Setup algorithm. It consists of obfuscation of each gate in the routing network as described in Figure 4.1. During each round t, the router receives n ciphertexts $\mathsf{CT}_1, \ldots, \mathsf{CT}_n$ from the n senders. Before processing the ciphertexts through the routing network, the router sets the $2n$ ciphertexts $\mathsf{CT}_{(1,1)}, \ldots, \mathsf{CT}_{(1,2n)}$ for the first layer as follows. For all $i \in [n]$, it sets $\mathsf{CT}_{(1,2i-1)} = \mathsf{CT}_i$ as the real ciphertexts and $\mathsf{CT}_{(1,2i)} = \perp_{\mathsf{filler}}$ as the *filler* ciphertexts, where $\perp_{\mathsf{filler}}$ is a special string. Next, the router uses the token tk to route the $2n$ ciphertexts in the first layer through the routing network to obtain the $2n$ ciphertexts in the last layer L: $\mathsf{CT}_{(L,1)}, \ldots, \mathsf{CT}_{(L,2n)}$. Finally, to all receivers $i \in [n]$, the router sends the ciphertexts $\mathsf{CT}'_i = \mathsf{CT}_{(L,2i-1)}$. More formally,

$\mathbf{Rte}(\mathsf{tk}, t, \mathsf{CT}_1, \mathsf{CT}_2, \ldots, \mathsf{CT}_n)$ on input the router token tk along with the round number t, and ciphertexts $\mathsf{CT}_1, \ldots, \mathsf{CT}_n$, does the following:

1. Parse $\mathsf{tk} = \{\overline{\mathsf{Gate}}_{(\ell,g)} : \ell \in [L-1], g \in [G]\}$.
2. Compute ciphertexts for the input layer:
   a) For all $k \in [n]$, set $\mathsf{CT}_{(1,2k-1)} = \mathsf{CT}_k$. // *Real ciphertexts*
   b) For all $k \in [n]$, set $\mathsf{CT}_{(1,2k)} = \perp_{\mathsf{filler}}$. // *Filler ciphertexts*
3. Compute network of iO obfuscated gates layer-by-layer, that is, for layer $\ell = 1, \ldots, L-1$, evaluate all the obfuscated gates at this layer as follows. For each $g \in [G]$, let $\mathsf{Input}_{(\ell,g)}$ and $\mathsf{Output}_{(\ell,g)}$ be the set of input and output wires of gate $\mathsf{Gate}_{(\ell,g)}$. Then, evaluate the circuit
$$\{\mathsf{CT}_{(\ell+1,i)} : i \in \mathsf{Output}_{(\ell,g)}\} = \overline{\mathsf{Gate}}_{(\ell,g)}(t, \{\mathsf{CT}_{(\ell,i)} : i \in \mathsf{Input}_{(\ell,g)}\}).$$
4. Output $(\mathsf{CT}'_1 = \mathsf{CT}_{(L,1)}, \mathsf{CT}'_2 = \mathsf{CT}_{(L,3)}, \ldots, \mathsf{CT}'_n = \mathsf{CT}_{(L,2n-1)})$.

Decryption Algorithm. A receiver u learns its intended message by just decrypting the received ciphertext using its PRF key. More formally,

$\mathsf{Dec}(\mathsf{rk}_u, \mathsf{CT}'_u, t)$ on input user u's receiver key rk_u, output ciphertext CT'_u, and a time step t, does the following: Output $y = \mathsf{CT}'_u \oplus \mathsf{PRF}(\mathsf{rk}_u, t)$.

4.6.2 Efficiency Analysis

Recall that we assume $\mathsf{len} = 1$ since for multi-bit messages, since we can always parallel-compose multiple NIAR schemes where $\mathsf{len} = 1$ to get a NIAR scheme for $\mathsf{len} > 1$. In the analysis below, we argue that the router computation per time is bounded by $\widetilde{O}_\lambda(n)$ where $\widetilde{O}_\lambda$ hides $\mathsf{poly}(\lambda, \log n)$ factors for some fixed $\mathsf{poly}(\cdot)$.

Recall that the routing network consists of layers $0, \ldots, L$, where $L = O(\log n)$. In each of the $L - 1$ intermediate layers, there are $G = 2n/W$ number of gates, where $W = O(\log^2 \lambda)$ is the number of incoming and outgoing wires in each gate.

Size of hardcoded values in each gate. Each incoming wire has the following hardcoded: PRF key of size $\mathsf{poly}(\lambda)$, route public parameters of size $\mathsf{poly}(\lambda)$ and route verification key of size $\mathsf{poly}(\lambda)$, message public parameters of size $\mathsf{poly}(\lambda)$ and message verification key of size $\mathsf{poly}(\lambda)$. Each outgoing wire has the following hardcoded: PRF key of size $\mathsf{poly}(\lambda)$, message public parameters of size $\mathsf{poly}(\lambda)$ and message signing key of size $\mathsf{poly}(\lambda, \mathsf{tlen}) = \mathsf{poly}(\lambda)$ as $\mathsf{tlen} = \log^2(\lambda)$.

Size of ciphertexts. Each route signature is of size $\mathsf{poly}_{\mathsf{rsig}}(\lambda)$. and each message signature is of size $\mathsf{poly}_{\mathsf{msig}}(\lambda)$. Therefore, the ciphertexts are of size $\mathsf{tlen} + 1 + L \cdot \log(2n) + L \cdot \mathsf{poly}_{\mathsf{rsig}}(\lambda) + \mathsf{poly}_{\mathsf{msig}}(\lambda) = \mathsf{poly}(\lambda, \log n)$.

Size and running time of each gate. Each gate has W incoming and outgoing wires and each gate processes W ciphertexts, where $W = O(\log^2 \lambda)$. Therefore, each gate has $\mathsf{poly}(\lambda)$ amount of hardcoded information and processes $\mathsf{poly}(\lambda, \log n)$ amount of inputs. Based on the operations inside each gate, we can then conclude that each gate is of size $\mathsf{poly}(\lambda, \log n)$. Then, accounting for the polynomial blowup of the iO obfuscator, we can conclude that the size of each obfuscated gate is still $\mathsf{poly}(\lambda, \log n)$ and the router can run each obfuscated circuit in time $\mathsf{poly}(\lambda, \log n)$.

Router computation per time step. Observe that for each time step, the router computes each of the obfuscated circuits at most once. Since there are at most $\tilde{O}(n)$ gates, we can conclude that the router

computation per time step is bounded by $\widetilde{O}_\lambda(n)$ where $\widetilde{O}_\lambda$ hides $\mathsf{poly}(\lambda, \log n)$ factors for some fixed $\mathsf{poly}(\cdot)$.

Sender and recevier key sizes, computation and communication per time step. Sender key size is bounded by the size of the route which is $\widetilde{O}_\lambda(1)$. For every sender, computation and communication per time step is $\widetilde{O}_\lambda(1)$. Each receiver's key contains a PRF key which is $O_\lambda(1)$ in size. For every receiver, computation and communication per time step is $O_\lambda(1)$.

4.6.3 Static Security Theorem

In Sections 4.7 and 4.8, we prove the following theorem, which shows that the above construction satisfies static security as long as the adversary always corrupts all receivers. In Section 4.9, we give a compiler that further compiles the scheme to one that satisfies full security under adaptive corruptions, and without any restrictions on the adversary.

Theorem 4.6.1. *Suppose* PRF *is a secure puncturable PRF,* Sig *is a secure deterministic SSU signature scheme, and* iO *is a secure indistinguishability obfuscation scheme. Then, our NIAR construction in Section 4.6.1 satisfies full static corruption security (Definition 4.3.1) subject to an all-receiver-corrupting adversary.*

We give a proof roadmap of Theorem 4.6.1 below.

Proof roadmap. We prove Theorem 4.6.1 through a sequence of steps.

- In Definition 4.7.1, we define indistinguishability w.r.t. inversions against an adversary that additionally satisfies the selective single-challenge restriction. Then, we present an Upgrade Theorem stated in Theorem 4.7.2 which shows how to remove the selective single-challenge and inversion restrictions.
- Next, to complete the proof of Theorem 4.6.1, it suffices to prove security under the selective single-challenge and single inversion restrictions. We show this in Theorem 4.8.1.

4.7 Upgrading Security Under Static Corruption

In this section, we prove an upgrade theorem which shows that to prove the static security of our scheme, it suffices to prove a weaker notion of security that imposes additional constraints on the adversary.

4.7.1 Definition: Selective Single-Challenge Security w.r.t. a Single Inversion

We now define a weaker security notion called selective single-challenge seucrity w.r.t. inversion. In the security game, the adversary $\mathcal{A}$ must be subject to a set of restrictions (besides having to corrupt all players upfront):

- $\mathcal{A}$ must submit a pair of permutations $\pi^{(0)}$ and $\pi^{(1)}$ that differ by only a single inversion, i.e., $\pi^{(1)}$ can be obtained from $\pi^{(0)}$ by swapping a pair of senders' destinations — henceforth we use s and s' to denote this pair of senders; and
- there is only a single challenge time step, and $\mathcal{A}$ must commit to the challenge time step and challenge plaintexts for honest senders upfront.

Introducing a simulated setup algorithm. To make such a definition possible, we must first impose an additional requirement on the syntax. Specifically, we introduce a simulated setup algorithm Setup^* that is never used in the real world, but needed for the security definition. Specifically, the simulated setup algorithm $\mathsf{Setup}^*(1^\lambda, \mathsf{len}, n, \pi^{(0)}, \pi^{(1)})$ takes in both permutations $\pi^{(0)}, \pi^{(1)}$ that differ by one inversion pertaining to a pair of senders s and s', and it outputs $\{\mathsf{ek}_u\}_{u\in[n]\setminus\{s,s'\}}$, $(\mathsf{ek}_s^{(0)}, \mathsf{ek}_{s'}^{(0)})$, $(\mathsf{ek}_s^{(1)}, \mathsf{ek}_{s'}^{(1)})$, $\{\mathsf{rk}_u\}_{u\in[n]}$, and tk. Importantly, observe that a single set of sender keys $\{\mathsf{ek}_u\}_{u\in[n]\setminus\{s,s'\}}$, for $[n]\setminus\{s,s'\}$ and the routing token tk must be simultaneously compatible with two different sets of sender keys $(\mathsf{ek}_s^{(0)}, \mathsf{ek}_{s'}^{(0)})$, $(\mathsf{ek}_s^{(1)}, \mathsf{ek}_{s'}^{(1)})$, for the swapped senders s and s', corresponding to the two worlds $b = 0$ and $b = 1$, respectively. More precisely, we want the following property:

Marginal distribution of simulated setup statistically close as the real setup:

- for either $b \in \{0,1\}$, the terms $\left(\{\mathsf{ek}_u\}_{u\in[n]\setminus\{s,s'\}}, (\mathsf{ek}_s^{(b)}, \mathsf{ek}_{s'}^{(b)}), \{\mathsf{rk}_u\}_{u\in[n]}, \mathsf{tk}\right)$ output by Setup^* has negligibly small statistical distance from the output of the real $\mathsf{Setup}(1^\lambda, \mathsf{len}, n, \pi^{(b)})$.

With this additional simulated setup algorithm, we are ready to define selective single-challenge security for a single inversion.

Experiment $\mathsf{NIARStatic\text{-}SelSingleCh\text{-}Inv}^{b,\mathcal{A}}(1^\lambda)$**.**

- $n, \mathsf{len}, \mathcal{K}_S, \mathcal{K}_R, \pi^{(0)}, \pi^{(1)}, t^*, \{x_{u,t^*}^{(0)}, x_{u,t^*}^{(1)}\}_{u\in\mathcal{H}_S} \leftarrow \mathcal{A}(1^\lambda)$;
- $(\{\mathsf{ek}_u\}_{u\in[n]\setminus\{s,s'\}}, \mathsf{ek}_s^{(0)}, \mathsf{ek}_{s'}^{(0)}, \mathsf{ek}_s^{(1)}, \mathsf{ek}_{s'}^{(1)}, \{\mathsf{rk}_u\}_{u\in[n]}, \mathsf{tk}) \leftarrow \mathsf{Setup}^*(1^\lambda, \mathsf{len}, n, \pi^{(0)}, \pi^{(1)})$;
- $\perp \leftarrow \mathcal{A}(\mathsf{tk}, \{\mathsf{ek}_u\}_{u\in\mathcal{K}_S}, \{\mathsf{rk}_u\}_{u\in\mathcal{K}_R})$;
- For $t = 1, 2, \ldots$:

- if $t \neq t^*$: $(\{x_{u,t}\}_{u\in\mathcal{H}_S}, \delta_t) \leftarrow \mathcal{A}(\perp)$, and for $u \in \{s, s'\}$, let $\mathsf{CT}_{u,t} := \mathsf{Enc}(\mathsf{ek}_u^{(\delta_t)}, x_{u,t}, t)$; for all other $u \in \mathcal{H}_S$, let $\mathsf{CT}_{u,t} := \mathsf{Enc}(\mathsf{ek}_u, x_{u,t}, t)$;
- else if $t = t^*$: for $u \in \{s, s'\}$, let $\mathsf{CT}_{u,t^*} := \mathsf{Enc}(\mathsf{ek}_u^{(b)}, x_{u,t^*}^{(b)}, t^*)$; for all other $u \in \mathcal{H}_S$, let $\mathsf{CT}_{u,t^*} := \mathsf{Enc}(\mathsf{ek}_u, x_{u,t^*}^{(b)}, t^*)$;
- $\perp \leftarrow \mathcal{A}(\{\mathsf{CT}_{u,t}\}_{u\in\mathcal{H}_S})$;

The adversary is said to be admissible, iff with probability 1, $\mathsf{Leak}^*(\pi^{(0)}, \mathcal{K}_S, \mathcal{K}_R, \{x_{u,t^*}^{(0)}\}_{u\in\mathcal{H}_S}) = \mathsf{Leak}^*(\pi^{(1)}, \mathcal{K}_S, \mathcal{K}_R, \{x_{u,t^*}^{(1)}\}_{u\in\mathcal{H}_S})$ where where the function Leak^* $(\pi, \mathcal{K}_S, \mathcal{K}_R), \{x_{u,t^*}\}_{u\in\mathcal{H}_S})$ contains the destination of each corrupt sender and the contents of the messages from honest senders to corrupt receivers, as defined below:

$$\mathsf{Leak}^*(\pi, \mathcal{K}_S, \mathcal{K}_R, \{x_{u,t^*}\}_{u\in\mathcal{H}_S}) := (\{(u, \pi(u))\}_{u\in\mathcal{K}_S}, \{(u, x_{\pi^{-1}(u),t^*})\}_{u\in\mathcal{K}_R, \pi^{-1}(u)\in\mathcal{H}_S})$$

Intuitively, the admissibility rule requires that the corrupt senders have the same destinations in the two worlds, and that corrupt receivers receive the same messages from honest senders in the two worlds.

Definition 4.7.1 (Selective single-challenge security w.r.t. inversion under static corruption)**.** *We say that a NIAR scheme (augmented with a* Setup^* *algorithm) satisfies selective security* w.r.t. inversion *under static corruption, iff for any non-uniform p.p.t. admissible adversary* $\mathcal{A}$ *which,* with probability 1, submits two permutations π_0 and π_1 that differ by a single inverstion*,* $\mathcal{A}$*'s views in* $\mathsf{NIARStatic\text{-}SelSingleCh\text{-}Inv}^{0,\mathcal{A}}(1^\lambda)$ *and* $\mathsf{NIARStatic\text{-}SelSingleCh\text{-}Inv}^{1,\mathcal{A}}(1^\lambda)$ *are computationally indistinguishable.*

4.7.2 Upgrade Theorem for Static Corruption

In this section, we prove an upgrade theorem for the static corruption setting: we start with a NIAR scheme that is secure when the adversary is subject to the single selective-challenge and single inversion restrictions, and show that the same scheme also satisfies security without the single selective-challenge and single inversion restrictions. It turns out that *we only need to prove the upgrade theorem for the special case when the adversary always corrupts all receivers with probability* 1. This is because later in Section 4.9, we show how to compile a NIAR scheme secure under static corruption as long as the adversary always corrupt all receivers, to a NIAR scheme that is fully secure even under adaptive corruptions, and without the restriction that all receivers must be corrupt.

All-receiving-corrupting adversary. Henceforth, if an adversary corrupts all receivers with probability 1, we say that it is an "all-receiver-corrupting adversary". Moreover, if corruption is static, we also refer to such an adversary as a "static, all-receiver-corrupting adversary".

Theorem 4.7.2 (Upgrade theorem for static corruption: removing the selective single challenge and single inversion restrictions)**.** *Given a NIAR scheme which works for single-bit messages, and moreover satisfies selective security w.r.t. inversion (i.e., Definition 4.7.1), under a static, all-receiver-corrupting adversary, then it also satisfies full static corruption security (Definition 4.3.1) subject to an all-receiver-corrupting adversary.*

4.7.3 Proof of the Upgrade Theorem

We prove Theorem 4.7.2 in the remainder of this section.

Removing the Selective Restriction

We define an adaptive single-challenge notion also w.r.t. inversion, which removes the restriction that the adversary $\mathcal{A}$ must commit to the challenge t^* and challenge plaintexts for honest users upfront.

Definition 4.7.3 (Adaptive single-challenge security w.r.t. inversion under static corruption)**.** *We say that a NIAR scheme (augmented with a* Setup^* *algorithm) satisfies adaptive single-challenge security w.r.t. inversion under static corruption, iff for any non-uniform p.p.t. admissible adversary* $\mathcal{A}$*, its views in the following experiments* $\mathsf{NIARStatic\text{-}AdSingleCh\text{-}Inv}^{0,\mathcal{A}}(1^\lambda)$ *and* $\mathsf{NIARStatic\text{-}AdSingleCh\text{-}Inv}^{1,\mathcal{A}}(1^\lambda)$ *are computationally indistinguishable.*

Experiment $\mathsf{NIARStatic\text{-}AdSingleCh\text{-}Inv}^{b,\mathcal{A}}(1^\lambda)$**.**

- $n, \mathsf{len}, \mathcal{K}_S, \mathcal{K}_R, \pi^{(0)}, \pi^{(1)} \leftarrow \mathcal{A}(1^\lambda)$;
- $(\{\mathsf{ek}_u\}_{u\in[n]\setminus\{s,s'\}}, \mathsf{ek}_s^{(0)}, \mathsf{ek}_{s'}^{(0)}, \mathsf{ek}_s^{(1)}, \mathsf{ek}_{s'}^{(1)}, \{\mathsf{rk}_u\}_{u\in[n]}, \mathsf{tk}) \leftarrow \mathsf{Setup}^*(1^\lambda, \mathsf{len}, n, \pi^{(0)}, \pi^{(1)})$;
- $\perp \leftarrow \mathcal{A}(\mathsf{tk}, \{\mathsf{ek}_u\}_{u\in\mathcal{K}_S}, \{\mathsf{rk}_u\}_{u\in\mathcal{K}_R})$;
- For $t = 1, 2, \ldots$:
  - $(\{x_{u,t}^{(0)}\}_{u\in\mathcal{H}_S}, \{x_{u,t}^{(1)}\}_{u\in\mathcal{H}_S}, \delta_t) \leftarrow \mathcal{A}(\perp)$ where $\delta_t \in \{0, 1, \text{“}\texttt{challenge}\text{”}\}$;
  - if $\delta_t \in \{0,1\}$, then for $u \in \mathcal{H}_S$, let $\mathsf{CT}_{u,t} := \mathsf{Enc}(\mathsf{ek}_u^{(\delta_t)}, x_{u,t}, t)$ where $\mathsf{ek}_u^{(\delta_t)} := \mathsf{ek}_u$ if $u \notin \{s, s'\}$;
  - else if $\delta_t =$ “$\texttt{challenge}$”, then for $u \in \mathcal{H}_S$, let $\mathsf{CT}_{u,t} := \mathsf{Enc}(\mathsf{ek}_u^{(b)}, x_{u,t}^{(b)}, t)$ where $\mathsf{ek}_u^{(b)} := \mathsf{ek}_u$ if $u \notin \{s, s'\}$;
  - $\perp \leftarrow \mathcal{A}(\{\mathsf{CT}_{u,t}\}_{u\in\mathcal{H}_S})$;

The adversary $\mathcal{A}$ is said to be admissible iff with probability 1, the following hold:

- There is a unique time step henceforth denoted t^* in which $\mathcal{A}$ sets δ_t to be “$\texttt{challenge}$”; and
- $\mathsf{Leak}^*(\pi^{(0)}, \mathcal{K}_S, \mathcal{K}_R, \{x_{u,t^*}^{(0)}\}_{u\in\mathcal{H}_S}) = \mathsf{Leak}^*(\pi^{(1)}, \mathcal{K}_S, \mathcal{K}_R, \{x_{u,t^*}^{(1)}\}_{u\in\mathcal{H}_S})$.

The lemma below shows that we can remove the selective single-challenge restriction for free and upgrade the security to adaptive single-challenge, under a single-inversion, all-receiver-corrupting, static-corruption adversary. The lemma considers a NIAR scheme where each sender's plaintext message in every time step is a single bit, i.e., $\mathsf{len} = 1$. This assumption is without loss of generality, since we can always parallel-compose multiple NIAR schemes for $\mathsf{len} = 1$ to get a NIAR scheme for $\mathsf{len} > 1$. We need the $\mathsf{len} = 1$ assumption for the proof to work because we want to make sure the reduction's guess is correct with $1/\mathsf{poly}(\lambda)$ probability — see the proof for more details.

Lemma 4.7.4 (Removing the selective restriction)**.** *Given any NIAR scheme which works for single-bit messages, and moreover satisfies selective single-challenge security w.r.t. a single inversion (Definition 4.7.1) under a static, all-receiver-corrupting adversary, then it is also adaptive single-challenge secure w.r.t. a single inversion (Definition 4.7.3) under a static, all-receiver-corrupting adversary.*

Proof. We consider a reduction $\mathcal{B}$ that interacts with an adaptive single-challenge adversary $\mathcal{A}$ and leverages it to break selective single-challenge security, w.r.t. inversion in both cases.

- At the start, $\mathcal{A}$ submits to $\mathcal{B}$ the terms n, len, the set of corrupt senders $\mathcal{K}_S$, and two permutations $\pi^{(0)}$ and $\pi^{(1)}$ that differ by a single inversion (recall also that $\mathcal{A}$ always corrupts all receivers). Let s, s' be the pair of senders whose destinations are swapped. By the admissibility rule on $\mathcal{A}$, s and s' must be honest.
- Let T be the maximum number of queries made by $\mathcal{A}$. $\mathcal{B}$ guesses at random $t^* \xleftarrow{\$} [T]$, guesses the plaintext messages $x^{(0)}_{s,t^*}$, $x^{(0)}_{s',t^*}$, and lets $x^{(1)}_{s,t^*} = x^{(0)}_{s',t^*}$, $x^{(1)}_{s',t^*} = x^{(0)}_{s,t^*}$.
- With its own challenger, $\mathcal{B}$ corrupts all receivers, and all senders except s and s'. It submits the same permutations $\pi^{(0)}$ and $\pi^{(1)}$, and the guessed t^*, and the guessed plaintext messages $x^{(0)}_{s,t^*}$, $x^{(0)}_{s',t^*}$, $x^{(1)}_{s,t^*}$, $x^{(1)}_{s',t^*}$.
- As a result, $\mathcal{B}$ obtains $\{\mathsf{ek}_u\}_{u\in[n]\setminus\{s,s'\}}$, $\{\mathsf{rk}_u\}_{u\in[n]}$, and tk from its own challenger. $\mathcal{B}$ now passes $\{\mathsf{ek}_u\}_{u\in\mathcal{K}_S}$, $\{\mathsf{rk}_u\}_{u\in[n]}$, and tk to $\mathcal{A}$.
- In the following, if $\mathcal{A}$ ever submits a challenge query during some $t \neq t^*$, or it does not submit a challenge query during t^*, $\mathcal{B}$ aborts.
- During every time step $t \neq t^*$, whenever $\mathcal{A}$ submits $\{x^{(0)}_{u,t}\}_{u\in\mathcal{H}_S}$, $\{x^{(1)}_{u,t}\}_{u\in\mathcal{H}_S}$, and $\delta_t \in \{0,1\}$ (assuming $\mathcal{B}$ has not aborted), $\mathcal{B}$ submits $\{x^{(\delta_t)}_{u,t}\}_{u\in\{s,s'\}}$ and δ_t to its own challenger, and it gets back the encryptions $\mathsf{CT}_{s,t}$ and $\mathsf{CT}_{s',t}$. For any $u \in \mathcal{H}_S\setminus\{s,s'\}$, $\mathcal{B}$ simply uses ek_u to encrypt $x^{(\delta_t)}_{u,t}$ and obtains $\mathsf{CT}_{u,t}$. $\mathcal{B}$ returns to $\mathcal{A}$ the resulting $\{\mathsf{CT}_{u,t}\}_{u\in\mathcal{H}_S}$.

- For $t = t^*$, assuming the reduction $\mathcal{B}$ has not aborted, $\mathcal{A}$ must have specified t^* to be the challenge time step. Now, check if the challenge plaintexts $\{x^{(0)}_{u,t^*}, x^{(1)}_{u,t^*}\}_{u\in\mathcal{H}_S}$ $\mathcal{A}$ has submitted are consistent with the reduction $\mathcal{B}$'s guesses $x^{(0)}_{s,t^*}$, $x^{(0)}_{s',t^*}$, $x^{(1)}_{s,t^*}$, $x^{(1)}_{s',t^*}$. If not, the reduction $\mathcal{B}$ simply aborts.

  Now, $\mathcal{B}$ receives from its challenger CT_{s,t^*} and CT_{s',t^*}. For any $u \in \mathcal{H}_S\backslash\{s, s'\}$, by the admissibility rule on $\mathcal{A}$, it must be that $x^{(0)}_{u,t^*} = x^{(1)}_{u,t^*}$. $\mathcal{B}$ now uses the corresponding ek_u to compute an encryption of $x^{(0)}_{u,t^*}$ and let the result be CT_{u,t^*}. $\mathcal{B}$ now returns $\{\mathsf{CT}_{u,t^*}\}_{u\in\mathcal{H}_S}$ to $\mathcal{A}$.

- At the end, if $\mathcal{B}$ has not aborted, it outputs whatever $\mathcal{A}$ outputs.

Observe that the Setup^* algorithm executed by $\mathcal{B}$'s challenger does not depend on the challenge time step t^* or the challenge plaintexts. Therefore, if $\mathcal{B}$'s challenger is in world $b = 0$, and if the reduction did not abort, then $\mathcal{A}$'s view is identically distributed as in $\mathsf{NIARStatic\text{-}AdSingleCh}^{0,\mathcal{A}}$ (subject to single inversion). Otherwise, if $\mathcal{B}$'s challenger is in world $b = 1$, and if the reduction did not abort, then $\mathcal{A}$'s view is identically distributed as in $\mathsf{NIARStatic\text{-}AdSingleCh}^{1,\mathcal{A}}$ (subject to single inversion). The lemma follows by observing as long as the message length is only one bit, the probability that the reduction $\mathcal{B}$ guesses correctly is $1/\mathsf{poly}(\lambda)$. □

Removing the Single Challenge Restriction

We now prove that we can further remove the single challenge restriction.

Lemma 4.7.5 (Removing the single challenge restriction)**.** *Given any NIAR scheme that satisfies adaptive single-challenge security w.r.t. inversion (Definition 4.7.3) subject to a static, all-receiver corrupting adversary, the same scheme also satisfies Definition 4.3.1 subject to a static, single-inversion, all-receiver-corrupting adversary.*

Proof. Let T be the maximum number of queries made by the adversary. We consider the following sequence of hybrids indexed by $i \in \{0, 1, \ldots, T\}$, and recall that the permutations $\pi^{(0)}, \pi^{(1)}$ submitted by the adversary $\mathcal{A}$ must differ by only one inversion where the swapped pair of senders is denoted s and s' below.

Hybrid experiment Hyb_i**:**

- $n, \mathsf{len}, \mathcal{K}_S, \mathcal{K}_R, \pi^{(0)}, \pi^{(1)} \leftarrow \mathcal{A}(1^\lambda)$;
- $(\{\mathsf{ek}_u\}_{u\in[n]\setminus\{s,s'\}}, \mathsf{ek}^{(0)}_s, \mathsf{ek}^{(0)}_{s'}, \mathsf{ek}^{(1)}_s, \mathsf{ek}^{(1)}_{s'}, \{\mathsf{rk}_u\}_{u\in[n]}, \mathsf{tk}) \leftarrow \mathsf{Setup}^*(1^\lambda, \mathsf{len}, n, \pi^{(0)}, \pi^{(1)})$;
- $\perp \leftarrow \mathcal{A}(\mathsf{tk}, \{\mathsf{ek}_u\}_{u\in\mathcal{K}_S}, \{\mathsf{rk}_u\}_{u\in\mathcal{K}_R})$;
- For $t = 1, 2, \ldots$:

- $(\{x^{(0)}_{u,t}\}_{u\in\mathcal{H}_S}, \{x^{(1)}_{u,t}\}_{u\in\mathcal{H}_S}) \leftarrow \mathcal{A}(\perp)$;
- if $t > i$: then for $u \in \mathcal{H}_S$, let $\mathsf{CT}_{u,t} := \mathsf{Enc}(\mathsf{ek}^{(0)}_u, x_{u,t}, t)$ where $\mathsf{ek}^{(0)}_u := \mathsf{ek}_u$ if $u \notin \{s, s'\}$;
- else if $t \le i$, then for $u \in \mathcal{H}_S$, let $\mathsf{CT}_{u,t} := \mathsf{Enc}(\mathsf{ek}^{(1)}_u, x^{(1)}_{u,t}, t)$ where $\mathsf{ek}^{(1)}_u := \mathsf{ek}_u$ if $u \notin \{s, s'\}$;
- $\perp \leftarrow \mathcal{A}(\{\mathsf{CT}_{u,t}\}_{u\in\mathcal{H}_S})$;

$\mathcal{A}$ is said to be admissible iff it is subject to the same admissibility rules as Definition 4.3.1; moreover, we also assume that $\mathcal{A}$ respects the single-inversion, and all-receiver-corrupting[5] constraints. Since the marginal distribution of Setup^* is statistical close to the real-world Setup algorithm, it holds that $\mathcal{A}$'s view in Hyb_0 is statistically close to $\mathsf{NIARStatic}^{0,\mathcal{A}}$, and its view in Hyb_T is statistically close to $\mathsf{NIARStatic}^{1,\mathcal{A}}$. It suffices to prove that every adjacent pair of hybrids are computationally indistinguishable to the adversary. This can be shown through a straightforward reduction to the adaptive single-challenge security w.r.t. inversion (Definition 4.7.3). □

Removing the Single Inversion Restriction

We next show how to remove the single inversion restriction on the adversary.

Lemma 4.7.6 (Removing the single inversion restriction (static corruption))**.** *Given a NIAR scheme that satisfies Definition 4.3.1 subject to a static, single-inversion, all-receiver-corrupting adversary, the same scheme also satisfies Definition 4.3.1 subject to a static, all-receiver-corrupting adversary.*

Proof. Given any two permutations $\pi^{(0)}$ and $\pi^{(1)}$ submitted by $\mathcal{A}$, let $\overline{C}(\pi^{(0)}, \pi^{(1)})$ be the set of senders that have different destinations in $\pi^{(0)}$ and $\pi^{(1)}$ — by the admissibility rule on $\mathcal{A}$, it must be that $\overline{C}(\pi^{(0)}, \pi^{(1)})$ are all honest senders. We define a sequence of permutations denoted $\pi^*_0, \ldots, \pi^*_n$ where $\pi^*_0 = \pi^{(0)}$, and for any $0 < i \le n$, π^*_i is almost the same as π^*_{i-1}, except that if $i \le |\overline{C}(\pi^{(0)}, \pi^{(1)})|$, then we additionally swap the destinations of the i-th honest sender in $\overline{C}(\pi^{(0)}, \pi^{(1)})$ denoted u^*_i and whoever is sending to $\pi^{(1)}(u^*_i)$ in π^*_{i-1} — by construction, the sender u^*_i is swapping destinations with another sender that must lie within the the set $\overline{C}(\pi^{(0)}, \pi^{(1)})$. Else if $i > |\overline{C}(\pi^{(0)}, \pi^{(1)})|$, then, $\pi^*_i = \pi^*_{i-1}$. By construction, in π^*_i, the first i honest senders in $\overline{C}(\pi^{(0)}, \pi^{(1)})$ have their correct destinations as in $\pi^{(1)}$, and thus $\pi^*_n = \pi^{(1)}$.

We now consider a sequence of hybrid experiments denoted Hyb_i where $i \in \{0, 1, \ldots, n\}$, in which a challenger interacts with an adversary $\mathcal{A}$ that has the same interface as a $\mathsf{NIARStatic}^{b,\mathcal{A}}$ adversary, and moreover, it always corrupts all receivers upfront. Namely, $\mathcal{A}$ submits $n, \mathsf{len}, \mathcal{K}_S, \mathcal{K}_R, \pi^{(0)}, \pi^{(1)}$ upfront where $\mathcal{K}_R$ is guaranteed to be $[n]$, and then in every time step t, it submits $\{x^{(0)}_{u,t}, x^{(1)}_{u,t}\}_{u\in\mathcal{H}_S}$. In Hyb_i, the challenger computes the responses to $\mathcal{A}$ as follows:

[5]The all-receiver-corrupting restriction is not important for this lemma, that is, the lemma and the proof still hold if we remove every occurrence of "all-receiver-corrupting".

- It calls the Setup algorithm on the input len, n and the permutation π_i^* (which is uniquely determined given $\pi^{(0)}$ and $\pi^{(1)}$ submitted by $\mathcal{A}$),
- During each time step t, upon receiving $\{x_{u,t}^{(0)}, x_{u,t}^{(1)}\}_{u \in \mathcal{H}_S}$, do the following: for each $u \in \mathcal{H}_S$, let $x_{u,t}^* = x_{u',t}^{(1)}$ where $u' = {\pi^{(1)}}^{-1}(\pi_i^*(u))$; now compute $\mathsf{ct}_{u,t} = \mathsf{Enc}(\mathsf{ek}_u, x_{u,t}^*, t)$.

By construction, and due to the admissibility rule on $\mathcal{A}$, Hyb_0 is the same as $\mathsf{NIARStatic}^{0,\mathcal{A}}$, and Hyb_n is the same as $\mathsf{NIARStatic}^{1,\mathcal{A}}$. It suffices to argue that the adversary's views in every pair of adjacent hybrids Hyb_i and Hyb_{i+1} are computationally indistinguishable. This can be achieved through a reduction to the static single-inversion security under all-corrupt-receivers.

If $\pi_{i+1}^* = \pi_i^*$, then by definition, Hyb_i and Hyb_{i+1} are identically. Henceforth we focus on the case when π_{i+1}^* and π_i^* differ by exactly one inversion. Consider a reduction $\mathcal{B}$ which receives $n, \mathsf{len}, \mathcal{K}_S, \mathcal{K}_R, \pi^{(0)}, \pi^{(1)}$ from $\mathcal{A}$ upfront where $\mathcal{K}_R$ is guaranteed to be $[n]$, $\mathcal{B}$ submits to its own challenger $n, \mathsf{len}, \mathcal{K}_S, \mathcal{K}_R, \pi_i^*, \pi_{i+1}^*$ and passes the responses to $\mathcal{A}$. In every time step t, $\mathcal{B}$ receives $\{x_{u,t}^{(0)}, x_{u,t}^{(1)}\}_{u \in \mathcal{H}_S}$ from $\mathcal{A}$. Now, for each $u \in \mathcal{H}_S$, let $x_{u,t}^* = x_{u',t}^{(1)}$ where $u' = {\pi^{(1)}}^{-1}(\pi_i^*(u))$, and let $y_{u,t}^* = x_{u',t}^{(1)}$ where $u' = {\pi^{(1)}}^{-1}(\pi_{i+1}^*(u))$. It submits to its own challenger the challenge plaintexts $\{x_{u,t}^*, y_{u,t}^*\}_{u \in \mathcal{H}_S}$, and passes the responses to $\mathcal{A}$. $\mathcal{B}$ outputs whatever $\mathcal{A}$ outputs.

If $\mathcal{B}$'s challenger is in world $b = 0$ and encrypts the plaintexts $\{x_{u,t}^*\}_{u \in \mathcal{H}_S}$, then $\mathcal{A}$'s view is identically distributed as in Hyb_i; else $\mathcal{B}$'s challenger is in world $b = 1$ and encrypts the plaintexts $\{y_{u,t}^*\}_{u \in \mathcal{H}_S}$, then $\mathcal{A}$'s view is identically distributed as in Hyb_{i+1}.

Finally, by construction, if $\mathcal{A}$ respects its admissibility rules, then $\mathcal{B}$ respects its admissibility rules as well. Moreover, as mentioned earlier, $\mathcal{B}$ respects the single-inversion constraint. Therefore, $\mathcal{B}$ can translate $\mathcal{A}$'s advantage in distinguishing Hyb_i and Hyb_{i+1} into its own advantage at breaking the single-inversion static-corruption security of NIAR (subject to all-corrupting receivers). □

Completing the Proof

The proof of Theorem 4.7.2 follows directly by combining Lemmas 4.7.4 to 4.7.6.

4.8 Proof of Selective Single-Challenge Security w.r.t. a Single Inversion

We now prove indistinguishability w.r.t. inversions against an adversary that additionally satisfies the selective single-challenge restriction. Formally, we have the following theorem.

Theorem 4.8.1. *Suppose that* PRF *is a secure puncturable PRF,* Sig *is a secure deterministic SSU signature scheme, and* iO *is a secure indistinguishability obfuscation scheme. Then our NIAR construction in Section 4.6.1*

satisfies selective single-challenge security w.r.t. inversion under static corruption (Definition 4.7.1) subject to an all-receiver-corrupting adversary.

In order to invoke Definition 4.7.1, we next describe the Setup^* algorithm. We will show in Claim 4.8.2 that the Setup^* algorithm satisfies the requirements that its marginal distribution is statistically close to the real setup.

$\mathsf{Setup}^*(1^\lambda, \mathsf{len}, n, \pi^{(0)}, \pi^{(1)})$: on inputs the security parameter 1^λ, the individual message length len, the number of parties n, the permutations $\pi^{(0)}$ and $\pi^{(1)}$, does the following:

1. Set $\mathsf{tlen} = \log^2(\lambda)$.
2. **Sampling Routes**: Let the two senders that $\pi^{(0)}$ and $\pi^{(1)}$ differ on be s and s'. Run the $\mathsf{AssignRoutes}^*$ procedure (Section 4.4.2) on inputs $(1^\lambda, n, \pi^{(0)}, \pi^{(1)})$. Abort if it outputs $\perp$. Else parse the output as $(\{\mathsf{rte}_u\}_{u\in[n]\setminus\{s,s'\}}, \{\mathsf{rte}_u^{(0)}, \mathsf{rte}_u^{(1)}\}_{u\in\{s,s'\}})$.
3. **Sampling Wire Keys**: Same as in Setup (Figure 4.2).
4. **Signing Routes**: For each sender $u \in [n] \setminus \{s, s'\}$ do the following:
   a) Parse $\mathsf{rte}_u = (j_1, \ldots, j_L)$. Sign rte_u using route signing keys for each wire along rte_u, that is, for $\ell \in [L]$ compute $\mathsf{rsig}_\ell = \mathsf{Sig.Sign}(\mathsf{rpp}_{(\ell,j_\ell)}, \mathsf{rsk}_{(\ell,j_\ell)}, 1, \mathsf{rte}_u)$. // *Same way as in* Setup *(Figure 4.2).*
   b) Set $\overline{\mathsf{rte}}_u = (\mathsf{rte}_u, \mathsf{rsig}_u = (\mathsf{rsig}_1, \ldots, \mathsf{rsig}_L))$. // *Same way as in* Setup *(Figure 4.2).*

   For each sender $u \in \{s, s'\}$ and for each $\beta \in \{0, 1\}$, do the following:
   a) Parse $\mathsf{rte}_u^{(\beta)} = (j_1, \ldots, j_L)$. Sign $\mathsf{rte}_u^{(\beta)}$ using route signing keys for each wire along $\mathsf{rte}_u^{(\beta)}$, that is, for $\ell \in [L]$ compute $\mathsf{rsig}_\ell^{(\beta)} = \mathsf{Sig.Sign}(\mathsf{rpp}_{(\ell,j_\ell)}, \mathsf{rsk}_{(\ell,j_\ell)}, 1, \mathsf{rte}_u^{(\beta)})$. // *Same way as in* Setup *(Figure 4.2).*
   b) Set $\overline{\mathsf{rte}}_u^{(\beta)} = (\mathsf{rte}_u^{(\beta)}, \mathsf{rsig}_u = (\mathsf{rsig}_1^{(\beta)}, \ldots, \mathsf{rsig}_L^{(\beta)}))$. // *Same way as in* Setup *(Figure 4.2).*
5. **Setting Routing Token**: Same as in Setup (Figure 4.2).
6. **Setting Sender Keys**: For each $u \in [n] \setminus \{s, s'\}$, set $\mathsf{ek}_u = (k_{(1,j_1)}, \mathsf{mpp}_{(1,j_1)}, \mathsf{msk}_{(1,j_1)}, \overline{\mathsf{rte}}_u)$. For each $u \in \{s, s'\}$ and $\beta \in \{0, 1\}$, set $\mathsf{ek}_u^{(\beta)} = (k_{(1,j_1)}, \mathsf{mpp}_{(1,j_1)}, \mathsf{msk}_{(1,j_1)}, \overline{\mathsf{rte}}_u^{(\beta)})$.
7. **Setting Receiver Keys**: For each $v \in [n]$, set $\mathsf{rk}_v = k_{(L,2v-1)}$.
8. Output $(\{\mathsf{ek}_u\}_{u\in[n]\setminus\{s,s'\}}, \{\mathsf{ek}_u^{(0)}, \mathsf{ek}_u^{(1)}\}_{u\in\{s,s'\}}, \{\mathsf{rk}_u\}_{u\in[n]}, \mathsf{tk})$.

Figure 4.3: The Setup^* algorithm for selective single-challenge security w.r.t. inversion under static corruption as defined in Definition 4.7.1.

Claim 4.8.2. *If the routing network satisfies obliviousness as defined in Definition 4.4.1, then the* Setup^* *algorithm in Figure 4.3 satisfies the requirement that its marginal distribution is statistically close to the real setup. More formally,*

- *for either* $b \in \{0,1\}$*, the terms* $\left(\{\mathsf{ek}_u\}_{u\in[n]\setminus\{s,s'\}}, (\mathsf{ek}_s^{(b)}, \mathsf{ek}_{s'}^{(b)}), \{\mathsf{rk}_u\}_{u\in[n]}, \mathsf{tk}\right)$ *output by* $\mathsf{Setup}^*(1^\lambda, \mathsf{len}, \pi^{(0)}, \pi^{(1)})$ *has negligibly small statistically distance from the output of the real* $\mathsf{Setup}(1^\lambda, \mathsf{len}, n, \pi^{(b)})$.

Proof. For either $b \in \{0,1\}$, the terms $\left(\{\mathsf{ek}_u\}_{u\in[n]\setminus\{s,s'\}}, (\mathsf{ek}_s^{(b)}, \mathsf{ek}_{s'}^{(b)}), \{\mathsf{rk}_u\}_{u\in[n]}, \mathsf{tk}\right)$ output by Setup^* differs from the output of the real Setup only in the following way: while Setup samples the senders' routes using $\mathsf{AssignRoutes}$, Setup^* samples them using $\mathsf{AssignRoutes}^*$ algorithm. Therefore, for either $b \in \{0,1\}$, the indistinguishability of the terms $(\{\mathsf{ek}_u\}_{u\in[n]\setminus\{s,s'\}}, (\mathsf{ek}_s, \mathsf{ek}_{s'}), \{\mathsf{rk}_u\}_{u\in[n]}, \mathsf{tk})$ output by the real $\mathsf{Setup}(1^\lambda, \mathsf{len}, n, \pi^{(b)})$ and the terms $\left(\{\mathsf{ek}_u\}_{u\in[n]\setminus\{s,s'\}}, (\mathsf{ek}_s^{(b)}, \mathsf{ek}_{s'}^{(b)}), \{\mathsf{rk}_u\}_{u\in[n]}, \mathsf{tk}\right)$ output by Setup^* follows from the obliviousness of the routing network (Definition 4.4.1).

□

4.8.1 The Hybrids for Theorem 4.8.1

We are now ready to prove Theorem 4.8.1. We do this via the sequence of hybrids below. In this sequence, $\mathsf{Hyb}_0^{(0)}$ implements $\mathsf{NIARStatic\text{-}SelSingleCh\text{-}Inv}^{0,\mathcal{A}}$, and $\mathsf{Hyb}_0^{(1)}$ implements $\mathsf{NIARStatic\text{-}SelSingleCh\text{-}Inv}^{1,\mathcal{A}}$. Thus it suffices to prove indistinguishability between each successive pair of hybrids $\mathsf{Hyb}_i^{(b)}$ and $\mathsf{Hyb}_{i+1}^{(b)}$ for each i and $b \in \{0,1\}$, and that the final hybrid $\mathsf{Hyb}_9^{(b)}$ is wholly independent of b.

In the following, we say that a wire is "*corrupt*" or "*honest*" if it is on a path which originates with a *corrupt* or *honest* sender respectively, and we say that a wire is an "*inversion*" wire if it is on one of the paths that originate with the two *honest* senders s and s'. We say all wires apart from *corrupt* and *honest* wires to be "*filler*" wires. Also, we sometimes use the shorthand "wire (ℓ, j)" to refer to j^{th} wire in the ℓth layer.

Informal Hybrids

In the real world hybrid $\mathsf{Hyb}_0^{(b)}$, the adversary's view contains the sender keys of corrupt senders, the receiver keys of corrupt receivers, the routing token, non-challenge and challenge ciphertexts for honest senders. Let's understand which all of these terms contain information about the challenge bit b.

- As $\mathsf{AssignRoutes}^*$ assigns a single route to each corrupt sender, hence, the sender keys of corrupt senders are independent of b. All receiver keys are independent of b.
- The non-challenge ciphertexts are independent of b. It is only the challenge ciphertexts that are dependent on b as follows. The challenger provides the following challenge ciphertexts to the adversary.
  - for all $u \in \{s, s'\}$: $\mathsf{CT}_{u,t^*} = \mathsf{Enc}(\mathsf{ek}_u^{(b)}, x_{u,t^*}^{(b)}, t^*)$.
  - for all $u \in \mathcal{H}_S \setminus \{s, s'\}$: $\mathsf{CT}_{u,t^*} = \mathsf{Enc}(\mathsf{ek}_u, x_{u,t^*}^{(b)}, t^*)$.

CT_{u,t^*} **contain information about** b **only for** $u \in \{s, s'\}$**:** Observe that for senders $u \in \mathcal{H}_S \setminus \{s, s'\}$, $\pi^{(0)}(u) = \pi^{(1)}(u)$. This means that the receiver remains the same across the two worlds for each such sender. And recall that we are proving security against an all-receiver-corrupting adversary. The admissibility criteria requires then that each corrupt receiver must receive the same message across the two worlds. This implies that $x^{(0)}_{u,t^*} = x^{(1)}_{u,t^*}$ for all $u \in \mathcal{H}_S \setminus \{s, s'\}$. Hence, it follows that CT_{u,t^*} is independent of b for all such honest senders u. Therefore, the adversary's view contains information about b in only the challenge ciphertexts for $u \in \{s, s'\}$. Removing this information about bit b is non-trivial and requires an intricate sequence of hybrids as discussed below.

- From the description of the circuit $\mathsf{Gate}_{(\ell,g)}$ in Figure 4.1, it seems that the routing token contains no information of b. But, observe that the obfuscated gates treat filler and non-filler wires differently. More importantly, notice that its possible that a wire is an *inversion* wire when $b = 0$ and *filler* wire when $b = 1$ or vice versa. This is not the case for a *corrupt* wire or a *honest non-inversion* wire as $\mathsf{AssignRoutes}^*$ only assigns a single route for all senders $u \notin \{s, s'\}$. Hence, one of the goals of our sequence of hybrids is to arrive at the final hybrid in which the obfuscated gates will treat the *inversion* and *filler* wires exactly the same in the challenge round t^*.

Our sequence of hybrids is as follows.

Hybrid $\mathsf{Hyb}^{(b)}_0$**:** This is the real world experiment $\mathsf{NIARStatic\text{-}SelSingleCh\text{-}Inv}^{b,\mathcal{A}}$.

Hybrids $\mathsf{Hyb}^{(b)}_1, \mathsf{Hyb}^{(b)}_2, \mathsf{Hyb}^{(b)}_3$**:** The foremost change we make is to ensure that for all the corrupt wires, the authenticated routes that the obfuscated gates obtain are as intended by Setup^*. This change is accomplished in three steps.

- $\mathsf{Hyb}^{(b)}_1$: For each corrupt wire in each layer, puncture the route signing key such that it can only sign the expected route that passes through that wire. $\mathsf{Hyb}^{(b)}_1$ is identical to $\mathsf{Hyb}^{(b)}_0$ as none of the route signing keys are in the view of the adversary.

- $\mathsf{Hyb}^{(b)}_2$: For each corrupt wire in each layer, bind the route signing and verification key such that the verification key for a corrupt wire only accepts signature for the expected route that passes through that wire. Indistinguishability follows from indistinguishability of punctured and binding modes of SSU signatures.

- $\mathsf{Hyb}^{(b)}_3$: For each corrupt wire in each layer, hardwire the expected route and expected route signatures. Further, update the route authentication check to directly compare routes and route signatures against

hardcoded values. Indistinguishability follows from statistical unforgeability of the SSU signatures at the binding points and iO security.

Hybrids $\mathsf{Hyb}_4^{(b)}$: For all filler/inversion wires of layer $\ell = 1$, puncture the message signing keys such that they can sign any message for non-challenge round $t \neq t^*$ and only the challenge message for the challenge round $t = t^*$. Then, the adversary's view is identical as before (formal argument can be found in the transition from $\mathsf{Hyb}_3^{(b)}$ to $\mathsf{Hyb}_4^{(b)}$ and Claim 4.8.6).

Hybrids $\mathsf{Hyb}_5^{(b)}$: For all filler/inversion wires of layer $\ell = 1$, bind the message signing and verification keys such that the verification key accepts signatures for any message for non-challenge round $t \neq t^*$ and only the challenge message for the challenge round $t = t^*$. Indistinguishability follows from indistinguishability of punctured and binding modes of SSU signatures.

Layered hybrids. Now that we have shown how to change the message signing and verification keys for all filler/inversion wires in layer $\ell = 1$, we show how to do the same for rest of the layers. This is done in a layer-by-layer fashion. Below, we describe how to do it for layer $\ell = 2$ through a sequence of hybrids $\mathsf{Hyb}_{6,1,1}^{(b)}, \mathsf{Hyb}_{6,1,2}^{(b)}, \mathsf{Hyb}_{6,1,3}^{(b)}, \mathsf{Hyb}_{6,1,4}^{(b)}, \mathsf{Hyb}_{6,1,5}^{(b)}$. Once we have reached $\mathsf{Hyb}_{6,1,5}^{(b)}$, similiar arguments can be made for layer $\ell = 3$ via hybrids $\mathsf{Hyb}_{6,2,1}^{(b)}, \mathsf{Hyb}_{6,2,2}^{(b)}, \mathsf{Hyb}_{6,2,3}^{(b)}, \mathsf{Hyb}_{6,2,4}^{(b)}, \mathsf{Hyb}_{6,2,5}^{(b)}$. These layer-by-layer changes carry on for rest of the layers till we finally arrive at hybrid $\mathsf{Hyb}_{6,L-1,5}^{(b)}$.

Hybrids $\mathsf{Hyb}_{6,1,1}^{(b)}, \mathsf{Hyb}_{6,1,2}^{(b)}, \mathsf{Hyb}_{6,1,3}^{(b)}, \mathsf{Hyb}_{6,1,4}^{(b)}, \mathsf{Hyb}_{6,1,5}^{(b)}$: The goal of this set of hybrids is to reach a distribution where the message signing and verification keys for all filler/inversion wires of layer $\ell = 2$ are in binding mode. We show how to reach this distribution via a sequence of steps.

- $\mathsf{Hyb}_{6,1,1}^{(b)}$: For all filler/inversion wires in layer $\ell = 1$, hardcode the expected message signatures and messages. Further, update the message authentication check to directly compare messages and message signatures against hardcoded values. Route authentication check remains the same as before.
- $\mathsf{Hyb}_{6,1,2}^{(b)}$: For all filler/inversion wires in layer $\ell = 1$, hardcode the expected input ciphertext and puncture the PRF keys. Further, replace message and route authentication checks with ciphertext comparison.

  At this point, observe that for all filler/inversion wires in layer $\ell = 2$, the unpunctured message signing keys can not be used to sign messages other than the challenge messages for the challenge round t^*.

- $\mathsf{Hyb}^{(b)}_{6,1,3}$: For all filler/inversion wires of layer $\ell = 2$, puncture the message signing keys such that they can sign any message for non-challenge round $t \neq t^*$ and only the challenge message for the challenge round $t = t^*$. These punctured message signing keys are hardcoded in the gates in the first layer $\ell = 1$. Importantly, changing these message signing keys does not change the gate's functionality because as noted in the previous hybrid, the unpunctured signing keys for layer $\ell = 2$ can not be used to sign messages other than the challenge messages for the challenge round t^* either. Consequently, the security of indistinguishability obfuscation can be invoked to transition from $\mathsf{Hyb}^{(b)}_{6,1,2}$ to $\mathsf{Hyb}^{(b)}_{6,1,3}$ (formal proof in Claim 4.8.10).

- $\mathsf{Hyb}^{(b)}_{6,1,4}$: For all filler/inversion wires of layer $\ell = 2$, bind the message signing and verification keys such that the verification key accepts signatures for any message for non-challenge round $t \neq t^*$ and only the challenge message for the challenge round $t = t^*$. Indistinguishability follows from indistinguishability of punctured and binding modes of SSU signatures.

- $\mathsf{Hyb}^{(b)}_{6,1,5}$: For all filler/inversion wires of layer $\ell = 2$, hardcode the expected output ciphertext for the challenge round, and puncture the PRF keys. Further, update preparing the output ciphertext to directly use the hardcoded values.

Hybrid $\mathsf{Hyb}^{(b)}_{7}$: In this hybrid, we invoke the pseudorandomness of PRF at punctured points to change all the hardcoded ciphertexts to be uniformly random values except for a select few following wires. If an inversion wire in the last layer has the destination that is corrupt, then, we do not make any change to the hardwired outgoing ciphertexts in the last layer. Further, the challenge ciphertexts given out to the adversary for $u \in \{s, s'\}$ are set in a manner consistent with the hardcoded input ciphertexts in layer $\ell = 1$.

Hybrids $\mathsf{Hyb}^{(b)}_{8}, \mathsf{Hyb}^{(b)}_{9}$: In $\mathsf{Hyb}^{(b)}_{7}$, the only sources of information of challenge bit b are the hardwired message signing and verification keys for all filler/honest wires in all the circuits. So, in these hybrids, we unbind and unpuncture all the message signing and verification keys.

Analysis of the final hybrid: In hybrid $\mathsf{Hyb}^{(b)}_{9}$, we claim that everything can be simulated from the leakage function which is identical in both worlds. In other words, this hybrid contains no information about the challenge bit b.

- Observe that the challenge ciphertext for $u \in \{s, s'\}$ for the challenge round t^* obtained by the adversary are random strings independent of challenge bit b.

- Observe that for all corrupt wires, while the punctured route verification keys and expected routes are hardwired in the obfuscated circuits, they are the same across the two worlds $b = 0$ and $b = 1$. Further, punctured PRF keys that are hardwired contain no information about b. Most hardwired ciphertexts in the obfuscated gates are uniformly random values and the ones that are not random (i.e., inversion wires with corrupt receiver as destination) have the same value across the two worlds by the admissibility criteria. Put in other words, for the challenge round t^*, the circuit description (Figure 4.14) treats filler and inversion wires exactly the same.

Hence, $\mathsf{Hyb}_9^{(0)}$ and $\mathsf{Hyb}_9^{(1)}$ are identical. Formal arguments for this can be found in Claim 4.8.16.

Formal Hybrids

Hybrid $\mathsf{Hyb}_0^{(b)}$: In this hybrid, the challenger plays the game $\mathsf{NIARStatic\text{-}SelSingleCh\text{-}Inv}^{b,\mathcal{A}}(1^\lambda)$ with $\mathcal{A}$.

Hybrid $\mathsf{Hyb}_1^{(b)}$: This hybrid is identical to $\mathsf{Hyb}_0^{(b)}$ except that during the algorithm Setup^*, for all the *corrupt* wires, the challenger punctures the route signing keys at the corresponding routes and uses these punctured keys to generate the route signatures. More specifically, for each wire $(\ell, i = j_\ell)$, where $j_\ell \in \mathsf{rte}_u^*$ for some sender $u \in \mathcal{K}_S$, we compute the route signatures as follows:

$$(\mathsf{rsk}_{(\ell,i)}, \mathsf{rsk}'_{(\ell,i)}, \mathsf{rvk}_{(\ell,i)}, \mathsf{rpp}_{(\ell,i)}) \leftarrow \mathsf{Sig}.\mathbf{PuncturedSetup}(1^\lambda, \mathsf{tlen}, \mathsf{len}_{\mathsf{rte}}, 1, \mathsf{rte}_u^*) \ ,$$
$$\mathsf{rsig}_\ell^* = \mathsf{Sig.PSign}(\mathsf{rpp}_{(\ell,i)}, \mathsf{rsk}'_{(\ell,i)}, 1, \mathsf{rte}_u^*) \ .$$

Hybrid $\mathsf{Hyb}_2^{(b)}$: This hybrid is identical to $\mathsf{Hyb}_1^{(b)}$ except that during the algorithm Setup^*, the challenger generates binding route signature keys for all the *corrupt* wires. More specifically, for each wire $(\ell, i = j_\ell)$, where $j_\ell \in \mathsf{rte}_u^*$ for some sender $u \in \mathcal{K}_S$, we compute the route signatures as follows:

$$(\mathsf{rsk}^*_{(\ell,i)}, \mathsf{rvk}^*_{(\ell,i)}, \mathsf{rpp}^*_{(\ell,i)}) \leftarrow \mathsf{Sig}.\mathbf{BindingSetup}(1^\lambda, \mathsf{tlen}, \mathsf{len}_{\mathsf{rte}}, 1, \mathsf{rte}_u^*) \ ,$$
$$\mathsf{rsig}_\ell^* = \mathsf{Sig.PSign}(\mathsf{rpp}^*_{(\ell,i)}, \mathsf{rsk}^*_{(\ell,i)}, 1, \mathsf{rte}_u^*).$$

Then, we replace the gates $\mathsf{Gate}_{(\ell,g)}$ for all $\ell \in [L-1], g \in [G]$ as described in Figure 4.4.

Notation. Let $\mathsf{Input}_{(\ell,g)}$ and $\mathsf{Output}_{(\ell,g)}$ be the set of input and output wires of gate $\mathsf{Gate}_{(\ell,g)}$.
Hardcoded values. Same as in $\mathsf{Hyb}_0^{(b)}$ except that for each *corrupt* input wire $i \in \mathsf{Input}_{(\ell,g)}$, route public parameter $\mathsf{rpp}^*_{(\ell,i)}$ and route verification key $\mathsf{rvk}^*_{(\ell,i)}$ are hardcoded instead of $\mathsf{rpp}_{(\ell,i)}$ and $\mathsf{rvk}_{(\ell,i)}$.
Procedure. Same as in $\mathsf{Hyb}_0^{(b)}$.

Figure 4.4: The circuit $\mathsf{Gate}_{(\ell,g)}$ in hybrid experiment $\mathsf{Hyb}_2^{(b)}$.

Notation. Let $\mathsf{Input}_{(\ell,g)}$ and $\mathsf{Output}_{(\ell,g)}$ be the set of input and output wires of gate $\mathsf{Gate}_{(\ell,g)}$.

Hardcoded values. Same as in $\mathsf{Hyb}_2^{(b)}$. Additionally, for each *corrupt* input wire $i \in \mathsf{Input}_{(\ell,g)}$, suppose it is on expected route $\overline{\mathsf{rte}}_u^* = (\mathsf{rte}_u^* = (j_1^*, \ldots, j_L^*), \mathsf{rsig}_u^* = (\mathsf{rsig}_1^*, \ldots, \mathsf{rsig}_L^*))$ for some sender $u \in [n]$. Then, hardcode $(i, \overline{\mathsf{rte}}_u^*)$.

Procedure.

- Step 1: For each input wire $i \in \mathsf{Input}_{(\ell,g)}$, if it is a *filler/honest* wire, then, compute as in $\mathsf{Hyb}_2^{(b)}$. Else:
  - Step (a) is same as in $\mathsf{Hyb}_2^{(b)}$.
  - Step (b): **Decrypt and authenticate the message/route:**
    - Steps i, ii, iii are same as in $\mathsf{Hyb}_2^{(b)}$.
    - Step iv: Parse $\overline{\mathsf{rte}}_u$ as $(\mathsf{rte}_u = (j_1, \ldots, j_L), \mathsf{rsig}_u = (\mathsf{rsig}_1, \ldots, \mathsf{rsig}_L))$ and perform the following checks to authenticate the route rte_u: If $\mathsf{rte}_u \neq \mathsf{rte}_u^*$, abort. If $\mathsf{rsig}_\ell \neq \mathsf{rsig}_\ell^*$, abort.
  - Step (c): **Prepare the output ciphertext** $\mathsf{CT}_{(\ell+1,j_{\ell+1})}$**:**
    - Step i: Let $j = j_{\ell+1}^*$. If $\mathsf{CT}_{(\ell+1,j)}$ has already been computed, then abort.
    - Step ii: If $\ell < L-1$, compute $\mathsf{msig}' = \mathsf{Sig.Sign}(\mathsf{mpp}_{(\ell+1,j)}, \mathsf{msk}_{(\ell+1,j)}, t, (x, \overline{\mathsf{rte}}_u^*))$ and $\mathsf{CT}_{(\ell+1,j)} = (x, \overline{\mathsf{rte}}_u^*, \mathsf{msig}') \oplus \mathsf{PRF.}\mathbf{Eval}(k_{(\ell+1,j)}, t)$.
    - Step iii is same as in $\mathsf{Hyb}_2^{(b)}$.
- Steps 2 and 3 are same as in $\mathsf{Hyb}_2^{(b)}$.

Figure 4.5: The circuit $\mathsf{Gate}_{(\ell,g)}$ in hybrid experiment $\mathsf{Hyb}_3^{(b)}$.

Hybrid $\mathsf{Hyb}_3^{(b)}$: In this hybrid, for the obfuscated gates generated by Setup^*, for each *corrupt* wire, we perform the route authentication checks by comparing with hardcoded routes and corresponding signatures. Specifically, in each gate we hardcode the relevant routes sampled by Setup^* along with the route signature as computed in hybrid $\mathsf{Hyb}_2^{(b)}$. Then, we replace the gates $\mathsf{Gate}_{(\ell,g)}$ for all $\ell \in [L-1], g \in [G]$ as described in Figure 4.5.

Hybrid $\mathsf{Hyb}_4^{(b)}$: This hybrid is identical to $\mathsf{Hyb}_3^{(b)}$, except that during Setup^*, the challenger punctures the message signing keys for all *filler/inversion* wires $(1,i)$ in the first layer at the challenge round t^* and challenge plaintext $\tilde{x}^* = (x_{u,t^*}^{(b)}, \overline{\mathsf{rte}}_u^{(b)})$ (or $\tilde{x}^* = (\bot_{\mathsf{filler}}, \bot_{\mathsf{filler}})$ in case of *filler*), that is, for each such wire $(1,i)$,

$$(\mathsf{msk}_{(1,i)}, \mathsf{msk}'_{(1,i)}, \mathsf{mvk}_{(1,i)}, \mathsf{mpp}_{(1,i)}) \leftarrow \mathsf{Sig.}\mathbf{PuncturedSetup}(1^\lambda, \mathsf{tlen}, \mathsf{len}_m, t^*, \tilde{x}^*).$$

Then, during Enc algorithm, whenever the challenger has to compute message signatures for the first layer, it computes them as follows:

$$\mathsf{msig}_{(1,i)} = \mathsf{Sig.PSign}(\mathsf{mpp}_{(1,i)}, \mathsf{msk}'_{(1,i)}, \cdot, \cdot).$$

Hybrid $\mathsf{Hyb}_5^{(b)}$**:** This hybrid is identical to $\mathsf{Hyb}_4^{(b)}$, except that during Setup^*, the challenger binds message signature keys for all *filler/inversion* wires $(1,i)$ in the first layer at the challenge round t^* and challenge plaintext $\tilde{x}^* = (x_{u,t^*}^{(b)}, \overline{\mathsf{rte}}_u^{(b)})$ (or $\tilde{x}^* = (\bot_{\mathsf{filler}}, \bot_{\mathsf{filler}})$ in case of *filler*), that is, for each such wire $(1,i)$,

$$(\mathsf{msk}^*_{(1,i)}, \mathsf{mvk}^*_{(1,i)}, \mathsf{mpp}^*_{(1,i)}) \leftarrow \mathsf{Sig.}\mathbf{BindingSetup}(1^\lambda, \mathsf{tlen}, \mathsf{len}_m, t^*, \tilde{x}^*).$$

Then, the challenger replaces the gates $\mathsf{Gate}_{(1,g)}$ for all $g \in [G]$ as described in Figure 4.6.

Notation. Let $\mathsf{Input}_{(1,g)}$ and $\mathsf{Output}_{(1,g)}$ be the set of input and output wires of gate $\mathsf{Gate}_{(1,g)}$.
Hardcoded values. Same as in $\mathsf{Hyb}_3^{(b)}$ except that for each *filler/inversion* wire $i \in \mathsf{Input}_{(1,g)}$, the message public parameter $\mathsf{mpp}^*_{(1,i)}$ and the message verification key $\mathsf{mvk}^*_{(1,i)}$ are hardcoded instead of $\mathsf{mpp}_{(1,i)}, \mathsf{mvk}_{(1,i)}$.
Procedure. Same as in $\mathsf{Hyb}_3^{(b)}$.

Figure 4.6: The circuit $\mathsf{Gate}_{(\ell,g)}$ in hybrid experiment $\mathsf{Hyb}_5^{(b)}$.

Hybrid $\mathsf{Hyb}_{6,\ell,1}^{(b)}$ **for each** $\ell \in [L-1]$**:** This hybrid is identical to $\mathsf{Hyb}_{6,\ell-1,5}^{(b)}$ (or $\mathsf{Hyb}_5^{(b)}$ if $\ell = 1$), except that during Setup^*, for all the gates in layer ℓ, i.e., $\mathsf{Gate}_{(\ell,g)}$ for all $g \in [G]$, for all *filler/inversion* wires (ℓ,i) in the input layer ℓ, the challenger hardcodes the expected challenge message $x^*_{(\ell,i)} = x^{(b)}_{u,t^*}$ (or $x^*_{(\ell,ß)} = \bot_{\mathsf{filler}}$ in case of *filler* wire), the expected route $\overline{\mathsf{rte}}^* = \overline{\mathsf{rte}}^*_u$ (or $\overline{\mathsf{rte}}^* = \bot_{\mathsf{filler}}$ in case of *filler* wire) and the message signature $\mathsf{msig}^*_{(\ell,i)} = \mathsf{Sig.PSign}(\mathsf{mpp}^*_{(\ell,i)}, \mathsf{msk}^*_{(\ell,i)}, t^*, (x^*, \overline{\mathsf{rte}}^*))$ for the challenge round t^* and compares the message, route and message signature in the decrypted plaintext against the respective hardcoded challenge message, expected route and message signature instead of checking via $\mathsf{Sig.Vf}$. Subsequently, for the outgoing wires, it uses the hardcoded $x^*_{(\ell,i)}$ and $\overline{\mathsf{rte}}^*$ for computing the outgoing ciphertexts in layer $\ell+1$. More formally, for all $g \in [G]$, the gates $\mathsf{Gate}_{(\ell,g)}$ are changed as described in Figure 4.7.

Notation. Let $\mathsf{Input}_{(\ell,g)}$ and $\mathsf{Output}_{(\ell,g)}$ be the set of input and output wires of gate $\mathsf{Gate}_{(\ell,g)}$.

Hardcoded values. Same as in $\mathsf{Hyb}^{(b)}_{6,\ell-1,5}$ (or $\mathsf{Hyb}^{(b)}_{5}$ if $\ell = 1$). Additionally, round t^* is hardcoded and for each *filler/inversion* input wire $i \in \mathsf{Input}_{(\ell,g)}$, the expected challenge message $x^*_{(\ell,i)}$, expected route $\overline{\mathsf{rte}}^*$ and signature $\mathsf{msig}^*_{(\ell,i)}$ are hardcoded.

Procedure. $\mathsf{Gate}_{(\ell,g)}$ takes as input a round t and a set of ciphertexts $\{\mathsf{CT}_{(\ell,i)} : i \in \mathsf{Input}_{(\ell,g)}\}$ corresponding to the input wires. Depending on the layer ℓ, it computes as follows.

- Step 1: For each input wire $i \in \mathsf{Input}_{(\ell,g)}$, if $t \neq t^*$ or wire i is *corrupt* or is a *non-inversion honest wire*, then, compute as in $\mathsf{Hyb}^{(b)}_{6,\ell-1,5}$ (or $\mathsf{Hyb}^{(b)}_{5}$ if $\ell = 1$). Else:
  - Step (a) is same as in $\mathsf{Hyb}^{(b)}_{6,\ell-1,5}$ (or $\mathsf{Hyb}^{(b)}_{5}$ if $\ell = 1$).
  - Step (b): **Decrypt and authenticate the message/route:**
    - Step i: compute the plaintext $(x, \overline{\mathsf{rte}}, \mathsf{msig}) = \mathsf{CT}_{(\ell,i)} \oplus \mathsf{PRF}.\mathbf{Eval}(k_{(\ell,i)}, t^*)$.
    - Step ii: If $x \neq x^*_{(\ell,i)}$ or $\overline{\mathsf{rte}} \neq \overline{\mathsf{rte}}^*$ or $\mathsf{msig} \neq \mathsf{msig}^*_{(\ell,i)}$, then abort.
    - Steps iii and iv are same as in $\mathsf{Hyb}^{(b)}_{6,\ell-1,5}$ (or $\mathsf{Hyb}^{(b)}_{5}$ if $\ell = 1$).
  - Step (c): **Prepare the output ciphertext** $\mathsf{CT}_{(\ell+1,j_{\ell+1})}$**:**
    - Step i is same as in $\mathsf{Hyb}^{(b)}_{6,\ell-1,5}$ (or $\mathsf{Hyb}^{(b)}_{5}$ if $\ell = 1$).
    - Step ii: If $\ell < L-1$, compute $\mathsf{msig}' = \mathsf{Sig.Sign}(\mathsf{mpp}_{(\ell+1,j)}, \mathsf{msk}_{(\ell+1,j)}, t^*, (x^*_{(\ell,i)}, \overline{\mathsf{rte}}^*))$ and $\mathsf{CT}^*_{(\ell+1,j)} = (x^*_{(\ell,i)}, \overline{\mathsf{rte}}^*, \mathsf{msig}') \oplus \mathsf{PRF}.\mathbf{Eval}(k_{(\ell+1,j)}, t)$.
    - Step iii: If $\ell = L-1$, compute $\mathsf{CT}^*_{(L,j)} = (x^*_{(\ell,i)}, \bot, \bot) \oplus \mathsf{PRF}.\mathbf{Eval}(k_{(L,j)}, t)$.
- Step 2: For each $j \in \mathsf{Output}_{(\ell,g)}$ such that $\mathsf{CT}_{(\ell+1,j)}$ has not been computed yet, compute filler ciphertexts as in $\mathsf{Hyb}^{(b)}_{6,\ell-1,5}$ (or $\mathsf{Hyb}^{(b)}_{5}$ if $\ell = 1$) if $t \neq t^*$. Else:
  - Step (a): Set $x = \bot_{\mathsf{filler}}$ and $\overline{\mathsf{rte}} = \bot_{\mathsf{filler}}$.
  - Steps (b) and (c) are same as in $\mathsf{Hyb}^{(b)}_{6,\ell-1,5}$ (or $\mathsf{Hyb}^{(b)}_{5}$ if $\ell = 1$).
- Step 3 is same as in $\mathsf{Hyb}^{(b)}_{6,\ell-1,5}$ (or $\mathsf{Hyb}^{(b)}_{5}$ if $\ell = 1$).

Figure 4.7: The circuit $\mathsf{Gate}_{(\ell,g)}$ in hybrid experiment $\mathsf{Hyb}^{(b)}_{6,\ell,1}$.

Hybrid $\mathsf{Hyb}^{(b)}_{6,\ell,2}$ for each $\ell \in [L-1]$: This hybrid is identical to $\mathsf{Hyb}^{(b)}_{6,\ell,1}$, except that during Setup^*, when generating all circuits for layer ℓ, the challenger punctures the hardcoded decryption keys and hardcodes the expected input ciphertexts and corresponding plaintexts for all *filler/inversion* input wires for the challenge round t^*. In other words, for each *filler/inversion* input wire (ℓ, i) that is on the route $\overline{\mathsf{rte}}^*$, do the following.

- The challenger hardcodes the expected input ciphertext $\overline{\mathsf{CT}}^*_{(\ell,i)} = (x^*_{(\ell,i)}, \overline{\mathsf{rte}}^*, \mathsf{msig}^*_{(\ell,i)}) \oplus \mathsf{PRF}.\mathbf{Eval}(k_{(\ell,i)}, t^*)$ and compares the input ciphertext with hardcoded ciphertext instead of decrypting and performing subsequent checks.

- In addition, the challenger hardcodes the punctured PRF key $k_{(\ell,i)}$ at challenge round t^*: $k^*_{(\ell,i)} \leftarrow \mathsf{PRF}.\mathbf{Puncture}(k_{(\ell,i)}, t^*)$.

Formally, the behavior of gate $\mathsf{Gate}_{(\ell,g)}$ for all $g \in [G]$ as described in Figure 4.8.

Notation. Let $\mathsf{Input}_{(\ell,g)}$ and $\mathsf{Output}_{(\ell,g)}$ be the set of input and output wires of gate $\mathsf{Gate}_{(\ell,g)}$.
Hardcoded values. Same as in $\mathsf{Hyb}^{(b)}_{6,\ell,1}$, except that for each *filler/inversion* wire $i \in \mathsf{Input}_{(\ell,g)}$, $\overline{\mathsf{CT}}^*_{(\ell,i)}$ is hardcoded along with the corresponding challenge message $x_{(\ell,i)}$ and route $\overline{\mathsf{rte}}^*$, and $\mathsf{msig}^*_{(\ell,i)}$ is not hardcoded anymore. In addition, the punctured PRF key $k^*_{(\ell,i)}$ is hardcoded instead of $k_{(\ell,i)}$, and is used for decrypting on wire (ℓ, i) during all rounds $t \neq t^*$.
Procedure. $\mathsf{Gate}_{(\ell,g)}$ takes as input a round t and a set of ciphertexts $\{\mathsf{CT}_{(\ell,i)} : i \in \mathsf{Input}_{(\ell,g)}\}$ corresponding to the input wires. Depending on the layer ℓ, it computes as follows.

- Step 1: For each input wire $i \in \mathsf{Input}_{(\ell,g)}$, if $t \neq t^*$ or wire i is *corrupt* or is a *non-inversion honest wire*, then, compute as in $\mathsf{Hyb}^{(b)}_{6,\ell,1}$. Else:
  - Steps (a) and (c) are same as in $\mathsf{Hyb}^{(b)}_{6,\ell,1}$.
  - Step (b): if $\mathsf{CT}^*_{(\ell,i)} \neq \overline{\mathsf{CT}}^*_{(\ell,i)}$, then abort. If wire i is filler, go to the next i.
- Steps 2 and 3 are same as in $\mathsf{Hyb}^{(b)}_{6,\ell,1}$.

Figure 4.8: The circuit $\mathsf{Gate}_{(\ell,g)}$ in hybrid experiment $\mathsf{Hyb}^{(b)}_{6,\ell,2}$.

Hybrid $\mathsf{Hyb}^{(b)}_{6,\ell,3}$ for each $\ell \in [L-1]$: This hybrid is identical to $\mathsf{Hyb}^{(b)}_{6,\ell,2}$, except that during Setup^*, the challenger punctures the message signing keys for all *filler/inversion* wires $(\ell+1, i)$ on the path $\overline{\mathsf{rte}}^* = \overline{\mathsf{rte}}^*_u$ (or $\overline{\mathsf{rte}}^* = \perp_{\mathsf{filler}}$ in case of *filler* wire) at the challenge round t^* and challenge message $x^*_{(\ell+1,i)} = x^{(b)}_{u,t^*}$ (or $x^*_{(\ell+1,i)} = \perp_{\mathsf{filler}}$ in case of *filler* wire), that is, for each such wire $(\ell+1, i)$,

$$(\mathsf{msk}_{(\ell+1,i)}, \mathsf{msk}'_{(\ell+1,i)}, \mathsf{mvk}_{(\ell+1,i)}, \mathsf{mpp}_{(\ell+1,i)}) \leftarrow \mathsf{Sig}.\mathbf{PuncturedSetup}(1^\lambda, \mathsf{tlen}, \mathsf{len}_m, t^*, (x^*_{(\ell+1,i)}, \overline{\mathsf{rte}}^*_u)).$$

Consequently, the associated message signatures in the gates will be computed using $\mathsf{Sig.PSign}$. Formally, the behavior of gate $\mathsf{Gate}_{(\ell,g)}$ for all $g \in [G]$ is changed as described in Figure 4.9.

Hybrid $\mathsf{Hyb}^{(b)}_{6,\ell,4}$ for each $\ell \in [L-1]$: This hybrid is identical to $\mathsf{Hyb}^{(b)}_{6,\ell,3}$, except that during Setup^*, the challenger uses the binding setup to bind message signature keys for all *filler/inversion* wires $(\ell+1, i)$ on the path $\overline{\mathsf{rte}}^*$ at the challenge round t^* and challenge message $x^*_{(\ell,i)} = x^{(b)}_{u,t^*}$ (or $x^*_{(\ell,i)} = \perp_{\mathsf{filler}}$ in case of *filler* wire), that is, for each such wire $(1, i)$,

$$(\mathsf{msk}^*_{(\ell+1,i)}, \mathsf{mvk}^*_{(\ell+1,i)}, \mathsf{mpp}^*_{(\ell+1,i)}) \leftarrow \mathsf{Sig}.\mathbf{BindingSetup}(1^\lambda, \mathsf{tlen}, \mathsf{len}_m, t^*, (x^*_{(\ell,i)}, \overline{\mathsf{rte}}^*)).$$

Then, the challenger replaces the gates $\mathsf{Gate}_{(\ell,g)}$ and $\mathsf{Gate}_{(\ell+1,g)}$ for all $g \in [G]$ as described in Figure 4.10 and Figure 4.11.

Notation. Let $\mathsf{Input}_{(\ell,g)}$ and $\mathsf{Output}_{(\ell,g)}$ be the set of input and output wires of gate $\mathsf{Gate}_{(\ell,g)}$.
Hardcoded values. Same as in $\mathsf{Hyb}^{(b)}_{6,\ell,2}$, except that for each *filler/inversion* wire $i \in \mathsf{Output}_{(\ell,g)}$ in the output layer $\ell+1$, the punctured message signing key $\mathsf{msk}'_{(\ell+1,i)}$ is hardcoded instead of $\mathsf{msk}_{(\ell+1,i)}$.
Procedure.

- Step 1: For each input wire $i \in \mathsf{Input}_{(\ell,g)}$, if $t \neq t^*$ or wire i is *corrupt* or is a *non-inversion honest wire*, compute as in $\mathsf{Hyb}^{(b)}_{6,\ell,2}$. Else:
  - Steps (a), (b) are same as in $\mathsf{Hyb}^{(b)}_{6,\ell,2}$.
  - Step (c): **Prepare the output ciphertext** $\mathsf{CT}_{(\ell+1,j_{\ell+1})}$**:**
    - Steps i and iii are same as in $\mathsf{Hyb}^{(b)}_{6,\ell,2}$.
    - Step ii: If $\ell < L-1$, compute $\mathsf{msig}' = \mathsf{Sig.PSign}(\mathsf{mpp}_{(\ell+1,j)}, \mathsf{msk}'_{(\ell+1,j)}, t^*, (x^{(b)}_{u,t^*}, \overline{\mathsf{rte}}^*_u))$ and $\mathsf{CT}^*_{(\ell+1,j)} = (x^{(b)}_{u,t^*}, \overline{\mathsf{rte}}^*_u, \mathsf{msig}') \oplus \mathsf{PRF}.\mathbf{Eval}(k_{(\ell+1,j)}, t)$.
- Step 2: For each $j \in \mathsf{Output}_{(\ell,g)}$ such that $\mathsf{CT}_{(\ell+1,j)}$ has not been computed yet, compute filler ciphertexts as in $\mathsf{Hyb}^{(b)}_{6,\ell,2}$ if $t \neq t^*$. Else:
  - Step (a): Set $x = \bot_{\mathsf{filler}}$ and $\overline{\mathsf{rte}} = \bot_{\mathsf{filler}}$.
  - Step (b): Compute $\mathsf{msig}' = \mathsf{Sig.PSign}(\mathsf{mpp}_{(\ell+1,j)}, \mathsf{msk}'_{(\ell+1,j)}, t^*, (x, \overline{\mathsf{rte}}))$.
  - Step (c): Compute $\mathsf{CT}^*_{(\ell+1,j)} \leftarrow (x, \overline{\mathsf{rte}}, \mathsf{msig}') \oplus \mathsf{PRF}.\mathbf{Eval}(k_{(\ell+1,j)}, t^*)$.
- Step 3 is same as in $\mathsf{Hyb}^{(b)}_{6,\ell,2}$.

Figure 4.9: The circuit $\mathsf{Gate}_{(\ell,g)}$ in hybrid experiment $\mathsf{Hyb}^{(b)}_{6,\ell,3}$.

Notation. Let $\mathsf{Input}_{(\ell,g)}$ and $\mathsf{Output}_{(\ell,g)}$ be the set of input and output wires of gate $\mathsf{Gate}_{(\ell,g)}$.
Hardcoded values. Same as in $\mathsf{Hyb}^{(b)}_{6,\ell,3}$ except that for each *filler/inversion* wire $i \in \mathsf{Output}_{(\ell,g)}$, message public parameter $\mathsf{mpp}^*_{(\ell+1,i)}$ and message signing key $\mathsf{msk}^*_{(\ell+1,i)}$ are hardcoded instead of $\mathsf{mpp}_{(\ell+1,i)}, \mathsf{msk}'_{(\ell+1,i)}$.
Procedure. Same as in $\mathsf{Hyb}^{(b)}_{6,\ell,3}$.

Figure 4.10: The circuit $\mathsf{Gate}_{(\ell,g)}$ in hybrid experiment $\mathsf{Hyb}^{(b)}_{6,\ell,4}$.

Notation. Let $\mathsf{Input}_{(\ell+1,g)}$ and $\mathsf{Output}_{(\ell+1,g)}$ be the set of input and output wires of gate $\mathsf{Gate}_{(\ell+1,g)}$.
Hardcoded values. Same as in $\mathsf{Hyb}^{(b)}_{6,\ell,3}$ except that for each *filler/inversion* wire $i \in \mathsf{Input}_{(\ell+1,g)}$, message public parameter $\mathsf{mpp}^*_{(\ell+1,i)}$ and message verification key $\mathsf{mvk}^*_{(\ell+1,i)}$ are hardcoded instead of $\mathsf{mpp}_{(\ell+1,i)}, \mathsf{mvk}_{(\ell+1,i)}$.
Procedure. Same as in $\mathsf{Hyb}^{(b)}_{6,\ell,3}$.

Figure 4.11: The circuit $\mathsf{Gate}_{(\ell+1,g)}$ in hybrid experiment $\mathsf{Hyb}^{(b)}_{6,\ell,4}$.

Hybrid $\mathsf{Hyb}^{(b)}_{6,\ell,5}$ for each $\ell \in [L-1]$: This hybrid is identical to $\mathsf{Hyb}^{(b)}_{6,\ell,4}$, except that during Setup^*, when generating all circuits for layer ℓ, for all *filler/inversion* wires $(\ell+1,i)$ on the path $\overline{\mathsf{rte}}^*$ the challenger hardcodes the expected output ciphertexts for the challenge round t^*: $\overline{\mathsf{CT}}^*_{(\ell+1,i)} = (x^{(b)}_{u,t^*}, \overline{\mathsf{rte}}^*_u, \mathsf{msig}^*_{(\ell+1,i)}) \oplus \mathsf{PRF}.\mathbf{Eval}(k_{(\ell+1,i)}, t^*)$ if $\ell < L-1$, else $\overline{\mathsf{CT}}^*_{(L,i)} = (x^{(b)}_{u,t^*}, \bot, \bot) \oplus \mathsf{PRF}.\mathbf{Eval}(k_{(L,i)}, t^*)$. Then, instead of dynamically computing the output ciphertext inside the gate, the challenger simply uses the hardcoded ciphertext. In addition, the challenger hardcodes punctured keys $k^*_{(\ell+1,i)} \leftarrow \mathsf{PRF}.\mathbf{Puncture}(k_{(\ell,i)}, t^*)$ for the output wires into $\mathsf{Gate}_{(\ell,g)}$.

At this point, since both inversion and filler wires are dealt with by comparing the inputs to fixed ciphertexts, and outputting fixed ciphertexts, the gate does not need to know which wires are inversion wires and which are fillers during round t^*. We change the flow of the program to reflect this.

Formally, the behavior of gate $\mathsf{Gate}_{(\ell,g)}$ for all $g \in [G]$ is as described in Figure 4.12.

Notation. Let $\mathsf{Input}_{(\ell,g)}$ and $\mathsf{Output}_{(\ell,g)}$ be the set of input and output wires of gate $\mathsf{Gate}_{(\ell,g)}$.
Hardcoded values. Same as $\mathsf{Hyb}^{(b)}_{6,\ell,4}$ except that for each *filler/inversion* wire $i \in \mathsf{Output}_{(\ell,i)}$ in the output layer $\ell+1$, the punctured PRF key $k^*_{(\ell+1,i)}$ is hardcoded instead of $k_{(\ell+1,i)}$.
Procedure. $\mathsf{Gate}_{(\ell,g)}$ takes as input a round t and a set of ciphertexts $\{\mathsf{CT}_{(\ell,i)} : i \in \mathsf{Input}_{(\ell,g)}\}$ corresponding to the input wires. Depending on the layer ℓ, it computes as follows.

1. Step 1: For each input wire $i \in \mathsf{Input}_{(\ell,g)}$, if $t \neq t^*$ or wire i is *corrupt* or is a *non-inversion honest wire*, compute as in $\mathsf{Hyb}^{(b)}_{6,\ell,5}$. Else:
   a) If $\ell = 1$ and i is even, continue to next i. // *It is a filler element and is ignored.*
   b) **Authenticate the message/route:** If $\mathsf{CT}^*_{(\ell,i)} \neq \overline{\mathsf{CT}}^*_{(\ell,i)}$, then abort.
2. For each $j \in \mathsf{Output}_{(\ell,g)}$ such that $\mathsf{CT}_{(\ell+1,j)}$ has not been computed yet, compute filler ciphertexts as in $\mathsf{Hyb}^{(b)}_{6,\ell,5}$ if $t \neq t^*$. Else compute *filler/inversion* ciphertexts as $\mathsf{CT}^*_{(\ell+1,j)} = \overline{\mathsf{CT}}^*_{(\ell+1,j)}$.
3. Output $\{\mathsf{CT}_{(\ell+1,i)} : i \in \mathsf{Output}_{(\ell,g)}\}$.

Figure 4.12: The circuit $\mathsf{Gate}_{(\ell,g)}$ in hybrid experiment $\mathsf{Hyb}^{(b)}_{6,\ell,5}$.

Hybrid $\mathsf{Hyb}^{(b)}_7$: This hybrid is same as $\mathsf{Hyb}^{(b)}_{6,L-1,5}$ except that during Setup^*, the challenger changes the hardcoded ciphertexts as follows: In gates $\mathsf{Gate}_{(L-1,g)}$ for all $g \in [G]$, for all *inversion* output wires $i \in \mathsf{Output}_{(L-1,g)}$, if the corresponding receiver is *corrupt*, then, do not change anything. Else, if it is an *inversion* output wire whose corresponding receiver is *honest*, or if it is *filler* wire $i \in \mathsf{Output}_{(L-1,g)}$, or if it *filler/inversion* wire i in any $\mathsf{Gate}_{(\ell,g)}$ for any $\ell < L-1$ and for any $g \in [G]$, then, change the hardcoded ciphertext to be a random string of appropriate length. Further, the challenge ciphertexts given out to the adversary for $u \in \{s, s'\}$ are set in a manner consistent with the hardcoded input ciphertexts in layer $\ell = 1$.

Hybrid $\mathsf{Hyb}_8^{(b)}$**:** This hybrid is same as $\mathsf{Hyb}_7^{(b)}$ except that during Setup^*, the challenger unbinds all the message signature tuples that were previously binding: for all the message signature tuples for all *filler/inversion* wires (ℓ, i) on the path $\overline{\mathsf{rte}}^*$ in all the layers, the challenger uses punctured setup instead of binding setup at the challenge round t^* and challenge message $x^*_{(L,i)} = x^{(b)}_{u,t^*}$ (or $x^*_{(L,i)} = \perp_{\mathsf{filler}}$ in case of *filler* wire), that is, for each such wire (ℓ, i),

$$(\mathsf{msk}_{(\ell,i)}, \mathsf{msk}'_{(\ell,i)}, \mathsf{mvk}_{(\ell,i)}, \mathsf{mpp}_{(\ell,i)}) \leftarrow \mathsf{Sig}.\mathbf{PuncturedSetup}(1^\lambda, \mathsf{tlen}, \mathsf{len}_m, t^*, (x^*_{(L,i)}, \overline{\mathsf{rte}}^*)).$$

Hybrid $\mathsf{Hyb}_9^{(b)}$**:** This hybrid is same as $\mathsf{Hyb}_8^{(b)}$ except that during Setup^*, the challenger unpunctures all the message signing keys that were previously punctured: for all the message signature key pairs for all *filler/inversion* wires (ℓ, i) on the path $\overline{\mathsf{rte}}^*$ in all the layers, the challenger uses setup algorithm and does not puncture anymore at the challenge round t^* and challenge message $x^*_{(L,i)}$, that is, for each such wire (ℓ, i),

$$(\mathsf{msk}_{(\ell,i)}, \mathsf{mvk}_{(\ell,i)}, \mathsf{mpp}_{(\ell,i)}) \leftarrow \mathsf{Sig.Setup}(1^\lambda, \mathsf{tlen}, \mathsf{len}_m).$$

Then, whenever the challenger has to compute a message signature during Enc algorithm or inside a gate, it computes as: $\mathsf{msig}_{(\ell,i)} = \mathsf{Sig.Sign}(\mathsf{mpp}_{(\ell,i)}, \mathsf{msk}_{(\ell,i)}, \cdot, \cdot)$.

To summarize, at this point the circuit $\mathsf{Gate}_{(\ell,g)}$ for all $\ell \in [L-1]$ and $g \in [G]$ is as in Figure 4.13 and Figure 4.14.

Notation. Let $\mathsf{Input}_{(\ell,g)}$ and $\mathsf{Output}_{(\ell,g)}$ be the set of input and output wires of gate $\mathsf{Gate}_{(\ell,g)}$.
Hardcoded values. $\mathsf{Gate}_{(\ell,g)}$ has hardcoded the following values:

- For each wire $i \in \mathsf{Input}_{(\ell,g)}$ in layer ℓ:
  - if *corrupt*: the PRF key $k_{(\ell,i)}$, the message public parameter $\mathsf{mpp}_{(\ell,i)}$, the message verification key $\mathsf{mvk}_{(\ell,i)}$, the route public parameter $\mathsf{rpp}^*_{(\ell,i)}$, the route verification key $\mathsf{rvk}^*_{(\ell,i)}$, the expected route $\overline{\mathsf{rte}}^*_u$.
  - if *filler/inversion*: the PRF key $k^*_{(\ell,i)}$, the message public parameter $\mathsf{mpp}_{(\ell,i)}$, the message verification key $\mathsf{mvk}_{(\ell,i)}$, the route public parameter $\mathsf{rpp}_{(\ell,i)}$, the route verification key $\mathsf{rvk}_{(\ell,i)}$, the expected challenge ciphertext $\overline{\mathsf{CT}}^*_{(\ell,i)}$.
- For each wire $i \in \mathsf{Output}_{(\ell,g)}$ in layer $\ell + 1$:
  - if *corrupt*: the PRF key $k_{(\ell+1,i)}$, the message public parameter $\mathsf{mpp}_{(\ell+1,i)}$. the message signing key $\mathsf{msk}_{(\ell+1,i)}$.
  - if *filler/inversion*: the PRF key $k^*_{(\ell+1,i)}$, the message public parameter $\mathsf{mpp}_{(\ell+1,i)}$, the message signing key $\mathsf{msk}_{(\ell+1,i)}$, the expected challenge ciphertext $\overline{\mathsf{CT}}^*_{(\ell+1,i)}$.
- The challenge round t^*.

Figure 4.13: The circuit $\mathsf{Gate}_{(\ell,g)}$ in hybrid experiment $\mathsf{Hyb}_9^{(b)}$: notation and hardcoded values.

Procedure. $\mathsf{Gate}_{(\ell,g)}$ takes as input a round t and a set of ciphertexts $\{\mathsf{CT}_{(\ell,i)} : i \in \mathsf{Input}_{(\ell,g)}\}$ corresponding to the input wires. Depending on the layer ℓ, it computes as follows.

1. For each input wire $i \in \mathsf{Input}_{(\ell,g)}$:
   a) If $\ell = 1$ and i is even, continue to next i. // *It is a filler wire and is ignored.*
   b) **Decrypt and authenticate the message/route:**
   If it is a *filler/inversion* wire and $t = t^*$: If $\mathsf{CT}^*_{(\ell,i)} \neq \overline{\mathsf{CT}}^*_{(\ell,i)}$, then abort.
   If it is a (*corrupt* wire or is a *non-inversion honest wire*) or (*any* wire and $t \neq t^*$):
      i. Compute the plaintext $(x, \overline{\mathsf{rte}}, \mathsf{msig}) = \mathsf{CT}_{(\ell,i)} \oplus \mathsf{PRF}.\mathbf{Eval}(k_{(\ell,i)}, t)$.
      ii. If msig is not a valid signature of $(x, \overline{\mathsf{rte}})$ w.r.t. $\mathsf{mpp}_{(\ell,i)}, \mathsf{mvk}_{(\ell,i)}$ and round t, then abort.
      iii. If $x = \perp_{\mathsf{filler}}$ and $\overline{\mathsf{rte}} = \perp_{\mathsf{filler}}$, go to the next i. // *It is a filler wire and is ignored.*
      iv. Parse $\overline{\mathsf{rte}}$ as $(\mathsf{rte} = (j_1, \ldots, j_L), \mathsf{rsig} = (\mathsf{rsig}_1, \ldots, \mathsf{rsig}_L))$ and perform the following checks to authenticate the route $\overline{\mathsf{rte}}$:
      If it is a *corrupt* wire: If $\mathsf{rte} \neq \mathsf{rte}^*_u$, abort. If $\mathsf{rsig}_\ell \neq \mathsf{rsig}^*_\ell$, abort.
      If it is an *honest* wire and $t \neq t^*$: Parse $\overline{\mathsf{rte}}$ as $(\mathsf{rte} = (j_1, \ldots, j_L), \mathsf{rsig} = (\mathsf{rsig}_1, \ldots, \mathsf{rsig}_L))$ and perform the following checks to authenticate the route $\overline{\mathsf{rte}}$: If $j_\ell \neq i$ or $j_{\ell+1} \notin \mathsf{Output}_{(\ell,g)}$, then abort. If rsig_ℓ is not valid signature of $(j_1, \ldots, j_L)$ w.r.t. $\mathsf{rpp}_{(\ell,i)}$ and $\mathsf{rvk}_{(\ell,i)}$, then abort.
   c) **Prepare the output ciphertext $\mathsf{CT}_{(\ell+1,j_{\ell+1})}$:**
   If it is a *corrupt* wire:
      i. Let $j = j^*_{\ell+1}$. If $\mathsf{CT}_{(\ell+1,j_{\ell+1})}$ has already been computed, then abort.
      ii. If $\ell < L-1$, compute $\mathsf{msig}' = \mathsf{Sig.Sign}(\mathsf{mpp}_{(\ell+1,j)}, \mathsf{msk}_{(\ell+1,j)}, t, (x, \overline{\mathsf{rte}}^*_u))$ and $\mathsf{CT}_{(\ell+1,j)} = (x, \overline{\mathsf{rte}}^*_u, \mathsf{msig}') \oplus \mathsf{PRF}.\mathbf{Eval}(k_{(\ell+1,j)}, t)$.
      iii. If $\ell = L-1$ (output layer), compute $\mathsf{CT}_{(L,j)} \leftarrow (x, \perp, \perp) \oplus \mathsf{PRF}.\mathbf{Eval}(k_{(L,j)}, t)$.
   If it is an *honest* wire and $t \neq t^*$:
      i. Let $j = j_{\ell+1}$. If $\mathsf{CT}_{(\ell+1,j_{\ell+1})}$ has already been computed, then abort.
      ii. If $\ell < L-1$ (intermediate layer), compute $\mathsf{msig}' = \mathsf{Sig.Sign}(\mathsf{mpp}_{(\ell+1,j)}, \mathsf{msk}_{(\ell+1,j)}, t, (x, \overline{\mathsf{rte}}))$ and $\mathsf{CT}_{(\ell+1,j)} \leftarrow (x, \overline{\mathsf{rte}}, \mathsf{msig}') \oplus \mathsf{PRF}.\mathbf{Eval}(k_{(\ell+1,j)}, t)$.
      iii. If $\ell = L-1$ (output layer), compute $\mathsf{CT}_{(L,j)} \leftarrow (x, \perp, \perp) \oplus \mathsf{PRF}.\mathbf{Eval}(k_{(L,j)}, t)$.
2. For each $j \in \mathsf{Output}_{(\ell,g)}$ such that $\mathsf{CT}_{(\ell+1,j)}$ has not been computed yet:
   If $t = t^*$, compute *filler/inversion* ciphertexts as $\mathsf{CT}^*_{(\ell+1,j)} = \overline{\mathsf{CT}}^*_{(\ell+1,j)}$.
   If $t \neq t^*$, compute *filler* ciphertexts:
   a) Set $x = \perp_{\mathsf{filler}}$ and $\overline{\mathsf{rte}} = \perp_{\mathsf{filler}}$.
   b) Compute $\mathsf{msig}' = \mathsf{Sig.Sign}(\mathsf{mpp}_{(\ell+1,j)}, \mathsf{msk}_{(\ell+1,j)}, t, (x, \overline{\mathsf{rte}}))$.
   c) Compute $\mathsf{CT}_{(\ell+1,j)} \leftarrow (x, \overline{\mathsf{rte}}, \mathsf{msig}') \oplus \mathsf{PRF}.\mathbf{Eval}(k_{(\ell+1,j)}, t)$.
3. Output $\{\mathsf{CT}_{(\ell+1,i)} : i \in \mathsf{Output}_{(\ell,g)}\}$.

Figure 4.14: The circuit $\mathsf{Gate}_{(\ell,g)}$ in hybrid experiment $\mathsf{Hyb}^{(b)}_9$: procedure.

4.8.2 Proofs of Indistinguishability

Let the number of corrupt senders be $\theta = |\mathcal{K}_S| \leq n$.

Claim 4.8.3. *For $b \in \{0,1\}$, assuming correctness of the SSU signatures scheme, adversary $\mathcal{A}$'s views in $\mathsf{Hyb}_0^{(b)}$ and $\mathsf{Hyb}_1^{(b)}$ are identical.*

Proof. The difference between $\mathsf{Hyb}_0^{(b)}$ and $\mathsf{Hyb}_1^{(b)}$ is that the for all the *corrupt* wires, the route signing keys are unpunctured in the former and punctured in the latter hybrid experiment. While no route signing keys are in the view of the adversary, the adversary does get route signatures for corrupt senders and these are computed differently across two hybrids. In the former, they are computed using Sig.Sign algorithm and in the latter they are computed using Sig.PSign algorithm. It follows from the correctness of the SSU signature scheme as defined in Section 4.5 that the input/output behaviour of these two algorithms is identical for all unpunctured points. As these signatures are generated by the challenger for some unpunctured points, hence, adversary $\mathcal{A}$'s views in $\mathsf{Hyb}_0^{(b)}$ and $\mathsf{Hyb}_1^{(b)}$ are identical. □

Claim 4.8.4. *For $b \in \{0,1\}$, assuming the SSU signature scheme satisfies computational indistinguishability of punctured and binding setups, adversary $\mathcal{A}$'s views in $\mathsf{Hyb}_1^{(b)}$ and $\mathsf{Hyb}_2^{(b)}$ are computationally indistinguishable.*

Proof. The difference between hybrids $\mathsf{Hyb}_1^{(b)}$ and $\mathsf{Hyb}_2^{(b)}$ is that in hybrid $\mathsf{Hyb}_1^{(b)}$, for all the *corrupt* wires, the route signature key tuple $(\mathsf{rsk}'_{(\ell,i)}, \mathsf{rvk}_{(\ell,i)}, \mathsf{rpp}_{(\ell,i)})$ is used where the signing key is punctured and the verification key and public parameter are unpunctured. Whereas in hybrid $\mathsf{Hyb}_2^{(b)}$, for all the *corrupt* wires, the route signature key tuple $(\mathsf{rsk}^*_{(\ell,i)}, \mathsf{rvk}^*_{(\ell,i)}, \mathsf{rpp}^*_{(\ell,i)})$ is used where all the keys are binding and are generated using binding setup. There are θ *corrupt* wires in each layer $\ell = 1, \ldots, L$. For the sake of simplicity, call these total of $L \cdot \theta$ wires to be $w_1, \ldots, w_{L\cdot\theta}$. We will show that the adversary $\mathcal{A}$'s views in $\mathsf{Hyb}_1^{(b)}$ and $\mathsf{Hyb}_2^{(b)}$ are computationally indistinguishable via a sequence of hybird $\mathbb{H}_{1,0}^{(b)}, \ldots, \mathbb{H}_{1,L\cdot\theta}^{(b)}$ where in $\mathbb{H}_{1,i}$, for wire w_j, the route signature key tuple $(\mathsf{rsk}^*_{w_j}, \mathsf{rvk}^*_{w_j}, \mathsf{rpp}^*_{w_j})$ is used if $j \leq i$, else the pair $(\mathsf{rsk}'_{w_j}, \mathsf{rvk}_{w_j}, \mathsf{rpp}_{w_j})$ is used. With this sequence, observe that $\mathbb{H}_{1,0}^{(b)}$ is identical to $\mathsf{Hyb}_1^{(b)}$ and $\mathbb{H}_{1,L\cdot\theta}^{(b)}$ is identical to $\mathsf{Hyb}_2^{(b)}$. We will show that $\mathbb{H}_{1,i-1}^{(b)}$ and $\mathbb{H}_{1,i}^{(b)}$ are computationally indistinguishable for all $i \in [L \cdot \theta]$ and then, by triangle inequality it follows that $\mathsf{Hyb}_1^{(b)}$ and $\mathsf{Hyb}_2^{(b)}$ are computationally indistinguishable.

All that remains to show now is that for all $i \in [L \cdot \theta]$, $\mathbb{H}_{1,i-1}^{(b)}$ and $\mathbb{H}_{1,i}^{(b)}$ are computationally indistinguishable. Let wire w_i be on the route rte^*_u for some $u \in \mathcal{K}_S$. Observe that the difference between the two hybrids is that for wire w_i in $\mathbb{H}_{1,i-1}^{(b)}$, the route signature key tuple $(\mathsf{rsk}'_{w_i}, \mathsf{rvk}_{w_i}, \mathsf{rpp}_{w_i})$ is used, whereas in $\mathbb{H}_{1,i}^{(b)}$, the route signature key tuple $(\mathsf{rsk}^*_{w_i}, \mathsf{rvk}^*_{w_i}, \mathsf{rpp}^*_{w_i})$ is used. Then, we can create a simple reduction from the computational indistinguishability of the two hybrids to the computational indistinguishability of punctured and binding setup of SSU signature scheme which states that the following two distibutions are computationally indistinguishable.

1. Let $(\mathsf{rsk}_{w_i}, \mathsf{rsk}'_{w_i}, \mathsf{rvk}_{w_i}, \mathsf{rpp}_{w_i}) \leftarrow \mathsf{Sig}.\mathbf{PuncturedSetup}(1^\lambda, 0, \mathsf{len}_{\mathsf{rte}}, 1, \mathsf{rte}^*_u)$, and output $(\mathsf{rsk}'_{w_i}, \mathsf{rvk}_{w_i}, \mathsf{rpp}_{w_i})$.

2. Let $(\mathsf{rsk}^*_{w_i}, \mathsf{rvk}^*_{w_i}, \mathsf{rpp}^*_{w_i}) \leftarrow \mathsf{Sig.}\mathbf{BindingSetup}(1^\lambda, 0, \mathsf{len}_{\mathsf{rte}}, 1, \mathsf{rte}^*_u)$, and output $(\mathsf{rsk}^*_{w_i}, \mathsf{rvk}^*_{w_i}, \mathsf{rpp}^*_{w_i})$.

□

Claim 4.8.5. *For $b \in \{0,1\}$, assuming that the SSU signature scheme is deterministic and statistically unforgeable at the binding point and the indistinguishability obfuscation scheme is secure, adversary $\mathcal{A}$'s views in $\mathsf{Hyb}_2^{(b)}$ and $\mathsf{Hyb}_3^{(b)}$ are computationally indistinguishable.*

Proof. The difference between the hybrids $\mathsf{Hyb}_2^{(b)}$ and $\mathsf{Hyb}_3^{(b)}$ is in the way route authentication checks are performed for all *corrupt* wires in all the obfuscated gates. In particular, for all gates $\mathsf{Gate}_{(\ell,g)}$ for $\ell \in [L-1]$ and $g \in [G]$, after decryption of incoming ciphertext on a *corrupt* wire (ℓ, i) and message authentication of the plaintext (containing route information $\overline{\mathsf{rte}}_u = (\mathsf{rte}_u = (j_1, \ldots, j_L), \mathsf{rsig}_u = (\mathsf{rsig}_1, \ldots, \mathsf{rsig}_L))$ for some user $u \in \mathcal{K}_S$), the route authentication in hybrid $\mathsf{Hyb}_2^{(b)}$ is performed by checking

$$j_\ell = i, j_{\ell+1} \in \mathsf{Output}_{(\ell,i)}, \mathsf{Sig.Vf}(\mathsf{rpp}^*_{(\ell,i)}, \mathsf{rvk}^*_{(\ell,i)}, \mathsf{rte}_u, 1, \mathsf{rsig}_\ell) = 1. \tag{4.1}$$

If any of these checks fail, the circuit aborts further computation. On the other hand, the route authentication in hybrid $\mathsf{Hyb}_3^{(b)}$ is performed by checking

$$\mathsf{rte}_u = \mathsf{rte}^*_u, \mathsf{rsig}_\ell = \mathsf{rsig}^*_\ell, \tag{4.2}$$

where the expected route for this wire $\overline{\mathsf{rte}}^*_u = (\mathsf{rte}^*_u = (j^*_1, \ldots, j^*_L), \mathsf{rsig}^* = (\mathsf{rsig}^*_1, \ldots, \mathsf{rsig}^*_L))$ is hardcoded in the gate circuit. If we can argue that for all gates $\mathsf{Gate}_{(\ell,g)}$ for $\ell \in [L-1]$ and $g \in [G]$, the gate circuit in the two hybrid experiments have identical input/output behaviour, then, the computational indistinguishability of the adversary $\mathcal{A}$'s views in the two hybrids follows from the security of the indistinguishability obfuscation scheme.

All that remains to be shown is that for any gate $\mathsf{Gate}_{(\ell,g)}$, the input/output behaviour of the circuits in the two hybrids is indeed identical. In other words, we want to show that it is equivalent to check either Equation (4.1) or Equation (4.2). If Equation (4.2) is satisfied, then, it is straighforward to observe that Equation (4.1) is also satisfied. It is non-trivial to see that whenever Equation (4.1) is satisfied, then, Equation (4.2) is also satisfied. To see this, notice that $\mathsf{rpp}^*_{(\ell,i)}$ and $\mathsf{rvk}^*_{(\ell,i)}$ are binding route public parameter and binding route verification key and satisfy statistical unforgeability at round 1 and value $(j^*_1, \ldots, j^*_L)$ as defined in Definition 4.5.1. Hence, the signature verification algorithm will only accept signature rsig^*_ℓ for $(j^*_1, \ldots, j^*_L)$ and no other signature for this or any other route. Then, it follows that Equation (4.2) is also satisfied. □

Claim 4.8.6. *For $b \in \{0,1\}$, assuming correctness of the SSU signatures scheme, adversary $\mathcal{A}$'s views in $\mathsf{Hyb}_3^{(b)}$ and $\mathsf{Hyb}_4^{(b)}$ are identical.*

Proof. The difference between $\mathsf{Hyb}_3^{(b)}$ and $\mathsf{Hyb}_4^{(b)}$ is that the for all the *filler/inversion* wires $(1,i)$, the message signing keys are unpunctured in the former and punctured in the latter hybrid experiment at the challenge rount t^* and the respective challenge plaintexts $\tilde{x}^* = (x_{u,t^*}^{(b)}, \overline{\mathsf{rte}}_u^{(b)})$ in case of *inversion* wire for some $u \in \mathcal{H}_S$ or $\tilde{x}^* = (\perp_{\mathsf{filler}}, \perp_{\mathsf{filler}})$ in case of *filler* wire. But, notice that none of these message signing keys are in the view of the adversary. The only difference then is the ciphertexts that the adversary obtains from the challenger for the *inversion* wires in the first layer. In $\mathsf{Hyb}_3^{(b)}$, the Enc algorithm internally uses $\mathsf{Sig.Sign}$ to compute the message signatures, whereas in $\mathsf{Hyb}_4^{(b)}$, the Enc algorithm internally uses $\mathsf{Sig.PSign}$ to compute the message signatures. It follows from the correctness of the SSU signature scheme as defined in Section 4.5 that the input/output behaviour of these two algorithms is identical for all unpunctured points. As these signatures are generated by the challenger for some unpunctured points, hence, it follows that the adversary $\mathcal{A}$'s views in $\mathsf{Hyb}_3^{(b)}$ and $\mathsf{Hyb}_4^{(b)}$ are identical.

□

Claim 4.8.7. *For $b \in \{0,1\}$, assuming the SSU signature scheme satisfies computational indistinguishability of punctured and binding setups, adversary $\mathcal{A}$'s views in $\mathsf{Hyb}_4^{(b)}$ and $\mathsf{Hyb}_5^{(b)}$ are computationally indistinguishable.*

Proof. The difference between hybrids $\mathsf{Hyb}_4^{(b)}$ and $\mathsf{Hyb}_5^{(b)}$ is that in hybrid $\mathsf{Hyb}_5^{(b)}$, for all the *filler/inversion* wires $(1,i)$, the message signature key tuple $(\mathsf{msk}'_{(1,i)}, \mathsf{mvk}_{(1,i)}, \mathsf{mpp}_{(1,i)})$ is used where the signing key is punctured and the verification key and public parameter are unpunctured. Whereas in hybrid $\mathsf{Hyb}_5^{(b)}$, for all the *filler/inversion* wires, the message signature key tuple $(\mathsf{msk}^*_{(1,i)}, \mathsf{mvk}^*_{(1,i)}, \mathsf{mpp}^*_{(1,i)})$ is used where all the keys are binding and are generated using binding setup. Suppose that there are $\eta < 2n$ *filler/inversion* wires in the first layer $\ell = 1$. For the sake of simplicity, call these η wires to be $w_1, \ldots, w_\eta$. We will show that the adversary $\mathcal{A}$'s views in $\mathsf{Hyb}_4^{(b)}$ and $\mathsf{Hyb}_5^{(b)}$ are computationally indistinguishable via a sequence of hybird $\mathbb{H}_{4,0}^{(b)}, \ldots, \mathbb{H}_{4,\eta}^{(b)}$ where in $\mathbb{H}_{4,i}$, for wire w_j, the message signature key tuple $(\mathsf{msk}^*_{w_j}, \mathsf{mvk}^*_{w_j}, \mathsf{mpp}^*_{w_j})$ is used if $j \leq i$, else the tuple $(\mathsf{msk}'_{w_j}, \mathsf{mvk}_{w_j}, \mathsf{mpp}_{w_j})$ is used. With this sequence, observe that $\mathbb{H}_{4,0}^{(b)}$ is identical to $\mathsf{Hyb}_4^{(b)}$ and $\mathbb{H}_{4,\eta}^{(b)}$ is identical to $\mathsf{Hyb}_5^{(b)}$. We will show that $\mathbb{H}_{4,i-1}^{(b)}$ and $\mathbb{H}_{4,i}^{(b)}$ are computationally indistinguishable for all $i \in [\eta]$ and then, by triangle inequality it follows that $\mathsf{Hyb}_4^{(b)}$ and $\mathsf{Hyb}_5^{(b)}$ are computationally indistinguishable.

All that remains to show now is that for all $i \in [\eta]$, $\mathbb{H}_{4,i-1}^{(b)}$ and $\mathbb{H}_{4,i}^{(b)}$ are computationally indistinguishable. Observe that the difference between the two hybrids is the treatment of message signature key tuple for wire w_i. If wire w_i is an *inversion* wire, then, the challenge plaintext is $\tilde{x}^* = (x_{u,t^*}^{(b)}, \overline{\mathsf{rte}}_u^{(b)})$ for some user $u \in \mathcal{H}_S$. Else, if it is a *filler* wire, then, the challenge plaintext is $\tilde{x}^* = (\perp_{\mathsf{filler}}, \perp_{\mathsf{filler}})$. In $\mathbb{H}_{4,i-1}^{(b)}$, the message signature key tuple $(\mathsf{msk}'_{w_i}, \mathsf{mvk}_{w_i}, \mathsf{mpp}_{w_i})$ is used, whereas in $\mathbb{H}_{4,i}^{(b)}$, the message signature key tuple $(\mathsf{msk}^*_{w_i}, \mathsf{mvk}^*_{w_i}, \mathsf{mpp}^*_{w_i})$ is used. In both the above hybrids the binding is done at challenge round t^*

and challenge plaintext $\tilde{x}^*$. We can create a simple reduction from the computational indistinguishability of the two hybrids to the computational indistinguishability of punctured and binding setups of SSU signature scheme which states that the following two distibutions are computationally indistinguishable.

1. Let $(\mathsf{msk}_{w_i}, \mathsf{msk}'_{w_i}, \mathsf{mvk}_{w_i}, \mathsf{mvk}_{w_i}) \leftarrow \mathsf{Sig}.\mathbf{PuncturedSetup}(1^\lambda, \mathsf{tlen}, \mathsf{len}_m, t^*, \tilde{x}^*)$, and output $(\mathsf{msk}'_{w_i}, \mathsf{mvk}_{w_i}, \mathsf{mvk}_{w_i})$.
2. Let $(\mathsf{msk}^*_{w_i}, \mathsf{mvk}^*_{w_i}, \mathsf{mpp}^*_{w_i}) \leftarrow \mathsf{Sig}.\mathbf{BindingSetup}(1^\lambda, \mathsf{tlen}, \mathsf{len}_m, t^*, \tilde{x}^*)$, and output $(\mathsf{msk}^*_{w_i}, \mathsf{mvk}^*_{w_i}, \mathsf{mpp}^*_{w_i})$.

□

Claim 4.8.8. *For $b \in \{0,1\}$ and $\ell \in [L-1]$, assuming the SSU signature scheme is a deterministic signature scheme and is statistically unforgeable at the binding point and that the indistinguishability obfuscation scheme is secure, adversary $\mathcal{A}$'s views in $\mathsf{Hyb}^{(b)}_{6,\ell-1,5}$ (or $\mathsf{Hyb}^{(b)}_5$ if $\ell = 1$) and $\mathsf{Hyb}^{(b)}_{6,\ell,1}$ are computationally indistinguishable.*

Proof. For the sake of simplicity, we define $\mathsf{Hyb}^{(b)}_{6,0,5} = \mathsf{Hyb}^{(b)}_5$ in this proof.

The only difference between $\mathsf{Hyb}^{(b)}_{6,\ell,1}$ and $\mathsf{Hyb}^{(b)}_{6,\ell-1,5}$ is in the way that the message signatures $\mathsf{msig}_{(\ell,i)}$ for all *filler/inversion* wires $i \in \mathsf{Input}_{(\ell,g)}$ are verified inside of each obfuscated program $\overline{\mathsf{Gate}}_{(\ell,g)}$, $g \in [G]$ during round t^*. In $\mathsf{Hyb}^{(b)}_{6,\ell-1,5}$, this is done by using the verification algorithm $\mathsf{Sig.Vf}$, whereas in $\mathsf{Hyb}^{(b)}_{6,\ell,1}$ this is done by checking $\mathsf{msig}_{(\ell,i)}$ is equal to to the hardcoded signature $\mathsf{msig}^*_{(\ell,i)}$, by checking that the signed message x is equal to the hardcoded message $x^*_{(\ell,i)}$. and by checking that the route $\overline{\mathsf{rte}}$ is equal to the hardcoded route $\overline{\mathsf{rte}}^*$.

We prove indistinguishability via a sequence of subhybrids Hyb'_α, where $\alpha \in \{1, \ldots, 2n\}$.

We define Hyb'_α to produce programs $\{\overline{\mathsf{Gate}}_{(\ell,g)}\}_{g\in[G]}$ which verify all message signatures $\mathsf{msig}_{(\ell,i)}$ as in $\mathsf{Hyb}^{(b)}_{6,\ell,1}$ when $i < \alpha$, and as in $\mathsf{Hyb}^{(b)}_{6,\ell-1,5}$ when $i \geq \alpha$. It is clear that $\mathsf{Hyb}'_1 = \mathsf{Hyb}^{(b)}_{6,\ell-1,5}$ and that $\mathsf{Hyb}'_{2n} = \mathsf{Hyb}^{(b)}_{6,\ell,1}$. Proving the claim thus reduces to proving indistinguishability between $\mathsf{Hyb}'_{\alpha-1}$ and Hyb'_α for all α. Note that the only difference between Hyb'_α and $\mathsf{Hyb}'_{\alpha-1}$ is in the behavior of a single $\overline{\mathsf{Gate}}_{(\ell,g)}$ for g such that $\alpha \in \mathsf{Input}_{(\ell,g)}$. Provided that the circuits obfuscated in $\mathsf{Hyb}'_{\alpha-1}$ and Hyb'_α to produce $\overline{\mathsf{Gate}}_{(\ell,g)}$ are functionally equivalent, a simple reduction to security of the obfuscation scheme shows that $\mathsf{Hyb}'_{\alpha-1}$ and Hyb'_α are computationally indistinguishable. Thus we have reduced the claim to showing functional equivalence of the two circuits in question.

To show functional equivalence, we observe that the only possible point at which the circuit outputs could differ is where $\mathsf{CT}_{(\ell,\alpha)}$ is an encryption of some message $(x_{(\ell,\alpha)}, \overline{\mathsf{rte}}, \mathsf{msig}_{(\ell,\alpha)})$ with respect to round t^* which was not generated honestly. It is clear that in Hyb'_α, because the *filler/inversion* message $x^*_{(\ell,i)}$, route $\overline{\mathsf{rte}}^*$ and signature $\mathsf{msig}_{(\ell,\alpha)}$ are hardcoded, $\mathsf{Gate}_{(\ell,g)}$ rejects all dishonestly generated ciphertexts. Because of statistical unforgeability of the SSU signature scheme at point $(t^*, x^*_{(\ell,i)})$ with respect to the key tuple $(\mathsf{msk}^*_{(\ell,\alpha)}, \mathsf{mvk}^*_{(\ell,\alpha)}, \mathsf{mpp}^*_{(\ell,\alpha)})$ which was generated by $\mathsf{Sig}.\mathbf{BindingSetup}$, the only message that is accepted

by $\mathsf{Sig.Vf}$ is $(x^*_{(\ell,i)}, \overline{\mathsf{rte}}^*)$, and by the fact that the scheme is a determistic signature scheme, the only accepted signature is $\mathsf{msig}^*_{(\ell,\alpha)}$. Thus, in $\mathsf{Hyb}'_{\alpha-1}$ at round t^*, $\mathsf{Gate}_{(\ell,g)}$ also rejects all inputs where $\mathsf{CT}_{(\ell,\alpha)}$ encrypts a message which was dishonestly generated. It follows that the circuits have identical behavior with respect to $\mathsf{CT}_{(\ell,\alpha)}$, and thus are functionally equivalent.

□

Claim 4.8.9. *For $b \in \{0,1\}$ and $\ell \in [L-1]$, assuming correctness of the punctured PRF scheme and the indistinguishability obfuscation scheme is secure, adversary $\mathcal{A}$'s views in $\mathsf{Hyb}^{(b)}_{6,\ell,1}$ and $\mathsf{Hyb}^{(b)}_{6,\ell,2}$ are computationally indistinguishable.*

Proof. The only difference between $\mathsf{Hyb}^{(b)}_{6,\ell,2}$ and $\mathsf{Hyb}^{(b)}_{6,\ell,1}$ is that in the obfuscated programs $\{\overline{\mathsf{Gate}}_{(\ell,g)}\}_g$ of $\mathsf{Hyb}^{(b)}_{6,\ell,2}$, the obfuscated programs inversion/filler wire keys are punctured at t^*, and during round t^* inversion/filler wire ciphertexts $\mathsf{CT}_{(\ell,i)}$ are not decrypted directly, and instead are compared with a fixed value $\overline{\mathsf{CT}}^*_{(\ell,i)}$, whose corresponding decryption is also hardcoded and used for the rest of the procedure.

We prove indistinguishability via a sequence of subhybrids Hyb'_α, where $\alpha \in \{1,\ldots,2n\}$.

We define Hyb'_α to produce programs $\{\overline{\mathsf{Gate}}_{(\ell,g)}\}_{g\in[G]}$ which are identical to those in $\mathsf{Hyb}^{(b)}_{6,\ell,1}$, except for the following differences:

- For all $i < \alpha$, if this is an inversion/filler wire, hardcode punctured key $k^*_{(\ell,i)}$. Treat input inversion/filler wire ciphertexts $\mathsf{CT}_{(\ell,i)}$ in the same way as in $\mathsf{Hyb}^{(b)}_{6,\ell,2}$ (i.e., do not decrypt directly, instead check whether $\mathsf{CT}_{(\ell,i)} = \overline{\mathsf{CT}}^*_{(\ell,i)}$, and if so, use corresponding hardcoded plaintext in the rest of the procedure.
- For all $i \geq \alpha$, hardcode non-punctured key $k_{(\ell,i)}$. Treat input inversion/filler wire ciphertexts $\mathsf{CT}_{(\ell,i)}$ in the same way as in $\mathsf{Hyb}^{(b)}_{6,\ell,1}$ (i.e., decrypt directly and proceed as normal).

It is clear that $\mathsf{Hyb}'_1 = \mathsf{Hyb}^{(b)}_{6,\ell,1}$ and that $\mathsf{Hyb}'_{2n} = \mathsf{Hyb}^{(b)}_{6,\ell,2}$. Proving the claim thus reduces to proving indistinguishability between $\mathsf{Hyb}'_{\alpha-1}$ and Hyb'_α for all α. Note that the only difference between Hyb'_α and $\mathsf{Hyb}'_{\alpha-1}$ is in the circuit $\mathsf{Gate}_{(\ell,g)}$ which is used to generate a single obfuscated program $\overline{\mathsf{Gate}}_{(\ell,g)}$ for g such that $\alpha \in \mathsf{Input}_{(\ell,g)}$. Provided that the circuits obfuscated in $\mathsf{Hyb}'_{\alpha-1}$ and Hyb'_α to produce $\overline{\mathsf{Gate}}_{(\ell,g)}$ are functionally equivalent, a simple reduction to security of the obfuscation scheme shows that $\mathsf{Hyb}'_{\alpha-1}$ and Hyb'_α are computationally indistinguishable. Thus we have reduced the claim to showing functional equivalence of the two circuits in question.

To show functional equivalence, note that the behavior of $\mathsf{Gate}_{(\ell,g)}$ only differs across $\mathsf{Hyb}'_{\alpha-1}$ and Hyb'_α in terms of the behavior for the input wire $(\ell, \alpha-1)$. We focus on this wire. In $\mathsf{Hyb}'_{\alpha-1}$, $\mathsf{Gate}_{(\ell,g)}$ only

accepts exactly one value for $\mathsf{CT}_{(\ell,\alpha-1)}$ during round t^*. This is because $\mathsf{Gate}_{(\ell,g)}$ decrypts $\mathsf{CT}_{(\ell,\alpha-1)}$ and then compares the decrypted plaintext to fixed hardcoded values, and aborts if they are unequal. Since decryption is deterministic, only one such ciphertext $\mathsf{CT}_{(\ell,\alpha-1)}$ does not cause an abort. Since in Hyb'_α $\mathsf{Gate}_{(\ell,g)}$ hardcodes this exact ciphertext and the corresponding plaintext, the behavior with respect to round t^* is identical. Because of correctness of the punctured PRF key at all non-punctured points $t \neq t^*$, the behavior of $\mathsf{Gate}_{(\ell,g)}$ between $\mathsf{Hyb}'_{\alpha-1}$ and Hyb'_α across all other rounds are also identical. Thus, $\mathsf{Gate}_{(\ell,g)}$ is functionally equivalent across these two subhybrids.

□

Claim 4.8.10. *For $b \in \{0,1\}$ and $\ell \in [L-1]$, assuming correctness of the SSU signature scheme, and assuming the indistinguishability obfuscation scheme is secure, adversary $\mathcal{A}$'s views in $\mathsf{Hyb}^{(b)}_{6,\ell,2}$ and $\mathsf{Hyb}^{(b)}_{6,\ell,3}$ are computationally indistinguishable.*

Proof. The only difference between $\mathsf{Hyb}^{(b)}_{6,\ell,3}$ and $\mathsf{Hyb}^{(b)}_{6,\ell,2}$ is that in $\mathsf{Hyb}^{(b)}_{6,\ell,3}$, for inversion/filler output wires $(\ell+1,i)$, the challenger hardcodes punctured message signing keys $\mathsf{msk}'_{(\ell+1,i)}$ instead of unpunctured ones. The obfuscated gates then use the punctured signing algorithm $\mathsf{Sig.PSign}$ when signing outgoing messages at layer $\ell+1$.

We prove indistinguishability via a sequence of subhybrids Hyb'_α, where $\alpha \in \{1,\ldots,2n\}$.

We define Hyb'_α to produce programs $\{\overline{\mathsf{Gate}}_{(\ell,g)}\}_{g\in[G]}$ which are identical to those in $\mathsf{Hyb}^{(b)}_{6,\ell,2}$, except for the following differences:

- For all $i < \alpha$, if wire i is an inversion/filler wire, hardcode punctured signing key $\mathsf{msk}'_{(\ell+1,i)}$, and sign messages for output wire $(\ell+1,i)$ using $\mathsf{Sig.PSign}$.
- For all $i \geq \alpha$, hardcode non-punctured signing key $\mathsf{msk}_{(\ell+1,i)}$, and sign messages for output wire $(\ell+1,i)$ using $\mathsf{Sig.Sign}$.

It is clear that $\mathsf{Hyb}'_1 = \mathsf{Hyb}^{(b)}_{6,\ell,2}$ and that $\mathsf{Hyb}'_{2n} = \mathsf{Hyb}^{(b)}_{6,\ell,3}$. Proving the claim thus reduces to proving indistinguishability between $\mathsf{Hyb}'_{\alpha-1}$ and Hyb'_α for all α. Note that the only difference between Hyb'_α and $\mathsf{Hyb}'_{\alpha-1}$ is in the circuit $\mathsf{Gate}_{(\ell,g)}$ which is used to generate a single obfuscated program $\overline{\mathsf{Gate}}_{(\ell,g)}$ for g such that $\alpha \in \mathsf{Output}_{(\ell,g)}$. Provided that the circuits obfuscated in $\mathsf{Hyb}'_{\alpha-1}$ and Hyb'_α to produce $\overline{\mathsf{Gate}}_{(\ell,g)}$ are functionally equivalent, a simple reduction to security of the obfuscation scheme shows that $\mathsf{Hyb}'_{\alpha-1}$ and Hyb'_α are computationally indistinguishable. Thus we have reduced the claim to showing functional equivalence of the two circuits in question. Functional equivalence follows directly from correctness of the SSU signature scheme and the observation that no message is ever signed with respect to round t^* except for the exact binding message. □

Claim 4.8.11. *For $b \in \{0,1\}$ and $\ell \in [L-1]$, assuming the SSU signature scheme satisfies computational indistinguishability of punctured and binding setups, adversary $\mathcal{A}$'s views in $\mathsf{Hyb}^{(b)}_{6,\ell,3}$ and $\mathsf{Hyb}^{(b)}_{6,\ell,4}$ are computationally indistinguishable.*

Proof. The only difference between $\mathsf{Hyb}^{(b)}_{6,\ell,4}$ and $\mathsf{Hyb}^{(b)}_{6,\ell,3}$ is that in $\mathsf{Hyb}^{(b)}_{6,\ell,4}$, the message signature key tuples for inversion/filler wires $(\ell+1,i)$, $i \in [2n]$ are all generated using Sig.**BindingSetup**, whereas in $\mathsf{Hyb}^{(b)}_{6,\ell,3}$ they are generated using Sig.**PuncturedSetup**.

We prove indistinguishability via a sequence of subhybrids Hyb'_α, where $\alpha \in \{1,\ldots,2n\}$.

We define Hyb'_α to generate message signature key tuples $(\mathsf{msk}_{(\ell+1,i)}, \mathsf{mvk}_{(\ell+1,i)}, \mathsf{mpp}_{(\ell+1,i)})$ using Sig.**BindingSetup** for all inversion/filler $i < \alpha$, and to generate $(\mathsf{msk}_{(\ell+1,i)}, \mathsf{mvk}_{(\ell+1,i)}, \mathsf{mpp}_{(\ell+1,i)})$ using Sig.**PuncturedSetup** for all $i \geq \alpha$. It is clear that $\mathsf{Hyb}'_1 = \mathsf{Hyb}^{(b)}_{6,\ell,3}$ and that $\mathsf{Hyb}'_{2n} = \mathsf{Hyb}^{(b)}_{6,\ell,4}$. Proving the claim thus reduces to proving indistinguishability between $\mathsf{Hyb}'_{\alpha-1}$ and Hyb'_α for all α. Note that the only difference between Hyb'_α and $\mathsf{Hyb}'_{\alpha-1}$ is in the key tuple $(\mathsf{msk}_{(\ell+1,\alpha-1)}, \mathsf{mvk}_{(\ell+1,\alpha-1)}, \mathsf{mpp}_{(\ell+1,\alpha-1)})$. As such, a simple reduction to computational indistinguishability of the punctured and binding setups of the SSU signature scheme shows this indistinguishability. This proves the claim.

□

Claim 4.8.12. *For $b \in \{0,1\}$, assuming the indistinguishability obfuscation scheme is secure, adversary $\mathcal{A}$'s views in $\mathsf{Hyb}^{(b)}_{6,\ell,4}$ and $\mathsf{Hyb}^{(b)}_{6,\ell,5}$ are computationally indistinguishable.*

Proof. The only difference between $\mathsf{Hyb}^{(b)}_{6,\ell,5}$ and $\mathsf{Hyb}^{(b)}_{6,\ell,4}$ is in the behavior of circuits $\{\mathsf{Gate}_{(\ell,g)}\}_{g\in[G]}$ used to generated the obfuscated programs $\{\overline{\mathsf{Gate}}_{(\ell,g)}\}_{g\in[G]}$. In $\mathsf{Hyb}^{(b)}_{6,\ell,5}$ during round t^*, for inversion/-filler output wires $(\ell+1,i)$, $\mathsf{Gate}_{(\ell,g)}$ sets $\mathsf{CT}_{(\ell+1,i)}$ to be a hardcoded value $\overline{\mathsf{CT}}^*_{(\ell+1,i)}$, where $\overline{\mathsf{CT}}^*_{(\ell+1,i)} = (x^{(b)}_{u,t^*}, \overline{\mathsf{rte}}^*_u, \mathsf{msig}^*_{(\ell+1,i)}) \oplus \mathsf{PRF}.\mathbf{Eval}(k_{(\ell+1,i)}, t^*)$ if $\ell < L-1$, else $\overline{\mathsf{CT}}^*_{(\ell+1,i)} = (x^{(b)}_{u,t^*}, \bot, \bot) \oplus \mathsf{PRF}.\mathbf{Eval}(k_{(L,i)}, t^*)$, and outputs this value for wire $(\ell+1,i)$.

We prove indistinguishability via a sequence of subhybrids Hyb'_α, where $\alpha \in \{1,\ldots,2n\}$.

We define Hyb'_α to produce programs $\{\overline{\mathsf{Gate}}_{(\ell,g)}\}_{g\in[G]}$ which are identical to those in $\mathsf{Hyb}^{(b)}_{6,\ell,4}$, except for the following difference. During round t^*:

- For all $i < \alpha$ such that $(\ell+1,i)$ is a filler/inversion wire, output $\mathsf{CT}_{(\ell+1,i)} = \overline{\mathsf{CT}}^*_{(\ell+1,i)}$ for this wire.
- For all $i \geq \alpha$, act as in $\mathsf{Hyb}^{(b)}_{6,\ell,4}$.

It is clear that $\mathsf{Hyb}'_1 = \mathsf{Hyb}^{(b)}_{6,\ell,4}$ and that $\mathsf{Hyb}'_{2n} = \mathsf{Hyb}^{(b)}_{6,\ell,5}$. Proving the claim thus reduces to proving indistinguishability between $\mathsf{Hyb}'_{\alpha-1}$ and Hyb'_α for all α. Note that the only difference between Hyb'_α and $\mathsf{Hyb}'_{\alpha-1}$ is in the circuit $\mathsf{Gate}_{(\ell,g)}$ which is used to generate a single obfuscated program $\overline{\mathsf{Gate}}_{(\ell,g)}$ for g

such that $\alpha \in \mathsf{Output}_{(\ell,g)}$. Provided that the circuits obfuscated in $\mathsf{Hyb}'_{\alpha-1}$ and Hyb'_{α} to produce $\overline{\mathsf{Gate}}_{(\ell,g)}$ are functionally equivalent, a simple reduction to security of the obfuscation scheme shows that $\mathsf{Hyb}'_{\alpha-1}$ and Hyb'_{α} are computationally indistinguishable. Thus we have reduced the claim to showing functional equivalence of the two circuits in question.

To show functional equivalence, observe that the only wire where the two circuits could possibly differ is the output wire $(\ell+1, \alpha-1)$. Functional equivalence then follows directly from the following facts:

- No corrupt wire from layer ℓ could be re-routed to a *filler/inversion* wire in layer $\ell+1$ because of the hardwired routes for *corrupt* wires in hybrid $\mathsf{Hyb}_3^{(b)}$ (Figure 4.5).
- During round t^*, the plaintext $(x, \overline{\mathsf{rte}}, \mathsf{msig}')$ encrypted by $\mathsf{Gate}_{(\ell,g)}$ to form $\mathsf{CT}_{(\ell+1,\alpha-1)}$ in $\mathsf{Hyb}'_{\alpha-1}$ has values x and $\overline{\mathsf{rte}}$ are fixed and exactly equal to the corresponding plaintext components of $\overline{\mathsf{CT}}^*_{(\ell+1,\alpha-1)}$ in Hyb'_{α}, along with the fact that the SSU signature scheme produces deterministic signatures (which means that the signature msig' is also fixed and equal to the corresponding component of $\overline{\mathsf{CT}}^*_{(\ell+1,\alpha-1)}$).

□

Claim 4.8.13. *For $b \in \{0,1\}$, assuming pseudorandomness of the PRF at punctured points, adversary $\mathcal{A}$'s views in $\mathsf{Hyb}^{(b)}_{6,L-1,5}$ and $\mathsf{Hyb}^{(b)}_7$ are computationally indistinguishable.*

Proof. The difference between the two hybrids is in (i) how are the hardcoded ciphertexts for the challenge round t^* for all the *filler/inversion* wires in all the layers (with the exception of *inversion* output wires directed towards corrupt receivers in the last layer) computed by the challenger, and (ii) the challenge ciphertexts for $u \in \{s, s'\}$ given to the adversary for the challenge round t^*. In $\mathsf{Hyb}^{(b)}_{6,L-1,5}$, the hardcoded ciphertext for wire (ℓ, i) is of the form $\overline{\mathsf{CT}}^*_{(\ell,i)} = (\tilde{x}^*, \mathsf{msig}^*_{(\ell,i)}) \oplus y^*$, where $\mathsf{msig}^*_{(\ell,i)} = \mathsf{Sig.PSign}(\mathsf{mpp}^*_{(\ell,i)}, \mathsf{msk}^*_{(\ell,i)}, t^*, \tilde{x}^*)$ and $y^* = \mathsf{PRF}.\mathbf{Eval}(k_{(\ell,i)}, t^*)$. If it is an *inversion* wire corresponding to route $\overline{\mathsf{rte}}^{(b)}_u$ for some $u \in \mathcal{H}_S$, then, $\tilde{x}^* = (x^{(b)}_{u,t^*}, \overline{\mathsf{rte}}^{(b)}_u)$. Else, if it is a *filler* wire, then, $\tilde{x}^* = (\perp_{\mathsf{filler}}, \perp_{\mathsf{filler}})$. On the other hand, in $\mathsf{Hyb}^{(b)}_7$, for a *filler/inversion* wire (ℓ, i), $\overline{\mathsf{CT}}^*_{(\ell,i)} = \textit{random}$. Further, in both hybrids $\mathsf{Hyb}^{(b)}_{6,L-1,5}$ and $\mathsf{Hyb}^{(b)}_7$, the challenge ciphertexts for $u \in \{s, s'\}$ given to the adversary for the challenge round t^* are set consistent with the hardcoded incoming ciphertexts in the obfuscated gates in layer $\ell = 1$ in those respective hybrids. To argue the computationally indistinguishability of adversary's view in these two hybrids, notice that the hardcoded PRF key $k^*_{(\ell,i)}$ is punctured at t^*, whereas y^* is the PRF evaluation at exactly this punctured point. So, one can use the pseudorandomness of the PRF at punctured points to show the computational indistinguishability.

For a *filler/inversion* wire (ℓ, i), we want to change from $\overline{\mathsf{CT}}^*_{(\ell,i)} = (\tilde{x}^*, \mathsf{msig}^*_{(\ell,i)}) \oplus y^*$ to $\overline{\mathsf{CT}}^*_{(\ell,i)} = \textit{random}$, where $y^* = \mathsf{PRF}.\mathbf{Eval}(k_{(\ell,i)}, t^*)$. We first invoke the psuedorandomness of PRF at punctured points to

change to $y^* = random'$. Then, we can change from $\overline{\mathsf{CT}}^*_{(\ell,i)} = (\tilde{x}^*, \mathsf{msig}^*_{(\ell,i)}) \oplus random'$ to $\overline{\mathsf{CT}}^*_{(\ell,i)} = random$ as both are identically distributed. This is exactly what we want in $\mathsf{Hyb}_7^{(b)}$.

□

Claim 4.8.14. *For $b \in \{0,1\}$, assuming the SSU signature scheme satisfies computational indistinguishability of punctured and binding setups, adversary $\mathcal{A}$'s views in $\mathsf{Hyb}_7^{(b)}$ and $\mathsf{Hyb}_8^{(b)}$ are computationally indistinguishable.*

Proof. Similar to Claim 4.8.7, except that here the message verification keys and message public parameters are changed from binding to unbinding for all the *filler/inversion* wires (ℓ, i) in all the layers.

□

Claim 4.8.15. *For $b \in \{0,1\}$, assuming the SSU signatures scheme is correct, and the indistinguishability obfuscation scheme is secure, adversary $\mathcal{A}$'s views in $\mathsf{Hyb}_8^{(b)}$ and $\mathsf{Hyb}_9^{(b)}$ are computationally indistinguishable.*

Proof. The difference between the two hybrids is that in $\mathsf{Hyb}_8^{(b)}$, for all the *filler/inversion* wires (ℓ, i), punctured message signing key $\mathsf{msk}'_{(\ell,i)}$ is used, whereas in $\mathsf{Hyb}_9^{(b)}$, unpunctured message signing key $\mathsf{msk}_{(\ell,i)}$ is used. This difference reflects in how message signatures are computed in all the circuit $\mathsf{Gate}_{(\ell,g)}$ and by the Enc algorithm for ciphertexts corresponding to *inversion* wires in layer 1. For ciphertexts corresponding to *inversion* wires in layer 1, identical distibution can be proven similar to Claim 4.8.6 assuming correctness of SSU signatures.

We will argue that for all the circuits $\mathsf{Gate}_{(\ell,g)}$, the input/output behaviour is still the same. Hence, the indistinguishability of the two hybrids follows through a sequence of intermediate hybrid transitions where we switch the circuit for one gate at a time and the computational indistinguishability of any two consequtive intermediate hybrids can be shown through a simple reduction to the security of the indistinguishability obfuscation scheme.

All that remains to be shown is that all the circuits $\mathsf{Gate}_{(\ell,g)}$ in the two hybrids have identical input/output behaviour. Observe that the only points where the circuits in the two hybrids may differ are precisely the points that were punctured. $\mathsf{msk}_{(\ell,i)}$ can be used to sign even for challenge round t^* and non-challenge messages $x \neq x^*$ but $\mathsf{msk}'_{(\ell,i)}$ can't sign for these points. But observe that in $\mathsf{Hyb}_9^{(b)}$, $\mathsf{msk}_{(\ell,i)}$ is never used to sign messages for the challenge round t^* as the hardcoded ciphertexts are directly used instead. Hence, it follows that all the circuits $\mathsf{Gate}_{(\ell,g)}$ in the two hybrids indeed have identical input/output behaviour.

□

Claim 4.8.16. *Adversary $\mathcal{A}$'s views in $\mathsf{Hyb}_9^{(0)}$ and $\mathsf{Hyb}_9^{(1)}$ are identical.*

Proof. Note that the ciphertexts received by $\mathcal{A}$ in round t^* on behalf of s and s' are random values independent of b, and the other ciphertexts received by $\mathcal{A}$ during that round are independent of b by the definition of the hybrids. Recall the formal description of circuits $\mathsf{Gate}_{(\ell,g)}$ described in Figure 4.13 and Figure 4.14. It suffices to argue that the router token received by $\mathcal{A}$ in $\mathsf{Hyb}_9^{(b)}$ and $\mathsf{Hyb}_9^{(1-b)}$ contain no information about the challenge bit b. Observe that the gate circuit had no information about challenge bit b in the real world description. So, the only new sources of information about it could be the changes done to the circuit that are highlighted in the figures. We will now argue that all of them have no information about the challenge bit b.

- For each *corrupt* wire $i \in \mathsf{Input}_{(\ell,i)}$, route information $\overline{\mathsf{rte}}^*_u$, binding route public parameter $\mathsf{rpp}^*_{(\ell,i)}$ and binding route verification $\mathsf{rvk}^*_{(\ell,i)}$ are hardwired, where the route public parameter and route verification key are binding at route $\overline{\mathsf{rte}}^*_u$. The admissibility criteria requires that for each $u \in \mathcal{K}_S$, $\pi^{(0)}(u) = \pi^{(1)}(u)$. Hence, the hardcoded routes for corrupt senders have no information about the challenge bit b. It also follows then that the binding route public parameters and the binding verification keys for corrupt senders do not contain any information about the challenge bit b.

- For each *filler/inversion* wire $i \in \mathsf{Input}_{(\ell,i)}$, the punctured PRF key $k^*_{(\ell,i)}$ is hardwired. This key is punctured at the challenge round t^*, and hence has no information about the challenge bit b. For these wires, the expected challenge ciphertext $\overline{\mathsf{CT}}^*_{(\ell,i)}$ is also hardcoded. As all of these are uniformly random values, hence, it follows that they have no information about the challenge bit b.

- For each *filler/inversion* wire $i \in \mathsf{Output}_{(\ell,i)}$, the punctured PRF key $k^*_{(\ell+1,i)}$ is hardwired. This key is punctured at the challenge round t^*, and hence has no information about the challenge bit b. For these wires, the expected challenge ciphertext $\overline{\mathsf{CT}}^*_{(\ell+1,i)}$ is also hardcoded. If $\ell \neq L-1$, all of these are uniformly random values, hence, it follows that they have no information about the challenge bit b. If $\ell = L-1$ and the wire's destination is not a *corrupt* receiver, then also all of these are uniformly random values. Hence, it follows that they have no information about the challenge bit b. If $\ell = L-1$ and the wire's destination is a *corrupt* receiver, then, the hardcoded value is the what the *corrupt* receiver would expect. The admissibility criteria dictates that if two different honest senders are sending some message to a corrupt receiver in the two worlds $b = 0$ and $b = 1$, then, the message values by the two honest senders should be the same. Hence, it follows that the same ciphertext value is hardcoded in the two worlds and consequently, it leaks no information about the challenge bit b.

- It is possible that a wire could be *filler* when $b = 0$ and *inversion* when $b = 1$. Hence, it could have different treatment by the circuit procedure in the two worlds. But notice that in both hybrids $\mathsf{Hyb}_9^{(0)}$

and $\mathsf{Hyb}_9^{(1)}$, for the challenge round $t = t^*$, the treatment of filler and inversion wires is exactly the same. Hence, it leaks no information about the challenge bit b.

From the above analysis, it follows that adversary $\mathcal{A}$'s views in $\mathsf{Hyb}_9^{(0)}$ and $\mathsf{Hyb}_9^{(1)}$ are identical.

□

4.9 From Static Corruption to Adaptive Corruption

4.9.1 A Compiler for Upgrading from Static to Adaptive Corruption

In this section, we present a generic compiler that lifts any NIAR scheme that satisfies static-corruption security to a NIAR scheme with *full* security under adaptive corruptions and adaptive queries (Definition 4.3.2).

Starting from a static corruption NIAR scheme denoted NIAR', we will use an extra pseudorandom function PRF to encrypt the plaintext first, before encrypting it again with NIAR', thus creating *two layers* of encryption. For the security proof, we will need the PRF to be secure against *selective-opening attacks* (Definition 4.4.6), which is implied by the standard security for pseudorandom functions as shown by Abraham et al. [ACD+19]. We formally describe the compiler below.

A compiler from static corruption to adaptive corruption

$\underline{\mathsf{Setup}(1^\lambda, \mathsf{len}, n, \pi)}$**:**

- For all $i \in [n]$: $k_i \leftarrow \mathsf{PRF.Setup}(1^\lambda)$.
- Let $(\{\mathsf{ek}'_u\}_{u\in[n]}, \{\mathsf{rk}'_u\}_{u\in[n]}, \mathsf{tk}') \leftarrow \mathsf{NIAR}'.\mathsf{Setup}(1^\lambda, \mathsf{len}, n, \pi)$.
- For each sender $u \in [n]$, let $\mathsf{ek}_u := (k_{\pi(u)}, \mathsf{ek}'_u)$; for each receiver $u \in [n]$, let $\mathsf{rk}_u := (k_u, \mathsf{rk}'_u)$; and let $\mathsf{tk} = \mathsf{tk}'$. Output $(\{\mathsf{ek}_u\}_{u\in[n]}, \{\mathsf{rk}_u\}_{u\in[n]}, \mathsf{tk})$.

$\underline{\mathsf{Enc}(\mathsf{ek}_u, \mathbf{x}_u, t)}$**:** *// Parse* $\mathsf{ek}_u := (k'_u, \mathsf{ek}'_u)$

- Output $\mathsf{ct}_{u,t} \leftarrow \mathsf{NIAR}'.\mathsf{Enc}(\mathsf{ek}'_u, \mathsf{PRF}.\mathbf{Eval}(k'_u, t) \oplus \mathbf{x}_u, t)$.

$\underline{\mathbf{Rte}(\mathsf{tk}, t, \mathsf{ct}_1, \ldots, \mathsf{ct}_n)}$**:**

- Let $(\mathsf{ct}'_1, \ldots, \mathsf{ct}'_n) \leftarrow \mathsf{NIAR}'.\mathbf{Rte}(\mathsf{tk}, t, \mathsf{ct}_1, \ldots, \mathsf{ct}_n)$ and output $(\mathsf{ct}'_1, \ldots, \mathsf{ct}'_n)$.

$\underline{\mathsf{Dec}(\mathsf{rk}_u, \mathsf{ct}'_u, t)}$**:** *// Parse* $\mathsf{rk}_u := (k_u, \mathsf{rk}'_u)$

- Let $\mathbf{y} = \mathsf{PRF}.\mathbf{Eval}(k_u, t) \oplus \mathsf{NIAR}'.\mathsf{Dec}(\mathsf{rk}'_u, \mathsf{ct}'_u, t)$, and output $\mathbf{y}$.

Figure 4.15: A compiler from static corruption to adaptive corruption

Theorem 4.9.1 (Static to adaptive corruption compiler)**.** *Suppose that* NIAR' *satisfies Definition 4.3.1 subject to a static-corruption, single-inversion, and all-receiver-corrupting adverasary, and suppose that* PRF *is a selective-opening secure pseudorandom function (i.e. Definition 4.4.6). Then,* NIAR *satisfies full security (i.e., Definition 4.3.2).*

Proof roadmap. In the remainder of this section, we prove Theorem 4.9.1. The proof goes through multiple intermediate steps.

1. We start from a NIAR secure w.r.t. inversion under a static-corruption, all-receiver-corrupting adversary. We prove that the same scheme is also secure w.r.t. inversion under an *adaptive*-corruption, all-receiver-corrupting adversary. (see Section 4.9.2 and Lemma 4.9.2).

2. Next, given a NIAR scheme secure w.r.t. inversion under an adaptive-corruption, all-receiver-corrupting adversary, we remove the "inversion" restriction, and prove that it is also secure for an arbitrary pair of permutations under an adaptive-corruption, all-receiver-corrupting adversary (see Section 4.9.3 and Lemma 4.9.3).

3. Finally, given a NIAR scheme that is secure under an adaptive-corruption, all-receiver-corrupting adversary (for an arbitrary pair of permutations), we show that the above static-to-adaptive compiler gives a full NIAR scheme secure under Definition 4.3.2 (see Section 4.9.4 and Lemma 4.9.4).

4.9.2 Adaptive Corruption Under Single-Inversion and All-Receiver-Corruption

Lemma 4.9.2. *If a NIAR scheme satisfies security under a static-corruption, single-inversion, and all-receiver-corrupting adversary, then it also satisfies security under a single-inversion adversary who corrupts all receivers upfront, and corrupts senders adaptively.*

Proof. Consider a non-uniform p.p.t. admissible adversary $\mathcal{A}$ who corrupts all receivers upfront and corrupts the senders adaptively. $\mathcal{A}$ is trying to distinguish its views in experiments $\mathsf{NIARFull}^{0,\mathcal{A}}(1^\lambda)$ and $\mathsf{NIARFull}^{1,\mathcal{A}}(1^\lambda)$ subject to single inversion. Then, we construct a p.p.t. admissible adversary $\mathcal{B}$ that plays a game with its own challenger $\mathcal{C}$ and leverages $\mathcal{A}$ to distinguish between $\mathsf{NIARStatic}^{0,\mathcal{B}}(1^\lambda)$ and $\mathsf{NIARStatic}^{1,\mathcal{B}}(1^\lambda)$ subject to single inversion and all-receiver-corruption. The description of $\mathcal{B}$ is as follows.

1. $\mathcal{B}$ gets $(n, \mathsf{len}, \pi^{(0)}, \pi^{(1)})$ from $\mathcal{A}(1^\lambda)$. Let s_1, s_2 be the two senders involved in the inversion. For its game with $\mathcal{C}$, it chooses to corrupt the senders $\mathcal{K}'_S = [n] \setminus \{s_1, s_2\}$, the receivers $\mathcal{K}'_R = [n]$, and it sends $(n, \mathsf{len}, \mathcal{K}'_S, \mathcal{K}'_R, \pi^{(0)}, \pi^{(1)})$ to $\mathcal{C}$. Then, $\mathcal{C}$ computes $(\{\mathsf{ek}_u\}_{u\in[n]}, \{\mathsf{rk}_u\}_{u\in[n]}, \mathsf{tk}) \leftarrow \mathsf{Setup}(1^\lambda, \mathsf{len}, n, \pi^{(b)})$ and gives the terms $(\mathsf{tk}, \{\mathsf{ek}_u\}_{u\in\mathcal{K}'_S}, \{\mathsf{rk}_u\}_{u\in[n]})$ to $\mathcal{B}$. $\mathcal{B}$ now passes $(\mathsf{tk}, \{\mathsf{rk}_u\}_{u\in[n]})$ to $\mathcal{A}$ since $\mathcal{A}$ always corrupts all receivers upfront.

2. In each time step t:

   - Upon receiving an encryption query $\{x_{u,t}^{(0)}, x_{u,t}^{(1)}\}_{u \in \mathcal{H}_S}$ from $\mathcal{A}$ where $\mathcal{H}_S$ is the set of senders $\mathcal{A}$ has not corrupted yet, $\mathcal{B}$ sends the encryption query $\{x_{u,t}^{(0)}, x_{u,t}^{(1)}\}_{u \in \{s_1, s_2\}}$ to $\mathcal{C}$, and gets back ciphertexts $\{\mathsf{ct}_{u,t}\}_{u \in \{s_1, s_2\}}$. For every $u \in \mathcal{H}_S \backslash \{s_1, s_2\}$, by the admissibility rule on $\mathcal{A}$, it must be that $x_{u,t}^{(0)} = x_{u,t}^{(1)}$; thus, $\mathcal{B}$ computes $\mathsf{ct}_{u,t} = \mathsf{Enc}(\mathsf{ek}_u, x_{u,t}^{(0)}, t)$ on its own since it knows ek_u. Now, $\mathcal{B}$ sends $\{\mathsf{ct}_{u,t}\}_{u \in \mathcal{H}_S}$ to $\mathcal{A}$.
   - Upon receiving a corruption query to corrupt some sender $u \in [n]$, by the admissibility rule on $\mathcal{A}$, u cannot be s_1 or s_2. Therefore, $\mathcal{B}$ returns ek_u to $\mathcal{A}$, as well as all random coins used in earlier encryption queries involving u.

3. $\mathcal{B}$ outputs whatever $\mathcal{A}$ outputs.

Observe that if $\mathcal{A}$ is admissible and respects single inversion, then, $\mathcal{B}$ is also admissible and respects single inversion. By construction, $\mathcal{B}$ respects all receiver corruption. Further, for $b \in \{0, 1\}$, when $\mathcal{C}$ uses the challenge bit b, $\mathcal{A}$'s view is identical to $\mathsf{NIARFull}^{b,\mathcal{A}}(1^\lambda)$. Therefore, if $\mathcal{A}$ has non-negligible advantage in distinguishing $\mathsf{NIARFull}^{0,\mathcal{A}}(1^\lambda)$ and $\mathsf{NIARFull}^{1,\mathcal{A}}(1^\lambda)$, then $\mathcal{B}$ has non-negligible advantage in distinguishing $\mathsf{NIARStatic}^{0,\mathcal{B}}$ and $\mathsf{NIARStatic}^{1,\mathcal{B}}$. □

4.9.3 Removing the Single Inversion Restriction

We next remove the single inversion restriction. The proof is very similar to that of Lemma 4.7.6 where we removed the single inversion restriction but for the static corruption scenario.

Lemma 4.9.3 (Removing the single inversion restriction (adaptive corruption))**.** *Suppose we are given a NIAR scheme that satisfies security subject to a single-inversion adversary who always corrupts all receivers upfront, and corrupts senders adaptively. Then, the same scheme is also secure for an arbitrary pair of permutations, subject to an adversary who always corrupts all receivers upfront, and corrupts senders adaptively.*

Proof. The proof is almost identical to that of Lemma 4.7.6, except that now, to prove each pair of adjacent hybrids indistinguishable, the reduction $\mathcal{B}$ also has to answer $\mathcal{A}$'s adaptive sender corruption queries. $\mathcal{B}$ can simply forward such queries to its own challenger and forward back the answers. By $\mathcal{A}$'s admissibility rule, $\mathcal{A}$ can never corrupt a sender that is in $\overline{C}(\pi^{(0)}, \pi^{(1)})$. Therefore, in the adaptive corruption case, it still holds that if $\mathcal{A}$ is admissible, then $\mathcal{B}$ is admissible too. □

4.9.4 Removing the All Receiver Corruption Restriction

Finally, given a NIAR scheme that is secure under an adversary who corrupts all receivers upfront but can adaptively corrupt senders, we can remove the all-receiver-corruption restriction with the compiler described in 4.9.1.

Lemma 4.9.4 (Removing the all receiver corruption restriction)**.** *Suppose that* NIAR' *satisfies security under an adversary who corrupts all receivers upfront but can adaptively corrupt senders. Then, the compiled* NIAR *scheme described in 4.9.1 satisfies full NIAR security (Definition 4.3.2).*

Proof. We first define what is a conspicuously honest receiver.

Conspicuously honest receiver. Consider the experiment $\mathsf{NIARFull}^{b,\mathcal{A}}$. Recall that $\mathcal{A}$ submits two permutations $\pi^{(0)}$ and $\pi^{(1)}$ upfront. Consider some receiver j, and let s_0 and s_1 be the corresponding senders to j in $\pi^{(0)}$ and $\pi^{(1)}$, respectively, that is, for $b \in \{0,1\}$, $\pi^{(b)}(s_b) = j$. Suppose during some time step t, the adversary $\mathcal{A}$ submits an encryption query for the plaintext vectors $\{x^{(0)}_{u,t}, x^{(1)}_{u,t}\}_{u\in\mathcal{H}_S}$ where $\mathcal{H}_S$ denotes the so-far honest senders, and it turns out that $x^{(0)}_{s_0,t} \neq x^{(1)}_{s_1,t}$, i.e., j is receiving two different messages in the two worlds. Then, at this moment, we know that the receiver j can never be corrupt by $\mathcal{A}$ due to the admissibility rules on $\mathcal{A}$. Henceforth, such a receiver is called a conspicuously honest receiver. Observe also the corresponding senders s_0 and s_1 (which could be the same) must be conspicuously honest too, by the admissibility rule on $\mathcal{A}$.

Consider the following hybrid experiment indexed by a bit $b \in \{0,1\}$.

Experiment Hyb_b**.** Hyb_b is almost the same as $\mathsf{NIARFull}^b$, except with the following modifications: whenever $\mathcal{A}$ submits an encryption query for the plaintext vectors $\{x^{(0)}_{u,t}, x^{(1)}_{u,t}\}_{u\in\mathcal{H}_S}$ where $\mathcal{H}_S$ denotes the so-far honest senders, the challenger responds with the following ciphertexts for each $u \in \mathcal{H}_S$:

- if $\pi^{(b)}(u)$ is conspicuously honest, then compute $\mathsf{ct}_{u,t} = \mathsf{NIAR}'.\mathsf{Enc}(\mathsf{ek}_u, r, t)$ for a randomly chosen r of appropriate length;
- else compute $\mathsf{ct}_{u,t} = \mathsf{NIAR}.\mathsf{Enc}(\mathsf{ek}_u, x^{(b)}_{u,t}, t)$ honestly.

Claim 4.9.5. *Suppose that the PRF satisfies selective opening security, and let* $b \in \{0,1\}$*. Then, for the compiled* NIAR *scheme, it holds that for any non-uniform p.p.t. admissible adversary* $\mathcal{A}$*, its views in* Hyb_b *and* $\mathsf{NIARFull}^b$ *are computationally indistinguishable.*

Proof. We prove it for $b = 0$, since the case when $b = 1$ is symmetric. Given an efficient $\mathsf{NIARFull}$ adversary $\mathcal{A}$, we will construct an efficient reduction $\mathcal{B}$ that breaks the selective-opening security of the PRF.

- At the beginning $\mathcal{B}$ creates n PRF instances with its own challenger $\mathcal{C}$, one corresponding to each receiver. Except for the PRF keys which $\mathcal{B}$ does not know at the beginning, $\mathcal{B}$ samples all other terms of the NIAR scheme using the honest algorithms.
- Whenever $\mathcal{A}$ makes a corruption query for a receiver $u \in [n]$, $\mathcal{B}$ corrupts the corresponding PRF key with its own challenger $\mathcal{C}$. It then responds to $\mathcal{A}$ with u's secret key in the NIAR scheme.
- Whenever $\mathcal{A}$ makes a corruption query for a sender $u \in [n]$, $\mathcal{B}$ corrupts the corresponding PRF key for the receiver $\pi^{(0)}(u)$ with its own challenger $\mathcal{C}$. It then responds to $\mathcal{A}$ with u's secret key in the NIAR scheme. Additionally, it returns the random coins consumed by u's Enc algorithm in all previous time steps.
- Whenever $\mathcal{A}$ makes an encryption query for a time step t with the challenge plaintexts $\{x^{(0)}_{u,t}, x^{(1)}_{u,t}\}_{u\in\mathcal{H}_S}$, $\mathcal{B}$ computes the answers in the following way: for every $u \in \mathcal{H}_S$,
  - if $\pi^{(0)}(u)$ is conspicuously honest, then $\mathcal{B}$ submits to $\mathcal{C}$ a challenge query for the PRF instance corresponding to receiver $\pi^{(0)}(u)$, with the challenge message t, and it obtains the response c^*;
  - else it submits an evaluation query to $\mathcal{C}$ for the the PRF instance corresponding to receiver $\pi^{(0)}(u)$, with challenge message t and obtains the responses c^*;
  - $\mathcal{B}$ computes $\mathsf{ct}_{u,t} = \mathsf{NIAR}'.\mathsf{Enc}(\mathsf{ek}'_u, c^* \oplus x^{(0)}_{u,t}, t)$.

Clearly, if the selective-opening PRF challenger $\mathcal{C}$ always encrypts the real challenge message, then $\mathcal{A}$'s view is the same as in $\mathsf{NIARFull}^0$. Otherwise, if $\mathcal{C}$ always returns random strings for challenge queries, then $\mathcal{A}$'s view is the same as in Hyb_0. As mentioned earlier, if a receiver is conspicuously honest, then the senders paired with it in either $\pi^{(0)}$ or $\pi^{(1)}$ must be conspicuously honest too, meaning that an admissible adversary $\mathcal{A}$ will never corrupt it. Therefore, $\mathcal{B}$ is admissible w.r.t. to its own security game. Summarizing the above, $\mathcal{B}$ can translate $\mathcal{A}$'s advantage in distinguishing $\mathsf{NIARFull}^0$ and Hyb_0 into its own advantage in breaking the selective-opening PRF security.

□

Claim 4.9.6. *Suppose that the underlying* NIAR' *scheme satisfies Definition 4.3.2 subject to an adversary that corrupts all receivers upfront and can adaptively corrupt senders. Then, for any non-uniform p.p.t. admissible adversary* $\mathcal{A}$*, its views in* Hyb_0 *and* Hyb_1 *are computationally indistinguishable.*

Proof. In Hyb_b where $b \in \{0,1\}$, in each time step, to answer the adversary $\mathcal{A}$'s encryption query $\{x^{(0)}_{u,t}, x^{(1)}_{u,t}\}_{u\in\mathcal{H}_S}$ for the so-far honest set $\mathcal{H}_S$, the challenge does the following: for every $u \in \mathcal{H}_S$,

- either call $\mathsf{NIAR}'.\mathsf{Enc}(\mathsf{ek}_u, x^{(b)}_{u,t} \oplus \mathsf{PRF}.\mathbf{Eval}(k_{\pi^{(b)}(u)}, t), t)$ to encrypt the *inner-message* $x^{(b)}_{u,t} \oplus \mathsf{PRF}.\mathbf{Eval}(k_{\pi^{(b)}(u)}, t)$, where $k_{\pi^{(b)}(u)}$ denotes the PRF key of the receiver paired with sender u, which is also sender u's PRF key.
- or call $\mathsf{NIAR}'.\mathsf{Enc}(\mathsf{ek}_u, r, t)$ to encrypt some random inner-message r.

If $\mathcal{A}$ respects its admissibilty rules, then for each receiver u, then the following must hold:

1. For every eventually corrupt receiver u, let s_0 and s_1 be its corresponding senders in $\pi^{(0)}$ and $\pi^{(1)}$, respectively; then it must be that for every time step t, the inner-messages $x^{(0)}_{s_0,t} \oplus \mathsf{PRF}.\mathbf{Eval}(k_u, t)$, and $x^{(1)}_{s_1,t} \oplus \mathsf{PRF}.\mathbf{Eval}(k_u, t)$ (to be passed to $\mathsf{NIAR}'.\mathsf{Enc}$ in Hyb_0 and Hyb_1 respectively) are the same;
2. For every eventually honest receiver u,
   - For any t before u becomes conspicuously honest, the inner-messages $x^{(0)}_{s_0,t} \oplus \mathsf{PRF}.\mathbf{Eval}(k_u, t)$ and $x^{(1)}_{s_1,t} \oplus \mathsf{PRF}.\mathbf{Eval}(k_u, t)$ are the same;
   - For any t during or after the round in which u becomes conspicuously honest, the inner-messages to be passed to $\mathsf{NIAR}'.\mathsf{Enc}$ in Hyb_0 and Hyb_1 are both random inner-messages.

Now, imagine that the challenger has some randomness tape $\Gamma[\cdot, \cdot]$ which can be viewed as a two dimensional array. The randomness tape is sampled at the beginning of the experiment Hyb_b where $b \in \{0, 1\}$. Later, whenever the challenger needs to sample a random inner-message for some sender whose destination is u in $\pi^{(b)}$, it will read the random coins $\Gamma[u, t]$ off the randomness tape where t denotes the current time.

It suffices to prove that *conditioned on any fixed choice of the randomness tape* Γ, $\mathcal{A}$'s views in Hyb_0 and Hyb_1 are computationally indistinguishable. This can be accomplished through a straightforward reduction to the security of the underlying NIAR' scheme. Basically, consider a reduction $\mathcal{B}$ that interacts with its own challenger as well as $\mathcal{A}$.

- $\mathcal{B}$ passes the terms $(n, \mathsf{len}, \pi^{(0)}, \pi^{(1)})$ sent by $\mathcal{A}$ directly to its own challenger, and gets back tk. It passes tk to $\mathcal{A}$.
- Further, $\mathcal{B}$ chooses the PRF key k_u for every receiver u.
- $\mathcal{B}$ corrupts all receivers upfront with its own challenger, and receives $\{\mathsf{rk}'_u\}_{u \in [n]}$.
- Whenever $\mathcal{A}$ submits an encryption query during some time step t, $\mathcal{B}$ computes the two corresponding inner-messages and pass them to its own challenger. It receives some ciphertexts from its own challenger and forwards them to $\mathcal{A}$.

- Whenever $\mathcal{A}$ makes a corruption query for some receiver u, $\mathcal{B}$ sends to $\mathcal{A}$ its key (k_u, rk'_u).
- Whenever $\mathcal{A}$ makes a corruption query for some sender, $\mathcal{B}$ forwards the query to its own challenger and passes the answer back to $\mathcal{A}$. Moreover, it reveals the corresponding player's PRF key to $\mathcal{A}$. If $\mathcal{A}$ is admissible, then a corrupt sender must have the same receiver in both $\pi^{(0)}$ and $\pi^{(1)}$, so $\mathcal{B}$ can always identify a unique PRF key to return to $\mathcal{A}$.
- $\mathcal{B}$ outputs whatever $\mathcal{A}$ outputs.

Now, if $\mathcal{B}$'s challenger is in world $b = 0$, then $\mathcal{A}$'s view is identically distributed as Hyb_0. Else if $\mathcal{B}$'s challenger is in world $b = 1$, then $\mathcal{A}$'s view is identically distributed as Hyb_1. Further, for a fixed randomness tape, $\mathcal{B}$ is admissible as long as $\mathcal{A}$ is admissible. □

Completing the proof of Lemma 4.9.4. Combining Claims 4.9.5 and 4.9.6, we have that $\mathsf{NIARFull}^0 \approx_c \mathsf{Hyb}_0 \approx_c \mathsf{Hyb}_1 \approx_c \mathsf{NIARFull}^1$ where $\approx_c$ denotes computational indistinguishability. This completes the proof of Lemma 4.9.4. □

4.9.5 Completing the Proof of Theorem 4.9.1

Theorem 4.9.1 follows directly by combining Lemmas 4.9.2 to 4.9.4.

Chapter 5

Conclusion and Future Directions

In this thesis, we have explored several advanced topics in functional encryption, including decentralized multi-authority attribute-based encryption (MA-ABE) in Chapter 2, fair exchange with privacy in Chapter 3, and non-interactive attribute-based reveal (NIAR) in Chapter 4. Each of these chapters has introduced novel constructions and techniques that significantly advance the state-of-the-art in their respective areas. In this concluding chapter, we summarize the key contributions of each chapter and outline potential directions for future research.

Decentralized Access Control. Our contributions in Chapter 2 significantly advance the state-of-the-art of MA-ABE, in terms of security, efficiency and expressivity of access policies. Specifically, we showed that the classic Lewko-Waters [LW11a] construction is in fact fully secure against adaptive corruptions. Towards this, we presented a new security proof that is simpler than earlier work of Datta et al. [DKW23]. Building on this, we designed the first MA-ABE scheme for arithmetic span programs, which are more expressive than prior MA-ABE schemes.

Future Directions. While our new proof techniques enable several efficiency and expressivity improvements, there are still many open problems for making MA-ABE practical. On a high level, there are two main sources of inefficiency in making MA-ABE practical: (1) the overhead from converting a real-world access policy to an MA-ABE friendly intermediate representation (such as boolean formulas), and (2) the overhead from the cryptograhic parameters of the underlying MA-ABE scheme itself.

Addressing the first aspect on a technical level requires investigation what computation models can be efficiently supported by MA-ABE schemes. For single authority ABE, there has been significant prior work on supporting various computation models such as circuits, finite automata, Turing machines, and random access machines. But in the context of MA-ABE, without using heavy hammers such as iO, most of prior

work has only been able to support boolean span programs. Our work in this thesis makes a step forward by supporting arithmetic span programs. Future research can further explore what other computation models can be supported efficiently in MA-ABE.

Addressing the second aspect on a technical level requires investigating various features of MA-ABE schemes such as attribute reuse in policy, number of attributes per policy / user, attribute universe size, and security model. In the single-authority ABE setting, there has been significant prior work on designing schemes that optimize for these features, specifically schemes that simultaneously support multi-use attributes, large-universe, and unbounded number of attributes per policy / user key, and fully secure. But in the context of MA-ABE, prior work has been able to only achieve multi-use with full security (also our work in this thesis), but constructions supporting large-universe and unbounded attributes per policy / user key either are proven adaptively secure in the GGM model or selectively secure under q-type assumptions. Future research can further explore designing MA-ABE schemes that simultaneously achieve all these features with full security under standard assumptions.

Addressing these questions would significantly advance the foundations of decentralized access control, and I plan to pursue them as part of my future research agenda.

Fair Exchange. Our contributions in Chapter 3 significantly advance the state-of-the-art of fair exchange with privacy. Specifically, we presented FE inspired generalization of classical fair exchange, called functional fair exchange (FFE), and designed the first FFE scheme that simultaneously achieves fairness and strong privacy. Our construction is based on a novel combination of functional encryption and adaptor signatures. But it could only support linear functions. To support more general functions, we presented a second construction based on fully-homomorphic encryption (FHE) and adaptor signatures; and our key contribution was reducing the overheads from FHE to make the overall protocol seller-optimized in terms of computation and communication.

Future Directions. While our current constructions demonstrate both feasibility and strong performance improvements, achieving practical functional fair exchange remains an open challenge. More broadly, I aim to make fairness, like privacy and security, a first-class design principle in digital systems. Examples include preventing front-running and transaction manipulation in decentralized exchanges, enabling fair payment-for-inference in AI marketplaces, and guaranteeing output fairness in multi-party computation so that no participant gains an advantage by deviating. By addressing these challenges, I aim to develop a general cryptographic framework where fairness, like privacy and security, becomes a first-class design principle in modern digital systems.

Anonymous Communication. Our contributions in Chapter 4 significantly advance the state-of-the-art of NIAR. Specifically, we presented the first NIAR construction with sub-quadratic computation for the router. On a technical level, our key contribution was using a network of small iO-obfuscated programs to emulate the router's functionality, and we built new authentication techniques for arguing security of such a network.

Future Directions. Although our iO-based construction demonstrates the feasibility of quasi-linear NIAR, its reliance on iO makes it largely theoretical. To move towards practicality, I aim to revisit the multi-client FE (MCFE) based approach. A key bottleneck is that current bilinear-group-based MCFE schemes require bit-by-bit encryption. Future research would involve developing lattice-based function-hiding MCFE for linear functions, thus enabling compact encodings and plausibly post-quantum security. This is also closely related to the study of post-quantum iO, which is a major open problem in cryptography.

Bibliography

[ABDP15] Michel Abdalla, Florian Bourse, Angelo De Caro, and David Pointcheval. Simple functional encryption schemes for inner products. In *PKC*, 2015.

[Abe99] Masayuki Abe. Mix-networks on permutation networks. In *ASIACRYPT*, 1999.

[ABGW17] Miguel Ambrona, Gilles Barthe, Romain Gay, and Hoeteck Wee. Attribute-based encryption in the generic group model: Automated proofs and new constructions. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 647–664. ACM Press, October / November 2017.

[ac22] arkworks contributors. `arkworks` zksnark ecosystem, 2022. `https://arkworks.rs`.

[ACD+19] Ittai Abraham, T-H. Hubert Chan, Danny Dolev, Kartik Nayak, Rafael Pass, Ling Ren, and Elaine Shi. Communication complexity of byzantine agreement, revisited. In *PODC*, 2019.

[ACGSV24] Diego F Aranha, Anamaria Costache, Antonio Guimarães, and Eduardo Soria-Vazquez. Heliopolis: Verifiable computation over homomorphically encrypted data from interactive oracle proofs is practical. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 302–334. Springer, 2024.

[ACJ24] Rashmi Agrawal, Anantha Chandrakasan, and Ajay Joshi. Heap: A fully homomorphic encryption accelerator with parallelized bootstrapping. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 756–769. IEEE, 2024.

[ACL+22] Martin R Albrecht, Valerio Cini, Russell WF Lai, Giulio Malavolta, and Sri AravindaKrishnan Thyagarajan. Lattice-based snarks: publicly verifiable, preprocessing, and recursively composable. In *Annual International Cryptology Conference*, pages 102–132. Springer, 2022.

[ACN+20] Gilad Asharov, T.-H. Hubert Chan, Kartik Nayak, Rafael Pass, Ling Ren, and Elaine Shi. Bucket oblivious sort: An extremely simple oblivious sort. In *SOSA*, 2020.

[ACR17] Konstantin Avrachenkov, Pavel Chebotarev, and Dmytro Rubanov. Kernels on graphs as proximity measures. In *Algorithms and Models for the Web Graph: 14th International Workshop, WAW 2017*, 2017.

[AEE+21] Lukas Aumayr, Oguzhan Ersoy, Andreas Erwig, Sebastian Faust, Kristina Hostáková, Matteo Maffei, Pedro Moreno-Sanchez, and Siavash Riahi. Generalized channels from limited blockchain scripts and adaptor signatures. In *Advances in Cryptology–ASIACRYPT 2021: 27th International Conference on the Theory and Application of Cryptology and Information Security*, 2021.

[AFV11] Shweta Agrawal, David Mandell Freeman, and Vinod Vaikuntanathan. Functional encryption for inner product predicates from learning with errors. In Dong Hoon Lee and Xiaoyun Wang, editors, *ASIACRYPT 2011*, volume 7073 of *LNCS*, pages 21–40. Springer, Berlin, Heidelberg, December 2011.

[AG21] Miguel Ambrona and Romain Gay. Multi-authority ABE, revisited. Cryptology ePrint Archive, Report 2021/1381, 2021.

[AHY15] Nuttapong Attrapadung, Goichiro Hanaoka, and Shota Yamada. Conversions among several classes of predicate encryption and applications to ABE with various compactness tradeoffs. In Tetsu Iwata and Jung Hee Cheon, editors, *ASIACRYPT 2015, Part I*, volume 9452 of *LNCS*, pages 575–601. Springer, Berlin, Heidelberg, November / December 2015.

[AIK11] Benny Applebaum, Yuval Ishai, and Eyal Kushilevitz. How to garble arithmetic circuits. In Rafail Ostrovsky, editor, *52nd FOCS*, pages 120–129. IEEE Computer Society Press, October 2011.

[AJS17] Prabhanjan Ananth, Abhishek Jain, and Amit Sahai. Indistinguishability obfuscation for turing machines: constant overhead and amortization. In *Annual International Cryptology Conference*, pages 252–279. Springer, 2017.

[AKTZ17] Nikolaos Alexopoulos, Aggelos Kiayias, Riivo Talviste, and Thomas Zacharias. Mcmix: Anonymous messaging via secure multiparty computation. In *Usenix Security*, 2017.

[AKY24] Shweta Agrawal, Simran Kumari, and Shota Yamada. Attribute based encryption for turing machines from lattices. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part III*, volume 14922 of *LNCS*, pages 352–386. Springer, Cham, August 2024.

[ALMT20] Shweta Agrawal, Benoît Libert, Monosij Maitra, and Radu Titiu. Adaptive simulation security for inner product functional encryption. In *IACR International Conference on Public-Key Cryptography*, pages 34–64. Springer, 2020.

[ALS16] Shweta Agrawal, Benoît Libert, and Damien Stehlé. Fully secure functional encryption for inner products, from standard assumptions. In *CRYPTO*, 2016.

[Ama93] Shun-ichi Amari. Backpropagation and stochastic gradient descent method. *Neurocomputing*, 5(4-5):185–196, 1993.

[AMY19] Shweta Agrawal, Monosij Maitra, and Shota Yamada. Attribute based encryption (and more) for nondeterministic finite automata from LWE. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 765–797. Springer, Cham, August 2019.

[AMYY25] Shweta Agrawal, Anuja Modi, Anshu Yadav, and Shota Yamada. Evasive lwe: Attacks, variants & obfustopia. *Cryptology ePrint Archive*, 2025.

[APY20] Ittai Abraham, Benny Pinkas, and Avishay Yanai. Blinder: Mpc based scalable and robust anonymous committed broadcast. In *ACM CCS*, 2020.

[ARYY23] Shweta Agrawal, Mélissa Rossi, Anshu Yadav, and Shota Yamada. Constant input attribute based (and predicate) encryption from evasive and tensor LWE. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part IV*, volume 14084 of *LNCS*, pages 532–564. Springer, Cham, August 2023.

[AT20] Nuttapong Attrapadung and Junichi Tomida. Unbounded dynamic predicate compositions in ABE from standard assumptions. In Shiho Moriai and Huaxiong Wang, editors, *ASIACRYPT 2020, Part III*, volume 12493 of *LNCS*, pages 405–436. Springer, Cham, December 2020.

[Att14] Nuttapong Attrapadung. Dual system encryption via doubly selective security: Framework, fully secure functional encryption for regular languages, and more. In Phong Q. Nguyen and Elisabeth Oswald, editors, *EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 557–577. Springer, Berlin, Heidelberg, May 2014.

[Att16] Nuttapong Attrapadung. Dual system encryption framework in prime-order groups via computational pair encodings. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *ASIACRYPT 2016, Part II*, volume 10032 of *LNCS*, pages 591–623. Springer, Berlin, Heidelberg, December 2016.

[Att19] Nuttapong Attrapadung. Unbounded dynamic predicate compositions in attribute-based encryption. In Yuval Ishai and Vincent Rijmen, editors, *EUROCRYPT 2019, Part I*, volume 11476 of *LNCS*, pages 34–67. Springer, Cham, May 2019.

[AY20] Shweta Agrawal and Shota Yamada. Optimal broadcast encryption from pairings and LWE. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part I*, volume 12105 of *LNCS*, pages 13–43. Springer, Cham, May 2020.

[AYY22] Shweta Agrawal, Anshu Yadav, and Shota Yamada. Multi-input attribute based encryption and predicate encryption. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part I*, volume 13507 of *LNCS*, pages 590–621. Springer, Cham, August 2022.

[BB11] Dan Boneh and Xavier Boyen. Efficient selective identity-based encryption without random oracles. *Journal of Cryptology*, 24(4):659–693, October 2011.

[BBGN19a] Borja Balle, James Bell, Adrià Gascón, and Kobbi Nissim. Differentially private summation with multi-message shuffling. *CoRR*, abs/1906.09116, 2019.

[BBGN19b] Borja Balle, James Bell, Adrià Gascón, and Kobbi Nissim. The privacy blanket of the shuffle model. In *CRYPTO*, 2019.

[BBS04] Dan Boneh, Xavier Boyen, and Hovav Shacham. Short group signatures. In Matthew Franklin, editor, *CRYPTO 2004*, volume 3152 of *LNCS*, pages 41–55. Springer, Berlin, Heidelberg, August 2004.

[BCCT13] Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. Recursive composition and bootstrapping for SNARKs and proof-carrying data. In *Proceedings of the Forty-Fifth Annual ACM Symposium on Theory of Computing*, STOC '13. Association for Computing Machinery, 2013.

[BDGM20] Zvika Brakerski, Nico Dottling, Sanjam Garg, and Giulio Malavolta. Factoring and pairings are not necessary for io: Circular-secure lwe suffices. Cryptology ePrint Archive, Report 2020/1024, 2020.

[Bei96a] Amos Beimel. Secure schemes for secret sharing and key distribution. *PhD thesis, Israel Institute of Technology, Technion*, 1996.

[Bei96b] Amos Beimel. Secure schemes for secret sharing and key distribution. *PhD thesis, Israel Institute of Technology, Technion*, 1996.

[BEM+17] Andrea Bittau, Úlfar Erlingsson, Petros Maniatis, Ilya Mironov, Ananth Raghunathan, David Lie, Mitch Rudominer, Ushasree Kode, Julien Tinnés, and Bernhard Seefeld. Prochlo: Strong privacy for analytics in the crowd. *CoRR*, abs/1710.00901, 2017.

[BF01] Dan Boneh and Matthew K. Franklin. Identity-based encryption from the Weil pairing. In Joe Kilian, editor, *CRYPTO 2001*, volume 2139 of *LNCS*, pages 213–229. Springer, Berlin, Heidelberg, August 2001.

[BG12] Stephanie Bayer and Jens Groth. Efficient zero-knowledge argument for correctness of a shuffle. In *Eurocrypt*, volume 7237, pages 263–280, 2012.

[BGG+14] Dan Boneh, Craig Gentry, Sergey Gorbunov, Shai Halevi, Valeria Nikolaenko, Gil Segev, Vinod Vaikuntanathan, and Dhinakaran Vinayagamurthy. Fully key-homomorphic encryption, arithmetic circuit ABE and compact garbled circuits. In Phong Q. Nguyen and Elisabeth Oswald, editors, *EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 533–556. Springer, Berlin, Heidelberg, May 2014.

[BGI+01] Boaz Barak, Oded Goldreich, Rusell Impagliazzo, Steven Rudich, Amit Sahai, Salil Vadhan, and Ke Yang. On the (im) possibility of obfuscating programs. In *Annual international cryptology conference*, pages 1–18. Springer, 2001.

[BGI14] Elette Boyle, Shafi Goldwasser, and Ioana Ivan. Functional signatures and pseudorandom functions. In *International workshop on public key cryptography*, pages 501–519. Springer, 2014.

[BGKS25] Ruben Baecker, Paul Gerhart, Jonathan Katz, and Dominique Schröder. Fair exchange for decentralized autonomous organizations via threshold adaptor signatures. Cryptology ePrint Archive, Paper 2025/388, 2025.

[BGL+15] Nir Bitansky, Sanjam Garg, Huijia Lin, Rafael Pass, and Sidharth Telang. Succinct randomized encodings and their applications. In *Proceedings of the forty-seventh annual ACM symposium on Theory of Computing*, pages 439–448, 2015.

[BGN05] Dan Boneh, Eu-Jin Goh, and Kobbi Nissim. Evaluating 2-dnf formulas on ciphertexts. In Joe Kilian, editor, *Theory of Cryptography*, pages 325–341, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.

[BGR98] Mihir Bellare, Juan A Garay, and Tal Rabin. Fast batch verification for modular exponentiation and digital signatures. In *Advances in Cryptology—EUROCRYPT'98: International Conference on the Theory and Application of Cryptographic Techniques*, 1998.

[BGV11] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. Fully homomorphic encryption without bootstrapping. Cryptology ePrint Archive, Paper 2011/277, 2011.

[BHMS21] Benedikt Bünz, Yuncong Hu, Shin'ichiro Matsuo, and Elaine Shi. Non-interactive differentially anonymous router. Cryptology ePrint Archive, Paper 2021/1242, 2021. https://eprint.iacr.org/2021/1242.

[bip] Bip 340: Schnorr signatures for secp256k1. https://github.com/bitcoin/bips/blob/master/bip-0340/reference.py.

[BK14] Iddo Bentov and Ranjit Kumaresan. How to use bitcoin to design fair protocols. In *Annual Cryptology Conference*, 2014.

[BKO22] Paul Bunn, Eyal Kushilevitz, and Rafail Ostrovsky. Anonymous permutation routing. Cryptology ePrint Archive, Paper 2022/1353, 2022. https://eprint.iacr.org/2022/1353.

[BL88] Josh Cohen Benaloh and Jerry Leichter. Generalized secret sharing and monotone functions. In *Proceedings of the 8th Annual International Cryptology Conference on Advances in Cryptology*, pages 27–35, 1988.

[Boy13] Xavier Boyen. Attribute-based functional encryption on lattices. In Amit Sahai, editor, *TCC 2013*, volume 7785 of *LNCS*, pages 122–142. Springer, Berlin, Heidelberg, March 2013.

[bri] Blockchain bridges. https://ethereum.org/en/bridges/.

[BS17] Dan Boneh and Victor Shoup. A graduate course in applied cryptography, 2017.

[BSW07] John Bethencourt, Amit Sahai, and Brent Waters. Ciphertext-policy attribute-based encryption. In *2007 IEEE Symposium on Security and Privacy*, pages 321–334. IEEE Computer Society Press, May 2007.

[BSW11] Dan Boneh, Amit Sahai, and Brent Waters. Functional encryption: Definitions and challenges. In *TCC*, 2011.

[BSW16] Mihir Bellare, Igors Stepanovs, and Brent Waters. New negative results on differing-inputs obfuscation. In Marc Fischlin and Jean-Sébastien Coron, editors, *Advances in Cryptology – EUROCRYPT 2016*, pages 792–821, Berlin, Heidelberg, 2016. Springer Berlin Heidelberg.

[BÜW24] Chris Brzuska, Akin Ünal, and Ivy K. Y. Woo. Evasive LWE assumptions: Definitions, classes, and counterexamples. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part IV*, volume 15487 of *LNCS*, pages 418–449. Springer, Singapore, December 2024.

[BV16] Zvika Brakerski and Vinod Vaikuntanathan. Circuit-ABE from LWE: Unbounded attributes and semi-adaptive security. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part III*, volume 9816 of *LNCS*, pages 363–384. Springer, Berlin, Heidelberg, August 2016.

[BV20] Zvika Brakerski and Vinod Vaikuntanathan. Lattice-inspired broadcast encryption and succinct ciphertext-policy ABE. Cryptology ePrint Archive, Report 2020/191, 2020.

[BW13] Dan Boneh and Brent Waters. Constrained pseudorandom functions and their applications. In *International conference on the theory and application of cryptology and information security*, pages 280–300. Springer, 2013.

[CBM15] Henry Corrigan-Gibbs, Dan Boneh, and David Mazières. Riposte: An anonymous messaging system handling millions of users. In *S & P*, 2015.

[CC09] Melissa Chase and Sherman S. M. Chow. Improving privacy and security in multi-authority attribute-based encryption. In Ehab Al-Shaer, Somesh Jha, and Angelos D. Keromytis, editors, *ACM CCS 2009*, pages 121–130. ACM Press, November 2009.

[CCG+23] Jie Chen, Qiaohan Chu, Ying Gao, Jianting Ning, and Luping Wang. Improved fully adaptive decentralized MA-ABE for NC1 from MDDH. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part V*, volume 14442 of *LNCS*, pages 3–32. Springer, Singapore, December 2023.

[CCH+19] Ran Canetti, Yilei Chen, Justin Holmgren, Alex Lombardi, Guy N Rothblum, Ron D Rothblum, and Daniel Wichs. Fiat-shamir: from practice to theory. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*, 2019.

[CD00] Jan Camenisch and Ivan Damgård. Verifiable encryption, group encryption, and their applications to separable group signatures and signature sharing schemes. volume 5, pages 331–345, 12 2000.

[CG97] Benny Chor and Niv Gilboa. Computationally private information retrieval (extended abstract). In *Proceedings of the twenty-ninth annual ACM symposium on Theory of computing*, STOC '97, pages 304–313, New York, NY, USA, 1997. ACM.

[CGF10] Henry Corrigan-Gibbs and Bryan Ford. Dissent: Accountable anonymous group messaging. In *CCS*, page 340–350, 2010.

[CGGI20] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Tfhe: fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1):34–91, 2020.

[CGKS95] Benny Chor, Oded Goldreich, Eyal Kushilevitz, and Madhu Sudan. Private information retrieval. In *IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 41–50, 1995.

[CGKW18] Jie Chen, Junqing Gong, Lucas Kowalczyk, and Hoeteck Wee. Unbounded ABE via bilinear entropy expansion, revisited. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part I*, volume 10820 of *LNCS*, pages 503–534. Springer, Cham, April / May 2018.

[CGW15] Jie Chen, Romain Gay, and Hoeteck Wee. Improved dual system ABE in prime-order groups via predicate encodings. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 595–624. Springer, Berlin, Heidelberg, April 2015.

[cha] Oracle network for smart contracts. https://shorturl.at/fpDEZ.

[Cha81] David L. Chaum. Untraceable electronic mail, return addresses, and digital pseudonyms. *Communications of the ACM*, 24(2):84–90, February 1981.

[Cha88] David L. Chaum. The dining cryptographers problem: Unconditional sender and recipient untraceability. *Journal of Cryptology*, 1(1):65–75, March 1988.

[Cha07] Melissa Chase. Multi-authority attribute based encryption. In Salil P. Vadhan, editor, *TCC 2007*, volume 4392 of *LNCS*, pages 515–534. Springer, Berlin, Heidelberg, February 2007.

[CHJV14] Ran Canetti, Justin Holmgren, Abhishek Jain, and Vinod Vaikuntanathan. Indistinguishability obfuscation of iterated circuits and ram programs. *Cryptology ePrint Archive*, 2014.

[CHK03] Ran Canetti, Shai Halevi, and Jonathan Katz. A forward-secure public-key encryption scheme. In *Advances in Cryptology—EUROCRYPT 2003: International Conference on the Theory and Applications of Cryptographic Techniques, Warsaw, Poland, May 4–8, 2003 Proceedings 22*, pages 255–271. Springer, 2003.

[CL05] Jan Camenisch and Anna Lysyanskaya. A formal treatment of onion routing. In *CRYPTO*, 2005.

[CLTV15] Ran Canetti, Huijia Lin, Stefano Tessaro, and Vinod Vaikuntanathan. Obfuscation of probabilistic circuits and applications. In Yevgeniy Dodis and Jesper Buus Nielsen, editors, *Theory of Cryptography*, pages 468–497, Berlin, Heidelberg, 2015. Springer Berlin Heidelberg.

[CLW25] Valerio Cini, Russell Lai, and Ivy Woo. Lattice-based multi-authority/client attribute-based encryption for circuits. *IACR Communications in Cryptology*, 1(4):1–67, 2025.

[cona] https://github.com/zama-ai/concrete.

[conb] https://github.com/zama-ai/concrete-ml.

[COPZ22] Melissa Chase, Michele Orrù, Trevor Perrin, and Greg Zaverucha. Proofs of discrete logarithm equality across groups. Cryptology ePrint Archive, Paper 2022/1593, 2022.

[CS02] Ronald Cramer and Victor Shoup. Universal hash proofs and a paradigm for adaptive chosen ciphertext secure public-key encryption. In Lars R. Knudsen, editor, *EUROCRYPT 2002*, volume 2332 of *LNCS*, pages 45–64. Springer, Berlin, Heidelberg, April / May 2002.

[CSU+19] Albert Cheu, Adam Smith, Jonathan Ullman, David Zeber, and Maxim Zhilyaev. Distributed differential privacy via shuffling, 04 2019.

[CW77] JL Carter and MN Wegman. Universal classes of hash functions (extended abstract), stoc'77: Proceedings of the ninth annual acm symposium on theory of computing, 1977.

[CW23] Valerio Cini and Hoeteck Wee. ABE for circuits with poly (λ)-sized keys from LWE. In *64th FOCS*, pages 435–446. IEEE Computer Society Press, November 2023.

[CW24] Valerio Cini and Hoeteck Wee. Unbounded ABE for circuits from LWE, revisited. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part IV*, volume 15487 of *LNCS*, pages 238–267. Springer, Singapore, December 2024.

[CW25] Valerio Cini and Hoeteck Wee. Faster abe for turing machines from circular evasive lwe. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 94–125. Springer, 2025.

[CZK+23] Dake Chen, Yuke Zhang, Souvik Kundu, Chenghao Li, and Peter A. Beerel. Rna-vit: Reduced-dimension approximate normalized attention vision transformers for latency efficient private inference. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, pages 1–9, 2023.

[DHM+24] Fangqi Dong, Zihan Hao, Ethan Mook, Hoeteck Wee, and Daniel Wichs. Laconic function evaluation and ABE for RAMs from (ring-)LWE. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part III*, volume 14922 of *LNCS*, pages 107–142. Springer, Cham, August 2024.

[DKL+18] Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium: A lattice-based digital signature scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 238–268, 2018.

[DKW21] Pratish Datta, Ilan Komargodski, and Brent Waters. Decentralized multi-authority ABE for DNFs from LWE. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part I*, volume 12696 of *LNCS*, pages 177–209. Springer, Cham, October 2021.

[DKW23] Pratish Datta, Ilan Komargodski, and Brent Waters. Fully adaptive decentralized multi-authority ABE. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part III*, volume 14006 of *LNCS*, pages 447–478. Springer, Cham, April 2023.

[dlPVA22] Antonio de la Piedra, Marloes Venema, and Greg Alpár. ABE squared: Accurately benchmarking efficiency of attribute-based encryption. *IACR TCHES*, 2022(2):192–239, 2022.

[dlPVA23] Antonio de la Piedra, Marloes Venema, and Greg Alpár. ACABELLA: Automated (crypt)analysis of attribute-based encryption leveraging linear algebra. In Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, editors, *ACM CCS 2023*, pages 3269–3283. ACM Press, November 2023.

[DLZ+23] Ye Dong, Wen-jie Lu, Yancheng Zheng, Haoqi Wu, Derun Zhao, Jin Tan, Zhicong Huang, Cheng Hong, Tao Wei, and Wenguang Chen. Puma: Secure inference of llama-7b in five minutes. *arXiv preprint arXiv:2307.12533*, 2023.

[DMNS06] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. Calibrating noise to sensitivity in private data analysis. In *TCC*, 2006.

[DMS04] Roger Dingledine, Nick Mathewson, and Paul Syverson. Tor: The second-generation onion router. In *USENIX Security Symposium*, 2004.

[DMVVZ25] Pierpaolo Della Monica, Ivan Visconti, Andrea Vitaletti, and Marco Zecchini. Public channel-based fair exchange protocols with advertising. *arXiv preprint arXiv:2503.10411*, 2025.

[DOT19] Pratish Datta, Tatsuaki Okamoto, and Katsuyuki Takashima. Efficient attribute-based signatures for unbounded arithmetic branching programs. In Dongdai Lin and Kazue Sako, editors, *PKC 2019, Part I*, volume 11442 of *LNCS*, pages 127–158. Springer, Cham, April 2019.

[DOY22] Wei Dai, Tatsuaki Okamoto, and Go Yamamoto. Stronger security and generic constructions for adaptor signatures. In *International Conference on Cryptology in India*, pages 52–77. Springer, 2022.

[DS18] Jean Paul Degabriele and Martijn Stam. Untagging tor: A formal treatment of onion encryption. In *EUROCRYPT*, 2018.

[DTV25] Pratish Datta, Junichi Tomida, and Nikhil Vanjani. Fully adaptive decentralized MA-ABE: Simplified, optimized, ASP supported. In *Asiacrypt*, 2025.

[EEE20a] Muhammed F Esgin, Oğuzhan Ersoy, and Zekeriya Erkin. Post-quantum adaptor signatures and payment channel networks. In *European Symposium on Research in Computer Security*, pages 378–397. Springer, 2020.

[EEE20b] Muhammed F. Esgin, Oğuzhan Ersoy, and Zekeriya Erkin. Post-quantum adaptor signatures and payment channel networks. In Liqun Chen, Ninghui Li, Kaitai Liang, and Steve Schneider, editors, *Computer Security – ESORICS 2020*, pages 378–397, Cham, 2020. Springer International Publishing.

[EFH+21] Andreas Erwig, Sebastian Faust, Kristina Hostáková, Monosij Maitra, and Siavash Riahi. Two-party adaptor signatures from identification schemes. In *IACR International Conference on Public-Key Cryptography*, pages 451–480. Springer, 2021.

[EFM+19] Úlfar Erlingsson, Vitaly Feldman, Ilya Mironov, Ananth Raghunathan, Kunal Talwar, and Abhradeep Thakurta. Amplification by shuffling: From local to central differential privacy via anonymity. In *SODA*, 2019.

[EHK+13] Alex Escala, Gottfried Herold, Eike Kiltz, Carla Rafols, and Jorge Luis Villar. An algebraic framework for Diffie-Hellman assumptions. In *CRYPTO*, 2013.

[ETS18] ETSI. *CYBER; Application of Attribute Based Encryption (ABE) for PII and personal data protection on IoT devices, WLAN, cloud and mobile services - High level requirements 2018*. ETSI TS 103 458, 2018.

[fil] filecoin-project/blstrs. https://github.com/filecoin-project/blstrs.

[FLS90] Uriel Feige, Dror Lapidot, and Adi Shamir. Multiple non-interactive zero knowledge proofs based on a single random string. In *Proceedings [1990] 31st Annual Symposium on Foundations of Computer Science*, pages 308–317. IEEE, 1990.

[Fre10] David Mandell Freeman. Converting pairing-based cryptosystems from composite-order groups to prime-order groups. In Henri Gilbert, editor, *EUROCRYPT 2010*, volume 6110 of *LNCS*, pages 44–61. Springer, Berlin, Heidelberg, May / June 2010.

[FSS+23] Rex Fernando, Elaine Shi, Pratik Soni, Nikhil Vanjani, and Brent Waters. Non-interactive anonymous router with quasi-linear router computation. In *Theory of Cryptography Conference*, pages 62–92. Springer, 2023.

[Für07] Martin Fürer. Faster integer multiplication. In *Proceedings of the thirty-ninth annual ACM symposium on Theory of computing*, pages 57–66, 2007.

[GDCC16] Junqing Gong, Xiaolei Dong, Jie Chen, and Zhenfu Cao. Efficient IBE with tight reduction to standard assumption in the multi-challenge setting. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *ASIACRYPT 2016, Part II*, volume 10032 of *LNCS*, pages 624–654. Springer, Berlin, Heidelberg, December 2016.

[GGH+13a] Sanjam Garg, Craig Gentry, Shai Halevi, Mariana Raykova, Amit Sahai, and Brent Waters. Candidate indistinguishability obfuscation and functional encryption for all circuits. In *IEEE Symposium on Foundations of Computer Science (FOCS)*, 2013.

[GGH+13b] Sanjam Garg, Craig Gentry, Shai Halevi, Amit Sahai, and Brent Waters. Attribute-based encryption for circuits from multilinear maps. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part II*, volume 8043 of *LNCS*, pages 479–499. Springer, Berlin, Heidelberg, August 2013.

[GGL24] Rachit Garg, Rishab Goyal, and George Lu. Dynamic collusion functional encryption and multi-authority attribute-based encryption. In Qiang Tang and Vanessa Teague, editors, *PKC 2024, Part II*, volume 14604 of *LNCS*, pages 69–104. Springer, Cham, April 2024.

[GGM86] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions. *Journal of the ACM (JACM)*, 33(4):792–807, 1986.

[GGW24] Sanjam Garg, Aarushi Goel, and Mingyuan Wang. How to prove statements obliviously? In *Annual International Cryptology Conference*, pages 449–487. Springer, 2024.

[GHKW16] Romain Gay, Dennis Hofheinz, Eike Kiltz, and Hoeteck Wee. Tightly CCA-secure encryption without pairings. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part I*, volume 9665 of *LNCS*, pages 1–27. Springer, Berlin, Heidelberg, May 2016.

[GIKM00] Yael Gertner, Yuval Ishai, Eyal Kushilevitz, and Tal Malkin. Protecting data privacy in private information retrieval schemes. *J. Comput. Syst. Sci.*, 60(3), 2000.

[GKLX22] S. Dov Gordon, Jonathan Katz, Mingyu Liang, and Jiayu Xu. Spreading the privacy blanket: Differentially oblivious shuffling for differential privacy. In *ACNS*, 2022.

[GKW17] Rishab Goyal, Venkata Koppula, and Brent Waters. Lockable obfuscation. In Chris Umans, editor, *58th FOCS*, pages 612–621. IEEE Computer Society Press, October 2017.

[GMR88] Shafi Goldwasser, Silvio Micali, and Ronald L Rivest. A digital signature scheme secure against adaptive chosen-message attacks. *SIAM Journal on computing*, 17(2):281–308, 1988.

[GMR89] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989.

[GOS06] Jens Groth, Rafail Ostrovsky, and Amit Sahai. Non-interactive zaps and new techniques for nizk. In *Advances in Cryptology-CRYPTO 2006: 26th Annual International Cryptology Conference, Santa Barbara, California, USA, August 20-24, 2006. Proceedings 26*, pages 97–111. Springer, 2006.

[GP20] Romain Gay and Rafael Pass. Indistinguishability obfuscation from circular security. Cryptology ePrint Archive, Report 2020/1010, 2020.

[GPSW06] Vipul Goyal, Omkant Pandey, Amit Sahai, and Brent Waters. Attribute-based encryption for fine-grained access control of encrypted data. In Ari Juels, Rebecca N. Wright, and Sabrina De Capitani di Vimercati, editors, *ACM CCS 2006*, pages 89–98. ACM Press, October / November 2006. Available as Cryptology ePrint Archive Report 2006/309.

[GPV19] Badih Ghazi, Rasmus Pagh, and Ameya Velingker. Scalable and differentially private distributed aggregation in the shuffled model. *CoRR*, abs/1906.08320, 2019.

[GRS99] David Goldschlag, Michael Reed, and Paul Syverson. Onion routing for anonymous and private internet connections. *Communications of the ACM*, 42:39–41, 1999.

[GSST24] Paul Gerhart, Dominique Schröder, Pratik Soni, and Sri AravindaKrishnan Thyagarajan. Foundations of adaptor signatures. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024*, 2024.

[GSW13] Craig Gentry, Amit Sahai, and Brent Waters. Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. Cryptology ePrint Archive, Paper 2013/340, 2013.

[Gui13] Aurore Guillevic. Comparing the pairing efficiency over composite-order and prime-order elliptic curves. In Michael J. Jacobson, Jr., Michael E. Locasto, Payman Mohassel, and Reihaneh

Safavi-Naini, editors, *ACNS 13International Conference on Applied Cryptography and Network Security*, volume 7954 of *LNCS*, pages 357–372. Springer, Berlin, Heidelberg, June 2013.

[GVW13] Sergey Gorbunov, Vinod Vaikuntanathan, and Hoeteck Wee. Attribute-based encryption for circuits. In Dan Boneh, Tim Roughgarden, and Joan Feigenbaum, editors, *45th ACM STOC*, pages 545–554. ACM Press, June 2013.

[GW20] Junqing Gong and Hoeteck Wee. Adaptively secure ABE for DFA from k-Lin and more. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part III*, volume 12107 of *LNCS*, pages 278–308. Springer, Cham, May 2020.

[GWW19] Junqing Gong, Brent Waters, and Hoeteck Wee. ABE for DFA from k-Lin. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 732–764. Springer, Cham, August 2019.

[GWZ22] Jiaxin Guan, Daniel Wichs, and Mark Zhandry. Incompressible cryptography. In *Advances in Cryptology–EUROCRYPT 2022: 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Trondheim, Norway, May 30–June 3, 2022, Proceedings, Part I*, pages 700–730. Springer, 2022.

[han] Secp256k1 python. https://github.com/hanabi1224/Programming-Language-Benchmarks/blob/main/bench/algorithm/secp256k1/1.py.

[HEK12] Yan Huang, David Evans, and Jonathan Katz. Private set intersection: Are garbled circuits better than custom protocols? In *NDSS*, 2012.

[HIJ+17] Shai Halevi, Yuval Ishai, Abhishek Jain, Ilan Komargodski, Amit Sahai, and Eylon Yogev. Non-interactive multiparty computation without correlated randomness. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology – ASIACRYPT 2017*, pages 181–211, Cham, 2017. Springer International Publishing.

[HJL25] Yao-Ching Hsieh, Aayush Jain, and Huijia Lin. Lattice-based post-quantum io from circular security with random opening assumption (part ii: zeroizing attacks against private-coin evasive lwe assumptions). *Cryptology ePrint Archive*, 2025.

[HKLR05] Mohammad T. Hajiaghayi, Robert D. Kleinberg, Tom Leighton, and Harald Räcke. Oblivious routing on node-capacitated and directed graphs. In *SODA*, 2005.

[HLL23] Yao-Ching Hsieh, Huijia Lin, and Ji Luo. Attribute-based encryption for circuits of unbounded depth from lattices. In *64th FOCS*, pages 415–434. IEEE Computer Society Press, November 2023.

[HLL24] Yao-Ching Hsieh, Huijia Lin, and Ji Luo. A general framework for lattice-based ABE using evasive inner-product functional encryption. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part II*, volume 14652 of *LNCS*, pages 433–464. Springer, Cham, May 2024.

[HLTW24] Lucjan Hanzlik, Julian Loss, Sri AravindaKrishnan Thyagarajan, and Benedikt Wagner. Sweep-uc: Swapping coins privately. In *2024 IEEE Symposium on Security and Privacy (SP)*, 2024.

[Hol19] Justin Holmgren. On round-by-round soundness and state restoration attacks. Cryptology ePrint Archive, Paper 2019/1261, 2019.

[IW14a] Yuval Ishai and Hoeteck Wee. Partial garbling schemes and their applications. In Javier Esparza, Pierre Fraigniaud, Thore Husfeldt, and Elias Koutsoupias, editors, *ICALP 2014, Part I*, volume 8572 of *LNCS*, pages 650–662. Springer, Berlin, Heidelberg, July 2014.

[IW14b] Yuval Ishai and Hoeteck Wee. Partial garbling schemes and their applications. In *International Colloquium on Automata, Languages, and Programming*, pages 650–662. Springer, 2014.

[JCT+25] Siddharth Jayashankar, Edward Chen, Tom Tang, Wenting Zheng, and Dimitrios Skarlatos. Cinnamon: A framework for scale-out encrypted ai. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, pages 133–150, 2025.

[JJ21] Abhishek Jain and Zhengzhong Jin. Non-interactive zero knowledge from sub-exponential ddh. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 3–32. Springer, 2021.

[JJ22] Abhishek Jain and Zhengzhong Jin. Indistinguishability obfuscation via mathematical proofs of equivalence. In *63rd IEEE Annual Symposium on Foundations of Computer Science, FOCS 2022, Denver, CO, USA, October 31 - November 3, 2022*, pages 1023–1034. IEEE, 2022.

[JKK+17] Zahra Jafargholi, Chethan Kamath, Karen Klein, Ilan Komargodski, Krzysztof Pietrzak, and Daniel Wichs. Be adaptive, avoid overcommitting. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part I*, volume 10401 of *LNCS*, pages 133–163. Springer, Cham, August 2017.

[JLL23] Aayush Jain, Huijia Lin, and Ji Luo. On the optimal succinctness and efficiency of functional encryption and attribute-based encryption. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part III*, volume 14006 of *LNCS*, pages 479–510. Springer, Cham, April 2023.

[JLS21] Aayush Jain, Huijia Lin, and Amit Sahai. Indistinguishability obfuscation from well-founded assumptions. In *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing*, pages 60–73, 2021.

[JW16] Zahra Jafargholi and Daniel Wichs. Adaptive security of yao's garbled circuits. Cryptology ePrint Archive, Report 2016/814, 2016.

[kag] https://www.kaggle.com/datasets/erdemtaha/cancer-data.

[KKC+23] Jongmin Kim, Sangpyo Kim, Jaewan Choi, Jaiyoung Park, Donghwan Kim, and Jung Ho Ahn. Sharp: A short-word hierarchical accelerator for robust and practical fully homomorphic encryption. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, pages 1–15, 2023.

[KLW15] Venkata Koppula, Allison Bishop Lewko, and Brent Waters. Indistinguishability obfuscation for turing machines with unbounded memory. In *Proceedings of the forty-seventh annual ACM symposium on Theory of Computing*, pages 419–428, 2015.

[KOS16] Marcel Keller, Emmanuela Orsini, and Peter Scholl. MASCOT: Faster malicious arithmetic secure computation with oblivious transfer. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 830–842. ACM Press, October 2016.

[KPTZ13] Aggelos Kiayias, Stavros Papadopoulos, Nikos Triandopoulos, and Thomas Zacharias. Delegatable pseudorandom functions and applications. In *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*, pages 669–684, 2013.

[KRA+22] Mohammad Monirujjaman Khan, Nesat Tasneem RoJa, Faris A Almalki, Maha Aljohani, et al. Revolutionizing e-commerce using blockchain technology and implementing smart contract. *Security and Communication Networks*, 2022.

[KW93] Mauricio Karchmer and Avi Wigderson. On span programs. In *Proceedings of Structures in Complexity Theory*, pages 102–111, 1993.

[KW19] Lucas Kowalczyk and Hoeteck Wee. Compact adaptively secure ABE for NC^1 from k-Lin. In Yuval Ishai and Vincent Rijmen, editors, *EUROCRYPT 2019, Part I*, volume 11476 of *LNCS*, pages 3–33. Springer, Cham, May 2019.

[KW20] Lucas Kowalczyk and Hoeteck Wee. Compact adaptively secure ABE for NC^1 from k-Lin. *Journal of Cryptology*, 33(3):954–1002, July 2020.

[LCLS08] Huang Lin, Zhenfu Cao, Xiaohui Liang, and Jun Shao. Secure threshold multi authority attribute based encryption without a central authority. In Dipanwita Roy Chowdhury, Vincent Rijmen, and Abhijit Das, editors, *INDOCRYPT 2008*, volume 5365 of *LNCS*, pages 426–436. Springer, Berlin, Heidelberg, December 2008.

[LHG+23] Wen-jie Lu, Zhicong Huang, Zhen Gu, Jingyu Li, Jian Liu, Cheng Hong, Kui Ren, Tao Wei, and WenGuang Chen. Bumblebee: Secure two-party inference framework for large transformers. *Cryptology ePrint Archive*, 2023.

[liv] Ethereum gas cost. https://milkroad.com/ethereum/gas/.

[LL20a] Huijia Lin and Ji Luo. Compact adaptively secure ABE from k-Lin: Beyond NC^1 and towards NL. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part III*, volume 12107 of *LNCS*, pages 247–277. Springer, Cham, May 2020.

[LL20b] Huijia Lin and Ji Luo. Succinct and adaptively secure ABE for ABP from k-Lin. In Shiho Moriai and Huaxiong Wang, editors, *ASIACRYPT 2020, Part III*, volume 12493 of *LNCS*, pages 437–466. Springer, Cham, December 2020.

[LLL22] Hanjun Li, Huijia Lin, and Ji Luo. ABE for circuits with constant-size secret keys and adaptive security. In Eike Kiltz and Vinod Vaikuntanathan, editors, *TCC 2022, Part I*, volume 13747 of *LNCS*, pages 680–710. Springer, Cham, November 2022.

[LOS+10] Allison B. Lewko, Tatsuaki Okamoto, Amit Sahai, Katsuyuki Takashima, and Brent Waters. Fully secure functional encryption: Attribute-based encryption and (hierarchical) inner product encryption. In Henri Gilbert, editor, *EUROCRYPT 2010*, volume 6110 of *LNCS*, pages 62–91. Springer, Berlin, Heidelberg, May / June 2010.

[LVV+23] Watson Ladd, Tanya Verma, Marloes Venema, Armando Faz-Hernández, Brendan McMillion, Avani Wildani, and Nick Sullivan. Portunus: Re-imagining access control in distributed systems. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, pages 35–52, 2023.

[LW10] Allison B. Lewko and Brent Waters. New techniques for dual system encryption and fully secure HIBE with short ciphertexts. In Daniele Micciancio, editor, *TCC 2010*, volume 5978 of *LNCS*, pages 455–479. Springer, Berlin, Heidelberg, February 2010.

[LW11a] Allison B. Lewko and Brent Waters. Decentralizing attribute-based encryption. In Kenneth G. Paterson, editor, *EUROCRYPT 2011*, volume 6632 of *LNCS*, pages 568–588. Springer, Berlin, Heidelberg, May 2011.

[LW11b] Allison B. Lewko and Brent Waters. Unbounded HIBE and attribute-based encryption. In Kenneth G. Paterson, editor, *EUROCRYPT 2011*, volume 6632 of *LNCS*, pages 547–567. Springer, Berlin, Heidelberg, May 2011.

[LW12] Allison B. Lewko and Brent Waters. New proof methods for attribute-based encryption: Achieving full security through selective techniques. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 180–198. Springer, Berlin, Heidelberg, August 2012.

[Lyn] Ben Lynn. Pbc library. https://crypto.stanford.edu/pbc/.

[Lyu12] Vadim Lyubashevsky. Lattice signatures without trapdoors. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 738–755. Springer, 2012.

[MKE09a] Sascha Müller, Stefan Katzenbeisser, and Claudia Eckert. Distributed attribute-based encryption. In Pil Joong Lee and Jung Hee Cheon, editors, *ICISC 08*, volume 5461 of *LNCS*, pages 20–36. Springer, Berlin, Heidelberg, December 2009.

[MKE09b] Sascha Muller, Stefan Katzenbeisser, and Claudia Eckert. On multi-authority ciphertext-policy attribute-based encryption. *Bulletin of the Korean Mathematical Society*, 46(4):803–819, 2009.

[MMSS+19] Giulio Malavolta, Pedro Moreno-Sanchez, Clara Schneidewind, Aniket Kate, and Matteo Maffei. Anonymous multi-hop locks for blockchain scalability and interoperability. In *Proceedings 2019 Network and Distributed System Security Symposium*. Internet Society, 2019.

[OS97] Rafail Ostrovsky and Victor Shoup. Private information storage (extended abstract). In *STOC*, pages 294–303, 1997.

[OSW07] Rafail Ostrovsky, Amit Sahai, and Brent Waters. Attribute-based encryption with non-monotonic access structures. In Peng Ning, Sabrina De Capitani di Vimercati, and Paul F. Syverson, editors, *ACM CCS 2007*, pages 195–203. ACM Press, October 2007.

[OT10] Tatsuaki Okamoto and Katsuyuki Takashima. Fully secure functional encryption with general relations from the decisional linear assumption. In Tal Rabin, editor, *CRYPTO 2010*, volume 6223 of *LNCS*, pages 191–208. Springer, Berlin, Heidelberg, August 2010.

[OT12] Tatsuaki Okamoto and Katsuyuki Takashima. Fully secure unbounded inner-product and attribute-based encryption. In Xiaoyun Wang and Kazue Sako, editors, *ASIACRYPT 2012*, volume 7658 of *LNCS*, pages 349–366. Springer, Berlin, Heidelberg, December 2012.

[OT20a] Tatsuaki Okamoto and Katsuyuki Takashima. Decentralized attribute-based encryption and signatures. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, 103(1):41–73, 2020.

[OT20b] Tatsuaki Okamoto and Katsuyuki Takashima. Decentralized attribute-based encryption and signatures. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, 103(1):41–73, 2020.

[PB23] René Peralta and Luís TAN Brandão. NIST first call for multi-party threshold schemes. 2023.

[PD] Joseph Poon and Thaddeus Dryja. The bitcoin lightning network: Scalable off-chain instant payments.

[PHGR13] Bryan Parno, Jon Howell, Craig Gentry, and Mariana Raykova. Pinocchio: Nearly practical verifiable computation. In *2013 IEEE Symposium on Security and Privacy*, pages 238–252. IEEE Computer Society Press, May 2013.

[PS19] Chris Peikert and Sina Shiehian. Noninteractive zero knowledge for NP from (plain) learning with errors. In *Annual International Cryptology Conference*, 2019.

[QPM+23] Xianrui Qin, Shimin Pan, Arash Mirzaei, Zhimei Sui, Oğuzhan Ersoy, Amin Sakzad, Muhammed F Esgin, Joseph K Liu, Jiangshan Yu, and Tsz Hon Yuen. Blindhub: Bitcoin-compatible privacy-preserving payment channel hubs supporting variable amounts. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 2462–2480. IEEE, 2023.

[RÖ02] Harald Räcke. Minimizing congestion in general networks. In *The 43rd Annual IEEE Symposium on Foundations of Computer Science, 2002. Proceedings.*, pages 43–52. IEEE, 2002.

[Ren10] Steffen Rendle. Factorization machines. In *2010 IEEE International Conference on Data Mining*, pages 995–1000, 2010.

[RS21] Vijaya Ramachandran and Elaine Shi. Data oblivious algorithms for multicores. In Kunal Agrawal and Yossi Azar, editors, *SPAA '21: 33rd ACM Symposium on Parallelism in Algorithms and Architectures, Virtual Event, USA, 6-8 July, 2021*, pages 373–384. ACM, 2021.

[RW15] Yannis Rouselakis and Brent Waters. Efficient statically-secure large-universe multi-authority attribute-based encryption. In Rainer Böhme and Tatsuaki Okamoto, editors, *FC 2015*, volume 8975 of *LNCS*, pages 315–332. Springer, Berlin, Heidelberg, January 2015.

[RW22] Doreen Riepel and Hoeteck Wee. FABEO: Fast attribute-based encryption with optimal security. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 2491–2504. ACM Press, November 2022.

[SA19] Nigel P. Smart and Younes Talibi Alaoui. Distributing any elliptic curve based protocol. In *IMACC*, 2019.

[SBS02] Rob Sherwood, Bobby Bhattacharjee, and Aravind Srinivasan. P5: A protocol for scalable anonymous communication. In *IEEE Symposium on Security and Privacy*, 2002.

[Sch90] Claus-Peter Schnorr. Efficient identification and signatures for smart cards. In *Advances in Cryptology—CRYPTO'89 Proceedings 9*, pages 239–252. Springer, 1990.

[sec] Bitcoin wiki: Secp256k1. https://en.bitcoin.it/wiki/Secp256k1.

[Sha71] Daniel Shanks. Class number, a theory of factorization, and genera. In *Proc. Symp. Math. Soc., 1971*, volume 20, pages 415–440, 1971.

[Sha79] Adi Shamir. How to share a secret. *Communications of the Association for Computing Machinery*, 22(11):612–613, November 1979.

[SS24] Nikola Samardzic and Daniel Sanchez. Bitpacker: Enabling high arithmetic efficiency in fully homomorphic encryption accelerators. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 137–150, 2024.

[sup] supranational/blst. https://github.com/supranational/blst.

[SV23] Elaine Shi and Nikhil Vanjani. Multi-client inner product encryption: Function-hiding instantiations without random oracles. In *IACR International Conference on Public-Key Cryptography*, pages 622–651. Springer, 2023.

[SW05] Amit Sahai and Brent R. Waters. Fuzzy identity-based encryption. In Ronald Cramer, editor, *EUROCRYPT 2005*, volume 3494 of *LNCS*, pages 457–473. Springer, Berlin, Heidelberg, May 2005.

[SW21] Elaine Shi and Ke Wu. Non-interactive anonymous router. In *Eurocrypt*, 2021.

[TBM+20] Sri Aravinda Krishnan Thyagarajan, Adithya Bhat, Giulio Malavolta, Nico Döttling, Aniket Kate, and Dominique Schröder. Verifiable timed signatures made practical. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, CCS '20, New York, NY, USA, 2020. Association for Computing Machinery.

[tfh] https://github.com/zama-ai/tfhe-rs.

[TGL+17] Nirvan Tyagi, Yossi Gilad, Derek Leung, Matei Zaharia, and Nickolai Zeldovich. Stadium: A distributed metadata-private messaging system. In *SOSP*, 2017.

[TM21] Sri Aravinda Krishnan Thyagarajan and Giulio Malavolta. Lockable signatures for blockchains: Scriptless scripts for all signatures. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 937–954. IEEE, 2021.

[TMMS22] Sri AravindaKrishnan Thyagarajan, Giulio Malavolta, and Pedro Moreno-Sanchez. Universal atomic swaps: Secure exchange of coins across all blockchains. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 1299–1316. IEEE, 2022.

[Tom22] Alin Tomescu. Pairings or bilinear maps, 2022. https://alinush.github.io/pairings.

[Tsa19] Rotem Tsabary. Fully secure attribute-based encryption for t-CNF from LWE. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part I*, volume 11692 of *LNCS*, pages 62–85. Springer, Cham, August 2019.

[TSZ+24] Ertem Nusret Tas, István András Seres, Yinuo Zhang, Márk Melczer, Mahimna Kelkar, Joseph Bonneau, and Valeria Nikolaenko. Atomic and fair data exchange via blockchain. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, 2024.

[V+12] Salil P Vadhan et al. Pseudorandomness. *Foundations and Trends® in Theoretical Computer Science*, 7(1–3):1–336, 2012.

[VAH23] Marloes Venema, Greg Alpár, and Jaap-Henk Hoepman. Systematizing core properties of pairing-based attribute-based encryption to uncover remaining challenges in enforcing access control in practice. *DCC*, 91(1):165–220, 2023.

[vdHLZZ15] Jelle van den Hooff, David Lazar, Matei Zaharia, and Nickolai Zeldovich. Vuvuzela: Scalable private messaging resistant to traffic analysis. In *SOSP*, 2015.

[Ven23] Marloes Venema. A practical compiler for attribute-based encryption: New decentralized constructions and more. In Mike Rosulek, editor, *CT-RSA 2023*, volume 13871 of *LNCS*, pages 132–159. Springer, Cham, April 2023.

[VGTS25] Nikhil Vanjani, Garrett Greiner, Sri Aravinda Krishnan Thyagarajan, and Pratik Soni. New constructions of functional adaptor signatures : Broader functions and improved efficiency: Implementation, 2025. https://github.com/nikhilvanjani/fas-from-he-impl.

[VGTS26] Nikhil Vanjani, Garrett Greiner, Sri Aravinda Krishnan Thyagarajan, and Pratik Soni. New constructions of functional adaptor signatures : Broader functions and improved efficiency. In *IEEE Security and Privacy (Oakland)*, 2026.

[VST24a] Nikhil Vanjani, Pratik Soni, and Sri Aravinda Krishnan Thyagarajan. Functional adaptor signatures: Beyond all-or-nothing blockchain-based payments. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*, 2024.

[VST24b] Nikhil Vanjani, Pratik Soni, and Sri AravindaKrishnan Thyagarajan. Functional adaptor signatures: Implementation, 2024. https://github.com/nikhilvanjani/fas-impl.

[Wat09] Brent Waters. Dual system encryption: Realizing fully secure IBE and HIBE under simple assumptions. In Shai Halevi, editor, *CRYPTO 2009*, volume 5677 of *LNCS*, pages 619–636. Springer, Berlin, Heidelberg, August 2009.

[Wat11] Brent Waters. Ciphertext-policy attribute-based encryption: An expressive, efficient, and provably secure realization. In Dario Catalano, Nelly Fazio, Rosario Gennaro, and Antonio Nicolosi, editors, *PKC 2011*, volume 6571 of *LNCS*, pages 53–70. Springer, Berlin, Heidelberg, March 2011.

[Wat12] Brent Waters. Functional encryption for regular languages. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 218–235. Springer, Berlin, Heidelberg, August 2012.

[Wee14] Hoeteck Wee. Dual system encryption via predicate encodings. In Yehuda Lindell, editor, *TCC 2014*, volume 8349 of *LNCS*, pages 616–637. Springer, Berlin, Heidelberg, February 2014.

[Wee16] Hoeteck Wee. New techniques for attribute-hiding in prime-order bilinear groups. Manuscript, 2016.

[Wee21a] Hoeteck Wee. ABE for DFA from LWE against bounded collusions, revisited. In Kobbi Nissim and Brent Waters, editors, *TCC 2021, Part II*, volume 13043 of *LNCS*, pages 288–309. Springer, Cham, November 2021.

[Wee21b] Hoeteck Wee. Broadcast encryption with size $N^{1/3}$ and more from k-lin. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part IV*, volume 12828 of *LNCS*, pages 155–178, Virtual Event, August 2021. Springer, Cham.

[Wee22] Hoeteck Wee. Optimal broadcast encryption and CP-ABE from evasive lattice assumptions. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part II*, volume 13276 of *LNCS*, pages 217–241. Springer, Cham, May / June 2022.

[Wee24] Hoeteck Wee. Circuit ABE with $\mathsf{poly}(\mathsf{depth}, \lambda)$-sized ciphertexts and keys from lattices. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part III*, volume 14922 of *LNCS*, pages 178–209. Springer, Cham, August 2024.

[Wee25] Hoeteck Wee. Almost optimal kp and cp-abe for circuits from succinct lwe. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 34–62. Springer, 2025.

[WN22] Gang Wang and Mark Nixon. Sok: tokenization on blockchain. In *Proceedings of the 14th IEEE/ACM International Conference on Utility and Cloud Computing Companion*, UCC '21, New York, NY, USA, 2022. Association for Computing Machinery.

[WW20] Hoeteck Wee and Daniel Wichs. Candidate obfuscation via oblivious LWE sampling. Cryptology ePrint Archive, Report 2020/1042, 2020.

[WW24] Brent Waters and Daniel Wichs. Adaptively secure attribute-based encryption from witness encryption. In Elette Boyle and Mohammad Mahmoody, editors, *TCC 2024, Part III*, volume 15366 of *LNCS*, pages 65–90. Springer, Cham, December 2024.

[WWW22] Brent Waters, Hoeteck Wee, and David J. Wu. Multi-authority ABE from lattices without random oracles. In Eike Kiltz and Vinod Vaikuntanathan, editors, *TCC 2022, Part I*, volume 13747 of *LNCS*, pages 651–679. Springer, Cham, November 2022.

[ZYH+24] Jiawen Zhang, Xinpeng Yang, Lipeng He, Kejia Chen, Wen-jie Lu, Yinghao Wang, Xiaoyang Hou, Jian Liu, Kui Ren, and Xiaohu Yang. Secure transformer inference made non-interactive. *Cryptology ePrint Archive*, 2024.

[ZZZR05] Li Zhuang, Feng Zhou, Ben Y. Zhao, and Antony Rowstron. Cashmere: Resilient anonymous routing. In *NSDI*, 2005.